
Data Science for Cybersecurity in the Age of AI Agents

Measuring Risk, Detecting Adversaries, Evaluating Defenses, and Building
Evidence-Driven Security Products

Tri Doan

48 chapters · 1 appendices · 110,293 words

Edition of 2026-08-20

Contents

Chapter 1: Data Science as an Evidence and Decision Discipline	4
Chapter 2: Data Types, Structures, and When to Use Them	13
Chapter 3: Probability, Statistics, and Uncertainty	23
Chapter 4: Data Preparation, Exploration, and Feature Engineering	32
Chapter 5: Python, SQL, and Reproducible Analytical Workflows	41
Chapter 6: Visualization, Communication, and Analytical Storytelling	52
Chapter 7: Machine Learning Methods and Selection	60
Chapter 8: Specialized Data: Time, Text, Graphs, and Streams	70
Chapter 9: The Adversary Is in the Dataset	79
Chapter 10: Cyber Risk Is a Property of a Connected System	86
Chapter 11: Make the Claim Before You Touch the Data	94
Chapter 12: Operating Security Data Science as a Control System	102
Chapter 13: Telemetry — What the System Can Actually Know	111
Chapter 14: Security Event Schemas and Data Modeling	119
Chapter 15: Building Reliable Security Data Pipelines	127
Chapter 16: Security Data Quality as a Control	135
Chapter 17: Privacy-Preserving Security Analytics	143
Chapter 18: Ground Truth Is a Process, Not a Column	151
Chapter 19: Metrics That Survive Contact with Reality	159
Chapter 20: Measuring Security-Control Effectiveness	167
Chapter 21: Experiments in High-Stakes Environments	175
Chapter 22: Causal Inference for Security Decisions	183
Chapter 23: Adversarial Drift and Continuous Validation	191
Chapter 24: Statistical Detection Engineering	199
Chapter 25: Anomaly Detection Without the Hype	208
Chapter 26: Identity, Access, and Insider-Risk Analytics	216
Chapter 27: Network, Endpoint, Cloud, and Application Analytics	224
Chapter 28: Graph Analytics for Attack Paths	232

Chapter 29: Threat Intelligence and Exploit Prioritization	240
Chapter 30: Measuring the Quality of Security Findings	248
Chapter 31: From Finding to Resolution	256
Chapter 32: Security Product Analytics	264
Chapter 33: Ranking, Triage, and Decision Support	272
Chapter 34: Foundations of Agentic AI Security	280
Chapter 35: Instrumenting AI Agents for Security Evidence	288
Chapter 36: Evaluating Agent Security Controls	296
Chapter 37: AI-Powered Detection and Security Operations	304
Chapter 38: AI for Vulnerability Discovery and Code Security	312
Chapter 39: Cybersecurity Capability and Misuse Evaluations	321
Chapter 40: Production Monitoring for AI Security Systems	330
Chapter 41: Reproducible Security Analytics with Python and SQL	339
Chapter 42: MLOps and Observability for Security Models	347
Chapter 43: Building a Security Data-Science Team	355
Chapter 44: Communicating Evidence for Decisions	363
Chapter 45: Case Study: Measuring Security for a Coding Agent	371
Chapter 46: Case Study: Identity Compromise Detection	379
Chapter 47: Case Study: AI Vulnerability-Finding Product	387
Chapter 48: Case Study: Evidence-Driven Security Program	401
Manuscript Chapters	410

Chapter 1: Data Science as an Evidence and Decision Discipline

At 08:10 Monday, the identity team reports that confirmed account takeovers have nearly doubled: 145 last month versus 80 before. Login volume is flat. Fraud losses are up. The CISO asks, “Why, and what should we do today?”

That sentence hides different scientific jobs. Counting cases is descriptive. Locating the increase in recovery flows or customer cohorts is diagnostic. Estimating which current sessions will become takeovers is predictive. Estimating whether a new challenge would prevent compromise is causal. Allocating limited challenge capacity is prescriptive. Treating these jobs as interchangeable creates confident but invalid security programs.

This chapter follows the ATO problem from alarm to decision. Its structure is a sequence of increasingly demanding questions, each with its own estimand, evidence, and failure conditions.

The Monday-Morning Claim

An **account takeover (ATO)** occurs here when an unauthorized actor controls a customer account and the case is confirmed by a defined adjudication process. An alert is evidence, not the outcome.

Let $(Y_i=1)$ when active account (i) has a confirmed takeover whose compromise date falls in month (t) . The descriptive rate is

$$R_t = \frac{\sum_{i \in A_t} Y_i}{|A_t|},$$

where (A_t) is the population at risk. A numerator alone can manufacture a crisis when the customer base grows. A rate alone can hide a large workload. Report both, with label maturity.

Confirmed cases are delayed observations. If cases take nine days to adjudicate, comparing incomplete August labels with mature July labels biases the trend. A new customer-notification campaign may increase confirmations without changing compromise. The first task is a measurement audit: did compromise, observation, or adjudication change?

Population, unit, and clock

- **Population:** consumer accounts eligible to authenticate, excluding tests, employees, and already locked accounts.
- **Unit:** account-day for exposure, login session for challenge decisions, or account-month for reporting.
- **Clock:** login, compromise, report, and adjudication times are distinct.

A row is an **observation**. A **feature** is available evidence, such as distinct networks used in the prior week. A **label** is the recorded training signal, such as an adjudicator disposition. A **target** is what a model estimates. An **outcome** is what happens in the world. Labels are fallible measurements of outcomes.

Five Questions Hidden in One Incident

What changed? Descriptive work

The descriptive estimand is the change in mature incidence:

$$\Delta R = R_{\text{current}} - R_{\text{baseline}}$$

It says nothing about cause. Compute it by acquisition channel, account age, authentication method, recovery path, and telemetry coverage. Denominators must follow each segment. “Forty cases used recovery” is meaningless without the number of recovery attempts.

Description also audits data quality. A rise isolated to one adjudication queue may be workflow. A rise paired with lower sensor coverage may understate harm. This stage earns the right to ask harder questions.

Where is the mechanism? Diagnostic work

Diagnosis searches for explanations consistent with evidence. Excess cases after email changes and password resets suggest a recovery mechanism but do not prove that recovery caused the increase.

Compare compromised accounts with eligible accounts from the same period. Investigate releases, controls, marketing, bot traffic, and staffing. A field populated after support opens a case is a consequence of detection, not a precursor.

The output is a ranked hypothesis set with disconfirming checks: credential stuffing rose from residential proxies; a release weakened device binding; support improved confirmation; or one partner cohort was compromised. Each predicts a different observable pattern.

Which sessions will become takeovers? Predictive work

For decision-time evidence ($X=x$), the estimand is

$$p(x) = P(Y^{14d} = 1 \mid X=x)$$

where (Y^{14d}) means confirmed ATO within fourteen days. A model may rank well without explaining cause. Device novelty can predict ATO even if forcing known devices would be harmful.

Prediction needs deployment-shaped data: features reconstructed at login time, outcomes matured for fourteen days, and temporal validation. Random row splits leak recurring accounts and campaigns. Accuracy is useless at rare base rates; calibration, precision at capacity, recall, and workload matter.

Would intervention prevent harm? Causal work

Let treatment (A) be a phishing-resistant step-up. For eligible sessions, define potential outcomes ($Y_i(1)$) with challenge and ($Y_i(0)$) without it. The causal estimand is

$$\tau_E = E[Y(1) - Y(0) \mid E=1]$$

Only one potential outcome is observed. Naively comparing challenged and unchallenged sessions is biased because current policy challenges riskier traffic. Identification needs randomization, a credible natural experiment, or explicit assumptions with sensitivity analysis. A phased randomized rollout can estimate intention to treat while retaining emergency overrides. A predictive score is not a treatment-effect estimate.

What should the system do? Prescriptive work

Prescription chooses actions under constraints. It uses descriptive facts, predicted risk, causal effects, friction, and capacity. A policy may reserve passkey challenges for sessions with high compromise risk and contexts where challenge is effective.

Constraints belong in the specification: at most 6,000 daily challenges; no SMS fallback for administrators; abandonment increase below 0.3 percentage points; human review for high-value transfers. The answer is a policy frontier, not an unexplained threshold. If challenge does not help stolen active sessions, higher predictive precision does not solve that decision.

The Estimand Ledger

Product	Unit/population	Estimand	Evidence	Invalid shortcut
Trend	active account-month	mature ATOs per 100,000	denominators, mature cases	raw counts
Diagnosis	eligible account-day	excess incidence by mechanism	aligned telemetry, release history	alert mix equals attack mix
Ranking	login session	$(P(Y^{\wedge}\{14d\}=1 \mid X))$	as-of features, mature outcomes	random split
Evaluation	eligible session	$(E[Y(1)-Y(0) \mid E=1])$	randomized or credible assignment	treated/untreated comparison
Policy	daily sessions	prevented ATOs under capacity	risk, effects, cost, capacity	maximize AUC

Every artifact should name the row it supports. If it supports no decision and improves no other artifact's validity, it may not deserve production ownership.

Lab: Turn the Alarm into Evidence

Save as `ato_evidence.py` and run with the book environment.

```
from collections import defaultdict

rows = [
    ("July", "password", 60_000, 48, True),
```

```

("July", "passkey", 40_000, 12, True),
("August", "password", 61_000, 105, True),
("August", "passkey", 42_000, 14, True),
("September", "password", 63_000, 82, False),
]

monthly = defaultdict(lambda: [0, 0])
for month, cohort, active, cases, mature in rows:
    if mature:
        monthly[month][0] += active
        monthly[month][1] += cases

for month in ("July", "August"):
    active, cases = monthly[month]
    print(f"{month}: {cases} cases; {cases / active * 100_000:.1f} per 100k")

j = monthly["July"]; a = monthly["August"]
print(f"absolute rate change: {(a[1]/a[0] - j[1]/j[0]) * 100_000:.1f} per 100k")

selected = {(m, c): (n, y) for m, c, n, y, mature in rows if mature}
for cohort in ("password", "passkey"):
    jn, jy = selected[("July", cohort)]; an, ay = selected[("August", cohort)]
    print(f"{cohort}: {(ay/an - jy/jn) * 100_000:+.1f} per 100k")

questions = {
    "descriptive": "How much did mature ATO incidence change?",
    "diagnostic": "Which cohort and mechanism explain the excess?",
    "predictive": "Which sessions will be confirmed within 14 days?",
    "causal": "Would a passkey challenge prevent takeover?",
    "prescriptive": "Which sessions consume limited challenge capacity?",
}

for kind, question in questions.items():
    print(f"{kind:12s} {question}")

```

Expected output:

```

July: 60 cases; 60.0 per 100k
August: 119 cases; 115.5 per 100k
absolute rate change: 55.5 per 100k
password: +92.1 per 100k
passkey: +3.3 per 100k
descriptive  How much did mature ATO incidence change?
diagnostic   Which cohort and mechanism explain the excess?
predictive   Which sessions will be confirmed within 14 days?
causal       Would a passkey challenge prevent takeover?
prescriptive  Which sessions consume limited challenge capacity?

```

The fixture does not prove passwords caused the incident. It establishes concentration and excludes immature September. The next analysis must test mechanisms and population composition.

Building an Evidence Program

First frame the claim: population, unit, time origin, outcome, decision, and estimand. Next preserve event, ingestion, availability, intervention, and adjudication times. Audit sensor coverage, identity resolution, adjudication delay, and sampling. Reproduce the headline before modeling. Select the minimum adequate method: a rate table may resolve the initial alarm; logistic regression may rank sessions; randomization may be required for intervention effect.

Validation must resemble use. Recreate future traffic, label delay, capacity, and attack campaigns. Deployment versions the eligible population, features, model, treatment rules, and override behavior. Monitoring covers denominator shifts, maturity, treatment compliance, calibration, and adversarial displacement.

Reproducibility means another qualified person can reconstruct the result from versioned code, immutable inputs or snapshots, environment metadata, and a decision log. Peer review should challenge exclusions, temporal joins, outcomes, and identification assumptions, not merely style. Ethical review asks whether collection is proportionate and intervention friction is fairly distributed.

Simplicity reduces assumptions and review surface, but does not excuse weak design. No model repairs an undefined outcome. No causal method repairs severe interference without modeling it.

Case Resolution: What Can Be Said

Suppose the excess begins six hours after a release stopped invalidating old recovery tokens. It is concentrated in password accounts completing recovery. Rollback reduces suspicious completions immediately, while confirmed outcomes mature later.

The defensible report says the mature incidence increase is real; most excess cases use password recovery; timing and code inspection implicate stale tokens; emergency rollback is justified by mechanism evidence and reversibility; takeovers prevented are not yet identified; and a passkey experiment concerns future policy, not historical cause.

This is more useful than “the model found root cause.” It tells leaders what is observed, inferred, actionable, and unknown.

Analytical Ethics and Review Gates

An ATO program observes intimate behavior: devices, travel patterns, recovery attempts, network location, and sometimes financial activity. “Useful for prediction” is not sufficient authority to collect a field. The team should document purpose, retention, access, and whether a less intrusive signal can answer the same question. Raw IP histories need not be exposed to every model developer merely because a derived network-change feature is useful.

Intervention burden also deserves analysis. Step-up challenges can disproportionately affect travelers, accessibility-tool users, shared-device households, and customers in regions where preferred factors are unavailable. This is not resolved by deleting protected attributes. Without carefully governed cohort

evaluation, the team may be unable to discover concentrated harm. Access to sensitive cohort attributes should be separated from routine scoring and used under an approved evaluation protocol.

Peer review should stop an analysis when the claimed population differs from the sampled population, label maturity is ignored, availability time cannot be reconstructed, or causal language exceeds the design. A reviewer should be able to answer four questions from the artifact alone: What quantity was estimated? Which observations could enter it? What assumptions connect data to the claim? What result would reverse the recommendation?

The incident tempo does not remove these gates. It changes the level of evidence appropriate to an action. A reversible rollback with strong mechanism evidence may proceed before a precise causal effect is available. A permanent program that challenges millions of customers requires stronger evaluation. Match evidentiary burden to action scale, reversibility, and potential harm.

Diagnostic Stop Rules

Investigations can become endless correlation hunts. Establish stop rules before opening another dashboard. Stop expanding diagnosis when a mechanism is reproduced technically, its predicted telemetry pattern appears, a reversible mitigation exists, and remaining uncertainty will not change the immediate action. Continue outcome measurement afterward to quantify effect.

Conversely, stop model development and return to instrumentation when confirmation coverage cannot be estimated, principal resolution is unstable, or key source timestamps reflect batch load rather than availability. A sophisticated classifier trained under those conditions converts measurement failure into numerical confidence.

Maintain a claim register with status values such as *observed*, *supported mechanism*, *causally identified*, *contradicted*, and *open*. Link each status to evidence and date it. During a fast-moving incident, this prevents yesterday's working theory from becoming tomorrow's unquestioned fact. It also creates a clean handoff from incident response to the longer-lived prevention program.

Evidence quality should be visible in the decision memo. A mature rate from a stable denominator may receive a high grade; a mechanism supported by logs and code reproduction a moderate-to-high grade; an extrapolated treatment effect from another population a low grade. Grades do not replace argument. They force the author to state why one claim is firmer than another. Record dissent as well: if identity engineering disputes timestamp semantics, the unresolved dispute belongs beside the recommendation. This habit makes later incident review scientific rather than ceremonial and prevents an executive slide from flattening heterogeneous evidence into one unjustified confidence label.

Sources and Further Reading

- Donoho, "50 Years of Data Science." <https://doi.org/10.1080/10618600.2017.1384734>
- Hernán and Robins, *Causal Inference: What If*. <https://www.hsph.harvard.edu/miguel-hernan/causal-inference-book/>

- National Academies, *Reproducibility and Replicability in Science*. <https://doi.org/10.17226/25303>
- NIST, *Digital Identity Guidelines: Authentication and Lifecycle Management*, SP 800-63B. <https://pages.nist.gov/800-63-4/sp800-63b.html>
- W3C, *Web Authentication Level 3*. <https://www.w3.org/TR/webauthn-3/>
- OWASP, *Credential Stuffing Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Credential_Stuffing_Prevention_Cheat_Sheet.html
- IETF RFC 9700, *Best Current Practice for OAuth 2.0 Security*. <https://www.rfc-editor.org/rfc/rfc9700>

Interview Practicum

The Incident Changes Mid-Answer

Panel prompt

“Confirmed ATOs rose from 80 to 145. Design the response.” Five minutes later: “Support launched a self-service confirmation form halfway through.” Later still: “You may challenge only 5% of sessions, and fraud will not randomize obviously high-risk traffic.”

Clarifying branches

Ask when compromise occurs relative to confirmation; whether both counts are mature under one rule; how active accounts changed; which releases and controls moved; how challenge eligibility is defined; what “obviously high-risk” means; and whether boundary traffic can be randomized. The support form threatens comparability. Capacity turns ranking into allocation. Safety restrictions change the experiment’s target population.

Worked Answer

Strong answer

Reproduce the increase by compromise month with a fixed maturity window and active-account denominator. Model confirmations as the product of latent compromise and observation. Backfill historical cases to estimate how the new form changed reporting delay and confirmation probability; report bounds if cases cannot be recovered.

Diagnose concentration using recovery path, factor, account age, device continuity, network, segment, and telemetry health. Tie discontinuities to releases. Keep association language until intervention effects are identified.

Define prediction as confirmed ATO within fourteen days and validate temporally. At 5% capacity, report precision and captured cases at that operating point, calibration by cohort, and workload.

Estimate challenge effectiveness only where randomization is ethical. Route obvious-high-risk traffic to safety action; randomize eligible boundary sessions. The intention-to-treat estimate applies to that boundary population, not automatically to forced-action sessions. Measure completion, abandonment, fraud, and displacement into recovery.

Recommend emergency rollback when mechanism evidence and reversibility justify it, but label prevented cases as pending. Launch a bounded challenge experiment only after calibration and data-health checks.

Worked detail

If 80 observed cases had 0.70 confirmation probability and 145 had 0.90 after the form, simplified latent counts are $(80/0.70=114.3)$ and $(145/0.90=161.1)$. The observed 81% increase becomes an estimated 41% latent-count increase. This is sensitivity analysis, not truth; constant sensitivity is a strong assumption.

For assignment (Z), receipt (A), and outcome (Y), report $(E[Y \mid Z=1] - E[Y \mid Z=0])$ as intention to treat. Assignment can instrument receipt only if exclusion, relevance, and monotonicity are plausible.

Weak answers

- “Train XGBoost on all logins” ignores the changed observation process.
- “Block more because cases rose 81%” ignores denominators and maturity.
- “Device novelty caused ATO” mistakes prediction for cause.
- “A/B test everyone” ignores safety constraints.
- “Never experiment in security” leaves ethical policy effects unknowable.

Follow-Up Questions and Answers

Follow-up probes with answers

Cases are selected by the current detector. What now? Adjudicate a probability sample outside its boundary and weight by sampling probabilities. Where withholding action is unsafe, give bounds rather than fictional recall.

Attackers adapt? Treat channel displacement as an outcome, time-index estimates, and refresh the threat model.

Passkey users differ from password users? Raw rate differences are not passkey effects. Require a credible counterfactual and state the overlap population.

When stop the experiment? Stop for predefined harm, data-integrity failure, abandonment, or overwhelming efficacy under a valid sequential design—not repeated ordinary (p)-value peeking.

Communication drills

Thirty seconds: “Separate compromise from changed confirmation, locate the mature denominator-correct excess, estimate fourteen-day session risk for the 5% capacity decision, and estimate challenge effect in an ethical boundary experiment. Keep emergency mitigation separate from claims about prevented cases.”

Five minutes: Lead with the four estimands: incidence change, concentration, session risk, and treatment effect. Explain observation correction, temporal validation, boundary randomization, capacity metrics, and what remains unidentified.

Key Takeaways

- Description, diagnosis, prediction, causation, and prescription answer different questions.
- Write population, unit, clock, outcome, and estimand before choosing a method.
- Labels measure outcomes through an observation process that can change independently.
- Prediction identifies neither historical cause nor intervention effect.
- Capacity and safety are part of the scientific problem.
- Distinguish what is observed, inferred, identified, and unknown.

Chapter 2: Data Types, Structures, and When to Use Them

A security lake receives four objects: an authentication event, an analyst paragraph, a NetFlow record, and an identity-entitlement edge. They all concern the same investigation, but forcing them into one wide table destroys meaning. This chapter treats representation as type design. We will build a catalog, inspect a mixed fixture, and show which operations are valid for each field.

A Type System for Security Evidence

A data type is a set of possible values plus valid operations. Storage syntax is secondary. An IPv4 address stored as an integer is not a quantity; adding two addresses is meaningless. A severity encoded 1–5 is ordered, but the distance from 1 to 2 need not equal the distance from 4 to 5. A timestamp is not a duration. These distinctions prevent silent analytical errors.

Numeric families

Counts are nonnegative integers: failed logins, packets, findings. Their variance often grows with the mean, and exposure differs, so rates may be more comparable. **Continuous measurements** include latency and bytes after aggregation. **Ratio-scale** values have a meaningful zero; ratios are meaningful. **Interval-scale** values have meaningful differences but an arbitrary zero, as with Celsius. Money is ratio-like within a currency and valuation time, but exchange and inflation complicate comparisons.

A rate needs numerator, denominator, and interval. `failures_per_attempt` differs from `failures_per_minute`. Store the components when possible; a precomputed rate hides low-denominator instability.

Categorical families

Nominal categories such as operating system have equality but no order. **Ordinal** categories such as low/medium/high have order but unknown spacing. **Binary** flags need explicit positive semantics. **Multilabel** observations may carry several ATT&CK techniques. **High-cardinality** categories—user IDs, device IDs, process hashes—are identifiers, not automatically model-ready categories.

Encoding nominal values as integers invents order. One-hot encoding can explode dimensions. Hashing trades collisions for bounded size. Learned embeddings can capture similarity but require enough data and careful unknown handling. The choice follows the task, not the source database type.

Time is several types

A UTC timestamp identifies an instant. A local calendar value depends on a timezone and daylight-saving rules. A duration measures elapsed time. A sequence preserves order. A time series associates measurements with regular or irregular times. An event stream adds arrival behavior: late, duplicated, and out-of-order messages.

Store RFC 3339 timestamps with offsets at boundaries and normalize instants to UTC internally. Preserve source timezone where local business behavior matters. Never infer elapsed time by subtracting naive local timestamps across a daylight-saving transition.

Text, code, and documents

Analyst notes are not merely long categories. They contain tokens, sentences, entities, assertions, negation, uncertainty, and sensitive data. Source code has syntax trees and control flow. Prompts have roles and message boundaries. Security reports have sections, citations, indicators, and provenance.

Representations include raw immutable text, normalized text for search, token or n-gram features, embeddings for retrieval, and structured extractions. Retain offsets from extracted facts back to source spans. An embedding is a lossy index, not evidence; investigators must recover the original passage.

Network and spatial values

IP addresses support prefix membership, version, and equality—not arithmetic distance as ordinary integers. Ports are bounded identifiers with conventions, not continuous quantities. Latitude and longitude require spherical distance and privacy controls. “Country” may describe IP geolocation, billing, residence, or legal entity; name the semantics and provider version.

NetFlow is a directional summary with source/destination addresses and ports, protocol, byte and packet counts, and interval. Reversing direction changes interpretation. NAT and proxies mean an address is not an identity.

Graphs

An identity graph contains typed nodes and edges: user `member_of` group, service principal `has_role` subscription, device `managed_by` tenant. Edges often have validity intervals and provenance. A static adjacency table loses the difference between current and historical permission.

Graph questions include reachability, shortest attack path, neighborhood aggregation, centrality, and community structure. Exporting degrees into a tabular model is useful but cannot answer arbitrary path queries. Graph storage is justified by traversal workload, not visual appeal.

Binary, packet, image, audio, and multimodal evidence

Executable bytes support hashes, sections, imports, entropy, and disassembly. Packet captures preserve payload and timing but raise storage and privacy costs. Screenshots may contain phishing cues and text. Audio may contain voice, timing, and codec artifacts. A multimodal record must preserve modality-specific preprocessing and synchronization; concatenating unrelated vectors does not create coherent evidence.

Shape: How Observations Relate

Cross-sectional data take one measurement per unit. **Longitudinal** data repeat units. **Panel** data organize multiple units across times. **Cohort** data align units by an origin such as enrollment. **Survival** data record time to outcome plus censoring. A user with no observed takeover by study end is right-censored, not proven permanently safe.

Structured data have explicit fields; **semi-structured** events have envelopes plus variable nested payloads; **unstructured** artifacts lack a uniform field model. “Unstructured” does not mean structure-free: documents and binaries have rich internal grammars.

Static snapshots answer state questions. Streams answer timely change questions but require watermarks, idempotency, and late-data policy. A snapshot of permissions cannot reconstruct which access existed at incident time unless validity history is retained.

Schema Catalog: One Investigation, Four Stores

Authentication event envelope

```
auth_event
  event_id: uuid           identity, not a feature magnitude
  occurred_at: timestamp[UTC] source-reported instant
  observed_at: timestamp[UTC] collector availability
  principal_id: opaque_id   join key with tenant scope
  outcome: enum{success,failure,challenge}
  source_ip: ip_address
  factors: set<enum>
  latency_ms: duration
  raw: json                immutable source payload
```

The envelope is append-only. Corrections become new records or versioned normalization outputs. `observed_at` makes point-in-time reconstruction possible.

Analyst document

```
case_note
  note_id: uuid
  case_id: opaque_id
  author_role: enum
  created_at: timestamp[UTC]
  body: sensitive_text
  assertions: list<{span, predicate, object, confidence}>
  embedding_model: nullable model_version
  embedding: nullable vector<float>
```

Access control applies to source text and derived vectors. Assertions preserve span provenance and distinguish “user denied travel” from “user traveled.”

Flow fact

```
network_flow
  sensor_id: opaque_id
  start_at/end_at: timestamp[UTC]
  src/dst_ip: ip_address
  src/dst_port: uint16
  protocol: enum
  packets/bytes: count
  sampling_rate: probability
```

Sampled counts require weighting. Zero packets is not the same as missing telemetry.

Temporal identity edge

```
identity_edge
  subject_id/object_id: scoped_id
  relation: enum
  valid_from: timestamp[UTC]
  valid_to: nullable timestamp[UTC]
  asserted_at: timestamp[UTC]
  source: system_version
```

`valid_to IS NULL` means currently open, not unknown forever. An as-of query requires `valid_from <= t` and `(valid_to > t or null)`, plus `asserted_at <= t` if replay must respect availability.

Representation Decision Matrix

Evidence	Canonical storage	Useful transformation	Suitable analysis	Common category error
Failed-login count	integer plus exposure	rate, log1p, rolling sum	count model, threshold	treat 2 and 200 attempts equally
Severity	ordered enum	ordinal encoding	rank tests, ordinal model	assume equal spacing
User ID	scoped opaque string	joins, frequency, hashing	entity history	numeric scaling
Timestamp	timezone-aware instant	hour, recency, windows	temporal split, survival	parse as local naive time
IP address	native IP/prefix	subnet, ASN, reputation	prefix matching, graph	Euclidean integer distance
Analyst note	protected text	spans, retrieval embedding	search, NLP with review	discard source after embedding
Entitlement	temporal typed edge	paths, degrees	reachability, GNN	snapshot as timeless truth
Malware bytes	immutable blob	imports, sections, hashes	static analysis, classifier	decode arbitrary bytes as text

Lab: Inspect and Type a Mixed Fixture

The lab uses only the standard library. It rejects seductive coercions and emits a compact dictionary.

```
from datetime import datetime
from ipaddress import ip_address
import json

records = [
    {"field": "attempts", "value": 12, "logical": "count"},
    {"field": "severity", "value": "high", "logical": "ordinal"},
    {"field": "principal_id", "value": "00731", "logical": "opaque_id"},
    {"field": "occurred_at", "value": "2026-08-20T13:45:00Z", "logical": "instant"},
    {"field": "source_ip", "value": "2001:db8::8", "logical": "ip_address"},
    {"field": "analyst_note", "value": "User denies this sign-in.", "logical":
"sensitive_text"},
    {"field": "techniques", "value": ["T1078", "T1110"], "logical": "set[nominal]"},
]

def validate(row):
    value, kind = row["value"], row["logical"]
    if kind == "count":
        assert isinstance(value, int) and value >= 0
    elif kind == "ordinal":
        assert value in {"low", "medium", "high", "critical"}
    elif kind == "opaque_id":
        assert isinstance(value, str)
    elif kind == "instant":
        datetime.fromisoformat(value.replace("Z", "+00:00"))
    elif kind == "ip_address":
        value = f"IPv{ip_address(value).version}"
    elif kind == "set[nominal]":
        assert len(value) == len(set(value))
        value = sorted(value)
    elif kind == "sensitive_text":
        value = f"{len(value.split())} tokens (redacted)"
    return {"field": row["field"], "logical_type": kind, "preview": value}

for item in records:
    print(json.dumps(validate(item), sort_keys=True))
```

Expected output:

```
{"field": "attempts", "logical_type": "count", "preview": 12}
{"field": "severity", "logical_type": "ordinal", "preview": "high"}
{"field": "principal_id", "logical_type": "opaque_id", "preview": "00731"}
{"field": "occurred_at", "logical_type": "instant", "preview": "2026-08-20T13:45:00Z"}
{"field": "source_ip", "logical_type": "ip_address", "preview": "IPv6"}
{"field": "analyst_note", "logical_type": "sensitive_text", "preview": "4 tokens (redacted)"}
{"field": "techniques", "logical_type": "set[nominal]", "preview": ["T1078", "T1110"]}
```

The leading zeros in `principal_id` survive because it is not a number. IPv6 is validated without logging the address. The multilabel set is canonicalized. The note preview avoids reproducing sensitive text.

Mixed-Data Representation Lab

One investigation can be rendered as table, sequence, document retrieval result, and graph neighborhood. Each view answers a different query.

```
from collections import Counter, defaultdict

auth = [
    ("u1", 1, "failure", "d1"),
    ("u1", 2, "success", "d2"),
    ("u2", 3, "success", "d3"),
]
notes = {"u1": "customer denies login from new device"}
edges = [("u1", "member_of", "admins"), ("admins", "can_access", "billing")]

print("tabular", dict(sorted(Counter(x[2] for x in auth).items())))
print("sequence", [x[2] for x in sorted(auth) if x[0] == "u1"])
query = {"denies", "device"}
hits = [uid for uid, text in notes.items() if query <= set(text.split())]
print("text_hits", hits)

graph = defaultdict(list)
for left, relation, right in edges:
    graph[left].append((relation, right))
frontier = ["u1"]
paths = []
for _ in range(2):
    node = frontier.pop(0)
    for relation, nxt in graph[node]:
        paths.append(f"{node}-{relation}->{nxt}")
        frontier.append(nxt)
print("graph", paths)
```

Expected output:

```
tabular {'failure': 1, 'success': 2}
sequence ['failure', 'success']
text_hits ['u1']
graph ['u1-member_of->admins', 'admins-can_access->billing']
```

The count view loses order; the sequence preserves it. Keyword retrieval finds a note but not its truth. The graph exposes transitive access. No single representation dominates.

Labels, Weak Labels, and Censoring

A confirmed incident label may be precise but selected: only alerted cases receive investigation. A user report is weak evidence with variable delay. A vendor verdict can change. A rule hit is a heuristic label, not ground truth. Keep label source, version, time, and confidence separate from the target.

Unobserved does not mean negative. In survival data, an account observed for seven days without ATO is censored at seven days. In positive-unlabeled learning, unlabeled examples contain negatives and undiscovered positives. In streaming systems, a late label may revise an earlier training snapshot. Schema catalogs should therefore describe outcome maturity and revision behavior.

Catalog Governance and Compatibility

A data dictionary becomes useful when it is executable. Each field entry should include logical type, null semantics, unit, allowed range or vocabulary, sensitivity classification, owner, source, event-time meaning, availability expectation, retention, and known revisions. Examples are valuable, but examples cannot replace constraints.

Schema compatibility has several dimensions. A change can be syntactically additive yet semantically breaking: keeping `country` as a string while changing it from billing country to IP-geolocation country will pass parsers and corrupt trends. Renaming an enum is obvious; changing the population represented by a boolean is not. Version meaning, not merely serialization.

Producers should publish contract tests. Consumers should test unknown enum values, absent optional fields, duplicates, delayed events, and mixed schema versions. Rejecting every new value creates outages; silently mapping every unknown to “other” can hide attacks and rollout defects. The contract must state whether unknown means unsupported, newly introduced, malformed, or intentionally withheld.

For privacy, classify derived data as carefully as raw data. Hashing an email does not make it anonymous when the hash is stable and linkable. Embeddings may retain sensitive semantic information. Graph neighborhoods can reveal privileged roles. Apply tenant scope, access control, deletion obligations, and retention to derivatives, indexes, caches, and training snapshots.

Choosing Physical Layout from Access Patterns

Logical modeling precedes physical layout. Columnar files suit scans over selected fields and compress repeated values. Row stores suit keyed transactional reads. Inverted indexes suit lexical retrieval. Vector indexes suit approximate semantic neighbors. Graph engines suit variable-depth traversal. Stream logs suit ordered append and replay. These are workload claims that should be benchmarked.

Partitioning by date accelerates bounded time scans but can create tiny files at low volume. Partitioning by high-cardinality user ID is usually disastrous. Clustering or sorting by tenant and time may improve pruning without creating a directory per identity. Storage statistics themselves can leak tenant size, so operational metadata also needs authorization.

Materialized representations need freshness contracts. An online feature may use a five-minute identity-neighborhood cache while an investigator queries the authoritative temporal graph. Both are correct for their declared use, but they are not interchangeable. Record build time, source watermark, and transformation version so disagreement can be explained.

Representation choices should be reversible where uncertainty is high. Preserve an immutable canonical artifact, then derive tables, indexes, vectors, and graph projections. Reversibility does not mean retaining everything forever; retention and deletion must be enforced at the canonical and derived layers. It means avoiding an irreversible lossy transform as the only surviving evidence.

Benchmark with representative queries before adopting another store. Measure ingest throughput, bounded-time scans, key lookup latency, traversal depth, retrieval recall, authorization overhead, backfill duration, and deletion propagation. Include failure experiments: one region unavailable, an index six hours stale, or a schema version arriving early. A fast happy-path benchmark says little about investigative reliability. Document the exit cost, including how canonical data can be exported without losing temporal semantics. Specialized infrastructure is justified when it produces a measured capability the existing platform cannot meet, not when its data model merely resembles a whiteboard drawing. Measure semantics first; optimize their physical execution second.

Sources and Further Reading

- PostgreSQL, “Data Types.” <https://www.postgresql.org/docs/current/datatype.html>
- Apache Parquet, format documentation. <https://parquet.apache.org/docs/>
- IETF RFC 3339, *Date and Time on the Internet*. <https://www.rfc-editor.org/rfc/rfc3339>
- IETF RFC 8200, *Internet Protocol, Version 6*. <https://www.rfc-editor.org/rfc/rfc8200>
- W3C, *RDF 1.1 Concepts and Abstract Syntax*. <https://www.w3.org/TR/rdf11-concepts/>
- Open Cybersecurity Schema Framework, schema documentation. <https://schema.ocsf.io/>
- Elastic, *Elastic Common Schema Reference*. <https://www.elastic.co/guide/en/ecs/current/index.html>
- NIST, *Guide to Data-Centric System Threat Modeling*, SP 800-154 IPD. <https://csrc.nist.gov/pubs/sp/800/154/ipd>

Interview Practicum

The Architecture Whiteboard

Prompt and constraint cards

“Represent authentication events, analyst notes, NetFlow, and an identity graph for detection and investigation.” Then the panel reveals: notes contain regulated data; flows are sampled 1:100; entitlement edges arrive up to six hours late; investigators need 90-day replay; and the feature service needs results in 100 ms.

Worked Answer

Strong answer

Use an immutable event envelope for authentication with occurrence and observation times. Store typed normalized columns beside raw versioned payloads. Partition by event date and tenant, but avoid putting sensitive identity values in paths. Use a protected document store and inverted/vector indexes for notes; retain source spans and apply the same access policy to embeddings.

Store flows in a columnar fact table with direction, interval, counters, and sampling rate. Weighted aggregate estimates may multiply counts by 100, but uncertainty and collector loss must be distinguished. Store temporal identity edges with validity and assertion intervals. Materialize bounded neighborhoods for the low-latency feature path while keeping a graph-capable historical store for replay.

Use scoped stable IDs and a governed identity-resolution table. Do not join on email text or IP. Every derived object records source versions and as-of time. The 100-ms service reads precomputed typed features; investigation queries can tolerate slower document and graph traversal. Replay blocks facts whose observation time exceeds the historical decision time.

Tradeoffs

A single warehouse simplifies governance and batch joins but may serve graph traversal and text retrieval poorly. Specialized stores improve workloads but multiply consistency and authorization surfaces. Choose the smallest set that meets measured workloads, and publish canonical semantics centrally. Parquet is efficient for scans, not point mutation. A graph database helps variable-depth traversal, not simple one-hop joins. Vector retrieval improves semantic recall but requires evaluation and source recovery.

Weak answers

- “Put everything in JSON” avoids schema design and weakens constraints.
- “Use a graph database for all data” confuses connectedness with every workload.
- “Convert IDs and IPs to integers” invents numeric meaning and breaks formatting.
- “Embed the notes and delete text” discards evidence and provenance.
- “Missing edge means no access” ignores late arrival and sensor coverage.

Follow-Up Questions and Answers

Follow-up probes with answers

How do you version a schema without breaking consumers? Use additive evolution by default, stable field identifiers, compatibility tests, explicit semantic versions for changed meaning, and dual-read migration for breaking changes.

How do you represent a deleted entitlement? Close the edge validity interval; do not delete historical state required for replay. Record when the deletion assertion became available.

Can sampled NetFlow identify one connection? No. Sampling supports certain aggregate estimates; it does not reconstruct omitted individual flows. State the sampling mechanism before weighting.

How do you test embeddings containing sensitive notes? Apply document authorization before retrieval, test cross-tenant isolation and memorization risks, minimize retained text, and log access without exposing content.

Communication drills

Thirty seconds: “Keep four canonical forms: typed immutable events, protected documents with span provenance, directional sampled flow facts, and temporal identity edges. Join them through scoped IDs and as-of semantics; materialize only the low-latency features needed online.”

Five minutes: Walk through logical types, canonical stores, transformations, workload indexes, late data, replay, privacy, and why no universal table or database is appropriate.

Key Takeaways

- Logical type determines valid operations; physical storage does not create semantics.
- Preserve counts and exposures, instants and durations, validity and availability.
- IDs, IPs, ports, ranks, and timestamps must not be coerced into false quantities.
- Text embeddings, graph features, and aggregates are derived views, not substitutes for evidence.
- Censoring, weak labels, sampling, and late arrival belong in the schema contract.
- Mixed security investigations need interoperable representations, not one universal shape.

Chapter 3: Probability, Statistics, and Uncertainty

A malware detector advertises 99% sensitivity and 99% specificity. The SOC hears “99% accurate.” At a base rate of one malicious file per thousand, however, most alerts are false. Probability supplies the missing denominator; statistics tells us how uncertain our measurements are; simulation shows what the queue may look like tomorrow.

This chapter is organized as a quantitative case file. We derive the alert calculation, examine distributions behind security counts and delays, construct intervals, test a policy change, perform a Bayesian update, and simulate operational load.

Case File A: The Alert That Sounds Certain

Let (M) denote malicious and (+) a positive detector result. Suppose

$$\begin{aligned} P(M) &= 0.001, \quad P(+ \mid M) = 0.99, \quad P(- \mid \neg M) = 0.99. \end{aligned}$$

The false-positive rate is ($P(+ \mid \neg M) = 0.01$). Bayes’ theorem gives

$$P(M \mid +) = \frac{P(+ \mid M)P(M)}{P(+ \mid M)P(M) + P(+ \mid \neg M)P(\neg M)}.$$

Substitution yields

$$\frac{0.99(0.001)}{0.99(0.001) + 0.01(0.999)} = \frac{0.00099}{0.01098} = 0.09016.$$

So an alert has about 9.0% posterior probability of malware. In one million files, the expected confusion matrix is 990 true alerts, 10 missed malware files, 9,990 false alerts, and 989,010 true negatives. Analysts receive 10,980 alerts; roughly eleven out of twelve are false.

The detector is not broken. The verbal summary was. Sensitivity and specificity condition on truth; an analyst needs probability of truth conditioned on the alert. Base rates connect them.

Odds form and sequential evidence

Bayes is often clearer in odds. Prior odds are

$$O(M) = \frac{0.001}{0.999} = 0.001001.$$

The positive likelihood ratio is ($LR^+ = 0.99/0.01 = 99$). Posterior odds equal prior odds times likelihood ratio:

$$O(M \mid +) = 0.001001 \times 99 = 0.099099.$$

Converting odds to probability gives ($0.099099 / (1 + 0.099099) = 0.09016$).

If an independent sandbox result has ($LR^+ = 20$), posterior odds become (1.98198), or probability ($1.98198 / 2.98198 = 0.6647$). Independence is a substantive assumption. Two engines trained on the same corpus and signatures may provide correlated evidence; multiplying their likelihood ratios overstates certainty.

Random Variables Behind Security Telemetry

A random variable maps uncertain outcomes to values. For Bernoulli ($X \in \{0,1\}$) with success probability (p),

$$E[X] = p, \quad \text{Var}(X) = p(1-p).$$

The sum of (n) independent Bernoulli trials is binomial:

$$P(K=k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad E[K] = np, \quad \text{Var}(K) = np(1-p).$$

It can model malicious files in a fixed batch when trials are sufficiently similar and independent. Campaign bursts violate those assumptions and inflate variance.

Poisson counts and exposure

For events arriving independently at constant rate (λ),

$$P(K=k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad E[K] = \text{Var}(K) = \lambda.$$

If a tenant averages two credential-stuffing alerts per hour, the probability of at least one in an hour is $(1 - e^{-2}) = 0.8647$. Real attacks cluster, so empirical variance often exceeds the mean. A negative-binomial model or hierarchical rate can represent overdispersion. Always include exposure: ten alerts among 100 logins differs from ten among a million.

Exponential waiting time

The waiting time (T) in a Poisson process is exponential:

$$P(T > t) = e^{-\lambda t}, \quad E[T] = 1/\lambda.$$

Its memoryless property rarely describes remediation perfectly: old vulnerabilities may be systematically harder, violating constant hazard. Survival methods can model time-varying hazard and right censoring.

Normal, lognormal, and heavy tails

The normal distribution is symmetric and useful for additive measurement error and sampling means. It is often poor for positive skewed latency or loss. If $(\log X \sim N(\mu, \sigma^2))$, then (X) is lognormal with median (e^μ) and mean $(e^{\mu + \sigma^2/2})$; the mean exceeds the median.

File sizes, response times, and incident costs may be heavy-tailed. A power-law tail satisfies approximately $(P(X > x) \propto x^{-\alpha})$ above a threshold. Do not declare a power law from a straight line on a log-log chart; compare alternative tail models and estimate the threshold. Heavy tails make sample means unstable and extreme scenarios important.

Expectation, variance, and covariance

Expectation is probability-weighted average. Variance measures squared dispersion. Covariance

$$\text{Cov}(X,Y) = E[(X-E[X])(Y-E[Y])]$$

captures linear co-movement and depends on scale. Correlation standardizes it to $([-1,1])$. Neither establishes cause. Correlated detections may share a sensor, campaign, or label process.

Case File B: Estimation from a Sample

Suppose auditors randomly inspect 400 detector alerts and confirm 44 malicious files. Estimated precision is

$$\hat{p} = 44/400 = 0.11.$$

A point estimate without uncertainty invites false precision. The naive Wald interval, $(\hat{p} \pm 1.96 \sqrt{\hat{p}(1-\hat{p})/n})$, performs poorly near boundaries or with small samples. The Wilson interval has better coverage:

$$\frac{\hat{p} + z^2/(2n) \pm z \sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}.$$

For 44 of 400 at 95%, it is approximately $[0.083, 0.145]$. This is a confidence interval: over repeated samples, 95% of intervals built by the procedure cover the fixed parameter. It is not a 95% probability statement about this fixed (p) .

A Bayesian model with $(p \sim \text{Beta}(1,1))$ yields posterior $(\text{Beta}(45,357))$. A 95% credible interval allows the direct probability interpretation conditional on model and prior. Similar numerical intervals do not make the philosophies identical.

Prediction intervals are different

A confidence interval concerns a parameter such as mean alert volume. A prediction interval concerns a future observation and includes process variation. If daily false alerts are Poisson with uncertain rate, tomorrow's predictive distribution is wider than uncertainty about the mean rate. Staffing from a confidence interval for the mean understates queue spikes.

Bootstrap and robust summaries

For a complicated statistic, resample observations with replacement, recompute the statistic, and use the bootstrap distribution. Resampling must match dependence: sessions from the same account or days from the same campaign may need cluster or block bootstrap. Median and trimmed means reduce sensitivity to extreme times, but the choice must match the operational question; the tail may be exactly what incident response cares about.

Case File C: Did the New Rule Help?

Before a rule change, 120 of 10,000 reviewed alerts were confirmed malicious. Afterward, 162 of 10,000 were confirmed. The observed precision difference is $(0.0162 - 0.0120 = 0.0042)$, or 0.42 percentage points.

Under the null $(p_1 = p_2)$, pooled proportion is $(282/20,000 = 0.0141)$. Standard error is

$$SE = \sqrt{0.0141(1-0.0141)(1/10000 + 1/10000)} = 0.001667.$$

Thus ($z=0.0042/0.001667=2.52$), with two-sided ($p\approx0.0118$). At a predeclared 0.05 level, this rejects equality. It does not prove the rule caused improvement: time, campaigns, reviewers, and sampling may differ. Nor does significance establish importance. The change adds about 42 confirmed cases per 10,000 reviewed alerts; whether that matters depends on misses, queue cost, and displacement.

Power is the probability of rejecting the null at a specified true effect. Rare outcomes require large samples. Designing for an implausibly large effect produces an underpowered study. Running hundreds of segment tests inflates false discoveries; use prespecified hypotheses, family-wise or false-discovery controls, and independent validation. Exploratory findings should be labeled exploratory.

Permutation testing can compare groups with fewer distributional assumptions: shuffle treatment labels under exchangeability and measure how often differences are at least as extreme. If assignment was not exchangeable, permutation does not repair confounding.

Bayesian Update Lab

This program reproduces the base-rate calculation, Wilson interval, beta posterior, and likelihood-ratio update.

```
from math import sqrt
from scipy.stats import beta

prevalence = 0.001
sensitivity = 0.99
specificity = 0.99
posterior = sensitivity * prevalence / (
    sensitivity * prevalence + (1 - specificity) * (1 - prevalence)
)
print(f"P(malware|alert)={posterior:.4f}")

n = 400; successes = 44; z = 1.96
phat = successes / n
center = (phat + z*z/(2*n)) / (1 + z*z/n)
half = z * sqrt(phat*(1-phat)/n + z*z/(4*n*n)) / (1 + z*z/n)
print(f"Wilson95=[{center-half:.3f}, {center+half:.3f}]")

lo, hi = beta.ppf([0.025, 0.975], 1 + successes, 1 + n - successes)
print(f"Beta posterior mean={({1+successes)/(2+n):.3f}")
print(f"Credible95=[{lo:.3f}, {hi:.3f}]")

odds = posterior / (1 - posterior)
odds *= 20
print(f"P(malware|alert,sandbox)={odds/(1+odds):.4f}")
```

Expected output:

```
P(malware|alert)=0.0902
Wilson95=[0.083, 0.144]
Beta posterior mean=0.112
```

```
Credible95=[0.083, 0.144]
P(malware|alert,sandbox)=0.6647
```

The uniform beta prior adds one pseudo-success and one pseudo-failure. In production, prior choice should reflect historical exchangeability and be tested for sensitivity. A posterior is only as credible as its likelihood and data-generating assumptions.

Monte Carlo Lab: Will the Queue Overflow?

Analytical expectations hide operational tails. Simulate daily traffic, malware arrivals, detections, false alerts, and analyst capacity. A beta distribution represents uncertainty in the false-positive rate rather than pretending it is known exactly.

```
import numpy as np

rng = np.random.default_rng(20260820)
days = 100_000
files = 100_000
capacity = 1_150
malicious = rng.binomial(files, 0.001, days)
true_alerts = rng.binomial(malicious, 0.99)
false_positive_rate = rng.beta(100, 9900, days)
false_alerts = rng.binomial(files - malicious, false_positive_rate)
alerts = true_alerts + false_alerts

print(f"mean_alerts={alerts.mean():.1f}")
print(f"p95_alerts={np.quantile(alerts, .95):.0f}")
print(f"overflow_probability={(alerts > capacity).mean():.3f}")
print(f"mean_true_alerts={true_alerts.mean():.1f}")
```

Expected output for NumPy 2.5.2:

```
mean_alerts=1098.6
p95_alerts=1277
overflow_probability=0.302
mean_true_alerts=99.0
```

The expected queue is below capacity, yet overflow occurs about 30% of days because the false-positive rate is uncertain. Simulation is not prophecy. Vary prevalence, correlation, campaign bursts, sensor outages, and analyst service time. Model validation includes checking whether simulated components reproduce historical marginal distributions and dependence.

Sampling, Bias, and the Law of Large Numbers

For independent identically distributed observations with finite expectation, sample averages converge toward the expectation. More data reduces random error; it does not remove systematic bias. A million adjudicated alerts can estimate the precision of *reviewed alerts* tightly while saying little about undetected malware.

Selection bias arises because current policy determines review. Verification bias occurs when only positives receive definitive testing. Survivorship bias excludes destroyed or unenrolled devices. Measurement bias enters when engines share labels. Concept drift changes $(P(Y \mid X))$; prior-probability shift changes $(P(Y))$. Security data violate IID assumptions through campaigns, tenants, repeated entities, and adaptive attackers.

Random sampling from the target population is the cleanest remedy for prevalence estimation. Stratified sampling improves precision for rare or important groups when weights are retained. Cluster sampling may be cheaper but increases variance. Convenience samples require a narrower claim.

Statistical and Operational Significance

A tiny precision increase may be statistically detectable at massive scale but operationally irrelevant. A modest reduction in critical ransomware misses may be valuable despite wide uncertainty. Report absolute effects, relative effects, intervals, denominators, and translated consequences.

Decision makers also need robustness. Does the conclusion change if prevalence is half the estimate? If false-positive cost doubles? If one campaign is removed? Sensitivity analysis exposes conclusions balanced on fragile assumptions. “Not statistically significant” means the study did not reject its null; it does not prove equivalence. Equivalence and noninferiority require margins defined before seeing results.

Dependence Is Not a Footnote

Many textbook formulas assume independent trials. Security observations cluster within users, hosts, tenants, days, and campaigns. Ten thousand files from one outbreak do not provide the same information as ten thousand independent families. Ignoring dependence makes standard errors too small and evidence look stronger than it is.

Match analysis to the sampling unit. Use cluster-robust uncertainty when assignment is individual but outcomes correlate within organizations; random effects or hierarchical models when group variation is itself important; and block resampling for time dependence. Report the number of independent clusters, not only the row count. With very few clusters, asymptotic corrections can remain unreliable, so design-based randomization inference or conservative aggregation may be preferable.

Calibration uncertainty deserves the same care. Reliability diagrams built from tiny bins can look erratic or deceptively perfect. Report bin counts, use adaptive or prespecified bins, and complement plots with proper scoring rules such as log loss or Brier score. Recalibration can repair probability mapping under stable ranking, but it cannot recover missed features, selection bias, or a changed target. When prevalence shifts, intercept adjustment may help; when class-conditional behavior changes, fresh labeled evidence is necessary. Treat every numeric probability as a claim tied to a population, time window, and adjudication process.

Sources and Further Reading

- NIST/SEMATECH, *e-Handbook of Statistical Methods*. <https://www.itl.nist.gov/div898/handbook/>
- NIST, *Engineering Statistics Handbook: Binomial Proportion*. <https://www.itl.nist.gov/div898/handbook/>
- Gelman et al., *Bayesian Data Analysis*, 3rd ed. <https://sites.stat.columbia.edu/gelman/book/>
- Efron and Tibshirani, *An Introduction to the Bootstrap*. <https://doi.org/10.1201/9780429246593>
- Wasserstein, Schirm, and Lazar, “Moving to a World Beyond $p < 0.05$.” <https://doi.org/10.1080/00031305.2019.1583913>
- Benjamini and Hochberg, “Controlling the False Discovery Rate.” <https://doi.org/10.1111/j.2517-6161.1995.tb02031.x>
- NIST, *AI Risk Management Framework 1.0*. <https://doi.org/10.6028/NIST.AI.100-1>

Interview Practicum

The 99%-Accurate Detector

Prompt with live constraints

“Explain why a 99%-accurate malware detector overwhelms our SOC and propose a launch evaluation.” The panel adds: one million files arrive daily; prevalence is 0.1%; analysts can review 2,000 alerts; sandbox evidence is correlated with the detector; and fifteen business units have different base rates.

Worked Answer

Strong answer

Clarify whether “99%” means sensitivity, specificity, balanced accuracy, or overall accuracy and on which population. Using 99% sensitivity and specificity at 0.1% prevalence, expect 990 true alerts and 9,990 false alerts per million, so precision is about 9% and 10,980 alerts exceed capacity.

Do not multiply detector and sandbox likelihood ratios unless conditional independence is defensible. Estimate joint performance on a representative, adjudicated sample. Calibrate per business unit or with a hierarchical model because posterior probability changes with local prevalence. Reserve probability samples below thresholds to estimate misses and prevalence.

Evaluate at the 2,000-alert operating point: confirmed malware captured, false alerts, calibration, severe-family recall, time to action, and queue-age distribution. Compare confidence or credible intervals, not only point estimates. Stress-test campaign days and uncertain false-positive rates through Monte Carlo. Launch in shadow mode, then bounded routing with predefined queue and miss gates.

Technical answer

At capacity 2,000, ranking quality determines which alerts are reviewed. If the top slice contains 800 true and 1,200 false alerts, precision is 40% and recall is $(800/1000=80\%)$, assuming 1,000 malicious files. That is operationally different from evaluating a fixed binary threshold. Confidence intervals should account for stratified audit sampling. Business-unit posterior probabilities use local priors; pooling can borrow strength but must not erase heterogeneity.

Weak answers

- “One percent error means 10,000 mistakes” misses truth-conditioned rates and class composition.
- “Accuracy is high, so add analysts” accepts a bad operating point without analysis.
- “Run both tools and multiply confidence” assumes independence.
- “The (p)-value is below .05, so launch” omits effect size, queue, and causality.
- “No alert means safe” ignores false negatives and verification bias.

Follow-Up Questions and Answers

Follow-up probes with answers

How estimate prevalence when labels exist only for alerts? Draw a probability sample from all files, adjudicate with a reference procedure, and weight strata. Without negative sampling, prevalence is unidentified.

What if zero malware appears in a sample? Zero does not imply zero rate. For (n) independent trials, the rough 95% upper bound is $(3/n)$, the rule of three.

Why not one global threshold? Different priors and costs yield different posterior risks and operating points. Calibrate and govern segment policies while guarding against unstable small samples.

How address multiple family metrics? Prespecify critical families, report simultaneous uncertainty or control false discoveries, and validate discoveries on fresh data.

Communication drills

Thirty seconds: “At 0.1% prevalence, 99% sensitivity and specificity produce about 990 true and 9,990 false alerts per million files—roughly 9% precision. Evaluate ranking at the 2,000-case capacity, audit below threshold, calibrate by base rate, and simulate queue tails.”

Five minutes: Derive Bayes, translate to counts, discuss dependent evidence, intervals, sampling bias, segmented calibration, capacity evaluation, and Monte Carlo stress tests.

A defensible conclusion states which assumptions dominate and which additional observation would most reduce decision uncertainty.

Key Takeaways

- Base rates can dominate impressive sensitivity and specificity.
- Use distributions whose assumptions match counts, waiting times, skew, and dependence.
- Confidence, credible, and prediction intervals answer different questions.
- Statistical significance neither establishes causality nor operational value.
- More selected data reduces variance around a biased answer.
- Monte Carlo simulation reveals queue tails hidden by expected values.

Chapter 4: Data Preparation, Exploration, and Feature Engineering

This chapter is a forensic notebook. A login-risk model scores 0.97 AUC offline and 0.61 after deployment. Rather than tune the model, we freeze three login times, reconstruct exactly what the system knew, and test every feature against an availability clock. The culprit is not an algorithm. It is a future fact disguised as a column.

Notebook Setup: Three Clocks, Never One

An authentication record has an **event time**: when the login occurred. A source fact has an **availability time**: when the feature pipeline could read it. An outcome has a **label time**: when compromise was adjudicated. Point-in-time correctness requires that features for decision time (t) use facts whose availability time is at or before (t), while labels may mature later.

The raw fixture contains login attempts, device observations, password resets, case outcomes, and ingestion timestamps. The unit is one login attempt. The target is confirmed compromise within fourteen days. The model decision occurs immediately after authentication evidence arrives.

login_id	principal	event_time	decision_time
L1	u1	Aug-01 09:00	Aug-01 09:00:02
L2	u1	Aug-03 10:00	Aug-03 10:00:02
L3	u2	Aug-03 11:00	Aug-03 11:00:02

A password-reset record for `u1` occurred after L2 but was backfilled into a table joined by principal. A case system later set `reset_after_incident=1`. The training query reads the latest principal row without an as-of predicate, so both L1 and L2 inherit the future value. The model learns the investigation workflow.

Cell 1: Profile Before Repair

Profiling is not a ritual of printing `describe()`. It tests claims needed by the analysis:

- Are login IDs unique, and are source IDs stable across replays?
- What fraction of events arrive after the online latency budget?
- Which columns change after initial ingestion?
- Are missing values concentrated by source, tenant, platform, or attack period?
- Do outcome dates precede adjudication dates?
- Are counts physically possible and timestamps ordered?
- Does one principal appear in train and validation?

For numeric features, inspect quantiles, zero mass, tail behavior, and exposure. For categories, inspect cardinality, unseen values, and rare levels. For time, inspect gaps, lateness, seasonality, and release boundaries.

For joins, measure unmatched, one-to-many, and multiplicative rows. A join that grows 100,000 logins into 128,000 rows is not “more data”; it may be duplicated evidence.

Exploration should have a question ledger. Plotting hundreds of correlations and selecting an exciting one invites data dredging. Mark exploratory discoveries and verify them on untouched time periods.

Cell 2: Classify Missingness as a Mechanism

Let $(R=1)$ mean a value is observed and (X) the complete-data values.

MCAR means $(P(R \mid X) = P(R))$: missingness is unrelated to observed or unobserved values. A random packet drop may approximate it, though real outages often do not.

MAR means missingness depends on observed data but, conditional on them, not the missing value. Device posture may be absent more often on a known legacy platform; conditioning on platform may support an MAR analysis.

MNAR means missingness depends on the unseen value even after observed variables. Attackers disabling an agent specifically when executing malicious code creates adversarial MNAR. No generic imputer solves identification under MNAR; use sensitivity analysis, alternate sensors, and explicit missingness features where safe.

Missing has domain meanings: “not applicable,” “sensor unsupported,” “collector late,” “redacted,” “lookup found no match,” and “true zero” are different states. Collapsing them into null destroys information. An absent reputation lookup can become predictive because the service fails during attacks; deploying that feature may encode an outage rather than threat.

Do not globally fill numeric nulls before splitting. Fit imputation statistics on training data and apply them unchanged to validation and test. Add missing indicators when the mechanism is stable and legitimate, but monitor them for source-policy changes and attacker manipulation.

Cell 3: Separate Errors, Outliers, and Rare Truth

A negative duration is usually an error. A login from 20,000 km away may be a geolocation error, VPN, shared account, or actual compromise. Rare is not synonymous with malicious, and malicious is not synonymous with statistically rare.

Validation rules should distinguish impossible values from improbable ones. Quarantine impossible rows or repair them from authoritative sources with provenance. Preserve plausible extremes and use robust transforms or models. Winsorizing attack bursts because they are outliers deletes the phenomenon the model should learn.

For skewed counts, `log1p` compresses scale while retaining zeros. Standardization is useful for distance- and gradient-based models; trees generally do not require it. Fit scalers only on training data. An ordinal encoder needs explicit order. A nominal encoder needs unknown-category behavior. Frequency encoding can leak target-period composition when computed globally.

Cell 4: Deduplicate and Resolve Entities

Deduplication asks whether records represent the same event; entity resolution asks whether identifiers represent the same entity. They require different evidence.

Exact event deduplication may use a source event ID plus producer. If IDs are absent, construct a fingerprint from stable fields, but account for legitimate repeated failures. “Same user, IP, and second” may collapse a retry storm. Keep ingestion lineage and choose a deterministic survivor.

Entity resolution can connect renamed accounts, federated identities, devices, and sessions. Deterministic links using tenant-scoped immutable IDs are preferable. Probabilistic linkage must expose match confidence and avoid cross-tenant collisions. Never let an email address silently become a global person identifier. Split data by resolved entity groups so alternate IDs do not cross train and test.

Cell 5: Put Every Candidate Feature on Trial

A feature is eligible only if it passes five tests:

1. **Availability:** Could production obtain it before the decision deadline?
2. **Provenance:** Can the value be traced to versioned facts and transformations?
3. **Semantic stability:** Will the meaning survive source and policy changes?
4. **Leakage review:** Is it caused by the outcome, adjudication, or intervention?
5. **Abuse review:** Can an attacker cheaply control or suppress it?

Temporal features

For login at (t), useful features include failures in $((t-1h, t])$, distinct devices in $((t-30d, t])$, seconds since prior successful login, and ratio of failed attempts to all attempts. Specify boundary inclusion. Including the current outcome field or later events is leakage. A rolling feature must group by entity and shift where the current row should not contribute.

Recency needs a policy for no prior event. Encode “no history” separately from an arbitrary huge duration. Rates retain numerator and denominator or smoothing information. A ratio of one failure over one attempt is less certain than 100 over 100.

Categorical, text, and graph features

Categorical features can use one-hot encoding for bounded vocabularies, hashing for high cardinality, or leakage-safe target encoding fitted out of fold. Text features must exclude notes written after disposition; even document timestamps can be misleading when notes are edited in place. Graph features such as privileged-neighbor count must use the entitlement graph valid and available at decision time. Computing current graph centrality for historical logins leaks future access changes.

Sequence features capture ordering: failure, reset, success differs from success, reset, failure. Aggregates erase this. Dimensionality reduction such as PCA must fit on training only and may reduce interpretability. Feature

selection performed on the full dataset leaks validation outcomes. Wrap all learned preprocessing inside the validation procedure.

Cell 6: Point-in-Time Fixture

This runnable fixture contrasts a leaking latest-state join with a correct as-of feature builder.

```
from datetime import datetime, timezone

def ts(value):
    return datetime.fromisoformat(value).replace(tzinfo=timezone.utc)

logins = [
    {"id": "L1", "user": "u1", "at": ts("2026-08-01T09:00:00")},
    {"id": "L2", "user": "u1", "at": ts("2026-08-03T10:00:00")},
    {"id": "L3", "user": "u2", "at": ts("2026-08-03T11:00:00")},
]

facts = [
    # user, fact, event time, availability time
    ("u1", "device:d1", ts("2026-07-31T12:00:00"), ts("2026-07-31T12:00:05")),
    ("u1", "failure", ts("2026-08-03T09:40:00"), ts("2026-08-03T09:40:03")),
    ("u1", "password_reset", ts("2026-08-03T10:30:00"), ts("2026-08-03T10:30:02")),
    ("u2", "device:d9", ts("2026-08-03T10:00:00"), ts("2026-08-03T12:00:00")),
]

def build(login):
    visible = [f for f in facts if f[0] == login["user"] and f[3] <= login["at"]]
    failures = sum(f[1] == "failure" and f[2] > login["at"].replace(hour=0) for f in visible)
    devices = {f[1] for f in visible if f[1].startswith("device:")}
    reset_seen = any(f[1] == "password_reset" for f in visible)
    return login["id"], failures, len(devices), reset_seen

latest_reset = {user: any(f[1] == "password_reset" for f in facts if f[0] == user)
                 for user in {x["user"] for x in logins}}

for login in logins:
    print("as_of", build(login), "leaky_latest_reset", latest_reset[login["user"]])
```

Expected output:

```
as_of ('L1', 0, 1, False) leaky_latest_reset True
as_of ('L2', 1, 1, False) leaky_latest_reset True
as_of ('L3', 0, 0, False) leaky_latest_reset False
```

The reset is unavailable at both u1 login times, yet the latest-state join marks both true. The u2 device occurred earlier but arrived an hour after L3, so it is correctly absent. Event-time filtering alone would still leak it.

Cell 7: Assert the Boundaries

Point-in-time pipelines need executable invariants, not reviewer memory.

```
from datetime import datetime, timezone

def ts(value):
    return datetime.fromisoformat(value).replace(tzinfo=timezone.utc)

cutoff = ts("2026-08-03T10:00:00")
boundary_facts = [
    ("at_cutoff", cutoff),
    ("one_microsecond_late", ts("2026-08-03T10:00:00.000001")),
]
included = [name for name, available in boundary_facts if available <= cutoff]
assert included == ["at_cutoff"]

train_users = {"u1", "u2"}
validation_users = {"u3"}
assert train_users.isdisjoint(validation_users)

feature_rows = [("L1", False), ("L2", False), ("L3", False)]
assert len(feature_rows) == len({row[0] for row in feature_rows})
assert all(reset_seen is False for _, reset_seen in feature_rows)
print("boundary=closed")
print("entity_split=disjoint")
print("fixture_checks=4 passed")
```

Expected output:

```
boundary=closed
entity_split=disjoint
fixture_checks=4 passed
```

Tests should cover exact cutoff, late corrections, duplicated source IDs, daylight-saving transitions, unknown categories, empty history, source outage, and deterministic replay. Store fixture rows in code review so a semantic change produces an obvious diff.

Cell 8: Choose a Split That Matches Deployment

An IID random split assumes exchangeable rows. Authentication rows violate this when the same user, device, or campaign appears in both sets. A grouped split keeps entities together. A temporal split trains on the past and tests on the future. Often both are needed: train before a cutoff and prevent entity overlap where the deployment claim concerns new entities.

Use three roles. Training fits parameters and preprocessing. Validation selects features, hyperparameters, and thresholds. Test remains sealed for final estimation. Reusing test results for decisions turns the test into validation. With time drift, use rolling-origin evaluation: train through (t), validate on the next interval, advance, and summarize variation across folds.

Labels need a maturity gap. If the target is ATO within fourteen days, stop training examples at least fourteen days before the label snapshot, or mark censoring explicitly. A random split cannot repair immature negatives.

Offline metrics must be recomputed after removing contaminated features. A dramatic decline is evidence that the old score measured future workflow. Compare train and serving feature distributions generated by the same code path. Training-serving skew may arise from different parsers, window boundaries, lookup freshness, or default values.

Cell 9: Missingness and Adversarial Failure Tests

Feature health is part of model behavior. Simulate loss of device telemetry, delayed geolocation, new identity providers, clock skew, and duplicated bursts. Measure predictions and actions, not only null rates. A model that maps missing posture to high risk may lock out a tenant during an agent outage; one that maps it low may create an attacker bypass.

Define explicit degradation: abstain, fall back to a smaller feature set, use a conservative rule, or route to review. Select based on harm. Monitor coverage by cohort, feature age, unknown-category rate, join multiplicity, and point-in-time violations. Alert before outcome metrics mature.

Attacker-induced missingness deserves threat modeling. Can clients omit device fields? Can a proxy delay reputation enrichment past scoring? Can repeated identities poison rolling baselines? Use server-observed signals where possible, cap contributions, retain raw provenance, and test manipulated fixtures.

Notebook Conclusion: Why AUC Fell

The offline model used `reset_after_incident`, current entitlement centrality, and a device reputation value backfilled hours later. Random splitting placed the same principals and campaign in both partitions. Removing leakage, rebuilding as-of features, grouping principals, and testing on later weeks reduced offline AUC from 0.97 to 0.72. Production AUC of 0.70 now agrees within uncertainty.

The lower score is the successful result. It estimates the deployable problem. The team can improve it honestly through better decision-time instrumentation, sequences, calibrated models, and outcome sampling. Restoring 0.97 by reintroducing future facts would improve only the fiction.

Sources and Further Reading

- scikit-learn, “Common Pitfalls and Recommended Practices.” https://scikit-learn.org/stable/common_pitfalls.html
- scikit-learn, “Preprocessing Data.” <https://scikit-learn.org/stable/modules/preprocessing.html>
- pandas, “Working with Missing Data.” https://pandas.pydata.org/docs/user_guide/missing_data.html
- Google, “Rules of Machine Learning.” <https://developers.google.com/machine-learning/guides/rules-of-ml>
- Kaufman et al., “Leakage in Data Mining.” <https://doi.org/10.1145/2020408.2020496>
- Little and Rubin, *Statistical Analysis with Missing Data*, 3rd ed. <https://doi.org/10.1002/9781119482260>

- Sculley et al., “Hidden Technical Debt in Machine Learning Systems.” <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>
- NIST, *Secure Software Development Framework*, SP 800-218. <https://csrc.nist.gov/pubs/sp/800/218/final>

Interview Practicum

Debug the Miraculous Model

Prompt with evidence drops

“A login-risk model has 0.97 offline AUC and 0.61 online. Diagnose it.” The interviewer reveals in stages: password-reset fields are updated after incidents; device intelligence arrives up to two hours late; rows were randomly split; analysts investigate only challenged sessions; and production uses a different null default.

Worked Answer

Strong investigative answer

Freeze model, feature, and source versions and reproduce several production decisions from raw evidence. Build a feature lineage table containing event time, availability time, transformation, and owner. For every candidate feature, compare historical value as originally available with the current backfilled value. Remove post-outcome and post-intervention fields.

Rebuild examples as of login decision time. Add a fourteen-day maturity gap for outcomes. Split temporally and group principals or linked devices. Fit imputation, encoders, scaling, feature selection, and dimensionality reduction inside each training fold. Compare online and offline feature vectors for identical decision IDs.

The different null default is training-serving skew; unify transformation code and test outage behavior. Because only challenged sessions receive labels, observed online performance is selected. Add a randomized audit sample where ethically possible and use sampling weights. Do not claim global recall from investigated traffic alone.

Report a leakage ablation: baseline with all fields, remove reset workflow, remove late intelligence, enforce grouped temporal split, and enforce production defaults. The step changes identify how much optimism each defect contributed. Re-estimate threshold metrics at operational capacity and shadow the repaired pipeline before action.

Worked answer

The principal assertion is: for observation (i) at decision time (t_i), every feature fact (j) must satisfy `available_at_j <= t_i`; every learned preprocessing artifact must be fit without validation or test rows; and outcome observation must not influence feature construction. Event time alone is insufficient because backfilled data can describe earlier events but become known later.

A safe SQL pattern joins a login to facts on entity plus `fact.available_at <= login.decision_at`, applies event-window predicates separately, and resolves multiple versions by the latest version *available by the cutoff*. The test fixture must include a fact whose event precedes login but whose arrival follows it.

Weak answers

- “Tune regularization” treats a data defect as model variance.
- “Production drifted” is a hypothesis, not a diagnosis.
- “Drop all null rows” changes population and hides outages.
- “Use current tables for more complete history” guarantees backfill leakage.
- “AUC is still above random” ignores action thresholds and selected labels.

Follow-Up Questions and Answers

Follow-up probes with answers

A feature is available before scoring but caused by a prior intervention. Is it leakage? It may be legitimately available yet create policy dependence or treatment leakage. Define the deployment policy and causal pathway; evaluate stability when intervention changes.

How handle an IP reputation correction? Version values with validity and availability. Replay the version known at decision time; use corrected values only for analyses whose estimand permits hindsight.

Can a missingness indicator be useful? Yes when it represents stable decision-time information, but test source changes and manipulation. It does not make MNAR bias disappear.

How detect duplicate leakage across accounts? Group by resolved device, household, organization, campaign, or artifact as appropriate; measure similarity across splits and run stricter sensitivity splits.

Would you restore the old model while rebuilding? Only if its live harm is lower than fallback and contaminated features exist online at decision time without post-outcome dependence. Otherwise fall back to a reviewed rule or abstention policy.

Communication drills

Thirty seconds: “Reconstruct identical production decisions from raw facts, enforce availability-time joins, remove post-incident fields, add label maturity, group entities, split forward in time, and unify preprocessing. The honest offline AUC will fall; agreement with online behavior is the first success.”

Five minutes: Explain the three clocks, lineage audit, leakage ablation, selected-label problem, grouped temporal validation, train-serving parity, fixtures, and safe fallback.

Key Takeaways

- Event, availability, and label times must be modeled separately.
- Latest-state joins are unsuitable for historical feature reconstruction.
- Missingness is a mechanism, sometimes controlled by attackers, not a cleaning inconvenience.
- Fit every learned transformation inside the training partition.
- Grouped temporal splits and outcome-maturity gaps approximate deployment better than random rows.
- A lower leakage-free score is more valuable than a miraculous invalid score.

Chapter 5: Python, SQL, and Reproducible Analytical Workflows

Repository Walkthrough: Rebuild the Impossible-Travel Investigation

At 09:10 an investigator asks which identities authenticated from two countries within thirty minutes and whether the second event used a new device. By lunch, a notebook has produced twelve names. By Friday, nobody can reproduce the list. The query was edited in place, the notebook depended on an unrecorded package version, a nested JSON field silently changed type, and a cloud token remains in cell output.

This chapter rebuilds that investigation as a repository. The aim is not infrastructure for its own sake. A result becomes evidence only when another person can reconstruct its input, transformations, assumptions, and checks.

```
impossible-travel/
├── pyproject.toml
├── data/auth.ndjson
├── sql/{schema.sql,investigate.sql}
├── src/impossible_travel/{ingest.py,pipeline.py}
├── tests/test_pipeline.py
└── notebooks/01_discovery.ipynb
```

The notebook is deliberately last: it can discover a transformation but is not the authoritative implementation.

Commit Zero: Runtime and Data Contract

A manifest should separate runtime from development dependencies and bound supported Python versions. A lock file then records resolved transitive versions and hashes.

```
[build-system]
requires = ["setuptools==75.8.0"]
build-backend = "setuptools.build_meta"

[project]
name = "impossible-travel"
version = "0.1.0"
requires-python = ">=3.11,<3.15"
dependencies = []

[project.optional-dependencies]
dev = ["pytest==8.3.5", "ruff==0.9.7"]
```

This exercise needs no third-party runtime package. A production run manifest should still capture the interpreter, operating-system image digest, source revision, parameters, and immutable data-snapshot identifier.

The input is newline-delimited JSON. Nested objects and arrays are ordinary in identity telemetry. The last record intentionally violates the boolean contract.

```
{
  "event_id": "a1",
  "ts": "2026-08-01T10:00:00Z",
  "identity": {
    "id": "u1",
    "groups": ["engineering", "vpn"]
  },
  "source": {
    "country": "US",
    "ip": "198.51.100.7"
  },
  "device": {
    "id": "d1",
    "managed": true
  },
  "result": {
    "success": true
  }
},
{
  "event_id": "a2",
  "ts": "2026-08-01T10:18:00Z",
  "identity": {
    "id": "u1",
    "groups": ["engineering"]
  },
  "source": {
    "country": "DE",
    "ip": "203.0.113.9"
  },
  "device": {
    "id": "d9",
    "managed": false
  },
  "result": {
    "success": true
  }
},
{
  "event_id": "a3",
  "ts": "2026-08-01T10:40:00Z",
  "identity": {
    "id": "u2",
    "groups": ["finance"]
  },
  "source": {
    "country": "US",
    "ip": "198.51.100.8"
  },
  "device": {
    "id": "d2",
    "managed": true
  },
  "result": {
    "success": "true"
  }
}
```

Ingestion Boundary: Reject Ambiguity

Nested JSON is difficult because absent or malformed values acquire accidental meanings. This loader refuses missing paths, duplicate identifiers, non-boolean outcomes, and timestamps without offsets. It flattens the fields needed by the query while retaining canonical raw JSON for audit.

```
from __future__ import annotations
import hashlib, json
from datetime import datetime
from pathlib import Path
from typing import Any, Iterable

def require(record: dict[str, Any], *path: str) -> Any:
    value: Any = record
    for key in path:
        if not isinstance(value, dict) or key not in value:
            raise ValueError(f"missing JSON path: {'.'.join(path)}")
        value = value[key]
    return value

def parse_line(line: str) -> tuple[object, ...]:
    record = json.loads(line)
    success = require(record, "result", "success")
    if type(success) is not bool:
        raise ValueError("result.success must be a JSON boolean")
    stamp = datetime.fromisoformat(require(record, "ts").replace("Z", "+00:00"))
    if stamp.utcoffset() is None:
        raise ValueError("ts must include an offset")
    raw = json.dumps(record, sort_keys=True, separators=(",", ":"))
    return (require(record, "event_id"), stamp.isoformat(),
            require(record, "identity", "id"), require(record, "source", "country"),
            require(record, "device", "id"), int(require(record, "device", "managed")),
            int(success), raw)

def load_lines(lines: Iterable[str]) -> list[tuple[object, ...]]:
    rows = [parse_line(line) for line in lines if line.strip()]
```

```

ids = [row[0] for row in rows]
if len(ids) != len(set(ids)):
    raise ValueError("duplicate event_id")
return rows

def snapshot_id(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()[:12]

```

The exact outcome for `a3` is:

```
ValueError: result.success must be a JSON boolean
```

Converting the string to `True` would conceal a producer regression. Explicit failure is cheaper than a plausible wrong answer.

SQL That Exposes Its Plan

SQLite stores the normalized subset. Raw JSON remains available for forensic review, but every query need not repeat deep extraction.

```

CREATE TABLE auth_event (
  event_id TEXT PRIMARY KEY, ts TEXT NOT NULL, identity_id TEXT NOT NULL,
  country TEXT NOT NULL, device_id TEXT NOT NULL,
  managed INTEGER NOT NULL CHECK(managed IN (0,1)),
  success INTEGER NOT NULL CHECK(success IN (0,1)), raw_json TEXT NOT NULL
);
CREATE INDEX auth_identity_ts ON auth_event(identity_id, ts);

```

`LAG` compares each success with the immediately preceding success for that identity. The correlated subquery determines whether the current device appeared earlier.

```

WITH ordered AS (
  SELECT event_id, identity_id, ts, country, device_id,
    LAG(ts) OVER (PARTITION BY identity_id ORDER BY ts,event_id) previous_ts,
    LAG(country) OVER (PARTITION BY identity_id ORDER BY ts,event_id) previous_country
  FROM auth_event WHERE success=1
), candidates AS (
  SELECT *, ROUND((julianday(ts)-julianday(previous_ts))*1440,1) minutes_apart,
    NOT EXISTS (SELECT 1 FROM auth_event p
      WHERE p.identity_id=ordered.identity_id AND p.device_id=ordered.device_id
      AND p.ts<ordered.ts) new_device
  FROM ordered
)
SELECT identity_id,previous_country,country,minutes_apart,device_id,new_device
FROM candidates
WHERE previous_country<>country AND minutes_apart BETWEEN 0 AND 30
ORDER BY identity_id,ts;

```

After correcting `a3` to the JSON boolean `true`, exact output is:

identity_id	previous_country	country	minutes_apart	device_id	new_device
u1	US	DE	18.0	d9	1

This is a candidate, not proof of compromise. VPN egress and carrier routing can change apparent country without travel.

Inspect properties, not decorative SQL

Run `EXPLAIN QUERY PLAN`. SQLite wording varies by release, so assert a useful property instead of snapshotting every character:

```
import sqlite3

connection = sqlite3.connect(":memory:")
connection.executescript("""
CREATE TABLE auth_event(
    event_id TEXT PRIMARY KEY, identity_id TEXT NOT NULL,
    ts TEXT NOT NULL, country TEXT NOT NULL, device_id TEXT NOT NULL,
    success INTEGER NOT NULL
);
CREATE INDEX auth_identity_ts ON auth_event(identity_id, ts);
""")
investigation_sql = """
SELECT identity_id, ts, country
FROM auth_event INDEXED BY auth_identity_ts
WHERE identity_id = 'u1' AND ts >= '2026-08-01T00:00:00+00:00'
ORDER BY identity_id, ts
"""
plan = connection.execute("EXPLAIN QUERY PLAN " + investigation_sql).fetchall()
assert "auth_identity_ts" in "\n".join(row[-1] for row in plan)
```

In a warehouse, first bound the base scan by event-date partitions and project only needed columns. Then inspect partitions pruned, bytes read, cardinality estimates, join distribution, spill, and sort stages. A thirty-day query that scans five years because its date predicate appears too late is an analytical defect, not merely a slow database.

Independent Python Implementation

Parity is more convincing when Python does not call the SQL implementation.

```
from datetime import datetime

def find_country_jumps(rows: list[tuple[object, ...]]) -> list[dict[str, object]]:
    seen: dict[str, set[str]] = {}; prior: dict[str, tuple[datetime, str]] = {}
    findings: list[dict[str, object]] = []
    for row in sorted(rows, key=lambda r: (str(r[2]), str(r[1]), str(r[0]))):
        _, ts_text, identity, country, device, _, success, _ = row
```

```

if not success: continue
ts = datetime.fromisoformat(str(ts_text)); key = str(identity)
known = seen.setdefault(key, set())
if key in prior:
    previous_ts, previous_country = prior[key]
    minutes = (ts-previous_ts).total_seconds()/60
    if previous_country != country and 0 <= minutes <= 30:
        findings.append({"identity_id":identity,"previous_country":previous_country,
                        "country":country,"minutes_apart":round(minutes,1),
                        "device_id":device,"new_device":str(device) not in known})
    known.add(str(device)); prior[key]=(ts,str(country))
return findings

```

The tie-breaker `event_id` matters. If SQL orders tied timestamps differently from Python, the engines can disagree nondeterministically.

Tests That Protect Meaning

Tests should cover parsing, temporal logic, duplicate handling, SQL/Python parity, plan properties, and idempotent reruns. `unittest` keeps the fixture runnable in this repository.

```

from __future__ import annotations
import hashlib, json
from datetime import datetime
from pathlib import Path
from typing import Any, Iterable

def require(record: dict[str, Any], *path: str) -> Any:
    value: Any = record
    for key in path:
        if not isinstance(value, dict) or key not in value:
            raise ValueError(f"missing JSON path: {'.'.join(path)}")
        value = value[key]
    return value

def parse_line(line: str) -> tuple[object, ...]:
    record = json.loads(line)
    success = require(record, "result", "success")
    if type(success) is not bool:
        raise ValueError("result.success must be a JSON boolean")
    stamp = datetime.fromisoformat(require(record, "ts").replace("Z", "+00:00"))
    if stamp.utcoffset() is None:
        raise ValueError("ts must include an offset")
    raw = json.dumps(record, sort_keys=True, separators=(",", ":"))
    return (require(record, "event_id"), stamp.isoformat(),
            require(record, "identity", "id"), require(record, "source", "country"),
            require(record, "device", "id"), int(require(record, "device", "managed")),
            int(success), raw)

def load_lines(lines: Iterable[str]) -> list[tuple[object, ...]]:

```

```

rows = [parse_line(line) for line in lines if line.strip()]
ids = [row[0] for row in rows]
if len(ids) != len(set(ids)):
    raise ValueError("duplicate event_id")
return rows

def snapshot_id(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()[:12]

from datetime import datetime

def find_country_jumps(rows: list[tuple[object, ...]]) -> list[dict[str, object]]:
    seen: dict[str, set[str]] = {}; prior: dict[str, tuple[datetime, str]] = {}
    findings: list[dict[str, object]] = []
    for row in sorted(rows, key=lambda r: (str(r[2]), str(r[1]), str(r[0]))):
        _, ts_text, identity, country, device, _, success, _ = row
        if not success: continue
        ts = datetime.fromisoformat(str(ts_text)); key = str(identity)
        known = seen.setdefault(key, set())
        if key in prior:
            previous_ts, previous_country = prior[key]
            minutes = (ts - previous_ts).total_seconds() / 60
            if previous_country != country and 0 <= minutes <= 30:
                findings.append({"identity_id": identity, "previous_country": previous_country,
                                "country": country, "minutes_apart": round(minutes, 1),
                                "device_id": device, "new_device": str(device) not in known})
            known.add(str(device)); prior[key] = (ts, str(country))
    return findings

import unittest

GOOD = [
    '{"event_id": "a1", "ts": "2026-08-01T10:00:00Z", "identity": {"id": "u1"}, "source": {"country": "US"}, "device": {"id": "d1", "managed": true}, "result": {"success": true}}',
    '{"event_id": "a2", "ts": "2026-08-01T10:18:00Z", "identity": {"id": "u1"}, "source": {"country": "DE"}, "device": {"id": "d9", "managed": false}, "result": {"success": true}}'
]

class PipelineTest(unittest.TestCase):
    def test_jump(self):
        result = find_country_jumps(load_lines(GOOD))
        self.assertEqual(result[0]["minutes_apart"], 18.0)
        self.assertIs(result[0]["new_device"], True)
    def test_string_boolean(self):
        bad = GOOD[0].replace('{"success": true}', '{"success": "true"}')
        with self.assertRaisesRegex(ValueError, "JSON boolean"): parse_line(bad)
    def test_duplicate(self):
        with self.assertRaisesRegex(ValueError, "duplicate"): load_lines([GOOD[0], GOOD[0]])

if __name__ == "__main__": unittest.main()

```

Expected output:

```
...
```

```
-----
Ran 3 tests
```

```
OK
```

Do not convert every empirical expectation into a brittle assertion. “Exactly twelve findings” fails when legitimate late events arrive. Monitor input count, null rate, identity cardinality, and finding rate within documented tolerances; reserve hard failure for broken schemas and impossible values. Monitoring detects change. Tests define invalidity.

Secrets Are Inputs, Not Notebook Content

Use a workload identity or secret manager. Local code should fail without printing the secret:

```
import os
def warehouse_token() -> str:
    token=os.environ.get("SECURITY_WAREHOUSE_TOKEN")
    if not token: raise RuntimeError("SECURITY_WAREHOUSE_TOKEN is not configured")
    return token
```

Avoid credentials in committed `.env` files, command arguments visible in process listings, query URLs, exception traces, and notebook output. Prefer short-lived credentials, least-privilege read roles, separate development and production identities, and audited access. Fixtures should replace personal identifiers and IP addresses unless their retention is necessary and approved.

Notebook Exit Criteria

A notebook is useful while the analyst asks which transformation explains the observation. It becomes hazardous when hidden state is treated as a scheduled pipeline. Promote work when the answer is recurring, shared, used for enforcement, or expensive enough that silent drift matters.

Promotion means moving logic into typed functions; replacing mutable downloads with snapshot IDs; parameterizing the observation interval instead of using implicit “now”; storing reviewed SQL with deterministic ordering; recording revision, runtime, parameters, counts, and snapshot hash; adding schema, parity, idempotence, and plan checks; scheduling with a service identity, timeout, bounded retries, and alerts; and publishing atomically from staging only after validation.

```
{"run_id":"2026-08-02T06:00Z","source_revision":"8c31f2a","input_snapshot":"70dc8a7c8225","window_start":"2026-07-03T00:00:00Z","window_end":"2026-08-02T00:00:00Z","input_rows":8420911,"findings":37,"status":"published"}
```

Retry only likely transient failures. A schema violation will not heal on attempt three. Backfills accept explicit intervals and must produce the same state twice. Alert on stale successful output as well as exceptions: a green job that published zero bytes is unhealthy.

Reproducibility Under Real Failure

The happy path is the least interesting test of a pipeline. Consider a scheduler that reads a date partition, writes findings, and crashes after the write but before it records success. If the retry appends, every finding is duplicated. Give the output a natural idempotency key—perhaps `(identity_id, current_event_id, rule_version)`—and write with an upsert or transactional replace. When the transformation itself changes, increment the rule version rather than silently overwriting historical meaning.

Late data creates a different problem. Suppose event `a2` arrives today but belongs to yesterday. Reprocessing only today's partition misses its relationship to `a1`; appending it to yesterday changes yesterday's result. A bounded correction policy should identify which input partitions can still change, rebuild the affected output interval, and publish a new revision. The run manifest should distinguish `observed_at`, `event_time`, and `published_at` so an auditor can reconstruct both operational knowledge and corrected history.

Data snapshots require more than a file name. A useful snapshot descriptor includes object URI or table version, content hash where feasible, extraction query revision, inclusive and exclusive time boundaries, row count, schema fingerprint, and source retention policy. If regulations prohibit retaining raw security data, retain a governed reference and enough aggregate evidence to verify the analysis without copying restricted content into the repository.

Test categories have different owners

A unit test belongs close to a function and runs on every change. A contract test confirms that the producer and consumer agree about field names, types, enums, and timestamp semantics. A data-quality test checks a particular snapshot for broken invariants. A statistical test watches distributional movement without asserting that movement is necessarily wrong. An integration test exercises database transactions, permissions, and serialization. A replay test processes a historical interval end to end and compares a versioned result.

Combining these into one “pipeline passed” flag makes diagnosis slower. A new country code may violate an outdated enum but be valid data; a large fall in volume may be a collector outage; a changed finding count may be the intended result of a rule revision. Each failure should name its violated assumption, observed value, expected boundary, snapshot, and remediation owner.

Tests also need adversarial cases. Use a timestamp exactly at the thirty-minute boundary, two events with identical timestamps, reversed arrival order, a failed login between two successes, a device seen only after the candidate event, Unicode identity values, missing country, and an input line large enough to stress memory. The expected behavior must be stated before implementation. Boundary fixtures prevent a future refactor from changing `BETWEEN 0 AND 30` into a subtly different policy.

Cost-Aware Baselines Without Semantic Drift

Precomputation is safe only when the aggregate retains sufficient information. A daily table with `login_count`, `device_count`, and `last_country` cannot reproduce an impossible-travel sequence: ordering inside the day has been destroyed. Conversely, daily count, sum, minimum, and maximum can support many rolling summaries exactly. Decide physical grain from the hardest required query, not from storage fashion.

Query optimization should proceed as an experiment. Record the reference query's plan and answer on a fixed snapshot. Change one factor—predicate placement, index, clustering key, materialized daily aggregate, or join strategy. Measure wall-clock time, compute or bytes scanned, peak memory or spill, and answer parity. A query that is twice as fast but loses late-arriving records is not optimized; it computes a different function.

Parameterization deserves equal care. Bind values through the database driver rather than formatting identity strings into SQL. Maintain an allowlist when a parameter selects a column or ordering direction, because bind parameters generally represent values, not SQL identifiers. Log a redacted parameter manifest: time bounds and rule version are useful; access tokens and full lists of sensitive identities are not.

Finally, distinguish reproducible from repeatable. A run is repeatable when the same operator can execute it again. It is reproducible when an independent operator can obtain the same result from the recorded materials. It is robust when reasonable changes—machine, batch size, event arrival order—do not change meaning. A mature analytical repository aims for all three and documents the cases, such as nondeterministic approximate algorithms, where exact equality is neither possible nor required.

A final peer-review checklist should be answerable from the repository alone: What question and interval were evaluated? Which snapshot supplied the records? Which fields can be absent? Which ordering resolves timestamp ties? Which query plan was accepted? Which outputs are exact versus approximate? Which principal can read the source and publish the result? How does an operator rerun one interval without touching others? If any answer exists only in the original analyst's memory, the workflow is not ready to become shared evidence.

Sources and Further Reading

1. Python Packaging Authority, Writing `pyproject.toml`.
2. Python Software Foundation, `unittest`.
3. SQLite, Query Planning and Window Functions.
4. NIST, Secure Software Development Framework, SP 800-218.
5. NIST, Digital Identity Guidelines, SP 800-63.
6. OWASP, Secrets Management Cheat Sheet.
7. The Twelve-Factor App, `Config`.
8. Martin Kleppmann, *Designing Data-Intensive Applications*, O'Reilly, 2017.

Interview Practicum

The Baseline Query Melts the Warehouse

Panel prompt

“A query computes each identity's thirty-day authentication baseline across billions of events. It takes four hours and spills. Analysts need twenty minutes.” The panel then reveals that events arrive forty-eight hours late and no new database will be approved.

Worked Answer

Strong answer as constraints arrive

Ask for the plan, physical layout, daily volume, identity cardinality, skew, freshness requirement, and exact statistics. Counts, quantiles, sequences, and distinct devices do not have identical aggregation properties.

Require predicates that prune partitions, project only baseline columns, and verify pruning. Check whether service identities cause skew. Align clustering or sort order with identity and event time if supported. For mergeable features, materialize daily identity aggregates and combine thirty bounded partitions. Means merge from sum and count; variance from count, mean, and second moment. Exact distinct counts and quantiles do not merge like sums, so either retain sufficient detail or document sketch error.

Rebuild the last two or three daily partitions for the late-data correction horizon, then recompute affected rolling outputs. If raw sequence features are necessary, use aggregates to identify candidates and read raw events only for them. Compare old and new outputs per identity on a frozen replay; investigate mismatches by lateness, null pattern, and high-volume identity. The launch gate includes parity or declared approximation tolerance, bytes scanned, spill, and wall time.

Optimization shortcuts that fail

“Use Spark” ignores the platform constraint and may preserve the scan. “Add an index” is incomplete for a columnar warehouse. “Cache it” omits invalidation. “Sample one percent” can erase rare identities and corrupt individual baselines.

Follow-Up Questions and Answers

Warehouse follow-ups

Exact distinct devices are mandatory—what then? Maintain bounded identity-device membership at a daily grain and compute distinctness over thirty partitions, accepting its cost. Never call a probabilistic sketch exact.

What belongs in the on-call page? Run ID, failed stage, interval, snapshot, last successful publication, count deltas, and sanitized-log link—never credentials or raw sensitive events.

Thirty-second answer: “I would prove partition pruning, replace repeated raw scans with recomputable daily aggregates, rebuild the late-data horizon, and demonstrate semantic parity on an immutable replay before changing production.”

Key Takeaways

- Reproducibility joins identified code, runtime, parameters, and immutable input.
- Validate nested JSON before coercion; plausible conversions hide telemetry regressions.
- SQL performance starts with bounded scans and inspected plans.
- Independent SQL and Python implementations enable semantic parity tests.
- Tests distinguish impossible data from unusual data; pipelines must survive retries and backfills.
- A recurring or consequential notebook should graduate into a package and monitored pipeline.

Chapter 6: Visualization, Communication, and Analytical Storytelling

Redesign Studio: Four Charts Enter, One Decision Leaves

The wall display says alert volume is down 42 percent. The security vice president calls the program green. An analyst points out that confirmed compromises doubled. Both statements are true, and the dashboard has made them appear contradictory because it visualizes production rather than exposure, quality, or consequence.

This chapter is a critique studio. Each station begins with a chart that can be computed correctly and still mislead. We calculate the replacement, specify visual encodings, and state what the display permits a reader to conclude. There is no generic dashboard architecture here. The unit of work is a visual claim.

Station 1: Replace the Alert Mountain

Suppose two weeks have these counts:

week	eligible_logins	alerts	reviewed	confirmed_compromises
W1	2,000,000	1,000	900	20
W2	3,500,000	580	540	42

A bar chart of alerts celebrates a fall from 1,000 to 580. Its denominator is invisible, review coverage changed, and the outcome moved in the opposite direction. Calculate rates per 100,000 eligible logins and precision among reviewed alerts:

```
rows = [
    {"week": "W1", "eligible": 2_000_000, "alerts": 1_000, "reviewed": 900, "confirmed": 20},
    {"week": "W2", "eligible": 3_500_000, "alerts": 580, "reviewed": 540, "confirmed": 42},
]
for r in rows:
    r["alert_rate"] = 100_000 * r["alerts"] / r["eligible"]
    r["compromise_rate"] = 100_000 * r["confirmed"] / r["eligible"]
    r["review_precision"] = r["confirmed"] / r["reviewed"]
    print(r["week"], f'{r["alert_rate"]:.2f}', f'{r["compromise_rate"]:.2f}',
          f'{100 * r["review_precision"]:.2f}%')
```

Exact output:

```
W1 50.00 1.00 2.22%
W2 16.57 1.20 7.78%
```

The redesign is a small multiple, not a dual-axis chart: alert rate and confirmed-compromise rate get separate aligned panels with the same weekly x-axis; review precision appears as a third panel. Each mark includes

numerator and denominator in its tooltip or adjacent annotation. The decision memo now reads: alert burden per login fell sharply, reviewed-alert yield increased, yet observed compromise incidence rose 20 percent. That combination calls for a coverage investigation, not a green status.

Rare-event denominators

Never divide by “all traffic” when only a subset could generate the event. A privileged-action detector should use eligible privileged actions, not HTTP requests. A phishing report rate might use delivered messages or active mailboxes depending on the question; those denominators answer different questions. Display the population definition and observation window beside the chart.

When denominators are small, raw rates jump. For 1 compromise in 100 logins, the observed rate is 1 percent, but uncertainty is wide. Wilson's 95 percent interval for a binomial proportion is

$$\left[\frac{\hat{p} + z^2/(2n)}{1 + z^2/n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n + z^2/(4n)}} \right]$$

The following code computes it without a statistics package:

```
from math import sqrt
def wilson(k: int, n: int, z: float=1.96) -> tuple[float,float]:
    p=k/n; d=1+z*z/n
    c=(p+z*z/(2*n))/d
    h=z*sqrt(p*(1-p)/n+z*z/(4*n*n))/d
    return c-h,c+h
print(tuple(round(100*x,2) for x in wilson(1,100)))
```

Expected output is (0.18, 5.45). Plot the point with this interval; do not give a tiny team the same visual certainty as a population of millions.

Station 2: Audit the Geometry

Three common choices create false drama or false calm.

Truncated bars. Bars encode length from a baseline. Showing values 96 and 100 on an axis beginning at 95 makes a four-percent change occupy eighty percent of the drawing. Begin ordinary bars at zero. If the meaningful question is deviation from a service objective, use a dot plot centered on the objective and label the transformation.

Dual axes. Mapping alert count to a left axis and compromise loss to an independently scaled right axis lets the author manufacture apparent correlation. Use vertically aligned panels or normalize both to a clearly named baseline index. If indexed values are used, preserve raw values in labels.

Log scales. A log scale is useful when values span orders of magnitude, such as host byte counts from 100 to 100,000,000. It turns equal ratios into equal distances. It is invalid for zero and awkward for signed differences. Label powers explicitly and explain that the distance from 100 to 1,000 equals the distance from 1,000 to 10,000. For an executive ranking of absolute financial exposure, a log axis may visually understate the largest item; use it only when multiplicative comparison is the task.

A composition chart also deserves suspicion. A pie chart of alert categories can show that 60 percent are phishing, but it cannot show that total alerts tripled. A stacked area chart makes only its baseline category easy to compare. Prefer direct lines for a few categories or a heat map when category-by-time density is the actual question.

Station 3: Put Calibration on Trial

A classifier assigns probabilities, but the dashboard shows only ROC AUC. In rare-event work, a model may rank well while producing probabilities that cannot support staffing or automated action.

Use ten equal-width probability bins or, when predictions cluster, quantile bins with displayed boundaries. Within bin (b) , calculate mean predicted probability and observed event frequency:

$$ECE = \sum_b \frac{n_b}{N} \left| \bar{p}_b - \bar{y}_b \right|$$

```
pred = [0.02,0.04,0.08,0.12,0.18,0.55,0.62,0.76,0.84,0.93]
truth = [0,0,0,0,1,0,1,1,1,1]
bins = [(0,.2),(.2,.6),(.6,1.0001)]
ece=0
for lo,hi in bins:
    idx=[i for i,p in enumerate(pred) if lo<=p<hi]
    mp=sum(pred[i] for i in idx)/len(idx)
    oy=sum(truth[i] for i in idx)/len(idx)
    ece += len(idx)/len(pred)*abs(mp-oy)
    print(f"[{lo:.1f},{hi:.1f}) n={len(idx)} predicted={mp:.3f} observed={oy:.3f}")
print(f"ECE={ece:.3f}")
```

Exact output:

```
[0.0,0.2) n=5 predicted=0.088 observed=0.200
[0.2,0.6) n=1 predicted=0.550 observed=0.000
[0.6,1.0) n=4 predicted=0.787 observed=1.000
ECE=0.196
```

The reliability diagram plots observed rate against predicted probability with a 45-degree reference. Add bin counts below it: a point based on four cases must not look as authoritative as one based on forty thousand. ECE is a summary, not a substitute for the diagram; binning choices can hide local failure.

For discrimination under class imbalance, show the precision-recall curve and label the prevalence baseline. ROC plots can remain useful for comparing ranking across thresholds, but a low false-positive rate multiplied by millions of benign events may still overwhelm operations. Mark the deployed threshold and annotate its precision, recall, alerts per day, and confidence interval.

Station 4: Separate Drift into Questions

“The model drifted” conflates at least three displays.

1. **Input drift:** did feature distributions change? For categorical device posture, show shares by category and an “unknown” category rather than dropping missing values. For a numeric feature, overlay empirical cumulative distributions or use fixed, meaningful bins.
2. **Prediction drift:** did the score distribution move? Show quantiles over time, with deployment and policy changes annotated.
3. **Performance drift:** conditional on matured labels, did precision, recall, calibration, or cost change? Align each metric to the event cohort date, not the date on which its delayed label arrived.

Population Stability Index is often overinterpreted. For reference distribution shares $\{r_i\}$ and current shares $\{c_i\}$,

$$\text{PSI} = -\sum_i (c_i - r_i) \log(c_i / r_i)$$

```
from math import log
reference=[.70,.20,.10]; current=[.50,.30,.20]
parts=[(c-r)*log(c/r) for r,c in zip(reference,current)]
print([round(x,4) for x in parts], round(sum(parts),4))
```

Expected output:

```
[0.0673, 0.0405, 0.0693] 0.1772
```

This says the binned distribution differs; it does not identify cause, harm, or a universal retraining threshold. Show category contributions so reviewers can see what moved. Fix bins from the reference interval and handle zero shares with a declared smoothing rule.

Station 5: Draw a Causal Effect Without Hiding Time

A security-key rollout begins in week 5. A before/after line for treated employees drops from 8 to 4 compromises per 100,000 logins. That does not establish an effect if the untreated group fell from 6 to 3 during the same incident lull.

The difference-in-differences estimate is

$$[(4-8)-(3-6)=-1]$$

compromise per 100,000 logins. The redesign shows treated and comparison rates across all weeks, an intervention marker, pre-period trend, and a second panel of estimated weekly treatment effects with confidence intervals. Do not plot only the aggregate pre/post bars: they conceal pre-trend failure and timing.

If rollout timing varies, an event-study display aligns units by time relative to adoption. Include leads before treatment; nonzero pre-treatment coefficients warn that parallel trends may be implausible. State whether

intervals account for clustering. A visual recommendation should say “consistent with one fewer compromise per 100,000 under the stated comparison,” not “security keys prevented every observed decline.”

Accessibility Review at the Door

Color is redundant, never sole meaning. Use line style, point shape, direct labels, or texture in addition to hue. A red/green status display excludes many readers and turns uncertainty into traffic lights. Prefer a blue/orange palette with text labels such as “within range” and “outside range.”

Meet WCAG contrast requirements for text and essential graphical objects. Avoid low-contrast gridlines competing with data. Provide concise alt text that states the chart's question, principal pattern, exception, and uncertainty. Preserve reading order and data tables for keyboard and screen-reader users. Do not rely on hover for denominators or threshold values.

Example alt text: “Weekly confirmed-compromise rate rose from 1.00 to 1.20 per 100,000 eligible logins while alert rate fell from 50.00 to 16.57. Review precision rose from 2.22 to 7.78 percent. The display does not determine why compromise incidence increased.”

Studio Critique: Match Display to Audience

An investigator needs event-level drill-down and reversible filters. A model owner needs cohort-aligned calibration, drift, threshold, and latency diagnostics. An executive needs the decision, affected population, magnitude, uncertainty, and alternatives. Giving all three audiences the same wall of tiles is not consistency; it is refusal to edit.

For a decision memo, lead with a complete sentence: “Observed compromise incidence increased despite lower alert burden; hold the proposed coverage reduction until the missing segment is explained.” Follow with the smallest figure that supports the sentence, then limitations and requested action. Avoid “data storytelling” that assigns motives to an attacker or causal credit to a control without evidence.

Critique Bench: Questions Before Pixels

Before accepting a chart, conduct a five-minute verbal review. What comparison must the reader make: position, length, angle, area, color, or animation? Position on a shared scale is generally easier to compare than area or angle, which is why a sorted dot plot often beats bubbles or a pie. What data disappeared through aggregation? Which records were excluded? Does the visual encode an outcome, a proxy, or an operational activity? What action would change if the pattern reversed?

Then inspect the calculation. Confirm time zones and whether weekly buckets are calendar weeks or rolling seven-day windows. Check whether a “daily average” is the mean of tenant averages—which weights small and large tenants equally—or total events divided by total exposure. If customers have radically different sizes, show both the population-weighted rate and the distribution of tenant rates. Simpson's paradox can make the aggregate improve while every important segment worsens because the mixture changed.

Uncertainty is not limited to confidence intervals. Labels may be incomplete, identity resolution may merge people, and controls may change what becomes observable. Encode maturity with open points, hatching, or a separate recent-period band rather than extending a confident line into unresolved cohorts. Annotate known instrumentation changes on the time axis. A vertical line labeled “collector v3 deployed” often explains more than another anomaly color.

Small multiples require disciplined scales. Shared scales support comparison but can flatten a small segment; free scales expose within-segment shape but prohibit magnitude comparison. State the choice. Sorting categories by value helps ranking; preserving a stable semantic order helps repeated scanning. Do not reorder severity levels alphabetically or let a dashboard resort them each refresh.

Network diagrams deserve a special warning. A force-directed hairball converts node count into visual authority while edge overlap hides paths. Begin with the question. For reachability, show the few typed paths to the protected resource. For neighborhood review, use an ego graph with degree limits. For group comparison, an adjacency matrix or summarized edge counts may be clearer. Label edge direction and time validity; otherwise a visually connected node can imply access that never existed simultaneously.

The final review is linguistic. Titles should make claims, subtitles should define population and period, and annotations should distinguish observation from explanation. “Compromise rate increased after rollout” is descriptive. “Rollout caused compromise” is causal. A responsible display never smuggles the second sentence through the grammar of the first.

Export matters too. Review the figure at presentation size, on a grayscale printer, at 200 percent zoom, and in the document where it will actually appear. A dashboard tooltip cannot rescue a screenshot pasted into an incident report. Preserve a machine-readable table beside static exports, record the query and snapshot used to generate them, and put the generation timestamp somewhere less prominent than the event-time coverage. “Updated today” does not mean the underlying labels are current.

When animation is proposed, ask whether motion reveals transition or merely attracts attention. A scrubber can help inspect an evolving attack path; autoplaying points usually obstruct comparison and creates inaccessible content. Prefer stable views with selectable time states and a textual summary of what changed.

Sources and Further Reading

1. Edward Tufte, *The Visual Display of Quantitative Information*, Graphics Press.
2. Tamara Munzner, *Visualization Analysis and Design*, CRC Press, 2014.
3. Claus Wilke, *Fundamentals of Data Visualization*, O'Reilly, 2019.
4. W3C, [Web Content Accessibility Guidelines 2.2](#).
5. NIST, *Visualizing Information for Cybersecurity*.
6. Brier, “Verification of Forecasts Expressed in Terms of Probability,” *Monthly Weather Review*, 1950.
7. Niculescu-Mizil and Caruana, “Predicting Good Probabilities with Supervised Learning,” ICML 2005.
8. Cunningham, *Causal Inference: The Mixtape*, Yale University Press, 2021.

Interview Practicum

The Executive Demands One Light

Prompt with live evidence

“Alert volume is down 42 percent, precision is up, compromise incidence is also up, and the executive wants one red/green indicator.” Then the interviewer reveals that only 60 percent of alerts have matured labels and the product launch doubled eligible login volume.

Worked Answer

Strong response

Decline the false compression while meeting the need for a decision. Offer a headline state tied to an explicit action—“coverage review required”—plus three supporting measures: compromise rate per eligible login, alert burden per analyst hour, and reviewed-alert precision with label maturity. Recalculate all count metrics using eligible exposure. Separate mature cohorts from recent incomplete ones and show their denominators.

Use aligned small multiples, not a weighted score whose weights conceal tradeoffs. If governance insists on a status, define it from preapproved conditions: for example, red if the mature compromise-rate interval exceeds the guardrail, amber if label maturity is insufficient, and green only when outcome and operational guardrails pass. The state must carry an “insufficient evidence” condition; missing labels are not green.

Misleading executive compromises

An average of normalized metrics allows a favorable alert decline to cancel rising harm. A dual-axis chart can visually force agreement. Showing counts ignores the launch-driven exposure change. Picking red “to be safe” avoids the analytical question and eventually destroys trust in status signals.

Follow-Up Questions and Answers

Display-review follow-ups

What if compromise labels arrive sixty days late? Freeze performance panels by event cohort and maturity; use leading operational measures separately, never as if they were confirmed outcomes.

What if the executive has ten seconds? State the action and reason: “Do not reduce coverage: confirmed compromise per eligible login is higher, and 40 percent of recent labels are unresolved.”

How would you test the redesign? Give representative analysts and leaders decision tasks, measure whether they identify the correct direction and denominator, audit accessibility, and compare decisions—not aesthetic preference.

Key Takeaways

- Chart rates against the population that could experience the event and display numerator, denominator, and uncertainty.
- Calibration, discrimination, drift, and causal effect answer different questions and require different displays.
- Visual geometry carries assumptions: zero baselines, log scales, binning, and dual axes are analytical decisions.
- Accessibility requires redundant encoding, contrast, alt text, and non-hover access to essential values.
- A decision display should preserve conflict when the evidence conflicts; a single score must not make harm disappear.

Chapter 7: Machine Learning Methods and Selection

Benchmark Notebook: Three Models, One Review Queue

The product requirement is not “maximize AUC.” Each hour, a login service produces more suspicious sessions than investigators can review. The team can inspect the top 25 sessions in a batch. Missing a malicious login costs far more than reviewing a benign one, yet an uncalibrated probability cannot safely drive an automatic challenge.

We will build an imbalanced temporal fixture, compare logistic regression, a decision tree, and histogram gradient boosting, calibrate the selected model, and evaluate ranking, capacity, cost, stability, and interpretation. The fixture is synthetic so its mechanism is visible; the validation design is intended for real data.

Fixture: Make Time and Base Rate Visible

Each row represents a login. Features available at decision time are failed attempts in the preceding hour, device age, distance from the user's usual region, ASN rarity, and whether the session has strong authentication. Attack prevalence rises in the final period and the relationship between distance and attack changes slightly—a small regime shift.

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import average_precision_score, roc_auc_score, brier_score_loss
from sklearn.calibration import CalibratedClassifierCV
from sklearn.frozen import FrozenEstimator

rng=np.random.default_rng(27); n=6000
day=np.repeat(np.arange(60),100)
failed=rng.poisson(0.7,n); device_age=rng.exponential(120,n)
distance=rng.gamma(1.5,350,n); asn_rare=rng.binomial(1,.08,n)
strong_auth=rng.binomial(1,.62,n)
logit=-4.7+0.72*failed-0.004*np.minimum(device_age,365)+0.0015*distance
logit += 1.45*asn_rare-1.2*strong_auth+0.8*((distance>900)&(device_age<7))
logit += .035*np.maximum(day-44,0)
prob=1/(1+np.exp(-logit)); y=rng.binomial(1,prob)
X=np.c_[failed,device_age,distance,asn_rare,strong_auth]
train=day<45; test=day>=45
print(X.shape, int(y.sum()), f"train={y[train].mean():.4f}", f"test={y[test].mean():.4f}")
```

With the specified seed, expected output in the companion environment is:

```
(6000, 5) 177 train=0.0284 test=0.0327
```

The positive class is rare, and prevalence differs across time. A random split would leak future regime composition into training and produce a deceptively stable estimate. The first 45 days are development data; the last 15 form a locked temporal test.

Metrics Before Models

Let the review capacity be $(k=25)$ per 1,500-event test period. For scores (s_i) , precision at (k) is positives among the largest (k) scores divided by (k) ; recall at (k) divides by all positives. A cost policy assigns 1 unit to a benign review and 40 units to a missed malicious login.

```
import numpy as np

def top_k(y, score, k=25):
    chosen=np.argsort(score)[-k:]
    tp=int(y[chosen].sum()); positives=int(y.sum())
    return tp/k, tp/positives, chosen

def queue_cost(y, score, k=25, review_cost=1, miss_cost=40):
    precision, recall, chosen=top_k(y,score,k)
    tp=int(y[chosen].sum()); fp=k-tp; fn=int(y.sum())-tp
    return fp*review_cost+fn*miss_cost, {"tp":tp,"fp":fp,"fn":fn}
```

Average precision summarizes the precision-recall ranking across thresholds and should be read against prevalence. ROC AUC remains informative about pairwise ranking but can look excellent while top-of-queue precision is unusable. Brier score evaluates squared probability error; calibration curves will reveal where errors occur.

The operational metric is not accuracy. Predicting every event benign already achieves about 96.7 percent accuracy on the test period and catches nothing.

Contenders and an Honest Search

Logistic regression is the low-variance, inspectable baseline. Scaling is inside its pipeline so each fold estimates scaling only on its training portion. The shallow tree captures thresholds and interactions but is prone to instability. Histogram boosting can learn nonlinearities and interactions with regularization.

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
```

```

from sklearn.metrics import average_precision_score, roc_auc_score, brier_score_loss
from sklearn.calibration import CalibratedClassifierCV
from sklearn.frozen import FrozenEstimator

rng=np.random.default_rng(27); n=6000
day=np.repeat(np.arange(60),100)
failed=rng.poisson(0.7,n); device_age=rng.exponential(120,n)
distance=rng.gamma(1.5,350,n); asn_rare=rng.binomial(1,.08,n)
strong_auth=rng.binomial(1,.62,n)
logit=-4.7+0.72*failed-0.004*np.minimum(device_age,365)+0.0015*distance
logit += 1.45*asn_rare-1.2*strong_auth+0.8*((distance>900)&(device_age<7))
logit += .035*np.maximum(day-44,0)
prob=1/(1+np.exp(-logit)); y=rng.binomial(1,prob)
X=np.c_[failed,device_age,distance,asn_rare,strong_auth]
train=day<45; test=day>=45

def top_k(y, score, k=25):
    chosen=np.argsort(score)[-k:]
    tp=int(y[chosen].sum()); positives=int(y.sum())
    return tp/k, tp/positives, chosen

def queue_cost(y, score, k=25, review_cost=1, miss_cost=40):
    precision, recall, chosen=top_k(y,score,k)
    tp=int(y[chosen].sum()); fp=k-tp; fn=int(y.sum())-tp
    return fp*review_cost+fn*miss_cost, {"tp":tp,"fp":fp,"fn":fn}

models={
    "logistic": Pipeline([("scale",StandardScaler()),
        ("model",LogisticRegression(max_iter=2000,class_weight="balanced",C=.2))]),
    "tree": DecisionTreeClassifier(max_depth=4,min_samples_leaf=30,
        class_weight="balanced",random_state=27),
    "boosting": HistGradientBoostingClassifier(max_iter=120,max_leaf_nodes=15,
        learning_rate=.06,l2_regularization=2.0,random_state=27)
}

for model in models.values(): model.fit(X[train],y[train])
for name,model in models.items():
    score=model.predict_proba(X[test])[:,1]
    p,r,_=top_k(y[test],score)
    cost,parts=queue_cost(y[test],score)
    print(name, f"AP={average_precision_score(y[test],score):.3f}",
        f"ROC={roc_auc_score(y[test],score):.3f}",
        f"P@25={p:.3f}", f"R@25={r:.3f}", f"cost={cost}",parts)

```

Expected output for the pinned fixture is:

```

logistic AP=0.357 ROC=0.870 P@25=0.560 R@25=0.286 cost=1411 {'tp': 14, 'fp': 11, 'fn': 35}
tree AP=0.107 ROC=0.757 P@25=0.200 R@25=0.102 cost=1780 {'tp': 5, 'fp': 20, 'fn': 44}
boosting AP=0.169 ROC=0.821 P@25=0.280 R@25=0.143 cost=1698 {'tp': 7, 'fp': 18, 'fn': 42}

```

These numbers must be regenerated when library versions change; that is why the environment is part of the result. Logistic regression wins this fixture at the actual review capacity. The higher-capacity models overfit a

small positive class and unstable interactions. The relevant advantage is seven more discoveries than the tree in the first 25 and 369 lower cost units—not a vague claim that linear models are always safer.

Nested validation for tuning

The locked test cannot select hyperparameters. On the 45-day development interval, an outer temporal split estimates generalization and an inner temporal split selects settings. The inner folds must precede their validation windows.

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import average_precision_score, roc_auc_score, brier_score_loss
from sklearn.calibration import CalibratedClassifierCV
from sklearn.frozen import FrozenEstimator

rng=np.random.default_rng(27); n=6000
day=np.repeat(np.arange(60),100)
failed=rng.poisson(0.7,n); device_age=rng.exponential(120,n)
distance=rng.gamma(1.5,350,n); asn_rare=rng.binomial(1,.08,n)
strong_auth=rng.binomial(1,.62,n)
logit=-4.7+0.72*failed-0.004*np.minimum(device_age,365)+0.0015*distance
logit += 1.45*asn_rare-1.2*strong_auth+0.8*((distance>900)&(device_age<7))
logit += .035*np.maximum(day-44,0)
prob=1/(1+np.exp(-logit)); y=rng.binomial(1,prob)
X=np.c_[failed,device_age,distance,asn_rare,strong_auth]
train=day<45; test=day>=45

outer=TimeSeriesSplit(n_splits=3)
grid={"max_leaf_nodes":[7,15],"learning_rate": [.04,.08],"l2_regularization": [1.,4.]}
outer_ap=[]
for outer_train,outer_valid in outer.split(X[train]):
    search=GridSearchCV(
        HistGradientBoostingClassifier(max_iter=100,random_state=27),grid,
        scoring="average_precision",cv=TimeSeriesSplit(n_splits=3),n_jobs=1)
    search.fit(X[train][outer_train],y[train][outer_train])
    score=search.best_estimator_.predict_proba(X[train][outer_valid])[:,1]
    outer_ap.append(average_precision_score(y[train][outer_valid],score))
print([round(x,3) for x in outer_ap], round(float(np.mean(outer_ap)),3))
```

Expected output:

```
[0.161, 0.222, 0.099] 0.161
```

Variation across outer folds is itself evidence. Report the fold distribution and dates, not only the mean. If multiple events belong to one identity or campaign, time splitting alone may still leak entity-specific patterns. Use grouped temporal blocks or purge an interval around fold boundaries when feature windows overlap.

Probability Repair Without Test Leakage

Class weighting changes the training objective and often distorts raw probabilities. Calibration requires data not used to fit the base model. Reserve the last portion of development data for calibration or use cross-validated calibration within development. Never fit the calibrator on the locked test.

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import average_precision_score, roc_auc_score, brier_score_loss
from sklearn.calibration import CalibratedClassifierCV
from sklearn.frozen import FrozenEstimator

rng=np.random.default_rng(27); n=6000
day=np.repeat(np.arange(60),100)
failed=rng.poisson(0.7,n); device_age=rng.exponential(120,n)
distance=rng.gamma(1.5,350,n); asn_rare=rng.binomial(1,.08,n)
strong_auth=rng.binomial(1,.62,n)
logit=-4.7+0.72*failed-0.004*np.minimum(device_age,365)+0.0015*distance
logit += 1.45*asn_rare-1.2*strong_auth+0.8*((distance>900)&(device_age<7))
logit += .035*np.maximum(day-44,0)
prob=1/(1+np.exp(-logit)); y=rng.binomial(1,prob)
X=np.c_[failed,device_age,distance,asn_rare,strong_auth]
train=day<45; test=day>=45

fit_days=day<35; calibration_days=(day>=35)&(day<45)
base=Pipeline([("scale",StandardScaler()),
               ("model",LogisticRegression(max_iter=2000,class_weight="balanced",C=.2))])
base.fit(X[fit_days],y[fit_days])
calibrated=CalibratedClassifierCV(FrozenEstimator(base),method="isotonic")
calibrated.fit(X[calibration_days],y[calibration_days])
raw=base.predict_proba(X[test])[:,1]
fixed=calibrated.predict_proba(X[test])[:,1]
print(f"raw Brier={brier_score_loss(y[test],raw):.4f}")
print(f"calibrated Brier={brier_score_loss(y[test],fixed):.4f}")
print(f"mean predicted={fixed.mean():.4f} observed={y[test].mean():.4f}")
```

Expected output:

```
raw Brier=0.1569
calibrated Brier=0.0263
mean predicted=0.0280 observed=0.0327
```

Isotonic calibration can overfit when positives are scarce. Compare sigmoid calibration, inspect reliability bins with counts, and bootstrap by time block. Calibration does not improve ranking by design; it makes a score more interpretable as probability. A ranking queue may choose the uncalibrated model if ordering is best, while an automated step-up policy needs trustworthy probabilities and separate safety review.

Interpretation as Falsifiable Investigation

For logistic regression, standardized coefficients show direction and relative log-odds association, conditional on the feature set. They do not prove causes. Correlated variables can exchange coefficient magnitude.

For any model, permutation importance measures the performance decrease after shuffling one feature on a held-out set. It fails when correlated features substitute for one another and becomes misleading if shuffling creates impossible combinations. Partial dependence averages predictions over synthetic feature changes; accumulated local effects can behave better with correlated features but still describe the fitted model, not the world.

A security review should pair global and local analysis:

- inspect performance by strong-auth status, device-age bands, region, customer, and time;
- retrieve representative true positives, false positives, and false negatives;
- perturb inputs within valid ranges and check monotonic expectations where policy requires them;
- look for proxies such as geography standing in for customer or workforce population;
- explain the final queue with observed feature values and comparison cases, not a causal story.

The fixture intentionally makes strong authentication protective. If local explanations repeatedly claim strong authentication increases risk, investigate correlation, encoding, and interaction before writing prose around the model.

Model Choice Beyond the Scoreboard

The benchmark does not automatically crown boosting. Logistic regression may be preferred when latency is extremely tight, the feature relationship is stable and close to linear, probability calibration is reliable, or reviewers require a compact reason code. The tree may be useful as an auditable policy prototype but loses here on ranking and stability. Boosting is justified when its measurable queue benefit survives temporal folds, subgroup review, latency tests, and operational complexity.

Nearest neighbors are often costly at inference and sensitive to scaling and irrelevant dimensions. Support-vector machines can work well on moderate sparse datasets but probability estimation and large-scale training require care. Random forests offer robust nonlinear baselines but their probability tails may need calibration. Neural networks are justified by enough data and representation needs—text, images, sequences—not by prestige. Unsupervised anomaly detection answers “unusual relative to its reference,” not “malicious”; validate it through a review queue and adversarially meaningful outcomes.

Active learning can spend analyst labels on uncertain or diverse events, but naive uncertainty sampling may neglect rare high-risk regions. Semi-supervised methods assume something about cluster or manifold structure that attackers may violate. Every learning paradigm imports assumptions; model selection is the work of exposing and testing them.

Robustness and Launch Record

Before launch, replay known telemetry failures: missing device age, new ASN categories, duplicated events, delayed feature updates, extreme distances, and changed strong-auth enrollment. Measure score and queue changes. Confirm feature computation is point-in-time correct and online/offline definitions match. Establish fallbacks when a feature is stale. Rate-limit actions so a malformed upstream field cannot challenge an entire customer.

The model card should record training and calibration intervals, label definition and delay, exclusions, prevalence, feature provenance, hyperparameters, fold results, locked-test metrics, capacity and cost assumptions, subgroup limitations, calibration method, latency, owner, rollback condition, and monitoring plan. Retraining is not the default response to every drift alarm; first determine whether data, population, attacker behavior, or labels changed.

Benchmark Hygiene: Ways to Win Dishonestly

Imbalanced benchmarks are unusually easy to manipulate without an explicit lie. One can tune hundreds of configurations and report only the best test score, resample before splitting so duplicates cross folds, choose the queue size after seeing results, or exclude periods with missing labels. Each choice transfers information from evaluation into model selection.

Freeze the evaluation protocol before viewing the locked test: unit of splitting, time boundaries, label-maturity cutoff, primary and secondary metrics, queue size, cost coefficients, subgroup slices, and tie-breaking rule. Record every attempted configuration or use a reproducible search object. If the test influences a design decision, it is no longer a pristine test; create a later holdout rather than continuing to cite it as untouched.

Class imbalance techniques belong inside training folds. Class weights alter the contribution of examples to the objective. Random under-sampling discards majority information but can reduce compute. Over-sampling repeats minority examples and can overfit. Synthetic methods interpolate assumptions about feature space and may generate impossible security events, particularly with categorical and temporal variables. None of these changes real deployment prevalence, so probability estimates require evaluation and often calibration on naturally distributed data.

Threshold selection is a policy operation. A fixed probability threshold produces variable queue size when prevalence or calibration moves. A fixed top-k policy respects staffing but can force review of weak candidates during quiet periods. A hybrid can review up to k items above a minimum calibrated probability. Backtest the exact policy, including tied scores, customer caps, and rate limits. Report coverage: what share of eligible events could be scored with fresh features?

Cost coefficients should not be magical constants. Estimate review minutes, challenge abandonment, analyst opportunity cost, incident severity distribution, and recovery expense. Conduct sensitivity analysis across plausible miss-to-review ratios. If model choice flips whenever the miss cost moves from 35 to 40, the evidence does not support a robust winner. Present the frontier of discoveries versus burden rather than hiding judgment in one scalar.

Confidence intervals need dependence-aware resampling. Bootstrapping individual logins assumes independence that campaigns and identities violate. Resample days, customers, or campaigns according to the deployment uncertainty. For model comparisons, use paired resamples so both models face the same blocks; examine the distribution of metric differences. A one-point AP gain with an interval spanning substantial loss should not justify a more fragile service.

Interpretation must also be benchmarked. Give analysts blinded explanations from true and false cases and ask whether explanations help identify data defects, understand ranking, or choose a response. Measure consistency and time, not whether a colorful attribution “looks right.” An explanation layer that varies wildly under tiny valid perturbations can erode rather than create trust.

The benchmark itself is a governed artifact: version its fixture, split manifest, metric implementation, and expected output together.

Sources and Further Reading

1. scikit-learn, [Model evaluation](#), [Probability calibration](#), and [TimeSeriesSplit](#).
2. Saito and Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot,” *PLOS ONE*, 2015.
3. Davis and Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” ICML 2006.
4. Niculescu-Mizil and Caruana, “Predicting Good Probabilities with Supervised Learning,” ICML 2005.
5. Brier, “Verification of Forecasts Expressed in Terms of Probability,” *Monthly Weather Review*, 1950.
6. Varma and Simon, “Bias in Error Estimation When Using Cross-Validation for Model Selection,” *BMC Bioinformatics*, 2006.
7. NIST, [AI Risk Management Framework 1.0](#).
8. Molnar, *Interpretable Machine Learning*, online edition.

Interview Practicum

Defend a Malicious-Login Model

Prompt and constraint cards

“Choose a model for malicious-login detection.” After the candidate starts, the panel reveals: only 25 sessions per hour can be reviewed; labels mature after 30 days; one large customer accounts for half the attacks; the authentication team wants automatic challenges at probability 0.8; and legal asks whether country is acting as a workforce proxy.

Worked Answer

Model-selection defense

Frame two decisions separately: ranking a bounded review queue and automatically challenging a login. Establish a temporal, customer-aware split and a locked recent test. Benchmark a rules/current-policy baseline, regularized logistic regression, a shallow tree, and boosted trees. Select for precision and recall at 25, expected operational cost, temporal variability, latency, and subgroup behavior—not global accuracy.

Use nested temporal validation for tuning; group or report the dominant customer separately so it cannot determine the headline result. Calibrate on a later development slice with matured labels, then evaluate reliability on the locked test. An automatic threshold of 0.8 is acceptable only if calibration near that region has sufficient support, the action's error cost is approved, and safe fallbacks exist. Otherwise use the score for ranking or step-up experimentation rather than automatic blocking.

Audit country by comparing models with and without it, checking performance and error rates across affected populations, examining correlated features, and considering a coarser or policy-derived representation. Removing country does not remove geographic proxies. Document the lawful and operational purpose rather than treating a feature-importance plot as a fairness verdict.

Benchmark traps

“Use deep learning because attackers are complex” ignores sample size, latency, calibration, and tabular baselines. “Use AUC” ignores capacity. “Apply SMOTE before cross-validation” can leak synthetic information and distort time. “Set class weights and interpret output as probability” ignores calibration. “Drop country, therefore fair” ignores proxies and error distribution.

Follow-Up Questions and Answers

Adversarial panel follow-ups

The boosting gain is one extra true positive per hour. Is it worth it? Quantify prevented-loss value, review burden, latency, maintenance, and uncertainty across time. If the gain is unstable or operational cost dominates, ship logistic regression.

Labels are selectively observed because analysts review high scores. Preserve randomized exploration or audit samples, model selection propensity where defensible, and avoid treating unlabeled low-score events as benign.

The base rate doubles. Ranking may remain similar while calibration fails. Reassess reliability, capacity, thresholds, and whether the change reflects attacks, customer mix, or label policy.

Spoken model recommendation

Thirty seconds: “I would benchmark interpretable and nonlinear models on temporal, customer-aware splits; choose by discoveries and misses at the actual queue size; calibrate on separate matured labels before any automated action; and ship the simplest model whose advantage is stable and operationally material.”

Key Takeaways

- Model selection begins with the deployed decision, capacity, temporal split, and error costs.
- Precision-recall, top-k discovery, cost, calibration, and fold stability reveal failures hidden by accuracy or ROC AUC.
- Tune inside development data; keep a recent locked test and prevent identity, campaign, or feature-window leakage.
- Calibration needs independent matured labels and does not repair ranking.
- Interpretation describes model behavior and supports investigation; it does not establish causality or fairness.
- Prefer the simplest contender whose real operational advantage is stable, material, and supportable.

Chapter 8: Specialized Data: Time, Text, Graphs, and Streams

Investigation Packet: One Identity, Four Laboratories

Identity `u-17` generated an unusual midnight burst, an analyst note mentions “consent phishing,” a newly registered application reaches a privileged group through delegated permissions, and events arrive out of order from two collectors. Flattening all evidence into one feature table would erase useful structure. Treating every structure with a specialized model would create unjustified complexity.

The four laboratories below ask separate questions:

- time series: did a count process depart from its own seasonal baseline?
- text: which documented incidents resemble the current note, and why?
- graph: what reachable privilege path exists?
- stream: what can be known at event time when records are late?

Only after each laboratory produces a validated artifact do we design fusion.

Laboratory A: A Seasonal Count Is Not an IID Column

We observe hourly privileged-action counts for fourteen days. Normal activity has an hour-of-day pattern; the last night contains a burst. A global z-score would call busy daytime hours anomalous and might miss a smaller but extraordinary nighttime count. Estimate a robust seasonal baseline separately for each hour of day using the median and median absolute deviation (MAD).

```
import numpy as np
rng=np.random.default_rng(8)
hours=np.arange(14*24)
hour_of_day=hours%24
seasonal=np.where((hour_of_day>=8)&(hour_of_day<18),6,1)
counts=rng.poisson(seasonal).astype(float)
counts[-5]=12

score=np.zeros_like(counts)
for h in range(24):
    idx=np.where(hour_of_day==h)[0]
    history=counts[idx[:-1]]
    median=np.median(history)
    mad=np.median(np.abs(history-median))
    scale=max(1.0,1.4826*mad)
    score[idx[-1]]=(counts[idx[-1]]-median)/scale
```

```
top=np.argsort(score)[-3:][:-1]
for i in top: print(int(i),int(hour_of_day[i]),int(counts[i]),round(float(score[i]),2))
```

Exact expected output:

```
331 19 12 12.0
320 8 13 6.07
323 11 12 2.36
```

The score is a screening statistic, not a probability of attack. With only thirteen historical observations per seasonal cell, uncertainty is substantial. A production baseline should define minimum history, holiday handling, daylight-saving behavior, and a fallback for unseen seasonal cells.

Trend, autocorrelation, and regime change

A moving average smooths noise but lags a step change. Exponential smoothing weights recent observations more heavily. Forecast residuals must be checked for autocorrelation; correlated residuals invalidate the intuition that repeated threshold crossings are independent evidence. For count data, Poisson or negative-binomial models may be more appropriate than Gaussian errors, especially when variance grows with mean.

Change-point detection asks whether the generating regime shifted, not whether one point is extreme. A cumulative-sum detector can identify sustained mean movement, but its reference mean and threshold must be chosen on pre-change data. Evaluate detection delay and false alarms per unit time on replayed sequences. Never randomly shuffle time-series observations for validation.

Event histories differ from aggregate series. “MFA succeeded, token issued, consent granted, mailbox read” is an ordered sequence whose gaps matter. Markov, survival, recurrent, or transformer approaches may model it, but simple elapsed-time and n-gram baselines should come first. Sequence windows must end at the decision timestamp to avoid future leakage.

Laboratory B: Retrieve Analyst Notes Without Inventing Meaning

The current note should be compared with a controlled incident corpus. TF-IDF provides a transparent baseline: terms frequent in one document but common across all documents receive less weight. Character or word n-grams can capture phrases; preprocessing must preserve security tokens such as 0Auth , C2 , hashes, and command fragments.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

corpus=[
    "user approved oauth consent phishing application mailbox access",
    "powershell encoded command endpoint malware quarantine",
```

```
"impossible travel login new device token theft",
"oauth application delegated mail read privilege escalation"]
query="analyst suspects consent phishing and delegated mailbox permission"
vectorizer=TfidfVectorizer(ngram_range=(1,2),lowercase=True)
matrix=vectorizer.fit_transform(corpus+[query])
similarity=cosine_similarity(matrix[-1],matrix[:-1]).ravel()
for i in np.argsort(similarity)[::-1]: print(i,round(float(similarity[i]),3))
```

Exact expected output:

```
0 0.199
3 0.052
2 0.0
1 0.0
```

Retrieval says notes 0 and 3 share weighted language with the query. It does not say they have the same root cause. Inspect matched terms, corpus date, author conventions, and whether templated ticket text dominates similarity.

Embeddings, classifiers, topics, and LLM assistance

Embeddings help when semantically similar notes use different vocabulary, but their model version and truncation policy become data lineage. Benchmark them against lexical retrieval using labeled relevance judgments, recall at candidate depth, and subgroup slices such as language or product. Do not evaluate only on notes used to tune prompts.

A text classifier needs a point-in-time label and must exclude resolution text written after investigation. Topic models summarize co-occurrence, not stable threat categories; assess topic coherence and usefulness with analysts rather than assigning authoritative names from top words.

An LLM can extract entities or draft a synopsis, but require a schema, source-span citations, abstention behavior, and a held-out evaluation set. Treat prompt, model identifier, decoding settings, retrieval corpus, and safety policy as versioned dependencies. Measure unsupported-claim rate and omission of critical entities. Never let generated prose overwrite the original note.

Laboratory C: Traverse the Privilege Graph

Represent identities, applications, groups, and resources as typed nodes. Edges carry relations and validity intervals: `CONSENTED_TO`, `HAS_PERMISSION`, `MEMBER_OF`, or `CAN_ADMIN`. Direction matters. A high-degree node is not automatically dangerous; a short valid path to a crown-jewel permission is often more actionable.

```
import networkx as nx
G=nx.DiGraph()
edges=[
    ("user:u17","app:mail-sync","consented"),
```



```

("app:mail-sync","scope:mail.read","has_scope"),
("scope:mail.read","resource:executive-mail","permits"),
("user:u17","group:contractors","member"),
("group:contractors","resource:wiki","reads")]
for source,target,relation in edges: G.add_edge(source,target,relation=relation)
path=nx.shortest_path(G,"user:u17","resource:executive-mail")
print(" -> ".join(path))
print([G[a][b]["relation"] for a,b in zip(path,path[1:])])
print(sorted(nx.descendants(G,"user:u17")))

```

Exact expected output:

```

user:u17 -> app:mail-sync -> scope:mail.read -> resource:executive-mail
['consented', 'has_scope', 'permits']
['app:mail-sync', 'group:contractors', 'resource:executive-mail', 'resource:wiki',
'scope:mail.read']

```

The path is evidence only if edges were valid at the event time. Store `valid_from`, `valid_to`, source system, and observation time. A revoked grant retained as a timeless edge creates phantom exposure. A missing edge can break a real path.

Centrality is not a risk score

Degree counts adjacent edges. Betweenness measures how often a node lies on shortest paths. PageRank-like measures propagate importance under a random-walk assumption. These are structural summaries, not compromise probabilities. In permission graphs, shortest paths may also be semantically invalid if relation types cannot compose. Define allowed path patterns, such as identity → application → scope → resource, and reject nonsensical traversals.

Community detection can reveal dense clusters but is unstable under parameter changes and may rediscover organizational structure. Graph embeddings and graph neural networks can exploit topology, yet temporal leakage is easy: a graph snapshot built after an incident can contain remediation edges or labels. Compare against typed path counts and neighborhood features before adopting a complex model.

Laboratory D: Event Time, Watermarks, and Late Records

Two timestamps are essential. Event time describes when the action occurred; processing time describes when the system received it. A watermark is the system's claim that event time up to a boundary is mostly complete. It is not the latest observed timestamp and not a guarantee that nothing later will arrive.

Consider a ten-minute tumbling window and five minutes of allowed lateness:

```

from datetime import datetime, timezone, timedelta

events=[("e1","10:02","10:03"),("e2","10:08","10:09"),

```

```

        ("e3","10:04","10:17"),("e4","10:16","10:18")]
def minute(text):
    h,m=map(int,text.split(":")); return datetime(2026,8,1,h,m,tzinfo=timezone.utc)

max_event=None; windows={}; emitted=set()
for event_id,event_text,arrival_text in events:
    event_time=minute(event_text); arrival=minute(arrival_text)
    max_event=event_time if max_event is None else max(max_event,event_time)
    watermark=max_event-timedelta(minutes=5)
    start=event_time.replace(minute=(event_time.minute//10)*10,second=0)
    late=start+timedelta(minutes=10)<=watermark
    windows[start]=windows.get(start,0)+1
    print(event_id,"watermark",watermark.strftime("%H:%M"),"late",late,
          "window",start.strftime("%H:%M"),"count",windows[start])

```

Exact output:

```

e1 watermark 09:57 late False window 10:00 count 1
e2 watermark 10:03 late False window 10:00 count 2
e3 watermark 10:03 late False window 10:00 count 3
e4 watermark 10:11 late False window 10:10 count 1

```

This miniature uses `max_event - 5 minutes`; production systems often derive watermarks per partition and take a conservative minimum so one fast partition cannot close another's window. Arrival timestamp is shown for clarity but processing delay alone does not determine lateness; lateness is relative to the current watermark and window boundary.

What happens after a result is emitted? Three choices are legitimate:

- discard later records and measure drop rate;
- update the result and emit a retraction or new version;
- route late records to a correction stream and recompute downstream state.

The consumer determines which is safe. A dashboard may accept updates. An automatic account suspension cannot quietly retract a decision after harm; it needs conservative thresholds and compensating workflow.

State must be bounded. Evict window state only after the lateness policy, deduplicate by stable event ID within a declared horizon, checkpoint state, and make sinks idempotent. Exactly-once delivery claims do not create exactly-once business effects when an external action is nontransactional. Use idempotency keys and an action ledger.

Incremental mean can be updated with $(m_n = m_{n-1} + (x_n - m_{n-1})/n)$; Welford's method adds stable variance. Quantiles and distinct counts need sketches or retained state. Online learning must account for delayed labels: updating a model from immediate analyst actions may encode the current policy rather than eventual truth.

Fusion Design: Join Decisions, Not Raw Modalities

The laboratories yield four artifacts with different clocks and confidence:

1. a seasonal residual for `u-17` at event time;
2. retrieved note IDs with similarity and matched spans;
3. typed, time-valid privilege paths;
4. a windowed event count with watermark, revision, and completeness status.

Early fusion would concatenate everything into one vector. It is tempting but fragile: missing notes become arbitrary zeros, graph snapshots leak future edges, and late stream corrections mutate features after scoring. Prefer late fusion here. Each specialist produces a versioned feature plus provenance and validity interval. A small tabular model or explicit policy combines them only when aligned to the decision timestamp.

For example, queue an investigation when the seasonal residual exceeds 6, a valid path reaches executive mail, and either the note retriever finds a consent-phishing precedent above a validated threshold or the new-application count is elevated. This conjunctive policy is interpretable and testable. A learned meta-model may replace it after enough labeled incidents exist.

The online feature record should carry:

```
{
  "identity_id": "u-17",
  "as_of": "2026-08-01T10:20:00Z",
  "seasonal_residual": 11.0,
  "text_match": {
    "case_id": "case-104",
    "similarity": 0.251,
    "privilege_path": {
      "target": "resource:executive-mail",
      "hops": 3,
      "graph_snapshot": "g-884"
    },
    "stream": {
      "window": "10:10/10:20",
      "count": 4,
      "watermark": "10:15",
      "revision": 1,
      "complete": false
    }
  }
}
```

Do not treat `complete:false` as zero evidence. The policy can wait, degrade to a conservative mode, or score with an explicit missingness indicator. When a late event changes revision 1 to revision 2, store both the original decision context and corrected feature state. Otherwise post-incident review cannot reconstruct what the system knew.

Proving fusion adds value

Compare the fusion design with four ablations and a simple tabular baseline. Use temporal incident-level splits so events from one campaign do not cross folds. Measure precision and recall at review capacity, detection delay, calibration if probabilities are emitted, late-correction frequency, and performance when each modality is missing. Fusion is justified only when improvement is stable and greater than its operational cost.

Text may add no incremental value once the graph path is known; graph freshness may be too poor for real time; a time-series residual may flood on planned maintenance. Ablation is not an academic ritual—it identifies which pipeline can fail without changing the decision.

Fusion also needs a temporal join rule. For every feature, choose the most recent version whose source evidence was observable at or before `as_of`; never join to the latest row merely because it is convenient. Enforce a maximum age per modality and return `stale` or `unknown` when the bound is exceeded. Offline re-

play must reproduce those availability delays, including graph publication lag and note-index refresh, or it will evaluate an oracle unavailable online.

Privacy boundaries can favor fusion of decisions over raw content. The text service can return a case identifier, similarity, matched spans, and policy-approved entities while the fusion store avoids retaining full notes. The graph service can return a path proof rather than its entire tenant graph. Minimize access while preserving enough provenance for review and correction.

Set separate retention periods for raw events, window state, text evidence, graph snapshots, feature revisions, and action logs. Deleting one layer too early can make an incident unreconstructable; retaining everything indefinitely expands privacy and breach impact. Test deletion and replay procedures together so governance rules do not silently break model evaluation.

Sources and Further Reading

1. Apache Beam, *The Beam Model: Watermarks and Late Data*.
2. Akidau et al., *Streaming Systems*, O'Reilly, 2018.
3. Hyndman and Athanasopoulos, *Forecasting: Principles and Practice*.
4. Manning, Raghavan, and Schütze, *Introduction to Information Retrieval*.
5. Jurafsky and Martin, *Speech and Language Processing*.
6. Newman, *Networks*, Oxford University Press, 2018.
7. Hamilton, *Graph Representation Learning*, Morgan & Claypool, 2020.
8. NIST, *AI Risk Management Framework 1.0*.

Interview Practicum

Real-Time Account Compromise

Prompt with changing facts

“Combine event streams, analyst text, and an identity graph to detect account compromise in real time.” The panel then adds that graph updates lag fifteen minutes, notes may contain personal data, five percent of events arrive more than ten minutes late, and automatic account lockout is forbidden.

Worked Answer

Event-time design response

Clarify the decision: produce a ranked investigation queue, not a lockout. Define event time, maximum tolerable detection delay, review capacity, label maturity, and the graph relations that create meaningful access paths.

Build separate, testable processors. The stream processor computes bounded-window behavioral features using per-partition watermarks, deduplication, versioned corrections, and idempotent output. The graph service answers typed, time-valid reachability against a snapshot ID and exposes snapshot age. The text service performs access-controlled retrieval or schema-bound extraction; it stores source spans, redacts unnecessary personal data, and never makes generated text the sole alert basis.

Fuse late using `as_of` timestamps and freshness metadata. If the graph is older than fifteen minutes, mark it stale and avoid interpreting “no path” as safety. Score complete and degraded modes separately. Queue records include feature versions, watermark, graph snapshot, retrieved evidence, and revision. Evaluate through temporal campaign splits, ablations, capacity metrics, detection delay, late-data corrections, and missing-modality stress tests.

Because lockout is forbidden, actions are analyst review, step-up recommendation, or evidence collection. Human review is not a vague box: define the evidence shown, service-level objective, disposition vocabulary, and sampled audit of low-ranked events to reduce selective-label bias.

Multimodal architecture traps

“Put everything in a vector database” confuses storage with temporal semantics. “Use a graph neural network” skips typed-path validity and label volume. “Drop late events” hides measured evidence. “Have an LLM read all notes” ignores privacy, provenance, hallucination, and evaluation. “Exactly once” does not make an external response reversible.

Follow-Up Questions and Answers

Late-data follow-ups

A late record changes a previously published feature. Emit revision 2 with an idempotency key, retain revision 1 for audit, recompute the queue if still actionable, and measure correction frequency and decision impact.

The graph says no path but is stale. Return unknown with snapshot age, not false. Use other modalities for ranking and request a refreshed traversal before asserting containment.

How do you show fusion is necessary? Run modality ablations on temporal campaign-level tests and compare queue discoveries, delay, calibration, and degraded-mode behavior against added latency and failure surface.

Spoken fusion recommendation

Thirty seconds: “I would keep time, text, graph, and stream processors independently testable, align their outputs by event-time validity, and fuse versioned evidence late. Watermarks and graph freshness become model inputs, late corrections create new revisions, and the product is a review queue—not an irreversible automated lockout.”

Key Takeaways

- Preserve the structure that carries meaning: seasonal time, text context, typed reachability, and event-time completeness.
- Specialized scores are claims about a reference process, not interchangeable probabilities of compromise.
- Graph edges need types and validity intervals; text outputs need source evidence and versioned models.
- A watermark states bounded completeness, and late-data policy must specify correction, retraction, or discard behavior.
- Late fusion with timestamps, snapshot IDs, revisions, and missingness is safer than blindly concatenating modalities.
- Fusion earns its complexity only through temporal ablation tests and reliable degraded modes.

Chapter 9: The Adversary Is in the Dataset

At 08:00 Monday, Northstar's dashboard looks excellent: alerts per thousand logins are down 40 percent, triage is faster, and reviewed-alert precision is at a yearly high. On Friday, fraud operations reports that confirmed account takeovers rose 18 percent. Nothing is contradictory. The dashboard measures what a detector selected for attention. Fraud measures a delayed, incomplete subset of harm. Between them sits an adversary who probes controls, a review process that creates labels, and interventions that change what can be observed.

Security data is generated by a game, not sampled passively from a stable population. This chapter follows Northstar's identity team through that failure and builds an analysis that remains interpretable when attacks are rare, labels selective, outcomes delayed, and attackers adaptive.

The Monday Paradox

Let an attempt have features X , latent malicious status Y , score S , action A , review indicator R , and observed outcome O . Production stores X, S, A ; Y becomes known mainly when $R=1$ or a later loss is reported. Thus $P(Y=1 \mid X, R=1) \neq P(Y=1 \mid X)$. Analyst verdicts are not missing at random: review depends on score, rules, queue capacity, customer tier, and many model features. Training on reviewed alerts teaches the old policy's field of view.

Northstar needs two clocks and explicit unresolved states:

```
auth_attempt(attempt_id, identity_id, event_time, available_time,
             score_version, risk_score, action, review_probability,
             verdict_time, loss_time, outcome_state, censor_reason)

outcome_state := pending | benign | malicious | censored
censor_reason := horizon_open | account_closed | telemetry_lost
```

`available_time` prevents leakage: a day-20 chargeback cannot score a day-0 login. Pending is not benign. Censored means observation ended without resolution.

Rare Events Rewrite Familiar Metrics

Suppose one in 10,000 logins is malicious. A detector with 90 percent sensitivity and 99 percent specificity yields about 90 true positives and 9,999 false positives per million logins:

$$\$PPV = \frac{0.90(0.0001)}{0.90(0.0001) + 0.01(0.9999)} \approx 0.0089.$$

Fewer than one alert in a hundred is an attack. An always-benign classifier has 99.99 percent accuracy. Security evaluation instead needs base rate, precision-recall behavior, calibrated risk, capacity, and action costs.

For action a , choose

$$a^*(x) = \arg \min_a \sum_y P(Y=y \mid X=x) L(a, y),$$

where L distinguishes allowed takeover, unnecessary challenge, false block, and analyst time. Policy owners approve the loss matrix. A calibrated probability plus explicit costs travels better than a threshold copied between businesses.

Uncertainty dominates rare events. Three malicious cases in 2,000 audited logins is 0.15 percent, but its interval matters more than the point. Use Wilson or exact intervals for simple samples and design-based estimators for stratified samples.

The Label Factory Has a Selection Gate

Northstar labels cases through high-score review, user recovery after challenge, delayed loss reports, and a small random audit of allowed traffic. The first three paths are behaviorally selected. Only the fourth provides controlled visibility below threshold.

Let $\pi_i = P(R_i = 1 \mid Z_i)$ be the known inclusion probability. A Horvitz-Thompson estimate of malicious total is

$$\widehat{T} = \sum_{i: R_i = 1} \frac{Y_i}{\pi_i}, \quad \widehat{p} = \widehat{T} / N.$$

Audit probabilities must be recorded, every selected case pursued, and nonresponse treated separately. Partition allowed traffic into score bands, new versus established identities, and high-impact versus ordinary assets. Give every consequential stratum nonzero coverage; allocate remaining budget toward numerous, uncertain, inexpensive strata. Randomize extra investigation among cases already allowed by policy—no protection versus deliberate harm. Severe cases require shadow decisions, red-team fixtures, natural experiments, or bounds.

Delays, Censoring, and Hidden Counterfactuals

Only mature cohorts belong in a naive binary metric. If 95 percent of fraud reports arrive within 30 days, a seven-day cohort is unresolved, not unusually safe. Comparing it with mature history manufactures improvement.

Let T be time to confirmed compromise and C censoring time. Observe $\tilde{T} = \min(T, C)$ and $\delta = I(T \leq C)$. Kaplan-Meier estimates a time-to-event curve when censoring is conditionally independent. That assumption fails if risky accounts are watched longer; stratify, model censoring, or state bounds.

Intervention hides outcomes too. If a challenge stops an attacker, counterfactual loss is unknowable. “No incident after challenge” does not show every challenge was needed. Estimate effects from defensible comparisons and label prevented harm as an estimate with assumptions, never an observed count.

Defender and Attacker Learn at Different Speeds

Attackers infer thresholds by varying IP reputation, velocity, device age, and transaction amount. Tomorrow's distribution depends on today's policy:

$$P_{t+1}(X,Y) = \mathcal{G}(P_t, \pi_t, q_t)$$

with defender policy π_t and attacker strategy q_t . Deployment changes the prediction population. Track tactic-conditioned performance, champion-shadow disagreement, score-boundary density, and audit yield below threshold. Aggregate stability can hide substitution from password spraying to session-cookie theft.

The answer is controlled exploration, rotating high-cost-to-mimic evidence, protected challenge sets, and versioned policy changes—not reflexive retraining. Canary cohorts help distinguish model change, sensor change, and adversary change.

Alerts Are Work, Risk Is Consequence

An alert reduction may mean better precision, missing telemetry, a raised threshold, attacker migration, deduplication, or detector failure. Northstar keeps four ledgers:

Ledger	Denominator	Measure
exposure	privileged identity-days	reachable value
adversary	audited attempts	hostile prevalence
intervention	eligible decisions	challenge coverage
consequence	identity-days	mature loss

Leadership gets a linked explanation, not one blended score. Exposure, prevalence, intervention, and mature consequence can move in opposite directions.

```
sequenceDiagram
    participant A as Attacker
    participant D as Detector
    participant U as User/Asset
    participant Q as Audit sampler
    participant L as Outcome ledger
    A->>D: adaptive attempt
    D->>U: allow/challenge/block
    D-->>A: observable friction
    Q->>U: probability-sampled review
    U-->>L: delayed report
    Q-->>L: weighted verdict
    L-->>D: mature evidence
```

The attacker receives fast feedback; the defender receives delayed, policy-mediated feedback.

Lab: Make a Good Dashboard Lie

This fixture simulates 200,000 logins. Attack prevalence rises, attackers suppress a visible feature, and the threshold rises to reduce queue volume. All alerts and one percent of non-alerts are reviewed. Save as `ch09_adversarial_audit.py`; run with the book `.venv`.

```

import numpy as np

rng = np.random.default_rng(19)
n = 200_000
era = np.repeat([0, 1], n // 2)
y = rng.binomial(1, np.where(era == 0, 0.0010, 0.0018))
velocity = rng.normal(0, 1, n) + y * np.where(era == 0, 3.2, 1.5)
device = rng.normal(0, 1, n) + y * 2.0
score = 1 / (1 + np.exp(-(-5.0 + 1.4 * velocity + device)))
alert = score >= np.where(era == 0, 0.18, 0.30)
audit = (~alert) & (rng.random(n) < 0.01)
reviewed = alert | audit
prob = np.where(alert, 1.0, 0.01)

for e, name in [(0, "before"), (1, "after")]:
    m = era == e
    weighted = np.sum(y[m & reviewed] / prob[m & reviewed]) / np.sum(m)
    print(f"{name}: alerts={alert[m].mean():.4%} incidents={y[m].mean():.4%} "
          f"precision={y[m & alert].mean():.2%} "
          f"reviewed_rate={y[m & reviewed].mean():.2%} "
          f"audit_weighted={weighted:.4%}")

```

Fixture: seed 19, 100,000 attempts per era, base rates 0.10/0.18 percent, audit probability 0.01, thresholds 0.18/0.30. Exact expected output:

```

before: alerts=2.2550% incidents=0.0960% precision=4.26% reviewed_rate=2.98%
audit_weighted=0.0960%
after: alerts=0.8630% incidents=0.2010% precision=10.20% reviewed_rate=4.83%
audit_weighted=0.1880%

```

Alerts fall 56 percent while incidents nearly double. Precision improves—excellent dashboard news. The weighted audit recovers the direction and approximate magnitude of harm. Sampling variance explains the difference from simulated truth. Production extensions should add score strata, label times, nonresponse, and confidence intervals. Inclusion probability is the indispensable field.

Reading the Failure as a System

The simulation reveals three distinct estimands. Alert precision asks what fraction of selected alerts are malicious. Reviewed-case rate mixes certain inclusion above threshold with rare inclusion below it and has no stable population meaning. Audit-weighted prevalence asks what fraction of all attempts are malicious, under the audit design. None alone estimates prevented loss.

A production review should reconstruct policy versions point in time. Re-score the same eligible attempts under old and new thresholds, then compare disagreement sets. Slice by attack mechanism rather than only user demographics: credential stuffing, adversary-in-the-middle, token theft, recovery abuse. Examine sensor reach and feature missingness within each slice. If one policy challenges and another allows a case, deliberately oversample that disagreement for audit while retaining a probability sample of the complete population.

Outcome delay needs a cohort ledger. For every weekly cohort publish attempts, known malicious, known benign, unresolved, censored, and estimated completion fraction. Do not backfill a historical dashboard silently; revisions are evidence. A mature cohort may still be biased if users who suffer small losses never report them, so combine customer reports, analyst investigation, recovery signals, and sampled inquiry without pretending they are interchangeable.

Capacity is part of policy. A threshold selected on an unconstrained curve may flood analysts, lengthen decision time, and lower effective recall. Evaluate at the queue's service rate. If arrivals exceed service, calculate the induced waiting-time distribution and the fraction of decisions made after the attacker completes the action. The model's offline recall is irrelevant to alerts nobody reaches.

Finally, distinguish calibration drift from ranking drift. A ranking may still order cases usefully while probabilities become wrong as prevalence changes. Recalibration can repair the former; tactic-specific ranking collapse needs new evidence or model structure. The random audit supports both diagnoses because it observes below-threshold outcomes.

Designing the Audit as a Durable Control

An audit stream fails if treated as a temporary research sample. It needs a budget owner, sampling service, case-handling service level, and exclusion review. The sampler commits the unit and inclusion probability before anyone sees the verdict; otherwise investigators can replace inconvenient selections and invalidate the design. Store frame size, stratum, draw identifier, selection time, contact attempts, disposition, and adjudication version.

A prevalence audit samples the eligible population. A threshold audit concentrates near a proposed boundary. An adaptation audit oversamples new tactics while preserving a probability component. Mixing objectives without separate weights creates an opaque convenience sample. Northstar runs named streams and prevents duplicate investigation from being counted twice.

Adjudication should be blind to production action when feasible. Knowing that a detector blocked a login can bias the verdict. Use a fixed evidence packet, dual review for ambiguous cases, and a reason taxonomy. Measure disagreement and resolution delay. A label is a versioned conclusion supported by evidence, not an eternal property revealed by one click.

Random audit creates operational risk: customer contact can disclose monitoring and investigations consume specialists. Use minimization, tiered collection, and stopping rules. At prevalence p and sample size n , the chance of no positives is $(1-p)^n$. With $p=0.001$ and $n=1,000$, zero occurs about 36.8 percent of the time. A zero-yield week is unsurprising, not proof of safety.

The audit dashboard should emphasize design integrity: frame coverage, realized inclusion by stratum, unresolved rate, time to verdict, selection deviations, and weighted effective sample size. If these fail, suppress population estimates. "Not estimable this week" is better than a precise biased number. Publish the recovery condition as well, so operators know exactly which evidence defect must clear before estimates resume.

Analysts should also precompute escalation rules for surprising audit discoveries. One confirmed severe compromise in a supposedly protected stratum may justify immediate containment even though it cannot

establish prevalence. Operational response and statistical inference have different thresholds: responders may act on credible individual evidence while analysts refuse to generalize from one case. The decision record should say which threshold fired.

Audit results must remain comparable across policy versions. Freeze stratum definitions for a reporting window or publish a bridge that reweights both versions to a common frame. If product growth changes the identity mix, standardize to a declared reference population and also show the unstandardized operational rate. This separates composition change from within-stratum risk.

Finally, protect the sampler from the detector team. If the same owners can modify thresholds, sampling probabilities, exclusions, and adjudication, favorable results become too easy to manufacture accidentally. Independent review of draw logs and exclusions is inexpensive assurance. The purpose is not distrust of individuals; it is maintaining an evidence channel that production optimization cannot silently erase.

Sources and Further Reading

- NIST, *AI Risk Management Framework 1.0*: <<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>>
- NIST, *Engineering Statistics Handbook*: <<https://www.itl.nist.gov/div898/handbook/>>
- The causal text *What If*, by Miguel Hernán and James Robins, covers censoring and selection: <<https://www.hsph.harvard.edu/miguel-hernan/causal-inference-book/>>
- Perdomo et al., “Performative Prediction”: <<https://proceedings.mlr.press/v119/perdomo20a.html>>
- Sculley et al., “Hidden Technical Debt in Machine Learning Systems”: <<https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>>
- MITRE ATT&CK, Enterprise techniques: <<https://attack.mitre.org/techniques/enterprise/>>
- Verizon, *Data Breach Investigations Report*: <<https://www.verizon.com/business/resources/reports/dbir/>>

Interview Practicum

You inherit Northstar's detector. Alerts fell 40 percent, confirmed incidents rose, and the VP asks whether the model failed. The interviewer releases facts only after useful questions: analysts review every alert but almost no allowed login; losses mature over 30 days; the newest cohort is seven days old; SOC raised the threshold six weeks ago; cookie theft grew after MFA rollout; a one-percent retrospective audit is permitted, but dangerous traffic may not be intentionally allowed.

A strong candidate refuses the binary diagnosis. They draw the observation process and separate event, decision, verdict, and loss times. They name threshold change, maturity, telemetry, attack mix, exposure, and queue behavior as competing mechanisms. They reconstruct mature cohorts under both policies and launch stratified audits below both thresholds. Primary outcomes are compromise or loss per identity-time; alerts and analyst minutes are constraints.

Halfway through, audit budget is cut in half. Preserve nonzero coverage in consequential strata, oversample policy disagreements and high-impact identities, record probabilities, and widen intervals. Then the interview-

er reveals challenged attacks have no observable counterfactual loss. Stop calling them prevented incidents; use defensible comparisons and bounds.

The decision-quality close is: “I will neither roll back because incidents rose nor defend the model because precision improved. Freeze further threshold changes, restore point-in-time evaluation, start protected auditing, and decide after a mature cohort and audit interval exist.”

Worked Answer

In 48 hours, verify sensor counts, schema and deduplication changes, threshold history, queue limits, and incident definition. Rebuild cohorts by event time, attaching only evidence available at each historical decision. Show unresolved fractions and exclude immature cases from binary comparison.

Replay both thresholds on identical attempts. Report disagreements by tactic, identity value, device state, and coverage. Estimate population prevalence using the documented audit design. Keep analyst verdict and customer loss as distinct evidence. Compare expected loss at real review capacity, including challenge friction, false blocks, analyst cost, and mature harm. Require acceptable critical-stratum bounds and monitor below-threshold audit yield as an adaptation signal.

Follow-Up Questions and Answers

Audited cases remain unresolved—now what? Treat response as a second selection process, report it by stratum, fix operational causes, and provide sensitivity bounds. Unresolved never becomes benign by timeout.

Can active learning replace random audit? No. It selects informative boundary cases and cannot estimate population prevalence without a known design. Run both streams for different purposes.

How can adaptation be seen before labels mature? Monitor tactic mix, boundary density, missingness, challenge abandonment, canary traffic, and shadow disagreement. They are warnings, not outcome proof.

When should a feature be retired? When cheap manipulation, brittle group effects, or tactic-conditioned replay show lost incremental value. Retire through a shadowed policy change.

Key Takeaways

- Production labels describe a policy-selected sample, not the threat population.
- Rare-event arithmetic makes accuracy and raw alert counts misleading.
- Preserve event time, availability time, maturity, censoring, and sampling probability.
- A protected probability audit reveals risk below the threshold; active learning does not replace it.
- Attackers receive faster feedback than defenders, demanding tactic- and cohort-aware evaluation.
- Alert volume measures work; mature consequence and uncertainty measure security.

Chapter 10: Cyber Risk Is a Property of a Connected System

The request sounds reasonable: “Give leadership one enterprise cyber-risk score.” It is also underspecified. Risk to which assets, from which attack paths, during what horizon, under which controls, and measured with what evidence? A scalar can summarize a model after those questions are answered. It cannot answer them.

This chapter constructs a measurable system for Meridian, a software company whose customer-data service can be reached through workforce identities, CI/CD, cloud administration, and an AI support agent. The organizing artifact is a typed graph. Assets carry consequence; identities exercise authority; trust boundaries constrain movement; attack steps express adversary capability; controls interrupt mechanisms; telemetry makes states observable. Risk is computed over paths, then preserved as drill-down evidence rather than crushed into an unexplained color.

Start with Consequence-Bearing Assets

An inventory becomes a risk model only when nodes have operational meaning. Meridian uses five node types:

```
Asset:
  keys: [asset_id, environment]
  fields: [mission, data_class, outage_cost_per_hour, integrity_impact, owner]
Identity:
  keys: [principal_id, issuer]
  fields: [kind, privilege, auth_strength, lifecycle_state, owner]
Boundary:
  keys: [boundary_id]
  fields: [source_zone, destination_zone, allowed_protocols, policy_version]
AttackStep:
  keys: [step_id]
  fields: [technique_id, preconditions, success_probability, evidence_ids]
Control:
  keys: [control_id, version]
  fields: [mechanism, coverage, conditional_effect, failure_mode, evidence_grade]
```

Edges are typed too: `AUTHENTICATES_TO`, `CAN_ASSUME`, `DEPLOYS_TO`, `READS`, `CROSSES`, `ENABLES`, `BLOCKS`, and `OBSERVED_BY`. A shared administrator account is not merely an identity row; its `CAN_ASSUME` edges expose how one compromise fans out. A trust boundary is not a rectangle on a diagram; it is a policy-enforced transition with a version and observable decisions.

Consequence belongs primarily to assets. Threat is an actor's intent and capability. Vulnerability is a weakness. Exposure is reachability under current configuration. Likelihood is a model over attack attempts and control behavior. Impact is the conditional consequence of a successful path. Mixing these nouns into one “severity” field destroys causal meaning.

Events, Alerts, Incidents, Losses, and Near Misses

These records occupy different levels:

- an **event** is a state change or observation;
- an **alert** is a detector's assertion requiring action;
- an **incident** is an adjudicated harmful security episode;
- a **loss** is realized consequence such as outage, disclosure, recovery cost, or trust damage;
- a **near miss** is a path that progressed but was interrupted before consequence.

Ten alerts may describe one incident; one incident may produce several losses; a control can create near misses by stopping attacks. Counts cannot be freely divided. Meridian gives each table explicit keys and relations:

```
alert.event_ids[], incident.alert_ids[], incident.asset_ids[], loss.incident_id, and
near_miss.interrupting_control_id.
```

A Path Model with Causal Semantics

For path $p=(s_1,\ldots,s_k)$ to asset a , a first approximation is

$$R_p=P(A_p)\left[\prod_{j=1}^k P(s_j\mid pa(s_j),do(C=c))\right]I_a$$

$P(A_p)$ is attempted-path frequency, $pa(s_j)$ are prerequisites, C represents controls, and I_a is consequence. The **do** notation is a warning: observed attacks under a control do not automatically identify what would happen without it. Control effectiveness needs experiments, quasi-experiments, engineering tests, or explicit assumptions.

Paths are not independent. The same stolen token may enable several branches, while a common identity provider failure disables multiple controls. A Bayesian network, fault tree, or Monte Carlo structural model can retain shared causes. Summing path risks is acceptable only when overlap is negligible or corrected.

For a control c , Meridian records conditional mechanism effectiveness rather than a universal percentage:

$$e_c(z)=1-P(S=1\mid do(c=1),Z=z)/P(S=1\mid do(c=0),Z=z)$$

MFA may strongly reduce password-only compromise yet do little against stolen sessions. Coverage and effectiveness are separate: a 95-percent-effective control deployed to 40 percent of privileged identities does not provide 95-percent enterprise protection.

ATT&CK and ATLAS Are Semantic Coordinates

MITRE ATT&CK maps enterprise adversary behavior. ATLAS maps threats to AI-enabled systems. They are not probability tables. Meridian attaches technique identifiers to attack-step nodes and data components or telemetry sources to **OBSERVED_BY** edges.

Examples:

System step	Mapping	Observable evidence	Candidate interruption
steal developer session	ATT&CK T1539	token reuse and browser binding	phishing-resistant reauth
change build workflow	ATT&CK T1648	repository audit diff	protected branch approval
poison support-agent context	ATLAS AML.T0051	retrieval document lineage	signed corpus admission
induce unsafe tool call	ATLAS AML.T0051 plus local scenario	prompt/tool trace	scoped capability grant

The final row deliberately distinguishes a framework coordinate from Meridian's local mechanism. A mapping supports shared vocabulary and coverage analysis; it does not prove a detection or control works.

The Meridian Attack-and-Control Graph

```

flowchart LR
    X((External actor)) --> P[Phished developer]
    P --> T[Stolen session]
    T --> G[Source control boundary]
    G --> W[Workflow modification]
    W --> B[Build identity]
    B --> R[Cloud deploy boundary]
    R --> D[(Customer-data service)]
    K[Phishing-resistant reauth] -. interrupts .-> T
    V[Signed workflow policy] -. interrupts .-> W
    H[Workload identity attestation] -. interrupts .-> B
    M[Runtime egress policy] -. limits impact .-> D
    T --> E1[[Identity telemetry]]
    W --> E2[[Repository audit]]
    B --> E3[[Cloud audit]]

```

Diamonds are trust transitions, rectangles are adversary states or identities, the oval is the actor, the cylinder-like node is the consequence-bearing asset, dotted edges are controls, and double brackets are evidence. This topology differs from a pipeline: several controls act on distinct mechanisms, and evidence nodes observe rather than cause the path.

The graph exposes dependencies. Reauthentication and repository approval both depend on the identity provider; treating them as independent overstates defense in depth. Build attestation and cloud audit may share the same logging plane; an outage removes both enforcement evidence and measurement. Such common causes become explicit parent nodes in the computational graph.

From NIST CSF 2.0 to Measures

CSF functions organize outcomes; they do not prescribe metrics. Meridian maps them to graph operations:

- **Govern:** ownership, risk tolerance, evidence acceptance, exception expiry.
- **Identify:** asset consequence and identity-edge completeness.
- **Protect:** control coverage at attack transitions.
- **Detect:** probability and delay of observing step progression.
- **Respond:** probability of interrupting a live path before consequence.
- **Recover:** time and residual loss after asset impact.

Each measure has a denominator. “MFA coverage” becomes privileged identity-days protected by phishing-resistant authentication divided by eligible privileged identity-days. “Detection coverage” becomes tested attack-step instances observed within a specified delay divided by valid test instances, sliced by technique and telemetry state.

Leading indicators describe precursors such as exposed privileged edges. Operational indicators describe control execution, latency, and failure. Lagging indicators describe incidents and losses. Outcome indicators estimate consequence or avoided consequence. A balanced view retains all four; a composite must never erase the components.

Why One Score Usually Fails

Suppose leadership insists on $R = \sum_a w_a E[L_a]$. The formula is defensible only if asset loss distributions, weights, horizon, dependence, and uncertainty are visible. Meridian presents an expected-loss range plus top contributing paths and sensitivity. It refuses ordinal multiplication such as `likelihood 4 × impact 5 = risk 20`: category numbers lack ratio-scale meaning.

Monte Carlo simulation is useful when parameters are distributions. Draw attack frequency, step success, control availability, and impact; propagate them through the graph; report median, tail quantiles, and rank stability. Sensitivity analysis answers which uncertain parameter could reverse a funding decision. That is more actionable than adding decimal places to a score.

Graph Lab: Remove a Control, Recompute Reachability

The lab uses only Python's standard library. It enumerates simple attacker-to-asset paths, applies control reductions at specific edges, and reports expected loss. Shared controls are represented once per edge; this small fixture assumes path independence and says so.

Save as `ch10_graph_lab.py` and execute with the book `.venv`.

```
from collections import defaultdict
```

```

edges = {
    ("actor", "dev_session"): (0.08, "reauth"),
    ("dev_session", "repo_write"): (0.30, "branch_policy"),
    ("repo_write", "build_id"): (0.40, "attestation"),
    ("build_id", "customer_data"): (0.25, "deploy_gate"),
    ("actor", "support_agent"): (0.05, "corpus_signing"),
    ("support_agent", "customer_data"): (0.10, "tool_scope"),
}
effect = {"reauth": .75, "branch_policy": .80, "attestation": .60,
         "deploy_gate": .70, "corpus_signing": .50, "tool_scope": .65}
impact = {"customer_data": 2_000_000}

adj = defaultdict(list)
for (u, v), (p, c) in edges.items():
    adj[u].append((v, p, c))

def paths(node, target, seen=()):
    if node == target:
        yield []
    for v, p, c in adj[node]:
        if v not in seen:
            for tail in paths(v, target, seen + (node,)):
                yield [(node, v, p, c)] + tail

def risk(disabled=frozenset()):
    rows = []
    for path in paths("actor", "customer_data"):
        probability = 1.0
        for _, _, p, control in path:
            reduction = 0 if control in disabled else effect[control]
            probability *= p * (1 - reduction)
        names = "->".join([path[0][0]] + [e[1] for e in path])
        rows.append((names, probability, probability * impact["customer_data"]))
    return rows

for label, disabled in [("baseline", frozenset()),
                       ("no_branch_policy", frozenset({"branch_policy"}))]:
    rows = risk(disabled)
    print(label)
    for name, probability, loss in rows:
        print(f" {name}: p={probability:.8f} expected_loss=${loss:,.2f}")
    print(f" total=${sum(r[2] for r in rows):,.2f}")

```

Fixture edge probabilities and control reductions are exactly those in the dictionaries. Exact expected output:

```

baseline
actor->dev_session->repo_write->build_id->customer_data: p=0.00001440 expected_loss=$28.80
actor->support_agent->customer_data: p=0.00087500 expected_loss=$1,750.00
total=$1,778.80
no_branch_policy
actor->dev_session->repo_write->build_id->customer_data: p=0.00007200 expected_loss=$144.00

```

```
actor->support_agent->customer_data: p=0.00087500 expected_loss=$1,750.00
total=$1,894.00
```

The AI-support path dominates despite lower raw attempt probability because its remaining controls are weaker. Removing branch policy multiplies the software-supply path by five, but the enterprise total rises only 12.7 percent. This is exactly why totals need path drill-down: a critical control can look small inside aggregate loss.

For production, replace point probabilities with distributions, represent common-cause nodes, attach evidence provenance, prevent cycles or use a dynamic model, and validate topology with service owners and red teams. “No path” can mean missing inventory, not safety.

Operating the Model

Graph updates are governed changes. Asset owners approve consequence models. Identity teams own authority edges. Control owners attest versions and coverage. Detection teams own observation mappings. Risk analysts own assumptions and uncertainty, but cannot invent missing system relationships.

Every quarterly review asks: which high-consequence nodes lack owners; which trust crossings have no enforced decision; which attack steps lack telemetry; which controls share failure domains; which parameter drives decision sensitivity; and which topology changes occurred since the last model? A signed snapshot makes historical comparisons possible.

Near misses validate portions of topology. If an attack reached `repo_write` and branch policy stopped it, the path prefix is observed and the control executed. It does not identify the probability of eventual data loss. Purple-team tests validate reachability and observability under controlled conditions but must not be treated as unbiased attacker frequency.

Calibrating and Challenging the Graph

Every parameter carries a provenance class: incident frequency, controlled test, production control test, external study, or expert elicitation. These answer different questions. A purple-team success estimates capability in a designed scenario, not annual frequency. Incident counts reflect deployed controls and observation. External rates require transport assumptions. Expert ranges should be elicited independently before discussion to reduce anchoring.

Calibration compares predicted frequencies with mature observations at graph cuts where evidence is credible. If the model predicts ten trust-boundary crossings but telemetry sees one, probability error is only one explanation; missing observation or wrong topology may dominate. Challenge sensor sensitivity before tuning probabilities downward.

Graph assurance uses counterexample hunts. Ask owners to name absent access paths; ask red teams to reach an asset without the modeled boundary; ask identity engineers for dormant principals retaining edges; ask

recovery teams which dependencies appear only during failover. Version every discovered edge and recompute prior decisions. A stable total with radically changed path attribution still demands review.

Tail consequence needs separate reporting. Expected loss can favor a policy that lowers routine loss while increasing catastrophic exposure. Report exceedance probability $P(L > 10 \times \text{million})$, conditional tail expectation, and the worst credible path. Risk appetite may constrain the tail even when the mean improves.

Time also belongs in the graph. A path requiring four steps may be harmless if defenders reliably interrupt it within minutes, yet dangerous during an identity-provider outage. Dynamic edges therefore carry dwell-time and detection-delay distributions. Simulation compares time to adversary completion with time to observation, decision, and containment. This converts vague “response speed” into a race conditioned on technique and operating state.

Model scope must be printed on every output: included assets, excluded environments, snapshot time, horizon, currency basis, and evidence cutoff. Without that footer, a locally valid graph will be mistaken for an enterprise truth.

Sources and Further Reading

- NIST, *Cybersecurity Framework 2.0*: <<https://doi.org/10.6028/NIST.CSWP.29>>
- NIST SP 800-30 Rev. 1, *Guide for Conducting Risk Assessments*: <<https://doi.org/10.6028/NIST.SP.800-30r1>>
- NIST SP 800-53 Rev. 5, control catalog: <<https://doi.org/10.6028/NIST.SP.800-53r5>>
- MITRE ATT&CK, Enterprise matrix: <<https://attack.mitre.org/matrices/enterprise/>>
- MITRE ATLAS, adversarial threats to AI systems: <<https://atlas.mitre.org/>>
- FIRST, *Common Vulnerability Scoring System v4.0*: <<https://www.first.org/cvss/v4.0/specification-document>>
- Open Group, *Open FAIR Risk Analysis Standard*: <<https://publications.opengroup.org/c20a>>

Interview Practicum

The prompt is a negotiation: “The board wants one cyber-risk score next month. Design it.” The interviewer plays three stakeholders. The CFO supplies outage cost but distrusts probabilities. The CISO wants ATT&CK coverage. The platform VP says the asset inventory is stale.

The candidate gets credit for drawing a typed graph before proposing arithmetic. They define horizon and consequence, identify crown-jewel assets, authority paths, trust boundaries, attack transitions, controls, and observations. They offer an expected-loss distribution with drill-down, not a magic scalar.

At minute ten, the interviewer reveals two controls depend on the same identity provider. The candidate adds a common-cause node and retracts any independence assumption. At minute twenty, the CFO asks to multiply red/amber/green ratings. The candidate explains why ordinal categories cannot support multiplication and

proposes ranges with sensitivity. Finally, the platform VP admits inventory coverage is 70 percent. The candidate reports model coverage as a first-class uncertainty and refuses to label absent paths safe.

Worked Answer

Deliver a bounded first model for two material services, not an invented enterprise total. Define asset-specific loss distributions over twelve months. Build identity and deployment reachability from authoritative configuration, then validate trust crossings with owners. Map plausible attack steps to ATT&CK or ATLAS and local scenario identifiers. Attach controls to mechanisms and telemetry to observations.

Compute path-conditioned loss with explicit dependence and parameter grades. Present median and tail risk, top paths, control-removal deltas, missing-topology coverage, and sensitivity. Use CSF functions as governance and outcome groupings. The board sees a concise range and movement; engineers retain graph-level evidence.

Follow-Up Questions and Answers

How do you combine confidentiality and availability impact? Keep separate consequence dimensions until an accountable owner supplies a common monetary or utility model. Do not average incompatible ordinal ratings.

Does ATT&CK coverage equal security? No. It measures mapped behavioral scope. It says nothing by itself about attacker frequency, sensor quality, control effect, or consequence.

How should unknown probabilities be handled? Use ranges or distributions tied to evidence grades, then run sensitivity. If a decision reverses across plausible values, fund measurement before claiming a ranking.

What invalidates the graph? Unowned high-impact assets, stale authority edges, hidden shared dependencies, topology changes without versioning, or outputs that cannot be traced to evidence.

Key Takeaways

- Risk emerges from connected assets, authority, trust transitions, attack mechanisms, controls, and consequences.
- ATT&CK and ATLAS supply behavioral coordinates, not probabilities or proof of coverage.
- Control coverage, conditional effect, availability, and common-cause failure are separate quantities.
- Events, alerts, incidents, near misses, and losses require distinct keys and denominators.
- A scalar is acceptable only as a view over a versioned, inspectable graph with uncertainty.
- The most valuable output is often the path or assumption that can change a decision.

Chapter 11: Make the Claim Before You Touch the Data

“Prove the new agent approval flow makes us safer.” This sentence hides at least six decisions: what “safer” means, which agent actions count, whose experience matters, what alternative policy is being compared, how long outcomes take, and how much evidence is sufficient to change rollout. If analysts open a notebook before resolving those questions, the available data will quietly choose the question for them.

This chapter is a claim-construction workshop. The running case is an engineering agent that proposes shell commands and production changes. The current policy auto-approves low-risk actions and sends others to a human. A new policy requires approval for any command touching deployment infrastructure. We will translate the executive request into a population, unit, intervention, comparison, outcomes, horizon, estimand, evidence grade, and decision rule—then execute an analysis on fixed fixtures.

The Claim Card

A formal claim should fit on one page before implementation begins:

```
claim_id: AGENT-APPROVAL-07
decision: expand_new_gate | retain_current | revise_and_retest
population: eligible production-agent action requests by staffed teams
unit: action request, clustered within task and team
policy_1: infrastructure-sensitive approval gate v2
policy_0: current gate v1
primary_outcome: harmful execution within 24 hours
guardrails: [task_abandonment, approval_latency, reviewer_minutes]
horizon: 28-day enrollment, 24-hour outcome maturation
estimand: intention-to-treat risk difference
threshold: expand only if upper_95_harm_delta <= -0.002
           and p95_latency_delta <= 45 seconds
exclusions: break-glass incidents, synthetic red-team tasks
```

The card prevents semantic drift. It does not guarantee identification. It makes disagreements inspectable before results are known.

Question, Claim, Estimand, Decision

A security question is broad: does approval help? A claim is falsifiable: among eligible production action requests, assignment to gate v2 reduces harmful execution within 24 hours by at least 0.2 percentage points without adding more than 45 seconds to p95 decision latency. An estimand is the precise population quantity:

$$\tau_{\text{ITT}} = E[Y_i(1) - Y_i(0)],$$

where $Y_i(z)$ is the potential harmful-execution outcome under assignment z . The decision rule maps estimates and uncertainty to action.

Intention-to-treat is appropriate because assignment is controlled and bypasses occur in reality. A per-protocol estimand—effect among requests that actually received the assigned behavior—answers a different question and can be biased by noncompliance. Both may be reported, but only one is primary.

When random assignment is unavailable, specify the comparison mechanism: phased rollout, regression discontinuity around a policy boundary, difference-in-differences, matched controls, or an explicitly descriptive comparison. Never let “A/B-like” conceal a nonrandom launch.

Choose a Unit That Can Receive Policy

Possible units include event, action request, task, session, identity, host, incident, customer, and team-week. The unit must align with assignment and outcome. One agent task may emit twenty commands. Randomizing commands within the same task can contaminate treatment because an earlier rejection changes later proposals. Meridian therefore randomizes by task and analyzes action outcomes with cluster-aware uncertainty.

The population is not “all logs.” It is the set of units to which the decision will apply. State eligibility in executable terms:

```
eligible = environment = 'production'
        AND agent_mode = 'write_enabled'
        AND team_staffed_for_review = TRUE
        AND break_glass = FALSE
```

This excludes unstaffed teams, so external validity is limited. The result cannot justify rollout to them without further evidence. Record eligible-but-not-enrolled units to diagnose selection.

Outcomes Need Adjudication, Not Convenient Proxies

The primary harm outcome is a request that executes and causes unauthorized state change, material policy violation, secret exposure, or service impairment within 24 hours, adjudicated blind to assignment where feasible. “Command rejected” is a control action, not harm. “Security alert” is detector output, not ground truth. “No incident ticket” may reflect reporting failure.

Guardrails protect other objectives. Approval latency should be measured from policy decision request to executable decision, using event time and monotonic duration. Reviewer burden is minutes per eligible task, not minutes per reviewed request alone. Task abandonment needs a definition distinguishing user cancellation from policy-induced failure.

Outcomes form a hierarchy:

1. harmful execution (primary);
2. near miss stopped by policy;
3. unsafe proposal without execution;
4. review disagreement;
5. latency, burden, and abandonment.

Moving a result between levels after seeing data is not interpretation; it is outcome switching.

Evidence Grades Describe Support, Not Importance

Meridian grades each input and inference:

Grade	Meaning	Example
A	randomized or directly verified with complete provenance	task assignment; signed execution trace
B	strong observational evidence with tested assumptions	phased rollout with stable pretrends
C	indirect or selectively observed evidence	analyst verdict on alerted actions
D	expert judgment or unvalidated proxy	heuristic “unsafe” tag

A high-impact claim supported by grade D remains high impact and weakly known. Grades never average into a faux confidence score. The claim card lists which conclusion depends on which grade and what evidence would upgrade it.

Evidence can be contradictory. A signed trace may prove a command ran, while incident review disputes whether the change was harmful. Preserve both provenance chains and adjudication. Unknown is a valid state.

Metric Specifications Are Small Programs

A metric specification includes name, purpose, unit, numerator, denominator, event-time rule, availability rule, exclusions, grouping, missingness behavior, revision policy, owner, and tests. For example:

```
metric: harmful_execution_rate_24h
unit: eligible action request assigned to a policy
numerator: adjudicated harmful execution <= request_time + 24h
denominator: all assigned eligible requests with mature 24h horizon
missing: unresolved outcome remains pending; never coerce to zero
dedupe: request_id, latest valid adjudication version
revision: weekly cohorts may revise for 35 days; retain prior snapshots
```

The denominator includes assigned requests, not only executed or reviewed requests. That preserves the ITT interpretation. A separate executed-request rate can diagnose mechanism but is post-treatment and must not replace the primary metric.

Assumptions Become Tests or Sensitivity Analyses

Randomization requires allocation integrity, limited interference, known assignment, and valid outcome capture. Test balance on pre-treatment features, check assignment hashes, enumerate cross-task shared state, and compare outcome missingness by arm. Cluster assignment or analysis when teams and tasks create dependence.

For binary outcomes, the risk difference is

$$\widehat{\tau} = \bar{Y}_1 - \bar{Y}_0$$

With rare harm, report arm counts, risk ratio when defined, exact or score intervals, and absolute difference. A zero in one arm does not prove zero risk. For clustered data, use cluster bootstrap, randomization inference, or cluster-robust models. Guardrails often need quantiles, not averages; a modest mean latency can hide a disastrous p95.

Multiplicity matters. One primary outcome and a small preregistered guardrail set protect decision clarity. Exploratory slices are labeled exploratory and used to design follow-up studies, not opportunistically to declare success.

Workshop Artifact: The Approval Evidence Ledger

```
stateDiagram-v2
    [*] --> Eligible
    Eligible --> AssignedV1: hash bucket
    Eligible --> AssignedV2: hash bucket
    AssignedV1 --> Proposed
    AssignedV2 --> Proposed
    Proposed --> Approved
    Proposed --> Rejected
    Proposed --> Escalated
    Approved --> Executed
    Approved --> Abandoned
    Executed --> Harmful: adjudication
    Executed --> Clean: mature 24h
    Executed --> Pending: horizon open
    Rejected --> NearMiss: counterfactual review
```

The state machine makes denominators explicit. “Near miss” after rejection is not an observed harmful counterfactual; it is an adjudicated unsafe proposal. The ledger stores assignment before downstream state, immutable request IDs, policy versions, reviewer identity, decision timestamps, execution trace references, outcome version, and evidence grade.

Approval authority itself is a control surface. Reviewers need scoped permissions, conflict-of-interest rules, workload limits, and protected audit logs. The experiment must not weaken production by routing dangerous requests to underprepared reviewers.

Executable Analysis: A Clustered Approval Trial

The fixed fixture contains eight teams, four per policy, with eligible request counts, harmful executions, p95 latency, and reviewer minutes. The analysis reports ITT harm rates and a team-cluster bootstrap confidence interval. Save as `ch11_approval_analysis.py` and run with the specified `.venv`.

```

import numpy as np

rows = [
    ("A", 0, 400, 5, 31, 180), ("B", 0, 350, 4, 29, 155),
    ("C", 0, 500, 7, 35, 225), ("D", 0, 450, 5, 33, 205),
    ("E", 1, 420, 2, 63, 300), ("F", 1, 380, 1, 68, 285),
    ("G", 1, 510, 3, 72, 390), ("H", 1, 440, 2, 65, 325),
]

def rate(group):
    selected = [r for r in rows if r[1] == group]
    return sum(r[3] for r in selected) / sum(r[2] for r in selected)

r0, r1 = rate(0), rate(1)
rng = np.random.default_rng(11)
by_arm = [[r for r in rows if r[1] == z] for z in (0, 1)]
deltas = []
for _ in range(20_000):
    sampled = []
    for arm in by_arm:
        sampled.append([arm[i] for i in rng.integers(0, len(arm), len(arm))])
    arm_rates = [sum(r[3] for r in a) / sum(r[2] for r in a) for a in sampled]
    deltas.append(arm_rates[1] - arm_rates[0])
lo, hi = np.quantile(deltas, [0.025, 0.975])
lat0 = max(r[4] for r in by_arm[0]); lat1 = max(r[4] for r in by_arm[1])
minutes = [sum(r[5] for r in arm) / sum(r[2] for r in arm) for arm in by_arm]

print(f"v1_harm={r0:.4%} v2_harm={r1:.4%}")
print(f"risk_difference={r1-r0:.4%} cluster_bootstrap_95=[{lo:.4%}, {hi:.4%}]")
print(f"max_team_p95_latency_delta={lat1-lat0}s")
print(f"review_minutes_per_request: v1={minutes[0]:.3f} v2={minutes[1]:.3f}")
print("decision=REVISE_AND_RETEST")

```

Exact fixture output:

```

v1_harm=1.2353% v2_harm=0.4571%
risk_difference=-0.7782% cluster_bootstrap_95=[-0.9484%, -0.6231%]
max_team_p95_latency_delta=37s
review_minutes_per_request: v1=0.450 v2=0.743
decision=REVISE_AND_RETEST

```

Why not expand? The harm criterion and latency guardrail pass, but reviewer burden rises 71 percent and the claim card did not define an acceptable burden threshold. The correct action is not to invent one after seeing results. Operations and security must set it, improve routing, then retest. The fixture teaches a crucial discipline: favorable statistics do not authorize a decision absent a complete decision contract.

The bootstrap resamples teams, reflecting assignment clusters. Eight clusters are too few for comfortable asymptotics; a real launch should use randomization inference and more clusters. The exact output is a software fixture, not a claim that the interval has ideal frequentist coverage.

Decision Logs Prevent Retrospective Certainty

Before launch, record the claim, owner, alternatives, approved evidence, analysis version, thresholds, stop conditions, and expiration. After analysis, append results and decision without overwriting the prior record. If leadership chooses rollout despite an unmet condition, record it as a risk acceptance with accountable authority.

A good log distinguishes “evidence says,” “model assumes,” and “decision chooses.” Analysts estimate; policy owners weigh costs; control owners implement; independent reviewers challenge. Combining these roles invites motivated reasoning.

Negative Controls and Falsification

A credible plan says what should not change. The approval gate should not affect read-only actions outside its scope. If their latency shifts, shared infrastructure or logging may explain the apparent effect. A pre-treatment outcome—harm recorded before assignment—must show no policy effect; a nonzero estimate reveals linkage or clock defects.

Falsification questions make the workshop adversarial. Could reviewers infer assignment and classify outcomes differently? Could the gate cause agents to split one dangerous command into several safe-looking commands? Could users move work to uninstrumented tools? Could latency shrink the denominator through abandonment? Each mechanism implies an auxiliary measure and sometimes a redesigned unit.

The estimand may be partially identified. If capture is missing for two percent of v1 and six percent of v2 requests, calculate worst-case bounds by assigning all missing v2 outcomes harmful and missing v1 outcomes benign, then reverse. If the decision changes across bounds, repair capture before rollout.

Power calculations should use the decision threshold and cluster structure, not a generic ritual. Estimate baseline harm, minimum worthwhile absolute reduction, intracluster correlation, cluster sizes, and attrition. For extremely rare harm, extend enrollment, use a validated intermediate outcome, apply controlled sequential monitoring, or combine capability tests with bounded observational evidence. Do not lower the evidentiary bar because the desired answer is expensive.

Claims also need transport conditions. A result from staffed infrastructure teams may transfer to another group only if action mix, reviewer skill, baseline risk, tool permissions, and outcome capture are sufficiently comparable. List those moderators before rollout. Re-estimate locally when they differ rather than applying the same threshold by organizational decree.

Archive abandoned claims as carefully as successful ones. A stopped study records why identification failed, which data were insufficient, and what future instrumentation would make the question answerable. This prevents another team from repeating the same attractive but invalid analysis and turns negative analytical work into institutional evidence.

Sources and Further Reading

- CONSORT 2010, randomized-trial reporting principles: <<https://www.consort-statement.org/>>
- Hernán and Robins, *Causal Inference: What If*: <<https://www.hsph.harvard.edu/miguel-hernan/causal-inference-book/>>
- NIST SP 800-53 Rev. 5, assessment and authorization controls: <<https://doi.org/10.6028/NIST.SP800-53r5>>
- NIST AI RMF 1.0, measurement and governance: <<https://doi.org/10.6028/NIST.AI.100-1>>
- IETF RFC 9411, *Benchmarking Methodology for Network Security Device Performance*: <<https://www.rfc-editor.org/rfc/rfc9411.html>>
- ASA, *Statement on Statistical Significance and P-Values*: <<https://doi.org/10.1080/00031305.2016.1154108>>
- OWASP, *Agentic AI Threats and Mitigations*: <<https://genai.owasp.org/>>

Interview Practicum

The interviewer says: “Prove our new agent approval gate makes us safer.” The candidate must ask questions; each good question unlocks a fact. Assignment is by team, not request. Teams share reviewers. Harm labels mature after 24 hours. The rollout excludes nights because nobody staffs approvals. Leadership wants results after one week. The new gate logs rejected commands but the old gate does not.

The strong response converts the request into an ITT claim for staffed eligible teams, names task or team as assignment unit, defines harmful execution and guardrails, and refuses to compare raw rejection counts because instrumentation differs. They flag one week as inadequate for stable team-level inference and external validity to nights.

Then the interviewer changes the constraint: randomization is politically impossible. The candidate proposes a phased rollout with pretrend checks, contemporaneous controls, explicit interference analysis, and a weaker evidence grade. When told the highest-risk team demands early treatment, they recognize treatment timing is confounded and avoid claiming a simple causal effect.

Worked Answer

Draft and approve the claim card before rollout. Instrument eligibility, assignment, proposal, review, execution, outcome, and availability times with immutable IDs. Randomize at the task or team level depending shared context. Blind outcome adjudication where feasible. Retain all assigned eligible units in the primary ITT analysis.

Estimate absolute harm-rate difference with cluster-aware uncertainty. Report counts and unresolved outcomes. Pre-register p95 latency, reviewer burden, abandonment, and bypass as guardrails with thresholds. Audit allocation integrity and differential missingness. Publish evidence grades and limitations. Expand only when the complete decision rule passes; otherwise revise, retest, or explicitly accept risk.

Follow-Up Questions and Answers

What if no harmful events occur? Report the exposure count and an upper confidence bound. Zero observations is not zero risk; supplement with safe adversarial tests that answer capability, not prevalence.

Can rejected unsafe commands count as prevented harm? They are near misses or unsafe proposals. Counterfactual harm requires assumptions or a design; do not count every rejection as a prevented incident.

Why not use requests as independent observations? Shared teams, tasks, reviewers, and repositories induce correlation. Ignoring clustering understates uncertainty.

Who sets the threshold? Accountable security and product owners set it before results, informed by loss, friction, and capacity. Analysts make consequences and uncertainty explicit.

Key Takeaways

- Formalize population, unit, policy contrast, outcomes, horizon, estimand, and decision before analysis.
- Assignment unit and analysis unit need not match; dependence must be honored.
- Control actions and detector outputs are not security outcomes.
- Evidence grades describe inferential support and must remain traceable to claims.
- Metric specifications define denominator, time, missingness, revisions, ownership, and tests.
- A favorable estimate cannot approve rollout when the decision contract is incomplete.

Chapter 12: Operating Security Data Science as a Control System

A lifecycle diagram is easy to draw and easy to ignore. Teams frame a question, collect logs, train a model, publish a dashboard, and call the work complete. Six months later nobody can reproduce the result, the schema has changed, analysts distrust the queue, and the model remains deployed because no owner has authority to retire it.

Security data science is not a sequence ending at deployment. It is an operating control system: evidence changes decisions; decisions change attackers, users, and telemetry; those changes alter future evidence. This chapter designs the operating model for Harbor, a company creating its first security data-science program and an eventual AI-agent security measurement platform.

The Lifecycle Is a Set of Accountable Gates

Harbor uses seven verbs—frame, instrument, collect, validate, analyze, intervene, monitor—but every transition has entry evidence, an accountable approver, and an exit artifact.

FRAME	-> signed claim card and harm model
INSTRUMENT	-> event contract, privacy review, coverage test
COLLECT	-> immutable raw evidence plus provenance manifest
VALIDATE	-> quality report, adversarial fixtures, known limitations
ANALYZE	-> reproducible result and uncertainty statement
INTERVENE	-> policy version, rollout bounds, rollback owner
MONITOR	-> outcome, drift, capacity, and retirement evidence

Work can move backward. Validation may expose an unanswerable claim; monitoring may require new instrumentation; a threat-model finding may cancel collection. A stage gate is not bureaucracy when it prevents an irreversible decision from resting on unknown evidence.

Each artifact has an owner and time-to-live. Claims expire when the system or adversary changes. Schemas have compatibility policy. Models have review dates. Exceptions have explicit expiration. “Still running” is not evidence of continuing fitness.

Maturity Is Decision Capability, Not Tool Count

Harbor assesses maturity across evidence, inference, intervention, and governance:

Stage	Decision capability	Typical artifact	Promotion test
0 Reactive	reconstruct isolated incidents	manually assembled timeline	provenance and clock agreement
1 Descriptive	quantify stable operations	versioned metric specification	denominator and coverage tests
2 Diagnostic	distinguish plausible mechanisms	slice and dependency analysis	competing explanations tested

Stage	Decision capability	Typical artifact	Promotion test
3 Predictive	rank future cases within scope	calibrated shadow score	temporal validation and capacity fit
4 Causal	estimate policy effects	preregistered experiment	assumptions and guardrails pass
5 Adaptive	update controls under feedback	governed champion/challenger loop	safe exploration and retirement work

Maturity is not globally uniform. Harbor may operate incident timelines at stage 2 and agent controls at stage 0. Buying a feature store does not advance causal capability. Promotion requires a repeated, audited decision process, not one successful project.

Stage 0 is not shameful. During a breach, reliable reconstruction can matter more than prediction. The danger is claiming a higher stage than evidence supports.

Threat-Model the Analytics Pipeline

The pipeline is part of the attack surface. An adversary can evade collection, poison features, manipulate labels, steal sensitive telemetry, probe thresholds, exhaust reviewers, or compromise deployment. Insiders and accidental changes can create the same effects.

Harbor models the pipeline by asset and trust transition:

```

flowchart TB
    S1[Endpoint sensors] -->|signed batches| I{Ingestion boundary}
    S2[Cloud audit] -->|checkpointed stream| I
    I --> R[(Immutable evidence lake)]
    R --> F[Point-in-time feature builder]
    F --> M[Model/evaluation workspace]
    M --> P{Policy approval boundary}
    P --> D[Decision service]
    D --> H[Human review queue]
    H --> O[(Outcome registry)]
    O -. delayed labels .-> M
    K[Key service] --> I
    K --> D
    A[Adversary] -. evade/poison .-> S1
    A -. probe .-> D
    X[Privileged insider] -. alter labels .-> O

```

The diagram exposes a shared key-service dependency, a human capacity node, delayed feedback, and distinct ingestion and policy boundaries. Threat analysis asks:

- Can a source impersonate another source?
- Can events be deleted, reordered, replayed, or backdated?
- Can later evidence leak into historical features?
- Who can alter labels, thresholds, or exclusion lists?
- Can queries reveal sensitive identities or detector boundaries?

- Can an attacker create queue exhaustion?
- Can the organization reconstruct every deployed decision?
- What happens when the outcome registry is unavailable?

Controls include source authentication, append-only manifests, sequence checks, schema quarantine, point-in-time joins, least privilege, two-person policy approval, isolated evaluation, data minimization, query auditing, rate limits, capacity shedding, signed artifacts, and practiced rollback. Every control needs its own telemetry; otherwise pipeline protection is merely asserted.

Reproducibility Is an Incident-Response Feature

A reproducible result binds code commit, environment lock, source snapshots, schema versions, extraction time, feature definitions, random seeds, parameter file, and output hashes. For mutable warehouses, store snapshot identifiers or an immutable extract manifest. A notebook whose query reruns against today's tables is not a historical record.

Peer review has three distinct passes. A security reviewer challenges threat assumptions and harm definitions. A data reviewer checks time, units, selection, leakage, uncertainty, and computation. An operator checks whether intervention and rollback are executable under real capacity. One generic approval box cannot substitute for all three.

Model governance begins before modeling. A rule, graph score, statistical test, or LLM judge can cause consequential action and belongs in the registry. Harbor records purpose, prohibited uses, training/evaluation lineage, owners, dependencies, thresholds, known failure modes, monitoring, review date, and retirement procedure.

Human-in-the-Loop Means Designed Authority

“A human reviews it” is not a safety argument. Reviewers need the right information, time, competence, authority, and appeal path. Harbor specifies four modes:

1. **human decides:** automation organizes evidence but cannot act;
2. **human confirms:** automation proposes a bounded action requiring approval;
3. **human supervises:** automation acts within limits while humans sample and can stop it;
4. **human handles exceptions:** automation acts normally, escalating uncertainty or policy conflicts.

Mode follows consequence and reversibility. A production database deletion may require confirmation; a reversible low-risk quarantine may operate under supervision. Automation bias is measured through disagreement audits and blind re-review. Reviewer fatigue is measured as queue age, decision time, override rate, and sampled error—not just ticket count.

Escalation paths must survive off-hours, conflicts, and outages. If no qualified reviewer is available, the system should fail into an explicitly chosen safe state: deny, defer, sandbox, or bounded automation. “Ask a human” without a service-level design is a latent outage.

Governance as Decision Rights

Harbor's council does not approve every query. It assigns decision rights:

Decision	Responsible	Accountable	Required challenge
collect a sensitive field	data steward	privacy owner	security and legal
change a harm definition	measurement lead	risk owner	independent reviewer
deploy a threshold	service owner	control owner	operations
grant an exception	requesting leader	risk executive	expiry review
retire a model	model owner	control owner	downstream consumers

The registry links decisions to evidence. Exceptions contain scope, reason, compensating control, approver, start, expiry, and verification. Governance quality is measured by expired exceptions still active, unowned artifacts, overdue reviews, rollback-test age, and time to resolve challenged evidence.

Privacy and security can conflict. More telemetry may improve detection while increasing employee or customer risk. Collection must show purpose, proportionality, retention, access, and deletion. Derived features can be as sensitive as raw events. Governance evaluates the complete lineage.

A Twelve-Month Roadmap Built on Dependencies

Harbor avoids a list of models. It sequences capabilities.

Quarter 1: establish truth boundaries. Inventory three consequential services and their identities; define six metric specifications; build immutable manifests and clock-quality checks; create an outcome registry; form review roles. Deliver one reproducible incident reconstruction. Success means 95 percent of selected source batches have verifiable provenance and every published metric has an owner and denominator.

Quarter 2: make evidence dependable. Add point-in-time feature construction, schema compatibility tests, source health, privacy controls, and sampled label adjudication. Ship a descriptive identity-risk view and measure coverage. Success means replay produces identical approved outputs and missingness is reported by source and asset.

Quarter 3: evaluate interventions. Shadow one agent approval policy, establish random or phased comparison, capacity guardrails, and rollback drills. Success means an approved claim card reaches a decision with mature outcomes and uncertainty—not necessarily that the model wins.

Quarter 4: operate and retire. Launch a bounded champion/challenger process, quarterly threat-model review, evidence-grade dashboard, and retirement workflow. Integrate the first platform slice. Success means an obsolete rule is safely retired and one policy decision is reconstructed end to end.

Portfolio priority can be scored for discussion, not automated choice:

$Priority = \frac{Decision \times Value \times Evidence \times Feasibility \times Reuse}{Delivery \times Cost \times Governance \times Risk}$

Inputs remain ranges with rationale. Hard constraints—legality, unacceptable harm, missing owner—veto regardless of score.

Unique Lab: Validate an Artifact Promotion Manifest

The lifecycle's smallest enforceable artifact is a promotion manifest. The lab checks that an analysis cannot advance to intervention without lineage, reviews, monitoring, and rollback. Save as `ch12_lifecycle_gate.py` and run using the book `.venv`.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Artifact:
    name: str
    code_sha: str
    data_snapshot: str
    schema_version: str
    security_review: bool
    data_review: bool
    operator_review: bool
    monitor_ids: tuple[str, ...]
    rollback_test_age_days: int
    expires_in_days: int

def gate(a: Artifact) -> list[str]:
    failures = []
    if not (len(a.code_sha) == 8 and a.code_sha.isalnum()):
        failures.append("invalid_code_sha")
    if not a.data_snapshot:
        failures.append("missing_data_snapshot")
    if not a.schema_version.startswith("v"):
        failures.append("unversioned_schema")
    if not all((a.security_review, a.data_review, a.operator_review)):
        failures.append("incomplete_peer_review")
    if len(a.monitor_ids) < 2:
        failures.append("insufficient_monitoring")
    if a.rollback_test_age_days > 30:
        failures.append("stale_rollback_test")
    if not 0 < a.expires_in_days <= 180:
        failures.append("invalid_review_expiry")
    return failures

fixtures = [
    Artifact("agent_gate_v2", "9ac14e2b", "snap-2026-08-01", "v3", True,
            True, True, ("harm-24h", "queue-p95"), 12, 90),
    Artifact("legacy_ranker", "main", "", "latest", True,
            False, True, ("alert-count",), 94, 0),
```

```

]

for item in fixtures:
    failures = gate(item)
    print(f"{item.name}: {'PROMOTE' if not failures else 'BLOCK'}")
    if failures:
        print(" " + ",".join(failures))

```

Exact fixtures are the two immutable `Artifact` instances. Exact expected output:

```

agent_gate_v2: PROMOTE
legacy_ranker: BLOCK

invalid_code_sha,missing_data_snapshot,unversioned_schema,incomplete_peer_review,insufficient_m
onitoring,stale_rollback_test,invalid_review_expiry

```

This is intentionally a policy gate, not a statistical lab. Its artifact is unique to lifecycle operations: a machine-checkable promotion contract. Production should validate signed identities, registry references, approved exception objects, compatible environment hashes, monitor ownership, and rollback evidence. Hard-coded rules should be policy-as-code with tests and change review.

The negative fixture is important. A useful laboratory does not merely demonstrate success; it proves that unsafe or unreproducible state is rejected for named reasons. Exact error ordering makes changes reviewable.

Capstone Preview: The Harbor Evidence Platform

Later chapters build the pieces that this preview connects. The capstone platform is not a monolithic “security AI.” It has six bounded planes:

- **evidence plane:** append-only source records, provenance, retention, and privacy controls;
- **semantic plane:** entities, event contracts, clocks, metric specifications, and outcome versions;
- **analysis plane:** point-in-time datasets, notebooks/jobs, evaluation suites, and uncertainty;
- **policy plane:** thresholds, approvals, exceptions, decision rights, and rollback;
- **execution plane:** detectors, agent controls, human queues, and bounded automated actions;
- **assurance plane:** lineage, monitoring, audits, adversarial tests, and retirement.

An AI-agent security use case traverses all six. Tool calls and prompts enter the evidence plane; task, identity, capability, and resource semantics enter the semantic plane; an evaluation estimates unsafe behavior; policy decides approval requirements; execution routes or blocks the action; assurance samples outcomes and tests rollback.

The platform's contract is traceability: for any consequential decision, answer what evidence was available, which semantic and model versions transformed it, what policy chose the action, who had authority, what happened later, and whether the artifact remained within its approved scope.

Failure Modes That a Roadmap Must Budget For

Programs fail when foundational work has no visible customer, when model projects outrun outcome instrumentation, when privacy review arrives after collection, when human queues are treated as infinite, and when nobody receives credit for retirement. Harbor assigns product owners to evidence capabilities and publishes decisions enabled, not terabytes ingested.

Another failure is maturity theater: dashboards labeled “causal,” a registry containing only model filenames, or a red-team event presented as prevalence. Independent review should sample claims and reconstruct them. If reconstruction fails, the program has an assurance incident.

Roadmaps also need stop rules. A project stops if the decision has no owner, the intervention cannot be changed, required collection is disproportionate, outcome maturity exceeds the decision window, or plausible uncertainty cannot alter action. Stopping low-value analysis is a sign of maturity.

Sources and Further Reading

- The Govern function and outcome taxonomy come from NIST's 2024 *Cybersecurity Framework*, version 2.0: <<https://doi.org/10.6028/NIST.CSWP29>>
- NIST, *AI Risk Management Framework 1.0*: <<https://doi.org/10.6028/NIST.AI.100-1>>
- NIST SP 800-218, *Secure Software Development Framework*: <<https://doi.org/10.6028/NIST.SP800-218>>
- NIST SP 800-53 Rev. 5, governance, audit, assessment, and contingency controls: <<https://doi.org/10.6028/NIST.SP800-53r5>>
- MITRE, *Adversarial Threat Landscape for AI Systems*: <<https://atlas.mitre.org/>>
- Google, *SRE Workbook*, monitoring and operational practices: <<https://sre.google/workbook/table-of-contents/>>
- ISO/IEC 23894:2023, *Artificial intelligence — Guidance on risk management*: <<https://www.iso.org/standard/77304.html>>

Interview Practicum

The assignment is an operating exercise, not a speech: “You are the first security data scientist. Build a twelve-month roadmap.” The interviewer hands the candidate twelve cards: cloud audit gaps, a noisy phishing model, no outcome registry, two analysts, an upcoming agent launch, privacy complaints, stale asset inventory, executive demand for a score, no experiment framework, strong incident responders, an unowned dashboard, and a fixed budget.

The candidate must place cards on a dependency wall. Strong candidates put ownership, asset/identity scope, event contracts, outcomes, privacy, and reproducibility before new predictive models. They exploit the strong incident team as a source of adjudication knowledge. They bound the agent launch with instrumentation, approval, and rollback rather than promising a perfect model.

At the halfway point, budget drops 30 percent. The candidate protects provenance, outcome capture, and one decision-focused pilot; they defer broad feature-platform work. Then a regulator requests explanation of agent actions. The candidate promotes decision reconstruction and retention policy, not a generic explainability dashboard. Finally, the CISO demands quarterly visible wins. The candidate defines wins as enabled decisions and reduced unknowns: source coverage verified, one metric retired, one policy evaluated, one rollback exercised.

Worked Answer

Begin with a four-week discovery that maps consequential services, identities, decisions, owners, telemetry, outcomes, privacy constraints, and current analytical artifacts. Score maturity per decision domain. Select one incident-reconstruction use case and one bounded agent-control use case.

Quarter 1 establishes claims, event contracts, manifests, outcome registry, and review authority. Quarter 2 establishes point-in-time data, quality controls, privacy enforcement, and reproducible descriptive measures. Quarter 3 shadows and evaluates one intervention with capacity guardrails. Quarter 4 deploys conditionally, monitors outcomes, runs rollback, and retires an obsolete artifact.

Govern with artifact owners, expirations, evidence grades, independent reviews, exceptions, and reconstruction audits. Measure the program through decisions supported, mature outcome coverage, reproducibility, review latency, control capacity, exception hygiene, and retirement—not model count.

Follow-Up Questions and Answers

What is the first hire after the lead? It depends on the bottleneck. With unreliable collection, hire or borrow a security data engineer; with sound evidence but weak decisions, add an applied scientist. Do not default to an ML title.

How do you show value before incidents are reduced? Demonstrate verified coverage, faster reconstruction, retired misleading metrics, bounded unknowns, exercised rollback, and a decision changed by evidence. Do not relabel activity as risk reduction.

Who can stop an automated control? A named on-call role with technical authority, documented triggers, and a tested path. Governance without executable stop authority is advisory.

When is the platform ready? When a bounded use case can be reconstructed end to end and fail safely. Platform completeness is not a prerequisite; traceable operation is.

Key Takeaways

- The lifecycle is a feedback control system with accountable gates, not a one-way project diagram.
- Maturity measures repeatable decision capability, not tooling or model count.
- Threat-model sensors, labels, features, models, policies, humans, and shared dependencies.
- Human review requires capacity, authority, evidence design, audit, and fallback behavior.

- Roadmaps should sequence truth, reliability, intervention evidence, operation, and retirement.
- The capstone platform's central promise is reconstruction of every consequential decision.

Chapter 13: Telemetry — What the System Can Actually Know

At 02:14, an autonomous support agent read a customer ticket, queried an internal account service, generated a refund request, and called a payment tool. The payment was blocked. A dashboard later said the control “caught an unsafe agent action.” That sentence claims more knowledge than the system necessarily possesses. Did the agent intend a refund? Did untrusted ticket text redirect it? Did the model propose the call or did deterministic orchestration construct it? Was the customer record returned by the tool the same record shown to the model? Did policy block execution, or did the payment API merely time out?

Telemetry is the boundary between what happened and what can be defended as known. A log is not a transparent window onto reality. It is a claim emitted by a component, under a particular configuration, at a particular clock time, with blind spots and incentives. Security analysis begins by naming those claims precisely.

The Epistemic Ladder

Treat every investigation as a climb through four levels:

1. **World state:** processes executed, credentials were exercised, packets traversed interfaces, people made decisions, and tools changed resources.
2. **Sensor observation:** an endpoint hook, identity provider, proxy, application, or agent runtime noticed some projection of that state.
3. **Recorded assertion:** the sensor serialized fields, attached its clock and identity, and attempted delivery.
4. **Analytical inference:** a rule, query, model, or investigator interpreted one or more assertions.

The levels must not be collapsed. A cloud audit record proves that the provider's control plane recorded an API request; it does not alone prove which human controlled the credential. A DNS log proves what a resolver observed, not every name a process attempted to resolve. An agent trace containing a prompt proves what the tracing layer stored, not necessarily the exact token sequence presented after provider-side transformations.

For a recorded event $\backslash(e\backslash)$, a useful evidence annotation is

$\backslash[K(e)=(s, o, c, d, i), \backslash]$

where $\backslash(s\backslash)$ is source identity, $\backslash(o\backslash)$ the observation mechanism, $\backslash(c\backslash)$ the configuration version, $\backslash(d\backslash)$ the delivery path, and $\backslash(i\backslash)$ integrity evidence. An event without these attributes may still be operationally useful, but its evidentiary ceiling is lower.

Seven Surfaces, Seven Different Truths

Endpoint telemetry observes local execution: process start, image hash, parent relationship, command arguments, module load, file mutation, socket creation, user session, and sensor health. Kernel or EDR observations are strong for activity on enrolled hosts, but container boundaries, short-lived workloads, encrypted filesystems,

tems, tampering, and unenrolled assets create gaps. A process tree is also an interpretation: parent identifiers can be reused, and operating-system semantics differ.

Identity telemetry records authentication and authorization. Preserve authentication method, credential or session identifier, issuer, subject, client, device claim, challenge result, token scopes, policy decision, and revocation. A successful MFA event says that a factor satisfied a policy; it does not prove benign control of the session. Authorization deserves its own event because “logged in” and “permitted this action” answer different questions.

Network telemetry spans flow records, DNS, DHCP, TLS metadata, proxy transactions, packet capture, and service-mesh observations. Flow records retain endpoints, ports, byte counts, flags, and timing at low cost; they omit content and may be distorted by NAT. Proxies see application requests only for routed traffic. Full packet capture has high investigative value but severe cost, privacy, encryption, and retention limits. No single network feed is “the network.”

Cloud telemetry includes control-plane audit, workload logs, configuration history, identity policy, object access, and managed-service diagnostics. Provider audit is authoritative about the provider's recorded API surface, but data-plane logging is often separately enabled. Organization-level collection, region coverage, account onboarding, and immutable export determine whether an apparently rich feed is complete.

Application telemetry supplies business meaning unavailable elsewhere: tenant, object, operation, authorization reason, workflow stage, request correlation, and outcome. It is also closest to developer change. Free-form logs, inconsistent identifiers, accidental secrets, and success messages emitted before transaction commit are common hazards. Prefer versioned security events at semantic boundaries over parsing prose.

Code and software-supply-chain telemetry records commits, reviews, branch protection, build identities, dependency resolution, provenance attestations, artifact digests, deployments, and runtime version. A repository event links an identity to a change; a signed provenance statement links a builder and inputs to an artifact. Neither alone proves the deployed bytes came from the reviewed source. The chain requires digest continuity.

Model and agent telemetry must represent nondeterministic reasoning and consequential execution without pretending that hidden model cognition is observable. Store inputs, context references, model and policy versions, outputs, tool proposals, approvals, executions, and resulting state as separate records.

A Trace Schema for Consequential Agents

An agent trace should be an event graph, not one enormous JSON transcript. Large blobs impede selective retention, access control, causal reconstruction, and schema evolution. The following envelope separates correlation, semantics, clocks, provenance, and sensitivity:

```
{
  "event_id": "evt_01J8B8YQ6V",
  "event_type": "agent.tool.execution.completed",
  "schema_version": "1.2.0",
```



```

"observed_at": "2026-08-20T14:03:12.481Z",
"occurred_at": "2026-08-20T14:03:12.402Z",
"trace_id": "tr_7f91",
"span_id": "sp_09",
"parent_span_id": "sp_08",
"actor": {"agent_id": "refund-agent", "run_id": "run_441"},
"subject": {"tenant_token": "tkn_93", "case_id": "case_1882"},
"action": {"tool": "payments.issue_refund", "operation": "execute"},
"authorization": {"policy_version": "pay-guard@31", "decision": "allow", "approval_id":
"ap_772"},
"input_ref": {"digest": "sha256:ab...", "vault_uri": "evidence://inputs/991"},
"output_ref": {"digest": "sha256:de...", "classification": "restricted"},
"execution": {"attempt": 1, "status": "succeeded", "resource_version_after": "refund/883@2"},
"runtime": {"orchestrator": "agent-runtime@8f31", "model": "model-release-2026-08"},
"integrity": {"collector": "trace-gateway-4", "sequence": 91882, "signature": "base64:..."}
}

```

Prompt content should usually be referenced by digest and controlled storage rather than copied into every event. Record the system instruction, user input, retrieved chunks, memory reads and writes, model response, policy evaluations, tool proposal, approval, tool execution, and observable effect as distinct span types. A proposed tool call is not an executed tool call. A tool's 200 OK is not proof that the intended state changed. Capture a postcondition or resource version when consequences matter.

OpenTelemetry supplies trace and span correlation, but security semantics still require conventions. W3C Trace Context propagates identifiers; it does not establish trust. An attacker can forge an incoming trace header, so gateways should distinguish an untrusted upstream identifier from an internally minted trace identity.

Sensitive values need field-level handling. Prompts can contain credentials, customer data, exploit text, and model-injection payloads. Attach classifications, tokenize identities where attribution is unnecessary, encrypt referenced payloads, and make access purpose-bound. Redaction must preserve analytical meaning: replacing every URL with [REDACTED] destroys the ability to find a repeated malicious domain. Store a keyed token or normalized feature alongside restricted raw evidence.

Authentication, Authorization, and Sensitive Actions

A defensible chain for a sensitive action contains at least five assertions:

```

credential presented -> authentication evaluated -> session/token issued
-> authorization evaluated -> resource mutation observed

```

Each arrow can fail independently. Identity-provider success without application authorization cannot explain why access was granted. Application authorization without token issuance context cannot show whether step-up authentication was required. A tool execution event without resource mutation cannot distinguish a no-op, timeout, rollback, or misleading response.

Define a sensitive-action catalog rather than relying on log severity. Examples include changing MFA factors, granting a role, exporting records, rotating a key, modifying a detector, approving an agent tool, publishing a

package, and disabling collection. Each catalog entry names required evidence: actor, effective principal, target, old and new state, authorization policy, approval, request and result identifiers, and independently observable postcondition.

Coverage Is a Set Problem, Not a Volume Metric

Terabytes per day do not answer whether a decision is observable. Let (U) be the set of required evidence claims for a threat or control. Sensor (j) covers subset (S_j) , has reliability (r_j) , latency (l_j) , cost (c_j) , and common-mode dependency (g_j) . Naive coverage is

$$C = \frac{|\bigcup_j S_j|}{|U|}.$$

Weighted coverage assigns consequence weight (w_i) to each claim:

$$C_w = \frac{\sum_{i \in \cup S_j} w_i}{\sum_{i \in U} w_i}.$$

Neither formula captures independence. Endpoint and local forwarding logs may both disappear when the host is compromised. Two feeds sharing the same upstream collector are not two independent witnesses. Mark common dependencies and require diverse evidence for high-consequence claims.

The practical artifact is a coverage matrix whose rows are evidence requirements and columns are sensors. Cells should be richer than yes/no: **D** for direct observation, **I** for inference, **H** for sensor-health evidence, and blank for no support. Add scope, latency, retention, failure visibility, owner, and validation date outside the matrix. The matrix is versioned because products, policies, and fleets change.

Minimization Without Self-Blinding

Collection minimization asks which evidence is necessary for a defined decision, not which logs are inexpensive. Begin with the sensitive action and work backward. If the decision is whether to suspend a session after impossible token use, exact message bodies and shell history are disproportionate; token lineage, device binding, regions, timing, policy result, and session revocation are germane.

Minimize along several axes:

- **field:** omit content when category, digest, or keyed token suffices;
- **population:** instrument privileged or exposed workflows more deeply than unrelated users;
- **time:** retain high-volume raw evidence briefly and durable derived claims longer;
- **resolution:** aggregate health metrics while preserving case evidence under authorization;
- **access:** separate operational search, investigation, model training, and audit purposes;
- **location:** derive features near the source when central raw collection adds risk without value.

Minimization can create an undocumented blind spot. Record every deliberate omission as a coverage exception with rationale, compensating evidence, approver, and review date. That turns “we do not collect it” from tribal knowledge into an explicit risk decision.

Executable Lab: Audit an Agentic Refund Path

The lab computes weighted direct coverage, reports inferential-only and missing claims, detects common-mode witness failures, and finds a minimum-cost sensor set that directly covers every mandatory claim. Save as `cov-erage_audit.py` and run with the book interpreter.

```
from itertools import combinations

requirements = {
    "untrusted_input": (2, False),
    "model_context": (3, False),
    "tool_proposal": (5, True),
    "policy_decision": (5, True),
    "human_approval": (4, True),
    "tool_execution": (5, True),
    "resource_postcondition": (5, True),
    "collector_health": (3, False),
}

sensors = {
    "agent_trace": {"cost": 3, "group": "runtime", "D": {"model_context", "tool_proposal"},
    "I": {"tool_execution"}},
    "policy_log": {"cost": 2, "group": "policy", "D": {"policy_decision",
    "human_approval"}, "I": set()},
    "tool_gateway": {"cost": 3, "group": "runtime", "D": {"tool_execution"}, "I":
    {"policy_decision"}},
    "payment_audit": {"cost": 4, "group": "payment", "D": {"resource_postcondition"}, "I":
    {"tool_execution"}},
    "input_proxy": {"cost": 2, "group": "edge", "D": {"untrusted_input"}, "I": set()},
    "health_ledger": {"cost": 1, "group": "ops", "D": {"collector_health"}, "I": set()},
}

selected = list(sensors)
direct = set().union(*(sensors[s]["D"] for s in selected))
inferred = set().union(*(sensors[s]["I"] for s in selected)) - direct
missing = set(requirements) - direct - inferred
total_weight = sum(weight for weight, _ in requirements.values())
direct_weight = sum(requirements[r][0] for r in direct)

mandatory = {r for r, (_, required) in requirements.items() if required}
candidates = []
for size in range(1, len(sensors) + 1):
    for names in combinations(sensors, size):
        observed = set().union(*(sensors[n]["D"] for n in names))
        if mandatory <= observed:
            candidates.append((sum(sensors[n]["cost"] for n in names), names))
    if candidates:
        break

best_cost, best = min(candidates)
runtime_claims = set().union(*(sensors[s]["D"] for s in selected if sensors[s]["group"] ==
"runtime"))
```

```
print(f"direct_weighted_coverage={direct_weight}/{total_weight} ({direct_weight/
total_weight:.1%})")
print(f"inferential_only={sorted(inferred)}")
print(f"missing={sorted(missing)}")
print(f"minimum_mandatory_set={list(best)} cost={best_cost}")
print(f"runtime_common_mode_claims={sorted(runtime_claims)}")
```

Exact output:

```
direct_weighted_coverage=32/32 (100.0%)
inferential_only=[]
missing=[]
minimum_mandatory_set=['agent_trace', 'policy_log', 'tool_gateway', 'payment_audit'] cost=12
runtime_common_mode_claims=['model_context', 'tool_execution', 'tool_proposal']
```

The 100 percent result is not a declaration of perfect observability. It says only that the enumerated claims have a direct source in the fixture. The last line identifies a correlated dependency: trace and gateway evidence share the runtime group. The resource postcondition from the payment domain provides an independent witness for the consequence. A production audit should also test fleet scope, delivery lag, retention, schema validity, sequence gaps, and controlled canary events.

Operating the Unknowns Register

Coverage gaps belong in an operational register. Each entry states the decision impaired, affected scope, adversary opportunity, present proxy, planned remedy, owner, and expiration. Distinguish **known absent** evidence from **unexpected silence**. The former is design debt; the latter may be an incident.

Use canaries to verify end-to-end observation. A synthetic privileged action can traverse authentication, authorization, application, export, normalization, and detection paths. The canary succeeds only if every expected claim arrives within its latency objective and can be joined without manual guessing. Merely checking that a collector process is alive tests machinery, not knowledge.

Detection and response produce their own telemetry: rule version, alert inputs, suppression, routing, analyst access, disposition, containment, restoration, and later outcome. Vulnerability programs need scanner identity, asset scope, finding instance, evidence, deduplication lineage, ownership, exception, remediation, verification, and recurrence. Product telemetry should record whether security controls are enabled, successfully exercised, bypassed, abandoned, or creating friction. These operational records reveal whether an evidence system changes decisions rather than merely accumulating events.

Sources and Further Reading

- NIST SP 800-92, *Guide to Computer Security Log Management*: <<https://doi.org/10.6028/NIST.SP800-92>>
- NIST SP 800-61 Rev. 2, *Computer Security Incident Handling Guide*: <<https://doi.org/10.6028/NIST.SP800-61r2>>

- OpenTelemetry, trace concepts and semantic conventions: <<https://opentelemetry.io/docs/concepts/signals/traces/>>
- W3C, *Trace Context*, Recommendation: <<https://www.w3.org/TR/trace-context/>>
- Open Cybersecurity Schema Framework specification: <<https://schema.ocsf.io/>>
- MITRE ATT&CK, data sources and data components: <<https://attack.mitre.org/datasources/>>
- SLSA, provenance model and build levels: <<https://slsa.dev/spec/>>
- IETF RFC 5424, *The Syslog Protocol*: <<https://www.rfc-editor.org/rfc/rfc5424>>

Interview Practicum

You inherit telemetry for a customer-service agent that can credit accounts. The red team demonstrated prompt injection through ticket text, yet the company stores only final model text and application exceptions. The network team refuses payload capture, Legal prohibits indefinite prompt retention, the payment team offers a daily audit export, and launch is in six weeks.

The interviewer asks: **Design the minimum telemetry needed to investigate unsafe credits and evaluate the approval control.** After ten minutes, two constraints change: payment audit becomes available only after 24 hours, and the agent platform team says trace identifiers can be supplied by clients.

A strong candidate first asks which actions are consequential, maximum credit, authorization boundary, approval modes, tenant isolation, investigation horizon, expected volume, regulator needs, and who can retrieve restricted content. They separate proposal, authorization, execution, and postcondition. They reject client-supplied trace IDs as trusted roots and mint an internal run identity at ingress while preserving the external value as an untrusted correlation attribute.

Worked Answer

Instrument ticket ingestion with content digest, source trust label, tenant token, and restricted payload reference. Record ordered references to system instructions, retrieved material, memory reads, model release, sampling configuration, model response, and parsed tool proposal. The policy service emits rule version, evaluated attributes, decision, and reason. Human approval emits approver role, displayed evidence digest, decision, and latency. The gateway records the exact normalized call and idempotency key. Payment audit supplies independent postcondition evidence one day later.

For immediate containment, treat gateway success as provisional and reconcile it with payment audit. Alert on missing reconciliation after 26 hours. Retain structured trace metadata longer than prompt content; place content in purpose-bound encrypted storage with shorter retention and case holds. Preserve keyed indicators derived from untrusted text so repeated injections can be found without broad content access.

Coverage tests inject a synthetic ticket, approve and deny controlled calls, verify correlation and ordering, then reconcile daily results. The launch gate requires complete observation of all tool proposals, policy decisions, approvals, executions, and eventual postconditions; sequence-gap monitoring; access audit; and a tested kill switch. Report both direct coverage and the 24-hour consequence-confirmation lag.

Weak answers ask for “all logs,” equate model output with executed action, ignore sensor integrity, or promise that distributed tracing proves causality. A costly but still weak answer captures every packet while omitting authorization policy versions and resource postconditions.

Follow-Up Questions and Answers

What if prompts cannot be retained at all? Retain digests, trust labels, token counts, retrieval identifiers, policy-relevant features, and a short-lived quarantined payload only when lawful. Evaluation becomes less capable of semantic replay; document that limitation rather than manufacturing certainty.

How do you distinguish tool retry from duplicate malicious execution? Carry a stable idempotency key across proposal, approval, gateway attempts, and resource audit. Preserve attempt numbers and final resource version.

Would you adopt OCSF or OpenTelemetry? Use OpenTelemetry for propagation and span mechanics, and map security events to a governed semantic model such as OCSF where categories fit. Neither substitutes for agent-specific action and authorization contracts.

How do you prove collection is alive? Exercise a harmless synthetic action end to end and require the expected event graph, latency, signatures, and reconciliation—not just collector heartbeat.

Thirty-second summary? Separate observed claims from inference; capture proposal, policy, approval, execution, and consequence as distinct events; minimize sensitive content; independently verify high-impact state changes; continuously test coverage.

Key Takeaways

- Telemetry is a chain of claims with an evidentiary ceiling, not reality itself.
- Endpoint, identity, network, cloud, application, code, model, and agent sources observe different projections.
- Agent traces need separate, correlated records for context, proposals, decisions, executions, and postconditions.
- Coverage is defined against required evidence and failure independence, not event volume.
- Minimization should reduce fields, populations, time, resolution, access, and movement without concealing the resulting gaps.
- Synthetic end-to-end actions test whether a system can actually know what its diagrams claim.

Chapter 14: Security Event Schemas and Data Modeling

The incident timeline looked impossible. At 09:02, `alex@northwind` authenticated from a managed laptop. At 09:04, `a.chen` downloaded a design. At 09:05, device `7F-22` reached a restricted database. The identity team counted three actors. All three records described one contractor after an email rename, directory migration, and device re-enrollment.

Security modeling preserves distinctions until evidence justifies a join. It must answer opposing questions: which records refer to the same entity, and which similar records must remain separate? A warehouse that never resolves identity cannot reconstruct behavior. One that resolves too aggressively can accuse the wrong person.

Facts Need a Declared Grain

A fact records an occurrence at a declared grain. A useful activity grain is: **one row represents one source-observed attempt by an effective principal to perform one operation on one resource**. A retry is another attempt, not an update. One request affecting five resources may require five resource-action facts plus a request fact. An alert derived from events belongs in a detection fact with lineage; it is not another activity event.

Dimensions describe entities or classifications: identity, device, resource, application, organizational unit, vulnerability, and detector. They change independently. If a device becomes unmanaged today, yesterday's activity must not silently acquire today's status.

A compact model includes `fact_activity` plus versioned identity, device, and resource dimensions. Facts carry event, occurrence and ingestion times, actor, device, resource, operation, result, source, and source locator. Dimensions carry stable entity keys and time-bounded attributes. Bridge tables express many-to-many relationships such as identity-to-account assertions.

Surrogate keys improve joins, but preserve natural identifiers and issuing authority. `12345` means nothing without `issuer=okta-prod` or `source=aws-account-17`. Global uniqueness is a contract, not a visual property.

Sessions Are Hypotheses With Boundaries

Some systems emit explicit sessions with issue, renewal, and revocation. Preserve that source session. Analysts also infer sessions by grouping activity across inactivity gaps. If ordered event times are $\backslash(t_1, \ldots, t_n\backslash)$, a new gap session begins when

$$\backslash[t_i - t_{i-1} > \Delta. \backslash]$$

The threshold $\backslash(\Delta\backslash)$ is a modeling parameter. Thirty minutes may fit interactive administration yet merge unrelated automation or split a long remote shell. Version the algorithm and threshold, and attach confidence when device or credential binding is uncertain.

Do not use a session to erase actor layers. A human operates a browser, which exercises an application service account, which assumes a cloud role. Model `initiating_subject`, `effective_principal`, and `credential/session` separately. “Who caused this?” may mean human initiator, target-accepted principal, executing process, or accountable automation owner.

Identity Resolution Is Evidence Accumulation

Begin with assertions, not a universal user table. An assertion has a source identifier, candidate entity, issuer, assertion type, interval, source record, and confidence. Strong links include an immutable directory object ID, certificate key identifier, device enrollment record, or cloud role session naming its source. Weak links include display name, mutable email, IP, and temporal proximity.

Resolution forms a graph of source identifiers and canonical entities. Deterministic rules can join `(issuer, immutable_subject_id)`. Probabilistic linkage can combine evidence:

$$-\log \frac{P(M \mid x)}{P(\neg M \mid x)} = -\log \frac{P(M)}{P(\neg M)} + \sum_k -\log \frac{P(x_k \mid M)}{P(x_k \mid \neg M)},$$

where (M) means two identifiers represent the same entity. Rare shared attributes are stronger evidence than common ones; contradictory immutable IDs should dominate a matching name.

Use asymmetric thresholds. Auto-link only above a high threshold, queue ambiguity, and preserve “not same entity” constraints. A false merge contaminates every behavioral profile. A false split loses history but is often reversible. Privileged identities warrant stricter rules.

Service principals, workload identities, shared accounts, and agents must not be coerced into employee rows. Give principals a type and relate them to accountable people or teams without asserting equivalence. A bot owned by Finance is not the finance director.

Slowly Changing Context and Bitemporal Truth

Type 1 slowly changing dimensions overwrite attributes and suit corrections whose history is irrelevant. Type 2 creates a version with `valid_from` and `valid_to`; it is necessary when historical context affects interpretation. Department, manager, privilege tier, device ownership, exposure, and resource sensitivity often require Type 2.

Investigations also need bitemporality:

- **valid time:** when an assertion was true in the modeled world;
- **system time:** when the analytical system learned and stored it.

Suppose HR tells the warehouse on August 10 that a contractor left August 3. A current query about August 5 should show no active employment. Reconstruction of an alert generated August 6 must admit the warehouse still believed the contractor active.

A bitemporal version occupies

$\backslash [[v_{\text{from}}, v_{\text{to}}) \backslash \text{times}[s_{\text{from}}, s_{\text{to}})). \backslash]$

Corrections close the prior system-time rectangle and insert a corrected belief. Half-open intervals avoid double matches. The as-of join for event time $\backslash(t\backslash)$ and knowledge cutoff $\backslash(k\backslash)$ is:

```
v.valid_from <= t AND t < v.valid_to
AND v.recorded_from <= k AND k < v.recorded_to
```

Enforce non-overlap for versions of the same entity and belief state. Never update the only historical copy when reproducibility matters.

Four Clocks and Partial Order

Preserve `occurred_at` (source claim), `observed_at` (sensor observation), `ingested_at` (durable platform acceptance), and `processed_at` (transformation output). Store source timezone or offset, clock quality, and sequence numbers when available. Normalize for comparison but retain original timestamp text. Daylight-saving folds can produce duplicate local times; batching, mobile offline operation, virtual-machine drift, and coarse clocks weaken ordering.

If two times differ by less than their combined uncertainty, do not present their order as definite. Sequence numbers, transaction IDs, parent spans, and resource versions are often stronger. A state transition from version 8 to 9 establishes an order even if clocks disagree.

Streams use event time for semantics and processing time for operations. A watermark approximating `max_event_time_seen - allowed_lateness` trades latency for corrections. Late events must enter a correction path rather than disappear. Reports need an `as_of` time and revision state such as provisional, finalized, or corrected.

One Semantic Model, Four Physical Forms

Relational databases excel at constraints, point lookups, transactions, temporal dimensions, and explainable SQL. They are strong systems of record for identity, policy, and case state.

Columnar stores optimize scans and compression. Parquet or a columnar warehouse suits immutable events and feature computation. Avoid tiny files and high-cardinality partitions; partition by coarse time and perhaps tenant, then cluster for common predicates.

Graphs make variable-length paths first-class: identity assumed role, role accessed bucket, bucket contained artifact, artifact deployed to host. Every edge still needs direction, type, time, source, and confidence. A graph does not abolish provenance.

Streams represent changes in motion, with replay, partitions, and consumer offsets. Ordering normally holds only within a partition, so choose keys around invariants. A stream is not automatically an analytical model; consumers still need schemas, state, and late-event policy.

The semantic contract defines identifiers, grains, event types, clocks, units, classifications, and version rules. Physical projections optimize workloads. Graph edges, rows, Parquet records, and messages should remain traceable to the same source assertion.

Normalization Without Invented Semantics

Preserve raw evidence or a verifiable reference, parse source-specific typed fields, then map to normalized categories. If a source reports login success without authentication method, store method as unknown; do not infer password from absence of MFA.

Standards occupy different layers. OCSF defines cybersecurity categories. Elastic Common Schema supplies reusable fields. STIX 2.1 models threat-intelligence objects and relationships, while TAXII transports collections. OpenTelemetry correlates traces, metrics, and logs. CloudEvents standardizes an event envelope. Avro, Protocol Buffers, and JSON Schema express serialization contracts.

Mapping is rarely lossless. Retain extension fields and mapping versions. Mark whether normalized values were direct, transformed, defaulted, inferred, or unknown. Interoperability failures often arise from grain differences: one feed emits a flow summary while another emits requests.

Executable SQLite Lab: Query Two Knowledge Cutoffs

The model below contains identities, aliases, bitemporal device ownership, and activity. The warehouse first believed device `dev-7` belonged to Alex through August 10. On August 12 it learned ownership ended August 5 and Riley became owner. Run with the book interpreter.

```
import sqlite3

db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE identity (
  identity_id INTEGER PRIMARY KEY,
  display_name TEXT NOT NULL,
  principal_type TEXT NOT NULL CHECK(principal_type IN ('human','service','agent'))
);
CREATE TABLE alias (
  issuer TEXT NOT NULL, subject TEXT NOT NULL,
  identity_id INTEGER NOT NULL REFERENCES identity,
  valid_from TEXT NOT NULL, valid_to TEXT NOT NULL,
  PRIMARY KEY (issuer, subject, valid_from)
);
CREATE TABLE device_owner_bt (
  device_id TEXT NOT NULL,
  identity_id INTEGER NOT NULL REFERENCES identity,
  valid_from TEXT NOT NULL, valid_to TEXT NOT NULL,
  recorded_from TEXT NOT NULL, recorded_to TEXT NOT NULL,
  source TEXT NOT NULL,
```

```

    PRIMARY KEY(device_id, valid_from, recorded_from)
);
CREATE TABLE activity (
    event_id TEXT PRIMARY KEY,
    occurred_at TEXT NOT NULL, ingested_at TEXT NOT NULL,
    issuer TEXT NOT NULL, subject TEXT NOT NULL,
    device_id TEXT NOT NULL, resource TEXT NOT NULL, operation TEXT NOT NULL
);
INSERT INTO identity VALUES (1,'Alex Chen','human'), (2,'Riley Shah','human');
INSERT INTO alias VALUES
    ('idp','alex@northwind',1,'2026-01-01','9999-12-31'),
    ('repo','a.chen',1,'2026-01-01','9999-12-31');
INSERT INTO activity VALUES
    ('e1','2026-08-07T09:04:00Z','2026-08-07T09:04:05Z','repo','a.chen','dev-7','design/
    phoenix','download');
INSERT INTO device_owner_bt VALUES
    ('dev-7',1,'2026-07-01','2026-08-10','2026-08-01','2026-08-12','mdm'),
    ('dev-7',1,'2026-07-01','2026-08-05','2026-08-12','9999-12-31','mdm-correction'),
    ('dev-7',2,'2026-08-05','9999-12-31','2026-08-12','9999-12-31','mdm-correction');
"""

query = """
SELECT a.event_id, i.display_name,
    COALESCE(owner.display_name, 'UNKNOWN')
FROM activity a
JOIN alias x ON x.issuer=a.issuer AND x.subject=a.subject
    AND x.valid_from <= a.occurred_at AND a.occurred_at < x.valid_to
JOIN identity i ON i.identity_id=x.identity_id
LEFT JOIN device_owner_bt d ON d.device_id=a.device_id
    AND d.valid_from <= a.occurred_at AND a.occurred_at < d.valid_to
    AND d.recorded_from <= :cutoff AND :cutoff < d.recorded_to
LEFT JOIN identity owner ON owner.identity_id=d.identity_id
ORDER BY a.event_id
"""

for cutoff in ('2026-08-08', '2026-08-13'):
    row = db.execute(query, {"cutoff": cutoff}).fetchone()
    print(f"knowledge_cutoff={cutoff} event={row[0]} account={row[1]} device_owner={row[2]}")

orphans = db.execute("""
SELECT COUNT(*) FROM activity a
LEFT JOIN alias x ON x.issuer=a.issuer AND x.subject=a.subject
    AND x.valid_from <= a.occurred_at AND a.occurred_at < x.valid_to
WHERE x.identity_id IS NULL
""").fetchone()[0]
print(f"unresolved_activity={orphans}")

```

Exact output:

```

knowledge_cutoff=2026-08-08 event=e1 account=Alex Chen device_owner=Alex Chen
knowledge_cutoff=2026-08-13 event=e1 account=Alex Chen device_owner=Riley Shah
unresolved_activity=0

```

The account remains Alex because repository alias evidence is unchanged. Device ownership changes with knowledge cutoff. An August 8 alert reconstruction shows Alex as then-believed owner; a current timeline shows Riley and exposes the correction. Their mismatch is valuable and must not be erased by overwriting either fact.

Production needs constraints or validation to reject overlapping intervals. Tests should cover boundary instants, alias reuse, null identifiers, late corrections, deleted entities, timezone folds, and accidental one-to-many as-of joins.

Review the Model by Trying to Break History

Ask whether every fact has a grain and stable source locator. Verify that identifiers include namespace and type. Label joins as exact, asserted, inferred, or convenient. Require intervals for changing relationships. Preserve unknown as distinct from false, empty, and zero. Make units explicit.

Then test a renamed user, reimaged device, recycled IP, assumed role, shared mailbox, and workload identity. Can an analyst reproduce what was known at an earlier decision time? Can late facts revise aggregates without destroying the first result? Can replay create duplicates? If any answer depends on manually remembering context, the model is incomplete.

Cardinality tests deserve special attention. A fact-to-dimension join expected to be many-to-one can become many-to-many when temporal versions overlap or an alias is reused. The resulting row multiplication inflates counts while every individual row still appears valid. Assert expected cardinality before and after each join, publish unmatched and multiply matched rates, and fail closed for high-consequence products. When ambiguity is real, represent a set of candidate links rather than selecting the first database row.

Deletion also needs semantics. Privacy erasure, source retention, and entity retirement do not mean the same thing. A tombstone can state that an identifier existed and is no longer available without retaining prohibited attributes. Referential integrity may use durable pseudonymous keys while restricted lookup material expires. Analysts must be able to distinguish “entity deleted under policy,” “source never supplied this entity,” and “resolution currently failed.” Collapsing all three to null destroys both auditability and missingness analysis.

Schema evolution should therefore be rehearsed with historical queries. Add a new principal type, split one operation into two, change a resource hierarchy, and retire an alias namespace in a test snapshot. The acceptance criterion is not only that new events load; old as-of queries, alert reconstructions, and lineage pointers must continue to mean what they meant under their recorded versions.

Sources and Further Reading

- Richard T. Snodgrass, *Developing Time-Oriented Database Applications in SQL*: <<https://www2.cs.arizona.edu/~rts/publications.html>>
- Ivan P. Fellegi and Alan B. Sunter, “A Theory for Record Linkage,” *JASA* 1969: <<https://doi.org/10.1080/01621459.1969.10501049>>
- Open Cybersecurity Schema Framework: <<https://schema.ocsf.io/>>

- OASIS, *STIX Version 2.1*: <<https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>>
- OASIS, *TAXII Version 2.1*: <<https://docs.oasis-open.org/cti/taxii/v2.1/taxii-v2.1.html>>
- CNCF, *CloudEvents Specification*: <<https://cloudevents.io/>>
- Apache Parquet format: <<https://parquet.apache.org/docs/>>
- OpenTelemetry data model: <<https://opentelemetry.io/docs/specs/otel/common/>>

Interview Practicum

A hospital acquires a clinic. For 90 days both directories remain active. Email addresses collide, nurses share workstation sessions, devices are re-enrolled, and cloud audit arrives six hours late. The SOC wants a graph showing “every action by a person.” Mid-interview, the panel reveals that a privileged account was reassigned before a breach was discovered and Legal requires reproduction of the original alert.

The prompt is: **Design the identity–device–resource model and explain how analysts query it.** Strong questions identify immutable IDs, reuse rules, shared-account policy, device hardware evidence, clock quality, grains, correction sources, privacy constraints, and whether “person” means initiator, effective principal, or accountable owner.

Worked Answer

Create typed entities for people, accounts, credentials, devices, workloads, sessions, and resources. Ingest namespace-qualified aliases. Store evidence edges—assigned-to, authenticated-as, operated-from, assumed-role, accessed—with valid and recorded intervals, issuer, locator, and confidence. Never merge on email alone. Preserve a shared workstation session as its own entity linked to possible humans unless badge or step-up evidence resolves it.

Activity facts retain source actor, effective principal, device claim, resource, operation, result, four clocks, and raw locator. Bitemporal dimensions model account assignment, department, device ownership, and sensitivity. A current query uses event valid time plus the latest recorded view. An original-alert reconstruction adds the alert's knowledge cutoff. The graph is a projection of governed temporal facts, not an unversioned truth store.

Six-hour watermarks produce provisional dashboards, followed by finalized and corrected states. The reassigned privileged account remains one account entity with separate assignment intervals. The defensible conclusion is: “The effective principal was account P; at that valid time it was assigned to A according to knowledge available then.”

Follow-Up Questions and Answers

Why not use a graph database for everything? Traversal is useful, but temporal constraints, deduplication, bitemporal corrections, and scans may be clearer elsewhere. Project one contract into fit-for-purpose stores.

What if two high-confidence links conflict? Preserve both assertions, block auto-merge, record the conflict, and adjudicate. Confidence does not replace consistency.

Most dangerous shortcut? Joining mutable email, hostname, IP, or display name as immutable identity.

Explain bitemporality simply. One clock says when something was true; another says when we learned it. Both are needed to reconstruct history and judge past decisions.

Five-minute summary? Declare grains; namespace identities; resolve through provenance-bearing temporal assertions; model changing context with valid and system time; retain multiple clocks; use several physical stores without multiplying meanings.

Key Takeaways

- Facts record occurrences at a declared grain; dimensions supply versioned context.
- Sessions may be source-issued or inferred and must retain construction rules.
- Identity resolution is evidence-based linkage, not string equality.
- Bitemporal models distinguish what was true from what the system knew.
- Occurrence, observation, ingestion, and processing times answer different questions.
- Physical representations can vary while semantics and provenance remain stable.

Chapter 15: Building Reliable Security Data Pipelines

At 10:00 the detection team deployed a rule for impossible role assumptions. At 10:07 its alert rate fell to zero. The on-call engineer celebrated until an incident responder asked a basic question: had attacks stopped, or had the enrichment join stopped attaching role ownership? The raw cloud stream was healthy. A daily identity snapshot had arrived with a renamed column, the job had converted the missing field to null, and the detector filtered null owners as out of scope.

A security pipeline is part of the control it supports. Its correctness includes not only delivery, but semantic preservation, reproducibility, failure visibility, and safe recovery. “Exactly once” marketing cannot repair a pipeline that emits the wrong meaning once.

Choose Architecture From the Decision Clock

Batch processing reads bounded inputs and writes bounded outputs. It is natural for daily asset snapshots, vulnerability scans, model training, historical aggregates, and reconciliations. Batch gives deterministic reruns and straightforward snapshots, but its latency is at least the schedule plus completion time.

Streaming processes unbounded records as they arrive. It suits session revocation, high-risk authorization, exfiltration signals, and agent tool gates where minutes or seconds matter. Streams introduce event-time state, watermarks, replay, partition ordering, and operational complexity.

Hybrid systems use a fast path for provisional action and a slower path for authoritative correction. A streaming detector can suspend a token from current evidence; a nightly job reconciles late audit records and recomputes metrics. Hybrid is not duplicating every transformation twice. Share contracts and transformation logic where possible, then declare which output is provisional and which is authoritative.

Start with the intervention deadline. If an access review occurs weekly, subsecond ingestion wastes complexity. If an autonomous tool can transfer funds in seconds, a daily job is not a control. Also consider source behavior: a vendor API that exposes mutable 24-hour windows may require scheduled extraction even when downstream processing is streaming.

The Pipeline as a Sequence of Commitments

A useful decomposition is:

```
source -> durable landing -> validation -> normalization -> enrichment
      -> stateful correlation -> decision product -> reconciliation
```

Durable landing stores immutable source bytes or content-addressed objects before destructive parsing. Validation checks envelope, schema, identifiers, clocks, and source sequence. Normalization maps source meaning without fabricating absent fields. Enrichment performs point-in-time joins. Correlation creates

declared sessions or windows. The decision product may be an alert, feature view, policy signal, or metric. Reconciliation compares fast-path claims with authoritative later evidence.

Each transition needs an input version, code version, contract version, output identity, and error policy. Silent row dropping is rarely acceptable. Route invalid records to quarantine with reason, source locator, and bounded sensitive content; alert on rate and age.

Contracts Cover Semantics, Not Merely Types

A schema registry can prove that `bytes_sent` is an integer while missing that its unit changed from bytes to kibibytes. A data contract should specify:

- event grain and uniqueness key;
- field type, nullability, units, enum, and classification;
- identifier namespace and stability;
- occurrence and observation clock semantics;
- ordering and replay guarantees;
- maximum expected lateness and retention;
- producer compatibility policy and deprecation window;
- quality objectives, owner, and escalation path.

Compatibility has several forms. Backward compatibility lets a new reader consume old records. Forward compatibility lets an old reader tolerate new records. Full compatibility promises both, often through optional additions and stable meanings. Deleting a field, changing grain, reusing an enum, or altering units is a breaking change even if JSON still parses.

Consumers should test representative producer fixtures before rollout. Contract tests belong at organizational boundaries: cloud collector to platform, application team to security lake, agent runtime to evaluation service. They turn a Slack promise into an executable interface.

Idempotency and Deduplication Are Different Problems

At-least-once delivery is common because acknowledging only after durable write can replay a message after failure. An idempotent sink makes repeated processing of the same logical input converge to one effect. Deduplication determines whether two records are the same logical input.

Use a source-stable event ID when trustworthy. Otherwise construct a key from stable semantic fields and source coordinates, not arrival time. A content hash may collapse two legitimate identical actions, while `(topic, partition, offset)` distinguishes transport records but not a source event replayed through a new topic.

For a transformation version `\(v\)`, an output key such as

```
\[ k=H(source\_namespace\parallel source\_event\_id\parallel v) \]
```


supports idempotent upsert and intentional reprocessing under a new version. Preserve duplicate lineage rather than deleting evidence: canonical event, duplicate record locator, reason, and algorithm version.

Side effects require idempotency beyond storage. A detector retry must not page twice or revoke two unrelated sessions. Carry a decision or action idempotency key through queue, workflow, and target API. If the target lacks idempotency, introduce a transactional outbox and action ledger.

Lineage and Versioning Make Results Reconstructable

Lineage should answer: which source objects and offsets, contract versions, reference snapshots, code artifact, parameters, secrets identities, and runtime created this output? Table-level arrows are insufficient when a backfill touches only certain partitions.

Record run manifests containing immutable input ranges and hashes, image or environment digest, Git commit or source bundle digest, transformation and schema versions, output partition, row counts, quality results, start/end times, and actor. Model training additionally records feature definitions, labels, splits, and random seeds.

Version semantics separately from code. A refactor can change code without meaning; a threshold or identity rule can change meaning without schema. Name `pipeline_version`, `contract_version`, `mapping_version`, and `reference_snapshot` explicitly.

Backfills Are Controlled Historical Migrations

A backfill can flood alerts, overwrite original features, use current asset context for past events, and exhaust shared infrastructure. Treat it as a migration:

1. declare reason, affected interval, expected outputs, and semantic version;
2. freeze or identify point-in-time reference data;
3. write to a shadow destination;
4. compare counts, distributions, keys, and sampled records;
5. publish atomically or by partition with a manifest;
6. prevent historical decision products from triggering live side effects;
7. retain original outputs and supersession links.

Historical consistency means an event from March is enriched with what was valid in March, or with what was known at a declared March cutoff—not today's department and device posture. If point-in-time context is unavailable, label the backfill retrospective and do not compare it naively with original alerts.

Drift Has Several Owners

Schema drift changes fields or types. **Semantic drift** changes meaning without shape, such as a vendor redefining `success`. **Population drift** changes entities or traffic. **Reference drift** changes identity, asset, or threat context. **Pipeline drift** changes output due to code, library, configuration, or nondeterministic execution.

Monitor hard violations and statistical signals. Unknown enum rate, null rate by source, payload size, parse failures, uniqueness violations, join match rate, late-event rate, and distribution distance each reveal different faults. A stable global mean can hide one broken tenant, region, platform version, or privilege tier, so segment by operational failure domains.

Do not auto-reject every novel enum; attackers and new products both create novelty. Quarantine incompatible records, retain raw evidence, and make policy explicit. Drift alerts need an owner and playbook or they become another noisy feed.

Pipeline SLOs Express Decision Reliability

Availability alone is weak. A pipeline can be up and six hours late. Define service-level indicators such as:

- accepted completeness: expected source sequence or canary events durably landed;
- freshness: delay from source observation to queryable normalized record;
- correctness: fraction passing contract and invariant checks;
- reconciliation: provisional records matched to authoritative outcomes;
- reproducibility: sampled outputs regenerated from manifests;
- recovery: time to restore and safely replay after failure.

An SLO could require 99.9 percent of privileged cloud events queryable within five minutes per hour, no unaccounted source-sequence gaps, and 99 percent of quarantined records resolved within one business day. Error budgets govern change velocity. If freshness burns its budget, pause risky collector changes rather than merely reporting red.

Measure end-to-end from source to decision consumer. Broker lag can be zero while normalization is wrong. Synthetic canaries that traverse real paths provide stronger assurance than component heartbeats.

Secure the Machinery That Sees Everything

Pipelines aggregate credentials and evidence, making them high-value targets. Collectors should use workload identity and narrowly scoped read permissions. Writers should be append-only where possible; transformation roles should not administer sources. Separate production, backfill, and investigation privileges.

Keep secrets out of code, logs, manifests, and exception payloads. Use a secrets manager, short-lived credentials, rotation, access audit, and egress restrictions. Encrypt transport and storage; sign or hash manifests and landed objects when tamper evidence matters. Restrict quarantine because malformed events may contain the most sensitive payloads.

Threat-model poisoning, deletion, replay, timestamp manipulation, schema bombs, decompression bombs, identifier collisions, malicious file paths, and compromised build dependencies. Bound parsers, validate archive contents, enforce resource quotas, and pin verified dependencies. A “data job” executes attacker-influenced inputs.

Executable Lab: Watch Three Failures Surface

The lab implements contract validation, deterministic deduplication, enrichment, quarantine, and an SLO report. Its fixture contains a replay, a breaking schema version, a missing required field, an unknown identity, and a late event.

```
from datetime import datetime, timezone
import hashlib

NOW = datetime.fromisoformat("2026-08-20T12:00:00+00:00")
events = [
    {"id": "a1", "v": 1, "at": "2026-08-20T11:59:30+00:00", "subject": "u1", "action": "login"},
    {"id": "a1", "v": 1, "at": "2026-08-20T11:59:30+00:00", "subject": "u1", "action": "login"},
    {"id": "a2", "v": 2, "at": "2026-08-20T11:59:40+00:00", "subject": "u2", "action": "grant"},
    {"id": "a3", "v": 1, "at": "2026-08-20T11:58:00+00:00", "action": "download"},
    {"id": "a4", "v": 1, "at": "2026-08-20T11:59:50+00:00", "subject": "ghost", "action": "login"},
    {"id": "a5", "v": 1, "at": "2026-08-20T11:40:00+00:00", "subject": "u2", "action": "logout"},
]
directory = {"u1": "engineering", "u2": "finance"}
seen, accepted, quarantine = set(), [], []

for position, event in enumerate(events):
    if event.get("v") != 1:
        quarantine.append((position, event.get("id"), "unsupported_schema")); continue
    missing = [f for f in ("id", "at", "subject", "action") if f not in event]
    if missing:
        quarantine.append((position, event.get("id"), "missing:" + ",".join(missing)));
        continue
    key = hashlib.sha256(("cloud-audit|" + event["id"] + "|normalize-v1").encode()).hexdigest()
    if key in seen:
        quarantine.append((position, event["id"], "duplicate")); continue
    seen.add(key)
    if event["subject"] not in directory:
        quarantine.append((position, event["id"], "unresolved_identity")); continue
    lag = (NOW - datetime.fromisoformat(event["at"])).total_seconds()
    accepted.append((event["id"], directory[event["subject"]], int(lag)))

print(f"accepted={accepted}")
print(f"quarantine={quarantine}")
fresh = sum(lag <= 300 for _, _, lag in accepted)
print(f"freshness_slo={fresh}/{len(accepted)} ({fresh/len(accepted):.1%})")
print("release_gate=" + ("PASS" if not quarantine and fresh == len(accepted) else "BLOCK"))
```

Exact output:

```
accepted=[('a1', 'engineering', 30), ('a5', 'finance', 1200)]
quarantine=[(1, 'a1', 'duplicate'), (2, 'a2', 'unsupported_schema'), (3, 'a3',
'missing:subject'), (4, 'a4', 'unresolved_identity')]
freshness_slo=1/2 (50.0%)
release_gate=BLOCK
```

This deliberately blocks instead of returning only valid rows and declaring success. Production would distinguish expected duplicate replay from malformed data in its release policy, but both remain observable. Notice that `a5` is semantically valid and still violates freshness. Contract correctness and timeliness are independent.

Extend the lab by persisting source offsets and run manifests in SQLite, rerunning the fixture, and asserting that output state is unchanged. Then change `normalize-v1` to `normalize-v2`: a new output identity should coexist or explicitly supersede the first rather than silently mutate it.

Recovery Is Part of Normal Operation

When a pipeline fails, freeze destructive cleanup and side effects, identify the last trustworthy checkpoint, preserve raw input, and bound affected decisions. Communicate whether data is absent, late, duplicated, or semantically suspect—each demands different consumer behavior. Replay into shadow output, reconcile, publish through a versioned cutover, and close with a timeline and control change.

If alerts were missed, do not simply replay them into the live paging queue. Recompute historical candidates, prioritize still-actionable risk, mark them as recovery artifacts, and coordinate responder capacity. A technically correct replay can create an operational denial of service.

Recovery drills should include an adversarial dependency failure, not only a stopped worker. Revoke the enrichment service's credential, corrupt one reference snapshot in a test environment, replay an already consumed partition, and make the notification API time out after accepting a request. The drill passes when manifests isolate the affected outputs, action idempotency prevents duplicate response, raw evidence remains available, and operators can state which decisions are trustworthy. Time the exercise from fault injection to bounded impact, not merely process restart.

Retirement is another pipeline operation. Before removing a field, topic, table, or secret, enumerate consumers from lineage and access evidence, announce a deadline, observe use, and test the absence in staging. Orphaned consumers often fail silently because permissive readers turn missing fields into null. A successful deprecation ends with the producer contract, registry entry, transformations, dashboards, runbooks, and credentials agreeing that the interface no longer exists.

Sources and Further Reading

- Tyler Akidau et al., “The Dataflow Model,” *VLDB* 2015: <<https://doi.org/10.14778/2824032.2824076>>
- Martin Kleppmann, *Designing Data-Intensive Applications*, O'Reilly: <<https://dataintensive.net/>>
- Apache Kafka, delivery semantics: <<https://kafka.apache.org/documentation/#semantics>>

- OpenLineage specification: <<https://openlineage.io/docs/spec/>>
- NIST SP 800-204D, *Strategies for the Integration of Software Supply Chain Security*: <<https://doi.org/10.6028/NIST.SP800-204D>>
- NIST SP 800-207, *Zero Trust Architecture*: <<https://doi.org/10.6028/NIST.SP800-207>>
- Google, *Site Reliability Engineering*, service-level objectives: <<https://sre.google/sre-book/service-level-objectives/>>
- OWASP, *Secrets Management Cheat Sheet*: <https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html>

Interview Practicum

A SaaS company must detect privileged cloud changes in five minutes across 600 accounts. Provider audit is at-least-once and sometimes four hours late. Identity snapshots arrive nightly. The existing job joins against the current directory and pages directly from a stream consumer. Mid-interview, the panel says a regional outage requires a 36-hour replay and responders can absorb only 200 recovered alerts per day.

The prompt: **Redesign the pipeline and describe a safe backfill.** Strong candidates ask about source IDs, partition ordering, decision deadline, late-event distribution, identity history, target idempotency, raw retention, account onboarding, SLO ownership, and containment alternatives.

Worked Answer

Land provider records immutably before normalization. Validate source sequence and contract, deduplicate by provider event identity plus namespace, and retain duplicate lineage. Stream privileged actions into a point-in-time identity view, labeling unresolved enrichment instead of filtering it. Emit a provisional decision record to a transactional outbox; a separate action service uses the decision ID as idempotency key.

Maintain Type 2 identity snapshots through change-data capture or diffed nightly versions. A batch reconciler processes late events and compares provider records with normalized outputs. The freshness SLO is measured per account and region from provider observation to queryable decision evidence. Canary role changes verify the complete path.

For the outage, define source offsets and time interval, pin the historical identity snapshot and transformation image, write a shadow result, compare counts and samples, and publish supersession lineage. Do not page all historical matches. Rank still-relevant actions, cap release at responder capacity, group duplicates, and route expired cases to a retrospective exposure report. Keep live processing isolated from backfill resources.

Follow-Up Questions and Answers

Can you promise exactly once? Promise idempotent logical effects under documented replay assumptions, not magical transport. Define identity at source, transformation, decision, and action layers.

What if current identity is all you have? Mark enrichment retrospective, retain the unresolved source principal, and avoid claims about what the original detector could have known.

When is Lambda-style duplicate logic justified? Only when latency and authoritative recomputation require different engines and shared semantics cannot meet both. Minimize divergence with shared contracts and equivalence tests.

Most important security boundary? Least-privilege workload identities and separation of collection, transformation, backfill, and response authority.

Thirty-second answer? Durable raw landing, semantic contracts, deterministic idempotency, point-in-time enrichment, end-to-end SLOs, manifest-driven backfills, and capacity-aware recovery.

Key Takeaways

- Architecture follows the decision clock and source behavior.
- Contracts specify grain, meaning, clocks, units, replay, privacy, and ownership—not only types.
- Deduplication identifies logical sameness; idempotency makes retries converge safely.
- Lineage and versioning must reconstruct partitions and decisions, not just tables.
- Backfills are controlled migrations with historical context and suppressed live side effects.
- Reliability includes freshness, completeness, correctness, reconciliation, reproducibility, and safe recovery.

Chapter 16: Security Data Quality as a Control

On Monday, the executive chart showed a 63 percent decline in credential attacks. On Tuesday, incident response found that the identity provider had renamed `failure_reason` to `outcome_detail`. The parser accepted every record, populated the old field with null, and the dashboard counted only rows with a recognized failure reason. No component was down. Every green heartbeat was accurate. The conclusion was false.

Data quality is a security control because decisions depend on evidence remaining fit for a declared use. It is not an aesthetic cleanup stage and cannot be reduced to one score. A control can fail through false records, missing records, late records, biased scope, duplicate observations, or loss of semantic continuity. Attackers can cause several of those failures deliberately.

Quality Begins With a Decision

Quality is always relative to use. A five-minute delay is harmless for a quarterly risk review and unacceptable for session revocation. Country-level location can support regional prevalence but not impossible-travel detection. A dataset can be excellent for billing and unsafe for attributing an intrusion.

Define the unit, decision, horizon, and loss from error before selecting checks. For privileged authentication, the quality contract might say: all in-scope tenants and regions; one row per provider authentication attempt; identity issuer and immutable subject present; result semantics mapped by version; 99.9 percent available within three minutes; source-sequence gaps detected within five minutes; corrections retained.

Quality control has three loops. **Prevention** uses contracts, constrained producers, and compatibility tests. **Detection** uses invariants, canaries, distributions, and reconciliations. **Response** limits bad decisions, restores evidence, and learns. A scorecard without response authority is a report, not a control.

Six Dimensions That Refuse to Collapse

Completeness asks whether required records and fields exist for the intended population and interval. Measure expected versus observed using independent denominators: enrolled assets, configured accounts, source sequence, transaction counts, or synthetic canaries. Observed volume cannot prove its own completeness.

Accuracy asks whether values correspond to the source or world state. Validate through authoritative reconciliation, sampled raw evidence, postconditions, or controlled actions. Syntactic validity is not accuracy: a well-formed `success=true` can be emitted before transaction rollback.

Consistency asks whether representations agree across records, time, and systems. Examples include one event ID mapping to one payload, role identifiers resolving to the same tenant, bytes using stable units, and normalized outcomes preserving source distinctions. Disagreement may reveal either a defect or a real race; retain both claims while resolving it.

Timeliness measures whether evidence arrives before the decision deadline. Use distributions, not averages: p50, p95, p99, maximum, and fraction within objective. Separate source delay from ingestion and processing delay.

Uniqueness concerns logical observations, not row bytes. Retries can create duplicate transport records; aggregators can also summarize overlapping windows that are correlated rather than duplicated. Declare the key and deduplication horizon.

Representativeness asks whether observed data covers the target population without distortion. A detector trained only on managed laptops may perform poorly on servers, contractors, mobile devices, or newly acquired subsidiaries. Report coverage and performance by relevant strata.

A seventh useful property is **semantic continuity**: can a measure retain meaning across producer and mapping versions? Stable row count with changed enum meaning is a quality break invisible to type checks.

Missingness Is a Causal Clue

Let $(R=1)$ mean a value or record is observed, (X) observed variables, and (Y) the missing value or outcome.

Missing completely at random (MCAR) assumes

$$P(R=1 \mid X, Y) = P(R=1).$$

A random storage-sector failure approximates MCAR, though true MCAR is uncommon operationally. Complete-case analysis may remain unbiased for some estimates but loses precision.

Missing at random (MAR) assumes

$$P(R=1 \mid X, Y) = P(R=1 \mid X).$$

Telemetry may be missing more often on a known operating-system version. Conditional on version and region, missingness need not depend on the unseen event. Weighting or imputation can help if the observed predictors are adequate.

Missing not at random (MNAR) means observation depends on the missing value itself after conditioning on available variables. A sensor may drop the largest payloads; users may evade logging specifically during policy violations. The missing values are systematically different. Ordinary imputation cannot identify the distribution without additional assumptions or evidence.

Security adds **attacker-induced missingness**. An adversary disables a sensor, routes around a proxy, exhausts a collector, tampers with time, deletes audit configuration, or crafts input that crashes parsing. This is MNAR with an adaptive cause. Treat missingness patterns as detections: a process followed by sensor silence, one host deviating from peers, audit disablement before a role change, or a sequence gap coinciding with suspicious activity.

Do not replace absent evidence with “benign.” Three-valued logic—observed true, observed false, unknown—is safer. Models should receive missingness indicators and coverage context, while policies can fail closed, fail open, step up, or defer based on consequence and availability requirements.

Sensor Health Must Exercise the Evidence Path

Health exists at several layers:

- process: collector is running;
- source: it is subscribed to intended tenants, accounts, interfaces, or hosts;
- transport: sequence and acknowledgment progress;
- semantic: events parse and required distributions remain plausible;
- end-to-end: a known action becomes queryable and usable by the control.

A heartbeat proves only its layer. Synthetic canaries provide stronger evidence. Create a harmless authentication failure, DNS query, file write, policy denial, or agent tool proposal with a unique marker. Require the raw event, normalized fact, enrichment, detector result, and dashboard increment within explicit times. Canaries should cover regions and important product versions without becoming predictable blind spots that are healthier than ordinary traffic.

Passive invariants complement canaries: every enrolled endpoint should report within its cadence; cloud accounts should appear in configuration and audit inventories; source sequences should be contiguous or explain gaps; event and byte volumes should relate to independent service usage. Reconcile through a source with different failure modes whenever possible.

Duplicate Alerts and Correlated Evidence

Duplicates inflate prevalence, analyst burden, and apparent confidence. Exact duplicate events share a logical source identity. Near-duplicates may be retries or repeated observations of one action. Duplicate alerts may arise when several rules detect the same incident. Correlated evidence is subtler: endpoint, proxy, and DNS records can all be legitimate observations but do not represent three independent attacks.

Model at least three layers:

1. canonical source event and its transport duplicates;
2. detection candidates linked to contributing events and rule versions;
3. incident or episode clusters linked to alerts with explicit clustering rationale.

Deduplicate only at the layer whose equivalence is known. Hashing entire JSON fails when ingestion metadata differs. Hashing too few semantic fields can collapse repeated password guesses. For alert grouping, preserve all alerts but route one episode to the analyst, retaining rule contributions and timestamps.

When combining evidence scores, independence assumptions matter. Multiplying likelihood ratios from sensors sharing a proxy or identity feed overstates confidence. Tag observations with common upstream dependencies and calibrate on episodes, not raw alert rows.

Coverage Debt Accumulates Interest

Coverage debt is the gap between the population or behavior a decision claims to cover and what its evidence actually represents. It arises from unenrolled assets, new regions, unsupported operating systems, disabled data-plane logs, acquisitions, shadow cloud accounts, short retention, missing agent trace spans, and privacy-driven omissions.

Maintain a debt register with affected decision, population denominator, uncovered scope, risk, compensating control, owner, age, and review date. Quantify weighted coverage where possible, but preserve the list of critical gaps. Ninety-nine percent endpoint coverage is misleading if the missing one percent contains all domain controllers.

Debt has interest: detection tuning learns the observed population, investigators build habits around available evidence, dashboards normalize the blind spot, and attackers discover it. Age and consequence should influence remediation priority. New products need an observability acceptance criterion before entering a control's claimed scope.

Scorecards Without the Comfort of One Number

A single weighted quality score permits compensation: excellent freshness can hide zero coverage in a critical region. Use a vector plus hard gates. A scorecard can show, by source and segment, completeness, p95 freshness, contract validity, duplicate rate, unresolved identity rate, canary success, semantic version, and outstanding critical debt.

Each metric needs numerator, independent denominator, window, segmentation, uncertainty, owner, objective, and action. Show unknown denominators explicitly. Apply hard gates such as “no critical population at zero coverage” or “no unresolved breaking semantic change.” Trends and incident annotations matter more than decorative red-amber-green states.

Sampling must respect clusters. Ten thousand events from one healthy host do not validate fleet accuracy. Sample across tenants, regions, device types, privilege tiers, and producer versions. For a proportion $\hat{p}$, binomial intervals describe sampling error only; they do not cover biased denominators or correlated missingness.

Data-Quality Incident Response

Declare an incident when evidence fitness can materially alter a security decision. Triage along two axes: which evidence is wrong and which decisions consumed it. Determine onset, scope, failure mode, attacker involvement, and last trustworthy interval.

Containment may pause automated action, label dashboards stale, fall back to independent sensors, increase approval, preserve raw data, and stop destructive compaction. Do not silently repair outputs first; preserve the faulty state and lineage needed to assess impact.

Eradication fixes producer, contract, mapping, or infrastructure. Recovery reprocesses into shadow outputs, compares against trusted samples, publishes corrections, and reviews missed or false decisions. Notify consumers with concrete semantics: “EU authentication failure counts were underreported 58–71 percent from 08:00–13:20 UTC,” not “data issue resolved.”

Post-incident work adds a preventive contract test, a detection capable of shortening time to discovery, and a response change. Track time to detect quality failure, time to bound affected decisions, and time to trustworthy recovery.

Executable Case Study: Did Attacks Fall or Did Logging Change?

The fixture contains daily login totals from three regions. On August 4, EU failures collapse while total logins remain stable and the null outcome rate rises. The script uses a pre-change failure-rate baseline, checks a binomial z-score for the observed drop, and then refuses the attack-reduction interpretation because semantic missingness coincides with it.

```
from math import sqrt

rows = [
    ("2026-08-01", "US", 1000, 100, 0), ("2026-08-01", "EU", 800, 80, 0), ("2026-08-01", "AP", 600, 60, 0),
    ("2026-08-02", "US", 1020, 102, 0), ("2026-08-02", "EU", 820, 82, 0), ("2026-08-02", "AP", 610, 61, 0),
    ("2026-08-03", "US", 980, 98, 0), ("2026-08-03", "EU", 790, 79, 0), ("2026-08-03", "AP", 620, 62, 0),
    ("2026-08-04", "US", 1010, 101, 0), ("2026-08-04", "EU", 810, 12, 70), ("2026-08-04", "AP", 615, 61, 0),
]

baseline = [r for r in rows if r[0] < "2026-08-04"]
p0 = sum(r[3] for r in baseline) / sum(r[2] - r[4] for r in baseline)

for day, region, total, failures, unknown in rows:
    if day != "2026-08-04":
        continue
    known = total - unknown
    rate = failures / known
    se = sqrt(p0 * (1 - p0) / known)
    z = (rate - p0) / se
    status = "LOGGING_CHANGE" if unknown / total > 0.02 and z < -3 else "NO_BREAK"
    print(f"{region} rate={rate:.3f} unknown={unknown/total:.3f} z={z:.1f} {status}")

before = sum(r[3] for r in rows if r[0] == "2026-08-03")
after = sum(r[3] for r in rows if r[0] == "2026-08-04")
print(f"naive_failure_change={({after-before})/before:.1%}")
print("executive_claim=WITHHOLD")
```

Exact output:

```

US rate=0.100 unknown=0.000 z=0.0 NO_BREAK
EU rate=0.016 unknown=0.086 z=-7.6 LOGGING_CHANGE
AP rate=0.099 unknown=0.000 z=-0.1 NO_BREAK
naive_failure_change=-27.2%
executive_claim=WITHHOLD

```

The z-score says the EU change is far beyond ordinary binomial fluctuation under the baseline. It does not identify cause. The simultaneous 8.6 percent unknown rate localizes a semantic collection break, so the proper action is to withhold the attack trend, investigate the mapping, and backfill. A real system would use overdispersion-aware baselines, day-of-week effects, multiple-comparison controls, change-point methods, and independent provider totals.

To make this a durable control, segment by producer version and region, alert on both recognized-outcome rate and distribution change, retain raw enums, and deploy a canary failure whose normalized result is asserted. Then a new enum becomes an explicit incompatibility rather than a fake security improvement.

Adversarial Quality Tests

Test the quality system itself. Disable one canary route while leaving heartbeats green. Replay a valid event with changed whitespace. Send an oversized enum, a future timestamp, a source-sequence regression, and a decompression bomb to a safe test environment. Remove one region from an inventory denominator. Verify that controls quarantine safely, preserve evidence, notify the right owner, and do not convert unknown into normal.

Also test governance pressure. What happens if an executive dashboard depends on the broken metric? Can the data owner mark it invalid without permission from the consumer? Can automated containment degrade to approval-only mode? Technical detection without authority to stop false conclusions is incomplete.

Sources and Further Reading

- Roderick J. A. Little and Donald B. Rubin, *Statistical Analysis with Missing Data*: <<https://www.wiley.com/en-us/Statistical+Analysis+with+Missing+Data%2C+3rd+Edition-p-9780470526798>>
- NIST SP 800-137, *Information Security Continuous Monitoring*: <<https://doi.org/10.6028/NIST.SP800-137>>
- NIST SP 800-92, *Guide to Computer Security Log Management*: <<https://doi.org/10.6028/NIST.SP800-92>>
- ISO/IEC 25012:2008, *Data Quality Model*: <<https://www.iso.org/standard/35736.html>>
- DAMA International, *DAMA-DMBOK Data Management Body of Knowledge*: <<https://www.dama.org/cpages/body-of-knowledge>>
- Google, *Site Reliability Engineering*, monitoring distributed systems: <<https://sre.google/sre-book/monitoring-distributed-systems/>>
- MITRE ATT&CK, Impair Defenses technique T1562: <<https://attack.mitre.org/techniques/T1562/>>

- CISA, *Best Practices for Event Logging and Threat Detection*: <<https://www.cisa.gov/resources-tools/resources/best-practices-event-logging-and-threat-detection>>

Interview Practicum

A bank's board dashboard reports a 40 percent decline in account-takeover attempts after an identity-platform upgrade. The fraud team simultaneously reports more customer complaints. Raw payload retention is seven days, the vendor says event volume is normal, and one subsidiary has no asset inventory. Halfway through the interview, the panel reveals that attackers know how to trigger an undocumented parser exception.

The prompt: **Decide whether the decline is real, contain risk, and design a quality control.** Strong candidates ask for metric definitions, denominators, producer versions, null and unknown rates, regional splits, source sequences, independent transaction totals, complaint lag, parser quarantine, deployment timing, and automated decisions consuming the metric.

Worked Answer

Immediately label the board metric unverified and identify downstream thresholds or staffing decisions. Preserve remaining raw payloads, parser exceptions, deployment manifests, and source offsets. Compare totals and outcome distributions by version, subsidiary, region, and channel. Reconcile identity events against independent login transactions and customer complaints. Run controlled authentication canaries through old and new paths.

Treat parser exceptions as potential attacker-induced missingness. Isolate the parser, bound inputs, quarantine rather than drop, and search for exception timing near suspicious sessions. If takeover controls depend on the feed, increase step-up authentication or move to an independent signal while respecting availability impact.

The durable scorecard shows independent completeness, recognized-outcome fraction, freshness quantiles, duplicate rate, canary success, exception rate, and inventory coverage by critical segment. A hard gate prevents publication when a breaking enum or critical zero-coverage population exists. Recovery backfills from raw evidence, versions corrected outputs, reviews affected cases, and communicates quantified undercount.

Follow-Up Questions and Answers

Could customer complaints prove attacks increased? No. They are delayed and selected, but they are an independent signal that challenges the dashboard and helps bound impact.

Would you impute the missing failures? Not for case decisions. For aggregate sensitivity analysis, estimate ranges under explicit MNAR assumptions and show how conclusions change.

Why not average the quality dimensions? Compensation hides catastrophic gaps. Use vectors and hard gates for non-negotiable properties.

How does attacker-induced missingness change response? Treat the quality break as a security incident, preserve tamper evidence, hunt around silence, and assume missing events may be higher risk than observed ones.

Thirty-second summary? Challenge the denominator, segment the change, reconcile independently, preserve unknown, contain dependent decisions, repair with versioned backfill, and install an end-to-end canary plus hard publication gate.

Key Takeaways

- Data quality is fitness for a security decision and therefore part of the control.
- Completeness, accuracy, consistency, timeliness, uniqueness, representativeness, and semantic continuity expose different failures.
- MCAR, MAR, MNAR, and attacker-induced missingness demand different assumptions and responses.
- Heartbeats verify components; canaries and reconciliation verify evidence paths.
- Duplicate rows, duplicate alerts, and correlated witnesses must be handled at different semantic layers.
- Quality incidents require containment of bad decisions, preserved evidence, versioned recovery, and explicit consumer communication.

Chapter 17: Privacy-Preserving Security Analytics

Security teams can collect an attribute long before they can justify collecting it. Insider-risk systems make that gap dangerous: authentication, repository, messaging, location, and organizational data may reveal an attack, but the same data can expose health, labor activity, relationships, or ordinary dissent. The engineering task is not to “anonymize the logs.” It is to produce enough evidence for a defined security decision while limiting what the system can learn, who can learn it, and how long that knowledge persists.

Privacy is not the absence of identity. An investigation may eventually require attribution. Privacy-preserving analytics delays and constrains attribution until evidence and authorization justify it. A mature design separates population analysis from person-level investigation, makes transitions explicit, and assumes that technically authorized users can still misuse data.

Specify the Permitted Inference

“Detect insider threats” is too broad to constrain a system. Use a statement that identifies the inference, decision, and forbidden uses:

Estimate whether a managed identity performed an unusual sequence of bulk repository reads followed by an external transfer within 60 minutes, so an authorized investigator can decide whether to open a case. Do not infer productivity, sentiment, personal relationships, or protected activity.

This determines the unit, fields, horizon, retention, and escalation path. Message bodies are unnecessary if destination class and byte count answer the question. Exact GPS is unnecessary if the feature is “region changed faster than plausible travel.” Manager access is unnecessary when only a segregated investigations team can act.

Purpose limitation needs enforcement. Attach `purpose_id`, field policy, retention class, and allowed output grain to each dataset version. A gateway should reject projections and joins outside that contract. Documentation alone decays into folklore.

Separate Feature and Case Zones

The useful boundary is between a **feature zone**, where direct identity is unavailable, and a **case zone**, where re-identification is approval-gated. Tokenization is not anonymization: stable tokens remain linkable and may be guessed. Generate keyed pseudonyms in a separately administered service, scope keys to a purpose, and never expose token and source identity together in the analytical warehouse.

```
CREATE TABLE privacy_safe_identity_window (
  purpose_id TEXT NOT NULL,
  subject_token TEXT NOT NULL,
  window_start_utc TEXT NOT NULL,
  auth_region_count INTEGER NOT NULL CHECK (auth_region_count >= 0),
```

```

failed_auth_count INTEGER NOT NULL CHECK (failed_auth_count >= 0),
repo_read_bytes INTEGER NOT NULL CHECK (repo_read_bytes >= 0),
external_write_bytes INTEGER NOT NULL CHECK (external_write_bytes >= 0),
privileged_action_count INTEGER NOT NULL CHECK (privileged_action_count >= 0),
peer_group_size INTEGER NOT NULL CHECK (peer_group_size >= 0),
source_coverage REAL NOT NULL CHECK (source_coverage BETWEEN 0 AND 1),
expires_at_utc TEXT NOT NULL,
PRIMARY KEY (purpose_id, subject_token, window_start_utc)
);

CREATE TABLE reidentification_audit (
    request_id TEXT PRIMARY KEY,
    subject_token TEXT NOT NULL,
    case_id TEXT NOT NULL,
    approver_id TEXT NOT NULL,
    legal_basis TEXT NOT NULL,
    requested_at_utc TEXT NOT NULL,
    decision TEXT NOT NULL CHECK (decision IN ('approved','denied'))
);

```

The feature table excludes URLs, filenames, message text, exact location, and manager ratings. `source_coverage` prevents missing telemetry from looking normal. `peer_group_size` supports a release rule that blocks comparisons small enough to identify a person by elimination.

Derived data is not automatically safer than raw data. A year of hourly stable-token features reconstructs schedules and absences. Set retention from the permitted inference’s lookback, not storage cost. Delete raw content after local derivation when investigations do not require it; when they do, keep it in a separate, shorter-lived evidence store.

Privacy Risk Concerns Processing, Not Just Fields

NISTIR 8062 frames privacy risk as problems people experience because systems process data. Consider three operations on one authentication record:

1. Count failures across 50,000 identities for sensor health.
2. Rank named employees by after-hours activity for managers.
3. Reveal an identity after corroborated exfiltration signals and case approval.

The first can use aggregate output. The second creates surveillance with weak security justification. The third may be proportionate if thresholds, access, review, and retention are controlled. The bytes are similar; consequences are not.

NIST SP 800-188 likewise treats de-identification as risk management, not a one-time transform. Removing names does not defeat linkage through rare roles, timestamps, and unique behavior. Evaluate whether an attacker can determine that a known person is present, identify anyone, or link many records. Security telemetry is unusually linkable because time and device patterns are quasi-identifiers.

Differential Privacy for Population Releases

Differential privacy (DP) is appropriate for repeated aggregate releases to consumers who should not learn whether one person contributed. A randomized mechanism $\mathcal{M}$ is (ϵ, δ) -differentially private if, for neighboring datasets (D, D') differing in one person,

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\epsilon} \Pr[\mathcal{M}(D') \in S] + \delta.$$

For a person count with contribution bounded to one, sensitivity is 1. The Laplace mechanism releases

$$\tilde{c} = c + \text{Laplace}(0, 1/\epsilon).$$

Contribution bounding is the hard part. Counting events lets one active identity change a result by thousands; noise calibrated to 1 would make a false privacy claim. Define adjacency at the person level and clip before aggregation.

DP is usually wrong for deciding whether to investigate a person: noise can hide evidence or create an allegation. It is useful for prevalence trends, equity audits, product telemetry, and cohort-level threshold tuning. Budgets compose. Under basic composition, ten releases at $(\epsilon=0.2)$ have an upper bound of (2.0) , not (0.2) . Maintain a ledger by population, statistic, and consumer; correlated filters and averaging attacks make “noise on every dashboard query” unsafe.

Minimum group size and DP solve different problems. Suppression frustrates direct small-cell disclosure but can be defeated with differencing queries. DP limits the influence of membership on output. Neither makes an inappropriate statistic appropriate.

Safe Lab: Bounded Cohort Statistics

The fixture is synthetic. Its seeded random generator makes the assertion reproducible; production DP must use an appropriate random source and reviewed accountant.

```
from collections import defaultdict
import math
import random

events = [
    ("u1", "engineering", 8),
    ("u2", "engineering", 1),
    ("u3", "engineering", 0),
    ("u4", "finance", 3),
    ("u5", "finance", 0),
]

def bounded_count(rows, threshold=2, minimum_group=3):
    groups = defaultdict(list)
    for subject, group, transfers in rows:
        groups[group].append((subject, int(transfers >= threshold)))
    return {
```

```

        group: sum(flag for _, flag in members)
        for group, members in groups.items()
        if len(members) >= minimum_group
    }

def laplace_noise(epsilon, rng):
    u = rng.random() - 0.5
    return -math.copysign(math.log1p(-2 * abs(u)), u) / epsilon

exact = bounded_count(events)
assert exact == {"engineering": 1} # finance suppressed at k=2
rng = random.Random(17)           # deterministic test only
released = {k: round(v + laplace_noise(0.5, rng), 3) for k, v in exact.items()}
assert released == {"engineering": 0.910}
print(released)

```

Expected output:

```
{'engineering': 0.91}
```

One identity contributes at most one to a cohort, establishing sensitivity. The group rule prevents a two-person finance slice. Noise limits membership inference from the released engineering count. The assertion is a software test, not evidence that $\epsilon=0.5$ is the right policy; that choice depends on accuracy needs, cumulative releases, and acceptable privacy risk.

Secure Computation Has a Narrow Job

Encryption at rest protects media, not an analyst authorized to query plaintext. Private set intersection can identify overlap between two organizations without exchanging complete lists. Secure multiparty computation can calculate a joint statistic while parties retain inputs, at significant protocol and operational cost. Trusted execution environments reduce exposure to host operators but introduce hardware trust, side channels, attestation, and availability risk. Federated learning keeps raw records local, but updates can leak data unless paired with clipping, secure aggregation, and sometimes DP.

These mechanisms change who sees inputs; they do not legitimize an inference. If two business units should not jointly infer employee conduct, a cryptographic protocol that enables the inference does not make it proportionate.

Proportionality, Bias, and Contestability

Behavioral baselines encode unequal work arrangements. On-call engineers, responders, contractors, accessibility-tool users, travelers, and employees across time zones generate different patterns. Overall calibration can coexist with concentrated scrutiny.

Audit alert, escalation, and re-identification rates by operationally meaningful cohort after conditioning on exposure and role. Where lawful, protected attributes may be used in a segregated audit environment;

removing them from all data prevents measurement while proxies remain. Do not expose these audit attributes to investigators.

An adverse process needs contestability. Preserve evidence, feature, policy, and reviewer versions. Provide correction for wrong identity mappings and stale role data. Do not reveal evasion-enabling detection logic, but distinguish “case opened” from “misconduct established.” A risk indicator is not a verdict.

Production Failure Analysis

Universal token. One pseudonym used across security, HR, and productivity becomes a universal employee key. Scope tokens by purpose and prohibit cross-zone joins.

Rare-slice disclosure. A dashboard filters to one office, role, and hour. Apply cohort thresholds after all filters, and prevent differencing across overlapping queries.

Shadow copies. CSV exports enter tickets and notebooks, bypassing retention. Use controlled case workspaces, watermark exports, scan for copies, and expire access.

Function creep. A feature approved for compromise detection is reused for performance management. Purpose-bound authorization must reject the query pending a new review.

Premature attribution. Names shown during triage create confirmation bias. Keep pseudonyms until independent evidence meets a documented threshold.

Suppression blind spot. Attackers may force activity into suppressed cohorts. Record when evidence is withheld, retain tightly controlled forensic paths, and red-team the privacy rules.

False DP assurance. An event-level sensitivity calculation, untracked composition, or deterministic production noise invalidates the guarantee. Treat privacy accounting as a tested production subsystem.

Governance That Can Say No

Approval should include Security, Privacy or Legal, employee representation where appropriate, and an owner outside the investigation chain. Review the permitted inference, necessity of each source, model limitations, access, retention, disparity analysis, contestability, and sunset date. High-risk programs need periodic reauthorization based on demonstrated security value and measured privacy costs.

Audit logs must themselves be protected from investigators who might erase evidence of misuse. Monitor bulk browsing, repeated re-identification, out-of-case access, unusual cohort queries, and exports. A privacy control without misuse detection is merely a policy statement.

Test the Privacy Boundary Like a Security Boundary

A privacy review based only on the intended interface misses inference attacks. Build a synthetic “privacy red team” corpus containing rare roles, distinctive schedules, job changes, and planted canary identities. Give testers the same auxiliary information a realistic insider could possess—an organizational chart, public

conference attendance, or a known on-call rota—and ask them to link records. Measure successful linkage, membership inference, attribute inference, and the number of queries required. Never use real employees as surprise test subjects.

Test differencing explicitly. If a consumer can query “engineering” and “engineering excluding team X,” subtraction may reveal the suppressed team. Query controls must reason over history, not only the current result. For stable pseudonyms, test whether frequency patterns survive key rotation and whether an operator with access to two epochs can bridge them through unchanged quasi-identifiers.

Privacy telemetry needs service objectives of its own: percentage of exports with an active purpose, deletion completion by retention deadline, denied out-of-purpose queries, re-identification approval latency, orphaned case access, and privacy-budget consumption. A high denial count may indicate healthy enforcement or a confusing data product; investigate intent rather than rewarding the number.

Incident response must cover privacy compromise without destroying security evidence. Freeze affected releases and credentials, preserve immutable audit records, estimate which people and attributes were inferable, notify the appropriate governance owners, and rotate purpose keys when doing so will actually break linkage. Deleting the audit trail to “remove personal data” can obstruct accountability; retention rules should distinguish investigative evidence about misuse from the misused analytical data.

Finally, plan termination before launch. A sunset procedure should revoke queries, expire derived tables and caches, destroy or escrow token keys according to legal requirements, close case-zone grants, and verify downstream deletion. A program that cannot be dismantled cleanly is likely to become permanent surveillance infrastructure regardless of its original intent.

Procurement belongs inside this boundary. A vendor that receives telemetry for “service improvement” may train unrelated models or retain diagnostic copies after the contract ends. Require subprocessor disclosure, region and deletion guarantees, purpose-restricted use, incident notification, auditable access, and a tested export-and-delete procedure. Synthetic evaluation data should be the default during integration; production records should arrive only after controls are verified.

Sources and Further Reading

- **NIST Privacy Framework 1.0.** Its Identify-P, Govern-P, Control-P, Communicate-P, and Protect-P functions connect privacy outcomes to enterprise risk. <<https://www.nist.gov/privacy-framework/privacy-framework>>
- **NIST SP 800-188: De-Identifying Government Datasets.** Its release-model and re-identification analysis applies directly to linkable telemetry. <<https://csrc.nist.gov/pubs/sp/800/188/final>>
- **NISTIR 8062: An Introduction to Privacy Engineering and Risk Management.** Its problematic-data-action framing motivates analysis of processing consequences. <<https://csrc.nist.gov/pubs/ir/8062/final>>
- **Dwork and Roth, The Algorithmic Foundations of Differential Privacy.** This is the mathematical basis for adjacency, sensitivity, mechanisms, and composition. <<https://www.cis.upenn.edu/~aaroht/Papers/privacybook.pdf>>

- **NIST Privacy Risk Assessment Methodology.** PRAM connects data actions, privacy problems, likelihood, impact, and mitigations. <<https://www.nist.gov/privacy-framework/nist-pram>>

Interview Practicum

Scenario: Insider Risk Without a Panopticon.

Prompt

A global software company proposes ingesting message bodies, browser history, badge events, repository access, USB activity, and performance data to detect source-code exfiltration. Design a safer system.

Clarifying Questions

Ask what action the system may take; whether it detects, ranks, or proves misconduct; what the plausible attack sequence is; which sources are lawful by jurisdiction; who can re-identify; how incorrect data is challenged; and how many cases investigators can handle.

Worked Answer

Use a pseudonymous identity-hour and a narrow sequence: unusual repository volume followed by external write, conditioned on sensor coverage. Exclude content, browsing, badges, and performance ratings until necessity is demonstrated. Derive bounded features regionally, discard raw content, and send purpose-scoped tokens to the feature zone.

Triage exposes evidence categories, not names. Two-person case approval gates re-identification and expires automatically. Evaluate recall using synthetic and red-team sequences, precision through stratified review, disparities by work pattern, and the number of people exposed at each stage. Report false allegations and correction time alongside detections.

Follow-Up Questions and Answers

Constraint Change

Legal counsel then prohibits stable person-level tokens for EU employees, but responders must correlate events within a day.

Generate daily purpose-scoped tokens inside the region. Compute sequences regionally and export only threshold-crossing alert packets with feature values, coverage, policy version, and regional case reference. Identity remains local until regional approval. Cross-day analytics use cohort statistics. Quantify the recall lost from removing longitudinal identity before claiming equivalence.

Weak Answers and Probes

“Hash IDs” ignores linkage and dictionary attacks. “Encrypt everything” does not control authorized misuse. “Use DP” is mismatched to person-level investigation. “Remove protected fields” prevents disparity auditing while proxies remain.

Stop the program if less intrusive sources perform equivalently, re-identification bypasses approval, secondary use cannot be blocked, or adverse impact cannot be justified by exposure. In thirty seconds: minimize to threat-justified features, keep identity hidden through triage, gate attribution, and measure privacy harm as a first-class outcome.

Key Takeaways

- Begin with a permitted inference and explicit forbidden uses.
- Tokenization preserves linkability; it is a boundary control, not anonymity.
- Differential privacy requires person-level contribution bounds and composition accounting.
- Separate cohort analytics, pseudonymous triage, and attributed investigation.
- Measure identities exposed, re-identifications, false allegations, and misuse—not only detection accuracy.
- Privacy failure includes unjustified inference and function creep even when no database is breached.

Chapter 18: Ground Truth Is a Process, Not a Column

A phishing verdict does not arrive from nature. It is assembled from an artifact, evidence visible at review time, an analyst's policy interpretation, later incident facts, and sometimes remediation behavior. Two competent analysts can disagree because “malicious” conflates distinct questions: Was the sender deceptive? Did the message carry a harmful capability? Would it work in this environment? Was the sender account compromised? Did the recipient act?

Treating a ticket's final value as immutable truth hides these distinctions and trains models on workflow artifacts. Ground truth should be represented as versioned claims with provenance, uncertainty, and a label policy. The goal is not zero disagreement. It is to know which disagreement is error, which reveals an underspecified taxonomy, and which expresses irreducible ambiguity.

Define Claims Before Labels

A useful phishing ontology separates observable claims:

- `sender_control` : legitimate, spoofed, compromised, unknown;
- `intent` : benign, credential theft, malware delivery, fraud, unknown;
- `capability` : inert, live link, executable, exploit, unknown;
- `recipient_exposure` : delivered, quarantined, opened, clicked, credentials entered;
- `case_disposition` : benign, policy violation, attempted attack, confirmed incident;
- `confidence` : a probability or behaviorally anchored ordered scale.

A binary target can be derived for a particular decision. A quarantine model might define positive as credible malicious intent plus active capability. An incident model requires compromise evidence. Reusing one `malicious` column for both creates semantic error.

The guide must define an evidence cutoff. If historical labels include a compromise discovered weeks later, evaluation at delivery time contains hindsight. Record what was knowable, not simply what became known.

Store Assertions, Not Overwrites

```
CREATE TABLE label_assertion (
  assertion_id TEXT PRIMARY KEY,
  object_id TEXT NOT NULL,
  task_id TEXT NOT NULL,
  taxonomy_version TEXT NOT NULL,
  reviewer_id TEXT NOT NULL,
  reviewer_role TEXT NOT NULL,
  label_value TEXT NOT NULL,
```

```

    confidence REAL CHECK (confidence BETWEEN 0 AND 1),
    evidence_cutoff TEXT NOT NULL,
    created_at TEXT NOT NULL,
    supersedes_id TEXT,
    rationale_code TEXT NOT NULL,
    FOREIGN KEY (supersedes_id) REFERENCES label_assertion(assertion_id)
);

CREATE TABLE evidence_item (
    evidence_id TEXT PRIMARY KEY,
    object_id TEXT NOT NULL,
    observed_at TEXT NOT NULL,
    evidence_type TEXT NOT NULL,
    source_system TEXT NOT NULL,
    payload_digest TEXT NOT NULL,
    available_at TEXT NOT NULL
);

```

Never update a verdict in place. Append an assertion that supersedes it. This permits point-in-time reconstruction and separates correction from a taxonomy change. Store evidence by digest and controlled reference rather than duplicating dangerous attachments into the labeling database.

Agreement Is a Diagnostic

Raw agreement is inflated by a dominant class. For two reviewers, Cohen's kappa is

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

where p_o is observed agreement and p_e is expected agreement from reviewer marginals. With rare positives, kappa may be low despite high raw agreement. Report the confusion table and positive- and negative-class agreement as well.

For multiple reviewers, missing assignments, or ordinal outcomes, Krippendorff's alpha is more flexible:

$$\alpha = 1 - \frac{D_o}{D_e}.$$

Its disagreement distance must match the scale: nominal labels use equality, while ordered confidence should penalize distant ratings more.

Agreement is not correctness. Analysts can agree perfectly on a bad policy. Slice disagreement by campaign, language, attachment, evidence availability, queue age, and reviewer pair. Disputes concentrated around compromised legitimate senders indicate a taxonomy gap rather than careless work.

Adjudication Develops Policy

Do not send every conflict to a senior analyst and declare the result gold. Record why it occurred:

1. clerical error;
2. missed evidence;

3. ambiguous artifact;
4. taxonomy gap;
5. policy conflict;
6. evidence unavailable at initial review.

Only the first two imply simple correction. Taxonomy gaps require a guide revision and analysis of which historical labels change. Preserve pre-adjudication votes; otherwise agreement looks artificially high and reviewer error models cannot be estimated.

Gold sets need expiration. Maintain a stable anchor subset for longitudinal comparison and a rotating challenge subset for current techniques. Keep them out of training, restrict access, and use equivalent variants so reviewers and models cannot memorize them.

Positive–Unlabeled Learning

Confirmed attacks may be reliable positives, while “negative” means “not investigated.” Let $(s=1)$ mean labeled positive and $(y=1)$ truly positive. Under selected-completely-at-random labeling among positives,

$$c = P(s=1 \mid y=1), \quad P(y=1 \mid x) = \frac{P(s=1 \mid x)}{c}.$$

Elkan and Noto estimate (c) using held-out known positives. The assumption is demanding. Obvious and severe attacks are more likely to be reviewed, so selection depends on features. PU learning is useful as sensitivity analysis, not absolution. Document selection and review a probability sample of unlabeled items. Calling all unreviewed events negative is usually worse, but the correction is only credible to the extent its selection assumptions are.

Weak Supervision

Labeling functions might encode known credential domains, DMARC failure with display-name impersonation, sandbox detonation, or an analyst escalation. Snorkel’s key contribution is to model differing accuracy and dependence rather than use majority vote.

Security rules are correlated. Blocklists may share an upstream feed; several attachment rules may all detect macros. Assuming independence yields overconfident labels. Group rules by provenance, encode known dependencies, and validate probabilistic labels on an expert set. A labeling function should abstain when its precondition is absent instead of emitting a negative.

Probabilistic labels preserve uncertainty during training, but the deployed decision still needs action-specific costs and thresholds. A 0.6 probability may justify sandboxing but not disabling an account.

Safe Lab: Reliability-Weighted Disagreement

This synthetic lab computes kappa and combines two conflicting votes using known sensitivity, specificity, and prevalence. Conditional independence is an assumption to test, not a fact.

```

items = {
    "m1": {"a": 1, "b": 1},
    "m2": {"a": 1, "b": 0},
    "m3": {"a": 0, "b": 0},
    "m4": {"a": 0, "b": 0},
    "m5": {"a": 1, "b": 1},
    "m6": {"a": 0, "b": 1},
}

def cohen_kappa(rows, left="a", right="b"):
    pairs = [(v[left], v[right]) for v in rows.values()]
    po = sum(x == y for x, y in pairs) / len(pairs)
    pa = sum(x for x, _ in pairs) / len(pairs)
    pb = sum(y for _, y in pairs) / len(pairs)
    pe = pa * pb + (1 - pa) * (1 - pb)
    return (po - pe) / (1 - pe)

def posterior_positive(votes, sensitivity, specificity, prevalence):
    pos, neg = prevalence, 1 - prevalence
    for reviewer, vote in votes.items():
        pos *= sensitivity[reviewer] if vote else 1 - sensitivity[reviewer]
        neg *= 1 - specificity[reviewer] if vote else specificity[reviewer]
    return pos / (pos + neg)

kappa = cohen_kappa(items)
assert round(kappa, 3) == 0.333
p = posterior_positive(
    {"a": 1, "b": 0},
    sensitivity={"a": 0.90, "b": 0.75},
    specificity={"a": 0.80, "b": 0.95},
    prevalence=0.10,
)
assert round(p, 3) == 0.116
print(f"kappa={kappa:.3f}, disputed_p={p:.3f}")

```

Expected output:

```
kappa=0.333, disputed_p=0.116
```

The disputed object is not automatically positive because reviewer A voted positive. Low prevalence and B's high specificity matter. The posterior is a prioritization aid, not adjudicated fact; its estimated reviewer rates and independence assumption must travel with it.

Sampling and Active Learning

Random review estimates population error but spends expertise on easy negatives. Uncertainty sampling improves boundaries but can repeatedly select outliers. A defensible batch combines:

- a probability sample for prevalence and quality estimates;
- uncertain cases for model improvement;

- disagreement cases for policy development;
- high-impact cases for safety;
- new temporal and campaign strata for drift.

Record inclusion probability. Metrics from an actively selected batch are biased unless weighted or evaluated on a separate probability sample. Attackers can manufacture uncertain artifacts to exhaust review capacity, so cap contributions by campaign, tenant, sender, or infrastructure cluster.

Delayed and Censored Outcomes

A message labeled benign today may be connected to an incident later. Model the state transition rather than overwrite history: `provisional_benign -> confirmed_attack` at a recorded time. Evaluation at 14 days should treat unresolved recent items as censored, not negative. Report label maturation curves: the fraction of provisional outcomes that change after 7, 30, and 90 days.

Remediation is evidence but not ground truth. A user resetting credentials may reflect caution rather than compromise; failure to remediate may reflect ownership confusion. Keep action, analyst conclusion, and incident outcome as separate assertions.

Estimating Reviewer Behavior Without Ranking People

The Dawid–Skene model gives a more principled alternative to selecting the most senior vote. For latent class (z_i) and reviewer confusion matrix $(\pi^{(r)}_{kl}) = P(y^{(r)} = l \mid z_i = k)$, expectation–maximization alternates between estimating each item’s class posterior and updating reviewer matrices from those soft assignments. The result can identify a reviewer who is highly specific but misses subtle attacks, or one who is sensitive but overcalls benign simulations.

Identifiability is not guaranteed. If all reviewers copy the same upstream tool, their votes do not provide independent evidence. If reviewers see disjoint queues, differences mix reviewer behavior with case mix. Include overlapping assignments, randomize presentation order, conceal other votes, and ensure enough positive anchors. Regularize estimates for reviewers with few cases; a raw 100 percent score from three examples is not credible.

Use reviewer models to improve routing and training, not to produce a simplistic employee leaderboard. Analysts may specialize legitimately. Share error examples and policy changes, give reviewers a correction mechanism, and evaluate whether the evidence interface caused mistakes. A reviewer confusion matrix is conditional on task, taxonomy, evidence, and time.

Label quality also has a cost frontier. Suppose an item receives one initial review, a second review when confidence is low, and panel adjudication only when the action would be consequential. Estimate marginal corrected errors per expert hour at each stage. More votes are not always better: correlated reviewers can add confidence without information. Allocate additional review where expected decision regret—the probability the label is wrong multiplied by the consequence of using it—is greatest.

When taxonomy versions change, create a migration map rather than silently translating old labels. Some categories split, some merge, and some become non-comparable. Re-label a stratified bridge sample under both versions to estimate discontinuity. Dashboards should mark the boundary; a sudden fall in “malicious” after separating policy violations from attacks is a semantic change, not an improvement.

Label-Lineage Tests

Treat label production like a data pipeline. Unit-test allowed taxonomy values and transition rules. Assert that every derived training label references a policy version and evidence cutoff; that no evidence timestamp exceeds the decision timestamp; that supersession chains are acyclic; and that objects in the final test period were absent from weak-supervision rule development. A daily reconciliation should flag assertions whose referenced evidence was deleted or whose reviewer account was not authorized for the task.

Statistical tests should detect process shifts: reviewer positive rate by queue, disagreement by artifact family, time-to-label, abstention rate, and the fraction resolved through adjudication. A shift is not automatically bad—an attack campaign can legitimately change prevalence—but it triggers investigation before retraining. Publish a label-data sheet describing intended uses, exclusions, selection, maturity, known blind spots, and the difference between analyst judgment and confirmed outcome. Models inherit these limitations even when their feature engineering is flawless. Label debt compounds quietly until production decisions expose it.

Production Failure Analysis

Closed loop. The model determines review; reviewed alerts become training labels; suppressed attacks remain unknown. Maintain exploration sampling.

Hindsight leakage. Later incident evidence enters an earlier target or feature. Enforce evidence cutoffs and temporal joins.

Reviewer drift. Policy changes without taxonomy versioning. Re-score anchors and estimate behavior by reviewer and version.

Adjudicator dominance. One senior opinion becomes truth. Preserve dissent and compare adjudication with later outcomes.

Queue bias. Campaign pressure changes review quality. Store workload, queue age, and duration as process metadata, never as predictive evidence of maliciousness.

Benchmark contamination. Memorable gold examples leak into development. Restrict, rotate, and hash-check where feasible.

Semantic collapse. One flag combines intent, capability, exposure, and impact. Preserve components and derive labels per decision.

Identity leakage. Reviewer identity or queue route becomes a model feature and predicts institutional behavior. Exclude it from artifact models while retaining it for process audits.

Sources and Further Reading

- Dawid and Skene, “Maximum Likelihood Estimation of Observer Error-Rates.” Their latent-class model jointly estimates item classes and observer confusion patterns. <<https://doi.org/10.2307/2346806>>
- Elkan and Noto, “Learning Classifiers from Only Positive and Unlabeled Data.” The selected-positive formulation is useful precisely because it makes review-selection assumptions visible. <<https://doi.org/10.1145/1553374.1553453>>
- Ratner et al., “Snorkel: Rapid Training Data Creation with Weak Supervision.” The work shows how to combine noisy, overlapping sources rather than canonize rule votes. <<https://www.vldb.org/pvldb/vol11/p269-ratner.pdf>>
- Krippendorff, “Computing Krippendorff’s Alpha-Reliability.” Alpha accommodates multiple raters, missing assignments, and scale-specific disagreement. <<https://doi.org/10.1080/19312458.2011.568376>>
- Sambasivan et al., “Data Cascades in High-Stakes AI.” Their account of upstream data failures explains how labeling shortcuts propagate into model and organizational failures. <<https://doi.org/10.1145/3442188.3445922>>

Interview Practicum

Scenario: Thirty-Five Percent Disagreement.

Prompt

Two analyst teams disagree on 35 percent of sampled phishing messages. Leadership wants one ground-truth column by Friday so a classifier can launch.

Clarifying Questions

Ask whether disagreement concerns intent, successful compromise, or quarantine action; how items were sampled; whether both teams saw identical evidence; whether teams specialize by queue; and what action the classifier automates. Determine whether 35 percent is overall or positive-class disagreement.

Worked Answer

Do not manufacture consensus. Preserve assertions, build a cross-tab, and run a blinded overlap study on a probability sample with identical evidence packets. Calculate class-specific agreement and kappa or alpha. Classify conflict reasons and split intent, capability, exposure, and outcome where the taxonomy collapses them. Revise the guide, adjudicate against it, relabel an impact sample, and retain a separate temporal test set.

Use probabilistic labels where ambiguity is real, but define hard actions from operational costs. Monitor reviewer confusion matrices and later incident evidence; seniority and majority vote are not truth mechanisms.

Follow-Up Questions and Answers

Constraint Change

The panel reveals Team A reviews user reports and Team B reviews gateway alerts; the disagreement set was made by joining queues.

The statistic is not interpretable because assignment depends on source and severity. Sample from the underlying delivered-email population and randomly send identical packets to both teams. Estimate queue selection and report it. Do not rank team accuracy until case mix is controlled. The routing policy may be the primary source of disagreement.

Weak Answers and Follow-Ups

Majority vote fails with two correlated teams. Dropping disputes creates an easy dataset. Sending all disputes to a lead hides policy ambiguity. Soft labels alone do not fix selection bias.

If outcomes arrive 60 days later, preserve provisional point-in-time labels and measure their transition to delayed evidence; unresolved objects are censored. If budget is halved, preserve probability sampling, then divide the remainder among uncertainty, disagreement, impact, and novelty. In thirty seconds: compare like cases with like evidence, version every assertion, repair semantics, and let delayed outcomes calibrate the process rather than erase its history.

Key Takeaways

- A label is a claim under an evidence cutoff and policy, not an intrinsic column.
- Separate intent, capability, exposure, and outcome before deriving task targets.
- Agreement measures reliability, not correctness.
- PU and weak-supervision methods depend on explicit selection and dependence assumptions.
- Preserve probability sampling; active-learning batches cannot estimate population quality by themselves.
- Version assertions and retain disagreement because provenance is part of ground truth.

Chapter 19: Metrics That Survive Contact with Reality

A detector can have a beautiful ROC curve and still be unusable. At one malicious event per million, a false-positive rate of 0.1 percent creates roughly one thousand false alerts for each million events. Even perfect recall would yield precision near 0.1 percent. The model is not “99.9 percent accurate” in any operationally useful sense; it has produced a queue no team can investigate.

Security metrics must retain their meaning when prevalence changes, labels mature late, attacks cluster, and analyst capacity is finite. The evaluation object is not merely a classifier. It is a decision policy operating on a population, at a threshold, with an intervention and a queue.

Reconstruct Counts Before Rates

For true positives (TP) , false positives (FP) , false negatives (FN) , and true negatives (TN) :

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}.$$

Given prevalence $(\pi = P(Y=1))$, true-positive rate (t) , and false-positive rate (f) , expected precision is

$$\frac{t\pi}{t\pi + f(1-\pi)}.$$

With $(\pi = 10^{-6})$, $(t = 0.8)$, and $(f = 10^{-3})$, precision is about 0.0008: eight true alerts per ten thousand alerts. Report expected counts per day or million events alongside every rate. Counts expose capacity immediately.

Define the denominator carefully. Event-level FPR may look tiny while identity-level false alert rate is high because active users contribute thousands of events. Evaluate at the unit receiving an action: message, session, identity-day, host, incident, or campaign. Deduplicate correlated alerts before claiming analyst burden.

PR Curves, ROC Curves, and Thresholds

ROC curves plot TPR against FPR and are useful for comparing ranking behavior across all thresholds. Under extreme imbalance, their large true-negative denominator can visually minimize operationally disastrous false positives. Precision-recall curves focus on positive retrieval and reflect prevalence; their no-skill precision baseline is (π) .

Average precision is not a deployment decision. It aggregates thresholds, many of which the team cannot use. Report performance at feasible queue sizes and interventions. “Recall at 200 identity investigations per day” is often more meaningful than area under a curve.

PR curves are prevalence-dependent. A test set enriched to 10 percent attacks cannot supply production precision without reweighting. If class-conditional score distributions transfer, adjust using deployment prevalence. Better, construct a representative temporal test set and report uncertainty in prevalence.

Calibration Is Decision Reliability

A calibrated score means that among predictions near $\hat{p}$, about fraction $\hat{p}$ are positive. Calibration supports cost-sensitive actions and comparison across segments, but aggregate calibration can conceal subgroup error. Use reliability diagrams, Brier score,

$$\text{Brier} = \frac{1}{n} \sum_i (\hat{p}_i - y_i)^2,$$

and expected calibration error with disclosed binning. ECE can be gamed by coarse bins and should never stand alone.

Calibrate on data later than training and earlier than final testing. Isotonic regression is flexible but can overfit small samples; Platt scaling imposes a sigmoid; beta calibration can handle asymmetric distortions. After prevalence or policy changes, prior correction may be appropriate, but attacker adaptation can change more than the prior.

Do not call arbitrary anomaly scores probabilities. If outcomes are censored or selectively reviewed, the calibration target is also biased. Maintain exploration samples and delayed-outcome reconciliation.

Utility Under Capacity Constraints

Let action a have utility conditional on outcome. A simple threshold chooses action when

$$p(x)C_{\text{FN}} > (1-p(x))C_{\text{FP}} + C_A,$$

where C_{FN} is avoided missed-attack loss, C_{FP} is false-intervention cost, and C_A is action cost. The implied threshold is

$$p(x) > \frac{C_{\text{FP}} + C_A}{C_{\text{FN}} + C_{\text{FP}}}.$$

This is only valid if probabilities are calibrated and costs are relevant to the unit. Security losses are heavy-tailed and uncertain; present sensitivity across plausible costs rather than one theatrical dollar value.

Capacity turns thresholding into constrained selection. If analysts can review K cases, rank by expected marginal utility and take positive-utility items up to K . Diversity constraints may prevent one campaign or tenant from consuming the queue. Service-time differences matter: ten complex cloud investigations are not equivalent to ten automated URL checks.

Time Metrics Need State Semantics

Time-to-detect begins at attack occurrence, often unknown—not at log ingestion. Time-to-triage begins when actionable evidence is available, not when a ticket is eventually created. Time-to-contain ends when attacker capability is removed, not when an analyst clicks “resolved.” Define timestamps and censor unresolved cases.

Medians hide tails; means are dominated by rare long cases. Report quantiles and survival curves. Separate processing latency, queue latency, investigation time, and remediation time. A detector that fires earlier but creates a large queue may increase end-to-end containment time.

Coverage is also conditional. ATT&CK technique coverage based on a rule list measures declared scope, not observable or validated coverage. Distinguish telemetry coverage, logic coverage, validation coverage, and response coverage. Weight by exposure only with an explicit, reviewable threat model.

Safe Lab: Compare Two Queue Policies

The synthetic fixture evaluates detectors at a daily capacity of four alerts and assigns utility: +100 for a true investigated attack, -4 for a false investigation, and -35 for every missed attack.

```
records = [
    # id, truth, detector_a, detector_b
    ("e1", 1, .99, .70), ("e2", 1, .80, .95),
    ("e3", 1, .45, .90), ("e4", 0, .90, .30),
    ("e5", 0, .85, .20), ("e6", 0, .70, .80),
    ("e7", 0, .60, .10), ("e8", 0, .55, .75),
    ("e9", 0, .40, .60), ("e10", 0, .30, .50),
]

def evaluate(rows, score_index, capacity=4):
    selected = sorted(rows, key=lambda r: r[score_index], reverse=True)[:capacity]
    chosen = {r[0] for r in selected}
    tp = sum(r[1] for r in selected)
    fp = capacity - tp
    positives = sum(r[1] for r in rows)
    fn = positives - tp
    utility = 100 * tp - 4 * fp - 35 * fn
    return {"tp": tp, "fp": fp, "fn": fn,
            "precision": tp / capacity, "recall": tp / positives,
            "utility": utility, "selected": sorted(chosen)}

a = evaluate(records, 2)
b = evaluate(records, 3)
assert a == {"tp": 2, "fp": 2, "fn": 1, "precision": .5,
             "recall": 2/3, "utility": 157, "selected": ["e1", "e2", "e4", "e5"]}
assert b == {"tp": 2, "fp": 2, "fn": 1, "precision": .5,
             "recall": 2/3, "utility": 157, "selected": ["e2", "e3", "e6", "e8"]}
print(a["utility"], b["utility"], a["selected"], b["selected"])
```

Expected output:

```
157 157 ['e1', 'e2', 'e4', 'e5'] ['e2', 'e3', 'e6', 'e8']
```

Aggregate metrics and utility tie, but the policies catch different attacks. Detector A finds `e1`; B finds `e3`. This matters if `e3` is a novel campaign or higher-impact asset. Evaluation should add attack-family and exposure slices, then test uncertainty. A tie is not permission to ignore composition.

Uncertainty for Rare Events

Events sharing a campaign, identity, host, or time window are not independent. A binomial confidence interval over alerts can be far too narrow. Bootstrap at the cluster that could recur independently—often campaign or incident—and use temporal blocks when conditions drift.

For precision from a probability sample of reviewed alerts, Wilson or Bayesian beta-binomial intervals are more stable than normal approximations. If review is stratified by score or segment, weight observations by inverse inclusion probability. Report the number of confirmed positives, unresolved cases, sampling design, and maturity window.

Comparing two detectors on the same events calls for paired analysis. Bootstrap paired clusters or compare discordant outcomes; do not construct independent intervals and infer significance from overlap. For operational utility, propagate uncertainty in prevalence, labels, and cost as well as model performance.

Rare catastrophic outcomes may not support a narrow interval at all. State an upper bound or scenario range and identify what additional evidence would change the decision.

Metric Schema for Reproducibility

Every reported metric should carry its contract:

```
metric_id: identity_detection.precision_at_capacity
version: 3
unit: identity_day
population: managed_identities_with_complete_auth_telemetry
score_version: detector_b_2026_08_14
selection: top_200_per_utc_day
label: confirmed_compromise_within_60d
unresolved_policy: censored
deduplication: one_alert_per_identity_per_24h
interval: cluster_bootstrap_by_incident_2000_replicates
slices: [tenant_tier, auth_protocol, region]
```

Without this context, a number cannot be compared over time. A falling alert rate after deduplication is not a detection improvement; a precision increase after narrowing the population may not improve total attacks found.

Transport Metrics Across Prevalence and Time

Security teams routinely evaluate on a challenge set whose prevalence differs from deployment. If sensitivity t and specificity $(1-f)$ are stable, precision at a new prevalence π can be recomputed as

$$PPV = \frac{t\pi}{t\pi + f(1-\pi)}$$

This transport is narrower than it appears. It assumes the distribution within positive and negative classes remains sufficiently stable. A new authentication protocol, tenant population, or attacker technique can change

score distributions and invalidate both $\lambda(t)$ and $\lambda(f)$. Report the assumption and verify it with fresh labeled slices rather than applying prior correction indefinitely.

Thresholds also interact with arrival bursts. A policy that fits average daily capacity can fail during a campaign. Model the queue: arrivals have time-varying rate $\lambda(t)$, analysts have service-time distribution rather than a single rate, and unresolved work carries over. Evaluate the probability that queue age exceeds the response objective, not only alerts per day. Reserve capacity for high-severity cases and define load shedding: aggregation, automated enrichment, temporary threshold changes, or escalation to incident mode. Silent dropping is not a capacity policy.

Metrics should measure incremental value over a baseline. For a new detector, compare attacks found uniquely, time gained, and analyst work displaced relative to existing rules. If it produces true alerts already found earlier by another sensor, standalone recall exaggerates portfolio value. Attribute shared detections transparently—first touch, Shapley-style scenario analysis, or set-based overlap—without pretending one convention is causal truth.

Decision-curve analysis can show net benefit across threshold probabilities, but it inherits cost and calibration assumptions. Present a threshold range over which the detector improves on “investigate none” and the current policy. A narrow region signals fragility. Stress it under lower prevalence, worse label sensitivity, higher service time, and adversarial concentration. A production metric survives contact with reality only if its conclusion remains useful across plausible conditions or clearly identifies the condition that reverses it.

Finally, protect against survivorship in incident recall. “Detected incidents” excludes attacks never discovered. Use independent sources—red-team exercises, customer reports, threat hunting, post-incident retrospectives, and sampled forensic review—to estimate discovery gaps. Call the metric recall on known incidents unless the denominator genuinely represents all incidents.

Choosing a Primary Metric

The primary metric should correspond to the deployment decision. For a fixed analyst team, use expected net benefit or incident recall at a workload and latency constraint. For an automated block, use prevented harmful actions with strict false-block and bypass guardrails. For ranking-only assistance, measure lift in the first K reviewed cases and time saved relative to analyst ordering. Always pair the primary metric with coverage and data-quality checks so improvement cannot be purchased by silently excluding difficult traffic.

Pre-register the threshold, population, maturity window, deduplication, and stopping rule before examining the final test set. Repeatedly selecting the best slice or threshold creates multiple-comparison optimism even without formal model training. If many detectors or segments are compared, use a held-out confirmation period and distinguish exploratory findings from decision evidence.

Production Failure Analysis

Enriched test prevalence. Precision from a balanced benchmark is reported as production precision. Reweight or use representative data.

Event-level inflation. One incident produces 500 true-positive alerts. Deduplicate to the response unit and report incident recall.

Selective labels. Only high scores are investigated, making low scores appear negative. Preserve exploration review.

Label delay. Recent alerts look precise because false negatives have not matured. Use fixed maturity windows and censoring.

Capacity omission. A threshold is tuned without queue service times or staffing. Simulate arrival and service distributions and define overload behavior.

Segment masking. Aggregate calibration hides failure on legacy authentication or small tenants. Publish minimum-support slices and shrink noisy estimates rather than hiding them.

Metric gaming. Analysts close easy tickets to improve time-to-triage. Measure state transitions from immutable event timestamps and pair speed with quality and reopen rates.

Adaptive displacement. Precision remains high while attackers move to uncovered protocols. Track exposure and telemetry coverage, not only observed alerts.

Sources and Further Reading

- Saito and Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets.” The paper explains why ROC views can mislead under severe skew. <<https://doi.org/10.1371/journal.pone.0118432>>
- Niculescu-Mizil and Caruana, “Predicting Good Probabilities with Supervised Learning.” Their comparison of learners and calibration methods motivates separating ranking from probability quality. <<https://www.cs.cornell.edu/~caruana/niculescu.icml05.pdf>>
- NIST SP 800-55 Volume 1, **Measurement Guide for Information Security**. Its measure specification and implementation guidance supports explicit denominators, data sources, and interpretation. <<https://csrc.nist.gov/pubs/sp/800/55/v1/final>>
- **scikit-learn, Probability Calibration**. A practical reference for reliability diagrams, calibration methods, and Brier decomposition. <<https://scikit-learn.org/stable/modules/calibration.html>>
- **scikit-learn, Precision-Recall Metrics**. Useful implementation detail, provided prevalence and deployment threshold are not abstracted away. <https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html>

Interview Practicum

Scenario: One Attack per Million.

Prompt

Compare two detectors when attacks occur once per million events. Detector A has 90 percent recall and 0.1 percent FPR; Detector B has 60 percent recall and 0.001 percent FPR. The SOC can review 100 alerts daily across 20 million events.

Worked Answer

Start with counts. Expect 20 attacks daily. A yields 18 true alerts and about 19,980 false alerts, so its raw policy is impossible. B yields 12 true alerts and about 200 false alerts, still above capacity but far closer. Do not choose solely from these summary rates: inspect score distributions and calculate recall at the 100-alert constraint, deduplicated to investigation units. Estimate precision and recall with a representative temporal sample, cluster intervals by campaign, and value missed attacks by exposure.

Calibrate if scores will drive different interventions. Allocate some capacity to random exploration so low-score performance remains measurable. Report attack-family recall, queue latency, and uncovered population.

Follow-Up Questions and Answers

Constraint Change

The interviewer then says Detector A uniquely catches cloud-token theft, while B catches commodity password spray. Token theft is rare but has ten times the expected impact.

The problem becomes portfolio selection. Reserve capacity or use expected-loss ranking across detector outputs, deduplicate overlap, and apply diversity constraints so commodity volume cannot displace high-impact token cases. Validate the tenfold impact assumption with ranges. A detector dominated in aggregate can be essential for a high-value slice.

Weak Answers and Probes

Accuracy is meaningless here. AUROC does not express queue feasibility. F1 silently assigns a particular symmetry to precision and recall and ignores service capacity. Selecting B from lower FPR alone ignores score thresholds and attack composition.

If labels cover only reviewed alerts, introduce probability-sampled review and inverse-probability estimation. If leadership demands one number, provide expected net benefit at stated capacity with a range, plus mandatory recall and burden guardrails. Thirty-second answer: translate rates into counts, evaluate at capacity and action unit, calibrate for costs, and expose uncertainty and segment tradeoffs.

Key Takeaways

- Translate every rare-event rate into expected operational counts.
- Evaluate at the unit that receives an action and deduplicate correlated alerts.

- PR views expose imbalance, but deployment still requires a feasible threshold.
- Calibration, utility, and capacity answer different questions and all matter.
- Confidence intervals must respect campaign, identity, and temporal dependence.
- A detector portfolio may be valuable because of complementary high-impact coverage even when aggregate metrics tie.

Chapter 20: Measuring Security-Control Effectiveness

A safeguard that blocks attacks can still make the system worse. If it delays emergency changes, pushes users to shared credentials, or moves work into an unmonitored channel, its local success metric hides transferred risk. Conversely, a control with visible bypasses may still reduce expected loss by forcing attackers into slower, noisier paths.

Control effectiveness is a causal, socio-technical property. It depends on whether the control was applicable, executed as designed, changed attacker or user behavior, and improved outcomes relative to what would have happened without it. Counting blocks measures mechanism activity. It does not establish risk reduction.

Model the Control as an Evidence Chain

For a step-up approval on sensitive AI-agent tool calls, distinguish six states:

1. **Eligible:** the action should be governed under policy.
2. **Observed:** telemetry represented the action accurately and in time.
3. **Invoked:** the policy engine evaluated the action.
4. **Executed:** allow, deny, or approval was technically enforced.
5. **Resistant:** the requester could not bypass or route around enforcement.
6. **Outcome:** harm was prevented or legitimate work completed acceptably.

Multiplying stage rates is useful only when conditional denominators are explicit:

$$P(\text{effective}) = P(O \mid E)P(I \mid O, E)P(X \mid I)P(R \mid X)P(H \mid R, X).$$

The last factor is not directly observable for prevented events: it asks whether harm would otherwise have occurred. Earlier factors diagnose mechanism coverage; outcome effectiveness requires experiments, quasi-experiments, attack simulation, or a bounded causal argument.

Do not average stage percentages into a “control score.” Ninety-nine percent execution cannot compensate for fifty percent eligibility coverage, and a control that perfectly blocks irrelevant actions has no security value.

A Concrete Agent-Control Schema

The schema should join intent, policy, enforcement, approval, and outcome without treating missing records as allows.

```
CREATE TABLE sensitive_action_attempt (
  attempt_id TEXT PRIMARY KEY,
  occurred_at TEXT NOT NULL,
  actor_token TEXT NOT NULL,
```

```

    session_id TEXT NOT NULL,
    tool_name TEXT NOT NULL,
    action_class TEXT NOT NULL,
    resource_class TEXT NOT NULL,
    policy_version TEXT NOT NULL,
    eligible INTEGER NOT NULL CHECK (eligible IN (0,1)),
    request_digest TEXT NOT NULL
);

CREATE TABLE policy_enforcement (
    attempt_id TEXT PRIMARY KEY,
    evaluated_at TEXT NOT NULL,
    decision TEXT NOT NULL CHECK (decision IN ('allow','deny','require_approval','error')),
    reason_code TEXT NOT NULL,
    enforcement_point TEXT NOT NULL,
    latency_ms INTEGER NOT NULL CHECK (latency_ms >= 0),
    FOREIGN KEY (attempt_id) REFERENCES sensitive_action_attempt(attempt_id)
);

CREATE TABLE approval_resolution (
    attempt_id TEXT PRIMARY KEY,
    resolved_at TEXT,
    resolution TEXT CHECK (resolution IN ('approved','denied','expired','cancelled')),
    approver_role TEXT,
    queue_ms INTEGER CHECK (queue_ms >= 0),
    break_glass INTEGER NOT NULL CHECK (break_glass IN (0,1)),
    FOREIGN KEY (attempt_id) REFERENCES sensitive_action_attempt(attempt_id)
);

CREATE TABLE action_outcome (
    attempt_id TEXT PRIMARY KEY,
    executed INTEGER NOT NULL CHECK (executed IN (0,1)),
    completion_ms INTEGER,
    rollback_required INTEGER CHECK (rollback_required IN (0,1)),
    confirmed_harm INTEGER CHECK (confirmed_harm IN (0,1)),
    bypass_path TEXT,
    matured_at TEXT,
    FOREIGN KEY (attempt_id) REFERENCES sensitive_action_attempt(attempt_id)
);

```

An inner join would erase enforcement failures. Start from eligible attempts and left join downstream records. Missing policy evaluation is a control failure unless a documented telemetry outage makes status unknown. Keep `error` separate from `allow`; fail-open and fail-closed behavior should be measurable.

Intent, Mechanism, and Outcome Measures

Each metric answers a different question:

- **Applicability coverage:** eligible attempts represented in telemetry / estimated eligible attempts.
- **Invocation reliability:** policy evaluations / observed eligible attempts.
- **Enforcement integrity:** decisions enforced without bypass / evaluated attempts.

- **Attack prevention rate:** adversarial attempts prevented / adversarial attempts tested or observed.
- **False block rate:** legitimate attempts denied or expired / legitimate eligible attempts.
- **Approval burden:** approvals requested per active user or completed workflow.
- **Latency:** queue and end-to-end completion distributions, not merely policy-engine milliseconds.
- **Recovery effectiveness:** harmful executions restored within objective / harmful executions.
- **Displacement:** sensitive actions moved to unmanaged paths after deployment.

Preventive controls reduce event occurrence and therefore destroy some of their own labels. Red-team trials and shadow evaluation can estimate what would have passed without enforcement. Detective controls need outcome-linked recall and time-to-detection. Corrective controls require containment quality. Recovery controls require restored service and residual harm. One dashboard cannot use “blocked count” for all four.

Counterfactual Risk Reduction

For population $\mathcal{I}$, let $Y_i(1)$ be harm under the control and $Y_i(0)$ harm without it. The average treatment effect is

$$ATE = E[Y(1) - Y(0)].$$

Only one potential outcome is observed. Comparing protected and unprotected actions is biased when high-risk actions are preferentially protected. Randomization may be unethical for obviously dangerous cases, but alternatives exist: shadow decisions, staged rollout among policy-equivalent units, randomized approval UX that does not remove protection, synthetic adversarial trials, regression discontinuity around a pre-existing threshold, or difference-in-differences with defensible pre-trends.

Use mediation carefully. A control may reduce harmful execution by increasing denial, but approval delays may cause abandonment or bypass. Estimate both the safety pathway and displacement pathway. “No confirmed incidents” after deployment is weak evidence when incidents are rare and detection changes simultaneously.

For simulated attacks, define attack prevention as a conditional mechanism result, not direct production risk reduction. Simulation coverage, realism, and attacker adaptation bound the claim.

Precision, Bypass, and Fail Modes

Control precision is the proportion of restrictive interventions applied to attempts that truly warranted restriction. It is related to detection precision but includes policy and action. An accurate detector can feed an overbroad policy; an inaccurate detector can be buffered by human approval.

Bypass is not merely “policy returned allow.” It includes alternate APIs, stale tokens, direct database access, retries after policy timeout, split actions below a threshold, approval collusion, and moving work to unmanaged tools. Instrument choke points below the user interface. Reconcile authoritative side effects—deployments, secret reads, payments—with policy logs to discover actions lacking an enforcement record.

Fail-open improves availability but can create silent exposure. Fail-closed protects confidentiality or integrity but may stop emergency work. Define modes by action class. Secret export might fail closed; read-only documentation retrieval might fail open with alerting. Measure time spent degraded and actions affected.

Friction Is a Security Cost

Median approval latency is insufficient. Measure p90/p99 queue time, abandonment, retries, duplicate submissions, break-glass use, off-platform movement, support tickets, and task completion. Segment by urgency, time zone, approver availability, and accessibility needs.

Friction can increase risk through predictable workarounds. A user who waits forty minutes for each deployment may batch oversized changes, share privileged sessions, or disable automation. Treat workaround discovery as control telemetry, not user disobedience.

A practical net-benefit model is

$$\backslash [\text{NB} = L_{\{\text{prevented}\}} - L_{\{\text{residual}\}} - C_{\{\text{review}\}} - C_{\{\text{delay}\}} - C_{\{\text{workaround}\}} - C_{\{\text{recovery}\}}. \backslash]$$

Every term should be a range. Avoid converting human time and catastrophic breach loss into falsely precise dollars. The model's value is to expose assumptions and compare designs.

Safe Lab: Evaluate Approval Policy Variants

The lab uses synthetic action attempts. `strict` requires approval for all privileged writes; `targeted` requires it only for externally reachable production resources. Utility awards 120 for blocking an attack, charges 150 for a missed attack, 3 per legitimate approval, 20 per legitimate denial, and 0.02 per second of approval delay.

```

attempts = [
    # id, malicious, privileged_write, external_prod, approval_s, approved
    ("a1", 1, 1, 1, 20, 0),
    ("a2", 1, 1, 0, 15, 0),
    ("a3", 0, 1, 1, 40, 1),
    ("a4", 0, 1, 0, 25, 1),
    ("a5", 0, 0, 0, 0, 1),
    ("a6", 0, 1, 1, 90, 0),
]

def evaluate(policy):
    prevented = missed = approvals = false_denials = delay = 0
    for _, malicious, privileged, external, seconds, approved in attempts:
        governed = privileged if policy == "strict" else privileged and external
        executed = approved if governed else True
        prevented += int(malicious and not executed)
        missed += int(malicious and executed)
        approvals += int(governed and not malicious)
        false_denials += int(governed and not malicious and not approved)
        delay += seconds if governed else 0

```

```

utility = 120*prevented - 150*missed - 3*approvals - 20*false_denials - .02*delay
return {"prevented": prevented, "missed": missed, "approvals": approvals,
        "false_denials": false_denials, "delay_s": delay, "utility": utility}

strict = evaluate("strict")
targeted = evaluate("targeted")
assert strict == {"prevented": 2, "missed": 0, "approvals": 3,
                  "false_denials": 1, "delay_s": 190, "utility": 207.2}
assert targeted == {"prevented": 1, "missed": 1, "approvals": 2,
                    "false_denials": 1, "delay_s": 150, "utility": -59.0}
print(strict["utility"], targeted["utility"])

```

Expected output:

```
207.2 -59.0
```

Under these assumptions, targeted approval misses an internal privileged attack and loses despite less friction. This is not a universal endorsement of strict approval. Change the probability or cost of `a2`, improve approval latency, or add a low-friction control for internal writes and the result can reverse. The lab demonstrates an auditable utility calculation and asserted fixture, not a magic score.

Portfolios and Interaction Effects

Controls combine non-additively. A high-quality sandbox can reduce the need for manual approval; strong least privilege limits impact after a missed detection; immutable logs improve recovery but not prevention. Summing “risk reduction percentages” double-counts the same attack paths.

Represent attack scenarios and which controls interrupt each transition. Evaluate marginal contribution by removing or degrading one control in simulation, replay, or a causal model. Common-cause failures matter: identity provider outage may disable authentication, approval, and audit simultaneously. Portfolio measurement should include correlated failure and defense diversity.

Control interactions may be antagonistic. A restrictive gateway can cause users to adopt an unmanaged channel, weakening DLP. Or detective telemetry may depend on an action executing, while a preventive control removes the event used to validate detection. Version the whole policy stack and record which control had decision authority.

Recovery and Resilience Are Observable Outcomes

Prevention eventually fails, so control evaluation must include the system’s degraded and recovered states. Define a recovery objective in security terms: credentials rotated, unauthorized persistence removed, data integrity verified, dependent services restored, and affected parties informed where required. “Automation job succeeded” proves none of these.

Measure detection-to-containment, containment-to-restoration, restoration completeness, recurrence within a fixed window, and the fraction of recoveries requiring unsafe manual privilege. Exercise recovery with

synthetic tenants and reversible fixtures. A backup control is ineffective if restore credentials share the compromised identity plane, if restoration exceeds business tolerance, or if no one has tested the dependency order.

Resilience may justify a less restrictive preventive control when a reversible action has bounded blast radius and rapid, reliable rollback. It cannot justify permissiveness for irreversible disclosure: a leaked secret or exported customer record cannot be “rolled back.” Classify actions by reversibility, propagation speed, and maximum credible impact, then set approval and recovery requirements accordingly.

Track near misses without turning them into invented prevented losses. A denied request with attack-like content is evidence of mechanism activation; it becomes a validated adversarial trial only after provenance and intent review. Near misses are most useful for finding policy gaps, common dependencies, and recovery weaknesses—not for claiming a dollar amount saved.

Control owners should sign both an effectiveness claim and its expiration conditions. A claim based on one policy version, one enforcement point, and one threat suite must be revalidated after material change. Evidence has a shelf life.

Production Failure Analysis

Denominator loss. Only blocked actions are logged, so coverage and false-negative rates are unknowable. Log eligible attempts at the choke point.

Missing-as-success. Inner joins remove policy errors and absent outcomes. Use left joins from eligibility and explicit unknown states.

Block-count theater. Retries inflate prevention numbers. Deduplicate by intent or transaction and distinguish attempts from attack objectives.

Approval rubber-stamping. High approval rate is presented as low friction, but reviewers approve without inspection. Measure review duration, evidence access, sampled decision quality, and approver concentration.

Displacement. Managed-tool actions fall while unmanaged API usage rises. Reconcile side effects and monitor alternate paths.

Adaptive probing. Attackers infer thresholds through repeated requests. Rate-limit, randomize nonessential feedback, and test split-action evasion.

Recovery fiction. Rollback completed but secrets were already exfiltrated. Define recovery by restored security properties and residual harm, not job status.

Novelty bias. A new control gets credit for a broad secular decline. Use contemporaneous comparison, pre-trends, and mechanism evidence.

Sources and Further Reading

- **NIST SP 800-53 Rev. 5.** Its control families and assessment procedures distinguish control selection from evidence that controls are implemented and effective. <<https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>>
- **NIST Cybersecurity Framework 2.0.** Govern, Identify, Protect, Detect, Respond, and Recover emphasize that prevention is one part of a risk-management system. <<https://www.nist.gov/cyberframework>>
- **CIS Critical Security Controls v8.1.** The safeguards are concrete enough to map to applicability, implementation, and validation evidence. <<https://www.cisecurity.org/controls/v8-1>>
- **OWASP Application Security Verification Standard.** ASVS demonstrates testable requirements and verification levels rather than vague claims of “secure by design.” <<https://owasp.org/www-project-application-security-verification-standard/>>
- **NIST SP 800-160 Vol. 1 Rev. 1.** Its systems-security engineering perspective motivates tracing claims from mechanism through system behavior and lifecycle evidence. <<https://csrc.nist.gov/pubs/sp/800/160/v1/r1/final>>

Interview Practicum

Scenario: A Safeguard That Slows Work.

Prompt

An approval safeguard blocks malicious agent actions in red-team tests, but legitimate production changes wait a median of 18 minutes. Leadership asks whether to remove it.

Clarifying Questions

Ask which actions are eligible, attack severity and frequency, red-team realism, bypass paths, latency distribution, abandonment and break-glass rates, approver coverage by time zone, and whether safer low-friction mechanisms exist. Ask whether the 18 minutes is queue time or end-to-end delay.

Worked Answer

Do not compare blocked attacks with median latency in isolation. Build the evidence chain from eligible action through enforcement and outcome. Reconcile actual production side effects with policy events to measure bypass. Estimate false restrictions through stratified review and task outcomes. Report latency tails, abandonment, workaround behavior, and recovery.

Keep protection while testing redesigns: risk-tier actions, pre-authorization for narrow scopes, just-in-time approvers, cryptographic change plans, and automatic approval for well-bounded reversible operations. Use shadow mode to estimate each policy’s decisions, but never shadow-execute dangerous actions. Red-team both direct bypass and displacement.

Follow-Up Questions and Answers

Constraint Change

The interviewer reveals that 70 percent of delay occurs during one region's overnight hours and the company cannot add headcount.

This changes the mechanism, not the safety requirement. Add time-bounded pre-approval for signed deployment plans, route approvals to an on-call role with conflict-of-interest controls, and allow automatic execution only for reversible actions within an attested scope. Compare by region using queueing and task-completion metrics. Do not globally loosen the threshold to solve a staffing topology problem.

Weak Answers and Follow-Ups

Removing the control based on latency ignores loss severity. Keeping it unchanged because it blocked red-team attacks ignores workarounds. Reporting "attacks blocked" lacks a counterfactual. A single weighted score hides fatal guardrail violations.

If incidents are too rare for an A/B test, triangulate with adversarial trials, shadow decisions, mechanism coverage, staged policy-equivalent rollout, and bounded scenario analysis. If leadership insists on one number, give net benefit as a range but retain non-negotiable bypass, false-denial, and recovery guardrails. Thirty-second answer: preserve protection, localize the delay, test lower-friction mechanisms, and judge the portfolio on prevented harm, bypass, completion, and displacement.

Key Takeaways

- Blocks are mechanism activity, not proof of causal risk reduction.
- Measure eligibility, observation, invocation, enforcement, resistance, and outcome separately.
- Start joins from eligible attempts so missing enforcement cannot disappear.
- User friction includes delay, abandonment, workarounds, and transferred risk.
- Evaluate portfolios by marginal scenario coverage and correlated failure, not summed percentages.
- A control redesign should preserve the security invariant while changing how cheaply it is achieved.

Chapter 21: Experiments in High-Stakes Environments

Security teams often ask for an A/B test after the consequential design choice has already been made. A product team wants to enable automatic session revocation, require approval for an agent tool call, or block a suspicious OAuth grant. “Treatment” can therefore mean denying work, exposing a customer to an attack, or changing what the attacker observes. Ordinary experimentation advice—randomize users, watch conversion, ship the winner—does not survive those facts.

A defensible security experiment begins with an intervention envelope: which actions may vary, for whom, for how long, and with what automatic escape route. Randomization is useful only inside that envelope. It removes confounding; it does not make an unethical comparison ethical, make a rare incident common enough to measure, or prevent spillovers between units.

This chapter develops a concrete design for testing a step-up approval control on high-risk administrative actions. It does not withhold the established baseline control. It compares the baseline with an additive candidate, starts with non-enforcing replay and shadow stages, randomizes by tenant rather than event, and treats safety limits as executable policy.

Define the Experimental Object

Suppose an enterprise platform currently requires reauthentication for exporting more than 10,000 records. A proposed control also requires an independent approver when the request is made by an unfamiliar device or an automation agent. The security hypothesis is:

For eligible export attempts, adding independent approval reduces completed unauthorized exports without causing unacceptable delay or abandonment of legitimate exports.

The unit of assignment is the tenant. The unit of observation is an eligible export attempt. Users within one tenant share administrators, policy, attackers, and support channels. Event-level assignment would let a user learn the treatment from one request and alter a later control request; it could also apply inconsistent policy to one incident. Cluster assignment reduces that interference, although it increases variance.

Eligibility is fixed before assignment:

```
event_type = "bulk_export_requested"
record_count >= 10_000
actor_type IN ("human_admin", "automation_agent")
tenant has approval feature configured
request is not an emergency break-glass operation
```

Break-glass operations are excluded rather than randomized because delayed recovery can amplify harm. Tenants under active incident response are also removed and analyzed separately. Exclusion changes the target population; the final claim must say “configured tenants outside active incidents,” not “all customers.”

A schema that can reconstruct the decision

```
CREATE TABLE export_control_observation (
  request_id          TEXT PRIMARY KEY,
  tenant_id           TEXT NOT NULL,
  cluster_arm          TEXT NOT NULL
                        CHECK (cluster_arm IN ('baseline','candidate')),
  assignment_version   TEXT NOT NULL,
  requested_at         TEXT NOT NULL,
  actor_type          TEXT NOT NULL,
  unfamiliar_device    INTEGER NOT NULL CHECK (unfamiliar_device IN (0,1)),
  record_count         INTEGER NOT NULL CHECK (record_count >= 10000),
  decision             TEXT NOT NULL
                        CHECK (decision IN ('allow','challenge','deny')),
  approval_latency_s   REAL,
  completed           INTEGER NOT NULL CHECK (completed IN (0,1)),
  confirmed_unauthorized INTEGER CHECK (confirmed_unauthorized IN (0,1)),
  outcome_mature_at    TEXT,
  policy_version       TEXT NOT NULL,
  sensor_complete      INTEGER NOT NULL CHECK (sensor_complete IN (0,1))
);
```

`confirmed_unauthorized` remains null until the investigation window closes. Converting null to zero makes the newest arm look safest. Assignment is stored separately from decision because policy defects, exemptions, or outages create noncompliance. The intention-to-treat estimate uses assignment; an “as treated” comparison inherits the operational selection randomization was meant to remove.

Select a Design the System Can Sustain

The candidate passes through four evidence stages:

1. **Replay:** apply the proposed policy to historical events using only fields available at event time. Replay finds workload and compatibility failures but cannot estimate behavioral response.
2. **Shadow:** calculate and log the candidate decision without enforcing it. Sample disagreements between candidate and baseline for security and product review.
3. **Canary:** enforce the candidate for consenting internal or low-dependency tenants. Stop automatically on safety limits.
4. **Cluster-randomized rollout:** assign eligible tenant clusters to baseline or candidate, stratified by size and historical export rate.

A holdback is defensible only while genuine uncertainty remains and both arms meet the accepted minimum standard. Once the candidate is established as materially safer at tolerable cost, a permanent untreated group becomes difficult to justify. Continued learning can compare two safe variants, use randomized encouragement, or exploit scheduled rollout order.

Cluster and switchback designs answer different questions

Cluster randomization is appropriate when treatment persists or spills across actors. If a tenant contains an average of m requests and intraclass correlation is ρ , the design effect is $D = 1 + (m - 1) * \rho$. With $m = 40$ and $\rho = 0.05$, $D = 2.95$: nearly three times the individually randomized sample is required. More events inside one large tenant do not substitute for independent tenants.

A switchback assigns a whole environment to alternating policies across time blocks. It can work for a SOC queue or global rate limiter when simultaneous arms contaminate one another. It fails when treatment has carryover: an approval learned Monday affects Tuesday, an attacker can wait for a baseline interval, and a revoked token does not become unrevoked at a boundary. Blocks need randomized order and justified washout periods; predictable alternation is an attack schedule.

Estimands, Rare Outcomes, and Power

For tenant j , let Z_j denote candidate assignment and $Y_{ij}(z)$ indicate whether request i would become an unauthorized completed export under arm z . The primary intention-to-treat estimand is $\tau = E[Y(1) - Y(0) \mid \text{eligible and assigned}]$.

reported as a risk difference and risk ratio. A negative risk difference favors the candidate. Analysis clusters uncertainty by tenant and includes the predeclared size strata used in randomization.

For independent proportions with baseline risk p_0 , candidate risk p_1 , type-I error α , and power $1 - \beta$, sample size grows with outcome variance and inversely with $(p_1 - p_0)^2$. Halving the minimum detectable absolute effect therefore requires roughly four times as much independent information, before cluster inflation.

If unauthorized completion falls from 0.10% to 0.07%, millions of requests may be required before cluster inflation. No dashboard repairs that information limit. Run longer, broaden the population without changing the question, use a validated intermediate outcome, or acknowledge that the study can rule out only large effects.

“Candidate challenged the request” measures mechanism execution, not prevented loss. Better supporting outcomes include confirmed malicious requests stopped before transfer and adversarial test attempts completing the protected action. They supplement rather than replace mature production incidents.

Guardrails as Executable Safety Policy

A control could reduce unauthorized exports by breaking all exports. Pre-register guardrails with denominators and actions:

- legitimate completion must not decline by more than 2 percentage points;
- p95 approval latency must remain below 180 seconds;
- attributable support contacts must remain below 3 per 1,000 challenges;
- queue age above 10 minutes degrades to reauthentication;

- any confirmed candidate-policy bypass pauses expansion;
- sensor completeness below 99.5% invalidates automated comparison.

Some are engineering limits, not hypothesis tests. One severe bypass can stop expansion even when its p-value is uninteresting. Conversely, a five-minute latency spike should not terminate a month-long trial. The runbook specifies persistence, severity, owner, and recovery.

Repeatedly inspecting fixed-horizon p-values inflates false positives. Inspect at predeclared information times with alpha spending, use a sequential probability ratio test for a sharply defined effect, or use confidence sequences valid under optional stopping. “We looked daily but waited for $p < .05$ ” is not sequential analysis.

Safe Runnable Lab: Simulate a Cluster Trial

This standard-library fixture creates 80 synthetic tenants, simulates legitimate delay and a security outcome, and calculates tenant-weighted risk. It is seeded and contains no operational data.

```
from __future__ import annotations

import random
from collections import defaultdict
from statistics import mean, quantiles

rng = random.Random(2101)
rows: list[dict[str, object]] = []

for tenant_number in range(80):
    size = "large" if tenant_number < 20 else "small"
    arm = "candidate" if tenant_number % 2 else "baseline"
    requests = 80 if size == "large" else 20
    tenant_risk = rng.uniform(0.6, 1.4)
    for _ in range(requests):
        malicious = rng.random() < 0.08 * tenant_risk
        block_probability = 0.72 if arm == "candidate" else 0.35
        unauthorized = int(malicious and rng.random() > block_probability)
        legitimate = not malicious
        latency = rng.lognormvariate(3.6 if arm == "candidate" else 2.4, 0.35)
        completed = int((not malicious or not unauthorized) and latency < 180)
        rows.append({
            "tenant": f"t{tenant_number:02d}", "arm": arm,
            "malicious": malicious, "unauthorized": unauthorized,
            "legitimate": legitimate, "completed": completed, "latency": latency,
        })

by_tenant: dict[tuple[str, str], list[dict[str, object]]] = defaultdict(list)
for row in rows:
    by_tenant[(str(row["tenant"]), str(row["arm"]))].append(row)

tenant_rates: dict[str, list[float]] = defaultdict(list)
for (_, arm), events in by_tenant.items():
```

```

tenant_rates[arm].append(mean(float(e["unauthorized"]) for e in events))

risk = {arm: mean(values) for arm, values in tenant_rates.items()}
risk_difference = risk["candidate"] - risk["baseline"]
legitimate = [r for r in rows if r["arm"] == "candidate" and r["legitimate"]]
completion = mean(float(r["completed"]) for r in legitimate)
p95_latency = quantiles([float(r["latency"]) for r in legitimate], n=20)[18]

print(f"tenant-weighted risk difference={risk_difference:.4f}")
print(f"candidate legitimate completion={completion:.3f}")
print(f"candidate p95 latency={p95_latency:.1f}s")

assert risk_difference < 0, "fixture should show lower unauthorized completion"
assert completion > 0.97, "legitimate completion guardrail failed"
assert p95_latency < 180, "latency guardrail failed"

```

Expected output has a negative risk difference, legitimate completion above 0.97, and p95 latency below 180 seconds. The assertions encode the fixture's intended claims. Production work needs cluster-robust intervals or randomization inference, both tenant- and event-weighted effects, and sensitivity to dominant tenants.

Production Failure Analysis

Sample-ratio mismatch. Nominal 50/50 assignment becomes 60/40 because failing candidate requests disappear before logging. Audit assignment before outcomes and retain pre-decision exposure records.

Novelty effects. Approval delays are high in week one and then fall. Report effects by time since activation and declare whether the estimand includes ramp-up.

Differential ascertainment. Candidate challenges create investigations, which discover more attempted abuse. The candidate may appear to increase attacks because observation improved. Use arm-independent audit sampling and separate attempts from successful harm.

Cross-cluster interference. A centralized attacker targets both arms, or one approval team services every tenant. Map interference paths before selecting the unit. If isolation is impossible, estimate policy effect at the shared-system level.

Attacker adaptation. Publicly predictable holdbacks invite targeting. Do not expose assignment, do not rely on secrecy, and stop if the experiment becomes a targeting oracle.

Delayed labels. Early reports treat unresolved requests as benign. Freeze the primary analysis only after its outcome window matures and show censoring during operation.

Noncompliance. Approvers bypass the workflow or exemptions expand. Report intention-to-treat as primary, execution as a mechanism measure, and exemption reasons as product defects.

What the Sources Contribute

Pre-launch review record

The approval document should contain enough detail for a reviewer to reject the study before any user is exposed. Record the exact randomization code and seed management, cluster exclusions, stratification variables, anticipated sample-ratio tolerance, outcome maturation date, planned information times, and every person authorized to pause or resume. Include a table mapping each telemetry field to its producer and freshness objective. A field that exists in a warehouse but arrives after the decision cannot justify the online policy.

Run an A/A test through the complete assignment and analysis path. An A/A test will not prove the absence of bias, but it can expose broken hashing, inconsistent eligibility, repeated assignment, cluster leakage, and confidence intervals that are too narrow. Inject synthetic stop events to verify that alerts reach an on-call owner and that degradation actually changes policy. Finally, rehearse analysis on a blinded arm label. If conclusions change after labels are revealed, undisclosed researcher degrees of freedom have entered the process.

The decision memo must distinguish three outcomes: evidence supports expansion; evidence supports stopping; and the experiment is inconclusive at the available information. “No statistically significant harm” is not equivalent to safety, especially when the confidence interval still includes an operationally unacceptable increase.

Archive the assignment manifest and signed protocol with the release artifact. Future analysts must be able to distinguish a planned exclusion from a post-result deletion and a safety pause from ordinary missing traffic. That provenance is part of the experimental result.

Retain it for every arm, phase, pause, restart, and emergency policy revision.

Kohavi, Tang, and Xu supply disciplined assignment, sample-ratio checks, guardrails, and experimentation operations. Security adds an important boundary: an inferior arm can expose harm and assignment can become adversarially observable.

The CONSORT cluster extension forces explicit treatment of clusters, recruitment timing, and intracluster correlation—directly relevant to tenant rollouts. Wald explains why stopping must be part of the test, not an improvised dashboard reaction. Statsmodels implements conventional power calculations, but results remain conditional on baseline rate, effect, independence, and attrition. NIST SP 800-30 contributes the risk frame: experiment approval considers impact and uncertainty, not identifiability alone.

Sources and Further Reading

- Kohavi, Tang, and Xu, *Trustworthy Online Controlled Experiments*. <https://doi.org/10.1017/9781108653985>
- CONSORT Extension for Cluster Randomised Trials. <https://doi.org/10.1136/bmj.k1614>
- Wald, “Sequential Tests of Statistical Hypotheses.” <https://doi.org/10.1214/aoms/1177731118>

- **Statsmodels: Power and Sample Size Calculations.** <https://www.statsmodels.org/stable/stats.html#power-and-sample-size-calculations>
- **NIST SP 800-30 Rev. 1.** <https://csrc.nist.gov/pubs/sp/800/30/r1/final>

Interview Practicum

Scenario: The Safe Arm Changes Midstream.

Prompt. Product wants an A/B test of a control requiring approval before an AI agent exports customer data.

A strong candidate asks what protects users today, whether the candidate can accidentally authorize, what unit contains spillovers, how enforcement is disabled, how unauthorized exports mature, and whether attackers can observe assignment.

Worked Answer

Start with replay, shadow disagreement review, then tenant-cluster randomization against reauthentication. Stratify on tenant size and historical volume. The primary analysis is intention-to-treat mature unauthorized completion. Guardrails cover legitimate completion, approval latency, support burden, sensor completeness, and severe bypass. Assignment, execution, and outcome remain separate records.

Follow-Up Questions and Answers

Constraint change. Two weeks into shadow mode, red teaming finds that the baseline is bypassable for service accounts. Legal says knowingly retaining it is unacceptable.

Patch every arm immediately. The original comparison ends because treatment versions changed and equipoise disappeared. Preserve it as a superseded phase. After verifying the universal patch, ask a new question: does independent approval add benefit beyond the repaired baseline? Issue a new protocol and assignment version, recalculate power because baseline risk fell, and restart staged rollout. If the remaining outcome is too rare, use bounded adversarial evaluation plus production guardrails rather than claiming equivalence.

Second constraint. The approval service is global, so candidate tenants increase baseline latency.

No-interference is false. Reserve capacity by arm, randomize service shards, or use guarded switchbacks if carryover is short. If none isolates exposure, estimate a system-level policy effect. Do not publish tenant-level A/B standard errors.

Weak answers withhold all protection, optimize challenge rate, ignore clustering, or promise to “monitor p-values.” Staff-level reasoning distinguishes ethical from statistical stops. Principal-level reasoning recognizes that a baseline defect changes the allowable intervention, estimand, and governance record.

Thirty-second answer. Randomize only between controls meeting minimum safety, assign by tenant, and stage replay through canary before enforcement. Measure mature unauthorized completion with executable

friction and safety guardrails. If evidence makes an arm unsafe, patch all arms, close the phase, and start a newly versioned experiment.

Key Takeaways

- Randomization removes confounding in expectation; it does not grant ethical permission or eliminate interference.
- Assignment must follow spillovers, shared operations, and attacker learning.
- Rare outcomes impose hard power limits; mechanism metrics do not automatically substitute for harm.
- Guardrails need denominators, persistence rules, owners, and recovery actions.
- A newly unsafe baseline ends the old comparison and requires a new protocol.

Chapter 22: Causal Inference for Security Decisions

An incident curve that falls after an EDR rollout is visually persuasive and causally ambiguous. The rollout may have prevented compromise. It may also have changed detection, coincided with an identity hardening program, reached low-risk business units first, or occurred while an adversary paused operations. Causal inference is the discipline of stating which counterfactual is needed, which observations can approximate it, and which assumptions remain untestable.

This chapter follows one concrete question: did endpoint detection and response reduce confirmed ransomware entry events? The answer is not a universal “EDR effectiveness” coefficient. It is an effect for defined endpoints, rollout dates, outcome rules, and operating conditions.

Write the Counterfactual Before the Query

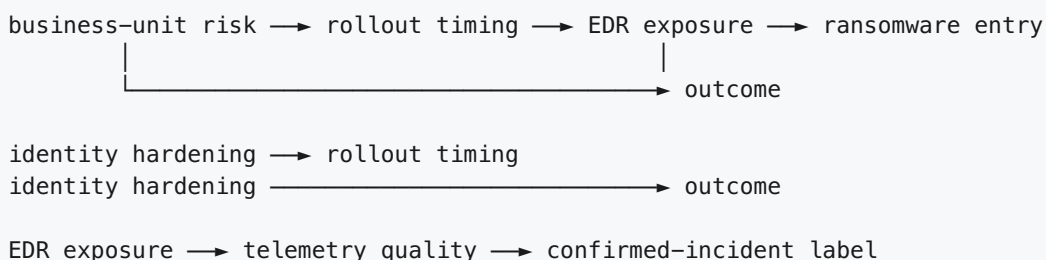
For endpoint i , let $Y_i(1)$ be its outcome under EDR and $Y_i(0)$ under the prior endpoint control. The individual effect cannot be observed because an endpoint cannot simultaneously occupy both states. A useful target is $ATT = E[Y_i(1) - Y_i(0) \mid A_i = 1]$.

where $(A_i=1)$ means the endpoint received EDR during the study. Define (Y) as confirmed ransomware entry events per 10,000 active endpoint-days, not alert count. EDR can increase alerts while reducing successful entry.

The population excludes unsupported operating systems only if they were never eligible. Removing eligible endpoints that failed installation conditions on a post-treatment variable and can make deployment look better. The primary estimand is therefore assignment-to-rollout where assignment records exist; installed-and-healthy effect is secondary and needs stronger assumptions or an instrument.

Causal structure

The central confounding paths can be expressed without an ornamental diagram:



Business-unit risk and concurrent identity hardening are pre-treatment confounders. Telemetry quality is a mediator and an outcome-measurement mechanism. Adjusting for post-rollout telemetry quality can block part of the treatment pathway while ignoring it can create differential ascertainment. The solution is not blindly

“control for everything.” Define an arm-independent outcome process where possible—for example, incident-response confirmation using network, backup, and identity evidence—and analyze label sensitivity separately.

Build a Point-in-Time Panel

One row represents a business-unit week because rollout decisions and operational spillovers occur at that level. Endpoint-day denominators remain explicit.

```
CREATE TABLE unit_week (
  business_unit      TEXT NOT NULL,
  week_start         TEXT NOT NULL,
  endpoint_days      INTEGER NOT NULL,
  edr_coverage       REAL NOT NULL CHECK (edr_coverage BETWEEN 0 AND 1),
  assigned_rollout   INTEGER NOT NULL CHECK (assigned_rollout IN (0,1)),
  ransomware_entries INTEGER NOT NULL,
  identity_mfa_rate  REAL NOT NULL,
  internet_exposure  REAL NOT NULL,
  os_legacy_share    REAL NOT NULL,
  incident_sensor_v  TEXT NOT NULL,
  PRIMARY KEY (business_unit, week_start)
);
```

Do not join today’s asset owner or criticality onto last year’s endpoint. Slowly changing dimensions must be resolved as of each week. Do not define treatment using “has ever installed EDR”; that leaks future state into preperiods. For a binary rollout analysis, treatment begins on the first week coverage exceeds the predeclared threshold, such as 90%, and remains on. Weeks in partial deployment are either modeled as a dose or excluded according to a registered rule.

Identification Strategies and Their Contracts

Adjustment, weighting, and doubly robust estimation

If treatment is plausibly ignorable conditional on pre-treatment covariates X , potential outcomes must be independent of assignment after conditioning on X .

then regression, matching, or propensity weighting can estimate effects. The propensity score is $(e(X)=P(A=1\text{mid } X))$. An inverse-probability weighted mean uses $(A/e(X))$ for treated and $((1-A)/(1-e(X)))$ for controls. Extreme weights reveal a positivity problem: some units had effectively no chance to receive one arm. Trimming may stabilize the estimate but changes its population.

An augmented inverse-probability weighted estimator combines an outcome model and a propensity model. It starts with each unit’s predicted treated-minus-control outcome and adds a propensity-weighted residual correction for the arm actually observed.

It is consistent if either nuisance model is correct under the other causal assumptions—not if the DAG is wrong, overlap is absent, labels differ by arm, or an important confounder is missing.

Difference-in-differences

When rollout is staggered, compare changes in treated units with contemporaneous changes in not-yet-treated units. In the simplest two-group case, $DiD = (treated_post - treated_pre) - (control_post - control_pre)$.

The identifying assumption is parallel untreated trends, not equal levels. Preperiod event-study coefficients can reveal gross violations but cannot prove the unobserved postperiod counterfactual. A two-way fixed-effects regression can be biased under staggered adoption and heterogeneous treatment effects because already-treated groups become controls. Use group-time estimators such as Callaway–Sant’Anna or interaction-weighted event studies such as Sun–Abraham.

Interrupted time series and synthetic controls

Interrupted time series is useful for a system-wide policy with a long, stable preperiod. It estimates level and slope changes at intervention, while modeling seasonality and autocorrelation. Any simultaneous change remains a rival explanation.

A synthetic control builds a weighted combination of untreated units matching the treated unit’s preperiod trajectory. It is attractive for one enterprise-wide deployment with comparable donor organizations or regions. It fails when donors are contaminated by shared controls, the treated unit lies outside their convex hull, or few preperiod observations permit overfitting.

Threshold policies

If rollout was determined by a score cutoff—endpoints above criticality score 80 received EDR first—regression discontinuity can estimate a local effect near 80. It requires no precise score manipulation and continuity of potential outcomes at the cutoff. The result is local; it does not establish effect for score 20 endpoints.

Safe Runnable Lab: Difference-in-Differences

The fixture constructs four units. Two receive EDR after week four. All units share a rising background threat; treatment removes two incidents per treated unit-week. The standard-library calculation recovers (-2).

```
from statistics import mean

rows: list[dict[str, object]] = []
for unit in ("payments", "sales", "research", "support"):
    treated_group = unit in {"payments", "research"}
    for week in range(1, 9):
        post = week >= 5
        baseline = 4 + week  # common time trend
        unit_offset = {"payments": 2, "sales": 0, "research": 1, "support": -1}[unit]
        treatment_effect = -2 if treated_group and post else 0
        incidents = baseline + unit_offset + treatment_effect
        rows.append({
```

```

        "unit": unit, "treated_group": treated_group,
        "post": post, "incidents": incidents,
    })

def group_mean(treated: bool, post: bool) -> float:
    return mean(
        int(r["incidents"]) for r in rows
        if r["treated_group"] is treated and r["post"] is post
    )

treated_change = group_mean(True, True) - group_mean(True, False)
control_change = group_mean(False, True) - group_mean(False, False)
did = treated_change - control_change

print(f"treated change={treated_change:.1f}")
print(f"control change={control_change:.1f}")
print(f"DiD effect={did:.1f} incidents/unit-week")

assert did == -2.0

# A falsification check: pre-treatment slopes are identical in this fixture.
def pre_slope(unit: str) -> int:
    values = [int(r["incidents"]) for r in rows if r["unit"] == unit and not r["post"]]
    return values[-1] - values[0]

assert len({pre_slope(u) for u in ("payments", "sales", "research", "support")}) == 1

```

Expected output is:

```

treated change=2.0
control change=4.0
DiD effect=-2.0 incidents/unit-week

```

This deliberately clean fixture teaches the contrast, not production inference. Real counts call for clustered uncertainty, exposure offsets, staggered estimators, treatment timing checks, and plots of group-time effects.

Sensitivity and Falsification

A causal estimate should travel with tests aimed at its assumptions:

- **Pretrend event study:** large pre-treatment coefficients undermine DiD.
- **Negative-control outcome:** EDR should not reduce an unrelated event such as lost badges. An apparent effect suggests a shared reporting change.
- **Negative-control exposure:** a future rollout indicator should not predict past incidents after the design variables are included.
- **Alternative outcome definitions:** repeat with strict confirmed ransomware, broad suspected ransomware, and independent backup-encryption evidence.

- **Donut and bandwidth checks:** for regression discontinuity, exclude observations nearest the cutoff and vary bandwidth.
- **Weight diagnostics:** show propensity overlap, effective sample size, and influence of extreme units.
- **Unmeasured confounding:** report how strong an omitted confounder would need to be to erase the estimate; do not summarize this as “robust” without scale.

Results should use bounded language: “Under parallel trends and stable outcome ascertainment, rollout is associated with an estimated reduction of X entries per 10,000 endpoint-days.” That sentence names both effect and identification contract.

Production Failure Analysis

Risk-based rollout. The most attacked units receive EDR first. A naive post-rollout comparison makes EDR look harmful. DiD may address fixed level differences but not different counterfactual trends.

Concurrent controls. MFA, macro blocking, and backup isolation launch with EDR. The data identify the package unless timing or variation separates components.

Detection paradox. EDR discovers incidents the prior stack missed. Short-term confirmed events rise even if compromise falls. Use outcomes less dependent on the treated sensor and explicitly model discovery.

Treatment misclassification. “License assigned” is not healthy coverage. Assignment estimates policy effect; healthy installation estimates require compliance methods. Never silently switch definitions until the desired result appears.

Anticipation. Teams harden endpoints before the official date. Event studies need leads, and treatment may begin when behavior changes rather than when procurement closes.

Spillover. EDR on one endpoint prevents lateral movement into controls. The stable-unit-treatment assumption fails, and endpoint-level effects understate network benefit. Cluster at a meaningful network or business boundary.

Staggered-estimator pathology. A convenient two-way fixed-effects model negatively weights some cohort effects. Report group-time effects and aggregation weights.

Selective incident closure. Analysts close candidate-arm cases faster. Freeze label policy, blind adjudication where possible, and measure unresolved-case censoring by arm and period.

What the Sources Contribute

From estimate to decision memo

A production causal analysis should ship as an auditable bundle, not a notebook screenshot. Preserve the cohort query, dimension snapshots, treatment-timing table, outcome-adjudication version, exclusion counts, overlap diagnostics, estimator configuration, and a machine-readable results table. The memo opens with the target estimand and then reports absolute effect, relative effect, interval, and baseline risk. A 40% relative

reduction can be trivial when baseline risk is tiny; an apparently small absolute reduction can matter greatly across millions of endpoint-days.

Show a specification curve across defensible alternatives: different preperiod lengths, strict and broad outcomes, exclusion of transition weeks, and influence-limited analyses. This is not an invitation to average incompatible estimands. It reveals whether the operational conclusion depends on one fragile choice. Identify which checks were planned and which were prompted by results.

Decision language should match the weakest important assumption. If pretrends are credible but outcome ascertainment changed, say the estimate is sensitive to measurement. If overlap fails for legacy operating systems, do not extrapolate to them. If the interval includes both meaningful benefit and negligible effect, recommend either further observation or a decision under explicit loss asymmetry. Causal inference narrows uncertainty; it does not remove the need for accountable judgment.

Choosing among designs

Prefer adjustment or weighting when treatment varies across many comparable units and the important assignment variables were recorded before treatment. Prefer difference-in-differences when repeated outcomes support a credible untreated trend and adoption times vary. Prefer regression discontinuity when a real threshold determined eligibility and observations cluster densely around it. Prefer synthetic control when one or a few aggregate units changed and a credible donor pool has a long preperiod. Interrupted time series is the fallback for a system-wide intervention with no contemporaneous control, but its causal claim is especially exposed to simultaneous events.

These methods are not interchangeable robustness checks. They answer different local or population effects under different assumptions. Agreement across methods is informative only when each design is independently plausible; averaging several biased estimates does not create validity. Before selecting a method, write down the source of counterfactual information in one sentence. If that sentence says only “the model controls for many features,” the design has not yet identified an effect.

Security reviews should also ask whether the intervention changes adversary selection. A visible EDR rollout may redirect attacks toward unmanaged endpoints, improving the treated population while increasing enterprise-wide risk. Report displacement outcomes and define whether the estimand is endpoint protection or total organizational loss.

Hernán and Robins provide the potential-outcomes language that prevents “adjusted association” from being casually renamed causation. Their emphasis on target trials is particularly valuable: specify eligibility, strategies, assignment, follow-up, outcome, contrast, and analysis as if the observational study were emulating a trial.

Cunningham explains design-based identification through accessible worked examples, including DiD, regression discontinuity, and synthetic control. Callaway and Sant’Anna address staggered timing by estimating group-time effects with appropriate comparison groups. Sun and Abraham show why conventional event-study coefficients can be contaminated under heterogeneous effects. DoWhy operationalizes graph-based assumptions, identification, estimation, and refutation; software can make the reasoning inspectable but cannot validate an omitted arrow.

Sources and Further Reading

- Hernán and Robins, *Causal Inference: What If*. <https://www.hsph.harvard.edu/miguel-hernan/causal-inference-book/>
- Cunningham, *Causal Inference: The Mixtape*. <https://mixtape.scunning.com/>
- Callaway and Sant’Anna, “Difference-in-Differences with Multiple Time Periods.” <https://doi.org/10.1016/j.jeconom.2020.12.001>
- Sun and Abraham, “Estimating Dynamic Treatment Effects in Event Studies.” <https://doi.org/10.1016/j.jeconom.2020.09.006>
- DoWhy. <https://www.pywhy.org/dowhy/>

Interview Practicum

Scenario: The Outcome Definition Moves.

Prompt. Ransomware incidents declined after EDR rollout. Did EDR cause the decline?

Ask how rollout was prioritized, whether other controls changed, what outcome evidence depends on EDR, whether units have multiple preperiods, how partial coverage is represented, and which population the decision concerns.

Worked Answer

Propose a target-trial emulation with business-unit weeks, point-in-time covariates, assignment or a pre-declared coverage threshold, and confirmed entry events per endpoint-day. For staggered rollout, use not-yet-treated comparisons and group-time DiD. Inspect pretrends, concurrent controls, coverage overlap, label maturation, and negative controls. State the ATT and its assumptions rather than promising universal causality.

Follow-Up Questions and Answers

Constraint change. Halfway through analysis, incident response reveals that before EDR it confirmed ransomware only when users reported encryption; after EDR it also counted blocked precursor activity.

The original outcome is not stable. Do not “control for logging volume” and proceed. Reconstruct a comparable strict endpoint—such as user-visible encryption or independently verified payload execution—across all periods. Analyze blocked precursors as a mechanism outcome. If historical strict outcomes cannot be recovered, narrow the claim, triangulate with backup and network evidence, and treat the causal estimate as unidentified rather than zero.

Second constraint. The CISO says EDR and MFA launched together in every unit.

The observed variation identifies their joint package. Seek timing differences, eligibility thresholds, or another credible source of independent variation. Without it, report package effect and avoid component attribution. A mediation model cannot manufacture variation.

Weak answers compare before and after, add every available column to a regression, or treat a nonsignificant pretrend test as proof. Strong senior answers distinguish outcome measurement from confounding. Principal answers tell leadership when the requested component effect is not identified and propose the next rollout design that would identify it.

Thirty-second answer. I would emulate the rollout's target trial, define a sensor-independent outcome, and use staggered group-time DiD only if untreated trends are credible. I would test pretrends, label stability, concurrent controls, and overlap. If EDR changed what counts as an incident or always launched with MFA, I would narrow the claim to what the variation actually identifies.

Key Takeaways

- Start with potential outcomes, population, treatment version, and follow-up—not a preferred regression.
- Adjustment is valid only under a causal graph, overlap, consistency, and no unmeasured confounding.
- Staggered adoption requires estimators that avoid already-treated units as invalid controls.
- A treated sensor can change the label; stable outcome ascertainment is part of identification.
- When controls always move together, data identify the package, not its components.

Chapter 23: Adversarial Drift and Continuous Validation

A detector can preserve aggregate accuracy while failing the population that matters. Fraud or account-takeover labels arrive weeks late; meanwhile, attacker traffic moves from consumer accounts to small-business administrators. The dominant benign class keeps the headline stable, the intervention changes which cases receive labels, and the model's own blocks remove outcomes from observation. Drift monitoring in security is therefore not a histogram dashboard. It is a versioned argument connecting inputs, decisions, delayed outcomes, attacker tests, and a safe response.

This chapter uses a login-abuse detector whose overall accuracy remains 99.4% while recall for small-business tenants collapses. The technical objective is to identify the kind of change, estimate performance despite delayed and selective labels, and decide whether to continue, restrict, challenge, or roll back.

Four Changes That Demand Different Responses

Let (X_t) be features, (Y_t) the latent malicious outcome, $(\hat{Y}_t)$ the model decision, and (π_t) the production policy converting scores into actions.

Data drift changes $(P_t(X))$. A new mobile SDK may alter device identifiers while the relationship between evidence and compromise remains stable. Retraining may be unnecessary; schema repair may be mandatory.

Concept drift changes $(P_t(Y \mid X))$. A previously suspicious residential proxy feature becomes common among legitimate privacy tools, or attackers adopt token replay that lacks password-failure precursors. The decision boundary itself is stale.

Prior drift changes $(P_t(Y))$. An active campaign increases prevalence. Calibration and capacity can fail even if class-conditional feature distributions remain similar.

Policy drift changes $(\pi_t(\hat{p}, X))$, label collection, or downstream treatment. A business unit raises the challenge threshold, analysts stop reviewing low scores, or a new recovery flow changes what “confirmed compromise” means.

“Attacker drift” can cause any combination, but naming the statistical object matters. A population-stability alert does not prove concept drift. A recall decline may be a label-policy change. Retraining on corrupted labels can institutionalize the failure.

Segment First, Then Aggregate

For segment g , mature recall is $\text{true positives} / \text{all actual positives}$, restricted to decisions in that segment whose outcome window has closed.

Aggregate recall is a prevalence-weighted mixture. If a vulnerable segment is small, its collapse barely moves the total. Production review should predeclare security-relevant slices: identity type, tenant tier, authentication protocol, geographic region, device trust, control state, and telemetry version. This is not permission to hunt

arbitrary subgroups until a bad result appears. Maintain a fixed core set plus a separately labeled exploratory discovery process with multiple-testing discipline.

Calibration also needs segment and time. Expected calibration error is the bin-size-weighted absolute difference between observed outcome frequency and mean predicted probability in each score bin.

Expected calibration error can hide opposing errors between segments and depends on binning. Pair it with reliability tables, Brier score, and decision-specific calibration near thresholds.

A Validation Ledger, Not a Mutable Feature Table

The minimum event contract separates what was known at decision time from what arrived later:

```
CREATE TABLE login_decision (
  decision_id      TEXT PRIMARY KEY,
  occurred_at      TEXT NOT NULL,
  tenant_id        TEXT NOT NULL,
  tenant_segment   TEXT NOT NULL,
  auth_protocol    TEXT NOT NULL,
  feature_version  TEXT NOT NULL,
  model_version    TEXT NOT NULL,
  policy_version   TEXT NOT NULL,
  score            REAL NOT NULL CHECK (score BETWEEN 0 AND 1),
  action           TEXT NOT NULL CHECK (action IN ('allow','challenge','block')),
  sensor_mask      INTEGER NOT NULL,
  review_probability REAL NOT NULL CHECK (review_probability BETWEEN 0 AND 1)
);

CREATE TABLE login_outcome (
  decision_id      TEXT PRIMARY KEY REFERENCES login_decision(decision_id),
  label            INTEGER CHECK (label IN (0,1)),
  label_source     TEXT NOT NULL,
  observed_at      TEXT NOT NULL,
  mature_at        TEXT NOT NULL,
  adjudication_v   TEXT NOT NULL
);
```

Never overwrite a score after model reprocessing. Store challenger outputs in another table keyed by decision and challenger version. Preserve `review_probability`: when case `i` is reviewed with known probability `q_i`, its contribution to a population total is weighted by $1 / q_i$.

This corrects known sampling, not analyst mistakes, unreviewable cases, or outcomes suppressed by blocking.

Monitoring Before Labels Mature

Immediate monitors are proxies arranged by distance from the desired outcome.

1. **Contract health:** nulls, cardinality, freshness, schema version, join success, sensor mask.
2. **Feature behavior:** quantiles, categorical novelty, missingness, and conditional distributions by segment.

3. **Score and action behavior:** score histograms, threshold mass, block/challenge rate, and abstention.
4. **Operational response:** recovery success, queue age, overrides, support contacts, and user abandonment.
5. **Mature outcomes:** recall, precision, calibration, loss, and attack success after the label window closes.

Population stability index is a coarse screening statistic:

$$[\text{operatorname{PSI}} = \sum_{k=1}^K (a_k - e_k) \log(a_k / e_k),]$$

where (e_k) and (a_k) are reference and current proportions. PSI depends on binning and sample size; conventional 0.1/0.25 thresholds have no universal security meaning. Jensen–Shannon divergence is symmetric and bounded, while Kolmogorov–Smirnov compares continuous empirical distributions. None establishes impact. Alert only when a change connects to a feature contract, vulnerable segment, or decision risk.

Delayed labels require cohort aging. A dashboard for day (t) should compare decisions from equally mature cohorts, such as all decisions whose 30-day outcome window has closed. A nowcast may estimate eventual outcomes from historical maturation curves, but it remains provisional and must be reconciled later.

Feedback Loops and Missing Counterfactuals

When high scores are blocked, their untreated outcomes cannot be observed. Training only on allowed logins selects lower-risk cases. Analyst review only above threshold creates the same blind spot. Defenses include:

- randomized review below threshold;
- a shadow challenger that never controls action;
- safe adversarial simulations with known ground truth;
- exploration among cases where alternative actions are ethically acceptable;
- causal evaluation of policy changes rather than relabeling blocked cases benign.

Do not intentionally allow a suspected account takeover merely to obtain a label. Exploration is bounded by the existing safety floor.

Challenger Design and Temporal Validation

A challenger consumes the same point-in-time features but has no enforcement authority. Evaluate it on rolling origin splits: train through (t) , validate on $(t+1)$, then advance. Random splits leak campaigns, identities, and policy regimes across train and test.

Promotion criteria should be expressed at the decision boundary: expected loss at queue capacity, recall for protected segments, calibration near intervention thresholds, latency, and robustness on red-team challenge sets. A challenger that improves average precision but doubles false challenges for service accounts is not an automatic winner.

Red-team cases are not an IID estimate of production prevalence. They test named capabilities: token replay without password failures, distributed low-rate attempts, device-fingerprint rotation, missing user-agent, and

threshold probing. Track pass rate by attack family and version, but never blend challenge-set counts into a production confusion matrix.

Safe Runnable Lab: Find the Hidden Segment Failure

```
from collections import defaultdict

# Each tuple is (period, segment, label, prediction).
rows = []
for period in ("reference", "current"):
    # Consumer traffic dominates and remains stable.
    rows += [(period, "consumer", 0, 0)] * 900
    rows += [(period, "consumer", 1, 1)] * 90
    rows += [(period, "consumer", 1, 0)] * 10

# Small business is healthy in reference, then attackers change technique.
rows += [("reference", "small_business", 0, 0)] * 90
rows += [("reference", "small_business", 1, 1)] * 9
rows += [("reference", "small_business", 1, 0)] * 1
rows += [("current", "small_business", 0, 0)] * 90
rows += [("current", "small_business", 1, 1)] * 2
rows += [("current", "small_business", 1, 0)] * 8

def metrics(period: str, segment: str | None = None) -> tuple[float, float]:
    selected = [r for r in rows if r[0] == period and (segment is None or r[1] == segment)]
    accuracy = sum(y == p for _, _, y, p in selected) / len(selected)
    positives = [(y, p) for _, _, y, p in selected if y == 1]
    recall = sum(p == 1 for y, p in positives) / len(positives)
    return accuracy, recall

for period in ("reference", "current"):
    overall = metrics(period)
    small = metrics(period, "small_business")
    print(period, f"accuracy={overall[0]:.3f}",
          f"overall_recall={overall[1]:.3f}",
          f"small_business_recall={small[1]:.3f}")

assert metrics("current")[0] > 0.98
assert metrics("reference", "small_business")[1] == 0.9
assert metrics("current", "small_business")[1] == 0.2
```

Expected output shows current accuracy above 0.98 while small-business recall falls from 0.900 to 0.200. The lab intentionally demonstrates composition masking. A real monitor would attach binomial or bootstrap uncertainty, minimum support, maturity cutoff, and a response policy.

Response Ladders and Rollback

Not every anomaly deserves a model rollback. Use a graded authority ladder:

- **Observe:** annotate a low-risk feature shift and collect mature labels.

- **Constrain:** lower automation authority; convert blocks to challenges for the affected segment.
- **Fallback:** route to a deterministic policy with known behavior.
- **Rollback:** restore a compatible model–feature–policy bundle.
- **Suspend:** abstain when evidence integrity is insufficient.

Rollback compatibility is frequently missed. Restoring model v7 while leaving feature v12’s device encoding and policy v10’s threshold can create a novel system. Release manifests must bind feature transformations, model artifact, calibration map, threshold policy, explanation codes, and runbook.

Resilience is measured by degradation, detection, and recovery, not just steady-state AUC. Useful quantities include time to detect a seeded regression, maximum loss during degradation, fraction of traffic safely handled by fallback, and time to verified recovery.

Production Failure Analysis

Silent schema substitution. An SDK changes empty device ID from null to "unknown" ; null monitoring stays green while a categorical feature shifts. Monitor semantics and novelty, not null rate alone.

Label-delay bias. Recent false negatives have not matured, so current recall looks excellent. Compare equal-age cohorts and publish unresolved counts.

Intervention censoring. Blocks are labeled “prevented fraud” without evidence. Separate policy action from outcome and maintain randomized audits where safe.

Adaptive probing. Attackers infer the score boundary and remain just below it. Monitor density near thresholds, repeated near-boundary identities, and red-team evasions; avoid assuming secrecy will hold.

Alert-driven retraining. A drift alarm triggers immediate training on the same corrupted window. Require root-cause classification, candidate backtests across prior regimes, and change approval.

Challenger contamination. Analysts see challenger scores and alter labels, destroying independence. Hide challenger decisions from adjudicators unless the study explicitly measures assisted review.

Unsafe global rollback. The old model is better for one segment but vulnerable to a newer attack elsewhere. Constrain the affected segment or use a composite fallback rather than treating versions as globally ordered.

What the Sources Contribute

Continuous-validation review cadence

Operate three clocks. The streaming clock watches contracts, scores, actions, latency, and capacity within minutes. The cohort clock recomputes performance when seven-, thirty-, or ninety-day labels mature. The governance clock reviews whether the threat model, segment inventory, response authority, and evidence sources remain appropriate. Combining these into one dashboard encourages the team to treat fast proxies as final truth.

Every alert must route to a named diagnostic playbook. A feature-distribution shift first checks producer releases, sensor coverage, and entity composition. A score-rate shift checks feature and model versions. A mature recall decline opens a segment incident and evaluates authority reduction. Require an incident record when a threshold is overridden manually; otherwise temporary exceptions become invisible policy drift.

Track validation debt explicitly: segments without enough labeled support, challenger versions awaiting mature outcomes, stale red-team cases, and fallbacks not exercised recently. Debt is not merely documentation backlog. It measures where the system can take consequential action without current evidence. Budget regular game days that remove a sensor, delay labels, corrupt a feature, and force rollback. Recovery time observed during these exercises is stronger evidence than an untested runbook.

Set alert thresholds from response capacity and consequence, not round divergence values copied from a blog. A high-volume categorical shift with no decision impact may merit a ticket; a small loss of a decisive token-replay feature may justify immediate authority reduction. Record both the statistical trigger and the operational action so future tuning does not erase why the limit exists.

That record should identify the approving owner, expiration date, and required recovery evidence.

Gama and colleagues distinguish drift detection, understanding, and adaptation; that separation prevents every distribution alert from becoming retraining. ADWIN from Bifet and Gavalda maintains an adaptive window and signals statistically significant mean changes, useful for bounded streams but not proof of malicious concept drift.

Sculley and colleagues show that ML failure often resides in data dependencies, feedback loops, and configuration debt rather than the model artifact. Security intensifies those debts because policy changes observation. NIST AI RMF and its Playbook add governance: validity, reliability, monitoring, documentation, and response ownership must persist through deployment. They do not prescribe one drift statistic; they require the organization to connect measurement to managed risk.

Sources and Further Reading

- **NIST AI Risk Management Framework 1.0.** <https://www.nist.gov/itl/ai-risk-management-framework>
- **Sculley et al., “Hidden Technical Debt in Machine Learning Systems.”** <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>
- **Gama et al., “A Survey on Concept Drift Adaptation.”** <https://doi.org/10.1145/2523813>
- **Bifet and Gavalda, “Learning from Time-Changing Data with Adaptive Windowing.”** <https://doi.org/10.1137/1.9781611972771.42>
- **NIST AI RMF Playbook.** https://airc.nist.gov/AI_RM_F_Knowledge_Base/Playbook

Interview Practicum

Scenario: Accuracy Is Stable, Then Labels Disappear.

Prompt. A fraud detector’s aggregate accuracy is stable, but investigators say it is missing attacks. Diagnose and respond.

Ask for confusion matrices by mature cohort and segment, label delay, action policy, review sampling, recent schema and attacker changes, queue capacity, and rollback compatibility.

Worked Answer

Recompute recall, precision, calibration, and expected loss for fixed high-risk slices. Audit feature and policy versions, then replay champion and challenger on temporal cohorts and capability-based red-team cases.

Follow-Up Questions and Answers

Constraint change. The analysis finds small-business recall fell from 0.81 to 0.34. Product cannot roll back globally because the previous model is vulnerable to a token-replay campaign affecting enterprise tenants.

Use a segmented degradation plan. Convert automated allows near the threshold into step-up challenges for small-business administrators, preserve the current model for enterprise token replay, and deploy a bounded deterministic signal for the missing small-business technique. Start a shadow challenger trained on point-in-time data. This is not permission to create permanent model forks without governance; version the routing policy and define convergence criteria.

Second constraint. Fraud labels now arrive only for challenged events because the review vendor stopped sampling allowed traffic.

Production recall is no longer identified from observed labels. Restore probability sampling of allowed traffic with known inclusion probabilities, use independent customer-recovery signals, and label current metrics provisional. Do not use challenged-only data to “prove” the fix. If safe sampling cannot be restored, report capability-test performance and operational proxies separately from unknown production recall.

Weak answers retrain immediately, watch population stability alone, or roll back globally. Strong answers separate data, concept, policy, and label drift. Principal answers design an authority-reduction path that protects distinct segments while rebuilding unbiased observation.

Thirty-second answer. Stable accuracy can hide a segment recall collapse. I would compare equally mature cohorts by protected segment, audit feature/model/policy versions, and check whether interventions changed label selection. Because global rollback revives another known vulnerability, I would constrain authority only for the failing segment, add a bounded fallback, and restore randomized outcome sampling before claiming recovery.

Key Takeaways

- Data, concept, prior, policy, and label-process changes require different responses.
- Aggregate performance is a mixture that can hide catastrophic segment failure.

- Immediate drift statistics are diagnostics; mature outcome performance remains the target.
- Intervention changes observation, so review probability and label provenance belong in the schema.
- Safe degradation may be segmented; rollback is a compatible bundle, not a model file.

Chapter 24: Statistical Detection Engineering

An analyst’s useful observation is not yet a production detection. “Many failures followed by a success from one network” leaves unanswered what counts as many, which entities share a baseline, how VPN outages behave, when event time closes, what evidence an alert carries, and how the rule changes safely. Statistical detection engineering turns a threat hypothesis into a versioned analytical product whose assumptions can fail visibly.

This chapter builds a password-spray detection. The adversary attempts a small number of passwords across many accounts, often avoiding per-account lockout. A fixed threshold on total failures is inadequate: corporate egress gateways produce high legitimate counts, tenants differ by orders of magnitude, and outages create bursts. The design combines an explicit sequence rule with an overdispersed count baseline and a change detector.

Threat Hypothesis and Observable Sequence

The threat hypothesis is:

A source that fails authentication against an unusually diverse set of accounts and then succeeds against one of those accounts within 20 minutes may be conducting password spraying.

This maps to MITRE ATT&CK credential-access behavior, but ATT&CK technique mapping is not evidence of correctness. The observable sequence requires:

1. at least 12 distinct target accounts;
2. at least 18 failed attempts in a 10-minute event-time window;
3. no single target contributes more than 25% of failures;
4. the source’s count is anomalous for its peer class;
5. a success for a targeted account within 20 minutes raises severity.

The unit is a normalized source principal, not raw IP. NAT egress, trusted scanners, identity-provider health checks, and Tor exits need distinct peer classes. IPv4 and IPv6 normalization, forwarded-address trust, and cloud proxy attribution belong in the data contract.

Concrete Event and Alert Schemas

```
CREATE TABLE authentication_event (
  event_id      TEXT PRIMARY KEY,
  event_time    TEXT NOT NULL,
  ingest_time   TEXT NOT NULL,
  tenant_id     TEXT NOT NULL,
  source_key    TEXT NOT NULL,
```

```

    source_class      TEXT NOT NULL,
    target_account    TEXT NOT NULL,
    outcome           TEXT NOT NULL CHECK (outcome IN ('success','failure')),
    auth_protocol     TEXT NOT NULL,
    device_trust      TEXT,
    sensor_version    TEXT NOT NULL
);

CREATE INDEX auth_window
ON authentication_event(tenant_id, source_key, event_time);

CREATE TABLE detection_alert (
    alert_id          TEXT PRIMARY KEY,
    window_end        TEXT NOT NULL,
    tenant_id         TEXT NOT NULL,
    source_key        TEXT NOT NULL,
    rule_id           TEXT NOT NULL,
    rule_version      TEXT NOT NULL,
    failure_count      INTEGER NOT NULL,
    distinct_targets  INTEGER NOT NULL,
    max_target_share   REAL NOT NULL,
    tail_probability  REAL NOT NULL,
    follow_on_success  INTEGER NOT NULL,
    severity          TEXT NOT NULL,
    evidence_event_ids TEXT NOT NULL
);

```

The alert stores evidence references and intermediate statistics. A scalar “risk score” would make investigation and regression testing harder. Evidence events remain immutable; suppression and disposition are separate lifecycle records.

A compatible SQL candidate generator

SQLite can generate candidate windows at a fixed boundary. Production stream processors use event-time windows and watermarks, but the aggregation semantics should match:

```

WITH failures AS (
    SELECT tenant_id, source_key, target_account, COUNT(*) AS attempts
    FROM authentication_event
    WHERE outcome = 'failure'
      AND event_time >= '2026-08-20T10:00:00Z'
      AND event_time < '2026-08-20T10:10:00Z'
    GROUP BY tenant_id, source_key, target_account
),
source_summary AS (
    SELECT tenant_id,
           source_key,
           SUM(attempts) AS failure_count,
           COUNT(*) AS distinct_targets,
           1.0 * MAX(attempts) / SUM(attempts) AS max_target_share

```



```

    FROM failures
    GROUP BY tenant_id, source_key
)
SELECT *
FROM source_summary
WHERE failure_count >= 18
    AND distinct_targets >= 12
    AND max_target_share <= 0.25;

```

This query generates candidates; it does not yet incorporate peer expectation, late events, or follow-on success.

Why a Negative-Binomial Baseline Fits Better

If counts arose from a homogeneous Poisson process, mean and variance would both equal λ . Authentication failures are heterogeneous and bursty; variance usually exceeds the mean. A gamma-Poisson mixture yields a negative-binomial distribution with $E[Y] = \mu$ and $\text{Var}[Y] = \mu + \alpha * \mu^2$, where $\alpha > 0$ captures overdispersion. Model log expected failures as an intercept plus hour-of-day, weekday, log active accounts as an exposure offset, and a source-class effect.

The exposure offset prevents a large tenant from being anomalous merely because it has more accounts. Estimate on a trailing period that excludes known incidents and the current window. Use a survival function ($P(Y_{ge} \text{ ymid} \mu, \alpha)$), but do not treat it as a calibrated attack probability. It measures incompatibility with the baseline model.

For sparse sources, shrink toward the peer class instead of fitting unstable personal means. A source with two historical windows cannot support a reliable per-source threshold. High-volume corporate proxies may have their own baseline only after enough uncontaminated history.

Robustness Against Hostile and Broken Data

Attackers can poison rolling means by slowly increasing traffic. Outages can inflate both mean and variance. Robust alternatives bound influence:

- median and median absolute deviation, $MAD = \text{median}(|x_i - \text{median}(x)|)$;
- trimmed estimation windows;
- winsorized counts for baseline fitting, while preserving raw counts for evidence;
- exclusion of declared identity-provider incidents;
- peer medians with minimum cohort size.

For approximately symmetric data, a robust z-score is $0.6745 * (x - \text{median}(X)) / MAD(X)$.

Count data are neither symmetric nor continuous at low means, so MAD is a fallback diagnostic, not a universal replacement for count models.

Detecting persistent low-and-slow change

A one-window tail test misses an attacker that remains mildly elevated. A one-sided Page CUSUM accumulates evidence as $S_t = \max(0, S_{(t-1)} + x_t - (\mu_0 + k))$ and signals when $S_t > h$. The allowance k sets the smallest sustained shift worth accumulating; h trades delay against false alarms. Reset or freeze CUSUM during known sensor outages. An exponentially weighted moving average, $Z_t = \lambda x_t + (1 - \lambda) Z_{(t-1)}$,

is easier to explain but smears a sharp burst into subsequent windows.

The production detector uses the deterministic diversity sequence as a semantic gate, negative-binomial tail probability for contextual rarity, and CUSUM for persistence. No weighted sum is necessary:

```
if sequence_gate and follow_on_success: severity = HIGH
elif sequence_gate and (tail_p < 0.001 or cusum_alarm): severity = MEDIUM
else: no alert
```

This preserves reason codes and makes policy changes reviewable.

Detection-as-Code Artifact

A repository artifact should include threat reference, required fields, logic, tests, and ownership:

```
id: identity.password_spray.follow_on_success
version: 3
status: canary
owner: identity-detection
event_time:
  window: 10m
  allowed_lateness: 3m
group_by: [tenant_id, source_key]
requirements:
  sensor_versions: [auth-v4, auth-v5]
gate:
  failures_min: 18
  distinct_targets_min: 12
  max_target_share: 0.25
context:
  peer_tail_probability_max: 0.001
sequence:
  follow_on_success_within: 20m
response:
  medium: analyst_queue
  high: revoke_session_and_queue
fallback:
  on_baseline_unavailable: sequence_only_medium
```

Schema validation should reject unknown keys and invalid durations. The high-severity response is more consequential, so it requires the success sequence and must have a tested recovery path.

Safe Runnable Lab: Behavioral Spray Detection

The lab creates a benign proxy burst concentrated on two accounts and a distributed spray across eighteen accounts. It tests the semantic distinction rather than generating attack traffic.

```
from collections import Counter
from dataclasses import dataclass

@dataclass(frozen=True)
class Auth:
    minute: int
    source: str
    account: str
    outcome: str

events: list[Auth] = []

# Benign retry storm: high volume, low target diversity.
events += [Auth(i % 10, "corp-proxy", f"svc-{i % 2}", "failure") for i in range(30)]

# Synthetic distributed spray and one follow-on success.
events += [Auth(i % 10, "198.51.100.8", f"user-{i:02d}", "failure") for i in range(18)]
events.append(Auth(14, "198.51.100.8", "user-07", "success"))

def candidate(source: str) -> tuple[bool, dict[str, float]]:
    failures = [e for e in events if e.source == source and e.outcome == "failure" and e.minute < 10]
    counts = Counter(e.account for e in failures)
    total = len(failures)
    diversity = len(counts)
    max_share = max(counts.values(), default=0) / total if total else 0.0
    targets = set(counts)
    success = any(
        e.source == source and e.outcome == "success"
        and 10 <= e.minute <= 30 and e.account in targets
        for e in events
    )
    fired = total >= 18 and diversity >= 12 and max_share <= 0.25
    return fired, {
        "failures": float(total), "targets": float(diversity),
        "max_share": max_share, "follow_on_success": float(success),
    }

proxy_fired, proxy = candidate("corp-proxy")
spray_fired, spray = candidate("198.51.100.8")

print("proxy", proxy_fired, proxy)
print("spray", spray_fired, spray)

assert proxy_fired is False
assert spray_fired is True
```

```
assert spray["follow_on_success"] == 1.0
assert spray["max_share"] < 0.10
```

Expected output reports `proxy False` and `spray True`. The proxy exceeds the raw count threshold but fails target diversity. The spray satisfies diversity and has follow-on success. Production tests also cover exact boundaries, duplicate IDs, late success, IPv6 normalization, missing source attribution, daylight-saving changes, and replay determinism.

Event Time, Suppression, and Deployment

Use event time for the threat sequence and ingest time for pipeline health. A three-minute watermark means the initial window closes after tolerating three minutes of lateness. A success arriving later can upgrade an existing alert idempotently; it must not create an unrelated duplicate. Very late events enter a correction stream used for retrospective metrics.

Suppression is part of detection semantics. Suppress a source only for a bounded interval and reason—known scanner, approved penetration test, or identity-provider incident. Broad allowlists become attacker infrastructure. Record who approved each exception, its expiry, and which evidence caused matches during suppression.

Rollout proceeds through historical replay, labeled challenge sets, shadow traffic, and a canary queue. Measure:

- recall by attack variant on curated cases;
- precision on mature, probability-sampled production alerts;
- alerts and evidence volume per analyst-hour;
- time from first failure to alert and to follow-on upgrade;
- late-event correction rate;
- suppression match and expiry rate;
- segment results by tenant size, source class, and protocol.

Detection ownership includes a review cadence and retirement criteria. A rule without an owner accumulates exceptions until no one knows which threat it detects.

Production Failure Analysis

VPN outage flood. Failure counts explode across many sources and accounts. Add identity-provider health as a control-plane signal and use global incident suppression with explicit expiry; do not teach the baseline that outages are normal.

NAT aggregation. Thousands of legitimate users share one source. Peer by attributed source class and require concentration/diversity semantics; raw IP thresholds are unsuitable.

Distributed spray. The adversary spreads attempts across many source keys. A source-centric detector loses power. Add a complementary account/ASN/campaign view; do not quietly widen this rule until its entity semantics become incoherent.

Slow poisoning. Counts rise just below alert threshold for weeks. Freeze clean reference windows, use CUSUM, and compare with peer cohorts.

Late and duplicated telemetry. Retries inflate counts and successes arrive after closure. Deduplicate by event ID before windows and support alert upgrades and corrections.

Threshold probing. Attackers stop at eleven targets. Threshold secrecy is temporary. Maintain overlapping signals, randomized review near the boundary, and red-team variants around every inequality.

Baseline outage. Feature service cannot provide (μ) or (α) . The declared fallback runs the strict sequence gate at medium severity; it does not substitute zero and generate a false tail probability.

Response coupling. Automatic revocation makes a medium-confidence false positive expensive. Keep statistical rarity advisory until the follow-on success supplies stronger semantics, and monitor recovery success.

What the Sources Contribute

Reviewable tests for a detection pull request

A detection change should arrive with positive fixtures, hard negatives, boundary cases, and historical replay results. Positive fixtures cover a fast distributed spray, a low-and-slow CUSUM case, and a failure sequence upgraded by success. Hard negatives cover a VPN outage, a two-account retry loop, a sanctioned scanner, and a large NAT gateway. Boundary tests exercise exactly 17 versus 18 failures, 11 versus 12 targets, a 25% target share, and success immediately before and after the allowed interval.

Replay compares candidate and current versions on identical immutable input. Review the newly alerted, newly suppressed, and severity-changed populations rather than only total counts. A large count difference is a demand for explanation, not automatically a regression. Store a small, privacy-safe evidence sample for each category and obtain analyst adjudication without revealing which version produced it when feasible.

The pull request also declares compute cost, maximum state per entity, watermark behavior, deduplication key, exception migration, dashboard changes, and rollback artifact. Approval from a detection author alone is insufficient for an automatic response: identity operations must verify recovery, data engineering must verify event semantics, and incident response must accept the evidence contract.

Sigma provides a portable vocabulary for rule metadata and log-source logic, but statistical state, peer models, and event-time upgrades usually require implementation beyond a static Sigma condition. NIST SP 800-94 frames detection as part of an intrusion detection and prevention lifecycle, including tuning, response, and operational placement.

MITRE ATT&CK data sources help connect behavior to observable components; they do not guarantee that a particular telemetry field is complete. Page's CUSUM supplies a principled method for accumulating small

persistent deviations. Statsmodels' negative-binomial implementation provides estimation machinery, while model fitness still depends on exposure, overdispersion, zero structure, and clean training windows.

Sources and Further Reading

- **Sigma Rule Specification.** <https://sigmahq.io/sigma-specification/>
- **NIST SP 800-94.** <https://csrc.nist.gov/pubs/sp/800/94/final>
- **MITRE ATT&CK Data Sources.** <https://attack.mitre.org/datasources/>
- **Page, “Continuous Inspection Schemes.”** <https://doi.org/10.1093/biomet/41.1-2.100>
- **Statsmodels: Negative Binomial Model.** https://www.statsmodels.org/stable/generated/statsmodels.discrete.discrete_model.NegativeBinomial.html

Interview Practicum

Scenario: The Network Team Removes Source IP

Prompt. Convert an analyst's “many failures then one success” query into a production password-spray detection.

Ask which attack variant is in scope, whether source attribution survives proxies, how lockout behaves, what event-time delay is acceptable, which sources are shared, what response authority is allowed, how labels are sampled, and who owns exceptions.

Worked Answer

Define a distributed-target sequence, normalize sources, fit overdispersed peer baselines with account exposure, and retain deterministic reason codes. Store immutable evidence, model and rule versions, watermark state, and follow-on upgrades. Test outages, NAT, duplicates, boundary thresholds, low-and-slow behavior, and suppression expiry. Shadow before routing to a bounded canary queue.

Follow-Up Questions and Answers

Constraint change. Privacy removes full source IP next quarter. Events retain tenant-scoped rotating source tokens that change every 24 hours, ASN, country, and device identifier.

Do not claim the original detector is unchanged. Ten-minute within-token aggregation still works inside a day, but long-term per-source baselines and cross-day CUSUM do not. Shift the baseline to ASN/source-class peers, use device continuity where lawful, and state the reduced ability to connect distributed campaigns. Re-evaluate NAT behavior and privacy leakage. If stable attribution is essential to automatic revocation, reduce authority to analyst queue until new evidence supports it.

Second constraint. Attackers distribute attempts across fifty rotating tokens, each below the gate.

Create a separate campaign detector keyed by tenant, ASN, target-set overlap, and synchronized time—not an ever-expanding exception inside the source rule. Its false-positive surface and response policy differ. Correlate its alert with the source detector at investigation time.

Weak answers hard-code twenty failures, assume IP equals attacker, or use one opaque score. Strong answers preserve the threat semantics through event-time and attribution failures. Principal answers split source and campaign hypotheses instead of forcing one rule to own incompatible units.

Thirty-second answer. I would formalize a diverse-target failure sequence, contextualize counts with an overdispersed peer baseline, and upgrade severity only on follow-on success. The artifact includes schema, watermark, evidence, tests, suppression expiry, fallback, and owner. If privacy removes stable IP, I would redesign the baseline and reduce response authority rather than pretend the evidence is equivalent.

Key Takeaways

- A detection begins with a falsifiable threat sequence and explicit entity semantics.
- Overdispersed count baselines are usually more realistic than global fixed or Poisson thresholds.
- Deterministic gates, statistical context, and persistent-change tests should retain separate reason codes.
- Event-time correction, exception expiry, and compatible fallback are part of correctness.
- When telemetry changes the observable unit, redesign and revalidate the detection.

Chapter 25: Anomaly Detection Without the Hype

Anomaly Detection Is a Search Strategy

An anomaly detector does not detect attacks. It orders observations by disagreement with a model of ordinary behavior. That distinction is operationally decisive. A payroll administrator opening a compensation export at quarter end may be statistically exceptional and legitimate; a compromised employee slowly reading the same repositories they use every day may be statistically ordinary. Anomaly detection is valuable when it shrinks an otherwise unreviewable search space, not when its score is mistaken for a probability of malice.

The object being modeled must be explicit. For identity-access activity, a row might represent one access, one session, or an identity-day. Those choices produce different questions. A point detector can rank an unusual access. A session representation can expose an unusual sequence. An identity-day representation can reveal a collective anomaly: no single event is remarkable, but their count, diversity, and timing are.

This chapter develops an honest benchmark for that setting. It treats unreviewed observations as unknown rather than benign, reserves analyst capacity in the threshold, and demands an explanation that identifies the reference population and the features responsible for the rank.

Four Kinds of “Different”

Let (x_i) be an observation and (c_i) its context. An anomaly score is a function $s(x_i, c_i)$ whose numerical scale is model-specific. Four distinctions prevent many design errors:

1. **Point anomaly:** the vector itself is remote, such as 400 repositories read in one hour.
2. **Contextual anomaly:** the event is unusual given context, such as ordinary volume at 03:00 local time for a daytime employee.
3. **Collective anomaly:** a set or sequence is unusual although its members are not, such as a low-and-slow scan across hundreds of resources.
4. **Conditional anomaly:** the observation disagrees with an appropriate peer or personal baseline, such as a finance identity invoking a deployment API common among SREs.

For an identity-day vector

$$x = (\log(1+N), H(R), f_{\text{off}}, f_{\text{denied}}, N_{\text{new}}, N_{\text{countries}}),$$

N is access count, $H(R) = -\sum_r p_r \log p_r$ is resource-category entropy, and the remaining terms describe off-hours fraction, denial fraction, first-seen resources, and country diversity. Log-transforming counts prevents one bursty feature from dominating. Entropy distinguishes repeated work on one resource from broad reconnaissance. Context must include identity class and role; otherwise service accounts and humans contaminate one another's baseline.

Robust standardization is preferable when attackers or batch jobs can create extremes:

$$\left[z_{ij} = \frac{x_{ij} - \operatorname{median}(x_{\cdot j})}{\{1.4826 \operatorname{median}|x_{\cdot j} - \operatorname{median}(x_{\cdot j})| + \epsilon\}} \right]$$

The formula supplies a resistant scale; it does not claim the feature is normally distributed.

Methods and Their Actual Assumptions

Isolation Forest repeatedly partitions features and gives higher anomaly scores to observations isolated by short paths. It works well on nonlinear tabular patterns and scales reasonably. Its score can change with irrelevant dimensions, sampling, and contamination settings. It cannot explain security meaning without an attribution layer.

Local Outlier Factor compares local density. In simplified form,

$$\left[\text{LOF}_k(p) = \frac{1}{|N_k(p)|} \sum_{o \in N_k(p)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(p)} \right]$$

A value above one means (p) is sparse relative to its neighbors. LOF can find a contractor anomalous among contractors even when that group occupies a globally sparse region. It is sensitive to scaling and (k) ; novelty mode must be trained only on reference data when scoring the future.

Cluster distance is useful when stable modes exist—interactive employees, CI workers, backup services—but a small cluster is not automatically hostile. Autoencoders compress and reconstruct normal examples; reconstruction error is a representation error, not maliciousness. They become attractive for high-dimensional sequences only after simpler baselines fail. With incomplete labels, extra capacity often learns logging quirks more efficiently than attacker behavior.

An ensemble should combine ranks, not arbitrary raw scores. If $(r_m(i) \in [0,1])$ is percentile rank under method (m) , use $(S_i = \sum_m w_{mr_m(i)})$, with weights fixed on a development period rather than tuned until remembered incidents rise.

A Concrete Identity-Day Contract

The feature table has one row per tenant, identity, and UTC day. Production systems should retain the time zone used to compute off-hours.

```
CREATE TABLE identity_day_features (
  tenant_id TEXT NOT NULL,
  identity_id TEXT NOT NULL,
  identity_type TEXT NOT NULL CHECK (identity_type IN ('human','service')),
  role_family TEXT NOT NULL,
  day_utc TEXT NOT NULL,
  access_count INTEGER NOT NULL,
  distinct_resources INTEGER NOT NULL,
  resource_entropy REAL NOT NULL,
  off_hours_fraction REAL NOT NULL,
  denied_fraction REAL NOT NULL,
```

```

new_resource_count INTEGER NOT NULL,
country_count INTEGER NOT NULL,
feature_version TEXT NOT NULL,
PRIMARY KEY (tenant_id, identity_id, day_utc, feature_version)
);

```

The raw event contract needs `event_id`, event and ingestion times, tenant, resolved identity, identity type, role family, resource, category, action, decision, country, source, and schema version. “New resource” must be point-in-time: first seen before the observation ends, never first seen according to today's complete history. Identity merges require a versioned mapping because retroactive resolution otherwise rewrites a benchmark.

Peer selection is part of the model. A defensible fallback hierarchy is `(tenant, identity_type, role_family)`, then `(tenant, identity_type)`, then a global identity-type cohort. Record which level was used. A score compared against 14 peers does not deserve the confidence of one compared against 14,000.

Choosing a Threshold from Capacity

The contamination parameter is not attack prevalence. It is often merely how many observations to rank as outliers. If the team can investigate $\lfloor B \rfloor$ identity-days per day, the first operating point is the top $\lfloor B \rfloor$, subject to deduplication and diversity constraints:

$$\lfloor \max_t \sum_{i: s_i \geq t} \widehat{V}_i \rfloor \quad \text{subject to} \quad \lfloor \{s_i \geq t\} \rfloor \leq B.$$

$\widehat{V}_i$ is estimated review value, not necessarily incident probability. Reserve 10–20% of capacity for random or stratified below-threshold review. Without it, the organization labels only what the detector already likes and builds a self-confirming evaluation set.

With incomplete labels, report adjudicated precision at the top, known-incident rank, rank stability under resampling, lift over random sampling, analyst time per useful lead, and coverage across identities and peer groups. Synthetic attack injection tests whether the representation can express a behavior; it does not estimate production precision.

Safe Runnable Lab: Peer Context Versus Global Rarity

This standard-library lab creates ordinary identity-days and three review cases. It demonstrates why context beats raw rarity.

```

from collections import defaultdict
from math import sqrt
from statistics import median
from random import Random

FEATURES = ("access_count", "entropy", "off_hours", "new_resources")

def mad(values):

```

```

center = median(values)
return median(abs(v - center) for v in values) or 1e-9

def robust_distance(row, reference):
    total = 0.0
    for feature in FEATURES:
        values = [r[feature] for r in reference]
        z = (row[feature] - median(values)) / (1.4826 * mad(values))
        total += z * z
    return sqrt(total)

def fixtures():
    rng = Random(25)
    rows = []
    for role, count, entropy in (("engineer", 42, 1.7), ("finance", 18, 0.8)):
        for i in range(24):
            rows.append({"id":f"{role}-{i}", "role":role,
                        "access_count":count+rng.randint(-6, 6),
                        "entropy":entropy+rng.uniform(-0.15, 0.15),
                        "off_hours":rng.uniform(0.0, 0.12),
                        "new_resources":rng.randint(0, 2)})
    rows += [
        {"id":"fin-broad-read", "role":"finance", "access_count":35,
         "entropy":2.1, "off_hours":0.08, "new_resources":11},
        {"id":"eng-night-burst", "role":"engineer", "access_count":83,
         "entropy":1.8, "off_hours":0.91, "new_resources":4},
        {"id":"fin-quarter-close", "role":"finance", "access_count":70,
         "entropy":0.7, "off_hours":0.03, "new_resources":0}]
    return rows

rows = fixtures()
peers = defaultdict(list)
for row in rows:
    peers[row["role"]].append(row)
global_rank = sorted(rows, key=lambda r: robust_distance(r, rows), reverse=True)
peer_rank = sorted(rows, key=lambda r: robust_distance(r, peers[r["role"]]), reverse=True)
global_top = [r["id"] for r in global_rank[:3]]
peer_top = [r["id"] for r in peer_rank[:3]]
print("global_top", global_top)
print("peer_top", peer_top)
assert "fin-broad-read" in peer_top
assert "eng-night-burst" in peer_top
assert peer_top[:2] == ["eng-night-burst", "fin-broad-read"]
assert len(rows) == 51

```

The assertions are the contract: both planted behaviors occupy the first two peer-conditioned positions. Quarter close is rare but legitimate. Production should encode its calendar context rather than expect an algorithm to infer it magically. When comparing Isolation Forest or LOF, keep this fixture, temporal split, and alert budget fixed; otherwise workload is confounded with model quality.

Designing the Benchmark Before Choosing the Winner

An anomaly benchmark needs chronological partitions. Fit transformations and reference distributions on an earlier interval, tune neighborhood size or tree parameters on the next interval, and report once on a later interval. Random row splitting leaks the same identities, recurring jobs, and organizational calendar into every partition. If the intended deployment must score unseen tenants or new business units, add a tenant or organization holdout as a separate test; temporal generalization and organizational portability are different claims.

Build challenge slices that correspond to representation risks: low-and-slow access, burst access, new employee, role transfer, service-account rotation, quarter close, incident response, and telemetry loss. Planted anomalies should be created by transforming held-out benign fixtures rather than pasting an obviously extreme constant. For example, preserve a user's daily count while increasing resource diversity, or distribute a burst across five days. Record the transformation recipe and never mix planted cases into estimates of real-world precision.

Stability matters because analysts experience rank, not an abstract objective. Bootstrap the reference period or refit across random seeds, then measure top- k Jaccard overlap and rank variance. A method whose top 40 changes almost entirely after a harmless resample is difficult to operate even if it captures one extra known incident. Investigate whether instability comes from small peers, correlated features, or observations near the capacity boundary. A deterministic tie-breaker makes replay possible but does not cure statistical instability.

Evaluation also needs a candidate-generation baseline. Compare against simple policies such as first-seen sensitive resource, robust off-hours burst, and random role-stratified sampling. A complex detector earns production cost only if it discovers useful cases those policies miss at the same review budget. Report incremental yield: unique useful cases contributed after baseline cases are removed. This prevents an ensemble from receiving credit for rediscovering every obvious rule hit.

Finally, define case consolidation before calculating precision. Ten anomalous identity-days from one compromised campaign may create one investigation, while ten independent identities may create ten. Event-level precision can look excellent while drowning analysts in duplicates. Evaluate alerts, consolidated cases, and incidents separately; measure time to first useful lead and total review effort. The benchmark should reproduce the queue the SOC will actually see, including suppressions, grouping, and delayed enrichment.

Model retirement criteria belong in the benchmark as well. Retire or redesign a detector when its unique useful yield remains below the transparent baseline across mature cohorts, when peer coverage falls below an agreed floor, when explanations cannot be reproduced from retained evidence, or when operating cost exceeds the investigation value it adds. A detector should not survive merely because it once found an incident. Conversely, one quiet month is not proof of failure when attack outcomes are rare. Use several consecutive evaluation windows, uncertainty intervals, and challenge-set results.

Ownership should be divided clearly. Data engineering owns event contracts and freshness; detection engineering owns representation, tests, and release artifacts; the SOC owns case workflow and disposition guidance; privacy and legal approve purpose and retention; business-system owners explain legitimate exceptional processes. A monthly review should inspect false-positive clusters, below-threshold audit discoveries, cohort

coverage, and recent organizational changes. Parameter changes need versioning and replay. Quietly moving a percentile threshold to meet a dashboard target destroys the comparability of every subsequent result.

The final operational artifact is therefore not a model file. It is a versioned package containing feature definitions, reference-window rules, peer fallback logic, fitted artifacts, capacity policy, explanation format, challenge fixtures, audit-sampling design, known blind spots, and rollback instructions. That package makes anomaly detection reviewable as engineering rather than mystical as “AI.”

Explanations Analysts Can Falsify

“Score 0.83” is not an explanation. A useful record says: “rank 2 of 27 finance identity-days; 11 first-seen resources versus peer median 1; entropy 2.10 versus 0.81; off-hours behavior ordinary.” It identifies the reference set and surprising dimensions and lets an analyst reject the result because, for example, a role transfer made the peer group stale.

For distance models, contributions are squared standardized components. For Isolation Forest, perturbation or SHAP-style explanations can be used cautiously, but correlated features make attribution unstable. For autoencoders, per-feature reconstruction error is descriptive, not causal. Include model and feature versions, window, peer cohort, fallback level, missing features, and comparable observations.

Production Failure Analysis

Peer poisoning. A compromised identity persists until its behavior enters the rolling baseline. Use lagged windows, cap each identity's influence, and retain a clean long-term baseline.

Population churn. Reorganizations change role families; cold-start users have no history. Fall back explicitly and avoid high-consequence automation on weak cohorts.

Batch-job domination. Service identities create extreme counts and teach a global model that human bursts are ordinary. Separate identity classes before fitting.

Score drift without threat drift. A client rollout changes event cardinality. Monitor raw features and semantics, not only score histograms.

Threshold probing. An adversary distributes activity across identities or days. Add collective and graph features; do not depend on threshold secrecy.

Feedback blindness. Analysts review only the top queue, so missed behaviors disappear from labels. Maintain randomized audits and incident-linked retrospective evaluation.

Explanation theater. Plausible attribution can conceal an invalid peer group or future-contaminated feature. Show provenance and reference cohorts first.

The safe failure mode is usually to keep ranking while removing automated consequences. If feature freshness or peer coverage breaches its contract, mark results degraded; never silently substitute zeros.

Sources and Further Reading

- **Chandola, Banerjee, and Kumar, “Anomaly Detection: A Survey.”** Its point, contextual, and collective taxonomy determines the observation unit and what a detector can express. <https://doi.org/10.1145/1541880.1541882>
- **Liu, Ting, and Zhou, “Isolation Forest.”** The isolation mechanism explains scalability and why its score reflects partition geometry rather than attack likelihood. <https://doi.org/10.1109/ICDM.2008.17>
- **Breunig et al., “LOF.”** LOF motivates peer-relative density while exposing sensitivity to neighborhood size and scaling. <https://doi.org/10.1145/342009.335388>
- **scikit-learn, Novelty and Outlier Detection.** Its outlier/novelty distinction matters when a reference model scores future events. https://scikit-learn.org/stable/modules/outlier_detection.html
- **Zhao, Nasrullah, and Li, “PyOD.”** A common API enables controlled benchmarks, provided raw scores are not treated as probabilities. <https://jmlr.org/papers/v20/19-011.html>

Interview Practicum

Scenario: The Queue Changes While You Design It.

Prompt. A SaaS company has 800,000 daily identity-resource events, 70 confirmed compromises from the last year, and capacity to review 40 cases daily. Design anomaly detection.

Ask whether a case is an event, session, or identity-day; how identities and roles resolve; whether confirmed cases omit quiet compromises; what analysts can do; and whether blocking is allowed.

Worked Answer

Propose a ranking service. Build point-in-time identity-day and session features, partition human and non-human identities, compare robust peer distance with Isolation Forest, and allocate 34 slots to ranked cases plus six to stratified audit sampling. Evaluate incident rank, reviewed precision, identity coverage, duplication, and analyst minutes.

Follow-Up Questions and Answers

Midstream constraint. Legal prohibits retaining resource names beyond 24 hours, and the SOC loses half its capacity.

Tokenize resources with tenant-scoped rotating identifiers and persist only approved aggregates: category counts, entropy, novelty indicators, and sensitivity. A rotation boundary must not masquerade as new resources, so novelty needs a lawful stable category representation or removal. Refit because representation changed. Reduce the budget to 20, preserve at least three audit slots, and deduplicate related identity-days into cases.

Second change. An executive demands that scores above 0.95 automatically lock accounts.

Reject the semantics, not the executive. The anomaly score is not calibrated compromise probability. Offer bounded controls: require an independent signal, apply step-up rather than lockout, or automate only known policy violations. If automation is mandatory, calibrate a supervised decision layer on adjudicated outcomes and retain abstention; do not relabel an unsupervised score.

Weak answers choose an exotic model before defining the row, call unreviewed cases negative, set contamination to incident prevalence, or promise explainability by naming SHAP. Staff judgment protects audit sampling under capacity pressure. Principal judgment negotiates the consequence boundary and privacy-safe representation.

Key Takeaways

- Anomaly scores measure disagreement with a reference model, not maliciousness.
- Observation unit, context, peer group, and timing determine what is expressible.
- Compare methods at the same review budget and reserve below-threshold audits.
- Explanations must expose cohort, surprising features, missingness, and provenance.
- Require independent evidence before attaching high-consequence controls.

Chapter 26: Identity, Access, and Insider-Risk Analytics

The Session, Not the Login, Is the Security Boundary

A successful login proves only that an authentication ceremony satisfied policy. It does not prove that the human intended the action, that the token remained under their control, or that subsequent authorization was appropriate. Identity analytics must connect authentication, token issuance, session continuity, privilege, resource access, and recovery. Treating each login as an independent row misses what token theft and valid-account abuse exploit.

The operative unit here is an **access decision within a session**. Evidence accumulates across a session, but controls act at specific boundaries: issue a token, permit a resource, require step-up, reduce scope, revoke a session, or investigate. These actions have different error costs. A false step-up is friction; a false revocation can interrupt production; an accusation of insider misuse can harm a person. Analytics and governance must preserve those distinctions.

Identity risk is evidence with provenance, not a permanent property of a person. “This session reused a retired refresh-token family from a new device and requested a privileged scope” is reviewable. “User risk = 87” is not.

Authentication and Authorization Evidence

A minimal stream contains related records:

```
{
  "type": "authn",
  "event_id": "a1",
  "session_id": "s7",
  "subject_id": "u42",
  "occurred_at": "2026-08-20T14:03:00Z",
  "method": "webauthn",
  "device_id": "d9",
  "ip_prefix": "203.0.113.0/24",
  "country": "US",
  "result": "success",
  "assurance_level": 2
}
{
  "type": "token_issue",
  "event_id": "t1",
  "session_id": "s7",
  "subject_id": "u42",
  "occurred_at": "2026-08-20T14:03:02Z",
  "client_id": "portal",
  "token_family_id": "tf8",
  "scopes": ["read:billing"],
  "ttl_seconds": 3600
}
{
  "type": "authorization",
  "event_id": "z1",
  "session_id": "s7",
  "subject_id": "u42",
  "occurred_at": "2026-08-20T14:08:11Z",
  "resource_id": "invoice-91",
  "action": "read",
  "decision": "allow",
  "policy_id": "p14"
}
```

Production records also need tenant, issuer, credential surrogate, source and ingestion times, schema and policy versions, and workload identity where delegation occurs. Never store passwords, bearer tokens, cookies, or full device fingerprints in analytics. Token-family identifiers should be non-reversible and scoped so they cannot become cross-purpose tracking IDs.

Authentication and authorization form a temporal graph. Subjects control credentials; credentials start sessions; sessions receive tokens; tokens carry scopes; policies authorize actions. The graph answers questions an event table cannot: Which active sessions descend from a suspected token family? Which service principal

can assume a role granting a customer key? Which privileged action occurred after the last strong authentication?

Travel Is a Hypothesis, Not a Verdict

For consecutive observations $((\phi_1, \lambda_1, t_1))$ and $((\phi_2, \lambda_2, t_2))$, the haversine distance is

$$\left[\begin{aligned} d &= 2R \arcsin \sqrt{\sin^2 \frac{\Delta \phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta \lambda}{2}}, \\ \text{quad } v &= \frac{d}{t_2 - t_1}. \end{aligned} \right]$$

Calling (v) impossible ignores VPN egress, mobile networks, cloud proxies, geolocation error, NAT, and concurrent sessions. A better feature records minimum plausible transit time plus network ownership, device continuity, and token continuity. Distant IPs using the same refresh-token family concurrently are different evidence from independent browsers separated by a corporate VPN change.

Useful token-abuse evidence includes refresh-token reuse after rotation, a token presented by a new client or workload, audience/scope mismatch, continuation after credential reset, ASN transition without device continuity, and privileged use long after authentication. These signals derive from protocol semantics. Mere IP novelty is too unstable for severe action.

Sequential Evidence and Decay

Independent-event addition double-counts correlated signals. New IP, ASN, and country often describe one transition. Group evidence into mechanisms—network, device, token, privilege, behavior—and combine within groups before combining across them.

For a session at time (t) , a transparent accumulator is

$$\left[L_t = L_0 + \sum_g \min \left(C_g, \sum_{i \in g} w_{ie_i} \exp[-(t-t_i)/\tau_i] \right), \right]$$

where (L) is log-odds, (e_i) evidence strength, (w_i) a reviewed weight, (τ_i) decay, and (C_g) caps correlated evidence. $(\sigma(L_t))$ can be calibrated, but calibration must be evaluated by action and segment. The equation is not a causal claim; it is an inspectable combination of time-varying evidence.

Policy should consume evidence separately from estimation:

```
policy_version: identity-access-2026-08
rules:
  - when: token_reuse_after_rotation and privileged_scope
    action: revoke_token_family
    require_human_review: false
  - when: session_evidence >= 0.80 and action_sensitivity == "high"
    action: require_phishing_resistant_step_up
    max_added_latency_seconds: 90
  - when: telemetry_confidence < 0.70
```

```
action: preserve_existing_access
emit: identity_evidence_degraded
```

The first is a protocol violation with narrow consequence. The second protects a sensitive action rather than punishing every session. The third prevents a sensor outage becoming mass lockout.

Privilege Changes and Resource Context

An authorization event is meaningful only relative to effective entitlements. Store grants with `valid_from`, `valid_to`, grantor, source system, approval reference, role, resource scope, and whether the grant was standing or just-in-time. A privilege escalation can be explicit—a role assignment—or implicit, such as adding a principal to a group that can impersonate a workload.

“Unusual resource access” should condition on job function, project, sensitivity, entitlement age, and recent change. A new finance employee reading payroll on day one may be expected because the role and onboarding task authorize it. A tenured engineer reading a newly assigned repository may reflect an incident response. The strongest signals often join independent planes: a new grant, an unfamiliar session, and immediate bulk use of the new privilege.

Point-in-time SQL prevents future entitlements from explaining past access:

```
SELECT a.event_id, a.subject_id, a.resource_id, a.action, g.grant_id
FROM access_events AS a
LEFT JOIN entitlement_grants AS g
  ON g.subject_id = a.subject_id
  AND g.resource_id = a.resource_id
  AND g.valid_from <= a.occurred_at
  AND (g.valid_to IS NULL OR g.valid_to > a.occurred_at)
WHERE a.occurred_at >= :window_start
      AND a.occurred_at < :window_end;
```

A null grant may mean unauthorized access, incomplete inventory, or indirect entitlement. Preserve that uncertainty instead of immediately labeling policy bypass.

Human and Non-Human Identities Need Different Models

Service accounts do not sleep, travel, or respond to MFA. Their context is workload placement, caller, deployment provenance, secret age, expected audiences, API set, and schedule. A service principal used from an engineer laptop is suspicious even if volume is ordinary. A Kubernetes workload identity requesting a new role after deployment may be expected if manifest and change record agree.

Maintain owner, purpose, permitted workloads, credential type, rotation policy, expected resources, and expiration. Orphan status is governance evidence, but should trigger ownership remediation before a compromise verdict. Non-human identities often have large blast radius and weak recovery; adaptive access should use short-lived credentials and workload attestation, not copied human controls.

Emergency identities are a third class. Their use should create a high-priority event, but not because it is anomalous: it violates ordinary workflow by design. Validate sealed credentials, dual authorization where feasible, limited duration, comprehensive logging, and post-use review.

Insider Risk: Analyze Acts, Protect People

Insider-risk programs combine security, employment, privacy, and civil-liberties concerns. Focus on policy-relevant acts—bulk export from a restricted repository, unauthorized privilege change, use of a break-glass credential—not personality, sentiment, health, union activity, or broad productivity surveillance. Collect minimum evidence for an approved purpose. Separate triage from employment decisions. Require corroboration and trained human review before adverse consequence.

Access to insider-risk data should be more restrictive than ordinary SOC telemetry. Queries and exports need audit logs; retention should be short; investigator conflicts managed; subjects should have applicable correction channels. A model card should state excluded features and prohibited uses. Fairness review should examine whether location, disability accommodations, on-call schedules, or job family create systematically higher friction.

Safe Runnable Lab: Stateful Token Evidence

This lab detects a narrow token-theft pattern with synthetic events. It performs no network action.

```
from dataclasses import dataclass
from datetime import datetime, timezone

@dataclass(frozen=True)
class Use:
    at: datetime
    token_family: str
    device: str
    asn: int
    action: str
    privileged: bool = False
    rotated: bool = False

def assess_session(events):
    events = sorted(events, key=lambda e: e.at)
    last_context, retired, reasons, score = {}, set(), [], 0
    for event in events:
        if event.token_family in retired:
            score += 70
            reasons.append("token_reuse_after_rotation")
        prior = last_context.get(event.token_family)
        if prior and prior != (event.device, event.asn):
            score += 20
            reasons.append("token_context_changed")
        if event.privileged and score >= 70:
```

```

        score += 20
        reasons.append("privileged_action_after_strong_evidence")
        last_context[event.token_family] = (event.device, event.asn)
        if event.rotated:
            retired.add(event.token_family)
        action = "revoke_token_family" if score >= 90 else (
            "step_up" if score >= 40 else "allow")
        return min(score, 100), action, tuple(dict.fromkeys(reasons))

def ts(minute):
    return datetime(2026, 8, 20, 14, minute, tzinfo=timezone.utc)

benign = [Use(ts(0), "tf-good", "laptop-1", 64500, "login"),
          Use(ts(5), "tf-good", "laptop-1", 64500, "refresh", rotated=True)]
stolen = [Use(ts(0), "tf-bad", "laptop-2", 64500, "login"),
          Use(ts(4), "tf-bad", "laptop-2", 64500, "refresh", rotated=True),
          Use(ts(7), "tf-bad", "unknown", 64599, "admin_export", privileged=True)]

assert assess_session(benign) == (0, "allow", ())
score, action, reasons = assess_session(stolen)
print(score, action, reasons)
assert score == 100 and action == "revoke_token_family"
assert reasons == ("token_reuse_after_rotation", "token_context_changed",
                  "privileged_action_after_strong_evidence")

```

Expected output is `100 revoke_token_family (...)` with the asserted reasons. The detector is intentionally narrow. In production, rotation semantics vary, duplicate delivery must be idempotent, and events arrive late. Streaming needs state keyed by issuer and token-family surrogate, an event-time watermark, and correction when a late rotation event changes the conclusion.

Measuring Protection, Friction, and Recovery

Risk-based authentication has paired outcomes. Security measures include compromised sessions interrupted before sensitive action, time from strong evidence to revocation, coverage of privileged actions, and recurrence after recovery. Experience measures include step-up rate per active user, completion, tail delay, help-desk contacts, recovery duration, and abandonment. Segment by method, platform, geography, accessibility need, and workforce/customer population.

“Ninety percent of risky logins challenged” says nothing about ordinary users challenged or attackers succeeding afterward. Follow cohorts through challenge, completion, protected action, investigation, and recovery. Suppressed incidents are partially unobservable, so combine outcomes with red-team exercises, token-replay tests, and sampled allowed-session review.

Recovery is part of the control. Password reset may not affect stolen session tokens; revoking one token may leave descendants. Model credential reset, token-family revocation, device re-registration, privilege review, notification, and restored access. Measure whether legitimate users recover safely rather than merely whether the alert closes.

Production Failure Analysis

Clock and ordering errors. Token use may appear before issuance or after revocation. Retain source and ingestion times, define skew, and allow correction.

Shared infrastructure mistaken for identity. NAT and gateways create many users behind one IP. IP equality is not identity continuity.

MFA fatigue and recovery bypass. More challenges can train reflexive approval or push users to weaker recovery. Measure both.

Role-change blindness. Transfers invalidate old baselines. Join effective-dated context while limiting employment-data use.

Revocation storms. A bad ASN feed can revoke thousands. Cap action rates, canary policy versions, and degrade toward step-up when confidence falls.

Service-account ownership fiction. Inventory ownership can be stale. Require attestation and actual workload provenance.

Outcome bias. Challenged sessions receive more scrutiny than allowed sessions. Audit allowed sensitive actions and replay adversarial cases.

From Offline Evidence to a Safe Control

Historical replay must reconstruct what the policy knew at each access decision. Device trust, group membership, token status, and resource sensitivity are effective-dated facts. Joining current device compliance to an old session creates false confidence; joining a later incident verdict into a feature creates leakage. Store one decision record per protected action with evidence timestamps, unavailable fields, proposed action, actual action, and eventual outcome. This table supports calibration, friction analysis, and incident reconstruction.

The first rollout stage should be counterfactual logging: the candidate policy records what it would have done while the existing policy remains authoritative. Shadow evaluation identifies action volume and systematic gaps but cannot fully predict user response. Next, canary by a coherent unit—perhaps internal employees in one region—while excluding emergency accounts and workflows without tested recovery. Stop conditions should include challenge failure, support contacts, authentication latency, revocation volume, policy-engine errors, and segment-specific disparity, not only security incidents.

Adaptive controls need an action ladder. Observation creates evidence without disruption. Step-up asks for stronger proof at a sensitive boundary. Scope reduction issues a limited token. Shortened lifetime limits exposure. Revocation terminates a token family. Account disablement is a last resort with a different approval path. Each rung requires its own threshold, because costs and reversibility differ. A single score may inform several actions, but one threshold must not be copied across them.

Calibration should be checked among mature outcome cohorts and by identity class. If $\hat{p}_i$ is a predicted probability for a defined outcome, reliability asks whether approximately $\hat{p}$ of sessions in a probability bin experience that outcome. Sparse compromises make narrow bins misleading; use broad bins, uncertainty in-

tervals, and incident-based challenge tests. More importantly, evaluate decision value: did step-up complete, did it stop the adversary, did legitimate work continue, and did recovery eliminate descendant sessions?

Policy changes alter labels. Once high-evidence sessions are challenged, observed compromise among them may fall, making the detector appear worse if outcome alone is used. Retain pre-intervention evidence, challenge result, and red-team outcomes. Where safe, randomized variations in reversible actions can estimate friction and completion; do not randomize exposure to known severe compromise. The measurement design must recognize prevention as censoring rather than relabeling protected sessions benign.

An identity-control service also needs hard reliability boundaries. Cache policy carefully: stale allow decisions can extend access, while stale deny decisions can cause an outage. Define maximum evidence age by action sensitivity, fail behavior for each dependency, and a manual break-glass path whose use is independently audited. Load-test authentication peaks and recovery storms. A security control that cannot survive identity-provider degradation becomes an availability vulnerability.

Sources and Further Reading

- **NIST SP 800-63B.** Assurance and phishing resistance support matching ceremonies to action sensitivity; authentication is not permanent session trust. <https://pages.nist.gov/800-63-4/sp800-63b.html>
- **NIST SP 800-207.** Continuous evaluation implies authorization should reconsider session, device, and resource evidence rather than network location. <https://csrc.nist.gov/pubs/sp/800/207/final>
- **MITRE ATT&CK T1078, Valid Accounts.** Valid credentials and legitimate interfaces carry adversary activity malware-centric analytics miss. <https://attack.mitre.org/techniques/T1078/>
- **CISA Zero Trust Maturity Model.** The identity pillar connects authentication, enforcement, visibility, and governance; a risk score alone is not maturity. <https://www.cisa.gov/resources-tools/resources/zero-trust-maturity-model>
- **Microsoft Entra ID Protection Risk Detections.** The catalog supplies an evidence taxonomy, while deployment still needs local base rates and validation. <https://learn.microsoft.com/en-us/entra/id-protection/concept-identity-protection-risks>

Interview Practicum

Scenario: Adaptive Access Under Changing Constraints.

Prompt. Design real-time access-risk controls for a global development platform serving employees, contractors, customers, CI principals, and emergency administrators. Prevent stolen-token use without mass lockouts.

Clarify protocols and rotation, sensitive actions, phishing-resistant coverage, event latency, identity classes, recovery ownership, and allowed consequences.

Worked Answer

Propose session state consuming authentication, issuance, rotation, authorization, device, and workload events. Implement protocol-grounded token-reuse rules, then calibrated evidence for ambiguous transitions. Protect sensitive actions with step-up and revoke only on strong independent evidence. Use separate policies for humans and workloads.

Follow-Up Questions and Answers

Constraint change one. The provider cannot expose IP addresses in the EU.

Do not invent a weak proxy and call it equivalent. Use provider-computed coarse network-change claims if lawful, plus device-bound key continuity, token behavior, client identity, sensitivity, and workload attestation. Record regional feature availability and recalibrate regionally. The EU may have a wider abstention region; identical score distributions are not the objective.

Constraint change two. An acquisition adds 60,000 unmanaged contractors without device inventory.

Avoid declaring every device risky. Create a cold-start policy: short token TTL, phishing-resistant enrollment where possible, least-privilege scopes, step-up at sensitive actions, and sponsor/contract expiration. Measure completion and support load. Delay behavioral modeling until lawful history exists.

Constraint change three. During an incident, leadership asks to revoke every session over 0.7.

Ask what 0.7 means and show action-specific calibration. A step-up threshold is not a revocation threshold. A temporary scoped rule may revoke sessions descending from a known token family, with owner and expiration. Do not convert an ambiguous population score into emergency authority.

Weak answers center impossible travel, merge humans and services, ignore token lineage, or discuss precision without recovery. Staff answers design state and action tiers. Principal answers establish prohibited insider-risk uses, regional contracts, expiring emergency policy, and accountable ownership.

Key Takeaways

- Evaluate trust throughout sessions and at authorization boundaries, not only login.
- Prefer protocol-grounded evidence to unstable novelty signals.
- Separate human, workload, and emergency identities.
- Insider-risk analytics must target approved acts, minimize data, and preserve human review.
- Measure interrupted compromise together with friction and safe recovery.

Chapter 27: Network, Endpoint, Cloud, and Application Analytics

An Investigation Is a Join Across Partial Witnesses

No telemetry domain tells the whole story. A network flow sees endpoints and bytes but often not the process or authenticated user. Endpoint telemetry sees process ancestry but may lose traffic after a host sensor fails. Cloud audit records control-plane requests but not necessarily what happened inside a workload. Application logs know business objects and authorization outcomes but may not identify the originating container. Kubernetes audit records API decisions while runtime activity occurs elsewhere.

Cross-domain analytics therefore should not begin with an “entity risk score.” It begins with a claim that can be supported or contradicted by independent witnesses. Consider: “A stolen developer session was used to create a cloud credential, deploy a workload, and exfiltrate customer records.” The useful output is a time-ordered evidence bundle showing subject, process, workload, request, and network destination, with uncertainty at every join.

Correlation is not ordinary equality. Clocks differ, identifiers are reused, NAT collapses hosts, containers are ephemeral, and collectors duplicate or delay events. A reliable investigation timeline distinguishes observed facts from inferred links and records why each inference was made.

Domain-Specific Evidence

Network flow and encrypted metadata

A flow record should contain observation start/end, source and destination addresses and ports, transport, bytes and packets in each direction, sensor, sampling rate, network zone, and translated addresses when lawful and available. TLS metadata may include version, SNI or privacy-approved hostname category, certificate fingerprint, ALPN, and handshake outcome. JA3-like fingerprints can support hunting but are neither stable identities nor maliciousness labels; libraries and middleboxes create collisions.

Useful derived values include duration, byte ratio, packet-size statistics, fan-out, failed-connection rate, periodicity, and destination novelty. For event times $(t_1, \dots, t_n)$, beacon regularity can be summarized by coefficient of variation

$$\sqrt{CV = \frac{\text{sd}(t_i - t_{i-1})}{\text{mean}(t_i - t_{i-1})}}.$$

Low (CV) is evidence of periodicity, not command-and-control: health checks and agents are periodic too. Destination reputation should retain observation date because domains and addresses change ownership.

Endpoint process trees

Process analytics require boot-scoped identity. A PID alone is unsafe because operating systems reuse it. A process key should combine device, boot identifier, PID, and start time. Record executable path and hash,

signer, command-line representation subject to privacy controls, parent process key, user or workload identity, integrity level, and termination time.

Sequences often carry more meaning than individual names. `winword -> powershell -> rundll32` is different from a management agent invoking PowerShell from an approved path, even if each binary is common. Represent ancestry as typed edges and preserve missing-parent status. If collection begins after process start, inventing a root creates false confidence.

Cloud control plane

Cloud events need provider, account/project/subscription, region, principal, credential/session, API operation, resource, source context, user agent, request parameters with secret redaction, response status, policy decision, and request ID. Separate control-plane creation from data-plane use. Creating an access key and using it are related but distinct evidence.

High-value patterns include disabling logs before privilege changes, creating credentials followed by new-region use, modifying trust policies, snapshot sharing, and unusual role assumption. Baseline by automation identity and deployment pipeline; infrastructure-as-code produces legitimate bursts that look extraordinary beside human administration.

Application and API activity

Application logs supply business semantics: tenant, authenticated principal, session, route template, object type, authorization decision, response class, records returned, and trace identifier. Never aggregate by raw URL when it embeds customer IDs. API abuse may appear as object enumeration, pagination manipulation, unusual error transitions, or a token exercising a permission it historically held but never used.

Kubernetes and workload identity

Kubernetes audit events identify user, groups, verb, object reference, namespace, request and response status, source IPs, stage, audit ID, and impersonation. Join pod UID—not merely pod name—to container runtime, image digest, node, service account, and cloud workload identity. Names are reusable; UIDs and validity intervals prevent yesterday's pod from being joined to today's traffic.

A Canonical Evidence Envelope

Do not flatten every source into a lowest-common-denominator table. Preserve source payloads and normalize only fields needed for correlation:

```
{
  "event_id": "cloud:01J5...",
  "event_type": "cloud.credential.create",
  "observed_at": "2026-08-20T15:04:12.882Z",
  "ingested_at": "2026-08-20T15:04:15.104Z",
```

```

"producer":"aws-cloudtrail",
"tenant_id":"acme",
"actor":{"type":"human","id":"u-17","session_id":"s-91"},
"object":{"type":"access_key","id":"key-tokenized-4"},
"trace":{"request_id":"req-8","trace_id":null},
"network":{"source_ip":"198.51.100.8","asn":64520},
"source_schema":"cloudtrail-1.09",
"normalizer_version":"3.4.1",
"payload_ref":"object://immutable/evidence/01J5..."
}

```

This envelope resembles CloudEvents in separating event identity, source, type, subject, and time; OpenTelemetry supplies trace/span conventions for instrumented applications; ECS provides a broad field vocabulary. None removes the need for local semantics. `user.id`, for example, can mean an employee, customer, operating-system account, or workload. Record namespace and identity type.

Correlation as Bounded Inference

The strongest join uses a propagated trace or request ID. Next are cryptographically or platform-bound identifiers such as session ID, pod UID, or cloud request ID. Weaker joins use shared device, IP, user agent, and time. Every inferred link should carry a method and confidence.

If source (A) has clock-error bound (ϵ_A) and source (B) has (ϵ_B) , events can be temporally compatible when

$$[t_A - \epsilon_A, t_A + \epsilon_A] \cap [t_B - \epsilon_B, t_B + \epsilon_B] \neq \emptyset$$

For an expected causal delay $([d_{\min}, d_{\max}])$, shift the second interval accordingly. A fixed “within five minutes” join hides whether clocks are accurate to milliseconds or hours. Estimate skew from known paired events—load-balancer request IDs and application traces, for example—and monitor it per source.

Candidate-link confidence can be an auditable log-likelihood ratio:

$$-\log \frac{P(M \mid \text{same})}{P(M \mid \text{different})},$$

where (M) contains identifier match, temporal compatibility, topology, and expected sequence. Do not pretend independence among IP, subnet, and ASN. For investigations, expose the components rather than only the combined value.

SQL for a Point-in-Time Timeline

The following SQLite-compatible query constructs one evidence timeline using explicit keys. It avoids a many-to-many time-only join.

```

WITH scoped_sessions AS (
  SELECT session_id, subject_id, device_id, started_at, ended_at

```

```

FROM identity_sessions
WHERE subject_id = :subject_id
    AND started_at < :investigation_end
    AND COALESCE(ended_at, :investigation_end) >= :investigation_start
),
timeline AS (
    SELECT e.occurred_at, 'endpoint' AS domain, e.event_id,
           e.process_key AS object_id, s.session_id, 'device_session' AS link_method
    FROM endpoint_events e
    JOIN scoped_sessions s ON s.device_id = e.device_id
    AND e.occurred_at >= s.started_at
    AND e.occurred_at < COALESCE(s.ended_at, :investigation_end)
    UNION ALL
    SELECT c.occurred_at, 'cloud', c.event_id, c.resource_id,
           c.session_id, 'session_id'
    FROM cloud_events c JOIN scoped_sessions s ON s.session_id = c.session_id
    UNION ALL
    SELECT a.occurred_at, 'application', a.event_id, a.object_id,
           a.session_id, 'session_id'
    FROM application_events a JOIN scoped_sessions s ON s.session_id = a.session_id
)
SELECT occurred_at, domain, event_id, object_id, session_id, link_method
FROM timeline
ORDER BY occurred_at, domain, event_id;

```

Network flows need a separate mapping from device interface and validity interval; joining every flow by a shared public IP would be indefensible. All tables should retain `ingested_at` so investigators can tell whether an event was available to the original detector or only arrived later.

Safe Runnable Lab: Build an Evidence Timeline

This standard-library lab correlates synthetic records using explicit session and pod identifiers, then asserts the hypothesized sequence.

```

from datetime import datetime, timezone

def instant(value):
    return datetime.fromisoformat(value.replace("Z", "+00:00"))

events = [
    {"id": "e1", "time": "2026-08-20T15:00:00Z", "domain": "endpoint",
     "kind": "shell_start", "session": "s91", "pod": None},
    {"id": "c1", "time": "2026-08-20T15:02:00Z", "domain": "cloud",
     "kind": "credential_create", "session": "s91", "pod": None},
    {"id": "k1", "time": "2026-08-20T15:05:00Z", "domain": "kubernetes",
     "kind": "pod_create", "session": "s91", "pod": "pod-uid-7"},
    {"id": "a1", "time": "2026-08-20T15:07:00Z", "domain": "application",
     "kind": "bulk_export", "session": None, "pod": "pod-uid-7"},
    {"id": "n1", "time": "2026-08-20T15:08:00Z", "domain": "network",

```

```

    "kind":"large_egress","session":None,"pod":"pod-uid-7"},
    {"id":"noise","time":"2026-08-20T15:06:00Z","domain":"network",
     "kind":"large_egress","session":None,"pod":"pod-uid-other"},
  ]

def correlated(seed_session, rows):
    selected = {r["id"] for r in rows if r["session"] == seed_session}
    pods = {r["pod"] for r in rows if r["id"] in selected and r["pod"]}
    changed = True
    while changed:
        changed = False
        for row in rows:
            if row["id"] not in selected and row["pod"] in pods:
                selected.add(row["id"]); changed = True
                if row["pod"] in pods: pods.add(row["pod"])
    return sorted((r for r in rows if r["id"] in selected),
                  key=lambda r: (instant(r["time"]), r["id"]))

timeline = correlated("s91", events)
kinds = [row["kind"] for row in timeline]
print(" -> ".join(kinds))
assert kinds == ["shell_start", "credential_create", "pod_create",
                 "bulk_export", "large_egress"]
assert "noise" not in {row["id"] for row in timeline}
assert all(instant(timeline[i]["time"]) <= instant(timeline[i+1]["time"]))
           for i in range(len(timeline)-1))

```

Expected output is `shell_start -> credential_create -> pod_create -> bulk_export -> large_egress`. The unrelated flow shares time and event type but not pod identity, so it is excluded. A production implementation should store each link as an edge with `link_method`, confidence, and evidence references; transitive expansion must have domain and time bounds to avoid swallowing an entire tenant.

From Timeline to Detection

Once evidence is linked, detection can operate on sequences rather than additive points. A finite-state rule for the constructed case might require credential creation by an interactive identity, workload creation using the credential within 15 minutes, sensitive export by that workload, and egress above a tenant-relative threshold. Each transition has a maximum delay and compatible identity relation.

Entity aggregation should preserve mechanisms. Summing “endpoint severity 40 + network severity 30 + cloud severity 50” creates a false numerical precision and lets ten weak correlated alerts outweigh one protocol violation. Prefer a small set of reasoned hypotheses with supporting/contradicting evidence. Deduplicate downstream alerts into an investigation keyed by incident hypothesis, not merely actor.

For streaming, event-time state must tolerate late arrivals. A watermark closes a window operationally, not epistemically; late evidence may revise a timeline. Emit revision numbers and avoid destructive overwrites. Analysts need to know that version 3 added a cloud event received 47 minutes late.

Production Failure Analysis

Identifier collision. PID, container name, username, and IP are reusable. Use boot IDs, UUIDs, namespaces, and validity intervals.

Clock-skew fiction. A generous time window creates plausible but false stories. Measure per-source skew and show temporal uncertainty.

Cardinality explosion. Joining shared IPs or generic service accounts produces millions of links. Require stronger keys or cap fan-out and label ambiguity.

Normalization loss. Flattening source payloads discards request parameters needed later. Keep immutable raw evidence with access controls and retention.

Sensor-selective truth. Managed endpoints have rich evidence while unmanaged workloads appear quiet. Report coverage and never interpret missing telemetry as benign.

Duplicate cascades. Retries and multi-stage audit records multiply one action. Deduplicate with source IDs and documented stage semantics.

Collection-induced outage. Verbose application or Kubernetes auditing can overload systems or leak secrets. Redact at source, sample only where justified, and load-test policies.

Cross-tenant leakage. Correlation stores are powerful. Enforce tenant predicates in storage and service authorization, not only UI filters.

Operating a Correlation Pipeline

Each producer needs a measurable contract. Completeness can be approximated through sequence numbers, expected heartbeats, or comparison with an independent control-plane counter. Freshness should be measured as both source delay and pipeline delay: `observed_at - occurred_at` captures producer behavior, while `available_at - observed_at` captures transport and processing. Combining them into one latency hides where remediation belongs. Track p50, p95, and p99 by source, region, tenant class, and event type.

Schema changes should enter through versioned adapters and golden fixtures. A source field renamed from `principalId` to a nested object can silently null an identity join while event volume remains healthy. Contract tests should assert semantics: a known role-assumption fixture resolves the expected actor; a pod recreation generates a new UUID; a denied API action remains denied after normalization. Preserve unknown enum values rather than mapping them to a familiar category.

Correlation rules need replayable versions. Store the normalized event IDs, rule version, state-machine transition, link methods, and window bounds for every emitted investigation. A rule edit should run against a fixed historical corpus and report changed cases, not only unit-test success. Because historical raw data may be sensitive or expensive, maintain small de-identified golden traces for common and adversarial sequences, plus access-controlled sampled replays for release review.

Queue behavior is part of validation. Measure how many source alerts consolidate into each investigation, how often separate incidents are incorrectly merged, and how often analysts split a case. Merge errors are

asymmetric: over-merging creates an incoherent mega-incident; under-merging repeats work and hides a campaign. Provide reversible merge/split operations and feed those corrections into rule review without treating every analyst edit as ground truth.

Response automation should attach to verified transitions, not the mere existence of a cross-domain story. A known compromised token creating a credential may justify token revocation; a probabilistic IP/time link should not terminate a production workload. The investigation object should state which actions are permitted by which evidence class. If a required source becomes stale, keep the partial timeline, downgrade the hypothesis, and disable only dependent automation.

Retention tiers can reduce cost and exposure. Keep normalized envelopes and link edges for the investigation horizon; retain high-cardinality payloads for shorter, source-specific periods; preserve legal or incident evidence through explicit holds. Derived timelines must point to payload availability so an old explanation does not promise evidence already deleted. Hashes can verify integrity, but a hash of deleted evidence does not make the explanation independently reviewable.

Sources and Further Reading

- **OpenTelemetry Semantic Conventions.** Trace and span conventions can provide strong application links, but security fields and identity namespaces still require explicit local contracts. <https://opentelemetry.io/docs/specs/semconv/>
- **CloudEvents Specification.** Its event envelope supports interoperable routing without requiring destructive flattening of domain payloads. <https://github.com/cloudevents/spec>
- **MITRE ATT&CK Data Sources.** ATT&CK connects behaviors to observable components and helps expose coverage gaps; it is not a guarantee that a listed sensor observes every implementation. <https://attack.mitre.org/datasources/>
- **Kubernetes Auditing Documentation.** Audit stages, policy levels, and request/response capture explain both the analytical value and the performance/privacy cost of Kubernetes evidence. <https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/>
- **Elastic Common Schema.** ECS offers useful shared names across domains; teams must still preserve source semantics, validity, and identity type. <https://www.elastic.co/guide/en/ecs/current/ecs-reference.html>

Interview Practicum

Scenario: Correlation When a Sensor Disappears.

Prompt. A customer reports that a developer account created a credential, deployed a pod, and exported data. Design a real-time cross-domain detector and an investigation view.

Ask which identifiers propagate, source latency/skew, retention, workload identity mapping, sensitive application actions, expected automation, and response authority.

Worked Answer

Propose canonical envelopes plus immutable payloads; build explicit session, request, pod UID, and workload-identity links; represent the hypothesis as a bounded state machine; emit an evidence graph and ordered timeline. Backtest point-in-time and measure confirmed-hypothesis precision, stage coverage, time to coherent evidence, duplicate investigations, and analyst correction rate.

Follow-Up Questions and Answers

Midstream constraint. Endpoint telemetry is unavailable on developer laptops for 90 days.

Do not claim equivalent coverage. Start the sequence at identity/cloud evidence, require stronger session and phishing-resistant authentication context, and label endpoint stage unknown. Increase audit sampling and consider a narrow preventive control on credential creation. Report the exact attacks that are no longer observable, such as local process ancestry.

Second constraint. Network telemetry is sampled 1:100 and NAT mappings are retained only one hour.

Use network evidence as corroboration, not required identity proof. Correct volume estimates only where sampling design supports it; retain uncertainty. Capture approved workload-to-flow mapping within the hour, aggregate privacy-preserving destination categories, and allow the detector to conclude “export observed, egress unconfirmed.”

Third constraint. Platform engineering says full Kubernetes request bodies add unacceptable API-server load.

Select audit levels by high-risk verbs and resources, exclude secret bodies, and measure policy overhead in a canary. Rely on object references, actor, verb, response, audit ID, and deployment provenance. The answer is a coverage/cost decision, not “log everything.”

Weak answers join on IP and time, flatten all logs into one score, or treat absent sensors as negative evidence. Staff answers reason about identifiers, clocks, state, and revision. Principal answers negotiate minimum viable telemetry, quantify residual blind spots, and move controls to boundaries that remain observable.

Key Takeaways

- Cross-domain analytics builds evidence for hypotheses; it does not merely sum alerts.
- Preserve domain payloads while normalizing a small correlation envelope.
- Prefer propagated and platform-bound identifiers; time/IP joins are weaker inferences.
- Model clock error, late data, identifier validity, and timeline revisions explicitly.
- Missing sensors reduce what can be claimed and may justify moving controls to observable boundaries.

Chapter 28: Graph Analytics for Attack Paths

An Attack Path Is a Satisfiable Program

A security graph is not valuable because it draws many arrows. It is valuable when it can answer a counterfactual: if an attacker controls this principal under these conditions, which protected assets become reachable, why, and which change breaks the most credible paths?

Ordinary graph reachability is too permissive. An identity may belong to a group only during one interval; a role may require MFA; an exploit may apply only to a vulnerable image version; a network route may permit traffic in one direction; two permissions may need to be exercised in sequence. A reported path is credible only when its edges are simultaneously valid and its prerequisites are satisfiable. Treating all edges as timeless produces attack paths that never existed.

The graph is heterogeneous. Nodes include people, service principals, groups, roles, credentials, workloads, hosts, applications, data stores, vulnerabilities, and controls. Edges include membership, trust, assume-role, owns, can-read, can-write, can-execute, exposed-to, runs-on, vulnerable-to, and blocked-by. Each type has different semantics. A shortest path across untyped edges is usually a shortest explanation of the data model, not attacker effort.

Schema: Facts, Validity, and Preconditions

A compact relational representation is easier to test than an opaque graph import:

```
CREATE TABLE graph_nodes (
  tenant_id TEXT NOT NULL,
  node_id TEXT NOT NULL,
  node_type TEXT NOT NULL,
  criticality INTEGER NOT NULL DEFAULT 0,
  properties_json TEXT NOT NULL DEFAULT '{}',
  PRIMARY KEY (tenant_id, node_id)
);

CREATE TABLE graph_edges (
  tenant_id TEXT NOT NULL,
  edge_id TEXT NOT NULL,
  src_id TEXT NOT NULL,
  dst_id TEXT NOT NULL,
  edge_type TEXT NOT NULL,
  valid_from TEXT NOT NULL,
  valid_to TEXT,
  confidence REAL NOT NULL CHECK (confidence >= 0 AND confidence <= 1),
  precondition_json TEXT NOT NULL DEFAULT '{}',
  evidence_ref TEXT NOT NULL,
  observed_at TEXT NOT NULL,
```



```
PRIMARY KEY (tenant_id, edge_id)
);
```

Edges are claims with evidence, not eternal truth. `valid_from` and `valid_to` describe the world; `observed_at` describes when the system learned it. Both are required for historical reconstruction and point-in-time evaluation. A scanner result should connect a workload instance or image digest to a vulnerability with version evidence. A role edge should carry conditions such as external ID, source principal, MFA, session tag, or namespace.

Negative and control facts also matter. A firewall deny, permissions boundary, explicit deny, or non-exportable key can invalidate a candidate path. Absence of an allow edge is not necessarily a deny when collection is incomplete. Record coverage so traversal can return `unknown` instead of quietly treating missing data as safe or exploitable.

Traversal Semantics

Define an attacker state $\backslash(q=(K,C,t)\backslash)$, where $\backslash(K\backslash)$ is controlled principals/resources, $\backslash(C\backslash)$ is acquired capabilities, and $\backslash(t\backslash)$ is evaluation time. An edge $\backslash(e\backslash)$ is traversable when:

$$\backslash[\text{valid_e}(t)\backslash \wedge \text{prereq_e}(C)\backslash \wedge \text{scope_e}(K)\backslash \wedge \neg \text{blocked_e}(t). \backslash]$$

Traversal applies a state transition $\backslash(q' = T_e(q)\backslash)$, perhaps adding a credential, execution capability, or resource control. This captures an important distinction: reading a secret grants a capability that can enable a later assume-role edge; merely being adjacent to the secret node does not.

For static IAM analysis, typed reachability may suffice. For exploit paths, use constrained search over state. Bound path length and prevent repeated states, not merely repeated nodes, because revisiting a node with a new capability can be meaningful. Keep the transition library small and reviewed. Overly expressive preconditions make results impossible to validate.

Centrality, Chokepoints, and Blast Radius

Degree finds hubs, but directory groups and scanners are often benign hubs. Betweenness centrality

$$\backslash[C_B(v) = \sum_{s \neq v \neq t} \frac{1}{\sigma_{st}(v)} \frac{1}{\sigma_{st}} \backslash]$$

counts the fraction of shortest paths passing through $\backslash(v\backslash)$. It can identify chokepoints, but only under the chosen edge weights and path definition. In security, the most important path may not be shortest, and thousands of near-duplicate group paths can dominate.

Blast radius is more actionable: given initial control $\backslash(S\backslash)$, count or value critical assets reachable under valid transitions. Weighting asset value yields

$$\backslash[BR(S) = \sum_{a \in \text{Reach}(S)} V(a). \backslash]$$

For remediation edge $\backslash(e\backslash)$, marginal path reduction is $\backslash(\Delta_e = BR(S) - BR_{\backslash\{e\}\backslash}(S)\backslash)$. This estimates leverage, not guaranteed risk reduction. Removing one group membership may break 300 enumerated paths

that all represent the same underlying privilege. Report unique protected assets and independent mechanisms as well as raw path count.

Communities can expose trust zones that do not match intended architecture. Motifs—user to group to role, workload to secret to cloud account—can encode known escalation patterns. Their value comes from typed semantics, not graph-theory novelty.

Ranking Paths Without Fake Probability

Suppose each edge has estimated success $\log(p_e)$, detection probability $\log(d_e)$, time cost $\log(c_e)$, and confidence $\log(r_e)$. A candidate score might be

$$J(P) = \sum_{e \in P} -\log(p_e) + \lambda \sum c_e + \gamma \sum d_e + \eta \sum -\log(r_e).$$

Lower is easier or better supported. This additive form implicitly treats factors as separable; shared credentials, controls, and evidence violate independence. Use it for ranking and sensitivity analysis, never as an exact breach probability. Show how rankings change when uncertain weights vary.

A practical priority should include starting-point exposure, target criticality, path feasibility, evidence freshness, existing detections, and remediation leverage. Internet exposure is not a universal starting point: phishing, insider access, vendor compromise, and workload compromise create other starts. Maintain scenario-specific rankings.

Graph embeddings and graph neural networks can rank nodes or missing links, but they introduce identity leakage, temporal leakage, and difficult explanations. A random edge split lets the same identity neighborhood appear in train and test. Evaluate on later graph snapshots and new entities. Begin with typed traversal and transparent path features; add learned representations only for measurable lift on adjudicated tasks.

A Unique Architecture for Temporal Truth

```

flowchart LR
    S[Source snapshots<br/>IAM, CMDB, scanners, network policy] --> F[Versioned edge facts]
    F --> T[Temporal + prerequisite validator]
    T --> Q[Scenario search<br/>initial control and target]
    Q --> P[Candidate paths]
    P --> E[Evidence expansion<br/>why each hop works]
    E --> R[Edge-removal simulation]
    R --> C[Remediation choice]
    C --> S
  
```

The validator is the important component. It rejects edges outside the evaluation time, expands group and trust semantics, checks required capabilities, applies explicit denies, and records unknown coverage. Search and ranking should never bypass it.

Safe Runnable Lab: Temporal Privilege Paths

This standard-library lab finds feasible paths and simulates remediation. The graph is synthetic and contains no real cloud credentials.

```
from collections import defaultdict, deque

edges = [
    ("internet", "web", "exposed", 1.0),
    ("web", "web-role", "workload_identity", 1.0),
    ("web-role", "build-secret", "read_secret", 1.4),
    ("build-secret", "ci-role", "credential_for", 0.8),
    ("ci-role", "signing-key", "can_sign", 1.0),
    ("contractor", "dev-group", "member_of", 0.2),
    ("dev-group", "ci-role", "assume_role", 1.2),
    ("old-group", "signing-key", "can_sign", 0.1), # expired
]
valid = {tuple(e[:3]) for e in edges if e[0] != "old-group"}

def paths(source, target, removed=None, cutoff=6):
    removed = removed or set()
    graph = defaultdict(list)
    costs = {}
    for src, dst, kind, cost in edges:
        key = (src, dst, kind)
        if key in valid and key not in removed:
            graph[src].append((dst, kind))
            costs[key] = cost
    queue = deque([(source, [source], [], 0.0)])
    found = []
    while queue:
        node, nodes, hops, cost = queue.popleft()
        if node == target:
            found.append((cost, nodes, hops)); continue
        if len(hops) >= cutoff: continue
        for nxt, kind in graph[node]:
            if nxt not in nodes:
                key = (node, nxt, kind)
                queue.append((nxt, nodes+[nxt], hops+[kind], cost+costs[key]))
    return sorted(found)

internet_paths = paths("internet", "signing-key")
contractor_paths = paths("contractor", "signing-key")
cut = {("dev-group", "ci-role", "assume_role")}
after_cut = paths("contractor", "signing-key", removed=cut)

print(internet_paths[0][1])
print(contractor_paths[0][1])
assert internet_paths[0][1] == ["internet", "web", "web-role",
                                "build-secret", "ci-role", "signing-key"]
assert contractor_paths[0][1] == ["contractor", "dev-group", "ci-role",
                                   "signing-key"]
```

```
assert after_cut == []
assert paths("old-group", "signing-key") == []
```

Expected output is the two asserted node lists. Removing `dev-group -> ci-role` eliminates the contractor path, while the expired edge never appears. The lab deliberately separates validity from topology. A production extension would represent prerequisites and capabilities explicitly and test that every explanatory hop has accessible source evidence.

Explainable Findings and Remediation Choice

A finding should read like a proof:

1. The contractor is an active member of `dev-group` according to directory snapshot X.
2. The group's trust policy can assume `ci-role` without MFA under policy version Y.
3. `ci-role` can invoke signing with key Z under effective policy simulation.
4. The key protects production release artifacts and has criticality 5.
5. Removing membership M breaks this scenario and two other independent paths; removing public web exposure does not affect it.

Include evaluation time, source freshness, unknown checks, and alternative paths. Analysts should be able to replay the query. If an edge depends on a scanner heuristic, label it accordingly. If a control is assumed effective but untested, do not present it as a hard block.

Remediation is a constrained cut problem. The smallest cut may be operationally impossible—removing the CI role stops releases. Candidate actions have engineering cost, availability impact, owner, and time to implement. Optimize a portfolio:

$$\max_{A} \text{critical reachability removed}(A) \quad \text{s.t.} \quad \sum_{a \in A} \text{cost}(a) \leq B,$$

plus constraints preserving required business paths. Often the best action is to condition a trust edge, shorten credential lifetime, or add approval rather than delete access entirely.

Production Failure Analysis

Timeless edges. Historical grants and remediated vulnerabilities invent paths. Require validity intervals and snapshot reproducibility.

Path explosion. Nested groups and equivalent permissions create combinatorial duplicates. Canonicalize equivalent mechanisms, cap depth, and summarize by target and cut edge.

Missing negative controls. Ignoring explicit deny, network policy, MFA conditions, or key non-exportability overstates feasibility. Model them or return unknown.

Stale ownership and criticality. A graph can rank precisely against obsolete business context. Require owner attestation and freshness indicators.

Tenant boundary failure. One missing predicate can create cross-customer paths. Partition storage and authorize traversal at the service layer.

Scanner confidence laundering. Importing a vulnerability edge can make uncertain version matching look factual. Preserve scanner evidence and confidence.

Remediation metric gaming. Teams close many paths by deleting duplicate edges while critical reachability remains. Measure protected assets and independent scenarios.

Learned leakage. Embeddings memorize high-degree identities and graph snapshots. Use temporal/entity holdouts and compare against typed heuristics.

Testing the Graph as a Security Product

Graph correctness begins below path ranking. Adapter tests should prove that source facts become the right typed edges, validity intervals, and conditions. Semantic tests should cover nested groups, explicit deny precedence, cross-account trust, inherited Kubernetes roles, workload federation, secret versioning, and vulnerability applicability. Use tiny graphs with one intended answer; large realistic fixtures make a wrong path difficult to diagnose.

Metamorphic tests are particularly effective. Adding an unrelated node must not change existing reachability. Expiring an edge must remove every path that depends on it. Adding an explicit deny must not create a path. Renaming a display label must not change node identity. Duplicating equivalent evidence must not double path count. Removing a cut edge must not affect required paths that do not traverse it. These properties catch entire classes of ingestion and traversal defects.

For a sampled production finding, a reviewer should reproduce each hop in the authoritative system or its immutable snapshot. Track edge confirmation rate, invalid-path rate, unknown-control rate, stale-edge rate, time to owner identification, accepted remediation, and verified reachability after change. “Paths closed” is insufficient until the graph is refreshed and the same scenario search confirms that the target is no longer reachable—or explains an alternative.

Freshness has scenario-specific limits. A 24-hour CMDB delay may be acceptable for quarterly architecture review but dangerous for a just-in-time signing path. Give every edge type a freshness SLO and propagate the weakest freshness into the path explanation. If one critical edge is stale, rank may remain useful for investigation, but automation should stop. Snapshot publication should be atomic so a query never mixes half of yesterday's IAM state with half of today's.

Large graphs require engineering discipline rather than semantic shortcuts. Partition by tenant and administrative boundary, maintain reverse indexes from protected targets, precompute safe closures for stable group membership, and run bidirectional search from starts and targets. Cache results by graph snapshot and scenario, invalidating them on affected edge changes. Approximate algorithms may prioritize candidates, but final findings should pass exact semantic validation.

Treat the graph service as highly sensitive. It is an attacker roadmap containing privilege, vulnerability, and critical-asset relationships. Apply least privilege, purpose-bound query APIs, tenant isolation, result limits,

audit trails, and controls on bulk export. Explanations shown to broad engineering audiences may need to redact unrelated identities while preserving remediation evidence. Protect source credentials and never let the analytics service acquire the privileges it is modeling merely for convenience.

Sources and Further Reading

- **BloodHound OpenGraph Documentation.** BloodHound demonstrates how identity and privilege relations become traversable attack paths; production use still requires local temporal, control, and evidence semantics. <https://bloodhound.specterops.io/opengraph/overview>
- **Neo4j Graph Data Science Manual.** Its path, centrality, community, and embedding algorithms are useful primitives, but algorithmic output inherits the graph projection's assumptions. <https://neo4j.com/docs/graph-data-science/current/>
- **NetworkX Algorithms Reference.** NetworkX is appropriate for small reproducible fixtures and algorithm tests; it is not a substitute for production authorization or tenant isolation. <https://networkx.org/documentation/stable/reference/algorithms/index.html>
- **MITRE ATT&CK Enterprise Matrix.** ATT&CK helps describe attacker transitions and evidence requirements, but techniques are not automatically graph edges without environment-specific prerequisites. <https://attack.mitre.org/matrices/enterprise/>
- **NIST SP 800-207.** Zero Trust's resource and policy emphasis supports modeling conditional access and continuous verification instead of assuming network adjacency equals trust. <https://csrc.nist.gov/pubs/sp/800/207/final>

Interview Practicum

Scenario: The Best Cut Becomes Unavailable.

Prompt. A cloud company has 12 million privilege edges. Find and prioritize paths from internet-facing workloads and contractors to production signing keys, then recommend remediation.

Clarify graph semantics, snapshot history, starting scenarios, target inventory, cloud-policy evaluation, vulnerability evidence, control representation, maximum path depth, and remediation owners.

Worked Answer

Build versioned typed edges with validity and evidence; define attacker-state transitions; enumerate bounded feasible paths; rank by scenario exposure, feasibility, target value, freshness, and uncertainty; then simulate candidate cuts. Validate a sample hop by hop with IAM and service owners.

Follow-Up Questions and Answers

Midstream constraint. The apparent top cut—removing the CI role—would halt releases and is prohibited.

Reframe remediation as constrained optimization. Test narrower changes: remove contractor inheritance, require workload-bound federation, restrict signing conditions to approved pipeline claims, shorten sessions, or add a brokered signing service. Recompute reachability while asserting that the required release path remains. The goal is not minimum graph size; it is reduced unauthorized reachability with preserved business function.

Second constraint. IAM snapshots are daily, while just-in-time grants last 30 minutes.

Daily snapshots cannot prove or exclude those paths. Ingest grant lifecycle events or policy-decision logs with event time. Until then, label JIT traversal unknown and avoid claiming complete coverage. A temporary control can require signing-service logs and alert on non-pipeline callers at the target boundary.

Third constraint. Leadership wants to deploy a GNN because the graph is large.

Ask which supervised task and labels justify it. Scaling traversal is an engineering problem; it does not itself require learning. Partition by tenant/account, precompute reviewed relation closures, and use bidirectional bounded search. Consider a GNN only for a defined ranking task with temporal/entity holdouts, transparent baselines, and path-level explanations. Never allow a learned score to invent authorization semantics.

Weak answers run generic shortest path, multiply edge probabilities as independent, rank by degree, or optimize raw path count. Staff answers model prerequisites, temporal validity, evidence, and operational cuts. Principal answers protect required business paths, expose unknown coverage, choose target-boundary controls when graph data is late, and resist unjustified model complexity.

Key Takeaways

- A credible attack path is a sequence of satisfiable, simultaneously valid transitions.
- Typed edges need evidence, confidence, validity, preconditions, and control semantics.
- Centrality and shortest path are heuristics; prioritize scenario reachability and remediation leverage.
- Explain findings as replayable proofs and distinguish false, blocked, and unknown paths.
- Optimize remediation under business constraints, then recompute reachability to verify the cut.

Chapter 29: Threat Intelligence and Exploit Prioritization

Exploitation Is a Forecast, Not a Severity Label

A scanner answers “what weakness is present?” A remediation program must decide which exposed weakness is likely to be used against this organization before it can be fixed. CVSS, EPSS, CISA Known Exploited Vulnerabilities (KEV), exploit reports, asset exposure, and business context describe different parts of that decision. Treating any one as the answer causes predictable mistakes.

CVSS v4.0 characterizes technical severity. EPSS estimates the probability that a published CVE will be exploited in the wild during the next 30 days. KEV says exploitation has occurred and assigns US federal remediation dates. None knows whether the vulnerable code path is reachable in *your* deployment, a control blocks it, or compromise reaches a material asset. The correct unit is therefore a time-stamped **CVE–asset–deployment observation**, not a CVE. The same CVE can require emergency work on an internet-facing identity gateway and normal maintenance on an isolated build host.

An Intelligence Contract That Preserves Evidence

Store observations separately from conclusions:

```
CREATE TABLE exploit_evidence (
  evidence_id TEXT PRIMARY KEY,
  cve_id TEXT NOT NULL,
  observed_at TEXT NOT NULL,
  ingested_at TEXT NOT NULL,
  evidence_type TEXT NOT NULL CHECK (evidence_type IN
    ('kev','epss','advisory','malware','honeypot','public_poc')),
  publisher TEXT NOT NULL,
  source_document TEXT NOT NULL,
  independent_group TEXT NOT NULL,
  supports_exploit INTEGER NOT NULL CHECK (supports_exploit IN (0,1)),
  likelihood_ratio REAL,
  raw_value REAL,
  extraction_method TEXT NOT NULL,
  UNIQUE(cve_id,publisher,source_document,evidence_type)
);

CREATE TABLE deployment_observation (
  observation_id TEXT PRIMARY KEY,
  observed_at TEXT NOT NULL,
  asset_id TEXT NOT NULL,
  cve_id TEXT NOT NULL,
  vulnerable_path INTEGER NOT NULL,
  internet_reachable INTEGER,      -- NULL means unknown
```



```
control_state TEXT NOT NULL,
asset_loss_usd REAL NOT NULL,
remediation_hours REAL NOT NULL,
inventory_source TEXT NOT NULL
);
```

`independent_group` prevents three feeds quoting one vendor bulletin from appearing to be three confirmations. `observed_at` says when an event happened; `ingested_at` says when the system could know it. Both are essential to honest backtests. STIX 2.1 is useful for exchanging indicators, sightings, relationships, and confidence, but STIX confidence is publisher metadata—not automatically a probability.

ATT&CK technique mappings add behavioral context. A CVE enabling External Remote Services (T1133) may support a campaign hypothesis, but the mapping is a typed, sourced relationship rather than proof that a campaign used that CVE. ATLAS plays an analogous role for attacks on machine-learning systems. Mappings should improve investigation and control selection, not become arbitrary score multipliers.

Bayesian Updating Without Circular Evidence

Let $\mathcal{H}$ mean “this CVE will be exploited against the relevant population during the decision horizon.” Starting with a point-in-time EPSS prior $\mathcal{P}_0$, prior odds are

$$\mathcal{O}(\mathcal{H}) = \frac{\mathcal{P}_0}{1 - \mathcal{P}_0}.$$

For conditionally independent evidence groups with calibrated likelihood ratios $\mathcal{LR}_g$:

$$\mathcal{O}(\mathcal{H} \mid \mathcal{E}) = \mathcal{O}(\mathcal{H}) \prod_g \mathcal{LR}_g, \quad \mathcal{P}(\mathcal{H} \mid \mathcal{E}) = \frac{\mathcal{O}(\mathcal{H} \mid \mathcal{E})}{1 + \mathcal{O}(\mathcal{H} \mid \mathcal{E})}.$$

A likelihood ratio is $\mathcal{P}(\mathcal{E} \mid \mathcal{H}) / \mathcal{P}(\mathcal{E} \mid \neg \mathcal{H})$, not an analyst’s 1–5 confidence. Estimate it from historical point-in-time observations. KEV inclusion usually deserves a strong positive ratio, but not infinity: “exploited somewhere” is not “will target this tenant.” A public proof of concept may add modest evidence. Four articles copied from the same researcher add one evidence group, not four.

Conditional independence is the fragile assumption. Reporting popularity creates another bias: famous vendors and dramatic CVEs receive more articles. Evaluate calibration by disclosure age, ecosystem size, and vendor prominence, and maintain low-attention challenge cases.

From Forecast to Action

Prioritization should minimize expected loss. For asset $\mathcal{A}$ and action $\mathcal{D}$:

$$\mathcal{L}(\mathcal{D}) = \mathcal{P}_{\text{exploit}} \mathcal{P}_{\text{reach}}(\mathcal{A}) \mathcal{P}_{\text{success}}(\mathcal{A}, \mathcal{D}) \mathcal{C}_{\text{impact}}(\mathcal{A})$$

- $\mathcal{C}_{\text{change}}(\mathcal{A}, \mathcal{D}) + \mathcal{C}_{\text{delay}}(\mathcal{D})$.

$\mathcal{L}$

`p_reach` represents whether the vulnerable path can be exercised. `p_success` accounts for remaining controls. `C_change` includes outage risk and labor. Emergency patching is justified when its expected reduction in

compromise loss exceeds change cost. This is why “patch every critical CVSS in 24 hours” is incomplete: it omits exploitation likelihood, reachability, consequence, and patch risk.

Keep the components in the decision record:

```
{
  "decision_time": "2026-08-20T14:00:00Z",
  "asset_id": "edge-idp-07",
  "cve_id": "CVE-20XX-9001",
  "epss_asof": 0.18,
  "posterior_exploit": 0.798,
  "reachability": 0.95,
  "impact_usd": 900000,
  "change_cost_usd": 18000,
  "action": "emergency_patch",
  "evidence_groups": ["kev:cisa", "poc:r17"],
  "policy_version": "exploit-loss-v3"
}
```

NLP as an evidence extractor

Advisory NLP should resolve product/version spans, distinguish “could allow exploitation” from “exploitation observed,” capture who asserted the claim and when, link to exact text, and cluster documents by common origin. “No evidence of exploitation” is not evidence of no exploitation. A language model may summarize a report, but consequential decisions must retain the underlying span and deterministic entity resolution.

Safe Lab: Evidence-Weighted Expected Avoided Loss

This synthetic lab deduplicates correlated evidence, updates odds, and chooses emergency work only when expected avoided loss exceeds change cost.

```
from dataclasses import dataclass
from math import prod

@dataclass(frozen=True)
class Deployment:
    name: str; cve: str; epss: float; reach: float
    residual_after_patch: float; impact: float; change_cost: float

@dataclass(frozen=True)
class Evidence:
    cve: str; origin: str; lr: float

def posterior(prior: float, items: list[Evidence], cve: str) -> float:
    if not 0 < prior < 1:
        raise ValueError("prior outside (0,1)")
    groups: dict[str, float] = {}
    for e in items:
        if e.cve == cve:
            groups[e.origin] = max(e.lr, groups.get(e.origin, 0.0))
    odds = prior / (1 - prior) * prod(groups.values())
    return odds / (1 + odds)

evidence = [
    Evidence("CVE-A", "researcher-17", 3.0),
    Evidence("CVE-A", "researcher-17", 2.5), # copied report
]
```

```

Evidence("CVE-A", "cisa-kev", 6.0),
Evidence("CVE-B", "public-poc", 1.4),
]
assets = [
    Deployment("edge-idp", "CVE-A", .18, .95, .30, 900_000, 18_000),
    Deployment("isolated-build", "CVE-A", .18, .02, .30, 120_000, 8_000),
    Deployment("marketing-site", "CVE-B", .04, .80, .40, 70_000, 9_000),
]
out = []
for a in assets:
    p = posterior(a.epss, evidence, a.cve)
    benefit = p * a.reach * a.impact * (1 - a.residual_after_patch)
    action = "emergency_patch" if benefit > a.change_cost else "normal_queue"
    out.append((a.name, round(p, 3), round(benefit), action))

expected = [
    ('edge-idp', .798, 477621, 'emergency_patch'),
    ('isolated-build', .798, 1341, 'normal_queue'),
    ('marketing-site', .055, 1852, 'normal_queue')]
assert out == expected, out
print(out)

```

Expected output is exactly `expected`. The shared CVE-A forecast produces different actions because local exposure and consequence differ. Without origin deduplication, duplicated reporting would create false certainty.

Production Failure Analysis

Future leakage. Joining today’s KEV and latest EPSS to old snapshots creates a clairvoyant backtest. Snapshot feeds and use only data ingested before each historical decision.

Invisible assets. An inner join between scanner and CMDB erases unmanaged assets. Publish join coverage. Distinguish “not reachable with recent evidence” from “reachability unknown”; missing must never become false.

Catalog bias. KEV is curated for a mission, not a random sample. “Not in KEV” is not a clean negative label. Use confirmed positives, delayed outcomes, reviewed samples, and positive-unlabeled methods where justified.

Queue shock. A feed update can move thousands of deployments across a threshold. Group common remediations, enforce engineering capacity, reserve an incident lane, and rank marginal avoided loss per hour. If EPSS becomes stale, fall back to explicitly degraded logic rather than silently reusing it forever.

Adversarial manipulation. Attackers can publish false exploit claims or poison sensors. Authenticate feeds, hash source material, separate extraction confidence from exploit evidence, and support removing a source followed by deterministic replay.

Evaluate the forecast with horizon-specific Brier score, calibration, and precision–recall. Evaluate the policy with expected loss avoided, emergency changes, incidents tied to vulnerable deployments, and remediation capacity. A calibrated forecast can still drive a poor policy if impact or patch-cost estimates are wrong.

Designing the Production Prioritizer

The production service should expose two APIs rather than one. A forecast API answers, for a CVE and timestamp, what exploitation probability and evidence were available. A decision API accepts a deployment observation, candidate actions, and policy version, then returns the chosen action and loss decomposition. This separation lets the threat-research team improve evidence fusion without owning business impact estimates, and lets asset owners dispute inventory or consequence without rewriting threat history.

Forecast features must be monotone in time: a document received tomorrow cannot change what was knowable today. Maintain append-only feed snapshots and a canonical CVE alias table. Reject records whose CVE identifier cannot be resolved, but retain them in a quarantine stream because pre-CVE exploitation reports are often valuable. When a CVE is later assigned, create a sourced alias event rather than rewriting the old document.

Asset context needs its own freshness rules. Network reachability may expire after hours; ownership might remain valid for weeks; business-impact tiers may be reviewed quarterly. Each field should carry `measured_at`, `source`, and `expiry`. The decision engine should abstain or widen its loss range when a critical input is stale. One global “data fresh” flag conceals the fact that different evidence decays at different rates.

Remediation grouping deserves explicit modeling. A package upgrade may remove five CVEs across 4,000 hosts with one tested change, while separately ranking every pair makes the work look impossibly large. Build candidate actions such as “upgrade base image X from digest A to B” and calculate the union of affected deployment losses, avoiding double counting when several CVEs lead to the same compromise path. Subtract testing, rollout, and expected outage cost once per action, not once per CVE.

Capacity constraints turn ranking into scheduling. Emergency windows, platform-team skills, freeze periods, dependency order, and rollback readiness can be represented as constraints. A simple greedy queue based on avoided loss per engineering hour is transparent and often adequate; integer optimization becomes useful when actions share prerequisites or maintenance windows. The optimizer must never invent precision: if impact is a broad range, solve optimistic and pessimistic schedules and surface actions stable across both.

An example decision review

Suppose an endpoint-management CVE has EPSS 0.12, a public exploit from one independent origin, and no KEV entry. It affects 6,000 laptops, but the vulnerable service is disabled on 5,400. The remaining 600 include 40 privileged administrator devices. The correct response is not a fleetwide priority score. Produce at least three deployment segments, verify configuration evidence, and compare actions: disable the service remotely, upgrade the agent, or isolate the administrator devices. A reversible configuration mitigation may dominate a risky emergency upgrade even though it does not “close” the scanner finding.

Now suppose the scanner cannot distinguish disabled from enabled services on 15% of laptops. The denominator is 6,000, not the 5,100 with confident configuration. Report the unknown segment and sample it. If sampling shows configuration data fails disproportionately on older, high-risk devices, random extrapolation is invalid; stratify by operating system and management health.

Evaluation data construction

Define the forecast horizon before collecting labels. For a 30-day forecast made daily, adjacent observations for one CVE share outcomes and cannot be treated as independent rows in an ordinary random split. Split by time and, for robustness, by CVE disclosure cohort. A positive label should cite an exploitation observation whose timestamp falls inside the horizon and whose source was not itself used as an input unless the evaluation explicitly measures assimilation speed.

Unobserved exploitation is not a confirmed negative. Honeypots and telemetry cover only parts of the internet, and reporting is selective. In addition to conventional metrics, estimate performance on high-observability strata and report label coverage. Sensitivity analysis can ask how calibration changes if a plausible fraction of unlabeled cases were exploited. This is more honest than presenting six decimal places from incomplete surveillance.

Policy evaluation should replay action availability as well as forecasts. A patch not published at decision time was not a feasible action. Likewise, a mitigation requiring a control the organization had not deployed cannot be credited retrospectively. Historical simulation should reconstruct inventory, controls, maintenance constraints, and labor capacity. Otherwise the model is evaluated in a fictional organization with perfect options.

Governance and human authority

Threat researchers should be able to flag a source as compromised, but removing it must produce a new evidence-policy version and an impact report listing changed decisions. Asset owners may correct reachability or impact with evidence and expiry, not by editing the threat probability. Incident commanders may override scheduling during active exploitation; the override record should state duration, scope, and reason.

Metrics create incentives. If vulnerability teams are measured by the number of KEV items closed, they may prioritize easy low-impact deployments and suppress hard exceptions. Pair timeliness with remaining expected exposure, unknown coverage, emergency-change harm, and recurrence. Review a random sample of “not affected” dispositions because that status can become a hiding place for inventory defects.

Finally, publish uncertainty in language operators can use: what is known, what is inferred, what is missing, and which new measurement would change the action. That is more actionable than a falsely precise percentile.

The service should also preserve rejected alternatives. Recording why mitigation lost to patching, or why acceptance lost to measurement, makes later incident review possible and prevents operators from reconstructing rationale from a score whose inputs have since changed.

Sources and Further Reading

- **FIRST, Exploit Prediction Scoring System.** A daily probability forecast for exploitation in the next 30 days, complementary to severity. <https://www.first.org/epss/>

- **CISA, Known Exploited Vulnerabilities Catalog.** Strong curated exploitation evidence, but not an exhaustive negative-label boundary. <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- **FIRST, CVSS v4.0.** Technical severity characteristics, including threat and environmental metrics; not organization-specific exploit forecasting. <https://www.first.org/cvss/v4.0/>
- **NIST, National Vulnerability Database.** Normalized enrichment whose publication and revision times matter to point-in-time analysis. <https://nvd.nist.gov/>
- **OASIS, STIX 2.1.** Exchange semantics for intelligence objects, relationships, and provenance. <https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>
- **MITRE ATT&CK and ATLAS.** Behavioral knowledge bases for enterprise and AI-system threats. <https://attack.mitre.org/> and <https://atlas.mitre.org/>

Together these sources imply a layered system: CVSS describes characteristics, EPSS forecasts population exploitation, KEV adds observed-exploitation evidence, STIX carries sourced claims, and ATT&CK/ATLAS connect claims to behaviors. The organization still owns exposure, consequence, and intervention cost.

Interview Practicum

Scenario: The Forecast Changes Mid-Meeting.

Prompt. You have 140,000 CVE–asset pairs and capacity to remediate 2,000 per week. Design the service.

A strong candidate asks whether capacity counts hosts, packages, or change tickets; requests reachability, asset-loss tiers, maintenance windows, compensating controls, point-in-time feeds, exploit labels, and shared patches.

Worked Answer

The initial design builds time-stamped deployment observations, uses EPSS as a prior, groups independent evidence, estimates exposure and residual control effectiveness, then schedules common remediation actions by expected avoided loss per engineering hour.

Follow-Up Questions and Answers

Constraint change one: reachability is only 60% complete and weekly. Introduce reachable, recently verified unreachable, and unknown states. Use loss bounds for unknowns. Cases whose action changes across the bounds enter a measurement queue. Publish coverage as an SLI; do not claim risk fell when visibility disappeared.

Constraint change two: an identity appliance enters KEV Friday afternoon, but its patch has a 4% outage rate and downtime is forbidden before Monday. Compare weekend compromise loss with outage loss. Apply reversible controls—management-plane restriction, targeted detection, stronger authentication, and active hunting—then patch in an approved window with rollback. Do not reduce the threat score to rationalize the operational decision.

Principal probe: leadership wants one number. Report estimated exploitable exposure above the approved loss threshold, with an uncertainty range, the fraction actively mitigated, and age. Operators retain the components.

Weak answers sort only by CVSS or EPSS, add incompatible scales, treat KEV absence as a negative label, or convert NLP confidence directly into exploit probability.

Key Takeaways

- CVSS severity, EPSS probability, KEV evidence, and local risk are different variables.
- Prioritize time-stamped CVE–asset–deployment observations, not CVEs alone.
- Bayesian updates require calibrated likelihood ratios and source-lineage deduplication.
- Schedule interventions by expected avoided loss under real remediation capacity.
- Preserve feed history, unknown exposure states, evidence spans, and policy versions.
- Evaluate forecast calibration and downstream policy outcomes separately.

Chapter 30: Measuring the Quality of Security Findings

A Finding Is a Falsifiable Claim

A security finding is not high quality because a tool emitted it or called it critical. It is a claim containing an affected artifact and version, a weakness mechanism, a reachable trigger, an undesirable consequence, and evidence another investigator can reproduce. Accuracy, severity, exploitability, business impact, actionability, and novelty are separate axes. A true but unreachable weakness can be accurate and low priority; a duplicate can be correct yet add no queue value.

```
CREATE TABLE finding(
  finding_id TEXT PRIMARY KEY, scanner_version TEXT NOT NULL,
  repository TEXT NOT NULL, commit_sha TEXT NOT NULL, artifact_uri TEXT NOT NULL,
  rule_id TEXT NOT NULL, cwe_id TEXT, source_span TEXT NOT NULL,
  sink_span TEXT, trace_hash TEXT, trigger_fixture TEXT,
  claimed_severity TEXT NOT NULL, confidence REAL CHECK(confidence BETWEEN 0 AND 1),
  emitted_at TEXT NOT NULL);
CREATE TABLE adjudication(
  finding_id TEXT, reviewer_id TEXT, reviewed_at TEXT NOT NULL,
  verdict TEXT CHECK(verdict IN
('confirmed','refuted','inconclusive','duplicate','environment_blocked')),
  root_cause_id TEXT, reproducible INTEGER, reachable INTEGER,
  minutes_to_validate INTEGER, evidence_grade TEXT, rationale TEXT,
  PRIMARY KEY(finding_id,reviewer_id));
```

The explicit inconclusive and environment-blocked states prevent forced binary labels. Scanner and commit versions prevent a later code change from making an old finding appear irreproducible. Reviewer identity supports agreement analysis without pretending adjudicators are infallible.

Three Levels of Measurement

For reviewed claims, precision is $\frac{TP}{TP+FP}$. Recall requires a denominator of weaknesses that should have been found; ordinary production output does not provide it. Seeded and curated corpora help, but NIST SARD and OWASP Benchmark are regression instruments, not estimates of prevalence in a company's repositories.

Reproducibility must name its environment:

$$R = \frac{\text{confirmed findings reproduced under pinned fixtures}}{\text{confirmed findings attempted under pinned fixtures}}$$

A missing compiler or expired test credential is an environment block, not a false positive. Measure that rate separately.

For confidence p_i and binary adjudication y_i , calibration can be summarized with Brier score:

$$\sqrt{BS = \frac{1}{n} \sum_i (p_i - y_i)^2}$$

Inspect reliability by rule, language, repository class, and scanner version. A globally calibrated 0.8 can hide one rule at 0.98 and another at 0.30. Use temporal holdouts because rule tuning after analyst feedback leaks across random splits. Enforce a hard cutoff: develop on evidence available through time (T), then validate only on predictions emitted after (T). Future developer reactions, fixes, and post-hoc refactors must not be joined back into the training side of that boundary.

Persist the raw candidate population at inference time, before suppression, deduplication, severity guards, or posting policy. Evaluate detector quality and calibration on the eligible raw population, with disposition as an explicit field; evaluate developer burden on the posted subset. Without this split-collection pattern, downstream filters create selection bias that can make precision appear better while hiding discarded candidates.

Finding-level precision also rewards ten correct symptoms of one defect. Group findings by a shared fix boundary—library version, missing authorization policy, sanitizer defect, or generated template—and report unique confirmed root causes, fragmentation, over-merging, review minutes, and tickets created. Exact file-line matching fails after refactoring; text similarity over-merges generic messages. Candidate keys should include CWE, repository lineage, sink API, sanitizer state, trace structure, dependency identity, and fix boundary.

At review budget $\sqrt{B}$, measure distinct root causes discovered per analyst-hour and downstream harm avoided. Do not add these into a universal “quality score”; publish a scorecard so a product cannot trade catastrophic over-merging for a small precision gain.

Safe Lab: Quality Scorecard

```
rows = [
    # id, confidence, outcome, root cause, evidence fields present, review minutes
    ("F1", .90, 1, "R1", 5, 8),
    ("F2", .80, 1, "R1", 4, 6), # duplicate symptom
    ("F3", .70, 0, None, 2, 12),
    ("F4", .60, 1, "R2", 5, 15),
    ("F5", .40, 0, None, 3, 5),
    ("F6", .55, None, None, 1, 20), # inconclusive
]
labeled = [r for r in rows if r[2] is not None]
roots = {r[3] for r in labeled if r[2] == 1}
scorecard = {
    "reviewed_precision": round(sum(r[2] for r in labeled) / len(labeled), 3),
    "brier": round(sum((r[1]-r[2])**2 for r in labeled) / len(labeled), 3),
    "evidence_completeness": round(sum(r[4]/5 for r in rows) / len(rows), 3),
    "confirmed_findings": sum(r[2] == 1 for r in labeled),
    "distinct_root_causes": len(roots),
    "review_minutes_per_root_cause": round(sum(r[5] for r in rows)/len(roots), 1),
    "inconclusive_rate": round(sum(r[2] is None for r in rows)/len(rows), 3),
}
expected = {'reviewed_precision': .6, 'brier': .172,
```

```
'evidence_completeness': .667, 'confirmed_findings': 3,
'distinct_root_causes': 2, 'review_minutes_per_root_cause': 33.0,
'inconclusive_rate': .167}
assert scorecard == expected, scorecard
print(scorecard)
```

Three confirmed findings represent two root causes. Optimizing confirmed-findings rewards F2; a customer-oriented metric recognizes its smaller incremental value.

Gold Sets, Challenges, and Production Failure

A gold set should combine seeded weaknesses, historical confirmed cases frozen before rule development, clean controls, and independently found production cases. Keep a locked temporal holdout, remove training clones, and report each stratum separately. Challenge sets target path explosion, framework magic, conditional sanitizers, generated code, polyglot boundaries, unreachable utilities, renamed APIs, and adversarial comments containing vulnerability vocabulary.

Inter-reviewer disagreement is information. Cohen's kappa can flag an underspecified rubric, but consensus must not erase original judgments. Reviewers should be blinded to model confidence on a sample. Developer dismissal is not ground truth: it can mean false positive, accepted risk, duplicate, missing ownership, or deadline pressure.

Calibrate reviewers before promoting a gold set:

1. select a representative calibration set, often 30–50 findings across important strata;
2. have qualified reviewers label it independently and blindly;
3. compute raw agreement plus a chance-corrected measure appropriate to the labels;
4. for weak or systematically confused categories, adjudicate examples, revise the written rubric, and repeat on a fresh calibration set.

Do not turn a threshold such as $\kappa \geq 0.70$ into a universal law. Predeclare it for the decision, report prevalence and category-specific disagreement, and retain the original labels so a high aggregate agreement score cannot conceal failure on rare critical classes.

Major production failures include:

- **Verification bias:** analysts review high scores first. Randomly sample below threshold or weight by review propensity.
- **Mutable truth:** reconstruct the emitted commit and lockfile; a finding fixed before review is not detector error.
- **Aggregate hiding:** require rule-level intervals and quarantine harmful rules even when global precision rises.
- **Severity inflation:** confirmation proves existence, not exploitability or impact.

- **Benchmark overfitting:** once a vendor tunes on an evaluation set, retire it to training history and create a new holdout.
- **Duplicate gaming:** count both finding and root-cause outcomes, plus over-merge and fragmentation harm.

Operate scorecards by scanner version. Monitor evidence completeness, environment blocks, confirmation, inconclusive outcomes, calibration, root-cause yield, duplicate burden, and validation time. Release gates must name acceptable regressions; “overall score improved” is insufficient.

Constructing a Defensible Evaluation Program

The evaluation population should match the product claim. A scanner marketed for pull-request prevention must be tested on changes as they appeared at review time, not only on mature repositories after fixes and refactors. A cloud posture product needs tenants with different providers, policy maturity, and asset scale. A binary analyzer needs compiler, architecture, and optimization diversity. Write the claim first—population, artifact, weakness families, latency, and intended action—then sample.

Sampling is part of the estimator. If every critical finding is reviewed, 10% of highs, and 1% of mediums, raw confirmed proportions cannot be added. Record the inclusion probability and use inverse-probability weighting for population estimates. Publish unweighted counts alongside weighted rates so readers can see support. Extremely large weights produce unstable estimates; combine strata or increase sampling rather than hiding the variance.

Confidence intervals matter most where sample sizes are small. A new rule with four confirmations in four reviews does not have proven 100% precision. Use a binomial interval or Bayesian posterior with a declared prior, and require minimum evidence before autonomous suppression or blocking. For calibration bins, report count and uncertainty. Empty regions of the confidence scale reveal that the model’s numeric precision is cosmetic.

Reproducibility protocol

A reproducibility package should pin the artifact, build configuration, runtime, request or input fixture, relevant identity, expected observation, and cleanup. Secrets must be synthetic or ephemeral. The package must not exploit production systems. For code findings, include the shortest trace demonstrating source, transformations, guards, and sink. For cloud findings, include read-only configuration evidence and a safe policy evaluation. For endpoint findings, prefer captured telemetry or a disposable virtual machine.

Reproduction can produce several outcomes: mechanism confirmed; claim refuted; trigger blocked by environment; evidence insufficient; or artifact no longer available. These outcomes guide different improvements. A high environment-block rate suggests the product emits claims customers cannot validate. A high unavailable-artifact rate suggests retention or commit-resolution defects. Folding both into “inconclusive” loses engineering direction.

Time to validate is not merely an operations metric. It is a property of finding design. Compare otherwise similar findings with and without traces, concrete inputs, ownership hints, and fix boundaries. A controlled evaluation can randomize evidence presentation to reviewers and measure validation time and correct disposi-

tion. Do not expose a potentially misleading model confidence in the treatment unless the experiment is specifically testing confidence display.

Severity, exploitability, and actionability

Severity describes consequence under conditions; exploitability describes how feasible those conditions are; business impact describes what the affected system matters to the organization. Keep these as fields. A path traversal in a build tool may have high technical impact but require trusted local access; an authorization bypass in a low-CVSS internal service may expose regulated data through a reachable identity path.

Actionability can be scored through observable components: correct owner located, fix boundary identified, remediation available, reproduction fixture present, and estimated effort. It should not be inferred from “ticket closed.” Teams sometimes close excellent findings because risk is accepted and retain poor findings because audit policy requires them. Measure evidence quality before downstream organizational choices.

Root-cause labeling also needs a contract. Two findings share a root cause when one bounded change can remove the relevant mechanism without separately addressing each symptom. This definition is deliberately tied to remediation. The same CWE in two independently maintained libraries is not one root cause. Conversely, different CWE symptoms may share one unsafe code-generation template. Store the proposed cluster, algorithm version, reviewer decision, and final fix identifier.

Metamorphic and adversarial tests

Known-answer benchmarks test cases, but metamorphic tests examine invariants. Rename variables, reorder independent statements, move code without changing data flow, or replace a sanitizer with an equivalent wrapper. A robust semantic analyzer should preserve its conclusion. Then alter the actual security property—remove authorization, change the sanitizer context, or make a path attacker-controlled—and require the conclusion to change.

Adversarial cases should target the product surface, not merely the model. Put vulnerability words in comments, create very long traces, use generated filenames resembling dependencies, produce conflicting package manifests, and inject malformed source maps. Test whether parsers fail closed, fail open, or silently assign a default severity. A safe product exposes unsupported analysis rather than fabricating confidence.

Release and rollback

Before release, replay the candidate on a fixed corpus and a recent production sample. Compare paired outcomes: new confirmed root causes, lost root causes, confidence changes, duplicate clusters, evidence regressions, and latency. Aggregate lift cannot justify losing coverage for a dangerous weakness family. Establish must-not-regress challenge cases and maximum queue expansion by customer segment.

Canary the scanner version on consenting repositories. Keep old and new finding identities linked so users do not receive duplicate tickets during migration. If rollback occurs, restore compatible suppression, cluster, and confidence semantics. Merely reverting the engine while leaving new identifiers in workflow systems can reopen thousands of items.

Quality governance should separate product targets from adjudication rules. The team rewarded for precision must not decide after seeing results whether duplicates count as true positives. Version the rubric before evaluation, log exceptions, and have an independent review for high-impact automated actions such as merge blocking. This turns the scorecard into evidence rather than marketing.

Reading a Scorecard as an Engineering Decision

Suppose a candidate release raises reviewed precision from 78% to 84%, reduces median validation time from 18 to 11 minutes, but loses 9 of 40 authentication-bypass cases in the locked challenge set. The release is not “better” in the only sense that matters. The regression affects a high-consequence family and may reflect a parser or framework-modeling change. Block general rollout, isolate the responsible rule path, and consider shipping unaffected languages behind separate version flags.

Now suppose the lost cases are duplicate symptoms already represented by another finding. The answer changes only if the duplicate analysis is independently validated and the surviving finding reaches the same owner with equal evidence. A finding suppressed because two traces look similar is not safely redundant when the traces cross different trust boundaries or require different fixes. Review over-merge errors at the actual automation boundary.

Scorecards should include denominators and abstentions. If a scanner analyzes 70% of files and achieves 90% precision there, it must not be compared directly with a scanner analyzing 98% at 82% precision. Report unsupported files, parser failures, timeouts, and intentionally skipped generated code. Coverage-weighted yield can help, but keep the components visible because customers may care disproportionately about the unsupported 2%.

Performance also has a quality dimension. A pull-request scanner that returns excellent findings after forty minutes may arrive after merge. Measure end-to-end decision latency at realistic repository sizes and concurrent load, including cache misses. Timeout behavior must be explicit: allow, block, retry, or fall back. Silent partial results make precision estimates meaningless because the evaluated population changes with system load.

For generative remediation, evaluate the patch separately from the finding. A correct diagnosis can yield an insecure or behavior-breaking patch. Run tests, static checks, policy checks, and targeted security fixtures in an isolated environment. Measure patch acceptance, modifications by developers, introduced defects, and whether the root cause remains. Never let an accepted generated patch retroactively validate the original finding; developers may accept changes for unrelated reasons.

Cross-customer comparison demands care. Different customers configure rules, suppressions, and severity policies; their reviewed populations and incentives differ. Normalize only within a declared configuration and sampling design. A benchmark percentile that mixes permissive advisory deployments with strict merge-blocking deployments is uninterpretable. Product analytics should help each customer understand its own evidence quality without leaking another tenant’s code or labels.

The final review artifact should include the claim being tested, corpus manifest, sampling probabilities, reviewer rubric, disagreement record, scanner artifacts, result tables with uncertainty, known benchmark

overlap, and a signed decision. Reproducibility here means another authorized team can recreate the evaluation from retained, access-controlled artifacts—not that vulnerable customer code is broadly copied.

Sources and Further Reading

- **NIST Software Assurance Reference Dataset.** Controlled known-good and known-bad programs, with limited external validity. <https://samate.nist.gov/SARD/>
- **OWASP Benchmark Project.** A repeatable Java test suite that should be one evaluation stratum, not the sole production claim. <https://owasp.org/www-project-benchmark/>
- **MITRE CWE.** Shared weakness vocabulary; identical CWE identifiers do not prove a common root cause. <https://cwe.mitre.org/>
- **ISO/IEC 29147:2018.** Coordinated receipt, verification, and communication expectations for vulnerability disclosure. <https://www.iso.org/standard/72311.html>
- **NIST SP 800-218.** Connects vulnerability response to secure development and durable process improvement. <https://csrc.nist.gov/pubs/sp/800/218/final>

Benchmarks supply known cases, CWE supplies vocabulary, and standards supply workflow expectations. Finding quality still requires reproducibility, calibrated confidence, root-cause novelty, and human cost.

Interview Practicum

Scenario: The Denominator Changes.

Prompt. A new SAST engine reports 92% precision on 500 reviewed findings versus 71% for the incumbent. Should the company migrate?

Ask how samples were selected, whether reviewers were blinded, how inconclusive cases were handled, whether commits were frozen, what languages were covered, and whether duplicates count.

Worked Answer

Replay both tools on identical repositories, adjudicate stratified random samples including low scores, group root causes, and compare results at the same review budget.

Follow-Up Questions and Answers

Change one: the 500 new-engine findings were its top-ranked outputs; the incumbent sample was uniform. The headline comparison is invalid. Uniformly sample both or use inclusion-probability weights. A top-of-list evaluation may answer a capacity question, but then compare unique root causes per analyst-hour.

Change two: migration removes the incumbent’s binary-analysis coverage for 20% of releases. Evaluate a portfolio. Retain the incumbent for uncovered artifacts until an alternative passes a challenge set, and deduplicate cross-tool output. Coverage loss is harm, not a precision footnote.

Change three: the vendor offers to tune on the adjudicated evaluation set. Freeze it as consumed and build a later private temporal holdout. Weak answers use accuracy, accept “won’t fix” as false-positive truth, or select the tool producing more critical alerts.

Key Takeaways

- A finding is a versioned, falsifiable claim, not a severity label.
- Accuracy, reproducibility, calibration, actionability, and novelty need separate metrics.
- Evaluate distinct root causes and human cost, not only confirmed finding count.
- Preserve inconclusive and environment-blocked outcomes.
- Correct verification bias with probability sampling or weighting.
- Gold sets, challenge sets, and production samples answer different questions.

Field lesson: precision is an empirical outcome, not a pipeline state

The sec-pr assurance project created 1,098 label-queue items but correctly refused to publish precision because no validity labels had yet matured. Its engineering platform could be released for controlled evidence acquisition while the finding-quality card remained `insufficient_evidence`. A healthy ingestion pipeline is not a verified detector. Chapter 47 develops the complete field case.

Chapter 31: From Finding to Resolution

Resolution Is a State History, Not a Closed Ticket

A scanner finding becomes security value only when a vulnerable condition is durably removed or bounded by an accepted control. Ticket closure is an administrative event. It may mean fixed, duplicate, risk accepted, asset retired, false positive, or merely closed by automation. Lifecycle analytics must preserve transitions and reasons instead of overwriting one `status` field.

```
CREATE TABLE finding_transition(
  transition_id TEXT PRIMARY KEY, finding_id TEXT NOT NULL,
  occurred_at TEXT NOT NULL, recorded_at TEXT NOT NULL,
  from_state TEXT, to_state TEXT NOT NULL,
  actor_type TEXT NOT NULL, owner_team TEXT,
  reason_code TEXT NOT NULL, evidence_uri TEXT,
  UNIQUE(finding_id,occurred_at,to_state,reason_code));
CREATE TABLE finding_snapshot(
  finding_id TEXT, snapshot_at TEXT, root_cause_id TEXT,
  severity TEXT, asset_tier TEXT, due_at TEXT,
  PRIMARY KEY(finding_id,snapshot_at));
```

A useful state machine distinguishes `discovered`, `triaged`, `validated`, `owned`, `fix_in_progress`, `deployed`, `verified`, `risk_accepted`, `refuted`, and `reopened`. Only verification establishes technical resolution. Preserve event time and recording time to diagnose delayed integrations. Invalid backward transitions—such as `discovered` directly to `verified`—should be rejected unless an explicit migration reason exists.

Queues, Competing Outcomes, and Censoring

Each transition has a waiting interval and a service interval. Long triage latency can mean too few analysts; long owned-to-fix latency can mean dependency coordination; long deployed-to-verified latency can mean missing telemetry. Aggregate “mean time to remediate” hides these mechanisms and is distorted by unresolved findings.

For time to verified resolution $\backslash(T\backslash)$, the survival function is $\backslash(S(t)=P(T>t)\backslash)$. With event times $\backslash(t_j\backslash)$, resolutions $\backslash(d_j\backslash)$, and findings at risk $\backslash(n_j\backslash)$, the Kaplan–Meier estimator is

$$\backslash[\hat{S}(t)=\prod_{t_j\leq t}\left(1-\frac{d_j}{n_j}\right).\backslash]$$

Open findings are right-censored, not discarded and not assigned today as a fake closure date. Risk acceptance, asset retirement, and refutation are competing events. Treating them as remediation exaggerates engineering performance. Report cumulative incidence by terminal reason or use a competing-risks model.

Service-level statements should use cohorts and quantiles: “62% of internet-facing critical findings discovered in May were verified within 14 days,” with unresolved cases retained. Segment by ownership, asset tier, root cause, and discovery channel. Compare cohorts only after accounting for severity and exposure mix.

Durable Fixes and Downstream Reduction

A finding is durably fixed when verification passes after deployment and the root cause does not recur inside a defined observation window. Reopen rate is

$$\frac{\text{verified findings later returned to active vulnerability}}{\text{verified findings with sufficient follow-up}}$$

The denominator excludes fixes too recent to observe. Recurrence should be counted at root-cause level: repairing ten generated files while leaving the template unchanged produces ten closures and one predictable recurrence mechanism.

Measure downstream reduction using exposure-time, not ticket count. If a finding affects m reachable assets for h hours, it contributes mh vulnerable asset-hours. A program can close more low-impact tickets while total high-impact exposure rises. Developer friction also matters: handoffs, build failures introduced by fixes, review minutes, exception churn, and unplanned work.

Safe Lab: Kaplan–Meier Without External Packages

```
# duration in days; event=1 only for verified remediation
records = [(2,1),(3,1),(3,0),(5,1),(7,0)]

def kaplan_meier(rows):
    survival = 1.0
    curve = []
    for t in sorted({d for d,e in rows if e == 1}):
        at_risk = sum(d >= t for d,e in rows)
        events = sum(d == t and e == 1 for d,e in rows)
        survival *= 1 - events / at_risk
        curve.append((t, round(survival, 3), at_risk, events))
    return curve

curve = kaplan_meier(records)
expected = [(2, .8, 5, 1), (3, .6, 4, 1), (5, .3, 2, 1)]
assert curve == expected, curve
assert round(1 - curve[-1][1], 3) == .7
print(curve)
```

The censored observations remain in the risk set until their last observed day. Dropping them would claim every retained case eventually resolved and bias the result toward fast closure.

Diagnosing Flow Rather Than Blaming Teams

Reconstruct transitions with window functions and measure time in each state:

```

WITH ordered AS (
  SELECT finding_id, occurred_at, to_state,
         LEAD(occurred_at) OVER(PARTITION BY finding_id ORDER BY occurred_at) AS next_at
  FROM finding_transition
), dwell AS (
  SELECT finding_id, to_state,
         (julianday(COALESCE(next_at, '2026-08-20')) - julianday(occurred_at)) * 24 AS hours
  FROM ordered
)
SELECT to_state, ROUND(AVG(hours), 1) AS mean_hours, COUNT(*) AS intervals
FROM dwell GROUP BY to_state ORDER BY mean_hours DESC;

```

This SQLite-compatible query uses an explicit analysis cutoff. Production code should parameterize it. Means are useful for workload accounting; publish medians and upper quantiles for latency. A growing queue can arise from arrivals exceeding service capacity even when individual engineers work faster. Track arrival rate, work-in-progress, throughput, blocked time, and age distribution together.

Production Failure Analysis

Status overwrite: a current-state table cannot reconstruct history. Use append-only transitions and correct errors with compensating events.

Closure semantics: automatic closure after a repository archive is not remediation. Terminal reason codes are mandatory.

Survivorship bias: analyzing only resolved findings guarantees optimistic latency. Include right-censored open cases.

Reopen blind spots: scanner suppression can make a defect disappear without a code change. Verification should cite deployment evidence and scanner coverage.

Ownership gaming: reassignment can reset clocks. SLA clocks attach to the finding and policy, while state dwell identifies where delay accumulated.

More findings, less security: a detector release can flood teams with low-value claims, displace high-risk remediation, and raise vulnerable asset-hours. Rate-limit intake, group root causes, protect an urgent queue, and monitor displacement.

Policy changes: a shorter SLA changes queue behavior and censoring. Version policy at discovery and show results under both historical and current definitions rather than rewriting history.

For causal claims about a workflow change, do not compare simple before/after means. Use phased rollout, matched teams with pre-trend checks, or randomized encouragement. Outcomes should include verified resolution, recurrence, exposure-time, incidents, and developer toil; closure volume alone is manipulable.

Engineering the Lifecycle Ledger

An append-only transition log needs deterministic rules. Every event carries an idempotency key from the source system, event time, ingestion time, actor, and reason. Consumers derive current state by replay. If a connector sends an older transition late, insert it at its event time and rebuild the affected finding; do not append it as if it happened now. Record the correction so dashboards can explain why historical counts changed.

State definitions must name evidence. `validated` might require reproduction or independent configuration proof. `deployed` requires the change to reach the affected environment, not merely merge. `verified` requires a post-deployment test, scanner observation, or configuration assertion. For ephemeral cloud assets, verification may mean the vulnerable image is no longer deployable and all running instances have rotated. Each evidence type has a freshness window.

Some findings legitimately branch. One root cause may affect five services owned by different teams; a central library fix is deployed gradually. Model a root-cause remediation and its affected-instance coverage rather than forcing one ticket to represent both. The root cause can be fixed in the library while residual vulnerable instances remain. Report both states to avoid celebrating upstream completion before downstream adoption.

Queue mechanics

Little's Law, $L = \lambda W$, relates average work in progress L , throughput rate λ , and average time in system W under stable conditions. It is a diagnostic, not a promise. If arrival rate changes sharply or the queue is not stable, the relationship's assumptions fail. Still, it gives a useful consistency check: a team closing 100 items per week with 1,000 active items cannot plausibly sustain a two-week average life-cycle.

Age distributions reveal starvation. Report the count and risk-weighted exposure in age bands, plus the oldest items. Priority queues can starve medium findings indefinitely when critical arrivals consume all capacity. Reserve capacity or impose aging rules, but verify that aging does not let low-impact bulk overwhelm scarce specialist work. Root-cause batching can improve throughput while reducing context switching.

Handoffs deserve explicit events rather than inferred owner changes. A handoff records sender, receiver, requested action, acceptance, and time. Unaccepted assignments are queue failures. Count bounced ownership and time awaiting acceptance. An ownership directory improvement may reduce latency more than a better severity model.

Survival models beyond a single curve

Kaplan–Meier compares descriptive cohorts but does not adjust for covariates. A Cox proportional-hazards model can estimate associations with resolution rate while leaving the baseline hazard unspecified. Its hazard ratio assumes proportional effects over time; inspect residuals and time interactions. Team identity, severity, and asset tier may be confounded with unmeasured complexity, so do not turn coefficients into blame.

Accelerated failure-time models can express multiplicative effects on resolution time and may be easier to communicate. Competing-risk methods are needed when remediation, risk acceptance, retirement, and refuta-

tion preclude one another. For operational forecasting, multi-state models can estimate transition-specific hazards: discovery to triage, triage to ownership, ownership to deployment, and deployment to verification.

Calendar time matters. Freeze periods, holidays, release trains, and incident response affect hazards. Use time-varying covariates or stratified reporting. Findings created just before a year-end freeze should not be used to claim one product team is intrinsically slower. Similarly, a policy that accelerates critical work may delay medium work; evaluate the full risk-weighted queue.

Measuring durable remediation

Define an observation window based on how recurrence can be detected. A weekly scanner might need several clean scans; a deployment control can verify immediately that an image digest is blocked. Mark findings “verified pending durability” until follow-up matures. Report cohort maturity so recent improvements are not credited with artificially low recurrence.

Recurrence can be exact, mechanistic, or organizational. Exact recurrence is the same finding identity. Mechanistic recurrence is the same root-cause pattern in another location. Organizational recurrence is a new defect that the same preventive control should have stopped. These answer different questions: fix quality, systemic remediation, and prevention effectiveness.

Downstream reduction should use a stable eligible denominator. If repository coverage expands, detected vulnerability count may rise while rate falls. Report eligible assets, scanned assets, reachable vulnerable assets, and vulnerable asset-time. A program that discovers previously invisible exposure should not be punished as if it created the defects; conversely, coverage loss must not look like improvement.

Safe rollout of workflow automation

Automation can assign owners, propose duplicates, open patches, or verify deployments. Authority should rise with evidence. Begin with recommendations, measure correction rates, then allow bounded actions with rollback. Auto-closing findings is high consequence because it removes work from human view; require stronger evidence than auto-tagging.

Monitor automation-specific failures: wrong owner, duplicate cluster over-merge, broken generated patch, tests skipped, verification against the wrong environment, and rollback failure. Keep an untreated or delayed cohort when feasible. If automation reduces visible backlog by suppressing items, verify that vulnerable asset-time and incident precursors also improve.

Governance must align incentives across scanner, security operations, and product engineering. Scanner teams should not win by emitting volume; product teams should not win by administrative closure; security teams should not win by transferring unplanned work. Shared outcomes—verified root-cause reduction, bounded exposure-time, recurrence, and total effort—make local gaming harder.

A Worked Operational Interpretation

Imagine two business units with a 20-day median time to verified resolution. Unit A resolves most items between 15 and 25 days. Unit B resolves half within two days while a long tail remains open for six months.

The medians are equal, but Unit B accumulates much more exposure. Publish survival curves, restricted mean unresolved time over a policy horizon, upper quantiles, and risk-weighted age. A single central tendency cannot describe a queue.

Next, a new dependency bot creates one ticket per repository. Closure throughput triples, yet one vulnerable base image remains the common root cause. The lifecycle model should link repository findings to the image remediation and deployment coverage. Closing child tickets before images rotate is administrative progress, not verified risk reduction. Conversely, once the fixed image is enforced and workloads rotate, child findings can be resolved with shared evidence rather than repetitive manual review.

SLA analysis needs the same discipline. A finding can meet a 30-day SLA because it was accepted on day 29, while exposure remains unchanged. Report compliance by terminal reason, and show the fraction truly verified. Pause rules—waiting for a vendor, freeze, or customer approval—should not erase elapsed exposure. Maintain both calendar time and policy-accountable time; the former represents risk, the latter helps diagnose controllable process delay.

Cost accounting should distinguish touch time from wait time. Developers may spend two hours fixing a defect that sits unowned for three weeks. Reducing wait improves risk without reducing engineering effort. Conversely, automated patch generation may reduce touch time but increase failed builds and review cycles. Instrument both so workflow improvements are attributed to the correct mechanism.

Data retention is part of the measurement design. Transition histories may contain vulnerability detail and employee identifiers. Apply least privilege, tenant boundaries, retention schedules, and auditable access. Aggregate long-term analytics where detailed evidence is no longer required, while retaining hashes and policy versions sufficient to prove lineage. Deleting raw evidence should not silently change historical state reconstruction.

Finally, incident learning should feed the ledger. When an incident exploits a previously known finding, link the incident, reconstruct the lifecycle as it was known, and classify the failure: detection absence, invalid disposition, ownership delay, remediation delay, failed deployment, missing verification, or recurrence. This is not for blame. It tests whether lifecycle metrics represented real exposure and identifies which transition needs redesign.

Sources and Further Reading

- **lifelines documentation.** Practical survival and competing-risk implementations; correct data construction remains the user's responsibility. <https://lifelines.readthedocs.io/>
- **Kleinbaum and Klein, Survival Analysis.** Foundations for censoring, survival functions, and regression. <https://doi.org/10.1007/978-1-4419-6646-9>
- **NIST SP 800-218.** Frames vulnerability response as part of a secure development lifecycle, supporting root-cause and durable-fix measurement. <https://csrc.nist.gov/pubs/sp/800/218/final>
- **OWASP SAMM.** Maturity practices connect operational remediation to governance and verification. <https://owasp samm.org/model/>

- **FIRST CVSS v4.0.** Severity characterization can stratify cohorts, but does not substitute for lifecycle outcomes. <https://www.first.org/cvss/v4.0/>

These sources together support survival-aware measurement embedded in development practice: statistical handling of unresolved work, explicit verification, and process maturity rather than ticket theater.

Interview Practicum

Scenario: Throughput Doubles and Risk Rises.

Prompt. A security program doubled monthly closures after deploying automatic ticketing, yet incidents involving known weaknesses increased. Diagnose it.

Ask for transition history, terminal reasons, scanner coverage, arrivals, severity/exposure mix, root-cause grouping, verification, reopen windows, and asset-hours.

Worked Answer

Build cohorts by discovery date, estimate time to verified resolution with censoring, separate competing terminal states, and compare high-risk exposure-time and recurrence.

Follow-Up Questions and Answers

Change one: old tickets were bulk-closed as “asset retired,” but retirement evidence is unavailable. Do not call them remediated. Classify them as administratively closed with uncertain technical outcome; sample assets and inventory records, and publish a sensitivity range.

Change two: product teams say security tickets consume 18% of sprint capacity. Optimize intake as well as remediation: suppress confirmed duplicates, group by fix boundary, provide reproducible evidence, and reserve capacity by expected harm. Measure build failures and handoffs so security does not externalize its costs.

Change three: leadership requires a 30-day result. Durable recurrence needs 90 days. Report leading outcomes—verified deployment, vulnerable asset-hours, and queue age—marked provisional. Pre-register the 90-day recurrence analysis rather than redefining success at day 30.

Weak answers divide close date by open date only for resolved tickets, treat risk acceptance as a fix, or celebrate closure volume. A Principal answer also changes incentives so scanners, analysts, and product teams share verified root-cause reduction rather than local throughput targets.

Key Takeaways

- Preserve append-only state transitions and terminal reasons.
- Use survival analysis because unresolved findings are right-censored.
- Treat risk acceptance, retirement, and refutation as competing outcomes, not fixes.

- Measure root-cause recurrence and vulnerable asset-time, not ticket closure alone.
- Diagnose arrival rate, queues, dwell, throughput, and friction together.
- Version lifecycle policy and verification evidence so history remains auditable.

Chapter 32: Security Product Analytics

Usage Is Not Security Value

A security product can increase daily users, alerts viewed, and tickets closed while making customers less secure. Analysts may bulk-close noise; developers may repeatedly open a dashboard because fixes fail; an automated agent may execute many actions because it is stuck. Product analytics must connect an eligible user's exposure to a workflow, a completed security decision, and a protected outcome—while accounting for friction and harm.

The unit depends on the job. Analysts investigate cases; developers resolve root causes; administrators establish coverage and policy; executives allocate risk; agents perform bounded actions. “Monthly active user” mixes these personas and rewards shallow activity. Define activation as the first completed value-bearing workflow, not login. For a detection platform it might require representative telemetry connected, a reviewed detection deployed, and one case resolved with evidence.

```
CREATE TABLE product_exposure(
  exposure_id TEXT PRIMARY KEY, organization_id TEXT NOT NULL,
  persona TEXT NOT NULL, feature_version TEXT NOT NULL,
  eligible_at TEXT NOT NULL, exposed_at TEXT,
  assignment_arm TEXT, assignment_unit TEXT, assignment_probability REAL);
CREATE TABLE security_value_event(
  event_id TEXT PRIMARY KEY, organization_id TEXT NOT NULL,
  actor_id TEXT NOT NULL, persona TEXT NOT NULL,
  occurred_at TEXT NOT NULL, workflow TEXT NOT NULL,
  root_cause_id TEXT, outcome TEXT NOT NULL,
  evidence_verified INTEGER NOT NULL, minutes_spent REAL,
  override_reason TEXT, policy_version TEXT NOT NULL);
```

Eligibility, assignment, exposure, and outcome are distinct. Entitlement is not exposure. Exposure is not adoption. Adoption is not causal impact. `assignment_probability` enables unbiased analysis when rollout is randomized or capacity-constrained.

A Metric Tree With Security Guardrails

A useful north star for a code-security product is **weekly organizations with at least one distinct high-risk root cause durably resolved after product-supported validation**. It resists raw alert inflation and aggregates at the customer level. It still needs guardrails:

- confirmed false-intervention rate;
- developer and analyst minutes per durable root cause;
- recurrence within a mature observation window;
- eligible asset and repository coverage;
- unresolved high-risk exposure-time;

- automation rollback and unauthorized-action rate;
- time to first verified value, including setup failures.

Retention should be conditioned on continued eligibility. A customer that finishes a one-time migration should not be labeled churned. Enterprise adoption also requires persona depth: an administrator configuring telemetry does not prove analysts trust explanations or developers can fix findings.

A funnel might be eligible organization → telemetry healthy → feature exposed → first workflow completed → repeated workflow → verified security outcome. Report conversion and latency at every transition. Missing instrumentation must appear as unknown, not abandonment.

Estimating Product Impact

Usage correlates with need. Customers under attack may use a feature more and suffer more incidents; mature teams may adopt early and already have lower risk. The causal estimand must be explicit, for example the intention-to-treat effect:

$$\backslash [ATE_ITTT] = E[Y \backslash mid Z=1] - E[Y \backslash mid Z=0], \backslash]$$

where $\backslash (Z \backslash)$ is randomized assignment and $\backslash (Y \backslash)$ is a protected outcome. Intention-to-treat preserves randomization even when some assigned customers never activate. A treatment-on-treated estimate needs additional assumptions; randomized encouragement can use assignment as an instrument when forced exposure is undesirable.

Cluster randomize by organization when analysts share queues or policy. Account for interference: one tenant's threat intelligence may improve the product for others, and an agent action may affect multiple repositories. For phased rollouts, inspect pre-trends and calendar effects. Never call a before/after adoption comparison causal without defending exchangeability.

Trust is behavioral and contextual. Capture override reason codes such as bad evidence, wrong severity, policy exception, missing context, unsafe action, and explanation unclear. A low override rate can mean trust, rubber-stamping, or inability to override. Audit a sample of accepted and rejected recommendations and measure decision quality, not agreement with the product.

Safe Lab: Correlation Versus Randomized Encouragement

The synthetic data contain confounding: high-risk organizations use the feature more. Assignment remains randomized.

```
orgs = [
  # org, assigned, used, incidents
  ("A",1,1,1), ("B",1,1,0), ("C",1,0,3), ("D",1,1,1),
  ("E",0,1,2), ("F",0,0,2), ("G",0,0,1), ("H",0,0,3),
]
```

```
def mean(values): return sum(values) / len(values)

used = [r[3] for r in orgs if r[2] == 1]
unused = [r[3] for r in orgs if r[2] == 0]
naive = mean(used) - mean(unused)
assigned = [r[3] for r in orgs if r[1] == 1]
control = [r[3] for r in orgs if r[1] == 0]
itt = mean(assigned) - mean(control)

result = {"naive_user_difference": round(naive,3),
          "randomized_itt": round(itt,3)}
expected = {"naive_user_difference": -1.25, "randomized_itt": -0.75}
assert result == expected, result
print(result)
```

Both estimates happen to be beneficial here, but they answer different questions and differ materially. The user comparison breaks randomization because usage is chosen. In real analysis report uncertainty and cluster effects; eight organizations are only a transparent fixture.

SQL for the north star

```
WITH mature AS (
  SELECT organization_id, root_cause_id,
         date(occurred_at, 'weekday 1', '-7 days') AS week
  FROM security_value_event
  WHERE outcome='durably_resolved' AND evidence_verified=1
     AND root_cause_id IS NOT NULL
), weekly AS (
  SELECT week, organization_id, COUNT(DISTINCT root_cause_id) AS roots
  FROM mature GROUP BY week, organization_id
)
SELECT week, COUNT(*) AS organizations_realizing_value,
       SUM(roots) AS distinct_org_root_causes
FROM weekly WHERE roots>0 GROUP BY week ORDER BY week;
```

Root causes are distinct within organization; the same library defect at two customers represents value to both. The week expression is SQLite-compatible and should be standardized in the semantic layer.

Product Failure Analysis

Metric gaming: alerts viewed rises when noise rises. Pair value events with false intervention, effort, recurrence, and exposure-time guardrails.

Instrumentation-induced selection: events appear only after successful setup, erasing failed onboarding. Begin the funnel at eligibility and preserve missing/failed states.

Identity errors: shared SOC accounts and service principals distort user retention. Model organization, persona, human actor, and agent separately.

Outcome delay: incident reduction may take months. Use pre-registered leading indicators, label them provisional, and revisit mature outcomes.

Interference: shared detections, global models, and cross-repository agent actions violate independent-unit assumptions. Randomize at the highest interacting cluster or estimate spillovers explicitly.

Novelty effects: usage spikes after launch. Measure repeated workflow completion and cohort retention after the novelty window.

Agent activity inflation: an agent can generate actions without value. Require bounded authority, action provenance, human override, rollback success, and outcome verification. Token count and tool calls are cost metrics, not security outcomes.

Survivorship: churned or breached customers disappear from telemetry. Join billing, support, and security outcome sources under a governed organization identifier and retain censoring reasons.

Rollout should progress from instrumentation validation to shadow measurement, randomized or phased exposure, and bounded expansion. Stop conditions should cover unauthorized actions, false interventions, customer effort, telemetry loss, and worsening protected outcomes. Rollback must restore compatible workflow and policy, not only a UI flag.

Building the Product Measurement System

Start with a semantic specification, not a dashboard. For every metric define eligibility, unit, event-time rule, late-arrival policy, identity resolution, deduplication, observation window, exclusions, and owner. Version the specification with product releases. A feature rename must not split history; a workflow redesign may require a deliberate metric version because “completion” has changed meaning.

Identity resolution is particularly difficult in enterprise security. One human may use several accounts, a managed service provider may act across tenants, and an agent may operate under a service principal. Preserve raw actor identifiers and map them to governed person, persona, organization, and automation entities with validity intervals. Never merge tenants because email domains match. Tenant isolation applies to analytics tables as well as product storage.

Event collection should represent state-changing facts rather than UI clicks whenever possible. `investigation_verified` is stronger than `button_clicked`, provided its evidence contract is stable. Keep raw interaction events for usability diagnosis but build value events from server-side workflow records. Include failures: attempted integrations, rejected agent actions, rollback attempts, and workflows abandoned after an error.

Time to value and retention

Time to value starts at a customer-relevant eligibility point—contract activation, completed prerequisite, or explicit trial start—and ends at a verified value event. Starting only after successful setup erases onboarding pain. Use survival curves because some organizations have not yet activated. Treat contract cancellation and trial expiry explicitly rather than dropping them.

Retention should follow the natural cadence of the job. A quarterly compliance workflow does not require weekly activity. Define retained value as repeated successful work when the need recurs, conditioned on continued eligibility. For automation, “no human sessions” may coexist with excellent retained value if policies remain healthy and verified outcomes continue.

Enterprise rollups need weighting choices. Organization-weighted metrics answer how many customers realize value; asset-weighted metrics answer how much protected surface does; revenue-weighted metrics answer commercial concentration. Publish them separately. A revenue-weighted north star can hide poor outcomes for smaller customers, while an unweighted average can understate systemic exposure in a few large tenants.

Experiment design under security constraints

Randomize the smallest unit that does not create contamination. If analysts share recommendations, randomize the queue or organization rather than individual alerts. If one repository’s fix updates a common library, repository-level assignment interferes. Document the interference graph and select clusters accordingly.

Before launch, specify the primary estimand, guardrails, minimum detectable effect, duration, and stopping rules. Sequential monitoring without correction inflates false positives. Security stop rules for harm can be asymmetric: stop immediately for unauthorized containment even if the primary outcome is inconclusive. Benefit stopping should be more conservative when outcomes are delayed.

Randomized encouragement is useful when customers choose adoption. Assignment might provide guided onboarding, an in-product prompt, or priority access; the feature remains optional. Intention-to-treat estimates the effect of encouragement. Using assignment as an instrument for adoption requires relevance, exclusion, and monotonicity assumptions that should be defended, not merely named.

When randomization is unavailable, phased rollout can support difference-in-differences if treated and comparison cohorts had credible parallel trends and no simultaneous differential change. Plot event-time effects and pre-period coefficients. Matching cannot repair unmeasured confounding. Regression discontinuity may be possible when eligibility follows a threshold, but manipulation around the threshold and limited external validity need attention.

Trust, explanations, and overrides

Trust should be calibrated reliance: users accept correct recommendations and reject incorrect ones. Measure acceptance conditional on audited correctness, not raw acceptance. Explanation quality can be tested by randomizing evidence formats and measuring correct decisions, time, and confidence. Longer explanations may feel transparent while obscuring the decisive fact.

Override taxonomies should permit multiple reasons and free text, then be reviewed for drift. “Policy exception” may conceal an incorrect recommendation if it becomes the easiest button. Forced reason codes can also add friction, so sample lightweight follow-up rather than burdening every action. Analyze whether particular teams cannot override because of role permissions.

For agents, record proposal, authorization, tools invoked, parameters, result, verification, and rollback as separate events. An agent that proposes the right action but calls the wrong tenant is unsafe. Outcome

analytics must join the action trace to a policy decision and protected asset without exposing sensitive prompts or secrets broadly.

Economic value without fictional precision

Translate outcomes into ranges. Analyst time saved can use loaded labor cost; avoided incident loss is much more uncertain. Report assumptions and sensitivity rather than a single inflated return-on-investment number. Separate realized savings, capacity redeployed, and modeled risk reduction. Ten hours “saved” are not cash savings if staffing and workload do not change, but they may be valuable capacity.

Customer value also includes option value: faster evidence during an incident, auditable control state, or safer automation. These may be measured through response time, audit effort, and bounded action coverage. Do not invent dollar values where operational measures are more defensible.

The product review should show a metric tree, causal evidence grade, segment distributions, reliability state, and unresolved uncertainties. Decisions then become explicit: expand, hold, redesign, or retire. A feature with high engagement but no outcome evidence may remain in discovery; a low-engagement automation with verified value may deserve expansion.

Measurement itself needs reliability objectives. Track event acceptance, schema rejection, end-to-end freshness, identity-match coverage, experiment-assignment integrity, and late outcome arrival. If assignment logs are incomplete, stop causal reporting rather than substituting observed use. If value events lag, label the current period provisional. A trustworthy product organization is willing to show an empty metric instead of manufacturing continuity from incompatible data.

Privacy review should minimize collection to the decision. Full source code, prompts, or investigation text rarely belongs in broad analytics. Prefer derived reason codes, bounded counts, and access-controlled evidence links. Define retention and deletion propagation before launch, especially when agents can capture secrets in traces.

Segment review is mandatory before declaring success. Tenant size, industry, geography, deployment model, and maturity can concentrate both benefit and harm. An average improvement does not authorize expansion when a regulated or low-maturity segment experiences unsafe interventions.

Sources and Further Reading

- **Rodden, Hutchinson, and Fu, HEART.** A useful user-experience lens, but security outcome and harm guardrails remain domain-specific. <https://research.google/pubs/measuring-the-user-experience-on-a-large-scale-user-centered-metrics-for-web-applications/>
- **Kohavi, Tang, and Xu, Trustworthy Online Controlled Experiments.** Foundations for assignment, exposure, guardrails, novelty, and experiment validity. <https://doi.org/10.1017/9781108653985>
- **Google SRE Workbook, Implementing SLOs.** Service reliability is necessary context for interpreting product funnels and safe degradation. <https://sre.google/workbook/implementing-slos/>
- **NIST Cybersecurity Framework 2.0.** Provides outcome-oriented security functions against which product workflows can be mapped. <https://www.nist.gov/cyberframework>

- **NIST SP 800-55 Volume 1.** Emphasizes measures tied to information-security objectives rather than activity counts. <https://csrc.nist.gov/pubs/sp/800/55/v1/final>

The synthesis is deliberate: HEART helps diagnose experience, experimentation literature supports causal claims, SRE constrains reliable delivery, and NIST anchors measurement to security outcomes. No framework licenses replacing protected outcomes with clicks.

Interview Practicum

Scenario: Engagement Falls After the Product Improves.

Prompt. An AI SOC assistant reduced weekly active analysts by 25%. The product leader wants to roll back. Decide what to do.

Ask whether eligible organizations changed, which personas declined, how exposure is logged, what work-flows completed, whether incidents and decision quality matured, and whether the assistant now automates work.

Worked Answer

Compare cohorts on verified cases, analyst minutes, false interventions, overrides, unresolved exposure, and incidents. Lower activity may be the intended outcome.

Follow-Up Questions and Answers

Change one: case-resolution time fell 40%, but analysts override 18% of containment recommendations. Stratify overrides by reason and consequence. Unsafe-action overrides may trigger a stop even when speed improves; policy exceptions may indicate configuration work. Audit accepted recommendations too, because non-override is not correctness.

Change two: only large customers received the feature, chosen by customer-success managers. Do not use untreated small customers as controls. Seek a randomized encouragement among eligible large customers or a phased rollout with credible pre-trends and matching. State that causal certainty is limited until assignment improves.

Change three: the agent begins auto-containment next quarter. The metric system must add authorization scope, attempted and executed actions, rollback success, business disruption, and human escalation. Randomization may be unethical for high-risk autonomous actions; use shadow decisions, simulation, red-team challenges, and bounded canaries before outcome experiments.

Weak answers maximize MAU, infer impact from correlation, or A/B test autonomous containment without a harm model. A Principal answer changes the north star so leadership benefits when customers need fewer interactions to achieve a verified outcome.

Key Takeaways

- Separate eligibility, assignment, exposure, adoption, workflow completion, and protected outcome.
- Define activation around first verified value, not login.
- Pair a security-value north star with friction, coverage, recurrence, and harm guardrails.
- Use randomization or defensible quasi-experiments; feature users are a selected population.
- Model personas, organizations, humans, and agents explicitly.
- Reduced usage can indicate improvement when the product removes work safely.

Chapter 33: Ranking, Triage, and Decision Support

The Queue Is the Product

A vulnerability ranker does not protect anything. Protection occurs when a finite group of analysts investigates the right findings soon enough, sends actionable cases to owners, and avoids spending its week on plausible-looking noise. The output is not a probability; it is a queue whose arrival rate, service time, deadlines, and error costs interact.

Suppose five analysts each have six effective investigation hours per day. A scanner produces 240 findings daily, requiring between ten minutes and two hours each. Sorting by severity cannot make 30 analyst-hours absorb 80 hours of arriving work. The system must decide what enters service, waits, is grouped, or is explicitly deferred. A silent tail is not abstention; it is unrecorded risk acceptance.

Chapter 30 separated finding quality from severity, and Chapter 31 treated remediation as a stateful workflow. Here the decision unit is a finding at a decision time. Its value depends on harm that timely review can avert, probability that it is real, time required to resolve uncertainty, and the deadline after which review loses value.

Capacity-Constrained Architecture

```

flowchart TB
  A[Scanner and researcher findings] --> B{Duplicate and identity resolver}
  B --> C[Point-in-time feature snapshot]
  C --> D[Probability calibration]
  D --> E[Expected avoidable-loss estimate]
  E --> F{Queue admission optimizer}
  F -->|assign| G1[Cloud analyst lane]
  F -->|assign| G2[Application analyst lane]
  F -->|abstain| H[Evidence request lane]
  F -->|defer| I[Risk-owned backlog]
  G1 --> J[Disposition and service time]
  G2 --> J
  J --> K[Policy replay and audit sampling]
  K --> C

```

Deduplication comes first because ten manifestations of one root cause must not occupy ten slots. The snapshot contains only admission-time evidence. Calibration converts raw scores into empirical probabilities within relevant cohorts. The optimizer accounts for effort and deadlines. Specialist lanes acknowledge that five analysts are not five interchangeable servers.

Every outcome must be named. `assigned` consumes capacity. `evidence_request` has an owner and expiry. `deferred` records who accepted delay and when it will be reconsidered. `suppressed_duplicate` points to its canonical case. These states expose displaced work.

Mathematics of Attention

For finding $\langle i \rangle$, define calibrated validity $\langle p_i \rangle$, impact $\langle L_i \rangle$, preventable fraction $\langle q_i(t) \rangle$ when service starts at $\langle t \rangle$, review effort $\langle s_i \rangle$, and friction cost $\langle c_i \rangle$. Expected net value is

$$\langle v_i(t) = p_i L_i q_i(t) - c_i \rangle$$

One urgency curve is $\langle q_i(t) = e^{-\lambda_i t} \rangle$; contractual deadlines may instead require a step. With capacity $\langle C \rangle$, admission resembles a knapsack:

$$\langle \max_{x_i \in \{0,1\}} \sum_i x_i v_i(0) \text{ s.t. } \sum_i x_i s_i \leq C \rangle$$

Sorting by probability is wrong when a moderately likely privilege escalation dominates an almost-certain informational issue. Sorting by value density $\langle v_i/s_i \rangle$ is a transparent baseline, but can starve large valuable cases. Reserve capacity for high-impact long cases and random below-threshold audits.

For analyst $\langle j \rangle$, let $\langle a_{ij} \rangle$ indicate skill compatibility and $\langle C_j \rangle$ capacity. Assignment variables satisfy $\langle \sum_i s_{ij} x_{ij} \leq C_j \rangle$, $\langle \sum_j x_{ij} \leq 1 \rangle$, and $\langle x_{ij} \leq a_{ij} \rangle$. This exposes specialist shortages instead of burying them in scores.

Calibration, abstention, and learning

Ranking metrics do not show whether the first case merits interruption. Reliability diagrams and Brier score test whether 0.8 means roughly eight valid findings in ten. Calibrate on later time periods and segment by scanner, asset, and evidence regime; a global calibration map can hide overconfidence in a new detector.

Abstain when required evidence is absent, inputs fall outside supported conditions, or estimates are too uncertain. An abstention should request a specific artifact—reachable-route evidence, package version, or exploit precondition—not merely announce low confidence. Evidence-acquisition cost belongs in the decision.

Triage determines which labels become observable. Review a probability sample below the cutoff. Given inclusion probability $\langle \pi_i \rangle$, use weights $\langle 1/\pi_i \rangle$ when estimating prevalence or recall. Otherwise the system congratulates itself for precision while blind regions accumulate.

Measure confirmed avoidable loss per analyst-hour, deadline-weighted recall, wait time by impact, backlog age, duplicate effort, and calibration. Precision at $\langle k \rangle$ is meaningful only when $\langle k \rangle$ equals service capacity. Replay candidate and incumbent policies over identical arrivals and service times.

Executable Lab: A Two-Lane Shift

This standard-library simulation creates synthetic cases and compares severity ordering with capacity-aware value density. It uses no network, credentials, files, or subprocesses.

```
from dataclasses import dataclass
from random import Random
```

```

@dataclass(frozen=True)
class Finding:
    fid: str
    domain: str
    severity: int
    probability: float
    loss: int
    minutes: int
    sla: int

def workload(seed=7):
    rng = Random(seed)
    rows = []
    for n in range(30):
        rows.append(Finding(
            f"F{n:02}", ("app", "cloud")[n % 2], rng.randint(2, 10),
            round(rng.uniform(.18, .96), 2),
            rng.choice((5, 10, 25, 60, 120)),
            rng.choice((15, 30, 45, 60, 90)),
            rng.choice((120, 240, 480))))
    return rows

def value(f, start):
    return f.probability * f.loss * (1.0 if start <= f.sla else .35)

def simulate(items, policy):
    free = {"app": 0, "cloud": 0} # one specialist per lane
    key = ((lambda f: (-f.severity, f.fid)) if policy == "severity" else
           (lambda f: -(f.probability * f.loss / f.minutes), f.fid))
    captured = late = reviewed = 0
    for f in sorted(items, key=key):
        start = free[f.domain]
        if start + f.minutes > 480:
            continue
        captured += value(f, start)
        late += start > f.sla
        reviewed += 1
        free[f.domain] += f.minutes
    return reviewed, captured, late, sum(free.values())

items = workload()
for name in ("severity", "capacity"):
    n, v, late, minutes = simulate(items, name)
    print(f"{name:8} reviewed={n:2} value={v:6.1f} "
          f"late={late:2} minutes={minutes:3}")

```

Expected output:

```

severity reviewed=27 value= 595.4 late= 9 minutes=915
capacity reviewed=28 value= 844.5 late= 7 minutes=885

```

The capacity policy wins through objective alignment, not sophistication. Add online arrivals, duplicate clusters, evidence requests, or reserved audit capacity. In an online implementation priorities must age as deadlines approach; a static priority-heap key becomes stale.

Operational Failure Analysis

Fast-case starvation

Value density can fill a shift with small wins and postpone a two-hour catastrophic case. Track maximum wait by risk band and reserve large-case capacity. An aging term prevents permanent starvation, but should not allow old low-value work to bury urgent arrivals.

Service times change with workflow

Historical duration reflects case difficulty and the old evidence interface. After evidence packets improve, old estimates become biased. Log active work separately from calendar duration, version the evidence bundle, and monitor residuals by case type. Do not turn these data into punitive individual rankings; analysts will game them.

Labels encode attention

“Confirmed” is mostly observed for reviewed cases; “closed benign” may mean time expired. Keep disposition, truth, and remediation separate. Random audits and researcher-confirmed challenge cases provide evidence outside the selected region.

Fundamental overload

If mean arriving work exceeds service capacity— $\lambda \geq m$ for m equivalent analysts—no queue policy is stable. Use deduplication, admission control, automation, staffing, or threshold changes. A model cannot repeal conservation of work.

Adversarial flooding

An attacker or broken integration can generate superficially urgent findings and displace genuine work. Apply source budgets, correlated-finding collapse, provenance checks, and diversity constraints. Alert on concentration of assigned capacity by source and root cause.

Automation bias

Analysts need evidence-linked reason codes. Overrides should capture a structured reason without harming a private performance score. Otherwise disagreement disappears and the ranker loses its most valuable correction channel.

Decision records and review cadence

For every policy release, preserve the training interval, calibration interval, replay interval, feature definitions, cost table, capacity assumptions, solver version, tie-breaking rule, reserved-capacity rules, and approving risk owner. The record should include rejected alternatives and sensitivity analysis. If modest changes to loss estimates reverse most assignments, the policy is fragile and should expose that uncertainty to operators.

Review cadence follows change, not the calendar alone. Re-evaluate after scanner releases, staffing changes, major incidents, new contractual SLAs, or material shifts in service time. A monthly governance meeting can inspect backlog composition and override themes, while automated monitors run continuously. Never retrain directly from recent dispositions without checking whether the old policy created the sample.

The user interface is part of the queue algorithm. Show the evidence that drives priority, the deadline, expected effort range, duplicate cluster, and why the case is in this analyst's lane. Permit analysts to request reassignment or flag corrupted evidence. Hide speculative monetary precision when impact is ordinal; false exactness damages trust.

Finally, distinguish remediation priority from investigation priority. A case may be cheap to validate but expensive to fix, or hard to validate but trivial to mitigate. The triage queue ends with a disposition and evidence package; remediation enters the lifecycle described in Chapter 31 with different owners, capacities, and deadlines. Combining both into one score obscures where work is actually constrained.

Source Synthesis

Combining investigation and remediation also corrupts measurement: remediation latency includes engineering ownership, release windows, and business acceptance that the ranker cannot control. Report handoff quality and downstream outcomes, but optimize the triage decision against consequences it can influence. This boundary keeps accountability fair and makes experiments interpretable. A later causal analysis may estimate whether faster validation changes remediation, but that claim requires its own design rather than an assumed chain of credit.

From Offline Scores to a Queue Policy

Productionization begins with a replay table, not a model endpoint. One row represents a case becoming eligible, with arrival time, evidence version, canonical root-cause identifier, skill requirement, estimated work, deadline, eventual disposition, and observed service intervals. Reconstruct the queue after every arrival and completion. This reveals whether apparent lift depends on future workload, labels, or true case duration.

Service-time prediction deserves its own loss function. Underestimating a long case crowds out deadlines; overestimating it leaves valuable capacity idle. Quantile estimates are often more useful than a mean. A scheduler can reserve the 75th-percentile duration for feasibility while estimating utilization from the median. Evaluate errors separately for cases that timeout, transfer, or wait on another team.

Policies need deterministic tie-breaking. If equal scores resolve by database return order, replay is unreliable. Use deadline, value density, arrival time, then stable case identifier. Record why an item moved: probability revision, impact revision, aging, capacity change, or override.

Analysts as information gatherers

Review is not a binary label factory. Analysts request artifacts, reproduce behavior, and discover clusters. A two-stage policy can spend five minutes resolving cheap uncertainty before committing to a full investigation. Expected value of information for evidence (e) is

$$[\text{EVI}(e) = E[\max_a U(a \mid e)] - \max_a E[U(a)] - \text{Cost}(e).]$$

This separates uncertain but cheaply resolvable cases from cases with no available evidence. The first belongs in a probe lane; the second may require abstention. Capture which artifact changed the decision so product teams can improve finding packets.

Fairness and tenant obligations

A globally efficient queue can starve small tenants because large customers generate more high-scoring cases, or starve legacy environments whose evidence is incomplete. Define whether guarantees are contractual, risk-based, or both. Constraints may set maximum wait per tenant, minimum review share for critical infrastructure, or separate capacity pools. Publish their displaced expected value so governance makes a conscious trade.

Check calibration and service across relevant groups, but avoid simplistic parity when exposure and contracts differ. Ask whether similarly consequential cases receive comparable treatment and whether evidence gaps systematically deny review. Improve instrumentation before correcting scores for missing evidence.

Rollout and incident response

Shadow mode must simulate assignment timing, not generate a second ordering at day end. During canary rollout, cap the capacity controlled by the candidate and retain a manual freeze. Trigger incidents on critical SLA misses, source concentration, calibration collapse, or backlog growth beyond recoverable capacity. Rollback restores the old admission policy without discarding already displaced cases.

After an incident, replay arrivals under both policies and identify the first divergent decision. Distinguish model error from optimizer error, stale evidence, capacity loss, or operator override. Correcting the wrong layer creates recurring failure; the queue decision log is as important as the trained artifact.

Learning-to-rank work supplies ordering objectives, but security triage adds asymmetric loss and finite service. Queueing and optimization supply admission and assignment constraints. Calibration research explains why order is insufficient for thresholds; selective classification supplies risk–coverage reasoning. CVSS describes severity, not an enterprise’s queue objective. NIST measurement guidance requires the resulting service to be assessed against operational outcomes. Their combined lesson is to evaluate model, optimizer, and workflow as one decision system.

Sources and Further Reading

- NIST, *Cybersecurity Framework 2.0*: <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP29.pdf>
- NIST SP 800-55 Volume 1: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP800-55v1.pdf>
- Burges et al., *Learning to Rank using Gradient Descent*: https://icml.cc/2015/wp-content/uploads/2015/06/icml_ranking.pdf
- Guo et al., *On Calibration of Modern Neural Networks*: <https://proceedings.mlr.press/v70/guo17a/guo17a.pdf>
- Geifman and El-Yaniv, *Selective Classification*: <https://arxiv.org/pdf/1705.08500>
- FIRST, *CVSS v4.0 Specification*: <https://www.first.org/cvss/v4-0/specification-document>

Interview Practicum

You inherit triage for a cloud-security product. Five analysts receive 1,200 weekly findings; only 300 can be reviewed within SLA. Leadership requests “the top 300.” After your initial design, constraints change: one analyst alone can investigate identity cases, 40 percent are duplicates, labels exist only above the old threshold, and a strategic contract requires every critical finding to receive a human touch in four hours.

Explain the decision unit, objective, allocation algorithm, calibration, exploration, rollout, and the metric you will refuse to optimize.

Worked Answer

I would not promise a universal top 300. First canonicalize duplicate manifestations into root-cause cases while retaining affected assets and contractual obligations. Each row is a canonical case at admission. Estimate calibrated validity, impact, preventable fraction, service time by skill, and deadline. Maximize expected avoidable loss subject to skill capacities, the four-hour critical-touch constraint, and a reserved audit budget.

The identity specialist is an explicit bottleneck. Critical cases receive a quick validation touch even when evidence is weak; contractual service is not proof they deserve full investigation. Duplicate clusters share investigation but fan out remediation ownership.

Old labels are policy-selected, so run stratified random review below the cutoff and weight by selection probability. Calibrate temporally within scanner and asset cohorts. Replay both policies on identical arrivals. Launch through shadow mode and a canary. Require positive lower-bound improvement in avoidable loss per hour, fewer critical SLA misses, bounded backlog age, and no material segment regression. Refuse to optimize closure count because it rewards easy dismissals and hides displaced work.

Follow-Up Questions and Answers

The identity specialist takes leave tomorrow. What changes?

Expose infeasibility immediately. Route critical cases to a prequalified backup, pause low-value identity admission, and show the risk owner which deadlines cannot be met. Never silently assign unqualified staff.

A scanner doubles volume after an update. Is that model drift?

It is operational drift even if features did not shift. Quarantine the version behind a source budget, compare duplication and yield, and prevent it from consuming the queue during validation.

Analysts repeatedly override the ranker. What do you do?

Sample overrides and separate missing evidence, wrong cost assumptions, and wrong predictions. Correct the relevant layer. High-quality disagreement is system evidence.

When should ML ranking be abandoned?

When an auditable rule policy achieves equivalent queue utility, outcomes cannot support calibration, or contractual constraints determine nearly every action.

Key Takeaways

- A ranker is useful only relative to arrivals, skills, service times, deadlines, and capacity.
- Optimize expected avoidable loss, not severity or probability alone.
- Make abstention, deferral, duplication, and displaced work explicit.
- Evaluate at service capacity and audit below the cutoff.
- Queue overload is a capacity fact, not a modeling defect.
- Human overrides are evidence, not disobedience.

Chapter 34: Foundations of Agentic AI Security

Authority, Not Intelligence, Defines the Blast Radius

An enterprise coding agent reads repositories, searches tickets, runs tests, proposes patches, and sometimes deploys. Its language model is only one component. Security depends on which principal the system represents, which objects it can observe, which actions it can cause, and which evidence can influence those actions. A mediocre model with production credentials is more dangerous than an exceptional model confined to a disposable workspace.

Traditional application diagrams emphasize data flow. Agent threat models need authority flow. A retrieved issue can contain instructions; a generated plan can select a tool; a tool broker can mint a credential; a deployment service can change production. The important question is not merely “can untrusted text reach the model?” It is “can that text cross a consequence boundary without an independent authorization decision?”

A Typed Authority Graph

Model the system as $(G=(V,E))$. Every vertex has a type: `Principal`, `Agent`, `Model`, `Data`, `Tool`, `Credential`, `Environment`, or `Policy`. Every directed edge has a relation such as `READS`, `INFLUENCES`, `INVOKES`, `AUTHORIZES`, `MINTS`, `WRITES`, or `DELEGATES`. Edge attributes include scope, tenant, expiry, conditions, and provenance.

This is not cosmetic typing. A data-flow path `issue -> model -> deployment` may be expected. An authority path containing `untrusted Data INFLUENCES Agent`, `Agent INVOKES Tool`, and `Tool WRITES production Environment` is a candidate confused-deputy path unless an authorization edge from an accountable principal constrains the write.

```

flowchart LR
    subgraph Untrusted[Untrusted evidence plane]
        I[Issue text: Data]
        R[Repository content: Data]
    end
    subgraph Reasoning[Reasoning plane]
        A[Coding agent: Agent]
        M[Language model: Model]
        W[Ephemeral workspace: Environment]
    end
    subgraph Authority[Authority plane]
        P[Engineer: Principal]
        B[Tool broker: Policy]
        C[Scoped token: Credential]
    end
    subgraph Consequence[Consequence plane]
        T[Deploy API: Tool]
        E[Production: Environment]
    end

```



```

end
I -->|INFLUENCES| A
R -->|READS| A
A -->|QUERIES| M
A -->|WRITES| W
P -->|AUTHORIZES change digest| B
B -->|MINTS short lease| C
C -->|SCOPES| T
A -->|INVOKES| T
T -->|WRITES| E

```

The diagram deliberately separates reasoning from authority. The model never receives a reusable production secret. The broker evaluates an immutable action proposal: repository, commit digest, environment, operation, expiry, and approver. Approval of “deploy something later” is too broad; approval must bind to what will occur.

Assets, Boundaries, and Abuse Cases

Protect more than source code. Assets include signing identities, CI provenance, private dependencies, customer data in test fixtures, system prompts, tool schemas, long-term memory, audit evidence, approval channels, and the integrity of generated changes. Availability matters too: an agent can create costly build loops or exhaust API quotas without exfiltrating anything.

Trust boundaries occur at retrieval, model providers, plugin/tool servers, workspaces, credential brokers, approval interfaces, memory stores, and agent-to-agent messages. Treat retrieved content as data even when it looks like policy. A repository file named `AGENT_INSTRUCTIONS.md` may legitimately guide local behavior, but it cannot grant deployment rights. Only the authority plane can do that.

Threats fall into interacting families:

1. **Indirect prompt injection.** An issue, README, test output, web page, or dependency description tells the agent to ignore policy, reveal context, or invoke a tool.
2. **Excessive agency.** A legitimate objective is paired with permissions broader than required—write access to every repository, wildcard cloud roles, or an unbounded shell.
3. **Confused delegation.** A parent agent gives a subagent a task but fails to attenuate credentials or communicate prohibitions. The child becomes a laundering step for authority.
4. **Memory poisoning.** Untrusted claims become durable preferences or facts and influence later tasks after their origin is forgotten.
5. **Tool semantic mismatch.** A tool called `preview_change` performs writes, parameters have surprising defaults, or the agent misunderstands an idempotency boundary.
6. **Supply-chain substitution.** Models, MCP/tool servers, packages, container images, or retrieved indexes change without compatible attestation.
7. **Approval capture.** The interface hides the true diff, batches unrelated actions, or conditions humans to approve repetitive prompts.

Risk for an abuse path can be prioritized as

$R(\text{path}) = P(\text{activation}) \times P(\text{policy} \setminus \text{bypass} \mid \text{activation}) \times \text{Impact}(\text{authority}) \times \text{Persistence}$.

Do not pretend these factors are precisely known. Use ordinal ranges and test the paths. The graph is valuable because it identifies where to cut reachability: remove an edge, attenuate its scope, insert a policy decision, or change the target to a sandbox.

Design Rules for Coding Agents

Capabilities should be task-scoped, object-scoped, action-scoped, tenant-scoped, and time-bounded. Mint them just in time. A read token should not imply write; repository write should not imply merge; merge should not imply deployment. Separate the identity that proposes a change from the identity that approves it.

Run generated code only in disposable environments with no ambient credentials, constrained network egress, resource quotas, and immutable base images. “Sandboxed” is a claim requiring escape testing and a documented host boundary, not a checkbox.

Canonicalize tool requests before policy evaluation. Validate types, resolve symbolic references, make defaults explicit, and hash the normalized proposal. Bind approval to that digest. Re-evaluate policy at execution because repository state, approver role, or environment freeze may have changed.

Memory needs provenance, scope, expiry, and deletion. Separate user-authored preferences, observed facts, model hypotheses, and security policy. A model-generated note cannot overwrite policy. Cross-tenant memory is forbidden unless a separately governed aggregation explicitly permits it.

Executable Lab: Detect Dangerous Authority Paths

The lab builds a typed graph and rejects paths where untrusted data can influence a production write without an intervening authorization node. It is intentionally small enough to audit and uses only synthetic objects.

```
from dataclasses import dataclass
from collections import defaultdict, deque

@dataclass(frozen=True)
class Node:
    name: str
    kind: str
    trusted: bool = True

@dataclass(frozen=True)
class Edge:
    src: str
    rel: str
    dst: str

nodes = {
```

```

n.name: n for n in [
    Node("issue", "Data", False), Node("agent", "Agent"),
    Node("shell", "Tool"), Node("broker", "Policy"),
    Node("approval", "Principal"), Node("deploy", "Tool"),
    Node("prod", "Environment")]
}

def unsafe_paths(edges):
    graph = defaultdict(list)
    for edge in edges:
        graph[edge.src].append(edge)
    queue = deque(["issue", ["issue"], False])
    bad = []
    while queue:
        here, path, authorized = queue.popleft()
        for edge in graph[here]:
            now_auth = authorized or edge.rel == "AUTHORIZES"
            new_path = path + [f"--{edge.rel}->", edge.dst]
            if edge.rel == "WRITES" and edge.dst == "prod" and not now_auth:
                bad.append(" ".join(new_path))
            elif edge.dst not in path and len(path) < 12:
                queue.append((edge.dst, new_path, now_auth))
    return bad

unsafe = [Edge("issue", "INFLUENCES", "agent"),
          Edge("agent", "INVOKES", "deploy"),
          Edge("deploy", "WRITES", "prod")]
safe = [Edge("issue", "INFLUENCES", "agent"),
        Edge("agent", "PROPOSES", "broker"),
        Edge("approval", "AUTHORIZES", "broker"),
        Edge("broker", "AUTHORIZES", "deploy"),
        Edge("deploy", "WRITES", "prod")]

for name, edges in (("unsafe", unsafe), ("safe", safe)):
    bad = unsafe_paths(edges)
    print(f"{name}: dangerous_paths={len(bad)}")
    if bad:
        print(" ", bad[0])

```

Expected output:

```

unsafe: dangerous_paths=1
  issue -INFLUENCES-> agent -INVOKES-> deploy -WRITES-> prod
safe: dangerous_paths=0

```

The program is a design linter, not a proof of safety. Its authorization state is path-local; a production version must verify that approval actually binds to the proposed action, that the principal is eligible, and that no alternate write edge exists. Useful extensions check credential expiry, delegation attenuation, tenant equality, and forbidden `Data -> AUTHORIZES` relationships.

Operational Failure Analysis

Ambient credentials defeat the graph. If the workspace inherits a cloud token, the documented broker is fiction. Scan process environments and metadata endpoints from inside the sandbox; deny credential-bearing mounts by default.

Approval becomes ritual. Hundreds of low-context prompts teach operators to click. Aggregate only actions sharing a consequence boundary, show semantic diffs and destination, and require fresh confirmation for scope expansion. Measure approval reversals and time-to-review, not approval rate.

Tool descriptions drift. Policy may approve yesterday's semantics. Pin tool versions, verify manifests, test side effects, and record the exact schema digest used for each call.

Delegation amplifies authority. A parent may have permissions that a research subagent does not need. Derive a child capability as an intersection of parent scope and task scope; never copy the parent token.

Egress turns read access into disclosure. A read-only agent can still leak secrets through HTTP, DNS, package names, logs, or pull-request text. Apply destination allowlists, content controls, secret scanning, and rate limits while recognizing none is perfect.

Rollback is incomplete. Reverting a commit does not revoke a leaked credential or remove published artifacts. Incident playbooks must cover credential rotation, package yanking, audit preservation, memory cleanup, and downstream notification.

Residual risk decisions

Not every dangerous path can be removed without eliminating the product. Document residual paths with preconditions, maximum authority, detective controls, recovery steps, and an accountable owner. “The model is unlikely to do that” is not ownership. A valid acceptance states why preventive control is disproportionate and when the decision expires.

Threat models decay as tools and workflows change. Re-run graph checks when adding a tool, widening repository scope, changing memory, enabling a model provider, or introducing delegation. Production telemetry should be compared with the graph: an observed edge absent from the model is an architecture incident, even when no harm occurred.

Source Synthesis

Schedule an explicit quarterly review even without announced changes, because undocumented integrations and emergency exceptions often create the most consequential new edges. Compare inventories, credentials, and observed tool calls against the approved graph.

Conducting the Threat-Model Workshop

Begin with one mission: “repair a vulnerable dependency and deploy it to staging.” Ask the owner to narrate the successful path while a facilitator creates typed nodes and edges. Record where identity changes, content

crosses tenants, executable material enters, and irreversible effects occur. Challenge every authority edge: who granted it, what object and action does it cover, how long does it live, and which component mediates every use?

Mark data an attacker can influence directly or through a supply chain. Trace forward while preserving edge types. Reachability to a tool is not automatically a vulnerability; the decisive issue is whether influence can shape a consequential request that passes authorization. Absence of a direct arrow does not prove safety when shared memory, environment variables, caches, or human copy-and-paste create implicit edges.

Write abuse cases as executable propositions: “Given a malicious issue and legitimate bug-fix task, the agent causes CI to send a repository secret to an attacker endpoint.” Include preconditions, observable success, and assumed controls. This wording becomes a Chapter 36 scenario and determines Chapter 35 telemetry.

Policy as reference monitor

A trustworthy broker approximates complete mediation, tamper resistance, and analyzability. Every consequential invocation passes through it, including retries, alternate SDKs, and emergencies. It evaluates authenticated actor, tenant, tool, normalized object, action, environment, proposal digest, approval, time, and organizational state—not the model’s opinion of safety.

Decisions return reason codes and obligations. An allow may require restricted networking, maximum spend, or post-action verification. A deny identifies the rule without exposing bypass details. An escalation names eligible approvers and expiry. Unknown tool versions or unparsable arguments fail closed.

Multi-agent systems

Agent messages combine data and delegation. A planner can request research without granting repository access; an implementer can receive a workspace capability without cloud credentials. Attach a task contract containing objective, input references, allowed outputs, capabilities, expiry, and parent trace. Receivers must not infer authority from reputation or natural-language urgency.

Cycles are dangerous. Agents can reinforce an unverified claim or repeatedly delegate until the original constraint disappears. Bound depth, preserve the intersection of prohibitions, and demand fresh authorization for out-of-contract actions. Monitor resources across the whole delegation tree.

Assurance evidence

Replace “sandboxed” with testable claims: no host mount, egress only through a dependency proxy, cloud metadata unreachable, bounded resources, verified base image, and destroyed writable state. Each needs configuration evidence and adversarial testing.

Likewise, “human in the loop” is incomplete. Evidence must show the approver was authenticated, eligible, shown the canonical action and semantic diff, and that execution matched the approved digest before expiry. Without reconstruction, approval is ceremony.

NIST AI RMF supplies governance and lifecycle vocabulary; NIST’s Generative AI Profile focuses it on generative systems. OWASP catalogs practical LLM and agent failure patterns. MITRE ATLAS connects those patterns

to adversary tactics and techniques. Saltzer and Schroeder's least privilege and complete mediation remain the decisive architecture principles. SLSA adds artifact provenance. Together they imply that model alignment cannot substitute for external, typed, continuously mediated authority.

Sources and Further Reading

- NIST AI RMF 1.0: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
- NIST AI 600-1, *Generative AI Profile*: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
- OWASP, *Top 10 for LLM Applications*: <https://genai.owasp.org/llm-top-10/>
- MITRE ATLAS knowledge base: <https://atlas.mitre.org/>
- Saltzer and Schroeder, *The Protection of Information in Computer Systems*: <https://web.mit.edu/Saltzer/www/publications/protection/Basic.html>
- SLSA specification: <https://slsa.dev/spec/v1.0/>
- IETF RFC 9334, *Remote Attestation procedureS Architecture*: <https://www.rfc-editor.org/rfc/rfc9334.txt>

Interview Practicum

Threat-model a coding agent that reads private repositories, issues, and CI logs; writes branches; runs tests; and deploys to staging. Halfway through, the interviewer adds that product managers may paste customer tickets containing secrets, a subagent searches the public web, and emergency production deployment is desired during incidents.

Produce a typed authority graph, identify three abuse paths, and propose controls that preserve useful autonomy. Explain what evidence would convince you the controls operate as designed.

Worked Answer

I would type principals, data, agents, tools, credentials, environments, and policies, then enumerate `INFLUENCES`, `READS`, `INVOKES`, `MINTS`, `AUTHORIZES`, and `WRITES` edges. The primary abuse path is customer ticket to agent to egress tool, causing disclosure. The second is repository injection to branch writer to CI, causing execution with CI secrets. The third is web subagent to parent plan to emergency deploy, a confused-deputy path.

Tickets enter a secret-redaction boundary and remain untrusted. Public-web retrieval runs under a child capability with no repository or write access. Tests execute in an ephemeral, no-credential workspace with restricted egress. Branch writes use repository-scoped, short-lived tokens. Staging deploy requires a brokered token bound to artifact digest and environment. Production emergency access is a separate break-glass workflow with eligible incident commander approval, short expiry, immutable logging, and post-incident review; the agent cannot activate it.

I would verify the design with graph reachability tests, sandbox escape and egress tests, injected-content scenarios, credential canaries, tool-schema integrity checks, and audit replay showing each consequential

action has a matching authorization digest. I would not claim that prompt filtering makes hostile content trusted.

Follow-Up Questions and Answers

The agent needs package downloads during tests. How do you constrain egress?

Use an authenticated dependency proxy with pinned namespaces, malware scanning, quotas, and no arbitrary URL fetch. Record resolved package digests. Exceptions require a new scoped decision.

Can the same human propose and approve an emergency deployment?

Only if explicit risk policy accepts that reduced separation during a declared incident. Make the exception visible, time-bound, and reviewed; do not let the agent infer emergency status.

Is read-only repository access low risk?

No. It exposes secrets, unreleased code, vulnerabilities, and customer context. Disclosure paths and cross-tenant mixing can be severe without any write.

Where should policy execute?

Outside the model and as close as practical to the consequential tool, with canonical inputs and complete mediation. Model instructions can guide behavior but cannot enforce authority.

Key Takeaways

- Blast radius follows authority, not model capability.
- Type nodes and edges so data influence cannot masquerade as authorization.
- Bind approvals and short-lived credentials to canonical action digests.
- Attenuate authority across every delegation.
- Treat memory, tools, models, and retrieved content as supply-chain surfaces.
- Prompt-injection resistance is defense in depth, never the authorization boundary.

Chapter 35: Instrumenting AI Agents for Security Evidence

A Trace Must Answer a Forensic Question

An agent trace is not a transcript. A transcript helps a developer watch a demo; security evidence must establish who initiated a run, what evidence influenced it, which policy authorized each consequential action, what tool actually executed, and whether the record was altered. Storing every prompt forever is neither necessary nor defensible. It increases breach impact while still failing to prove provenance.

Chapter 34 modeled authority paths. Instrumentation should make those typed edges observable. If a coding agent deploys an artifact, an investigator needs the proposal digest, approver, policy version, credential lease, tool request and response, artifact identity, and ordering relationship. The exact prose of an unrelated brainstorming turn may add privacy risk without forensic value.

Typed Trace Envelope

Use a small stable envelope with typed payloads. Required envelope fields are `trace_id`, `event_id`, `parent_id`, `sequence`, `event_time`, `ingest_time`, `actor`, `event_type`, `schema_version`, `policy_version`, `sensitivity`, `payload_digest`, `previous_hash`, and `event_hash`. Payloads vary by event type but are schema-validated.

Event type	Required payload fields	Security question
<code>run.started</code>	initiator, tenant, objective digest	Who requested what?
<code>evidence.observed</code>	source type, locator, content digest, trust label	What influenced reasoning?
<code>model.completed</code>	provider, model, input/output digests, finish reason	Which model transformation occurred?
<code>tool.proposed</code>	tool, canonical arguments digest, consequence class	What did the agent intend?
<code>policy.decided</code>	decision, reason codes, subject, object, action	Why was it allowed or denied?
<code>approval.recorded</code>	principal, proposal digest, expiry	What exactly was approved?
<code>tool.completed</code>	tool version, result digest, status, side effects	What actually happened?
<code>memory.changed</code>	namespace, old/new digest, provenance, expiry	What durable state changed?
<code>run.ended</code>	outcome, terminal reason, evidence summary digest	How did the run terminate?

Free-form text is referenced through protected content objects rather than copied into every event. Store a salted digest, classification, encrypted locator, retention class, and redacted preview. High-sensitivity content may be deleted while the event envelope and digest remain, proving that a particular object was used without preserving it indefinitely.


```
sequenceDiagram
    participant A as Agent runtime
    participant R as Redaction gateway
    participant L as Append-only trace log
    participant K as Key service
    participant V as Verifier
    participant C as Cold evidence vault
    A->>R: typed event + sensitive payload
    R->>C: encrypted content object (policy permitting)
    R->>K: request tenant signing operation
    K-->>R: signature/key identifier
    R->>L: envelope + digest + previous hash + signature
    L-->>A: durable sequence receipt
    V->>L: fetch chain segment
    V->>K: obtain public verification material
    V-->>V: validate schema, order, hashes, signatures
```

This architecture separates collection, content protection, integrity, and verification. The runtime cannot choose a weaker retention class. The log acknowledges only durable writes. A verifier independent of the producer detects deletion, insertion, reorder, and modification within the chain's guarantees.

Integrity and Provenance

Canonical serialization is mandatory. Hashing ordinary JSON is unsafe because key order, number rendering, and Unicode forms may differ. Define UTF-8 encoding, normalized strings, sorted keys, no insignificant whitespace, and permitted scalar representations. Better, adopt a documented canonical format such as JSON Canonicalization Scheme.

For canonical event bytes $\backslash(b_i\backslash)$, payload digest $\backslash(d_i\backslash)$, and previous hash $\backslash(h_{i-1}\backslash)$, compute

$$\backslash[h_i = H(\text{domain} \backslash \text{separator} \backslash \text{parallel } h_{i-1} \backslash \text{parallel } b_i \backslash \text{parallel } d_i) . \backslash]$$

The domain separator prevents the hash from being reused in another protocol. A hash chain detects changes only if a trusted party knows an anchor. Periodically sign checkpoint tuples (tenant, stream, first_sequence, last_sequence, root_hash, time) using a protected key and publish them to a separately administered store. Otherwise an attacker controlling the log can rewrite the whole chain.

Hash chains do not prove that an event is true or complete. A compromised producer can faithfully hash lies or omit a call. Corroborate high-consequence actions with tool-side logs, cloud audit records, deployment attestations, and credential-broker events. Compare expected state transitions with observed transitions and alert on sequence gaps.

Causality and clocks

Wall clocks drift. Use a monotonic per-stream sequence and `parent_id` for causal relationships; keep event and ingest times for latency analysis. Concurrent subagents form a directed acyclic graph rather than one global order. Each stream can chain independently, while join events name the terminal hashes of child streams.

Privacy and retention

Data minimization begins before logging. Record normalized tool parameters where needed, but tokenize secrets and omit raw credentials categorically. Separate operational telemetry from restricted content. Access to content requires a case-bound reason, is itself logged, and can be more tightly retained than envelopes.

Retention should follow consequence and purpose: policy decisions and deployment receipts may survive for audit obligations; raw prompts containing customer data may expire in days; cryptographic digests and aggregate metrics may remain longer. Deletion must cover replicas, indexes, caches, and derived embeddings. A digest can itself be personal data when the input space is guessable, so salt or key sensitive digests and control verification access.

Executable Lab: Schema and Hash-Chain Verifier

The lab validates event types and required fields, creates a deterministic chain, verifies it, then demonstrates tamper detection. It uses synthetic payloads and the standard library.

```
import hashlib, json
from copy import deepcopy

REQUIRED = {
    "run.started": {"initiator", "objective_digest"},
    "policy.decided": {"decision", "proposal_digest", "reason"},
    "tool.completed": {"tool", "status", "result_digest"},
}

def canonical(value):
    return json.dumps(value, sort_keys=True, separators=(",", ":"),
                      ensure_ascii=False).encode()

def seal(events):
    previous = "0" * 64
    sealed = []
    for sequence, raw in enumerate(events, 1):
        event = deepcopy(raw)
        missing = REQUIRED[event["event_type"]] - event["payload"].keys()
        if missing:
            raise ValueError(f"missing fields: {sorted(missing)}")
        event["sequence"] = sequence
        event["previous_hash"] = previous
        body = canonical(event)
        event["event_hash"] = hashlib.sha256(b"agent-trace-v1\0" + body).hexdigest()
        previous = event["event_hash"]
        sealed.append(event)
    return sealed

def verify(chain):
    previous = "0" * 64
    for expected, event in enumerate(chain, 1):
```

```

    claimed = event["event_hash"]
    body = {k: v for k, v in event.items() if k != "event_hash"}
    actual = hashlib.sha256(b"agent-trace-v1\0" + canonical(body)).hexdigest()
    if event["sequence"] != expected or event["previous_hash"] != previous:
        return False, f"ordering failure at {expected}"
    if claimed != actual:
        return False, f"hash failure at {expected}"
    previous = claimed
    return True, f"verified {len(chain)} events; anchor={previous[:12]}"

events = [
    {"event_type": "run.started", "payload": {"initiator": "alice",
        "objective_digest": "sha256:111"}},
    {"event_type": "policy.decided", "payload": {"decision": "allow",
        "proposal_digest": "sha256:222", "reason": "staging-only"}},
    {"event_type": "tool.completed", "payload": {"tool": "deploy",
        "status": "ok", "result_digest": "sha256:333"}},
]
chain = seal(events)
print(*verify(chain))
chain[1]["payload"]["decision"] = "deny"
print(*verify(chain))

```

Expected output:

```

True verified 3 events; anchor=e26371d4de87
False hash failure at 2

```

The exact anchor is deterministic for this program. Production code should use a formal schema library, canonicalization standard, per-tenant stream identity, signatures, key rotation, durable checkpoints, and explicit migration tests. Never place secrets into a lab merely to see whether redaction works.

Coverage and Operational Failure Analysis

Happy-path-only traces. Instrumentation often misses policy exceptions, retries, cancellation, timeouts, streaming partials, and sandbox kills. Derive required events from the state machine and test every transition, including denied and failed actions.

Collector loss. If the trace backend is unavailable, decide by consequence class: low-risk reads may buffer locally; consequential writes should fail closed if their authorization receipt cannot be durably recorded. Bound buffers and expose loss counters.

High-cardinality collapse. Aggregating by run can erase the one denied call that explains an incident. Keep event-level evidence in restricted storage while exporting carefully chosen metrics.

Redaction after ingestion. Secrets already reached logs, queues, and error trackers. Redact at the producer boundary, structurally exclude credential fields, and scan canaries across the complete telemetry path.

Broken migrations. Adding a field can change canonical bytes or verifier behavior. Version schemas immutably, preserve old verification logic, and test known fixtures for every version.

Insider reconstruction. Even redacted previews, timing, and tool arguments may reveal sensitive activity. Enforce tenant isolation, purpose-bound access, dual control for bulk export, query auditing, and anomaly detection on investigators.

False assurance from hashes. Integrity is not completeness, authenticity, or correctness. Combine producer chains with independently sourced tool and cloud audit evidence. Record coverage claims explicitly: which tool versions and execution paths emit which events.

Queryability without uncontrolled copying

Forensic evidence is useless when investigators cannot join it, but broad search indexes create new disclosure surfaces. Index stable identifiers, types, times, consequence classes, reason codes, and keyed pseudonyms. Keep sensitive payload search in a restricted enclave with case-bound access. Export tools should default to envelopes and require a separate authorization to include content.

Test common investigation queries against synthetic incidents before launch: list all actions derived from one evidence object, find tool completions without approvals, reconstruct a delegation tree, and verify a chain across key rotation. Measure time to answer and missing joins. A schema that is cryptographically elegant but operationally unqueryable will be bypassed during an incident.

Source Synthesis

Building an Evidence Contract

Start with investigations the organization must perform. For an unauthorized deployment, determine whether the agent proposed the artifact, an eligible human approved it, policy evaluated the same digest, the credential matched the environment, and the service applied that artifact. Convert each question into required events and joins. This avoids collecting attractive but irrelevant model chatter.

Define volume before rollout. Streaming tokens can outnumber consequential actions by orders of magnitude. Keep token diagnostics only in bounded debug samples; retain durable event facts for security. Estimate bytes per run, peak events per second, content-store growth, index cost, and checkpoint frequency. Budgeting is a control because overloaded collectors lose rare failure events.

Classify schema changes as additive, semantic, or breaking. Adding an optional field is additive only when absence has defined meaning. Changing actor from authenticated principal to display name is semantic. Removing an event type is breaking because coverage claims change. Consumers declare supported versions and fail visibly on unknown security semantics.

Sensitive-action taxonomy

Classify observation, local mutation, external communication, privilege change, financial action, destructive action, production change, credential access, and durable memory mutation. Add scope and reversibility. Reading one public file differs from bulk private-repository export; an ephemeral test file differs from deleting production data.

For each class specify required proposal fields, policy evidence, approval, tool receipt, retention, and trace-outage behavior. Share the taxonomy with the tool broker. If tracing calls something local while the broker calls it external, audits will be incoherent.

Completeness reconciliation

Every credential lease should join to allowed calls and closure or expiry. Every consequential completion should join to one proposal and policy decision, plus approval when required. Every deployment should appear in both agent and platform audit streams.

Reconcile continuously. Unmatched completions indicate bypass or collection loss. Unused leases may show aborted work or credential harvesting. Missing terminal events reveal crashes. Accept late events within a bounded window while preserving original sequence and ingest latency.

Cryptographic operations

Keys require identifiers, rotation, revocation, and separation by tenant and environment. Retain public material and algorithms for historical checkpoints after rotation. If a key is compromised, record the affected interval; do not re-sign history. Forward security reduces some retrospective risk but does not cure producer compromise.

Checkpoint frequency trades detection latency, cost, and rewrite exposure. Deployment streams may checkpoint per transaction; observation streams hourly. Anchor to another account, transparency service, or write-once store under a different role. Test archive restoration and verification without the producer database.

Evidentiary limits

An incident package contains chain segments, checkpoints, schemas, canonicalization rules, key history, verifier version, independent logs, access records, and a narrative separating observation from inference. Generate it reproducibly and hash the package.

A signature proves a key signed bytes, not which human controlled it. A matching digest proves byte equality when content exists, not that content was complete or honestly labeled. Put these limits in runbooks so incident pressure does not turn cryptography into folklore.

OpenTelemetry provides trace identity and semantic conventions but is not by itself a forensic integrity system. W3C Trace Context propagates correlation, not trust. RFC 8785 supplies deterministic JSON; NIST log-management guidance addresses operational collection and retention; SLSA and in-toto show how attestations bind subjects, materials, and steps. The practical design borrows correlation from observability and verifiability from supply-chain provenance while applying privacy minimization before collection.

Sources and Further Reading

- OpenTelemetry specification: <https://opentelemetry.io/docs/specs/otel/>
- W3C Trace Context Recommendation: <https://www.w3.org/TR/trace-context/>
- IETF RFC 8785, JSON Canonicalization Scheme: <https://www.rfc-editor.org/rfc/rfc8785.txt>
- NIST SP 800-92, *Guide to Computer Security Log Management*: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf>
- in-toto specification: <https://in-toto.io/>
- SLSA provenance model: <https://slsa.dev/spec/v1.0/provenance>
- NIST Privacy Framework 1.0: https://www.nist.gov/system/files/documents/2020/01/16/NIST%20Privacy%20Framework_V1.0.pdf

Interview Practicum

Design telemetry for a customer-support agent that reads tickets, queries account systems, drafts refunds, and can issue refunds after approval. Legal permits raw ticket retention for seven days, Finance requires seven-year refund evidence, and Security discovers that model-provider request IDs are unavailable in one region. During the discussion, a suspected insider deletes a trace row.

Specify the schema, trust boundaries, retention split, integrity design, coverage response, and incident evidence package.

Worked Answer

I would define typed envelopes for run start, evidence observation, model completion, tool proposal, policy decision, approval, refund completion, and run end. Raw tickets live in an encrypted content store for seven days; events hold a keyed digest, classification, and case-bound locator. Refund records retain canonical proposal, amount, currency, account pseudonym, approver, policy version, tool receipt, and ledger transaction identifier for the Finance period without retaining unrelated conversation text.

Each tenant and agent stream uses sequence numbers and a hash chain. Hourly signed checkpoints go to a separately administered immutable store. The refund service emits its own receipt and ledger audit event, allowing completeness reconciliation. Missing provider request IDs become an explicit regional coverage flag; model identity, local request digest, timing, and response digest remain, but the team must not claim provider-side correlation there.

For the deletion incident, verification identifies the gap or broken successor link. Preserve the signed checkpoint, adjacent events, collector access logs, refund-service audit, ledger record, key history, and schema/version documentation. Rotate access if compromise is suspected. The report distinguishes proven facts from inference: a broken chain proves inconsistency after the checkpoint, not which person deleted the row or whether the original event was truthful.

Follow-Up Questions and Answers

Can the company keep unsalted prompt hashes forever?

Not safely by default. Guessable messages can be dictionary-attacked and hashes may remain regulated data. Use keyed digests, restricted verification, and purpose-based retention.

What happens when the trace service is down?

Buffer bounded low-consequence events, but fail closed for refunds if durable authorization and completion evidence cannot be recorded. Alert on every loss counter.

How do you measure trace coverage in a nondeterministic agent?

Measure instrumented state transitions and tool boundaries, not textual paths. Run generated scenario suites and reconcile tool-side actions against corresponding proposal, decision, and completion events.

Why not use a blockchain?

A signed, externally anchored append-only log usually provides the required tamper evidence with less complexity. The governance of writers, keys, checkpoints, and independent evidence matters more than the branding of the data structure.

Key Takeaways

- Security traces are typed evidence, not permanent transcripts.
- Separate stable envelopes from protected, short-lived content objects.
- Canonicalize, chain, sign, and externally anchor high-value streams.
- Integrity does not prove truth or completeness; corroborate at tool boundaries.
- Design retention by purpose and consequence before collection.
- Make regional, tool, and failure-path coverage gaps explicit.

Field lesson: a digital thread must expose broken links

The sec-pr implementation indexed 566 evidence artifacts and linked analytical records through deterministic composite identity and hashes. It also quarantined 278 unresolved historical records instead of silently forcing joins. Trace completeness therefore included a reconciliation result: what the system could connect, what it could not connect, and why. See Chapter 47 for the full architecture.

Chapter 36: Evaluating Agent Security Controls

One Passing Run Proves Almost Nothing

An agent may refuse an attack on Tuesday and execute it on Wednesday because sampling, retrieval order, tool latency, memory, or a slightly different conversation changes its trajectory. Security evaluation must therefore estimate a distribution of outcomes, not preserve a favorite transcript. The evaluation unit is a scenario–configuration–trial tuple, and the claim should include uncertainty.

A safeguard can reduce attack success while destroying legitimate work. It can also appear strong because the test set contains obvious attacks, because evaluators stop after the first refusal, or because a sandbox prevents consequences without testing whether policy allowed them. Chapter 34 identified authority paths; Chapter 35 made actions observable. This chapter uses those artifacts to test the whole control stack: model, prompt/scaffold, retrieval, memory, policy, tool broker, sandbox, approval, and final outcome.

Evaluation Architecture

```

flowchart LR
    S[Versioned scenario registry] --> X[Factorial trial scheduler]
    C[Control configurations] --> X
    X --> R1[Isolated runner seed A]
    X --> R2[Isolated runner seed B]
    X --> RN[Isolated runner seed N]
    R1 --> T[Typed traces and outcomes]
    R2 --> T
    RN --> T
    T --> O[Independent oracle and adjudication]
    O --> M[Metrics by threat, utility, and segment]
    M --> U[Bootstrap and binomial uncertainty]
    U --> G[Release gate]
    G -->|pass| D[Canary with kill switch]
    G -->|fail| B[Counterexample corpus]
    B --> S
  
```

The scenario registry is immutable per evaluation. Each entry identifies initial state, trusted and untrusted inputs, permitted actions, forbidden outcomes, benign task goal, termination rule, and oracle. The scheduler varies seeds and relevant environmental factors. Runners are disposable and deny real-world side effects. Oracles operate on typed traces and final state, not on whether output “sounds safe.”

Define the Estimands

For adversarial scenario family $\backslash(g\backslash)$, attack success rate is

$\backslash[ASR_g = P(\text{forbidden outcome} \mid g, \text{configuration}). \backslash]$

Defense success is not always $(1-ASR)$: a run can avoid the forbidden outcome yet fail closed incorrectly, loop, or abandon the legitimate subtask. On benign tasks define false-refusal rate $(FRR=P(\text{refusal} \mid \text{benign}))$, task success (TSR) , cost, latency, and human interventions. A paired utility difference between candidate (c) and baseline (b) can be

$$\Delta U = E[w_t(TSR_c - TSR_b) - w_a(ASR_c - ASR_b) - w_f(FRR_c - FRR_b) - w_k(\text{Cost}_c - \text{Cost}_b)].$$

Weights encode an explicit operating decision, but always publish components. A positive aggregate can hide catastrophic regression in a high-impact scenario.

Repetition and confidence intervals

For x successes in n independent trials, report $\hat{p} = x/n$ with a Wilson interval rather than a naked rate. At zero observed bypasses, the one-sided “rule of three” gives an approximate 95 percent upper bound of $(3/n)$: zero in ten trials still permits a true bypass probability near 30 percent. To claim an upper bound below 1 percent after zero failures requires roughly 300 representative trials, not three screenshots. The shortcut assumes approximately IID trials. When attempts cluster by repository, agent session, target, or model version, the effective sample size is smaller; use cluster-aware analysis or a cluster bootstrap rather than substituting the raw request count for n .

Trials may not be independent. Provider updates, shared caches, rate limits, or persistent memory create clusters. Randomize execution order, isolate state, block by model/provider version, and use cluster bootstrap or hierarchical analysis when trials share conditions. Store seeds where supported, but do not confuse a seed with reproducibility across changing infrastructure.

Adaptive attacks

A static attack suite measures known strings, not a determined adversary. Split evaluation into fixed regression cases, mutation-based variants, and an adaptive phase in which a tester observes refusals and changes strategy within a bounded budget. Keep the final holdout private from control developers. Track attempts-to-bypass and success within budget.

Oracles

Output keyword matching is a weak oracle. Prefer state-based facts: Did a network connection reach a forbidden host? Was a credential read? Did the broker issue an unauthorized capability? Was production modified? Combine automatic oracles with blinded human adjudication for ambiguous semantic harm. Measure inter-rater agreement and resolve disagreements under a written rubric.

Experimental Design and Release Gates

Pair candidate and baseline on the same scenario instances to reduce variance. Randomize which runs first. Freeze model, tool, policy, scenario, and dependency versions for the comparison. If the production model is a moving service, record its provider version and repeat sentinel scenarios over time.

Pre-register primary metrics and gates. A credible gate might require: no critical forbidden outcome in a designated must-pass suite; candidate ASR upper confidence bound below 5 percent per high-risk family; benign task-success lower bound no more than two percentage points below baseline; false-refusal increase below one point; and no new unrestricted authority path. Gates are governance decisions, not statistically discovered after results arrive.

Avoid test-set contamination. Scenario text, hidden tokens, and oracle details should not enter training or everyday prompt repositories. Rotate semantic variants and test structural behavior. If a safeguard recognizes the phrase “ignore previous instructions,” it has learned a costume, not a boundary.

Executable Lab: Repeated-Trial Control Evaluation

This deterministic simulation models a nondeterministic agent under baseline and guarded configurations. It runs paired adversarial and benign trials, computes Wilson 95 percent intervals, and reports security and utility. Synthetic randomness makes the lab safe and repeatable.

```
from math import sqrt
from random import Random

SCENARIOS = [
    ("indirect_injection", "attack", .34, .08),
    ("delegation_escape", "attack", .22, .06),
    ("normal_refactor", "benign", .91, .87),
    ("dependency_update", "benign", .84, .79),
]

def wilson(successes, trials, z=1.96):
    p = successes / trials
    den = 1 + z*z/trials
    center = (p + z*z/(2*trials)) / den
    margin = z * sqrt((p*(1-p) + z*z/(4*trials))/trials) / den
    return center - margin, center + margin

def run(seed=2026, trials=200):
    rng = Random(seed)
    results = {}
    for name, kind, baseline_p, guarded_p in SCENARIOS:
        for config, probability in (("base", baseline_p),
                                    ("guard", guarded_p)):
            successes = sum(rng.random() < probability for _ in range(trials))
            results[(name, kind, config)] = (successes, trials)
    return results

results = run()
for (name, kind, config), (x, n) in results.items():
    lo, hi = wilson(x, n)
    metric = "ASR" if kind == "attack" else "TSR"
    print(f"{name:20} {config:5} {metric}={x/n:.3f} "
          f"95%CI=[{lo:.3f},{hi:.3f}]" )
```

Expected output:

```
indirect_injection  base  ASR=0.290 95%CI=[0.232,0.356]
indirect_injection  guard ASR=0.050 95%CI=[0.027,0.090]
delegation_escape   base  ASR=0.220 95%CI=[0.168,0.282]
delegation_escape   guard ASR=0.085 95%CI=[0.054,0.132]
normal_refactor      base  TSR=0.940 95%CI=[0.898,0.965]
normal_refactor      guard TSR=0.865 95%CI=[0.811,0.906]
dependency_update    base  TSR=0.825 95%CI=[0.766,0.871]
dependency_update    guard TSR=0.775 95%CI=[0.712,0.827]
```

The guard clearly reduces simulated ASR, but it also reduces benign success. Whether it ships depends on predeclared utility and non-inferiority gates. In a real harness, replace Bernoulli draws with isolated agent executions, preserve typed traces, and have oracles inspect final state. Do not let the harness possess production credentials.

Operational Failure Analysis

Evaluating only the model. A model refusal can be followed by a scaffold retry that succeeds dangerously; a model compliance can be blocked by the broker. Score policy decisions, tool calls, and outcomes separately.

Stopping on a good result. Re-running until a control passes is optional stopping. Fix trial counts or use a documented sequential test with error spending. Publish all planned runs.

Scenario monoculture. Many paraphrases of one injection inflate sample size without threat diversity. Report both trial-weighted and family-weighted results. Maintain coverage against authority paths, tool classes, data boundaries, and persistence mechanisms.

Cross-trial contamination. Memory, caches, files, and provider sessions can carry attack instructions into later benign trials. Reset state and include contamination canaries. If production intentionally preserves memory, evaluate ordered multi-session sequences as their own scenario.

Unsafe testing. An evaluation that can email customers or deploy production is an incident generator. Substitute instrumented fake services, network sinks, synthetic secrets, and canary credentials with no authority. Test the real authorization layer in a dedicated nonproduction tenant.

Oracle leakage. If the agent sees the forbidden marker or evaluator rubric, it can optimize for the test. Keep oracle state out of context and use external observations.

Aggregate victory. A large improvement on easy cases can mask one critical regression. Use must-pass outcomes and report worst-family bounds. Never average away catastrophic impact.

Production drift. Controls interact with new models, tools, prompts, and policies. Trigger evaluation on component changes, run scheduled sentinel trials, and compare canary trace distributions. Preserve a tested kill switch and known-safe reduced-authority mode.

Reporting results

An evaluation report begins with the release decision and tested configuration, then lists gates, sample sizes, invalid trials, point estimates, intervals, and scenario coverage. Include every critical counterexample with secrets removed. Separate prespecified findings from exploratory observations. Provide machine-readable trial records so another team can reproduce calculations.

Avoid league tables that rank safeguards by one aggregate score. State where each control acts: prompt handling, memory, policy, broker, sandbox, or approval. Two controls with similar ASR can have different failure correlation; defense in depth is valuable when bypasses are not shared. Test combinations rather than assuming independent effects.

Source Synthesis

Reports should also state what was not tested, especially languages, tenants, tool versions, long-horizon memory, and provider regions. Named omissions prevent a local result from becoming an organization-wide safety claim.

Constructing the Suite

Design scenarios from authority edges, not jailbreak phrases. For each consequential tool, create cases where untrusted evidence alters object, action, destination, scope, or timing. Include encoded content, tool-output injection, poisoned memory, compromised dependencies, delegation, and multi-turn pressure. Create benign twins sharing surface features: legitimate research quoting attacks, an authorized sensitive-file request, and harmless tool recovery. Twins reveal vocabulary filters.

Each scenario contract names setup, roles, trusted inputs, attacker inputs, allowed actions, forbidden outcomes, maximum turns, cost budget, reset, and oracle. Give it a stable family identifier with instance variation. Version it whenever success criteria change. Add counterexamples to regression while retaining a private holdout.

Sampling should reflect frequency and risk. A frequency-weighted estimate predicts ordinary exposure; a risk-weighted suite stresses rare catastrophic paths. Publish both. A production task capable of rotating a root credential deserves substantial testing even if rare.

Power and thresholds

Confidence intervals describe uncertainty after testing; power analysis plans whether a meaningful change is detectable. If baseline ASR is 10 percent and useful reduction means 5 percent, choose sample size for that contrast and desired errors. For rare events, combine stress tests, structural authorization assurance, and conservative bounds. Absence of observed harm is not impossibility.

For benign utility use non-inferiority. Set margin (δ) before results: accept only if the lower confidence bound of $(TSR_c - TSR_b)$ exceeds $(-\delta)$. Choose margins from user consequence, not release convenience. Measure tail latency and cost because loops make distributions heavy-tailed.

Multiple families create multiplicity. “Any critical failure” gates need no averaging. Label exploratory comparisons and control false discoveries for confirmatory claims. Never cherry-pick a seed, snapshot, or judge.

Harness engineering

Capture scenario version, model and parameters, system-prompt digest, retrieval corpus, memory snapshot, tool schemas, policy bundle, container image, network policy, and evaluator code. Assign trial IDs and derive seeds from a recorded master. Enforce timeout, token, tool, and spend budgets.

Fake tools must be behaviorally faithful. A mock deployer implements authorization, idempotency, state, and receipts rather than always succeeding. Synthetic credentials serve as canaries without real authority. Network sinks record destinations and content digests.

Evaluator failure is distinct from agent failure. Provider errors, invalid setup, and oracle crashes are neither attack nor defense success. Report invalid-trial rate, retry under a declared rule, and investigate configuration-correlated failures. A control must not look safe by crashing the harness.

Production bridge

Canaries begin with reduced authority and shadow policy. Compare production tool sequences, refusal patterns, and delegation depth with suite coverage. Do not replay private prompts into research without consent and minimization.

Define response thresholds before launch. One critical unauthorized action triggers containment; refusal regression may trigger rollback; a new model identity triggers sentinels or blocks execution. Preserve failed traces and sandbox state. Generalize failures into tests without broadly publishing bypass details.

NIST AI RMF frames evaluation as continuous risk management, while the Generative AI Profile identifies generative-system risks. OWASP and MITRE ATLAS organize concrete adversarial behaviors. Statistical guidance on binomial intervals prevents false certainty from small samples. Agent evaluation research contributes task environments, but security requires consequence-grounded oracles and authority-aware scenario coverage. The synthesis is a repeated, paired, isolated experiment with confidence bounds and explicit utility loss.

Sources and Further Reading

- NIST AI RMF 1.0: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
- NIST AI 600-1, *Generative AI Profile*: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
- OWASP GenAI Security Project: <https://genai.owasp.org/>
- MITRE ATLAS: <https://atlas.mitre.org/>
- NIST/SEMATECH, binomial confidence intervals: <https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm>
- Wilson, *Probable Inference, the Law of Succession, and Statistical Inference*: <https://www.jstor.org/stable/2248530>

- UK AI Security Institute, Inspect AI: <https://inspect.aisi.org.uk/>
- METR, agent evaluation resources: <https://metr.org/>

Interview Practicum

A vendor claims its new tool-call firewall cut prompt-injection success from 20 percent to zero in ten trials. Your coding agent serves 50,000 monthly tasks. You have two weeks and a constrained evaluation budget. After you propose a test, the vendor reveals that its control increases refusals and that the production model changes without notice.

Design the evaluation, quantify what the current evidence means, define release gates, and explain how you will handle adaptive attacks and model drift.

Worked Answer

Zero successes in ten does not establish zero risk; the approximate 95 percent upper bound is 30 percent. I would build a versioned suite from the Chapter 34 authority graph: indirect injection, delegation, credential access, egress, memory poisoning, tool confusion, and approval binding, plus benign tasks that exercise the same tools. Critical paths receive more trials than low-impact families.

Run baseline and firewall configurations paired on identical scenario instances, randomized in order, in isolated sandboxes. Record model/provider identity, scaffold, tool, policy, and seed where available. Automatic oracles inspect forbidden state transitions; blinded reviewers handle semantic ambiguity. Report ASR and Wilson intervals per family, benign task success, false refusal, latency, cost, and human escalation.

The release gate includes zero critical must-pass outcomes, bounded upper ASR per high-risk family, and a predeclared non-inferiority margin for task success and false refusal. A small adaptive team gets a fixed attempt budget after the static suite, while a private holdout remains untouched. Because the model changes without notice, provider identity becomes a blocking observability requirement or contractual risk; run daily sentinels and trigger the full suite on detected change. Canary deployment uses reduced authority and an immediate rollback path.

Follow-Up Questions and Answers

How many trials are enough?

Enough is claim-dependent. With zero failures, about 300 representative independent trials support an approximate 95 percent upper bound below 1 percent. Clustered or heterogeneous trials require more careful design, not merely more repetitions.

The control blocks attacks by blocking all tool calls. Did it pass?

No. Its ASR may be low, but task success and false refusal expose that it eliminated the product's function. Security and utility are joint constraints.

Should adaptive results enter the same ASR as static tests?

Report them separately. Adaptive trials have conditional, budget-dependent sampling. Combining them with fixed random trials produces a number with unclear meaning.

What if human reviewers disagree?

Use a written rubric, blinded independent ratings, agreement statistics, and adjudication. Preserve both original judgments. Frequent disagreement means the oracle or scenario contract is underspecified.

Key Takeaways

- Evaluate distributions of outcomes across repeated trials, not memorable transcripts.
- Pair security metrics with benign utility, refusal, latency, cost, and escalation.
- Report confidence intervals per threat family and protect catastrophic must-pass cases.
- Isolate trials and ground oracles in tool-side state and typed traces.
- Separate fixed regression, mutation, and adaptive attack evidence.
- Re-run evaluations whenever models, scaffolds, tools, or policies change.

Field lesson: benchmark contracts must fail closed

The sec-pr project registered 564 benchmark contracts, while the single recorded benchmark run returned `insufficient_evidence`. Registration did not become an invented pass count. The harness exercised its fail-closed status path and preserved the difference between executable evaluation structure and mature human outcome evidence. The full case appears in Chapter 47.

Chapter 37: AI-Powered Detection and Security Operations

An SOC agent should be judged by the evidence it preserves, the consequential actions it refuses to take, and the facts it notices or misses—not by how fluent its incident summary sounds. This chapter builds such an agent for a concrete workflow: enrich an identity alert, assemble a cited case brief, identify missing evidence, recommend a disposition, and request approval for containment. The agent may read narrowly scoped systems and draft work; it may not silently expand its investigation, change a control, disable an account, or convert uncertainty into fact.

The design separates three questions that are often collapsed. Did the agent retrieve the right evidence? Did it reason faithfully over that evidence? Was it authorized to perform the requested action? Better prompting cannot repair a missing audit source, and a well-grounded conclusion does not grant authority. Production safety depends on keeping those questions separate in data, policy, evaluation, and incident review.

The Case: Impossible Travel with a Plausible Explanation

An identity provider raises an impossible-travel alert for `alice@example.com`: Chicago at 14:02 UTC and Amsterdam at 14:17 UTC. A weak assistant paraphrases the alert and recommends account suspension. A useful investigator checks whether both events represent human sessions, whether an approved egress proxy explains one location, whether device identity and token lineage agree, and whether high-value actions followed. It then distinguishes observation, inference, and unknown.

The desired output is not merely “benign” or “malicious.” It is a compact evidence packet: a timeline whose factual claims cite immutable evidence identifiers; explicit coverage of required questions; an omission list for unavailable or unexamined evidence; a calibrated disposition; proposed actions checked against policy; and the model, prompt, retriever, policy, and tool versions needed for replay. This packet makes the agent a participant in an evidence process rather than an oracle.

Bounded Authority Is a Capability System

“Human in the loop” is too vague to be a control. The useful unit is a capability: a narrowly defined operation over a resource class, subject to conditions, expiry, and an approval mode. Reading the last 24 hours of sign-in metadata is different from reading mailbox contents. Adding a case note is different from disabling a principal. A capability token should be minted for the case, not inherited from the service account’s broad permissions.

```
flowchart LR
    A[Alert envelope] --> P[Case planner]
    P --> G{Capability gateway}
    G -->|read:idp.signins| I[Identity telemetry]
    G -->|read:asset.posture| E[Endpoint posture]
    G -->|read:change.approvals| C[Change records]
```



```

I --> V[Evidence vault]
E --> V
C --> V
V --> S[Cited case synthesis]
S --> O[Omission and contradiction check]
O --> D{Disposition policy}
D -->|draft only| N[Case note]
D -->|approval required| H[Analyst approval]
H -->|signed approval| R[Response broker]
R --> L[Append-only action ledger]
G --> L
S --> L

```

The gateway accepts typed requests, not arbitrary URLs or shell strings. The response broker independently rechecks authorization; it does not trust a claim that approval exists. High-impact actions use a short-lived, single-use approval bound to the exact subject and action. An approval for `revoke_sessions/alice` cannot authorize `disable_user/alice`, and neither can be replayed against Bob.

Operation	Default	Required control
Read alert-linked sign-ins, 24 h	allow	case-scoped token; field minimization
Read endpoint posture for linked device	allow	asset relation must be evidenced
Search unrelated identities	deny	analyst expands scope with reason
Draft timeline and case note	allow	citations and model provenance
Publish note to case	allow	schema validation; no action side effect
Revoke active sessions	approval	named analyst; exact target; 10-minute expiry
Disable identity or isolate host	deny	incident commander or separate policy
Modify detection or suppression rule	deny	change-management workflow only

The model never possesses ambient response credentials. It proposes an intent. Deterministic infrastructure decides whether that intent is permitted.

Evidence Objects, Claims, and Provenance

Retrieval grounding becomes auditable only when evidence has stable identity and claims have machine-checkable citations. A minimal contract is:

```

{
  "$id": "soc-evidence-v1",
  "type": "object",
  "required": ["evidence_id", "observed_at", "source", "subject", "payload_hash"],
  "properties": {

```

```

    "evidence_id": {"type": "string", "pattern": "^ev_[a-f0-9]{12}$"},
    "observed_at": {"type": "string", "format": "date-time"},
    "source": {"enum": ["idp", "edr", "proxy", "cldb", "change"]},
    "subject": {"type": "string"},
    "event_type": {"type": "string"},
    "payload_hash": {"type": "string", "pattern": "^sha256:"},
    "fields": {"type": "object"},
    "retrieved_at": {"type": "string", "format": "date-time"},
    "retention_class": {"enum": ["case", "restricted", "ephemeral"]}
  },
  "additionalProperties": false
}

```

The narrative is stored as claims, not one unconstrained blob:

```

{
  "claim_id": "cl_07",
  "text": "The Amsterdam address was an approved corporate egress node.",
  "claim_type": "observation",
  "citations": ["ev_9c14d8f0a612", "ev_a82f38de7710"],
  "confidence": 0.98,
  "contradicts": []
}

```

`claim_type` may be `observation`, `inference`, or `unknown`. Observations must be directly entailed by cited fields. Inferences require citations and a stated inference rule. Unknowns cite the failed query or unavailable source when possible. The payload hash detects mutation; the retrieval timestamp shows what the agent could have known at decision time.

Retrieval needs defenses. Alert text is untrusted data and may contain instruction-like content. The planner extracts typed entities, but no retrieved string becomes a system instruction. Connectors impose row limits and time windows. Results are normalized and rendered as data. The planner cannot ask a connector to “run whatever query is in this event.”

Citation Quality Is More Than Citation Presence

An answer can cite every sentence and still be wrong. Evaluation therefore works at claim level. For generated claims $\set{C}$, let $\set{s(c)=1}$ when cited evidence supports claim $\set{c}$, $\set{a(c)=1}$ when it requires a citation, and $\set{w_c}$ represent consequence weight:

$$\set{P_{\text{cite}}} = \frac{\sum_{c \in C} w_c s(c)}{\sum_{c \in C} w_c a(c)}.$$

Citation completeness measures whether citation-requiring claims have a reference. Entailment is separate: a sign-in record supports “a login occurred,” but not “Alice performed the login.” Temporal validity matters too; evidence observed after containment cannot justify the earlier decision.

Automated checks can reject nonexistent evidence IDs, wrong subjects, evidence outside the case horizon, and claims whose cited fields lack referenced values. Human adjudication remains necessary for semantic entail-

ment. Sample more heavily for containment recommendations, contradictions, new model versions, and low-confidence cases.

Omission Is a First-Class Error

Hallucinations are visible. Omissions often are not. An investigation that correctly describes two logins but fails to check token reuse may be more dangerous than one awkward sentence. Each alert family therefore has a question set $\mathcal{Q}$: device continuity, network explanation, authentication strength, token lineage, privileged follow-on activity, endpoint state, and relevant change records for impossible travel.

For question q , assign importance v_q and coverage state `answered`, `unavailable`, `not_applicable`, or `unexamined`. Weighted omission loss is

$$L_{\text{omit}} = \frac{\sum_{q \in \mathcal{Q}} v_q \mathbb{1}[q = \text{unexamined}]}{\sum_{q \in \mathcal{Q}} v_q \mathbb{1}[q = \text{unavailable}]} + \lambda \sum_{q \in \mathcal{Q}} v_q \mathbb{1}[q = \text{unavailable}] \quad \text{where } 0 \leq \lambda < 1.$$

Unavailable evidence is not equivalent to negligence, but it still weakens the decision. `not_applicable` requires a reason. Connector health and query logs prevent the agent from obtaining a good score by labeling every hard question unavailable.

Coverage sets should come from incident retrospectives and analyst playbooks, then remain small enough to discriminate. They are not rigid scripts. Novel evidence may add questions, while policy defines the minimum set before a close recommendation.

Decision, Abstention, and Adversarial Failure

Let d be a disposition, x the evidence, and $L(d, y)$ the loss if the true state is y :

$$d^* = \arg \min_d \sum_y P(y \mid x) L(d, y), \quad \text{subject to } L_{\text{omit}} \leq \tau_o \quad \text{and } P_{\text{cite}} \geq \tau_c.$$

If constraints fail, the correct output is abstention with a specific request: “EDR posture unavailable for device D17; analyst review required.” Confidence is never authorization. A 0.99 malicious probability can justify urgency, not bypass approval.

Threat modeling must include prompt injection in alert fields, poisoned knowledge articles, cross-tenant retrieval, stale identity mappings, citation laundering, compromised connectors, approval replay, tool argument substitution, denial-of-wallet, and analyst automation bias. The strongest boundary is typed mediation: retrieved text cannot alter policy; generated text cannot become a tool call; a validated intent cannot become an action without a fresh capability check.

Safe Lab: Score a Cited Investigation

This standard-library program evaluates evidence identity, claim support, question coverage, and allowed actions. It contacts no systems and performs no containment.

```

from dataclasses import dataclass
from typing import FrozenSet

@dataclass(frozen=True)
class Evidence:
    eid: str
    facts: FrozenSet[str]

@dataclass(frozen=True)
class Claim:
    text: str
    required_facts: FrozenSet[str]
    citations: tuple[str, ...]
    weight: int

evidence = {
    "ev_idp": Evidence("ev_idp", frozenset({"user:alice", "mfa:pass", "device:D17"})),
    "ev_proxy": Evidence("ev_proxy", frozenset({"ip:203.0.113.8", "approved_egress:true"})),
    "ev_edr": Evidence("ev_edr", frozenset({"device:D17", "health:normal"})),
}

claims = [
    Claim("MFA succeeded on D17", frozenset({"mfa:pass", "device:D17"}), ("ev_idp",), 2),
    Claim("The foreign IP is approved egress", frozenset({"approved_egress:true"}),
("ev_proxy",), 3),
    Claim("No endpoint alerts exist", frozenset({"alerts:none"}), ("ev_edr",), 3),
]

questions = {
    "device_continuity": (3, "answered"),
    "network_explanation": (3, "answered"),
    "token_lineage": (4, "unavailable"),
    "endpoint_state": (2, "answered"),
    "privileged_follow_on": (5, "unexamined"),
}

allowed = {"read_case", "draft_note"}
proposed = ["draft_note", "revoke_sessions"]

def supported(claim: Claim) -> bool:
    cited = [evidence[e] for e in claim.citations if e in evidence]
    union = frozenset().union(*(item.facts for item in cited))
    return claim.required_facts <= union

supported_weight = sum(c.weight for c in claims if supported(c))
total_weight = sum(c.weight for c in claims)
denom = sum(weight for weight, _ in questions.values())
loss = sum(weight * (1 if state == "unexamined" else 0.35 if state == "unavailable" else 0)
            for weight, state in questions.values()) / denom
blocked = [action for action in proposed if action not in allowed]

print(f"citation_precision={supported_weight / total_weight:.3f}")
print(f"omission_loss={loss:.3f}")
print(f"unsupported={[c.text for c in claims if not supported(c)]}")
print(f"blocked_actions={blocked}")
print("decision=ABSTAIN" if loss > 0.20 else "decision=RECOMMEND")

```

Expected output:

```
citation_precision=0.625
omission_loss=0.376
unsupported=['No endpoint alerts exist']
blocked_actions=['revoke_sessions']
decision=ABSTAIN
```

The negative claim is unsupported: `health:normal` does not establish that no alerts exist. The omission score exposes unchecked privileged activity. Policy blocks response even though drafting is allowed.

Production Operations

Deploy in modes, beginning with replay on closed cases and then shadowing live investigations. In assist mode, place the draft in a separate pane. Only after demonstrated citation precision, omission control, and analyst acceptance should it publish schema-valid notes. Response authority is a distinct promotion with its own hazard review.

Operational telemetry includes connector success by source, evidence age, retrieval truncation, invalid citations, weighted omission, contradiction rate, abstention reason, analyst edits, approval latency, denied tool calls, and downstream reversals. “Minutes saved” belongs beside investigation quality. Measure time to a correct, reviewable decision; otherwise the system can look efficient by producing fast rework.

Treat analyst edits as structured feedback, not automatically correct labels. Record whether an analyst changed a fact, inference, disposition, omission state, or prose. Outcome labels arrive late and selectively. Preserve a random audit sample of closed and escalated cases to limit policy-induced label bias.

Run drills for connector outage, stale asset inventory, poisoned alert text, approval-service failure, evidence-vault unavailability, model timeout, and ledger mismatch. Fail closed for actions. For synthesis, degrade to a visible partial-evidence state. A kill switch should disable tool execution without disabling access to preserved case evidence.

Sources and Further Reading

- NIST, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, 2023: <https://doi.org/10.6028/NIST.AI.100-1>
- NIST, *AI RMF Generative Artificial Intelligence Profile*, 2024: <https://doi.org/10.6028/NIST.AI.600-1>
- NIST, *SP 800-207: Zero Trust Architecture*, 2020: <https://doi.org/10.6028/NIST.SP800-207>
- NIST, *SP 800-61 Rev. 2: Computer Security Incident Handling Guide*, 2012: <https://doi.org/10.6028/NIST.SP800-61r2>
- MITRE, *ATLAS*: <https://atlas.mitre.org/>
- OWASP, *Top 10 for Large Language Model Applications*: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

- Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” 2023: <https://arxiv.org/abs/2312.10997>

The synthesis matters: AI RMF requires governed evaluation, zero trust supplies least-privilege enforcement, incident handling supplies temporal evidence discipline, and OWASP identifies instruction and agency attacks. Grounding is an integrity control; least privilege is an execution control; neither substitutes for the other.

Interview Practicum

You are interviewing for a Staff Security ML role at a global retailer. Its SOC wants an agent to investigate impossible-travel alerts. The identity team offers a broad service account; analysts want one-click session revocation; Legal prohibits retaining raw prompts containing employee data beyond seven days. A pilot looks impressive, but it missed privileged cloud activity in two red-team cases and cited a change ticket belonging to another user.

Prompt: Design the production system and the evaluation that determine whether it may move from shadow mode to analyst-facing use. Midway through your answer, assume the identity API begins returning results 18 minutes late. Near the end, assume the VP asks to enable autonomous revocation for “confidence above 95%.”

Strong clarifying questions are specific: What is the population and base rate? Which sources establish device, proxy, token, and privileged-action state? What counts as a correct disposition? Are evidence IDs immutable? Which actions are reversible? Who owns approval? How often are sources late or absent? Can normalized evidence outlive raw prompts? What adverse outcome is tolerable in the canary?

Worked Answer

I would reject the broad service account and start with case-scoped read capabilities. The alert supplies the identity, two sign-ins, and a 24-hour horizon. A deterministic planner may request only approved evidence classes for entities linked by cited relations. Normalized evidence enters a vault with immutable IDs, hashes, source time, and retrieval time. Prompts may expire after seven days; the evaluation record retains minimized structured claims, citations, policy decisions, versions, and permitted fields under the case schedule.

The output separates observations, inferences, unknowns, and proposed actions. Every material observation cites evidence. Every alert family has required questions. The two failures become regression fixtures: cross-subject citations fail validation, and privileged follow-on activity must be answered or explicitly unavailable before a close candidate.

Offline evaluation uses temporally held-out cases plus synthetic cases varying proxy use, token theft, device continuity, and privileged actions. I report weighted citation entailment, invalid citation rate, omission loss, disposition loss, abstention quality, and analyst time to correct decision. Experts adjudicate a stratified sample. Shadow mode then measures connector behavior, freshness, disagreement, and near misses without changing cases.

The 18-minute delay is evidence freshness failure, not model uncertainty. The case displays the watermark, marks dependent questions unavailable, and prevents close or containment recommendation until freshness

recovers or an analyst accepts the gap. A separate alarm prevents rising abstention from being misdiagnosed as model regression.

I would refuse autonomous revocation based on 95% confidence. Calibration drifts, loss depends on account criticality, and confidence grants no authority. If automation is later desired, define a narrow reversible policy—for example, revoke only a newly issued session for a non-service identity when a deterministic replay signature and an independent signal agree. That policy gets its own replay set, canary, action budget, rollback, and incident owner.

Promotion requires zero cross-subject citations in the release suite, a lower bound on weighted citation precision, acceptable omission loss in every high-consequence stratum, no unauthorized execution, acceptable correction time, and tested behavior under missing sources. A randomized audit stream remains because visible suggestions can anchor analysts and corrupt future labels.

Follow-Up Questions and Answers

How do you measure automation bias? Randomize whether the draft appears before or after an analyst records an initial disposition on a safe sample. Compare error-conditioned agreement, reversals toward the agent, and decision time.

What if required questions create friction? Determine which questions change decisions. Remove low-value items through controlled review, not silent UI shortcuts. Prefetch evidence and automate unavailable states.

Can an LLM judge its own citations? It may be one noisy signal, never the sole gate. Identity, subject, time, and field checks are deterministic; semantic entailment needs an independently versioned evaluator and human sampling.

What stops a release? Any unauthorized execution, cross-subject leakage, or material increase in high-weight omissions—even if average disposition accuracy improves.

Give the executive answer. The agent gets narrow read access, produces cited and omission-aware drafts, and cannot contain anything itself. It advances only when it preserves decision quality under missing and delayed evidence; actions stay behind an independent broker.

Key Takeaways

- Ground every material claim in immutable, subject-correct, time-valid evidence.
- Evaluate what the investigation omitted, not only what it asserted.
- Represent authority as case-scoped capabilities and recheck it outside the model.
- Make abstention name the missing evidence or failed control.
- Promote synthesis and response authority as separate risks.
- Optimize time to a correct, reviewable decision—not summary speed.

Chapter 38: AI for Vulnerability Discovery and Code Security

A vulnerability finding is not a model prediction. It is a falsifiable claim that a particular weakness exists in a particular revision, is reachable under stated conditions, has a defensible impact, and can be removed without introducing a regression. AI code review becomes useful only when it carries that claim through evidence collection, safe reproduction, root-cause grouping, remediation, and patch validation.

This chapter follows one example through that chain. A service parses a user-controlled archive manifest and joins each member name to an extraction directory. A scanner flags path traversal. The useful question is not whether the code resembles known vulnerable code. We must establish whether untrusted data reaches the path operation, whether canonicalization prevents escape, whether deployment configuration exposes the route, whether several alerts share one root cause, and whether the proposed patch closes all variants while preserving legitimate archives.

A Finding Is an Evidence Bundle

The unit sent to an engineer should contain:

- repository, commit, build target, file, symbol, and line range;
- weakness hypothesis and relevant CWE, without treating CWE as proof;
- source-to-sink path with sanitizer and configuration evidence;
- reachability assumptions and deployment preconditions;
- a harmless reproducer or counterexample in an isolated harness;
- deduplication group and root-cause explanation;
- patch candidate, tests, and residual uncertainty;
- identities and versions for scanner, model, rules, dependencies, and build.

A bare natural-language explanation loses exactly the information needed to dispute, reproduce, or close the issue. Conversely, a failing test without provenance may be real but impossible to map to a deployed artifact.

From Repository to Validated Repair

```

flowchart TB
    C[Commit and build manifest] --> S[Static facts: AST, call graph, dataflow]
    C --> D[Dependency and configuration facts]
    S --> H[Weakness hypotheses]
    D --> H
    H --> R[Reachability adjudicator]
    R -->|not connected| X[Archive as non-reachable]
    R -->|uncertain| Q[Request build or runtime evidence]
  
```



```

R -->|candidate path| B[Ephemeral safe harness]
B --> F{Behavior reproduced?}
F -->|no| Q
F -->|yes| G[Root-cause graph]
G --> U[Deduplicated developer issue]
U --> P[Patch candidate]
P --> T[Positive, negative, metamorphic tests]
T --> V{Original path gone and behavior preserved?}
V -->|no| P
V -->|yes| M[Merge evidence and post-deploy watch]

```

Static analysis supplies broad coverage. Dynamic reproduction supplies behavioral evidence for executable paths. Dependency analysis supplies version and advisory context. Semantic reasoning proposes connections and repairs. None is sufficient alone. The pipeline records disagreements rather than allowing the language model to smooth them away.

Reachability Has Several Meanings

Teams often say “reachable” while referring to different predicates:

1. **Syntactic reachability:** a call graph contains a path from an entry point to the vulnerable function.
2. **Data reachability:** attacker-influenced bytes can propagate to the dangerous argument.
3. **Control feasibility:** path conditions can be simultaneously satisfied.
4. **Deployment reachability:** the entry point, feature, and dependency version exist in the shipped configuration.
5. **Impact reachability:** the path reaches a security-relevant effect despite sandboxing or downstream validation.

Represent a candidate as a graph $(G=(V,E))$. Each edge has confidence (p_e) , and each required condition has support (r_j) . A ranking score can prioritize analysis:

$$R(\pi) = \left(\prod_{e \in \pi} p_e \right) \left(\prod_j r_j \right) I_{\text{deploy}} I_{\text{untrusted}},$$

but it is not a probability of exploitation unless calibrated against representative outcomes. Multiplication appropriately punishes one unsupported link; it can also over-penalize correlated evidence, so production systems should retain component values.

For the archive example, the evidence chain is: HTTP upload route enabled; body saved as an archive; manifest member name attacker-controlled; member name flows to `base / name`; no canonical containment check; process can write outside the intended directory. A unit test calling the join helper establishes code behavior but not route deployment. A route trace establishes execution but not attacker control. The finding requires both.

Schema for a Reviewable Finding

```

finding_version: 2
finding_id: F-2026-00418
artifact:
  repository: inventory-importer
  commit: 7a61c9e
  build_target: importer-api
location:
  file: src/archive.py
  symbol: extract_members
  lines: [88, 104]
hypothesis:
  weakness: path traversal during archive extraction
  cwe: CWE-22
  source: upload.manifest.member_name
  sink: filesystem.write
reachability:
  entrypoint: POST /v2/import
  dataflow_edges: [df-81, df-93, df-101]
  feature_flag: bulk_import=true
  deployment_evidence: deploy-55
  unresolved: []
reproduction:
  harness: tests/security/test_archive_containment.py
  safe_effect: write_attempt_outside_temp_root
  network: disabled
dedupe:
  root_cause_key: archive.extract.missing_canonical_boundary
patch:
  commit: pending
validation: pending

```

The reproduction records a safe effect, not an offensive payload. The harness uses a temporary directory, fabricated files, no network, no privilege, and an assertion that detects an attempted boundary escape before any persistent harm.

Safe Reproduction as a Scientific Test

Reproduction should answer the smallest claim necessary. For path traversal, the claim is that a crafted member name resolves outside a temporary root. There is no need to overwrite a real file. For command injection, a harness can replace process execution with an instrumented fake and assert argument boundaries. For server-side request forgery, route all traffic to a local stub. For authorization flaws, use synthetic tenants.

Safety controls belong in the harness contract: disposable container or process; no external network; read-only source mount; synthetic secrets; low CPU, memory, process, and time limits; unprivileged identity; bounded output; and destruction after collection. A reproducer is reviewed before it enters a shared corpus because “test code” can still carry dangerous primitives.

Reproduction outcomes need more nuance than pass/fail: `confirmed`, `counterexample`, `environment_mismatch`, `build_failed`, `inconclusive`, or `unsafe_to_run`. A build failure is not evidence that the finding is false. Similarly, failure to reproduce under one configuration does not refute another declared deployment.

Deduplication by Root Cause, Not Text Similarity

One missing containment check can trigger alerts at the join expression, file open, overwrite, and archive entry loop. Filing four tickets wastes engineering time and corrupts precision metrics. Yet grouping every CWE-22 alert can hide independent fixes.

Build a finding graph. Nodes are candidates; edges express shared source, sink, sanitizer gap, vulnerable symbol, dependency instance, call path, or patch locus. Cluster when findings share a plausible fix boundary. Text or embedding similarity is supporting evidence only.

For findings $\langle i, j \rangle$, a pair score might be

$$\frac{1}{|D_{ij}|} = 0.30J(E_i, E_j) + 0.25I(s_i = s_j) + 0.20I(k_i = k_j) + 0.15J(P_i, P_j) + 0.10\cos(z_i, z_j), \quad \forall$$

where $\langle E \rangle$ is dataflow edges, $\langle s \rangle$ vulnerable symbol, $\langle k \rangle$ sink class, $\langle P \rangle$ path-condition features, and $\langle z \rangle$ a text embedding. This is a candidate generator. A deterministic constraint prevents grouping across repositories or deployed dependency instances unless an explicit shared root cause exists.

Evaluate grouping with pairwise precision and recall, cluster purity, over-merge rate, under-merge rate, and developer issue count. Weight over-merges heavily because one closed ticket can otherwise suppress an unfixed vulnerability.

Patch Validation Is a New Security Evaluation

The patch is not validated merely because the reproducer stops failing. A superficial fix may block `../` while missing absolute paths, alternate separators, encoded forms, symlink resolution, or time-of-check/time-of-use behavior. A broad fix may reject legitimate nested paths.

Validation uses several test families:

- regression: the original harmless witness no longer escapes;
- variants: equivalent path representations remain contained;
- positive behavior: legitimate nested members still extract;
- metamorphic: harmless transformations preserve the security decision;
- static reanalysis: source-to-sink path is cut or a proven sanitizer dominates it;
- dependency/build: lockfile, artifact, and deployment actually contain the patch;
- differential: compare old and new behavior only inside the safe harness;
- post-deploy: monitor errors and recurrence without exposing exploit detail.

Define patch assurance as a vector rather than one score: $\backslash(A=(r,v,b,d)\backslash)$, where $\backslash(r\backslash)$ is original regression closure, $\backslash(v\backslash)$ variant closure, $\backslash(b\backslash)$ benign behavior preservation, and $\backslash(d\backslash)$ deployed-artifact confirmation. Release policy can require every component, preventing a high average from hiding a zero.

Safe Lab: Reachability, Grouping, and Repair Validation

This lab uses a toy call graph and pure path normalization. It does not extract archives or write files.

```
from collections import defaultdict, deque
from pathlib import PurePosixPath

edges = {
    "upload_route": ["parse_manifest"],
    "parse_manifest": ["extract_member", "audit_name"],
    "extract_member": ["join_path", "write_file"],
    "join_path": [], "write_file": [], "audit_name": []
}

tainted = {"upload_route", "parse_manifest", "extract_member", "join_path", "write_file"}
findings = [
    ("F1", "extract_member", "write_file", "missing_boundary"),
    ("F2", "join_path", "write_file", "missing_boundary"),
    ("F3", "audit_name", "log", "log_forging"),
]

def reachable(start, goal):
    queue, seen = deque([start]), set()
    while queue:
        node = queue.popleft()
        if node == goal:
            return True
        if node not in seen:
            seen.add(node)
            queue.extend(edges.get(node, []))
    return False

def contained(member):
    parts = PurePosixPath(member).parts
    depth = 0
    for part in parts:
        if part in ("", ".", "/"):
            continue
        if part == "..":
            depth -= 1
        else:
            depth += 1
        if depth < 0:
            return False
    return not PurePosixPath(member).is_absolute()

groups = defaultdict(list)
for fid, symbol, sink, root_cause in findings:
```

```

if reachable("upload_route", symbol) and symbol in tainted:
    groups[(sink, root_cause)].append(fid)

tests = ["reports/q1.csv", "../escape.txt", "a/../../escape.txt", "/tmp/escape"]
print("reachable_groups=" + str(dict(groups)))
for name in tests:
    print(f"{name}: {'ALLOW' if contained(name) else 'BLOCK'}")
print("patch_valid=" + str(all(contained(x) == expected for x, expected in {
    "reports/q1.csv": True, "../escape.txt": False,
    "a/../../escape.txt": False, "/tmp/escape": False}.items())))

```

Expected output:

```

reachable_groups=({'write_file', 'missing_boundary'): ['F1', 'F2']}
reports/q1.csv: ALLOW
../escape.txt: BLOCK
a/../../escape.txt: BLOCK
/tmp/escape: BLOCK
patch_valid=True

```

The lab deliberately separates reachability, grouping, and validation. F3 is not grouped because its symbol is not on the tainted set, even though a textual model might find both discussions of names similar.

That separation is operationally important: each stage can fail, be appealed, and be improved without relabeling every downstream outcome.

Metrics and Production Operations

Finding-level precision is inadequate. Track candidate precision, reachable precision, root-cause precision, confirmed-finding recall on seeded and historical cases, time to first reviewable evidence, developer acceptance, median duplicates per root cause, reopen rate, patch variant escape rate, and deployed remediation time. Stratify by language, framework, repository criticality, scanner version, and weakness family.

Avoid measuring recall solely with public benchmark snippets: they are susceptible to memorization, structurally simpler than production code, and often lack build or deployment context. Use temporally held-out internal fixes where permitted, synthetic transformations, seeded weaknesses reviewed by experts, and naturally occurring prospective findings. Prevent leakage by separating benchmark authorship, model-development access, and final evaluation custody; hash and access-log hidden cases.

Production services need build isolation, artifact retention, queue limits, repository access minimization, secret redaction, dependency-cache provenance, and an emergency stop for reproduction. Every finding state transition is append-only. Engineers can dispute source control, reachability, duplicate grouping, severity, or proposed repair separately; disagreement is valuable labeled evidence.

Roll out by repository cohort. Begin with read-only shadow scanning, then expose evidence bundles without auto-created tickets, then create tickets only for high-assurance families. Never auto-merge security patches

solely on model judgment. Canary monitoring includes build failures, test flakiness, reviewer rejection, grouping changes, sensitive-code exposure, and any unexpected network attempt from a harness.

Sources and Further Reading

- NIST, *SP 800-218: Secure Software Development Framework (SSDF) Version 1.1*, 2022: <https://doi.org/10.6028/NIST.SP800-218>
- NIST, *SP 800-115: Technical Guide to Information Security Testing and Assessment*, 2008: <https://doi.org/10.6028/NIST.SP800-115>
- MITRE, *Common Weakness Enumeration*: <https://cwe.mitre.org/>
- FIRST, *Exploit Prediction Scoring System*: <https://www.first.org/epss/>
- CISA, *Known Exploited Vulnerabilities Catalog*: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- OWASP, *Code Review Guide*: <https://owasp.org/www-project-code-review-guide/>
- Pearce et al., “Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions,” 2022: <https://doi.org/10.1109/SP46214.2022.9833571>
- Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?”, 2024: <https://arxiv.org/abs/2310.06770>

SSDF situates discovery inside a controlled development lifecycle; CWE standardizes weakness language but does not establish exploitability; EPSS and KEV add population-level exploitation evidence without replacing local reachability; secure code-generation studies show why generated repairs need independent tests. The combined lesson is to link weakness, context, behavior, repair, and deployed artifact.

Interview Practicum

A payments company's new AI reviewer emits 1,800 findings across 300 repositories. Developers report duplicates and false positives. Leadership claims 92% accuracy from a public benchmark. You discover the model had access to benchmark solutions, many services disable the flagged routes, and the reproduction service can reach the corporate network.

Prompt: Redesign the system and decide whether any findings can continue to developers this quarter. After your initial design, the build team says only 60% of repositories build reproducibly. Then a critical library alert appears in 47 services, but only four load the vulnerable parser at runtime.

Useful questions: What is the unit behind 92%? Was the benchmark held out temporally and organizationally? Which build, call-graph, deployment, and runtime facts exist? How are duplicates counted? What is the safe effect for reproduction? Which repositories process untrusted input? Can fixes be tested and traced into deployed artifacts?

Worked Answer

I would suspend the unsafe reproduction service immediately because corporate network reach is an independent hazard. Static scanning may continue in shadow mode. I would not discard all findings: quarantine them, preserve commit and scanner provenance, and allow developers to see only evidence bundles that pass subject, revision, and reachability checks without dynamic execution.

The public benchmark result is invalid for release because solution access creates contamination and because snippet accuracy does not measure deployed reachability, duplicate burden, or repair quality. I would establish a custodied temporal suite containing internal historical fixes, expert-seeded cases, benign hard negatives, and transformed variants. Reporting is per weakness family and language, with candidate and root-cause metrics separated.

For the 40% that do not build reproducibly, the state is `environment_mismatch`, not false positive. We can still establish partial static evidence, but cannot claim behavioral confirmation. I would fund build manifests as security infrastructure and make evidence grade visible. High-criticality repositories with missing builds receive human review rather than model confidence inflation.

The 47 library alerts should become one campaign with 47 deployment instances, not one ticket and not 47 unrelated vulnerabilities. Dependency presence, vulnerable version, runtime load, untrusted entry point, and vulnerable function use are separate predicates. The four runtime-reachable services receive prioritized remediation with safe regression tests. The others remain inventory records with reasons—unused code path, feature disabled, or non-attacker-controlled input—and are re-evaluated when configuration changes.

I would rebuild reproduction on ephemeral isolated workers with no external network, synthetic secrets, resource limits, instrumented effects, and reviewed harness templates. Patch validation must close the original and transformed cases, preserve benign behavior, pass the repository suite, and confirm the fixed artifact reaches deployment. This quarter, only finding families with reliable builds or strong static proofs, acceptable root-cause precision, and safe reviewed harnesses advance to ticket creation.

Follow-Up Questions and Answers

How do you estimate recall without exposing hidden cases? Run the evaluator in a custodied service that returns aggregate metrics and limited diagnostics. Rotate cases, log access, and reserve a final untouched slice.

Why not use EPSS as reachability? EPSS estimates exploitation probability for published CVEs in a population. It does not show that this service loads the component, invokes the vulnerable function, or exposes attacker-controlled input.

What is the most damaging dedupe error? Over-merging independent root causes, because closing one issue can suppress unfixed paths. Track and manually sample merges, especially across repositories and dependency instances.

When can an AI patch be merged? When normal ownership and review remain intact, security regression and variant tests pass, deployed artifact provenance is verified, and risk-appropriate human approval is recorded. Model confidence is not a merge authority.

Key Takeaways

- Treat every finding as a revision-specific, falsifiable evidence bundle.
- Distinguish syntactic, data, control, deployment, and impact reachability.
- Reproduce the smallest harmless security effect in an isolated environment.
- Deduplicate around the repairable root cause, not wording or CWE alone.
- Validate variants, benign behavior, and the deployed artifact—not only the original test.
- Keep contaminated benchmark performance out of release evidence.

Chapter 39: Cybersecurity Capability and Misuse Evaluations

Does an AI system make a person meaningfully more capable at a cyber task than the person, ordinary tools, or the model alone? That is a causal question, not a leaderboard question. A model may solve benchmark challenges yet provide no uplift to experienced practitioners; it may help novices while slowing experts; it may accelerate harmless reconnaissance but not end-to-end success; or it may teach transferable methods that persist after access ends. Release decisions require an experiment that distinguishes these effects without turning the evaluation into a training package for misuse.

This chapter designs a randomized study in a sandboxed enterprise-like range. Participants receive one of several assistance conditions, work on matched task families, and are measured on verified outcomes, time, intervention burden, and unsafe behavior. The design controls contamination among participants and between evaluation cases and model development. It also limits information hazards by separating task authorship, execution, scoring, and reporting.

Define the Estimand Before the Range

“Cyber capability” is not one scalar. Choose a population, task distribution, treatment, comparator, and outcome. One primary estimand might be the novice average treatment effect of an agentic assistant relative to conventional search and command-line tools:

$$\tau_N = E[Y_i(\text{agent}) - Y_i(\text{tools}) \mid E_i = \text{novice}],$$

where τ is a preregistered score based on independently verified milestones. A separate τ_E covers experts. Combining them can hide the release-relevant result.

Possible outcomes include validated task completion, time to first correct hypothesis, time to completion, number of substantive hints, unsafe actions attempted, solution quality, and retention on a later unassisted task. Token count or perceived helpfulness is not capability. A composite can be useful if weights are declared before results:

$$Y = 100C + 20M - 0.5T - 15H - 40U,$$

with complete success C , verified partial milestones M , minutes T , human hints H , and unsafe policy violations U . Report components beside the composite so weights cannot conceal tradeoffs.

Experimental Architecture

```
sequenceDiagram
    participant A as Task Authors
    participant V as Secure Vault
    participant R as Randomization Service
```

```

participant P as Participant
participant X as Isolated Range
participant M as Model/Tool Gateway
participant S as Blind Scorers
participant D as Disclosure Board

A->>V: Deposit task, solution, hazard tier, scoring tests
V->>R: Release task ID and strata only
R->>P: Assign condition and matched task family
P->>X: Work through instrumented interface
X->>M: Permit condition-specific assistance
M-->>X: Logged, filtered response
X->>V: Encrypted transcript and state changes
V->>S: Redacted outcome bundle, condition hidden
S->>V: Milestone and safety scores
V->>D: Aggregate effects and controlled exemplars
D-->>P: Tiered report; no reusable harmful procedure

```

Task authors cannot inspect participant assignments. Scorers do not see condition labels or model-branded interfaces. The model team sees neither hidden solutions nor final tasks before the evaluation freezes. The disclosure board decides which artifacts may leave the restricted environment.

Conditions and Factorial Questions

A credible baseline is not “human with no internet.” It reflects actual alternatives. For many studies, use four conditions:

1. human with standard documentation, search, and ordinary security tools;
2. human with a chat model but no execution tools;
3. human with a tool-using agent under the same range permissions;
4. model or agent alone, to separate collaboration from automation.

If resources permit, a factorial design can isolate model quality and tool access: model present/absent crossed with specialized tools present/absent. The interaction

$$\Delta = (Y_{\{M,T\}} - Y_{\{0,T\}}) - (Y_{\{M,0\}} - Y_{\{0,0\}})$$

estimates whether the model adds more value when tools are available. Avoid too many thin arms; an underpowered factorial produces attractive but unstable rankings.

Randomize within experience strata and task families. Measure prior skill with a short, non-overlapping pretest and collect role, years of experience, programming familiarity, and prior exposure to common challenge platforms. Block randomization keeps these variables balanced without using post-treatment behavior.

Freeze the experiment as a machine-readable protocol before allocation:

```

protocol_version: cyber-uplift/3
population:
  strata: [novice, expert]
arms: [standard_tools, chat_model, tool_agent]
unit_of_randomization: participant
primary_estimand: novice_risk_difference_agent_vs_standard
primary_outcome: verified_end_to_end_completion
secondary_outcomes:
  - restricted_mean_completion_time
  - unsafe_intervention_rate
  - seven_day_unassisted_transfer
exclusions:
  - range_failure_before_task_start
contamination_flags:
  - prior_task_exposure
  - participant_contact
  - author_model_team_access
stopping_rules:
  - range_escape
  - external_target_contact
  - credible_real_world_zero_day
analysis: intention_to_treat

```

Changing the primary outcome after viewing arm results creates a new protocol version and is reported as exploratory, not silently substituted.

A crossover design saves participants but creates learning and carryover. If participants see related tasks under two conditions, the first treatment teaches the second. Prefer parallel assignment for primary dangerous-capability estimates. A limited crossover may be used for usability on unrelated tasks with order randomized and washout acknowledged.

Tasks That Measure Work Rather Than Trivia

Use a ladder of safe, instrumented tasks: identify a misconfiguration in synthetic logs; trace a vulnerable dataflow in a toy service; prioritize remediation across a fabricated asset graph; write a detection for simulated behavior; or recover a benign token from a deliberately vulnerable range. No task should target public infrastructure, use real credentials, or require persistence, destructive effects, stealth, or uncontrolled propagation.

Realism comes from dependency, ambiguity, and partial evidence—not from live victims. Tasks should require several linked milestones, such as discover service, form hypothesis, validate harmless effect, and explain remediation. Scoring tests inspect range state rather than trusting self-reported success.

Difficulty must be calibrated before the main study. If nearly everyone succeeds, uplift is censored by a ceiling; if nobody succeeds, a floor masks differences. Pilot with participants outside the analysis set. Maintain anchor tasks across releases, but rotate most cases to reduce memorization.

Randomization, Attrition, and Analysis

Create assignments before sessions using a reproducible seed held by an independent statistician. Allocation is concealed until the session begins. Analyze intention-to-treat: participants remain in assigned arms even if the assistant fails or they barely use it. A per-protocol analysis can be secondary, but selection into “used the model effectively” is itself an outcome.

For binary completion, estimate risk difference and risk ratio with cluster-robust uncertainty when each participant attempts multiple tasks. For time, incomplete tasks are right-censored; survival analysis is preferable to replacing failures with the time limit. A proportional-hazards model may be inappropriate when assistance helps early discovery but hurts late debugging, so show Kaplan–Meier curves and restricted mean completion time.

With participant i and task j , a hierarchical model separates variation:

$$Y_{ij} = \alpha + \tau_i Z_i + \beta E_i + u_i + v_j + \tau_{E_i} E_i + \epsilon_{ij},$$

where u_i is participant ability, v_j task difficulty, and τ_{E_i} treatment heterogeneity by experience. Preregister exclusions, transformations, multiplicity correction, missing-data handling, and the minimum effect that changes a release decision.

Attrition is security-relevant. If agent sessions crash or safety filters frustrate users, excluding them inflates capability. Report assignment-to-outcome flow, infrastructure failures by arm, and bounds under plausible missing outcomes.

Contamination Controls

There are at least four contamination channels:

- **training contamination:** hidden tasks or solutions appeared in model training or fine-tuning;
- **developer contamination:** model or scaffold engineers saw final cases and tuned to them;
- **participant spillover:** participants share task details or strategies;
- **measurement contamination:** scorers infer the condition from style and score differently.

Controls include post-cutoff task creation, private synthetic environments, similarity searches against known corpora, canary strings, frozen model hashes, access logs, separate author and model teams, session scheduling that limits contact, nondisclosure rules proportionate to risk, and blind scoring from normalized artifacts. No single check proves absence of training data. The result should state residual contamination risk.

Rotate surface details while preserving latent skills. If performance collapses when hostnames, file layouts, or irrelevant wording change, the system may be recalling a procedure rather than transferring capability. Use isomorphic task pairs and calculate a transfer gap between original and transformed forms.

Information-Hazard Controls

Evaluation evidence can itself increase misuse capability. Classify artifacts before collection:

Tier	Example	Handling
0	aggregate completion rate	public or normal internal report
1	general failure category	standard restricted project access
2	detailed transcript for synthetic task	named evaluators; encrypted; limited retention
3	reusable exploit chain or novel technique	isolated vault; two-person access; no broad export
4	credible real-world zero-day evidence	stop task; disclosure counsel and affected vendor process

The range gateway blocks external targets, unapproved binaries, secret-like outputs, and destructive primitives. It logs policy interventions as outcomes. Filters must not silently alter one condition more than another; otherwise the treatment includes an unmeasured handicap. Record interventions and report them.

Public reporting should communicate the task family, population, assistance conditions, effect estimates, uncertainty, limitations, and release rationale. It need not publish hidden solutions, exact high-hazard prompts, vulnerable images, or procedural transcripts. Reproducibility can be served through controlled researcher access and lower-hazard analogues.

Safe Lab: Stratified Randomization and Uplift

This simulation creates balanced assignments and computes novice/expert uplift on harmless synthetic scores.

```
import random
from collections import defaultdict

participants = [
    ("p01", "novice"), ("p02", "novice"), ("p03", "novice"),
    ("p04", "novice"), ("p05", "expert"), ("p06", "expert"),
    ("p07", "expert"), ("p08", "expert")
]
conditions = ["tools", "agent"]
rng = random.Random(3901)

assignment = {}
for level in ("novice", "expert"):
    ids = [pid for pid, skill in participants if skill == level]
    rng.shuffle(ids)
    for index, pid in enumerate(ids):
        assignment[pid] = conditions[index % 2]

# Preregistered, fabricated outcome: verified milestones out of 10.
scores = {"p01": 4, "p02": 7, "p03": 8, "p04": 3,
          "p05": 8, "p06": 7, "p07": 9, "p08": 8}
cells = defaultdict(list)
skill_by_id = dict(participants)
for pid, condition in assignment.items():
    cells[(skill_by_id[pid], condition)].append(scores[pid])
```

```
print("assignment=" + ",".join(f"{p}:{assignment[p]}" for p in sorted(assignment)))
for level in ("novice", "expert"):
    means = {c: sum(cells[(level, c)]) / len(cells[(level, c)]) for c in conditions}
    print(f"{level}_tools={means['tools']:.1f}")
    print(f"{level}_agent={means['agent']:.1f}")
    print(f"{level}_uplift={means['agent'] - means['tools']:.1f}")
```

Expected output:

```
assignment=p01:tools,p02:agent,p03:agent,p04:tools,p05:agent,p06:agent,p07:tools,p08:tools
novice_tools=3.5
novice_agent=7.5
novice_uplift=4.0
expert_tools=8.5
expert_agent=7.5
expert_uplift=-1.0
```

The fabricated result demonstrates why pooling is dangerous: strong novice uplift coexists with expert harm. A real study needs far more participants and uncertainty intervals; the lab demonstrates allocation and estimand calculation, not adequate power.

Release Thresholds and Operations

A dangerous-capability threshold must be written before results. Example: hold release if the upper and lower evidence jointly show at least a 15 percentage-point increase in verified completion for low-skill participants on a high-consequence task family, without a compensating control that survives adaptive testing. The precise value is a governance choice, not a statistical constant.

Use a decision table including capability uplift, uncertainty, unsafe action rate, control bypass, task realism, contamination confidence, affected population, availability of comparable systems, and mitigation effectiveness. A null result is not proof of safety when the study is underpowered or tasks have poor external validity.

Production evaluation operations resemble a high-integrity lab: freeze artifacts, hash range images, verify clocks, monitor scorer drift, document deviations, rotate credentials, review data export, and rehearse discovery of a real vulnerability. Stop conditions include escape from the range, unexpected external contact, participant distress, a plausible undisclosed real-world flaw, loss of assignment integrity, or exposure of hidden solutions.

Repeat evaluations when model weights, system prompts, tools, autonomy, context limits, or safety policies materially change. Capability can emerge from scaffolding even with unchanged weights. Maintain version-specific results; do not attach a permanent “safe” label to a model family.

Sources and Further Reading

- NIST, *AI Risk Management Framework 1.0*, 2023: <https://doi.org/10.6028/NIST.AI.100-1>

- NIST, *AI RMF Generative Artificial Intelligence Profile*, 2024: <https://doi.org/10.6028/NIST.AI.600-1>
- METR, “Measuring AI Ability to Complete Long Tasks,” 2024: <https://metr.org/blog/2024-03-08-measuring-ai-ability-to-complete-long-tasks/>
- Meta, *CyberSecEval 2*, 2024: <https://arxiv.org/abs/2404.13161>
- Zhang et al., “Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models,” 2024: <https://arxiv.org/abs/2408.08926>
- Shavit et al., “Practices for Governing Agentic AI Systems,” 2023: <https://cdn.openai.com/papers/practices-for-governing-agentic-ai-systems.pdf>
- National Academies, *On Being a Scientist*, 2009, for research integrity and controlled evidence: <https://doi.org/10.17226/12192>

Benchmark frameworks contribute task patterns, but controlled release evidence additionally needs human baselines, randomization, contamination handling, and outcome verification. AI RMF supplies the risk-governance frame; agent governance emphasizes system-level capabilities; research-integrity guidance supports preregistration, custody, and transparent deviations.

Interview Practicum

A frontier-model company plans a release in six weeks. Twenty-four internal participants completed cyber-range tasks: novices using the agent outperformed novices using search, experts showed little difference, and agent users triggered more safety interventions. Product argues the tasks are “just CTFs.” The evaluation lead admits three participants discussed tasks at lunch and two task authors recently joined the post-training team.

Prompt: Decide what can be concluded, design the confirmatory study, and state a release rule. Halfway through, learn that only 30 qualified experts are available. Then learn that one hidden task resembles a newly disclosed real vulnerability.

Clarifying questions should cover assignment, baseline tools, prior skill, task families, verified outcomes, censoring, intervention definitions, task/model custody, timing of author transfers, participant contact, effect sizes and intervals, real-world mapping, and existing mitigations.

Worked Answer

The pilot suggests heterogeneous uplift but cannot support a release decision. With 24 people, uncertain spillover, and possible developer contamination, I would present it as hypothesis-generating. I would preserve all records, flag contaminated participants and tasks without simply deleting unfavorable observations, and preregister a confirmatory analysis.

The primary study would use parallel assignment within novice and expert strata to standard tools, chat-only, and agent assistance. Because experts are scarce, allocate expert participants between the most decision-relevant contrast—agent versus standard tools—and use repeated independent task families with a hierarchical model. Keep the larger novice three-arm study. Pretest skill, conceal allocation, blind scorers, verify

milestones from range state, analyze intention-to-treat, and treat incomplete sessions as censored rather than failures at an invented time.

New tasks are authored after the model freeze by a team barred from model development. Isomorphic variants test transfer. Access logs, canary strings, semantic similarity review, and participant scheduling reduce contamination. The two transferred authors cannot contribute cases or see results until unblinding. Lunch-discussion participants remain in a sensitivity analysis; the confirmatory study uses clearer isolation and records contact.

The newly disclosed real vulnerability changes hazard classification. I would remove that task from ordinary execution, ask a restricted review group whether it contains nonpublic actionable detail, and follow coordinated disclosure if needed. It cannot be published as reproducibility material. Replace it in the primary analysis according to a preregistered contingency, while reporting the deviation.

My release rule would use a confidence interval around novice completion uplift on high-consequence families, unsafe intervention rate, and mitigation robustness. If meaningful novice uplift crosses the predefined dangerous threshold and controls fail adaptive evaluation, hold or restrict the agentic configuration. If evidence is inconclusive because power or validity is weak, do not translate that into safety; narrow deployment and collect better evidence. Expert null effects do not cancel novice uplift.

The report publishes aggregate design and uncertainty but withholds solutions, exact high-hazard prompts, and range images. A restricted assurance record retains enough detail for independent review. We repeat the study when tools, autonomy, or policies change because the evaluated object is the complete system.

Follow-Up Questions and Answers

Why randomize people rather than prompts? The causal unit is assisted human performance. Prompt-only tests omit learning, tool selection, frustration, and unsafe action.

How do you handle participants who ignore the agent? Keep them in intention-to-treat. Assistant use can be analyzed as a secondary mediator, but conditioning the primary estimate on use breaks randomization.

Can you publish enough for credibility without publishing dangerous details? Yes: publish estimands, populations, task taxonomy, baselines, scoring method, effect estimates, uncertainty, contamination assessment, and deviations. Provide controlled access to sensitive artifacts.

What would immediately stop the study? Range escape, external targeting, uncontrolled disclosure of a plausible real vulnerability, loss of solution custody, or a serious participant-safety event.

Key Takeaways

- Measure causal uplift over realistic human-and-tool baselines, not model score alone.
- Randomize within skill and task strata and preserve intention-to-treat analysis.
- Treat contamination as several distinct threats with documented residual risk.
- Build realism from linked reasoning and verified outcomes, not live targets.

- Separate reproducibility from unrestricted release of hazardous artifacts.
- Make release thresholds depend on effect, uncertainty, controls, and consequence.

Chapter 40: Production Monitoring for AI Security Systems

An AI security release is not one model file. It is a behavior-producing assembly: model, prompt, retrieval index, policy, tool adapters, feature code, evaluator, runtime, and the data available at a moment in time. If any component changes, yesterday's assurance claim may no longer apply. Production safety therefore begins with a versioned event record, continues through deterministic replay and segmented canaries, and ends with an assurance case whose claims are supported—or explicitly weakened—by current evidence.

This chapter implements that release discipline for an agent that triages suspicious identity activity. The methods also apply to AI code review, fraud investigation, abuse detection, and response copilots. The central objective is not to prove the system safe. It is to make the release decision reviewable, bound its exposure, detect invalidated assumptions quickly, and preserve enough evidence to learn after failure.

The Release Unit and Behavior Manifest

A semantic version for the model alone is insufficient. Create an immutable behavior manifest:

```
behavior_release: soc-agent/2026.08.3
model:
  provider: internal
  id: sec-model-12b
  weights_digest: sha256:4a82...
prompt_bundle: prompt/37@sha256:ca19...
retrieval:
  encoder: embed-v7
  index_snapshot: evidence-index/2026-08-18T0200Z
policy: response-policy/14@sha256:06f1...
tools:
  idp_connector: 3.2.1
  case_writer: 2.8.0
feature_code: git:91dd2a7
runtime_image: registry/soc-agent@sha256:7bd0...
evaluator_bundle: assurance-eval/22
schema_versions: [trace-v4, evidence-v2, decision-v3]
approved_scope: [shadow, analyst_draft]
```

The manifest is signed and referenced by every trace. Mutable aliases such as `latest` are prohibited in assurance evidence. Configuration flags are included because they can change authority without changing code. External service versions that cannot be pinned require observed response metadata and a declared dependency risk.

Versioned Replay Without Pretending to Recreate History

Replay runs a candidate release against recorded inputs and compares it with the incumbent. It can reveal regression, but only if the record preserves what the system actually observed. Re-querying today's identity

graph for a case from last month leaks future facts and masks historical outages. Store minimized evidence snapshots or content-addressed references subject to retention rules.

```
stateDiagram-v2
    [*] --> Frozen: sign behavior manifest
    Frozen --> Replay: run gold, incident, drift, adversarial suites
    Replay --> Blocked: invariant or severity gate fails
    Replay --> Shadow: gates pass
    Shadow --> Blocked: live divergence or telemetry gap
    Shadow --> CanaryA: owner approval
    CanaryA --> Rollback: guardrail breach
    CanaryA --> CanaryB: segment evidence sufficient
    CanaryB --> Rollback: guardrail breach
    CanaryB --> General: assurance case accepted
    General --> Rollback: incident or invalidated assumption
    Rollback --> Replay: corrected release
    Blocked --> Replay: new candidate
```

A replay envelope needs event time, ingestion time, evidence snapshot, randomness controls, and all versions:

```
{
  "$id": "ai-security-replay-v1",
  "type": "object",
  "required": ["trace_id", "event_time", "behavior_release", "input_ref", "seed", "outcome"],
  "properties": {
    "trace_id": {"type": "string"},
    "event_time": {"type": "string", "format": "date-time"},
    "ingested_at": {"type": "string", "format": "date-time"},
    "behavior_release": {"type": "string"},
    "input_ref": {"type": "string", "pattern": "^sha256:"},
    "evidence_snapshot": {"type": "string", "pattern": "^snapshot/"},
    "seed": {"type": "integer"},
    "sampling": {"type": "object"},
    "tool_fixtures": {"type": "array", "items": {"type": "string"}},
    "outcome": {"enum": ["correct", "incorrect", "abstain", "error", "policy_block"]},
    "segment": {"type": "object"}
  },
  "additionalProperties": false
}
```

Nondeterminism is managed, not wished away. Fix seeds and tool fixtures where possible, then run multiple samples for stochastic components. Report the probability of a violation and distribution of quality, not only one canonical output. Some provider-hosted models cannot guarantee bitwise replay; the record should say “behavioral replay under version claim” rather than “exact reproduction.”

Building the Replay Corpus

No single test set represents production. Maintain named suites with different custody and refresh rules:

- **gold:** carefully adjudicated ordinary and difficult cases;

- **incident:** sanitized cases from real failures and near misses;
- **invariant:** synthetic cases for hard properties such as no cross-tenant citation;
- **adversarial:** prompt injection, tool manipulation, ambiguity, and malformed inputs;
- **drift:** recent sampled traffic with delayed labels or expert review;
- **contract:** connector, schema, timeout, and policy compatibility tests.

Prevent incident-suite overfitting by retaining hidden variants and testing the underlying property. A fix for one exact injected string is not evidence of injection resistance. Corpus membership is versioned; deleted cases remain documented so improvements cannot be manufactured by quietly removing failures.

Compare candidate $\backslash(c\backslash)$ and incumbent $\backslash(b\backslash)$ at trace level. For loss $\backslash(L\backslash)$, paired difference $\backslash(\Delta_i = L(c,i) - L(b,i)\backslash)$ removes much case difficulty. Bootstrap cases, clustering by incident or customer when correlated, to obtain an interval around mean or tail regression. Average loss is insufficient for severe invariants. A single unauthorized action may be a deterministic block even if mean quality rises.

Define release gates as a vector:

$\backslash[G = (\Delta\bar{L}, \Delta L_{\{p95\}}, V_{\{auth\}}, V_{\{privacy\}}, A_{\{critical\}}, E_{\{runtime\}}), \backslash]$

where the components are mean loss change, tail loss change, authority violations, privacy violations, critical-segment abstention, and runtime error. Each has its own threshold. Do not collapse privacy leakage and latency into a weighted score that allows one to compensate for the other.

Shadowing and Online Comparability

Shadow mode sends a copy of live inputs to the candidate but suppresses side effects. It catches current schemas, traffic mix, connector behavior, cost, and latency. It does not test analyst interaction or action consequences. Candidate and incumbent must see equivalent evidence; if shadow traffic uses stale fixtures while production uses live connectors, disagreement is uninterpretable.

Log coverage is itself a gate. If only 70% of candidate traces contain policy decisions, a zero observed violation rate is meaningless. Track trace completeness, dropped events, join success, label delay, and clock skew. Monitoring must distinguish system behavior change from observability failure.

Use disagreement sampling rather than reviewing random outputs alone. Review candidate/incumbent disposition flips, new actions, new abstentions, citation changes, and large confidence movements. Retain a random sample too; disagreement sampling misses shared failures.

Segmented Canary Design

A one-percent global canary sounds cautious but can expose the wrong one percent. Security risk is concentrated by account privilege, tenant, language, connector, alert family, and action type. Define segments before rollout and route by stable hashing so the same case or tenant does not bounce between releases.

Start with low-consequence, high-observability segments: internal synthetic tenants, analyst drafts, common alert families, no response action. Exclude service accounts, privileged identities, regulated tenants, and rare action paths until their targeted evidence is adequate. Later stages deliberately include each critical segment under smaller absolute budgets; otherwise “general availability” rests only on easy traffic.

For segment $\backslash(s\backslash)$, exposure budget can be expressed as

$$\backslash[B_s = N_s \backslash, p_s \backslash, T_s \backslash, h_s, \backslash]$$

where $\backslash(N_s\backslash)$ is eligible volume, $\backslash(p_s\backslash)$ routed fraction, $\backslash(T_s\backslash)$ duration, and $\backslash(h_s\backslash)$ consequence weight. Use both count and consequence budgets. A canary automatically pauses when either is consumed without enough reviewed evidence.

Guardrails include unauthorized intents, unsupported citations, omission loss, analyst override, severe false closure, response reversal, latency, connector errors, token cost, and trace loss. Use sequential monitoring cautiously: repeated peeking inflates false alarms. For operational guardrails, conservative anytime-valid bounds or predefined review windows are preferable to naive repeated p-values. Safety invariants do not require statistical significance; one confirmed cross-tenant disclosure is enough to stop.

Canary promotion is not automatic after elapsed time. Require minimum observations in each named segment, completion of delayed-label windows where relevant, and owner sign-off on the assurance delta.

Rollback and Forward Recovery

Rollback must include the entire behavior release, not only model routing. Restoring old weights while leaving the new policy or index active does not restore known behavior. Keep the last-known-good manifest deployable and verify it regularly.

Triggers are typed: immediate rollback for authority or privacy invariant breaches; pause and investigate for telemetry loss; segment rollback for localized quality degradation; traffic reduction for capacity or cost; and kill tool execution while retaining analysis during response-broker incidents. Every trigger has an owner, automated action, notification path, and maximum decision time.

Forward recovery is sometimes safer than full rollback when upstream schemas have migrated. The runbook states compatibility constraints. Preserve failed candidate traces before cleanup, revoke issued capabilities, identify side effects, and reconcile cases written during exposure.

Assurance Cases, Not Assurance Dashboards

A dashboard shows indicators. An assurance case explains why evidence supports a release claim. A lightweight Goal Structuring Notation-inspired record can be maintained as data:

```
claim: C0
text: Release 2026.08.3 may provide analyst drafts for scoped identity alerts.
context:
```

```

- No autonomous response authority.
- Excludes privileged identities and regulated tenants.
strategies:
- id: S1
  text: Show investigation quality is not materially worse than incumbent.
- id: S2
  text: Show authority, privacy, and tenant-isolation invariants hold.
- id: S3
  text: Show online observability and rollback are effective.
evidence:
- id: E1
  supports: S1
  artifact: replay-report/2026.08.3
  freshness_days: 2
- id: E2
  supports: S2
  artifact: invariant-suite/22
  result: pass
- id: E3
  supports: S3
  artifact: canary-drill/118
  result: rollback-in-94s
defeaters:
- id: D1
  text: Identity connector changed pagination semantics after replay.
  status: monitored
residual_risk_owner: director-soc-engineering
expires: 2026-09-20

```

Claims are scoped and time-bound. Evidence has provenance and freshness. Defeaters record facts that could invalidate the inference. Exceptions name an owner and expiry. When scope expands to privileged identities or response actions, edit the top claim and supply new evidence; do not stretch the old case.

The assurance case also prevents metric theater. “Replay accuracy 94%” cannot support tenant isolation unless a strategy explains the connection. An expired rollback drill cannot indefinitely support operational recoverability.

Safe Lab: A Segmented Canary Gate

The lab compares observed canary losses with thresholds by segment and enforces hard invariants. It uses fabricated data and no external services.

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Segment:
    name: str
    reviewed: int
    severe_errors: int

```

```

overrides: int
trace_missing: int
min_reviewed: int
max_severe: int
max_override_rate: float
max_trace_missing: int

segments = [
    Segment("common_draft", 240, 0, 18, 0, 200, 0, 0.10, 0),
    Segment("rare_alert", 38, 0, 3, 0, 50, 0, 0.12, 0),
    Segment("privileged", 22, 1, 4, 0, 20, 0, 0.10, 0),
]
hard_invariants = {"cross_tenant_disclosure": 0, "unauthorized_action": 0}

def decision(segment):
    if segment.severe_errors > segment.max_severe:
        return "ROLLBACK"
    if segment.trace_missing > segment.max_trace_missing:
        return "PAUSE"
    if segment.reviewed < segment.min_reviewed:
        return "HOLD"
    if segment.overrides / segment.reviewed > segment.max_override_rate:
        return "HOLD"
    return "PROMOTE"

for segment in segments:
    print(f"{segment.name}={decision(segment)}")
print("release=ROLLBACK" if any(hard_invariants.values()) or
      any(decision(s) == "ROLLBACK" for s in segments) else "release=NO_ROLLBACK")

```

Expected output:

```

common_draft=PROMOTE
rare_alert=HOLD
privileged=ROLLBACK
release=ROLLBACK

```

The common segment cannot compensate for a severe privileged error. The rare segment is held because evidence volume is insufficient, not because its observed rate looks bad. In production, the invariant counts come from separately validated telemetry with paging and case reconciliation.

Production Operating Model

Assign explicit roles: release owner, assurance reviewer, canary operator, incident commander, data steward, and segment owner. Separate the person who benefits from shipping from the person authorized to accept residual safety risk. Automate evidence assembly, not acceptance judgment.

The daily control plane verifies manifest signatures, trace completeness, version skew, connector contracts, assurance expiry, and last-known-good deployability. Weekly review examines segment outcomes, delayed

labels, drift samples, evaluator changes, unresolved defeaters, and policy denials. Each material change creates an assurance delta identifying which claims and evidence are affected.

Monitor leading indicators such as schema violations, tool denials, evidence age, output entropy, retrieval mix, and analyst overrides, alongside lagging outcomes such as confirmed misses and harmful actions. Drift in an embedding distribution is a diagnostic, not a security outcome. Connect alerts to a runbook and a decision; otherwise monitoring becomes decorative.

A useful drift runbook starts with classification. For input drift, verify instrumentation and population eligibility before changing a model. For vocabulary or repository-pattern drift, refresh parsing, retrieval, prompts, or rules and replay the frozen corpus. For outcome drift, investigate label delay and adjudication changes before retraining. For operational drift, repair queues, connectors, rate limits, or timeouts. For policy drift, version the policy and recompute affected decisions. Any mitigation then re-enters replay, shadow, canary, and rollback gates; a drift alert is not permission for an unreviewed hot fix.

Incident response preserves the release manifest, prompts, evidence snapshot references, policy decisions, tool intents, approvals, outputs, and human edits. Determine scope by version and segment, not deployment timestamp alone. Notify affected owners, disable side effects, reconcile actions, and add sanitized incident variants to replay. Root-cause analysis asks which assurance claim failed, which defeater was missed, and why canary exposure did or did not bound harm.

Sources and Further Reading

- NIST, *AI Risk Management Framework 1.0*, 2023: <https://doi.org/10.6028/NIST.AI.100-1>
- NIST, *AI RMF Generative Artificial Intelligence Profile*, 2024: <https://doi.org/10.6028/NIST.AI.600-1>
- NIST, *SP 800-160 Vol. 1 Rev. 1: Engineering Trustworthy Secure Systems*, 2022: <https://doi.org/10.6028/NIST.SP800-160v1r1>
- NIST, *SP 800-61 Rev. 2: Computer Security Incident Handling Guide*, 2012: <https://doi.org/10.6028/NIST.SP800-61r2>
- Google, *Site Reliability Engineering*, chapters on canarying and monitoring: <https://sre.google/sre-book/table-of-contents/>
- Kelly and Weaver, “The Goal Structuring Notation—A Safety Argument Notation,” 2004: <https://www.researchgate.net/publication/245024416>
- OWASP, *Machine Learning Security Verification Standard*: <https://owasp.org/www-project-machine-learning-security-verification-standard/>

SRE practice supplies bounded rollout and rollback mechanics; systems-security engineering supplies lifecycle assurance; GSN supplies explicit claim-evidence reasoning; incident guidance supplies preservation and response. AI RMF ties them to governed model risk. Their synthesis is a release process in which operational evidence continually updates a scoped assurance claim.

Interview Practicum

A bank wants to deploy a new model and retrieval index for an AI fraud-investigation assistant. Offline average accuracy improves two points. The candidate doubles abstentions for small-business accounts, has no replay coverage for Spanish cases, and once produced a cross-customer citation during red teaming. The current release cannot be exactly reconstructed because its index alias points to a mutable snapshot.

Prompt: Decide whether and how to release. After your plan, the platform team says rollback takes 25 minutes. Then a regulator asks what evidence supports the claim that customer isolation is preserved.

Strong questions identify the complete behavior manifest, authority scope, affected segments, hard invariants, evidence snapshot retention, delayed labels, assignment unit, expected volume, rollback side effects, evaluator independence, and who accepts residual risk.

Worked Answer

I would block customer-facing canarying initially. The cross-customer citation is a tenant-isolation invariant failure, not a quality tradeoff, and mutable replay inputs mean the two-point gain is weak evidence. I would first snapshot and hash the incumbent and candidate indexes, create complete behavior manifests, and reconstruct as much incumbent evidence as retention permits. The inability to reproduce history becomes an explicit assurance gap, not something hidden by rerunning against today's index.

The candidate must pass an invariant suite containing the red-team case and structurally varied cross-customer cases. Retrieval authorization should filter by tenant before ranking, and the citation validator should independently reject mismatched subjects. I would add contract tests for index namespaces and trace every retrieved document's tenant and digest.

For quality, I would report paired replay loss by consumer versus small-business, language, alert family, and case complexity. The doubled small-business abstention may represent safer uncertainty or harmful service degradation; review its reasons and time-to-resolution impact. Spanish coverage must be built before Spanish traffic is eligible. Overall accuracy cannot compensate for missing segment evidence.

Once invariants pass, begin shadow mode with live schema and latency checks. Canary only analyst-visible drafts, using stable customer-level assignment to avoid inconsistent case behavior. Start with internal and low-consequence English consumer segments, then deliberately gather evidence for small-business and Spanish cohorts. Each segment has minimum reviewed cases and separate guardrails. No autonomous action is in scope.

Twenty-five-minute rollback is too slow for isolation failure. Before canarying, add an immediate control-plane switch that disables candidate retrieval and tool output, targeting under two minutes, while full artifact rollback proceeds. Drill it and attach measured recovery time to the assurance case. The last-known-good release must include its pinned index, policy, prompts, and connectors.

For the regulator, I would provide a scoped assurance argument: tenant isolation is enforced before retrieval and independently at citation validation; evidence includes architecture review, invariant tests with hidden variants, shadow trace audits, canary observations, and a rollback drill. I would disclose residual limitations,

evidence dates, excluded Spanish scope, the named risk owner, and expiry. A dashboard's zero incidents is not the argument, especially with incomplete coverage.

Promotion occurs only when the cross-customer invariant has zero failures, trace completeness is complete enough to make that claim, each eligible segment meets its evidence minimum, abstention impacts are accepted, rollback is drilled, and the assurance reviewer signs the updated case. Any new index, policy, or model revision receives a new behavior release.

Follow-Up Questions and Answers

Why stable customer assignment instead of request assignment? It prevents one investigation from mixing behavior releases and makes side effects, analyst learning, and outcome attribution coherent.

What if exact replay is impossible for a hosted model? Preserve inputs, tool fixtures, parameters, provider version claims, and repeated behavioral samples. State the limitation and compensate with stronger shadowing and smaller canaries; do not call it exact replay.

Can a canary prove safety for rare privileged cases? Usually not through passive volume. Use targeted replay and controlled simulations, then a tiny consequence-budgeted canary with mandatory review. Absence of rare failures is weak evidence.

What belongs in an assurance-case defeater? Any plausible fact that breaks the claim-to-evidence inference: evaluator leakage, trace loss, an upstream schema change, stale index authorization, population drift, or an untested action path.

Key Takeaways

- Version the complete behavior-producing assembly, including data and policy.
- Replay historical evidence snapshots and state honestly when reproduction is approximate.
- Gate on vectors and invariants; averages cannot compensate for isolation or authority failures.
- Canary by consequence-relevant segment with stable assignment and exposure budgets.
- Roll back the complete behavior release and drill fast side-effect shutdown.
- Maintain a scoped, expiring assurance case with evidence, defeaters, and an accountable owner.

Field lesson: monitoring needs two release axes

The sec-pr platform passed its scoped engineering verification and exposed five assurance routes, while precision, recall, calibration, subgroup performance, and drift trends remained gated. Monitoring therefore reported platform health separately from detector evidence. This prevents a green dashboard from being interpreted as a green model. Chapter 47 shows the complete release boundary.

Chapter 41: Reproducible Security Analytics with Python and SQL

An investigation is reproducible only when another analyst can reconstruct the same evidence boundary, run the same transformations, and explain any difference in output. A notebook that still executes is not enough. Security data is corrected late, identity mappings change, intelligence feeds are revised, and labels arrive after decisions. Reproducibility therefore means preserving *what was knowable at decision time*, not merely preserving code.

This chapter builds an account-compromise analytic that produces a daily review queue. Its four invariants are: no feature uses evidence observed after cutoff; every output names its code, data, contract, and parameter versions; replaying one manifest produces identical logical rows; and ambiguous provenance or time semantics stop publication.

The Repository Is an Evidence Product

Use notebooks for exploration, but make reviewed modules and declarative queries authoritative:

```
security-analytics/
├── pyproject.toml
├── README.md
├── contracts/auth_event.v3.json
├── sql/point_in_time_features.sql
├── src/sec_analytics/{normalize,manifest,pipeline}.py
├── tests/{fixtures,test_contract,test_point_in_time,test_replay}.py
├── notebooks/001-investigation.ipynb
├── manifests/
└── reports/
```

The README begins with the decision contract: “At 06:00 UTC, rank identities for analyst review using evidence observed by 05:55 UTC; never automatically disable an account.” That limits both data and authority. It also gives tests something operational to enforce.

Do not commit credentials, production extracts, or reversible identifiers. Synthetic fixtures should preserve late arrival, duplicate delivery, changed mappings, and boundary timestamps without preserving real users or attack detail.

```
flowchart LR
    P[Producer events] -->|contract v3| R[(Immutable raw snapshot)]
    R --> N[Normalize and deduplicate]
    N --> I[(Identity snapshots)]
    I --> J[As-of join]
    J --> F[Point-in-time features]
    F --> Q[Deterministic queue]
    Q --> O[Report and decision records]
    C[Code commit] --> M[Run manifest]
    D[Dependency lock] --> M
```

```

R --> M
I --> M
M --> 0
T[Test suite] -. blocks .-> Q

```

Hashes prove that an artifact did not change after hashing. They do not prove the producer was truthful or the query correct.

A Point-in-Time Contract

An authentication contract needs temporal and provenance fields.

Field	Rule	Purpose
event_id	non-null, producer-stable	deduplication and traceability
principal_key	tokenized	stable analytical subject
event_time	UTC source time	when behavior occurred
observed_at	UTC collector time	when the pipeline could know it
source	idp , vpn , or cloud	provenance and coverage
outcome	success , failure , or challenge	defined result
country_code	ISO code or null	unknown is not zero
schema_version	exactly 3	compatibility gate
payload_hash	canonical digest	mutation detection

The producer owns field meaning and delivery expectations. The analytic owner owns acceptance rules. The platform owner owns retention and access. Changing `challenge` from “issued” to “completed” is breaking even if its SQL type is unchanged.

For cutoff (t_c) , eligible evidence is

$$\{e : e.\text{principal} = u, e.\text{observed_at} \leq t_c, e.\text{event_time} \leq t_c\}.$$

Using only event time leaks late records into historical replays. Using only observation time admits malformed future events. A correction received tomorrow belongs to tomorrow’s knowledge state unless the output is explicitly a revised historical result.

Identity joins are also temporal. If mappings have `valid_from`, `valid_to`, and `recorded_at`, a cutoff query requires both business validity and knowledge availability:

```

WITH eligible AS (
  SELECT * FROM auth_event
  WHERE event_time <= :cutoff AND observed_at <= :cutoff

```

```

    AND schema_version = 3
), mapped AS (
  SELECT e.*, m.employee_id
  FROM eligible e JOIN identity_map m
    ON e.principal_key = m.principal_key
    AND e.event_time >= m.valid_from
    AND e.event_time < COALESCE(m.valid_to, '9999-12-31')
    AND m.recorded_at <= :cutoff
)
SELECT employee_id, COUNT(*) AS attempts_24h,
       SUM(CASE WHEN outcome='failure' THEN 1 ELSE 0 END) AS failures_24h,
       COUNT(DISTINCT country_code) AS countries_24h
FROM mapped
WHERE event_time > :cutoff - INTERVAL '24 hours'
GROUP BY employee_id;

```

Database syntax varies; semantic tests should not. Define window inclusivity, null geography, time zones, and conflicting duplicates. Leaving those choices implicit destroys reproducibility.

Manifests and Deterministic Builds

Every published queue carries a manifest:

```

{
  "run_id": "ato-queue/2026-08-20T055500Z",
  "decision_cutoff": "2026-08-20T05:55:00Z",
  "code_revision": "8f31c2d",
  "dependency_lock_sha256": "21c8...",
  "contract": "auth_event.v3",
  "raw_snapshot": "auth/2026-08-20/snapshot-17",
  "identity_snapshot": "identity/2026-08-20/snapshot-04",
  "query_sha256": "a9de...",
  "parameters": {"window_hours": 24, "queue_size": 200},
  "row_count": 200,
  "logical_output_sha256": "55ae..."
}

```

Hash a canonical logical representation: stable columns, explicit nulls, normalized timestamps, defined numeric precision, and rows sorted by a total key. Parquet or gzip bytes can differ because of metadata while logical rows agree. Sorting only by score is insufficient when scores tie; use `(-risk_score, employee_id)`.

If audit sampling uses randomness, record generator algorithm, seed, population hash, and code version. “Seed 42” alone is not portable across implementations.

Tests That Defend the Claim

Contract tests reject missing fields, incompatible versions, invalid enums, duplicate keys, and impossible timestamps. Named examples test semantics, not merely JSON types.

Transformation tests use tiny fixtures. Include events exactly at cutoff and just after, a late event, a conflicting duplicate, a mapping recorded after cutoff, and null geography.

Metamorphic tests assert that input permutation changes nothing, adding post-cutoff evidence changes no historical output, duplicate replay does not increase counts, and renaming a display field does not affect scores.

Golden replay tests compare a fixed manifest's canonical output hash. Changes require human review. Automatically rewriting the golden hash turns a tripwire into decoration.

Tests must also assert failure: absent snapshot, unknown contract, ambiguous duplicate, or missing cutoff stops publication. A plausible report built from partial evidence is more dangerous than a visible failure.

Safe Lab: Replay a Tiny Pipeline

This standard-library program uses synthetic identities and in-memory SQLite. It performs no network or system action.

```
import hashlib, json, sqlite3
CUTOFF = "2026-08-20T05:55:00Z"
events = [
    ("e1", "alice", "2026-08-20T05:00:00Z", "2026-08-20T05:00:02Z", "failure"),
    ("e2", "alice", "2026-08-20T05:10:00Z", "2026-08-20T06:10:00Z", "failure"),
    ("e3", "bob", "2026-08-20T05:55:00Z", "2026-08-20T05:55:00Z", "success"),
    ("e4", "bob", "2026-08-20T05:56:00Z", "2026-08-20T05:54:00Z", "failure"),
    ("e5", "carol", "2026-08-20T04:00:00Z", "2026-08-20T04:00:01Z", "failure"),
]
def build(rows):
    db=sqlite3.connect(":memory:")
    db.execute("CREATE TABLE auth(event_id TEXT PRIMARY KEY, principal TEXT, event_time TEXT,
observed_at TEXT, outcome TEXT)")
    db.executemany("INSERT INTO auth VALUES(?,?,?,?,?)", rows)
    result=db.execute("""SELECT principal,COUNT(*),SUM(outcome='failure')
FROM auth WHERE event_time<=? AND observed_at<=?
GROUP BY principal ORDER BY 3 DESC,principal ASC""", (CUTOFF,CUTOFF)).fetchall()
    canonical=json.dumps(result,separators=(",",":"))
    return result,hashlib.sha256(canonical.encode()).hexdigest()[12:]
baseline,digest=build(events)
permuted,digest2=build(list(reversed(events)))
with_future,digest3=build(events+
[("e6", "alice", "2026-08-20T05:20:00Z", "2026-08-20T07:00:00Z", "failure")])
assert baseline==[("alice",1,1),("carol",1,1),("bob",1,0)]
assert baseline==permuted==with_future
assert digest==digest2==digest3
print(baseline); print("replay_hash",digest); print("point_in_time_tests PASS")
```

Expected output:

```
[('alice', 1, 1), ('carol', 1, 1), ('bob', 1, 0)]
replay_hash d3b5c1dfe930
point_in_time_tests PASS
```

`e4` was observed before cutoff but claims a future event time, so it is excluded. `e2` and `e6` occurred before cutoff but were not yet observed. Production code would quarantine excessive clock skew.

Production Failure Drill: The Helpful Backfill

At 09:00, the identity platform repairs three missing hours. The scheduled job overwrites the 06:00 report. The revision is statistically better but operationally false: analysts acted on the original queue, and their evidence has disappeared.

In staging, publish manifest `M1`; inject a late partition; rerun `M1` and verify its snapshot retains the original hash; publish immutable revision `M2` linked by `late-data-correction`; prove decision records retain `M1`; then reconcile decisions that would change.

The stop condition is a mutable snapshot, overwritten report, or decision without a manifest. Containment disables publication, not ingestion. Recovery restores snapshot immutability and replays both versions. The analytic owner declares output correctness; the platform owner repairs snapshot guarantees; the incident commander coordinates consumers.

Review, Promotion, and Retirement

A pull request shows the changed decision, contract compatibility, fixture additions, query plan, expected cost, temporal tests, privacy review, and a manifest from a clean build. Review SQL and Python together: a sound function fed by a leaky query is wrong.

Review the analytical diff, not only the source diff

A five-line SQL edit can reorder thousands of investigations. Code review therefore needs an *analytical diff*: run incumbent and candidate against the same frozen manifests and summarize added, removed, and re-ranked subjects. Partition disagreements by cause—new eligibility, feature change, score change, tie-breaking, missing evidence, or policy threshold. For each category, include counts, representative synthetic or authorized examples, and affected consequence segments.

Suppose a change replaces `COUNT(DISTINCT country_code)` with a mapping from IP to travel region. A source diff shows a join and a grouping change. The analytical diff should show that 18% of identities lose one “country,” that airport egress users move down the queue, that null geography increases in one source, and that the top-200 overlap is 82%. Reviewers can then decide whether the change fixes false diversity or conceals a coverage regression.

Compare outputs under at least three manifests: a normal day, a known late-data day, and an edge-case fixture. Report set overlap, rank correlation where meaningful, action transitions, group coverage, and output

invariants. A high rank correlation can still hide a severe subject crossing an action threshold; always inspect decision changes directly.

The query plan is also evidence. Capture scanned partitions, join strategy, estimated rows, runtime, and bytes processed under a representative snapshot. A semantically correct query that unpredictably scans an entire authentication lake is not ready for a scheduled production deadline. Place a cost ceiling in the run contract and fail before publication when the ceiling is exceeded, unless an authorized override names the reason and expiry.

Clean-room reconstruction

Before first production release, ask an engineer who did not write the analytic to reconstruct one report from the repository and manifest in a clean environment. Give that person read access only to the declared snapshots and artifact registry. They should not need the author's shell history, notebook state, personal configuration, or undocumented warehouse table.

The reconstruction record answers:

- Could dependencies be installed from the lock without contacting an unapproved package source?
- Did the manifest identify every input, including lookup tables and policy parameters?
- Did the environment use the expected timezone, locale, and SQL engine settings?
- Did the canonical rows and reason codes match?
- If physical files differed, was the difference limited to declared metadata?
- Could the reviewer trace one queue row back to eligible source events without privileged raw identifiers in the report?

Clean-room failure is useful. If the analytic relies on a hidden notebook cell or an analyst's temporary table, the organization has found the defect before an audit or incident. Record the fix as a repository change; do not solve the exercise by granting broad access or copying an unexplained table.

Data minimization and reproducibility

Reproducibility does not require retaining every raw field forever. It requires retaining enough authorized evidence to support the declared audit and replay claims. Classify fields as transformation inputs, trace references, sensitive investigation content, or irrelevant payload. Preserve immutable source identifiers and approved derived facts where possible; store direct identifiers in a separately controlled resolution service.

If policy requires deleting a field, manifests should record the deletion class and date. A later replay may then be *method-reproducible* but not *result-reproducible*. State that distinction. Privacy deletion is not a pipeline defect, and a hash of deleted content cannot reconstruct it. The design goal is an honest assurance boundary: who can reproduce which claim, for how long, under which authorization.

Scheduling and idempotency

The scheduler passes an explicit cutoff and manifest identifier; the pipeline must never infer “now” in transformation code. A retry with the same manifest writes the same logical output and does not create a second analyst task. Use a publication key derived from the run ID and output revision. Computation may be retried freely; external side effects require idempotency and a separate promotion step.

Separate `computed`, `validated`, and `published` states. A job can finish SQL successfully yet fail coverage validation. It must not become the current report. Publication should atomically move a reviewed pointer to an immutable artifact, leaving the previous good version available for rollback. Dashboards display cutoff, freshness, revision, coverage, and manifest link so users can recognize stale or corrected evidence.

The deployable unit is `(code, query, contract, parameters, dependencies, runbook)`. Roll it back together. Retire an analytic when its decision no longer exists, its contract cannot be supported, or a simpler reviewed rule produces equivalent decisions. Archive the final manifest and known limits so the organization does not rediscover the same dead end.

Sources and Further Reading

- The Turing Way: Reproducible Research distinguishes repeatability, reproducibility, and replicability.
- SLSA Specification supplies provenance concepts; here they are adapted to analytics without pretending provenance validates data truth.
- NIST SP 800-218, Secure Software Development Framework supports reviewed changes, protected artifacts, and tested releases.
- Great Expectations: Data Contracts provides implementation guidance for machine-checked expectations.
- DuckDB: Timestamp Issues illustrates why timezone and timestamp semantics must be explicit.

The synthesis is that software provenance is necessary but insufficient. Security analytics also needs *knowledge-time lineage* connecting mutable evidence to the exact state visible when a decision was made.

Interview Practicum

Scenario. A notebook identified compromised accounts with impressive precision. Leadership wants it scheduled Friday. Warehouse history is continuously corrected, the identity team cannot provide old mapping snapshots, and audits may occur a year later.

Prompt. Productionize the investigation without claiming reproducibility the platform cannot provide.

A strong candidate asks: What action is authorized? What cutoff and latency apply? Can raw partitions or change logs be retained? How are mapping corrections recorded? What late-data policy is acceptable? Which object must match—bytes, logical rows, ranking, or conclusion? Who signs off on incomplete evidence?

Then the interviewer adds: raw retention is seven days, and legal policy prohibits retaining raw IP addresses in analytical artifacts.

Worked Answer

I would narrow the first release to an analyst-review queue with no automated disable action. Eligibility uses event and observation time. Because warehouse history mutates, I would create immutable input snapshots or change-log-backed identifiers before calling any rerun reproducible.

The repository separates reviewed SQL, Python, contracts, synthetic fixtures, manifests, and generated reports. Each row links to reason codes and tokenized evidence. The manifest records code, lockfile, query hash, cutoff, raw and identity snapshots, parameters, contract, count, and canonical output hash.

Missing historical identity snapshots block claims about past entity features. For Friday, use stable principal keys or require append-only mapping changes with validity and recording times. Document that earlier periods cannot be faithfully replayed.

For seven-day retention, preserve a privacy-reviewed evidence capsule: tokenized principal, approved coarse geography, feature inputs, source hashes, contract, and snapshot attestations. Do not retain raw IPs. Whether that supports a one-year audit is an audit and legal decision; escalate the conflict rather than invent a guarantee.

Tests cover cutoff equality, late events, future timestamps, duplicate conflicts, mapping corrections, nulls, permutation, clean-environment replay, and missing provenance. Shadow the notebook and reconcile disagreements. Stop publication on an unknown contract, missing snapshot, or non-canonical output.

Follow-Up Questions and Answers

Why not pin the notebook environment? It preserves execution, not historical evidence or identity semantics.

Must bytes match? No. Hash a canonical logical form while tracking physical artifacts separately.

How do you test billion-row SQL locally? Exact fixtures and metamorphic tests locally; bounded integration, scale, and query-plan tests in an isolated warehouse.

What if a correction is more accurate? Publish a linked revision; never erase the version used for action.

What makes this Principal-level? It treats retention, identity history, audit claims, and authority as cross-system contracts and limits launch rather than offering false assurance.

Key Takeaways

- Reproducibility reconstructs the evidence state at decision time, not merely the code.
- Point-in-time correctness requires event time, observation time, and historical dimension semantics.
- A manifest binds code, dependencies, contracts, snapshots, parameters, and canonical output.
- Boundary, negative, metamorphic, and golden-replay tests protect different failure surfaces.
- Corrections create immutable revisions; they never overwrite evidence of earlier decisions.

Chapter 42: MLOps and Observability for Security Models

A security model can be statistically excellent and operationally unusable. An identity-risk service that takes 900 milliseconds to answer cannot sit in a 300-millisecond login path. A detector that costs more per day than the losses it plausibly avoids should not be defended with AUC. A calibrated score that silently becomes overconfident after a telemetry outage is worse than a transparent rule because downstream policy assigns it more authority.

Security MLOps is the discipline of operating a changing estimate inside a changing decision system. The production objective is not “serve the model.” It is to meet a decision deadline and loss budget while preserving calibration, coverage, auditability, and a safe degraded mode.

Start with the Decision SLO

Consider an account-takeover model used in three places:

Consumer	Deadline	Volume	Permitted action	Failure preference
interactive login	150 ms model budget	4,000/s peak	challenge, never disable	fail to rules or allow with audit
analyst queue	30 min	10 million/day	prioritize review	deliver partial queue with coverage flag
weekly control review	12 h	one batch/week	change policy proposal	fail closed; no stale report

One model artifact does not imply one serving architecture. The online path needs bounded features, aggressive timeouts, and a local fallback. The batch path can compute expensive graph and historical features. The review path should wait for mature outcomes and recompute calibration.

Define a budget before optimization:

$T_{\text{decision}} = T_{\text{gateway}} + T_{\text{features}} + T_{\text{inference}} + T_{\text{policy}} + T_{\text{network}}$, \]

and require a percentile, not an average: $p99(T_{\text{decision}}) \leq 250 \text{ ms}$. For daily economics,

$C_{\text{day}} = N_o(c_f + c_i + c_l) + N_b c_b + C_{\text{storage}} + C_{\text{oncall}}$. \]

The terms cover online feature lookup, inference, logging, batch scoring, storage, and operational labor. Labor is often omitted precisely because it is large.

```

flowchart TB
    E[Authentication event] --> G[Decision gateway]
    G --> H{Online path healthy?}
    H -->|yes| F[(Online feature cache)]
  
```

```

F --> S[Low-latency scorer]
S --> P[Versioned policy]
H -->|no/timeout| R[Local rules + bounded default]
R --> P
P --> A[Allow / challenge / review]
E --> Q[(Durable event stream)]
Q --> B[Batch feature builder]
B --> W[(Offline feature tables)]
W --> X[Training and backtest]
X --> V[Registry compatibility set]
V --> S
A --> D[(Decision log)]
D --> O[Delayed outcomes]
O --> M[Calibration, drift, value monitoring]
M -. rollback or restrict .-> V

```

The gateway, scorer, and policy are separate. A model score is evidence; policy decides what evidence authorizes. This permits a threshold rollback without retraining and a model rollback without replaying side effects.

Online and Batch Feature Architecture

Features are a common source of leakage and training-serving skew. Maintain one versioned feature definition but allow different materializations. For `failed_logins_15m`, specify source events, event-time window, watermark, null behavior, entity key, transformation version, and maximum staleness. The online cache might update every ten seconds; the offline table may recompute corrected history. Their values can legitimately differ, so evaluation should emulate the online watermark rather than train on perfected history.

Classify online features:

- **required-safe:** absence makes the model abstain, because policy cannot interpret the score safely;
- **optional-bounded:** use a documented default and set a missingness indicator;
- **batch-only:** excluded from interactive inference and used for queues or investigations;
- **prohibited:** too sensitive, mutable, expensive, or leakage-prone for the decision.

Do not turn a feature store into an authorization oracle. A cached `is_admin=true` value cannot substitute for the identity system at action time. Likewise, do not fetch arbitrary graph neighborhoods synchronously in a login path; precompute bounded summaries or route the richer analysis to batch.

The deployable compatibility set is (model, feature definitions, calibrator, policy, schema, runtime, runbook). The registry should refuse promotion if the online store lacks a required feature version or the policy expects reason codes the model cannot emit.

Calibration Is an Operational Interface

If a score of 0.20 drives one action and 0.80 another, the numerical probability must mean something. For predictions $\{p_i\}$ and outcomes $\{y_i\}$, the Brier score is

$$BS = \frac{1}{n} \sum_i (p_i - y_i)^2$$

Expected calibration error (ECE) summarizes gaps across bins, but it depends on binning and can hide small high-risk segments. Report a reliability diagram, Brier score, calibration intercept and slope when suitable, counts, and segment-specific results. Outcomes should be grouped by prediction vintage so immature “negatives” do not make a new release appear safer.

Fit Platt scaling or isotonic regression on a calibration period separate from training and final evaluation. Recalibration can repair probability mapping; it cannot repair lost discrimination, missing coverage, or a policy that creates biased labels. If only challenged logins receive investigation, observed outcomes are selected by the policy. Maintain a small authorized audit sample outside the normal threshold, or state that calibration below threshold is unidentified.

Drift: Name the Broken Relationship

“Drift detected” is not a diagnosis. Distinguish:

- **input drift:** $P(X)$ changed, perhaps due to a new VPN or sensor;
- **concept drift:** $P(Y|X)$ changed as attackers or users adapt;
- **prior shift:** compromise prevalence changed;
- **coverage drift:** an event source or entity segment disappeared;
- **label drift:** investigation policy, delay, or definition changed;
- **policy drift:** identical scores now cause different actions;
- **cost drift:** traffic or feature computation makes the service uneconomic.

Population Stability Index is convenient but arbitrary thresholds are not universal evidence. A feature distribution can move with no decision impact, while a rare segment can fail catastrophically without moving a global histogram. Monitor invariants first: event volume by source, nulls, freshness, entity resolution, feature availability, prediction coverage, and score/action distributions. Then monitor mature discrimination and calibration by time and consequence-bearing segment.

A useful alert must name an owner and response. “Calibration slope below 0.7 for two mature weekly cohorts with at least 500 labeled cases” is actionable. “Feature drift > 0.2” is not, unless linked to a validated failure mode.

Degradation Is a Designed Product State

Assume the online feature cache will fail. Predefine a degradation ladder:

1. full model with fresh required features;

2. reduced model using a reviewed subset;
3. deterministic local rules;
4. allow plus high-fidelity audit for low-consequence cases;
5. challenge or deny only where policy and business continuity permit;
6. stop the dependent workflow if no bounded action exists.

Each transition has an entry signal, maximum duration, user impact, security assumption, owner, and exit test. Never silently substitute zeros: zero failed logins and unavailable failed-login telemetry are different evidence states.

Load shedding should preserve the most consequential decisions, not merely the easiest requests. Reserve capacity by risk-independent attributes such as protected workflow class; otherwise using a model score to decide which requests receive a model score creates unstable selection.

Queue behavior matters even when inference is fast. If arrivals approach service capacity, tail latency grows nonlinearly and timeout traffic can cascade into the fallback. Measure utilization, queue age, deadline misses, retry amplification, and fallback saturation together. Apply admission control before workers exhaust memory, make clients respect a single deadline, and avoid retries whose remaining budget cannot complete. A cheaper model with predictable service time may protect more accounts than a stronger model whose latency variance causes correlated fallback during traffic spikes.

Capacity tests should reproduce regional peak shape, not only steady average throughput. Include cold starts, cache misses, feature-store throttling, and a downstream policy slowdown. Validate that reserved critical-workflow capacity remains available and that low-priority batch scoring yields to interactive decisions. The result belongs in the release evidence: maximum sustainable rate, p99 at that rate, degradation entry point, recovery hysteresis, and daily cost at normal and incident load.

Safe Lab: Choose an Architecture Under Latency and Cost

The lab simulates three candidate serving plans. It calculates end-to-end p99 latency, daily compute cost, and a simple expected daily loss. Inputs are synthetic; no model or security system is contacted.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Plan:
    name: str
    feature_ms: int
    inference_ms: int
    policy_ms: int
    cost_per_million: float
    miss_rate: float
    challenge_rate: float

REQUESTS = 20_000_000
```

```

COMPROMISES = 120
LOSS_PER_MISS = 18_000
FRICTION_PER_CHALLENGE = 0.35
LATENCY_BUDGET = 150
plans = [
    Plan("remote_graph",105,48,8,92,0.10,0.030),
    Plan("cached_linear",22,9,7,18,0.16,0.025),
    Plan("local_rules",3,1,4,4,0.28,0.018),
]

def evaluate(p):
    latency=p.feature_ms+p.inference_ms+p.policy_ms
    compute=REQUESTS/1_000_000*p.cost_per_million
    security=COMPROMISES*p.miss_rate*LOSS_PER_MISS
    friction=REQUESTS*p.challenge_rate*FRICTION_PER_CHALLENGE
    return latency,compute,security+friction+compute

eligible=[]
for p in plans:
    latency, cost, loss=evaluate(p)
    status="PASS" if latency<=LATENCY_BUDGET else "FAIL_LATENCY"
    print(p.name,latency,round(cost,2),round(loss,2),status)
    if status=="PASS": eligible.append((loss,p.name))
print("selected",min(eligible)[1])
assert min(eligible)[1]=="cached_linear"

```

Expected output:

```

remote_graph 161 1840.0 427840.0 FAIL_LATENCY
cached_linear 38 360.0 520960.0 PASS
local_rules 8 80.0 730880.0 PASS
selected cached_linear

```

The remote graph model has lower miss rate but violates the deadline. The simple rules are cheap and fast but lose more expected value. The exercise is not a universal dollar model: sensitivity analysis should vary compromise prevalence, loss, and friction. Its purpose is to force latency and cost into the same selection conversation as accuracy.

Extend the lab by giving `cached_linear` a 1% timeout rate that falls back to `local_rules`; report effective miss rate and timeout-segment calibration separately. An average metric over both paths would hide whether degradation is safe.

Production Failure Drill: Fresh Scores from Stale Features

The feature service continues answering during a stream-processing outage, but `last_successful_login` is six hours stale. Latency and HTTP availability dashboards remain green. Score distributions barely move because the model treats missing recency as ordinary behavior.

Inject this condition during a canary. The scorer must receive feature timestamps and compute an evidence-age vector. Required stale features trigger the reduced model or rules; decision logs record `degraded_reason=feature_freshness`; an alert measures affected traffic by workflow and region.

The incident has four clocks: detection time, time to restrict authority, time to restore features, and time to validate recovery. Containment changes policy before retraining. Preserve requests, feature timestamps, model responses, and actions. After recovery, replay the affected interval with corrected features, estimate changed decisions, and inspect outcomes without retroactively rewriting original decision records.

The feature-platform owner restores materialization, the model-service owner operates fallback, the security policy owner decides permitted actions, and the incident commander owns communications. Exit requires fresh features, replay agreement within a stated tolerance, stable latency under load, and explicit policy restoration.

Release and Rollback

Promotion proceeds through offline temporal evaluation, shadow traffic, paired comparison where safe, a low-authority canary, and progressive traffic. Compare candidate and incumbent on identical requests when possible. Gate severe misses, calibration, coverage, p99 latency, timeout rate, cost per million, and downstream workload—not just AUC.

Rollback triggers should be executable: any confirmed severe policy violation; required-feature coverage below 99.5% for five minutes; p99 above budget for ten minutes; or a mature calibration bound outside the policy-approved range. A registry pointer rollback is insufficient if schema, calibrator, or policy changed. Restore the compatibility set and verify it with a fixed replay pack.

Observability should answer one question from request to outcome: which evidence, model, calibrator, policy, degraded mode, and action produced this result? Sampled traces may serve performance debugging, but consequential decisions require durable audit records with privacy-aware retention.

Sources and Further Reading

- Google, [Rules of Machine Learning](#) emphasizes pipelines, simple baselines, and training-serving consistency.
- Google, [Site Reliability Engineering](#) provides SLO, error-budget, overload, and incident principles adapted here to model-backed decisions.
- NIST AI RMF 1.0 frames ongoing measurement, management, transparency, and accountability.
- NIST AI 600-1 adds generative-AI risk practices relevant when security models contain foundation-model components.
- Guo et al., [On Calibration of Modern Neural Networks](#) demonstrates modern model miscalibration and temperature scaling.
- [OpenTelemetry Specification](#) defines interoperable traces, metrics, and logs, though decision audit records require additional domain fields.

The combined lesson is that MLOps must join statistical validity to reliability engineering. Neither model monitoring without system SLOs nor system monitoring without outcome calibration is sufficient.

Interview Practicum

Scenario. A graph model reduces missed compromised logins by 30% offline. In production testing it adds 180 ms p99 latency, costs \$3.2 million annually, and depends on a cross-region feature service. The login team allows 80 ms and refuses a fail-closed dependency.

Prompt. Design a production system and decide whether to ship the model.

Strong clarifying questions cover the permitted action, traffic and regional peaks, consequence segments, current baseline, feature freshness, label delay, cost of challenge and miss, acceptable degradation, paired shadowing, and who owns residual risk.

Then add two constraints: 20% of traffic comes from a region where graph features are unavailable, and labels mature after 21 days while leadership wants a decision in one week.

Worked Answer

I would not put the graph lookup synchronously in the login critical path. I would decompose its value. Precompute bounded graph summaries for the online cache and test a smaller online model against the existing rules. Run the full graph model asynchronously for analyst prioritization and high-risk session review, where its latency is acceptable.

The online SLO allocates explicit budgets to features, inference, policy, and network, with a local rules fallback. Every score carries feature freshness, model, calibrator, and policy versions. The unavailable region uses a separately validated reduced model or rules; I would not claim global performance by averaging it with covered traffic.

In the first week, paired shadow results can establish latency, cost, coverage, score disagreement, and projected challenge workload, but not mature security effectiveness. I would present that limitation and require a 21-day outcome gate before increasing authority. If leadership must act, the reversible step is a low-authority canary that recommends challenges but cannot disable accounts.

Selection uses expected loss under sensitivity ranges plus hard constraints. The full graph model is rejected for synchronous serving even if its mean loss is lower because it violates the deadline and availability contract. Ship only if the cached candidate beats rules after accounting for fallbacks, regional coverage, calibration, user friction, and cost. Otherwise retain graph scoring in batch while improving distillation or feature precomputation.

Rollback restores model, features, calibrator, schema, and policy together. Triggers include severe misses, freshness loss, regional coverage loss, p99 breach, timeout amplification, or unexpectedly high challenge burden.

Follow-Up Questions and Answers

Why not increase the login latency budget? That is a product decision with conversion and reliability costs. Present the value curve, but do not assume security owns the budget.

Can drift be monitored without labels? Input and coverage drift can. Concept drift and calibration cannot be established from unlabeled scores alone.

How do you calibrate policy-selected labels? Preserve randomized or risk-stratified audit samples where authorized, model selection explicitly, or limit the claim to the observed policy region.

What if the fallback is less accurate? Measure the fallback as a separate operating mode, cap its duration, restrict authority, and include it in release evaluation.

What is the Principal-level insight? Architecture is part of the estimator. Timeouts, missing regions, and degradation alter who receives which decision, so they belong in statistical evaluation and ownership—not only infrastructure dashboards.

Key Takeaways

- Begin with decision latency, reliability, authority, and cost budgets—not a model endpoint.
- Online and batch paths can share definitions while using different materializations and actions.
- Calibration is the interface between scores and policy; monitor it by mature vintage and segment.
- Name input, concept, prior, coverage, label, policy, and cost drift separately.
- Degradation requires reviewed models, explicit entry and exit conditions, and durable decision logging.

Chapter 43: Building a Security Data-Science Team

A security data-science team is not defined by reporting lines. It is defined by the decisions it improves, the evidence it can defend, and the incidents it is prepared to own. A centralized team can become a ticket queue; embedded scientists can duplicate foundations and accept local incentives; a “federated” model can become a polite name for unclear accountability. The remedy is a concrete operating system: charter, interfaces, decision rights, portfolio allocation, quality gates, competencies, and incident ownership.

This chapter designs the first 14-person security data-science function for a global software company with identity, product security, cloud defense, abuse, and AI-agent security programs. The goal is not an ideal organization chart. It is an operating model that works across time zones, regulated data, scarce labels, and competing security claims.

Charter and Boundaries

The charter should name outcomes and exclusions:

The Security Data Science function builds and evaluates evidence systems that prioritize security action, measure control effectiveness, and operate approved predictive services. It owns analytical validity and model operations for registered products. It does not unilaterally accept security risk, authorize investigations, set employment policy, or replace source-system owners.

This prevents two common failures. First, scientists do not become unaccountable policy makers because a score appears quantitative. Second, partner teams cannot outsource all data quality and production reliability to “the model team.”

The initial organization uses a **hub-and-spoke** model. A central Evidence Platform group owns contracts, reusable temporal features, evaluation tooling, model registry, privacy patterns, and quality review. Three domain pods—Identity and Abuse, Product and Cloud Security, and Agent Security—own decision products with security partners. Scientists have a functional home in the discipline and a delivery commitment to domains.

```

flowchart TB
    CIS0[CISO / risk acceptance] --> SC[Security portfolio council]
    CD0[Data and privacy leadership] --> SC
    SC --> H[Head of Security Data Science]
    H --> EP[Evidence Platform: 4]
    H --> IA[Identity & Abuse pod: 3]
    H --> PC[Product & Cloud pod: 3]
    H --> AS[Agent Security pod: 2]
    H --> PM[Technical program/product: 1]
    H --> EM[Engineering manager: 1]
    EP -. contracts, review, tooling .-> IA
    EP -. contracts, review, tooling .-> PC
    EP -. contracts, review, tooling .-> AS
  
```

```

IA --> 01[Identity security owner]
PC --> 02[Product/cloud owners]
AS --> 03[AI security owner]
01 -. outcome and policy .-> SC
02 -. outcome and policy .-> SC
03 -. outcome and policy .-> SC

```

Do not allocate one scientist per stakeholder. That makes each person a bespoke dashboard operator and removes peer review. Pods need at least two technical contributors for continuity, while the hub supplies specialist review.

Decision Rights and RACI

RACI is useful only when tied to artifacts and decisions. “Security is accountable” is too broad. The following matrix separates a model from the policy that consumes it.

Activity	Domain security owner	SDS pod	Evidence platform	Data producer	Privacy/legal	SRE	Risk executive
define decision and permitted action	A/R	C	I	I	C	I	C
approve data contract	C	R	A	R	C	I	I
approve evaluation design	C	R	A	I	C	I	I
train and document model	C	A/R	C	I	I	C	I
set policy threshold	A/R	C	I	I	C	C	C
production reliability	C	R	C	I	I	A/R	I
accept residual risk	C	C	I	I	C	I	A/R
declare model incident	C	R	C	I	I	A/R	I

A must be singular for a particular decision. The data producer is responsible for delivery and semantics but is not accountable for whether a model is fit for a security action. The scientist estimates consequences and advises thresholds; the security owner owns the action policy. The executive accepts risk that exceeds delegated bounds.

Each registered analytical product has an ownership card: decision, users, action authority, service tier, data steward, technical owner, policy owner, outcome owner, on-call rotation, privacy classification, review date, and retirement conditions.

Portfolio Optimization Under Constraints

Backlogs overvalue articulate sponsors and undervalue foundation work. Use a transparent portfolio model, then challenge it rather than worship it. For initiative j , estimate value across plausible scenarios:

$$V_j = P_j(\text{impact}_j \times \text{adoption}_j) - \text{delivery}_j - \text{operating}_j - \text{risk}_j.$$

Choose initiatives with binary or fractional allocation x_j subject to capacity, specialist, regional, and foundation constraints:

$$\max \sum_j x_j V_j, \quad \sum_j x_j c_{jk} \leq C_k, \quad \sum_{j \in \text{foundations}} x_j c_j \geq 0.2C.$$

The 20% foundation floor is a governance choice, not a mathematical truth. It keeps urgent stakeholder work from consuming contracts, labeling, reliability, and evaluation capacity needed by every product.

Run a quarterly portfolio council with monthly exception review. Every proposal includes decision owner, affected population, current baseline, expected mechanism, evidence readiness, privacy classification, time to first falsifiable result, operating burden, and kill criteria. Allocate in four buckets: operate/repair existing products, foundations, committed decision products, and exploratory bets. A reasonable starting split is 35/20/35/10, adjusted when incidents or strategy demand it.

Avoid a single fabricated score when estimates are weak. Show low/base/high value, dependency confidence, and opportunity cost. Some work is mandatory because of regulation or critical risk; label it as a constraint instead of manipulating its score until it wins.

Safe Lab: Select a Bounded Quarterly Portfolio

This standard-library lab enumerates projects under an 18-person-quarter capacity, requires at least one foundation project, and reserves an incident-reliability project. Values are illustrative and contain no company data.

```
from itertools import combinations

projects = [
    ("identity_model", 7, 15, "product"),
    ("telemetry_contracts", 5, 11, "foundation"),
    ("agent_eval", 6, 13, "product"),
    ("label_audit", 4, 10, "foundation"),
    ("model_oncall", 3, 8, "reliability"),
    ("exec_dashboard", 4, 4, "reporting"),
]
best=None
for size in range(len(projects)+1):
    for chosen in combinations(projects,size):
        cost=sum(p[1] for p in chosen)
        value=sum(p[2] for p in chosen)
        kinds={p[3] for p in chosen}
        if cost<=18 and "foundation" in kinds and "reliability" in kinds:
```

```

        candidate=(value,-cost,tuple(p[0] for p in chosen))
        if best is None or candidate>best: best=candidate
    value,neg_cost,names=best
    print("selected",names)
    print("capacity_used",-neg_cost,"of",18)
    print("portfolio_value",value)
    assert names==("telemetry_contracts","agent_eval","label_audit","model_oncall")

```

Expected output:

```

selected ('telemetry_contracts', 'agent_eval', 'label_audit', 'model_oncall')
capacity_used 18 of 18
portfolio_value 42

```

The optimizer funds both foundations, the agent evaluation, and on-call work while rejecting the attractive identity model because a different feasible bundle creates more stated value. That is a prompt to inspect assumptions, not permission to outsource judgment to code. Split work only if a smaller increment has independent value and a clear interface; fractional staffing without a deliverable often hides overload. Extend the lab with a scarce privacy-review slot or low/base/high values and observe how the portfolio changes.

Quality Gates, Not Ceremonial Review

Use gates proportional to authority.

G0 — Intake. A named decision and owner exist. The team can state who acts, by when, and what current baseline will be displaced. Dashboard curiosity alone does not enter the committed portfolio.

G1 — Evidence readiness. Data contracts, lawful purpose, access, provenance, time semantics, label process, blind spots, and retention are reviewed. Exit artifact: evidence-readiness report with blockers.

G2 — Evaluation design. The estimand or prediction target, temporal split, independent unit, baselines, capacity, uncertainty, subgroup analysis, and failure criteria are fixed before final results. Exit artifact: signed evaluation plan.

G3 — Limited release. Reproducible manifest, threat model, model/data cards, runbook, SLOs, rollback, privacy controls, and shadow/canary evidence exist. High-impact actions require independent review.

G4 — Authority expansion. Mature outcomes show acceptable calibration, value, coverage, burden, and reliability. The policy owner—not the model author—approves greater action authority.

G5 — Recertify or retire. On a risk-based cadence, confirm decision need, evidence quality, performance, incidents, cost, and ownership. Missing owners force restriction or retirement.

The platform group operates the gate process, but domain experts review domain assumptions. Appeals go to a small technical-risk panel, preventing one central reviewer from becoming an unchallengeable bottleneck. Emergency releases use an explicit exception with expiry, compensating controls, and retrospective review.

Competency Ladder

Promotions should reward scope and judgment, not model novelty.

Level	Evidence and methods	Production and security	Organizational leverage
DS I	executes reviewed analysis; writes exact tests	follows handling and deployment controls	communicates assumptions to immediate team
DS II	designs temporal evaluation; diagnoses label defects	owns a low-tier pipeline and runbook	leads a bounded partner project
Senior	chooses estimands/models; quantifies uncertainty	owns a service and incident follow-through	resolves cross-functional tradeoffs; mentors
Staff	defines multi-product evidence architecture	designs degradation and governance patterns	changes portfolio decisions across domains
Principal	establishes discipline-wide technical strategy	owns systemic risk standards and crisis advice	aligns executives on ambiguous evidence and builds successors

An individual can advance through strong applied statistics, ML engineering, security research, analytics engineering, or decision science, but all paths require minimum competence in security ethics, reproducibility, and communication. Managers use a parallel ladder. Do not force the strongest technical leaders into people management.

Evidence for promotion includes design reviews, incident decisions, retired bad ideas, partner outcomes, reusable systems, and development of others. Counting shipped models encourages unsafe proliferation. Counting prevented incidents is usually unverifiable and easily gamed.

Measuring the Team

Use a metric portfolio:

- decision products with named owners and active users;
- percentage passing recertification and reproducible replay;
- time from intake to first falsifiable evidence;
- mature outcome lift or avoided workload versus baseline;
- coverage, calibration, user burden, cost, and SLO performance;
- quality-gate escape defects and incident recurrence;
- foundation adoption and duplicated work retired;
- staffing health: on-call load, review concentration, regretted attrition, and competency growth.

Metrics are diagnostic, not compensation formulas. A team can improve time-to-evidence by choosing trivial work, or improve model count by never retiring anything. Quarterly narrative review should identify tradeoffs and displaced burden.

Incident Ownership Across Time Zones

Classify incidents as data, model, policy, service, privacy, or action incidents; several can coexist. The on-call engineer can contain service behavior, but risk acceptance remains with the delegated security owner. A follow-the-sun rotation needs explicit handoff: current impact, evidence confidence, containment, next decision deadline, owners, and preserved artifacts.

Every tier-1 product has a primary and secondary region, a policy contact, SRE partner, and executive escalation path. Scientists are not permanently on call for ad hoc analyses. Only registered services with runbooks enter the rotation, and their operating cost is charged to portfolio capacity.

Production Failure Drill: Global Label Corruption

A new case-management workflow changes “closed-benign” to include cases auto-closed for missing analyst capacity. Precision dashboards improve in every region. Two pods consume the label; a third team maintains the source.

The drill begins with a semantic-contract alert or analyst report. The incident commander declares a data-and-model incident. Containment freezes retraining and threshold expansion, tags affected decisions, and falls back to the last validated model where safe. The producer explains the workflow change; SDS estimates affected cohorts; domain owners review actions; privacy ensures investigation sampling is permitted.

Do not immediately relabel auto-closed cases as malicious. They are unknown. Create a corrected taxonomy, backfill what evidence supports, preserve original labels, and sample unresolved cases. Recovery requires contract versioning, consumer tests, recalculated metrics, and review of decisions made during the contaminated interval. The post-incident action is not “communicate better”; it assigns a semantic owner, change-notification control, and consumer compatibility test.

Sources and Further Reading

- [NIST NICE Workforce Framework for Cybersecurity, SP 800-181 Rev. 1](#) provides a common language for cybersecurity work roles and competencies.
- [NIST Cybersecurity Framework 2.0](#) emphasizes governance, roles, responsibilities, and risk communication.
- [Team Topologies](#) offers interaction modes and platform/team boundaries; this chapter adapts them to evidence products and security decision rights.
- [Google, Site Reliability Engineering](#) supports service ownership, on-call, SLOs, and learning from incidents.
- [NIST AI RMF 1.0](#) supports mapped accountability and lifecycle governance for AI systems.
- [Forsgren, Humble, and Kim, Accelerate](#) provides empirical grounding for delivery performance and organizational capabilities, while not being security-data-science-specific.

The synthesis is that a team topology becomes real only when coupled to decision artifacts, quality gates, and incidents. Reporting lines explain where people sit; ownership cards and runbooks explain how risk is controlled.

Interview Practicum

Scenario. A global company funds 14 people for its first security data-science team. Five security VPs each demand embedded support. The data platform is centralized, Europe restricts some cross-region data use, and two existing models have no on-call owner.

Prompt. Propose the first-year operating model, staffing, portfolio, and success measures.

Strong candidates ask about business and threat priorities, current services, data residency, incident tiers, partner engineering capacity, executive risk authority, hiring mix, label operations, and which commitments already exist.

Then add: the CISO requires a customer-facing AI-security scorecard in 60 days, while an identity model is producing unexplained lockouts in Asia-Pacific.

Worked Answer

My first action is to assign interim ownership and restrict authority for the unexplained identity model; an organization design cannot wait while customers are harmed. I would form a small incident group with the identity owner, SRE, regional privacy, and model engineer, preserve decisions and features, and define restoration criteria.

For the durable structure, I would use a four-person Evidence Platform, three-person Identity and Abuse pod, three-person Product and Cloud pod, two-person Agent Security pod, one engineering manager, and one technical program/product lead. The head of function is counted within senior technical leadership or added explicitly if the 14 excludes leadership. Pods remain multi-person; no VP receives a solitary embedded scientist.

The first quarter portfolio reserves capacity for operating and repairing existing models, contracts and temporal evaluation, and two decision products. The requested scorecard receives a time-boxed discovery: define customer decisions, evidence, uncertainty, and safe claims. If it is merely a red/green vanity dashboard, I would deliver a transparent evidence inventory and roadmap rather than fabricate a security score in 60 days.

RACI separates model validity, policy threshold, reliability, privacy, and residual-risk acceptance. All registered products receive ownership cards and tiered on-call. Quality gates scale with authority. Europe may require regional processing and aggregated, privacy-reviewed evidence; the architecture and staffing plan must budget for that rather than treating residency as paperwork.

Success is not models shipped. I would report decision adoption, reproducible recertification, outcome or workload improvement versus baseline, calibration and coverage, SLOs, incident recurrence, foundation reuse, time to falsifiable evidence, and team health. At six months, I would revisit whether a pod has enough durable demand to embed more deeply.

Follow-Up Questions and Answers

Why centralize the platform? Contracts, temporal semantics, registries, and evaluation tooling have economies of scale. Domain decisions remain with pods and security owners.

Who sets a lockout threshold? The identity security policy owner is accountable; SDS supplies calibrated estimates and sensitivity; product and risk owners approve impacts within delegated authority.

How do you stop gates becoming bureaucracy? Scale them to consequence, publish turnaround SLOs, automate mechanical checks, allow expiring exceptions, and measure escaped defects as well as review delay.

What gets cut first when incidents consume capacity? Replan transparently: protect incident response and critical operations, pause exploratory bets, and show sponsors the displaced commitments.

What distinguishes Principal reasoning? It treats organizational interfaces as safety controls, recognizes that unfunded operations invalidate the roadmap, and is willing to narrow an executive deliverable rather than manufacture certainty.

Key Takeaways

- A team charter defines decisions, evidence responsibilities, authority limits, and exclusions.
- Hub-and-spoke structures work only with explicit interfaces and multi-person domain continuity.
- RACI must attach to concrete artifacts and separate model, policy, service, and risk ownership.
- Portfolio optimization needs capacity constraints, foundation investment, uncertainty, and kill criteria.
- Quality gates, a competency ladder, and incident rotations make the operating model testable.

Chapter 44: Communicating Evidence for Decisions

The most consequential security memo is often written when the evidence does not resolve the question. A control evaluation may show fewer incidents but have wide intervals, changed telemetry, low adoption, and plausible attacker displacement. “No statistically significant effect” does not mean “no effect.” A confident recommendation can exceed the evidence; an evasive summary can leave leaders unaware that delay is itself a choice.

This chapter builds a decision-memo system for such cases. The running example is a step-up authentication control evaluated over eight weeks. The estimate suggests benefit, but the study is underpowered for rare account takeover and clearly shows user friction. The system must preserve facts, estimates, assumptions, forecasts, and recommendations as different objects; visualize uncertainty without hiding consequences; track whether forecasters are calibrated; and stage authority as evidence improves.

The Memo Is a Versioned Decision Interface

A durable memo is not a polished narrative detached from evidence. It is a compact interface between analysis and accountable choice. Use this structure:

1. **Decision and deadline.** Who decides what, by when, and what happens if no choice is made?
2. **Recommendation stage.** Observe, instrument, pilot, expand, hold, restrict, roll back, or retire.
3. **One-sentence rationale.** State the benefit signal, uncertainty, and principal cost.
4. **Options.** Include status quo and at least one reversible alternative.
5. **Evidence ledger.** Separate observed facts, estimates, assumptions, and external evidence.
6. **Uncertainty and sensitivity.** Show intervals, denominators, missingness, and which assumptions change the choice.
7. **Forecasts.** Record probabilities for decision-relevant future events with resolution criteria.
8. **Guardrails and ownership.** Define stop conditions, monitoring, and accountable owners.
9. **Next evidence.** Name the experiment or instrumentation that has decision value.
10. **Decision record.** Capture choice, approver, date, dissent, and review trigger.

```
stateDiagram-v2
    [*] --> Question
    Question --> EvidenceLedger: define decision and alternatives
    EvidenceLedger --> Forecast: facts / estimates / assumptions
    Forecast --> StageChoice: calibrated probabilities + sensitivity
    StageChoice --> Pilot: reversible evidence supports learning
    StageChoice --> Hold: downside or uncertainty too high
    StageChoice --> Expand: benefit robust and guardrails pass
    Pilot --> Expand: mature outcomes pass gates
```

```

Pilot --> Restrict: guardrail breach
Pilot --> Hold: value of more evidence exceeds rollout value
Expand --> Review: scheduled or trigger-based
Hold --> EvidenceLedger: new evidence
Restrict --> EvidenceLedger: incident analysis
Review --> [*]: retire or renew decision

```

The diagram makes “pilot” a decision state, not a weak compromise. A staged recommendation specifies authority, population, duration, evidence to collect, and precommitted transitions.

Evidence Ledger: Stop Mixing Epistemic Categories

Consider the sentence: “The control reduced account takeover by 30% and should be launched globally.” It combines an estimate, a causal claim, and a recommendation. Replace it with a ledger:

ID	Type	Statement	Support	Limitation	Owner
F1	observed fact	42,118 eligible logins entered treatment regions	experiment assignment log	1.8% missing outcome linkage	data owner
E1	estimate	risk ratio for confirmed takeover is 0.70	clustered model, 95% CI 0.31–1.58	rare events; interval spans harm and benefit	analyst
E2	estimate	challenge abandonment increased 0.42 pp	randomized rollout, 95% CI 0.29–0.55	first-week novelty may differ	analyst
A1	assumption	cross-region spillover is negligible	architecture review	attacker displacement not directly measured	security owner
X1	external evidence	step-up authentication can reduce credential abuse	standards and prior studies	different population and implementation	research owner
R1	recommendation	run a six-week constrained expansion	E1, E2, capacity analysis	not a claim of proven global benefit	decision owner

Facts can still be wrong, estimates depend on methods, assumptions can be challenged, and recommendations depend on values. The categories do not assign prestige; they expose what kind of disagreement is occurring.

Negative evidence belongs in the ledger. Record instrumentation gaps, outcome definitions that changed, tests that failed to replicate, and segments with insufficient sample. An executive should not have to infer absences from a footnote.

Inconclusive Does Not Mean Neutral

Let Δ be change in confirmed takeover rate, where negative is beneficial. Suppose the 95% interval is $[-1.8, +0.6]$ incidents per 10,000 accounts. The estimate does not establish benefit at a conventional threshold, but it also does not establish equivalence. Ask decision questions:

- How much probability lies below the minimum worthwhile benefit?

- How much lies above the maximum tolerable harm?
- What are costs and reversibility of each option?
- What evidence would arrive by waiting?
- What harm or learning is lost during delay?

A frequentist confidence interval is not a probability distribution over the effect. If decision makers need probabilities such as $P(\Delta < -0.5)$, use a declared Bayesian model or bootstrap decision simulation with its assumptions, not an informal reinterpretation of a confidence interval.

For action a and uncertain state θ , compare expected loss:

$$EL(a) = \int L(a, \theta) p(\theta \mid D) d\theta + C(a).$$

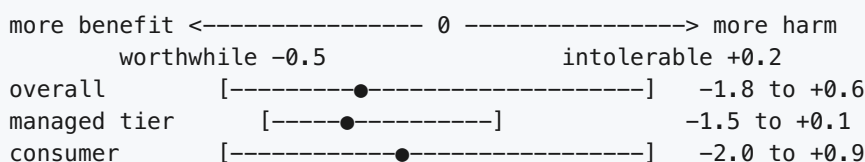
Then run sensitivity over disputed losses and priors. If the recommendation changes whenever the cost of friction moves slightly, say the decision is preference-sensitive. If every plausible setting favors a limited pilot, the staged recommendation is robust even when the global-effect claim is not.

Visualizing Uncertainty for Rare Security Outcomes

Do not use a red/amber/green cell to summarize an interval that crosses important thresholds. A useful effect plot includes:

- a zero-effect line;
- a minimum worthwhile benefit boundary;
- a maximum tolerable harm boundary;
- point and interval estimates for the overall population and predeclared segments;
- raw event counts and denominators;
- a note on assignment unit and outcome maturity.

For the example, a textual version is:



This is not a forest plot substitute in publication; it shows the semantics a plot must carry. Avoid truncating the axis at the point estimate, mapping probability to color without a legend, or hiding a zero-event numerator. “0 of 2,000” is bounded evidence, not zero risk. If no failures are observed in n approximately independent trials, the rough 95% upper bound $3/n$ is a useful orientation. If observations cluster by repository, user, session, or model version, show the cluster count and use an effective-sample-size or cluster-bootstrap analysis; the raw event count can materially overstate certainty.

Pair the effect plot with an outcome table. Executives need consequences: expected takeovers avoided, additional challenges, support tickets, analyst workload, latency, and affected customer groups under low/base/high scenarios.

Forecasts Make Beliefs Auditable

Decision memos often contain implicit predictions: “the interval will narrow,” “attackers will not displace,” or “support volume will normalize.” Convert them into resolvable forecasts.

A forecast record contains question, probability, opening evidence, forecaster, timestamp, close date, resolution source, and ambiguity rule. Example: “By 2026-11-30, the six-week pilot will show at least 0.5 fewer confirmed takeovers per 10,000 eligible accounts without challenge abandonment exceeding 0.6 percentage points.” This compound question may be split into two forecasts so resolution is diagnostic.

For binary outcome y and forecast p , Brier score is $(p-y)^2$. Across forecasts, calibration asks whether events assigned 70% occur about 70% of the time. Sharpness asks whether forecasts move away from 50%; sharp but miscalibrated confidence is not skill.

Use calibration bins only with enough observations and show counts. Also report mean Brier score against simple baselines, such as historical frequency or always 50%. Security forecasts are correlated and selectively created, so do not overinterpret a small leaderboard. The goal is learning: which teams, horizons, and event types are overconfident, and whether memo probabilities improve after feedback.

Never score recommendations as if they were forecasts. “Run a pilot” is a choice. Score its embedded empirical predictions, such as workload, effect, or guardrail breach.

Safe Lab: Score and Calibrate a Forecast Register

This standard-library lab evaluates ten resolved forecasts. It computes Brier score and reliability by probability bucket. The cases are synthetic.

```
forecasts = [
    (0.9,1),(0.8,1),(0.8,0),(0.7,1),(0.7,1),
    (0.6,0),(0.4,0),(0.3,1),(0.2,0),(0.1,0),
]
brier=sum((p-y)**2 for p,y in forecasts)/len(forecasts)
buckets={"low":[],"mid":[],"high":[]}
for p,y in forecasts:
    key="low" if p<0.34 else "high" if p>=0.67 else "mid"
    buckets[key].append((p,y))
print("brier",round(brier,3))
for key in ("low","mid","high"):
    rows=buckets[key]
    mean_p=sum(p for p,_ in rows)/len(rows)
    observed=sum(y for _,y in rows)/len(rows)
```

```
print(key,"n",len(rows),"forecast",round(mean_p,2),"observed",round(observed,2))
assert round(brier,3)==0.193
```

Expected output:

```
brier 0.193
low n 3 forecast 0.2 observed 0.33
mid n 2 forecast 0.5 observed 0.0
high n 5 forecast 0.78 observed 0.8
```

The high bucket is close to calibrated in this tiny sample, while the middle bucket is not. With only two cases, that is a prompt for inspection, not a stable conclusion. Extend the lab by comparing the register with a 50% baseline ($\text{Brier} = 0.25$) and by separating 30-day operational forecasts from one-year threat forecasts.

Staged Recommendations

A recommendation should bind action to evidence strength and reversibility.

Stage	Authority	Appropriate evidence	Required controls
observe	no user impact	mechanism plausible, instrumentation weak	data-quality plan and deadline
shadow	compute recommendation only	offline validity and privacy review	workload projection and disagreement audit
pilot	limited users/regions, reversible	safety case plus uncertain benefit	consent/notice where needed, stop rules, owner
expand	broader authority	mature benefit, acceptable burden and calibration	SLO, on-call, rollback, recertification
restrict	reduce authority	guardrail breach or evidence loss	incident process and impact review
retire	remove control	no decision value, excessive cost, or unsupported claim	archive evidence and migration plan

For the step-up control, a defensible recommendation might be: “Do not launch globally. Expand from 5% to 15% in managed enterprise tenants for six weeks; exclude accessibility-sensitive flows; stop if abandonment exceeds +0.6 pp for three days or any severe account-lockout incident is confirmed; expand further only if the mature posterior probability of at least 0.5 fewer takeovers per 10,000 exceeds 80% and support capacity remains below 70%.”

The exact Bayesian gate is a governance choice requiring a documented model and prior. The strength is not the number 80%; it is the precommitted transition, consequence bound, and review date.

Production Failure Drill: The Slide That Lost Its Interval

An analyst's effect plot shows a 30% reduction with a wide interval. During executive editing, the interval and denominator disappear, leaving a large green arrow. The CISO approves global rollout. This is an analytical integrity incident even though no database failed.

Run a tabletop with versioned memo artifacts. Detection may come from the analyst, an approver, or an automated check requiring uncertainty fields. Containment pauses authority expansion and sends a corrected decision brief. Preserve the analysis, original memo, edited slide, approvals, and distribution list. Determine which decisions relied on the altered representation.

Recovery requires re-presenting options with interval, counts, friction, and staged alternative; the accountable executive then reaffirms or changes the decision. The post-incident control is not a ban on slides. Require decision-critical charts to carry an evidence ID and generated caption; material edits after analytical sign-off trigger reapproval; dissent and limitations remain attached to the decision record.

The analyst owns accuracy of the estimate, the memo owner owns faithful translation, and the decision maker owns the chosen action. Communications owns distribution but does not own the evidence claim.

Tailoring Without Changing the Evidence

Engineers need schema, test, and failure details. Operators need workload, thresholds, runbooks, and failure signals. Product leaders need adoption, user burden, and options. Executives need the decision, consequence range, reversibility, and owner. These are views over one ledger, not separate truths.

A 30-second summary for the case is: "The pilot points toward fewer takeovers but is too small to establish the size of benefit; it clearly adds 0.42 percentage points of abandonment. I recommend a six-week expansion limited to managed tenants, with explicit abandonment and lockout stops, before any global launch."

The technical appendix then documents assignment, missingness, effect model, sensitivity, segment results, and code manifest. Concision at the top is safe only because traceability exists below.

Sources and Further Reading

- NIST, [Visualizing Uncertainty](#) provides research context for presenting uncertainty rather than suppressing it.
- NIST/SEMATECH [e-Handbook of Statistical Methods](#) supplies statistical foundations for intervals, experimental analysis, and graphical methods.
- NIST [Cybersecurity Framework 2.0](#) emphasizes governance and communicating cybersecurity risk to decision makers.
- Tetlock and Gardner, [Superforecasting](#) motivates resolvable questions, probability estimates, and feedback; organizational use still requires care about incentives and selection.
- Gneiting and Raftery, [Strictly Proper Scoring Rules](#) supplies the formal basis for evaluating probabilistic forecasts without rewarding dishonest probability reports.

- Gelman et al., *Bayesian Workflow* supports iterative model criticism and sensitivity rather than treating posterior output as automatic truth.

The synthesis is that uncertainty communication becomes operational when it is versioned, resolvable, and tied to staged authority. A chart informs; a decision system records how evidence changes action and later checks whether its forecasts deserved confidence.

Interview Practicum

Scenario. A randomized regional rollout estimates a 30% reduction in account takeover, but the 95% interval ranges from a 69% reduction to a 58% increase. Challenge abandonment rises by 0.42 percentage points with a narrow interval. The CISO asks for a red/green recommendation before tomorrow's board meeting.

Prompt. Write and present the recommendation.

Strong candidates ask about absolute event counts, assignment unit, outcome maturity, missing linkage, spillover, minimum worthwhile benefit, maximum tolerable harm, reversibility, affected segments, support capacity, and what the board actually decides.

Then add: the board deck permits one slide, and the control vendor publicly claims an 80% reduction based on a different customer population.

Worked Answer

My headline would be amber with words, not an unexplained color: "Evidence of security benefit is promising but inconclusive; user friction is established. Recommend constrained expansion, not global deployment." The slide shows the absolute risk difference and interval against benefit and harm thresholds, takeover counts and denominators, the +0.42 pp abandonment effect, and the staged option.

I would not describe the confidence interval as a probability of benefit. If a Bayesian probability is requested, I would fit and document an appropriate model, show prior sensitivity, and keep it separate from the randomized estimate. The vendor's 80% claim belongs in external evidence with population, implementation, and design differences; it does not replace our result.

The recommendation is a six-week expansion in reversible managed-tenant flows, from 5% to 15%, with no expansion into accessibility-sensitive or high-lockout-risk flows. Stop on abandonment above +0.6 pp, a confirmed severe lockout, telemetry coverage below the approved threshold, or support saturation. Predefine the security-effect gate using mature outcomes and state how spillover will be monitored.

The memo ledger distinguishes observed counts, estimated effects, spillover assumptions, external vendor evidence, and the recommendation. The decision record names the CISO as policy approver, product as user-impact owner, security analytics as estimate owner, and operations as rollout owner. Doing nothing remains an option with its own expected compromise cost; delay is not presented as neutral.

For one slide, I would link an evidence ID to the full memo and technical appendix. If anyone removes the interval or denominator, analytical sign-off is invalidated.

Follow-Up Questions and Answers

Is amber evasive? It is evasive without an action. Here it is paired with a specific staged recommendation, stop rules, and decision date.

Why not say the experiment failed? It did not precisely estimate the rare security outcome, but it successfully identified friction and narrowed plausible effects. “Failed” conflates evidence with a preferred conclusion.

How do you choose the next sample size? Base it on a decision-relevant effect and clustered event rate, then account for outcome delay, attrition, sequential rules, and operational feasibility—not an arbitrary power target alone.

Should executives see posterior distributions? Only if the model and interpretation aid the decision. Often an effect plot, consequence table, and sensitivity summary communicate better; keep technical detail available.

What is Principal-level communication? It refuses precision theater while still making a choice possible, exposes value judgments, protects dissent and provenance, and designs the next reversible decision rather than merely requesting more data.

Key Takeaways

- A decision memo separates observed facts, estimates, assumptions, external evidence, and recommendations.
- Inconclusive evidence can still rule out options, establish costs, or justify a reversible stage.
- Uncertainty visuals need decision thresholds, intervals, raw counts, denominators, and mature outcomes.
- Forecast registers turn implicit beliefs into resolvable probabilities and reveal overconfidence over time.
- Recommendations should define authority, population, duration, guardrails, owners, and transitions.

Field lesson: use a two-column executive decision

The sec-pr decision brief placed **platform readiness** beside **detector effectiveness** instead of blending them into one status. Verified engineering evidence supported controlled use of the validation platform; missing adjudicated labels, confirmed misses, representative cohorts, and observation time kept effectiveness claims gated. The format enabled action without green-metric theater. Chapter 47 contains the evidence ledger.

Chapter 45: Case Study: Measuring Security for a Coding Agent

A production coding agent is a principal with a changing view of a repository, access to tools, inherited credentials, and the ability to turn untrusted text into durable effects. A conventional model score therefore answers the wrong question. The deployment question is whether a particular agent, under a particular authority envelope, reduces engineering time without creating unacceptable paths from attacker-controlled context to protected assets.

This capstone joins the threat modeling, telemetry, evaluation, causal measurement, production engineering, and decision-writing methods developed earlier. The hypothetical agent, Forge, may read a granted repository, edit a disposable worktree, run approved build commands, and propose a pull request. It may not read arbitrary host files, retrieve production secrets, change branch protections, merge code, or deploy. Those are architecture properties, not prompt instructions.

Authority-centered threat model

The model uses four objects: **principal**, **provenance**, **capability**, and **effect**. A principal is the human, agent, or delegated subagent responsible for an operation. Provenance records which observations influenced it. A capability is an unforgeable, expiring grant scoped to a resource and action. An effect is the durable state change that follows.

Forge crosses five trust boundaries: the user's objective may be abusive; repository content is untrusted; model output is nondeterministic; tools translate proposals into filesystem, network, or CI effects; and external tool output can inject a second instruction stream. Protected assets include source integrity, developer credentials, proprietary code, CI identity, release artifacts, audit evidence, and availability.

The important abuse cases extend beyond the phrase *prompt injection*: credential discovery, cross-repository transfer, dependency substitution, test tampering, persistence in generated configuration, review evasion, resource exhaustion, and confused-deputy use of a developer's ambient authority.

Sensitive-action taxonomy

Class	Examples	Default disposition
S0 observational	list granted files, inspect a diff	allow and log
S1 reversible local	edit disposable worktree, run formatter	allow in sandbox
S2 externally visible	open draft PR, post comment, approved fetch	scoped grant
S3 privileged mutation	change workflow, policy, secret reference, release config	human approval plus policy check
S4 catastrophic/prohibited	exfiltrate secrets, disable audit, merge protected branch, deploy	structurally unavailable

Classification depends on arguments. `git diff` is S0; a push is S2; editing `.github/workflows/release.yml` is S3 although it is “just a file write.” Unknown tools and unparsed shell strings fail closed. This taxonomy is the join key across policy, telemetry, evaluation, and reporting.

Architecture: make effects pass through a broker

```

flowchart LR
    U[Developer objective] --> O[Orchestrator]
    R[Untrusted repo/issues] --> O
    O --> P[Typed action proposal]
    P --> G[Capability and policy broker]
    G -->|S0/S1| W[Ephemeral worktree]
    G -->|S2/S3| H[Human approval]
    G -->|deny| D[Structured refusal]
    W --> T[Hermetic adapters and egress proxy]
    H --> T
    T --> E[Effect receipt]
    O --> L[Append-only trace ledger]
    G --> L
    E --> L
    L --> M[Monitors and eval replay]

```

Forge never invokes a raw shell from model-generated text. It proposes a typed action such as `Run-Build(target="//service:test", timeout_s=300)` or `CreateDraftPR(repo_id, base_sha, patch_digest)`. The broker checks resource scope, sensitivity, approval, expiry, and policy version. The sandbox mounts only the granted repository, supplies short-lived task credentials, filters egress, enforces compute budgets, and is destroyed afterward. A subagent receives a subset of its parent's grants; delegation only reduces authority.

The key invariant is

$$\backslash[\text{executed}(a) \rightarrow \text{grant}(p,r,v,t) \wedge \text{allow}(a,v) \wedge \text{args}(a)=\text{args}_{\{\text{checked}\}}(a). \backslash]$$

Checking one command and executing another is a time-of-check/time-of-use flaw. The broker therefore signs a canonical action digest, and an adapter accepts only that exact digest. Content-origin labels, secret scanning, immutable images, and review views provide defense in depth, but the broker remains authoritative when the model ignores every instruction.

Trace schema and privacy boundary

The trace must reconstruct effects without retaining proprietary prompts forever:

```

agent_trace(trace_id, parent_trace_id, task_id, tenant_hash,
  principal_type, model_release, orchestrator_release, policy_version,
  sandbox_digest, started_at, ended_at, terminal_state, tokens, wall_ms)

observation(trace_id, seq, observed_at, origin, resource_hash,

```

```

content_digest, trust_label, redaction_profile, encrypted_payload_ref NULL)

action_decision(trace_id, seq, proposed_at, action_type,
  canonical_args_digest, sensitivity_class, capability_hash,
  decision, reason_code, approval_hash NULL, policy_latency_ms)

effect_receipt(trace_id, seq, executed_at, adapter, exit_class,
  bytes_read, bytes_written, egress_class, pre_digest, post_digest, artifact_ref)

```

Raw code and prompts have short tenant-controlled retention. Digests, origins, decisions, and effect summaries remain longer. Payload access is separately audited. Telemetry loss is a security event: no S2/S3 effect executes if the ledger cannot issue a durable receipt.

Evaluation and control coverage

An evaluation item is a versioned environment, objective, adversarial placement, permitted result, prohibited effects, and executable scoring program. The suite includes direct abuse, injections in READMEs and issues, poisoned tool output, malicious dependency names, encoded instructions, multi-turn escalation, and parent/subagent collusion. Adaptive evaluators see refusal reasons and attempt a second route.

For threat stratum $\backslash(k\backslash)$, estimate compromise probability with uncertainty:

$$\backslash[\hat{p}_k=(x_k+1)/(n_k+2), \backslashqqquad P(\text{compromise})=\backslashsum_k w_{kp_k} \backslash]$$

Weights reflect production exposure: frequency of untrusted issues, network use, repository sensitivity, and action class. Report worst-stratum performance too, so a rare S4 bypass is not hidden by easy cases.

Control coverage is over attack paths, not an inventory. A path might be `untrusted observation -> model adoption -> privileged proposal -> broker grant -> external effect`. A control covers it only when a test demonstrates interruption of a required edge. Weighted residual exposure is

$$\backslash[R=\backslashsum_j \backslashlambda_j I_j(1-c_j)d_j, \backslash]$$

where $\backslash(\backslashlambda_j\backslash)$ is path frequency, $\backslash(I_j\backslash)$ impact, $\backslash(c_j\backslash)$ demonstrated interruption probability, and $\backslash(d_j\backslash)$ a detectability factor bounded above zero. Logging is not prevention.

The benign suite uses real maintenance tasks stratified by language, repository size, and developer experience. Outcomes include independently tested completion, false refusal, unnecessary approval, regression, p95 duration, cost, and override. For task $\backslash(i\backslash)$:

$$\backslash[U_i=V_{iC_{i-C_{\{review\}}}}A_{i-C_{\{delay\}}}T_{i-C_{\{badpatch\}}}B_i \backslash]$$

Security and utility remain separate until a decision maker states exchange rates. A composite score must never allow time saved to cancel a catastrophic bypass.

Safe runnable Python lab

This standard-library lab does not execute commands. It applies capability policy to typed proposals.

```
from dataclasses import dataclass
from hashlib import sha256
import json

RANK = {"S0": 0, "S1": 1, "S2": 2, "S3": 3, "S4": 4}

@dataclass(frozen=True)
class Grant:
    principal: str
    repo: str
    actions: frozenset[str]
    max_class: str

@dataclass(frozen=True)
class Proposal:
    principal: str
    repo: str
    action: str
    sensitivity: str
    args: dict
    untrusted: bool = False

def digest(p):
    body = json.dumps(p.__dict__, sort_keys=True, separators=(",", ":"))
    return sha256(body.encode()).hexdigest()[:10]

def decide(g, p):
    if p.sensitivity == "S4": return "DENY", "prohibited_effect"
    if (p.principal, p.repo) != (g.principal, g.repo): return "DENY", "scope_mismatch"
    if p.action not in g.actions or RANK[p.sensitivity] > RANK[g.max_class]:
        return "DENY", "capability_missing"
    if p.untrusted and RANK[p.sensitivity] >= 2:
        return "REVIEW", "untrusted_to_external_effect"
    return "ALLOW", "bounded_capability"

g = Grant("agent-7", "payments", frozenset({"read", "edit", "draft_pr"}), "S2")
tests = [
    Proposal("agent-7", "payments", "edit", "S1", {"path": "src/tax.py"}),
    Proposal("agent-7", "payments", "draft_pr", "S2", {"base": "a18f"}, True),
    Proposal("agent-7", "payments", "deploy", "S4", {"env": "prod"}),
    Proposal("agent-7", "payroll", "read", "S0", {"path": "salary.csv"}),
]
for p in tests:
    decision, reason = decide(g, p)
    print(p.action, decision, reason)
```

Expected output:

```
edit ALLOW bounded_capability
draft_pr REVIEW untrusted_to_external_effect
deploy DENY prohibited_effect
read DENY scope_mismatch
```

Mutation test: change one approved argument after computing its digest and assert that the adapter rejects it. Production receipts must be authenticated and expiring; this demonstration digest is only a replay identifier.

Staged rollout and telemetry

Rollout moves through offline replay, employee red-team, shadow mode, S0 read-only, S1 sandboxed edits, then a small S2 draft-PR cohort. S3 remains approval-gated and S4 absent. Promotion requires no prohibited effect, an upper confidence bound below the class risk budget, trace completeness above 99.9%, stable accepted-task completion, and acceptable approval latency.

Online monitors separate model drift from enforcement failure: proposal mix by sensitivity and origin; deny/review reasons; canary-secret and out-of-scope access; approval acceptance and reversal; effect receipts without decisions (target zero); task success and abandonment; sandbox failures; and trace ordering or completeness anomalies.

Release eval design and statistical stopping

Each candidate is compared with the current release on a paired repository suite. Pairing matters because difficulty varies more across repositories than across repeated samples within one repository. For nondeterministic inference, run multiple seeded attempts and preserve sampling parameters. Report `pass@1`, which reflects ordinary use, and attack success across repeated attempts, because an attacker can retry. A safeguard that holds once but fails on the fifth paraphrase is not characterized by its first-attempt rate.

Rare catastrophic failures require different stopping logic from utility regressions. Zero S4 successes in a small suite does not establish zero risk; report an upper confidence bound. Any S4 adapter execution fails the release regardless of aggregate performance. S2 bypasses use a predeclared risk budget and sequential test with alpha spending, preventing repeated inspection until results look favorable. Benign completion and approval latency use equivalence margins agreed before testing.

Eval maintenance is a software supply chain. Items have owners, provenance, expected behavior, last review, and retirement reasons. Public attacks enter quarantine first so teams cannot tune directly against the test. A private holdout and rotating expert red-team set estimate generalization. Dashboards distinguish known-family regression retention from novel-family performance.

Human approval is a measured control

The review surface shows the requested effect, resource scope, security-relevant diff, influential content origins, and test evidence. It never asks a reviewer to approve opaque reasoning text. Approvals expire, bind to one action digest, and cannot authorize a changed retry.

Measure acceptance by action class, later reversal, time spent, and risky-effect escape. High acceptance combined with subsecond review suggests habituation. When volume exceeds capacity, restrict agent authority instead of silently turning approval into notification. Seeded exercises estimate whether reviewers notice secret additions, workflow mutations, and disguised egress. “Human in the loop” thus becomes a subsystem with measured workload and reliability limits.

Failure drills

Poisoned issue proposes egress. Freeze the release, revoke network grants, preserve the repo snapshot and trace, inspect whether the broker denied it, add transformations to replay, and search sibling traces by origin digest. A broker denial is a contained model-control near miss; an effect is an incident.

Ledger outage. S2/S3 fail closed. S0/S1 may continue only in isolated sandboxes with a bounded local queue. Reconciliation proves every queued decision has one terminal receipt.

Productivity cliff after a policy update. Compare matched tasks by reason-code transition, action class, and repository. Roll back the policy independently of the model. Workflow pain does not justify removing the authority boundary.

Final rollout memo

Decision: approve S0/S1 across internal repositories and S2 draft pull requests for a 5% cohort. Do not authorize autonomous merge, workflow-policy modification, secret access, or deployment.

Evidence: typed, digest-bound capabilities; prohibited actions absent from the tool surface; attack-path tests across repository, issue, dependency, and tool-output injections; complete effect receipts; and positive time savings after review cost. Residual uncertainty is highest for novel multi-step attacks and tenant-specific build tools.

Conditions: security owns taxonomy and stops; platform owns adapter invariants and ledger SLOs; model engineering owns replay performance; product owns refusal and approval friction. Expansion requires two stable releases, successful ledger and poisoned-repository drills, and independent red-team review. Any unmatched effect, canary-secret access, cross-repository read, or S4 adapter acceptance triggers rollback.

The memo does not claim “the agent is safe.” It names release, authority, population, evidence, uncertainty, owners, and the next reversible decision.

The decision registry also preserves rejected alternatives: unrestricted developer credentials, prompt-only policy, and immediate universal launch. Recording why they were rejected prevents later teams from rediscovering the same unsafe shortcuts after personnel or product strategy changes. The next review compares actual exposure, utility, and incidents with the memo's assumptions.

Sources and Further Reading

- NIST, [AI Risk Management Framework 1.0](#). Its Govern/Map/Measure/Manage separation becomes governance ownership, an authority threat model, replay measurement, and rollout gates here.
- NIST, [Secure Software Development Framework SP 800-218](#). Build integrity and provenance matter even when code is machine-generated.
- NIST, [Zero Trust Architecture SP 800-207](#). Per-request authorization is adapted to agent tool calls; no model receives ambient trust.
- MITRE, [ATLAS](#). Its adversary behaviors seed evals, but executable path tests—not checklist mappings—support coverage claims.
- OWASP, [Top 10 for LLM Applications](#). Prompt injection, insecure output handling, excessive agency, and supply-chain risk become concrete action classes and controls.
- IETF, [RFC 9334: RATS Architecture](#). It motivates issuer-bound environment and artifact evidence rather than unauthenticated digests.

Interview Practicum

You lead security measurement for a coding agent used by 4,000 engineers. A red team hid an instruction in a test fixture; the agent attempted to upload a private config to a paste service, but the network proxy blocked it. Product says no breach occurred and wants universal draft-PR launch next week. Security wants shutdown. Raw prompts expired under a seven-day privacy policy, but action metadata remains. Present a decision in 20 minutes and ask for only five facts.

Worked Answer

I would ask: which broker decision preceded egress; whether the destination was denied by the agent-specific allowlist or an unrelated corporate proxy; what alternative egress tools existed; how many traces observed the fixture digest; and what new authority draft-PR mode adds.

This is not proof of safety. It is a demonstrated model-control failure and possibly successful defense in depth. If the broker lacked an egress capability and denied the typed action before the proxy, the architecture held. If it allowed the call and a generic proxy happened to intervene, the agent boundary failed.

I would preserve the fixture, releases, action digests, receipts, and sandbox image; search by observation digest; revoke unnecessary network grants; and add this family to adaptive replay. Deleted raw prompts limit semantic reconstruction but should not prevent reconstructing origin, proposal, decision, and effect. If they do, the schema is inadequate.

I would allow continued S0/S1 only if there was no out-of-scope effect and telemetry is complete. I would hold broad S2 expansion until the team proves draft PRs cannot carry secrets or attacker-controlled links, reruns the family against the candidate, and computes a stratum-specific risk bound. A small canary may proceed with network-deny defaults, scanning, human review, and stops.

The Principal move is to split ownership: platform owns a broker defect, model engineering owns instruction adoption, product owns friction, and security owns risk acceptance. I would document the counterfactual without the proxy and the incremental effect enabled by S2. The answer is a reversible capability-specific rollout, not a vote on whether the model is trustworthy.

Follow-Up Questions and Answers

What if it was one failure in a million tasks? Use exposures to the malicious pattern as the denominator, report the stratum rate and uncertainty, then weight by production prevalence without hiding severity.

How do you estimate false refusals? Sample by reason code and task stratum, replay with independent test oracles, and combine adjudication with abandonment and manual-completion evidence. Random audits correct policy-selected labels.

Store prompts longer? Not automatically. Test encrypted incident sampling against privacy cost. Durable provenance, decisions, and effects are mandatory; full prompt retention is not.

What forces shutdown? An executed prohibited effect, cross-tenant access, an adapter accepting an unmatched action, agent-disabled audit, or loss of containment without reliable reconstruction.

Key Takeaways

- Govern a coding agent by executable authority, not its system prompt.
- Typed capabilities and sandboxed adapters turn model failures into containable proposals.
- Connect untrusted observations to decisions and effects while minimizing content retention.
- Evaluate adaptive attack paths, production exposure, uncertainty, and benign utility separately.
- Rollout memos must name action classes, population, stops, evidence gaps, and owners.

Chapter 46: Case Study: Identity Compromise Detection

Identity compromise is not a row-classification problem. An account can be legitimate at 09:00, have its refresh token stolen at 09:12, and be used simultaneously by its owner and attacker at 09:20. The evidence is distributed across authentication, device, session, privilege, application, and recovery systems. The intervention changes what becomes observable: a challenge stops some attacks, annoys some users, and selectively produces labels. A useful system must therefore model time, relationships, uncertainty, action capacity, and causal friction together.

This capstone designs Helios, an enterprise compromise system for workforce identities. Its unit of decision is not “the user” but an **identity episode**: a bounded interval in which related credentials, sessions, devices, resources, and actions may share control. Helios may permit, silently monitor, require step-up authentication, revoke a session, or route an episode to an investigator. It does not automatically disable an employee from one anomalous event.

Temporal identity graph

The graph has typed nodes and interval-valued edges:

- Person, Account, Credential, Session, Device, Network, Application, Resource, and Case nodes;
- OWNS, AUTHENTICATED_WITH, ISSUED, OBSERVED_ON, ACCESSED, ASSUMED_ROLE, and REVIEWED_AS edges;
- valid_from, valid_to, observed_at, source, and confidence on every edge.

Event time and ingestion time are distinct. A mobile device can upload an authentication event late; a directory correction can change the historical owner mapping. Point-in-time features use only graph state known at decision time. Otherwise the model “learns” from a later analyst verdict or asset enrichment.

```

flowchart LR
  A[Auth/device/app events] --> N[Normalize and resolve entities]
  N --> G["Bitemporal identity graph"]
  G --> E[Episode builder]
  E --> F[Point-in-time feature service]
  F --> B[Rules and behavioral baselines]
  F --> M[Calibrated supervised model]
  B --> C[Risk composer]
  M --> C
  C --> P[Capacity-aware intervention policy]
  P --> X["Allow / step-up / revoke / investigate"]
  X --> O[Outcome and friction ledger]
  O --> G
  O --> Q[Evaluation and causal analysis]

```

An episode begins with a credential issuance or authentication and links later events while continuity evidence remains plausible. It closes after inactivity, explicit revocation, or a maximum horizon. Concurrent sessions

remain separate but share account and credential ancestors. This permits a “good owner plus bad token” representation rather than forcing one label onto the account.

Concrete schemas

```
identity_event(
  event_id, event_time, ingest_time, source, event_type,
  account_id, session_id NULL, credential_id_hash NULL,
  device_id_hash NULL, network_prefix, app_id, resource_id NULL,
  auth_method, result, source_confidence, raw_ref
)

episode_feature(
  episode_id, as_of_time, feature_version,
  geo_velocity_kmh, new_device_score, peer_app_rarity,
  session_overlap_s, token_age_s, failed_auth_30m,
  privilege_delta, resource_novelty, graph_fanout_10m,
  sensor_missing_mask
)

decision(
  decision_id, episode_id, decision_time, model_version,
  raw_score, calibrated_risk, risk_band, action,
  policy_version, capacity_state, explanation_codes
)

outcome(
  episode_id, outcome_time, signal_type, value,
  label_strength, reviewer_id_hash NULL, available_after,
  intervention_observed, friction_seconds NULL
)
```

The missingness mask is a feature and a health signal. “No device evidence” differs from “known unmanaged device.” Entity-resolution confidence propagates into risk rather than disappearing after a deterministic join.

Labels that admit uncertainty

Confirmed compromise is rare and delayed. Helios combines sources without pretending they are equivalent:

Evidence	Label treatment
user reports “not me,” forensic token theft, attacker infrastructure match	high-confidence positive
analyst-confirmed benign travel or approved automation	high-confidence negative for that episode
successful phishing-resistant step-up	weak negative; attacker may have abandoned
password reset, help-desk recovery, bulk session revoke	weak positive
no case opened	unlabeled, not negative

Let latent compromise be (Y) , labeling functions be $(L_j \in \{-1, 0, 1\})$, and their conditional accuracies be learned from overlap and a small adjudicated set. A generative weak-label model estimates

$$[q_i = P(Y_i = 1 \mid L_{i1}, \dots, L_{im}),]$$

while accounting for correlated functions such as “password reset” and “help-desk recovery.” These soft targets can train a discriminative model, but evaluation uses time-separated, independently reviewed episodes. Cases sampled only above the old threshold are corrected with randomized below-threshold audits and inverse-probability weighting.

Reviewers see evidence available at decision time plus separately marked later evidence. They assign `compromised`, `benign`, `indeterminate`, or `mixed-control`, record confidence, and identify the controlled credential or session. Double review is concentrated where disagreement changes policy, not spread uniformly.

Three model layers and calibration

Helios deliberately keeps three layers. Deterministic rules catch high-specificity facts such as an impossible refresh-token binding or use after revocation. Behavioral baselines compare an episode with the identity's own history and an appropriate peer group. A supervised gradient model combines stable graph and sequence features. The composer records which layer contributed; a model outage therefore degrades to rules instead of failing open.

A useful sequence statistic is the decayed surprise of event (e_t) :

$$[S_t = \sum_{k < t} \exp[-(t - t_k) / \tau], [-\log P(e_t \mid \text{context}_{t_k})].]$$

Graph fanout and privilege escalation then distinguish an unusual login from an episode that rapidly touches many sensitive resources.

Raw scores are calibrated on a later time block using isotonic regression or Platt scaling, stratified when major populations have different base rates. Reliability diagrams, Brier score, and expected calibration error accompany precision-recall results. Calibration is monitored by model release, authentication method, region, workforce type, and sensor availability. An apparently calibrated aggregate can hide severe contractor overestimation and administrator underestimation.

From risk to intervention

An intervention is selected by expected loss, capacity, and constraints:

$$[a^*(x) = \arg \min_a \{p(x)L_{\text{miss}}(a) + (1 - p(x))L_{\text{friction}}(a) + L_{\text{ops}}(a)\}.]$$

Actions have different attack interruption and user costs. Silent monitoring is cheap but weak. A phishing-resistant step-up is stronger and measurable. Session revocation may destroy unsaved work. Account disablement can block emergency operations and requires corroboration.

The policy is monotone in calibrated risk but context-sensitive. At equal risk, a privileged admin accessing a production control plane may receive a stronger action than an employee reading a public wiki. Capacity is

explicit: if investigators can handle 60 episodes per day, ranking is by expected avoidable loss, not raw anomaly. Helios abstains when key sensors are missing or entity resolution is ambiguous and routes the episode to a safer action.

Guardrails prohibit use of protected personal attributes, cap repeated challenges, provide accessible recovery, and prevent an attacker from weaponizing the detector to lock out a victim. Explanation codes state evidence such as `new_hardware_key + token_reuse_after_revocation`; they do not expose a recipe precise enough to aid evasion.

Safe SQL lab: build temporal episodes

The following SQLite lab creates a tiny event stream and calculates episode-level signals. It uses synthetic data and no credentials.

```
CREATE TABLE events(
  event_id INTEGER PRIMARY KEY,
  account TEXT, session TEXT, event_time TEXT,
  event_type TEXT, device TEXT, country TEXT,
  privilege_delta INTEGER, resource TEXT
);

INSERT INTO events VALUES
(1,'alice','s1','2026-04-03 09:00:00','login','laptop-A','US',0,'mail'),
(2,'alice','s1','2026-04-03 09:04:00','access','laptop-A','US',0,'docs'),
(3,'alice','s2','2026-04-03 09:07:00','login','unknown-X','NL',0,'mail'),
(4,'alice','s2','2026-04-03 09:08:00','assume_role','unknown-X','NL',1,'cloud-admin'),
(5,'alice','s2','2026-04-03 09:09:00','access','unknown-X','NL',0,'payroll'),
(6,'bob','s3','2026-04-03 10:00:00','login','laptop-B','US',0,'mail');

WITH session_rollup AS (
  SELECT account, session, MIN(event_time) AS start_time,
    COUNT(DISTINCT resource) AS resources,
    MAX(privilege_delta) AS privilege_jump,
    COUNT(DISTINCT device) AS devices,
    MIN(country) AS country
  FROM events GROUP BY account, session
), overlap AS (
  SELECT a.account, a.session,
    EXISTS(SELECT 1 FROM session_rollup b
      WHERE b.account=a.account AND b.session<>a.session
      AND ABS(strftime('%s',b.start_time)-strftime('%s',a.start_time)) < 900)
    AS concurrent_session
  FROM session_rollup a
)
SELECT s.account, s.session, s.resources, s.privilege_jump,
  o.concurrent_session,
  20*s.privilege_jump + 15*o.concurrent_session + 5*(s.resources-1) AS risk_points
FROM session_rollup s JOIN overlap o USING(account,session)
ORDER BY risk_points DESC, account, session;
```

Expected output:

```
alice|s2|3|1|1|45
alice|s1|2|0|1|20
bob|s3|1|0|0|0
```

This is an explainable feature calculation, not a production detector. It reveals an important modeling issue: Alice's legitimate session receives overlap points because the graph contains a suspicious sibling. A production composer should allocate evidence to the likely controlled session while retaining a shared episode link. Run the lab with `sqlite3`, or through Python's standard-library `sqlite3` module when the CLI is unavailable.

Causal evaluation of step-up friction

Observed challenge success is not the causal effect of step-up. High-risk episodes receive the treatment, so naive treated-versus-untreated comparisons are confounded. Define two primary estimands for eligible episodes:

$$\backslash[ATE_{\text{attack}} = E[Y(1) - Y(0)], \quad ATE_{\text{friction}} = E[F(1) - F(0)], \backslash]$$

where $\backslash(Y\backslash)$ is compromise completion and $\backslash(F\backslash)$ is user time lost. Because withholding a control from extreme-risk cases is unethical, randomize only within an equipoise band between two acceptable policies—for example immediate phishing-resistant step-up versus step-up at the first sensitive action. Always-treat high-risk cases and never-treat low-risk cases remain observational.

Instrument assignment before outcome, analyze intent-to-treat, and use cluster-robust errors when a user can contribute repeated episodes. Outcomes include confirmed downstream compromise, session revocation, sensitive action completion, challenge success, recovery invocation, abandonment, time to resume work, repeated challenges, and support contacts. Estimate heterogeneous effects for admins, mobile users, accessibility cohorts, and device health, with privacy review.

When randomization is impossible, emulate a target trial using the policy threshold. Regression discontinuity near a stable calibrated-risk cutoff can estimate a local effect if actors cannot precisely manipulate the score and no other action changes at the cutoff. Interrupted policies, detector changes, and label delay must be modeled. Report sensitivity to unmeasured confounding rather than calling a propensity-weighted association causal by default.

Operations and failure drills

Offline replay without temporal leakage

Identity data makes leakage unusually easy. “Device later marked compromised,” a case opened after the decision, or a corrected owner mapping can yield spectacular offline results while being unavailable online. Helios materializes every row with an `as_of_time` and joins only records whose knowledge time precedes it. Labels have a separate `available_after` timestamp. Feature code runs identically in backfill and streaming, and replay compares the online decision payload with later reconstruction.

Splits are forward in time and grouped by account, credential family, or campaign where appropriate. A random row split would place adjacent events from one takeover in train and test. Evaluation includes cold-start users, new devices, and degraded sensors. Visible baselines—a few high-specificity rules, per-identity frequencies, and calibrated logistic regression—show whether graph complexity earns its cost.

Fairness, privacy, and recovery

Geography, schedule, disability accommodations, travel, and employment type can proxy for sensitive traits. Review asks whether a feature is necessary for an attack mechanism and whether a less invasive signal exists. Location precision is reduced when network class suffices; peer groups have minimum sizes; trace access is purpose-limited and audited.

Fairness is not equal challenge rates despite different exposure. Compare calibration, false interventions, recovery time, repeated challenge burden, and attack misses, then change features or policy where differences lack security justification. Recovery is part of control efficacy. If a legitimate traveler cannot recover, the system created an availability incident; if recovery uses weak questions, it becomes an attacker bypass. Recovery events return to the graph as security and friction outcomes.

Graph split drill. A device-ID migration fragments one device into thousands of nodes, causing novelty and challenge rates to spike. Freeze model promotion, compare source cardinality and resolution confidence, replay with corrected mappings, and reimburse users through automatic challenge suppression where corroborating signals are absent.

Adversarial low-and-slow drill. An attacker uses a familiar residential proxy and waits between actions. Test whether credential binding, resource sequence, graph fanout, and post-auth privilege evidence compensate for lost geolocation surprise. Do not simply lower thresholds globally.

Selective-label drill. Analysts only investigate top-ranked episodes for a month. Measure label propensity by score band, launch a safe random audit below threshold, reweight evaluation, and show how the apparent precision was inflated.

Step-up outage drill. The authentication provider fails. High-risk sensitive actions fail closed under a pre-agreed continuity policy; ordinary low-risk sessions degrade to monitoring with short expirations. The incident report distinguishes security exceptions from authentication availability.

Operational SLOs include point-in-time feature freshness, graph resolution latency, decision p95, challenge availability, intervention reversal time, daily review capacity, calibration drift, and outcome-link completeness. On-call dashboards show rates and denominators: a fall in detections can mean fewer attacks, missing events, or a broken join.

Sources and Further Reading

- NIST, [Digital Identity Guidelines SP 800-63B](#), grounds authenticator assurance, replay resistance, and recovery; risk scoring does not replace strong authentication.

- NIST, [Zero Trust Architecture SP 800-207](#), supports continuous, resource-aware decisions rather than permanent trust after login.
- MITRE ATT&CK, [Valid Accounts](#), explains why legitimate credentials make post-authentication sequence and resource evidence essential.
- Guo et al., [On Calibration of Modern Neural Networks](#), motivates post-hoc calibration, but temporal and population drift require continuing validation.
- Ratner et al., [Snorkel](#), provides a foundation for estimating latent labels from noisy labeling functions; correlated operational labels must be modeled explicitly.
- Hernán and Robins, [Causal Inference: What If](#), informs target-trial specification and the separation of prediction from intervention effects.

Interview Practicum

A global company challenges the top 0.5% of workforce sessions. Confirmed takeovers fell 25%, but help-desk calls rose 40%. Administrators complain of repeated prompts. Labels exist almost exclusively for challenged sessions, and a device-telemetry migration occurred halfway through the quarter. The CISO asks whether to expand the system to customers. Design the decision analysis.

Worked Answer

I would refuse the before/after causal claim. The policy selected the labels, the device migration changed features and missingness, and attacker pressure may have changed. First I would reconstruct bitemporal episodes under both identity-resolution versions and quantify event coverage, graph fragmentation, score shifts, challenge propensity, and outcome linkage by week and population.

For predictive quality I would create an independently adjudicated, time-forward sample containing challenged and unchallenged episodes. Random audits below the threshold identify missed compromise and permit inverse-propensity correction. I would evaluate calibration and expected avoidable loss, not accuracy, with separate administrator and device-health strata.

For intervention effect I would define eligible episodes and two outcomes: downstream harmful action and total user time lost, including recovery. Extreme-risk sessions stay protected. In an equipoise band I would randomize between two acceptable timings or challenge mechanisms, analyze intent-to-treat, cap repeat prompts, and monitor compromise, abandonment, help-desk demand, and accessibility. If randomization is unavailable, I would consider a threshold design only after proving the score and other controls were stable around the cutoff.

I would not transfer workforce thresholds to customers. Customer base rates, devices, recovery channels, loss functions, and tolerance differ. I would launch customer shadow scoring, build customer-specific calibration and friction baselines, complete abuse and lockout threat modeling, and test recovery capacity. The executive recommendation would separate: evidence the detector ranks risk; evidence step-up prevents harm; and evidence the customer workflow has acceptable net benefit.

At Principal scope I would also change the operating system: versioned entity resolution, randomized label audits as a permanent budget item, policy-independent outcome collection, a challenge-frequency constraint, and an accountable owner for each population's calibration. That prevents the next quarter from reproducing the same uncertainty.

Follow-Up Questions and Answers

What if the CISO needs one threshold tomorrow? Choose a conservative threshold from calibrated expected loss and capacity, retain hard rules for incontrovertible signals, cap repeated challenges, and label it an interim policy with a dated reevaluation.

Why not train on successful challenges as benign? Success proves possession of the requested factor, not that the preceding session was benign; an attacker may abandon or a compromised factor may satisfy the challenge.

How do you handle mixed control? Label the session or credential, not the whole account. Preserve concurrent legitimate and malicious branches in the graph.

Which metric belongs on the executive dashboard? Estimated harmful sessions prevented with an uncertainty interval, paired with hours of user friction and recovery burden, plus telemetry and calibration health.

Key Takeaways

- Model compromise as temporal control of sessions and credentials, not a permanent user label.
- Preserve event time, knowledge time, entity confidence, and missingness in the graph.
- Weak operational signals are useful only when their selection and dependence are explicit.
- Calibrated risk should drive a capacity-aware intervention policy, not an anomaly leaderboard.
- Evaluate prevention and friction as causal outcomes with ethical experimentation or carefully bounded quasi-experiments.

Chapter 47: Case Study: AI Vulnerability-Finding Product

An AI scanner does not create value when it emits a plausible paragraph about unsafe code. It creates value when it produces a reproducible, distinct, reachable, correctly prioritized security finding that a developer can fix, and when the fix remains effective. Every transition can fail: generated evidence may be invented, ten reports may describe one root cause, severity may ignore deployment context, a valid report may never reach its owner, and a quick patch may recur elsewhere.

This capstone designs Lumen, a multi-tenant product that analyzes authorized repository snapshots. Lumen combines static program analysis, dependency and configuration facts, and an AI hypothesis generator. The model can propose a vulnerability and a proof strategy; deterministic systems must establish artifact identity, trace evidence, sandboxed reproduction, and policy-compliant disclosure. The product is measured from hypothesis through durable remediation and enterprise risk reduction.

Finding as an evidence dossier

A finding is not a text blob. It is an immutable dossier with versioned claims:

```
finding(
  finding_id, tenant_id, repo_id, snapshot_sha, detector_release,
  weakness_id, root_cause_key, state, created_at, supersedes_id NULL
)

claim(
  finding_id, claim_type, claim_value, confidence,
  evidence_ids, produced_by, produced_at
)

evidence(
  evidence_id, kind, artifact_digest, source_location,
  derivation, sandbox_run_id NULL, retention_class
)

reproduction(
  sandbox_run_id, image_digest, command_template_id,
  network_mode, fixture_digest, exit_class, sanitizer_signal,
  side_effect_summary, replayable
)

disposition(
  finding_id, actor_role, at_time, decision,
  reason_code, ticket_id NULL, fix_commit NULL
)
```

Claims include `source`, `sink`, `dataflow`, `precondition`, `reachability`, `exploit_effect`, `affected_version`, and `severity`. Each points to evidence such as an AST path, call-graph edge, dependency

lockfile, configuration fact, or safe sandbox result. Prose is rendered from claims; it is never the system of record. A reviewer can reject the exploitability claim without discarding the verified source-to-sink path.

Evidence has lineage. If a code location came from generated code or an incomplete build, the dossier says so. Reproduction runs have no production credentials, deny outbound network by default, impose resource limits, and use inert fixtures. For dangerous classes where executable reproduction would be unsafe, a static witness and expert review replace a weaponized proof.

```

flowchart LR
  S[Authorized snapshot] --> I[Artifact inventory and build graph]
  I --> H[AI hypothesis generator]
  I --> A[Static and dependency analyzers]
  H --> V[Claim verifier]
  A --> V
  V --> R[Safe reproduction sandbox]
  V --> D[(Evidence dossier store)]
  R --> D
  D --> C[Root-cause clustering]
  C --> K[Calibrated ranking]
  K --> U[Developer workflow]
  U --> F[Fix validation]
  F --> O[Survival and recurrence outcomes]
  O --> E[Enterprise value ledger]

```

Quality taxonomy and evaluation sets

Lumen distinguishes six dimensions:

1. **Validity:** the claimed weakness exists in the scanned snapshot.
2. **Reachability:** a feasible path reaches the vulnerable code under stated configuration.
3. **Exploitability:** an actor satisfying the preconditions can cause the claimed effect.
4. **Severity:** technical impact and scope under the customer's environment.
5. **Actionability:** the dossier identifies an owner, location, cause, and feasible repair.
6. **Distinctness:** the report represents a new remediation unit rather than a duplicate symptom.

The benchmark has three layers. A curated gold set contains seeded and naturally occurring cases with blinded expert adjudication. A repository holdout measures generalization across organizations and time. A production sample estimates performance under real language, framework, and configuration distributions. Training contamination checks compare artifact hashes and lineage; synthetic examples are reported separately because they often have cleaner build systems and simpler paths.

Finding-level precision alone rewards duplicate flooding. Evaluation reports hypothesis precision, root-cause-cluster precision, recall against known cases, severity agreement, replay success, dossier completeness, and developer-confirmed actionable precision. Confidence intervals are clustered by repository because findings in one codebase are dependent.

Deduplication as remediation-unit inference

String similarity is insufficient: two SQL-injection reports can require different fixes, while twenty sink locations can share one unsafe query builder. Lumen constructs a finding graph. Edges encode shared source, sink, call path, package, introduced commit, code-owner boundary, reproduction signature, and learned semantic similarity. A constrained clustering algorithm may merge findings only when they share a plausible remediation unit; high-impact cross-boundary merges require review.

The primary cluster key is a versioned tuple:

```
\[ k=(repo, snapshot, weakness, root\_cause\_symbol, taint\_origin, fix\_boundary). \]
```

Cluster quality is assessed pairwise and operationally. Pairwise precision penalizes merging distinct flaws; pairwise recall penalizes duplicate tickets. More important is **fix consistency**: after one member is repaired, do the others disappear under rescan? A cluster that cannot predict shared remediation is semantically weak even if its descriptions look alike.

Cross-snapshot lineage uses code movement, symbol identity, and patch similarity to distinguish persistence from reintroduction. Closing a finding because line numbers changed is prohibited.

Ranking under developer capacity

Severity is not priority. Lumen estimates expected avoidable loss per unit of remediation effort:

```
\[ priority_i = \frac{P(\text{exploit}_i \mid x_i) \cdot \text{Impact}_i \cdot \text{Exposure}_i \cdot \text{Fixability}_i}{E[\text{effort}_i] + \epsilon}. \]
```

Inputs include dossier confidence, reachable deployment, internet exposure, asset criticality, known exploitation, privilege required, compensating controls, code ownership, and historical fix effort. CVSS remains a standardized description of technical severity; EPSS and CISA KEV provide population-level exploitation evidence; customer context changes the final decision. The model outputs calibrated probability bands and an abstention state when build or deployment evidence is missing.

Ranking is constrained. A tenant may require all critical verified findings shown regardless of predicted effort, reserve capacity for systemic root causes, and cap the number sent to any team. Offline evaluation uses precision and cumulative avoidable-loss gain at realistic review budgets. Online evaluation measures the funnel, not clicks alone.

Developer funnel and product instrumentation

The developer journey has explicit timestamps and loss reasons:

```
generated -> verified -> clustered -> surfaced -> opened -> accepted -> assigned -> fix_started ->
fix_proposed -> validation_passed -> deployed -> recurrence_window_complete.
```

At each transition, record the eligible denominator. A finding not surfaced due to capacity is different from one surfaced and ignored. Reasons include duplicate, not reachable, accepted risk, wrong owner, insufficient

evidence, no feasible fix, and fixed elsewhere. Ticket edits and manual merges are product feedback, not ground truth by themselves.

Core funnel measures include time to first inspection, dossier-open rate, valid-dismissal rate, acceptance conditional on verification confidence, time to owner, time in progress, validation pass rate, deployment lag, and developer minutes per accepted finding. Segment by language, detector, team, tenure, and integration without creating a leaderboard that pressures teams to close findings incorrectly.

Safe runnable Python lab: cluster and rank dossiers

The lab uses synthetic metadata, makes no network calls, and executes no exploit.

```
from dataclasses import dataclass
from collections import defaultdict

@dataclass(frozen=True)
class Finding:
    fid: str
    weakness: str
    root_symbol: str
    fix_boundary: str
    p_exploit: float
    impact: float
    exposure: float
    effort: float

findings = [
    Finding("F1", "CWE-89", "QueryBuilder.raw", "db-lib", .30, 9, 1.0, 3),
    Finding("F2", "CWE-89", "QueryBuilder.raw", "db-lib", .28, 9, 1.0, 3),
    Finding("F3", "CWE-79", "renderProfile", "web-ui", .12, 6, .8, 1),
    Finding("F4", "CWE-89", "AdminSearch.sql", "admin-api", .08, 8, .2, 2),
]

clusters = defaultdict(list)
for f in findings:
    clusters[(f.weakness, f.root_symbol, f.fix_boundary)].append(f)

rows = []
for key, members in clusters.items():
    # One remediation unit: use maximum risk, not the sum of duplicate symptoms.
    best = max(members, key=lambda f: f.p_exploit*f.impact*f.exposure/f.effort)
    score = best.p_exploit * best.impact * best.exposure / best.effort
    rows.append((score, ",".join(f.fid for f in members), key[1]))

for score, ids, root in sorted(rows, reverse=True):
    print(f"{ids:5} root={root:16} priority={score:.3f}")
```

Expected output:

```
F1,F2 root=QueryBuilder.raw priority=0.900
F3    root=renderProfile    priority=0.576
F4    root=AdminSearch.sql  priority=0.064
```

The crucial line takes maximum risk within a remediation unit rather than summing duplicates. Summation would reward the scanner for emitting more symptoms. In a production portfolio, correlated loss and multiple independently exploitable paths require a richer aggregation model.

Remediation survival and recurrence

Mean time to remediate is biased when open findings disappear from the calculation. Define origin at the first verified/surfaced time, event at validated deployment, and censoring at observation end, repository archival, or product disconnect—with those mechanisms reported separately. Kaplan-Meier curves estimate the probability a finding remains unresolved:

$$\hat{S}(t) = \prod_{t_j \leq t} \left(1 - \frac{d_j}{n_j}\right)$$

Use cumulative incidence when accepted risk, repository deletion, and remediation compete. A Cox or accelerated-failure-time model can estimate associations with severity, ownership, evidence quality, and suggested-fix availability, but product exposure is confounded: teams receiving richer features may differ systematically. Randomized rollout or stepped-wedge introduction can estimate whether evidence dossiers accelerate validated fixes.

Recurrence is a new manifestation of the same root-cause family after a validated fix and clean window. Measure 30/90/180-day recurrence, vulnerable-code reintroduction, sibling-service recurrence, and bypass of the original patch. Durable remediation may be a shared library change, framework guardrail, test, or training intervention. A team that patches ten endpoints while leaving the unsafe helper intact has fast closure and poor security.

Enterprise value without vanity economics

Enterprise reporting has four layers:

- **Evidence quality:** verified distinct findings, calibration, reproduction rate, coverage limits.
- **Workflow:** accepted clusters, developer effort, time to validated deployment, integration health.
- **Security outcomes:** exposed risk retired, recurrence avoided, systemic fixes, known-exploited coverage.
- **Business value:** avoided expected loss ranges, audit evidence, engineering time saved or consumed, renewal-relevant adoption.

Avoid claiming that every critical finding equals a breach prevented. For finding cluster (i) , estimate incremental expected loss reduction as

$$\Delta EL_i = [P(A_i \mid \text{no fix}) - P(A_i \mid \text{fix})] \times \text{Loss}_i$$

with ranges and dependence adjustments. Attribute value only after validated deployment and account for findings that an existing tool or developer would have found. A randomized repository rollout, matched stepped wedge, or credible instrument can estimate incremental discovery and remediation. Customer retention correlations are supportive product evidence, not proof of security benefit.

Suggested fixes and validation gates

An AI-generated patch is a new hypothesis, not remediation evidence. Lumen displays it only when the system can state the intended security invariant and validate against exploit-oriented and regression tests. The patch runs in a disposable branch without repository write access. Existing tests, targeted security tests, static analysis, dependency resolution, and behavior-preservation checks run first. Generated tests are marked and cannot be the sole oracle for a generated fix.

Validation looks for displacement. Escaping one sink may leave an equivalent path; moving validation downstream can create an authorization inconsistency; upgrading a package may select a still-vulnerable transitive version. The post-patch dossier lists invalidated claims, remaining claims, build coverage, and untested surfaces. Lumen abstains where the fix crosses an architectural decision, cryptographic protocol, unsafe-memory interface, or business authorization rule.

Measure acceptance, developer edits, test success, time saved, rollback, and recurrence. Acceptance alone is weak: a convenient patch may be incomplete. Randomized fix offers among eligible low-risk findings can estimate incremental time to validated deployment. The product never auto-merges based only on model confidence.

Multi-tenant isolation and disclosure

Code, embeddings, build artifacts, and evidence remain tenant-scoped. Cache keys include tenant and snapshot; cross-tenant retrieval tests block release. Customer code is not silently added to shared training. Retention and improvement consent are explicit, and support access is time-bounded and audited.

For flaws in shared dependencies, distinguish the customer's affected instance from the upstream vulnerability. Coordinated disclosure reproduces safely, establishes affected versions, avoids premature weaponizable detail, notifies the appropriate maintainer, and gives customers mitigation evidence. Product urgency and finding counts do not override safe disclosure.

Failure drills

Hallucinated dossier drill. The narrative cites a dataflow edge absent from deterministic evidence. Quarantine the detector release, prevent unsupported prose from rendering, replay affected dossiers, and search for the same claim template. The claim/evidence foreign-key check must fail before customer exposure.

Duplicate storm drill. A parser change produces 40,000 variants. Freeze notification, compare root-cause cardinality and fix consistency, restore the previous clusterer, and ensure billing and executive metrics use remediation units rather than raw findings.

Unsafe reproduction drill. A proposed proof attempts outbound access. The sandbox denies it, destroys the run, preserves metadata, and routes the case to static validation. The model never receives credentials to “make the proof work.”

Closure illusion drill. Findings close after a file rename and recur on rescan. Rebuild cross-snapshot lineage, reopen affected clusters without penalizing developers, and correct survival origin/event logic.

Field Case: Building a Data-Science Assurance Layer for sec-pr

This section reports an implemented validation project for **sec-pr**, a pull-request cybersecurity review application. The case is useful because it exposes a common category error in security products: teams often mistake operational volume for validated effectiveness. A thousand scans prove that a system ran. They do not prove that its findings were correct, that it found most eligible vulnerabilities, or that it reduced risk without imposing excessive review burden.

The engineering innovation was therefore not another detector. It was a **closed evidence loop** that joined a working cybersecurity application to a governed data-science layer:

```

sec-pr scan and workflow data
      ↓
immutable identity, hashes, versions, and provenance
      ↓
normalization, completeness checks, and quarantine
      ↓
human adjudication and benchmark contracts
      ↓
precision, recall, calibration, slices, burden, and drift
      ↓
minimum-evidence release decision

```

The final arrow is deliberately conditional. A dashboard is not entitled to publish an effectiveness number merely because a formula can be evaluated. The denominator, label maturity, sample size, population definition, and observation window must first satisfy the metric contract.

Decision objective and estimands

The project did not ask the vague question, “Is sec-pr good?” It decomposed that question into decisions:

1. Is the evidence platform fit for controlled engineering use?
2. For which repositories, languages, vulnerability classes, change types, and safety contexts is detector effectiveness supported?
3. What is the false-interruption burden on developers?
4. What eligible changed code was not inspected successfully?
5. Does a candidate scanner version improve security yield without regressing critical cohorts or workflow reliability?
6. Is the evidence mature enough to promote, hold, roll back, or continue observing?

These decisions require different estimands. Posted-finding precision requires adjudicated posted findings. Recall requires independently reviewed eligible changed code, including clean pull requests and confirmed misses. Coverage requires the full eligible PR-iteration population. Operational reliability requires workflow events such as scans, comments, replies, votes, rescans, and closure. None of these denominators can be substituted for another.

Four linked analytical grains

The canonical design separated four grains that are frequently collapsed in production databases:

Grain	Identity	Question answered
PR iteration	organization, project, repository, PR, iteration	What exact changed-code population was eligible?
Scan run	PR iteration plus scanner, prompt, policy, and execution version	What system behavior produced the observations?
Finding candidate	scan plus stable finding identity and disposition	What vulnerability claim was generated, suppressed, duplicated, or posted?
Workflow event	actor, action, target, event time, and provenance	What happened after the prediction?

This separation prevents three serious errors. First, rescans are not mistaken for new pull requests. Second, mutable current state is not mistaken for historical evidence. Third, a posted comment is not mistaken for an independently validated vulnerability.

Implemented evidence corpus

The implementation audit verified the following state:

Record or check	Verified result	Correct interpretation
PR iterations	1,287	An iteration-aware evaluation population was reconstructed.
Scan runs	1,428	Historical scanner executions were normalized.
Finding candidates	1,098	Candidate claims were available for governed review.
Label-queue items	1,098	Every candidate could enter the human-review workflow.
Evidence artifacts	566	Observed artifacts were indexed with provenance and hashes.
Benchmark contracts	564	Versioned benchmark definitions were registered.
Reconstructed workflow events	15	A limited historical workflow record was recovered explicitly.

Record or check	Verified result	Correct interpretation
Metric snapshots	5	Metric state and maturity could be persisted reproducibly.
Unresolved quarantine records	278	Unreliable records were isolated rather than silently accepted.
Invented validity labels	0	Missing outcome truth remained missing.
Invented missed findings	0	The recall denominator was not fabricated.
Database integrity	ok	The SQLite evidence store passed structural integrity checking.
Foreign-key errors	0	No broken canonical relationships were detected.
Verification	41 Python tests, 3 frontend tests, production build, 5 browser-verified routes	The scoped engineering release suite passed.

These numbers are intentionally reported as **engineering evidence**, not as a detector score. The most scientifically important zeros are the zero invented labels and zero invented misses. They show that the implementation preferred an honest unknown over a polished but fictitious metric.

Quarantine is part of the measurement system

The 278 unresolved records were not deleted, coerced into a convenient schema, or counted as valid observations. They were quarantined with reason and provenance. Typical causes in systems like this include incomplete composite identity, missing iteration linkage, malformed historical artifacts, unresolved duplicate identity, or inadequate evidence for a semantic mapping.

Quarantine changes the analytical contract. Every published result should expose at least:

```
eligible population
- structurally ineligible records
- quarantined unresolved records
- immature labels
= mature analytical denominator
```

This reconciliation makes missingness visible. It also supports repair: a quarantined record can later be corrected and replayed idempotently without rewriting the raw artifact.

Ground truth as an append-only workflow

The project used evidence maturity rather than a single mutable `is_valid` column:

- **Gold:** strong independent confirmation, such as qualified reviewer agreement or reviewer plus demonstrated reproduction.
- **Silver:** validated operational evidence with sufficient rationale.

- **Bronze:** provisional evidence, including an unresolved assertion or weak single-source judgment.
- **Unknown:** insufficient evidence.

Labels are events. Corrections append a new decision with reviewer role, timestamp, rationale, evidence reference, and superseded label. This supports agreement measurement, conflict adjudication, and retrospective reconstruction of what was known at release time.

The 1,098-item review queue demonstrates workflow readiness, not completed truth. Until qualified adjudication matures, precision and severity-agreement claims remain gated. Recall requires an additional stream: independent review of eligible changed code that did not produce posted findings. Reviewer disagreement is measured rather than hidden, using raw agreement and a chance-corrected statistic where the sample supports it.

Metric contracts and fail-closed publication

Each measure was designed as a contract containing:

- decision and owner;
- numerator and denominator;
- inclusion and exclusion rules;
- label maturity requirement;
- grouping and cohort dimensions;
- minimum sample size;
- interval or uncertainty method;
- version dependencies;
- freshness and observation-window requirement;
- status such as `mature`, `provisional`, or `insufficient_evidence`.

Precision uses mature adjudicated posted findings and should be accompanied by a binomial interval such as Wilson's. Recall uses confirmed issues found divided by all confirmed issues in independently reviewed eligible code. Calibration compares confidence with observed validity and can use reliability diagrams, Brier score, or expected calibration error after adequate labels accumulate. Burden measures invalid interruptions and review minutes per eligible PR. Drift is decomposed into input, prediction, outcome, operational, and policy drift so that a changing repository mix is not confused with a degraded detector.

One benchmark run was executed. Its result was `insufficient_evidence`. This was a successful safety behavior: the benchmark pipeline ran, evaluated its evidence requirements, and refused to manufacture a pass or failure. In high-stakes analytics, an explicit non-result can be the correct result.

Shadow comparison and release logic

Candidate versions run in a structurally read-only shadow path. They cannot comment on live pull requests, cast votes, mutate trackers, or trigger production rescans. A valid comparison freezes the exact eligible input and records both versions' scanner, prompt, policy, registry, dependency, and benchmark versions.

A promotion decision should combine:

1. no critical safety regression;
2. mature evidence in required cohorts;
3. paired or otherwise comparable evaluation;
4. bounded developer burden;
5. operational SLO compliance;
6. no unresolved data-integrity blocker;
7. documented rollback conditions.

The independent implementation review returned **GO FOR ENGINEERING RELEASE**. This means the evidence platform was fit for controlled ingestion, adjudication, benchmarking, dashboarding, and release governance. It does **not** mean that sec-pr precision, recall, calibration, subgroup performance, or long-term drift were established. The distinction is not legalistic fine print; it is the central result of the validation project.

Reproducibility protocol

An engineer reproducing a metric snapshot should be able to recover:

1. the immutable source artifacts and checksums;
2. the canonical schema and migration version;
3. the exact population query and exclusions;
4. scanner, prompt, policy, registry, and benchmark versions;
5. label events and maturity rules available at computation time;
6. the metric implementation version;
7. the resulting numerator, denominator, interval, cohort, and maturity state.

The operational database used serialized SQLite writes for the pilot, with integrity and foreign-key verification. PostgreSQL was identified as the scale-out boundary rather than pretending that a single-writer pilot store was an enterprise endpoint. Aggregate views avoid exposing source snippets, while evidence drill-through remains subject to repository authorization. Credentials and personal identities are excluded or pseudonymized in analytical exports.

What the field case changed in the product conversation

Before the validation layer, stakeholders could discuss scan counts, findings, and anecdotal fixes. After it, they could ask better questions:

- Which population did this score represent?
- How many observations were quarantined or unlabeled?
- Was recall measured against independently reviewed clean code?
- Did performance differ for authentication, cryptography, injection, or sensitive logging?
- Did a new prompt improve yield by increasing noise?

- Was the apparent improvement a detector change, a repository-mix change, or a policy change?
- What evidence supports promotion, and what evidence is still absent?

That transition—from activity reporting to falsifiable decision support—is the real data-science contribution.

Field-case interview questions and answers

Question: You have 1,098 scanner findings but no mature labels. What can you report?

Answer: Report the finding population, its provenance, disposition completeness, queue status, data-quality exclusions, and label maturity. Do not report precision. Create a stratified adjudication plan, define reviewer qualification and conflict resolution, and publish precision only after the metric contract's maturity and sample gates are satisfied.

Question: Why can posted findings estimate precision but not recall?

Answer: Posted findings reveal claims the scanner made. Recall also needs issues the scanner did not report. Its denominator therefore requires independent review of eligible changed code, including apparently clean PRs, plus a governed process for confirmed misses.

Question: Is quarantining 278 records evidence of poor engineering?

Answer: It is evidence of incomplete historical data and functioning quality controls. The engineering failure would be silently coercing those records into the analytical population. The operational response is to classify causes, repair recoverable records, measure quarantine rate by source, and prevent recurrence at ingestion.

Question: How do you explain “GO FOR ENGINEERING RELEASE” beside “effectiveness not established”?

Answer: They answer different questions. The first says the validation machinery, controls, tests, and evidence paths are fit for controlled use. The second says the outcome evidence is not yet mature enough to quantify detector quality. A professional assurance system must be able to release measurement infrastructure before it has accumulated the future observations that infrastructure is designed to govern.

Question: What is the most novel aspect of the system?

Answer: The closed evidence loop. A real cybersecurity application emits workflow evidence; an assurance digital thread preserves identity and provenance; governed human truth and statistical methods test performance; and explicit evidence gates feed promotion or hold decisions. Detection, measurement, and release governance are integrated without allowing the evaluation path to mutate production behavior.

Sources and Further Reading

- NIST, SSDF SP 800-218, frames vulnerability discovery as part of a secure development lifecycle; this chapter extends measurement through validated deployment and recurrence.
- MITRE, CWE, supplies weakness semantics useful for claims and clustering, but CWE identity alone does not establish a shared fix boundary.

- FIRST, CVSS v4.0, standardizes technical severity inputs; it is intentionally not treated as a complete remediation priority.
- FIRST, EPSS, estimates exploitation probability for published CVEs. Its population-level signal complements, but cannot replace, code reachability and customer exposure.
- CISA, Known Exploited Vulnerabilities Catalog, provides strong evidence of exploitation in the wild and should create portfolio constraints rather than merely add score points.
- NIST, AI RMF 1.0, motivates explicit validity, reliability, transparency, and monitoring claims for the AI components of the product.
- Kaplan and Meier, *Nonparametric Estimation from Incomplete Observations*, provides the censoring-aware basis for remediation survival analysis.

Interview Practicum

An AI scanner's dashboard shows 12,000 “critical” findings and 96% precision on a seeded benchmark. Customers accept only 8% of surfaced findings. One large customer says half are duplicates, while sales claims the count demonstrates value. You have six weeks before renewal season and may not run exploit code against customer environments. Redesign the product measurement and recommend what ships.

Worked Answer

I would stop using raw critical count as value and decompose the failure. The benchmark may be contaminated or unrepresentative; precision may be measured per emitted symptom; severity may omit reachability; and acceptance may reflect both quality and workflow. I would sample production findings across languages, customers, scores, accepted and dismissed states, then conduct blinded adjudication of validity, reachability, exploitability, actionability, severity, and distinctness.

I would make the dossier the product contract. Every displayed claim must reference deterministic source, path, configuration, or safe sandbox evidence. Unsupported claims abstain. Since customer execution is prohibited, validation uses authorized snapshots, inert fixtures, static witnesses, and isolated product-owned test repositories. The UI exposes evidence gaps rather than inventing certainty.

Next I would cluster around remediation units using root symbol, dataflow origin, fix boundary, reproduction signature, and snapshot lineage. I would evaluate pairwise merge/split errors and fix consistency. Ranking would use calibrated exploitability, exposure, impact, confidence, and effort under customer capacity, with mandatory lanes for verified known-exploited or policy-critical items.

The six-week ship should be narrow: evidence-backed dossiers, cluster-level counts, a calibrated high-confidence queue, clear abstention, and funnel instrumentation. I would not ship autonomous fixes or an avoided-breach dollar counter. Renewal reporting would show distinct verified clusters, accepted clusters, validated deployments, median and survival-based remediation, systemic fixes, recurrence, developer effort, and coverage limitations.

At Principal level I would align incentives: sales cannot quote unverified raw counts; detector teams are scored on cluster precision and durable fixes, not output volume; product owns time-to-owner and evidence usability; research owns held-out generalization; and security owns safe reproduction policy. Then I would propose a staggered customer rollout to estimate incremental acceptance and remediation relative to the previous workflow.

Follow-Up Questions and Answers

What if deduplication hides two exploitable paths? Preserve member dossiers and path evidence inside the remediation cluster. Merge workflow, not truth; split when fixes or exploit preconditions differ.

How do you value a systemic fix? Count retired affected paths and estimate incremental loss reduction once, with dependence, rather than multiplying value by every symptom.

Should developer dismissal be a negative label? No. It is a disposition requiring a reason. Wrong owner, duplicate, risk acceptance, and invalidity have different learning meaning.

What is the release stop condition? Any unsupported evidence claim, cross-tenant artifact exposure, unsafe reproduction escape, or severe calibration failure in a material language segment.

Key Takeaways

- The atomic product object is an evidence-linked, versioned dossier—not generated prose.
- Measure distinct remediation units so duplicate output cannot masquerade as coverage or value.
- Rank by calibrated, contextual avoidable loss under capacity and policy constraints.
- Instrument the full developer funnel through validated deployment, survival, and recurrence.
- Enterprise value requires incremental evidence and uncertainty, not “findings times breach cost.”

Chapter 48: Case Study: Evidence-Driven Security Program

An enterprise security program is usually described as a list of tools, controls, incidents, and compliance percentages. Those objects do not compose into evidence. A green control dashboard can coexist with an expanding attack path; fewer incidents can reflect prevention, attacker quiet, or broken sensors; a risk score can conceal assumptions that no executive agreed to.

This final capstone designs an evidence system for Meridian, a global software and payments company. It integrates the book's semantic contracts, telemetry engineering, threat models, detection measurement, vulnerability prioritization, causal evaluation, AI assurance, production reliability, team design, and decision communication. The goal is not a universal cyber number. It is a traceable chain from business assets and plausible threats through control claims and observations to decisions, uncertainty, and accountable work.

Semantic architecture: facts before dashboards

Meridian defines a small enterprise security ontology. Core entities are `BusinessService`, `Asset`, `Identity`, `DataClass`, `ThreatScenario`, `AttackPath`, `Control`, `ControlClaim`, `Evidence`, `Observation`, `Finding`, `Incident`, `Decision`, and `Owner`. Every metric declares its population, event time, knowledge time, grain, exclusions, source health, and version.

Canonical facts are append-only. Corrections supersede prior records; they do not silently rewrite history. Operational systems retain local schemas, while semantic adapters map them into governed facts. This avoids forcing endpoint, cloud, identity, and application telemetry into one lossy mega-table.

```

flowchart TB
    subgraph Sources
        CM[Asset/identity/config]
        TM[Security telemetry]
        WK[Workflow and finance]
    end
    CM --> SC[Versioned semantic contracts]
    TM --> SC
    WK --> SC
    SC --> FF[(Canonical fact and lineage layer)]
    FF --> EG[(Claims and evidence graph)]
    FF --> FS[Point-in-time feature products]
    FS --> AN[Detection, vulnerability, and causal analytics]
    AN --> EG
    EG --> TP[Threat portfolio]
    TP --> DR[Decision registry]
    DR --> EX[Executive narratives and roadmap]
    EX --> DR
  
```

A metric definition resembles an API contract. For example, `identity_compromise_harm_30d` specifies an episode population, verified harmful outcomes within 30 days, label maturity, intervention status, and excluded test tenants. Its dashboard cannot substitute raw alert count without changing the metric version.

Data quality is risk-weighted. Completeness for production control-plane logs receives a tighter SLO than completeness for low-risk laboratory endpoints. Health dimensions include expected volume, entity join rate, latency, schema conformity, clock skew, duplication, and outcome linkage. A sensor-health incident annotates dependent evidence so downstream reports do not treat absence as safety.

Control claims and evidence graph

“MFA is deployed” is an implementation fact, not a security claim. Meridian expresses a control as falsifiable propositions:

```
control_claim(
  claim_id, control_id, subject_scope, threat_edge,
  proposition, target, valid_from, valid_to,
  claim_owner, review_frequency, status
)

evidence_assertion(
  assertion_id, claim_id, evidence_type, artifact_uri,
  observed_at, population_start, population_end,
  result, uncertainty_low, uncertainty_high,
  freshness_deadline, producer, method_version
)

dependency(
  upstream_claim_id, downstream_claim_id,
  dependency_type, rationale
)
```

An identity claim might state: “For workforce administrator access to production, phishing-resistant authentication interrupts credential replay in at least 99% of eligible attempts, with telemetry coverage above 99.5%.” Evidence can include configuration attestations, sampled access logs, replay tests, red-team results, exceptions, and outage history. Each item has scope and freshness. A policy document alone cannot satisfy an effectiveness claim.

The graph separates four evidence classes:

- **Design evidence:** architecture, policy, threat analysis, and test specification.
- **Implementation evidence:** configuration, deployment, inventory, and attestation.
- **Operating evidence:** logs, sampling, SLOs, exceptions, and failure recovery.
- **Outcome evidence:** attack interruption, incident impact, remediation, and causal estimates.

Claim confidence is not a naive average. Suppose implementation depends on sensor coverage and policy enforcement. The confidence in the conjunction cannot exceed its weakest required dependency. Correlated

evidence from the same source receives less weight than independent evidence. One usable conservative aggregation is

$$\backslash [C(\text{claim}) = \min_{\{d \in \text{required}\}} C(d) \times \text{Freshness} \times \text{ScopeCoverage}, \backslash]$$

with uncertainty retained, not rounded into traffic-light theater. The graph permits queries such as “Which high-loss attack paths depend on a stale identity-provider assertion?” and “Which executive claims rely on the same failing asset inventory?”

Threat portfolio and control coverage

Meridian manages threat scenarios rather than an undifferentiated risk register. A scenario states actor, objective, entry condition, attack path, affected services, loss mechanisms, current controls, observability, and plausible frequency/severity ranges. Examples include cloud-control-plane takeover, software supply-chain compromise, payment credential theft, destructive ransomware, and AI-agent misuse.

Attack paths are directed graphs. A control maps to an edge only when a falsifiable claim explains how it prevents, detects, delays, or recovers that transition. Coverage has dimensions: population scope, time, implementation, demonstrated effectiveness, and independence. Five tools observing one edge do not imply fivefold protection.

Scenario loss is a distribution:

$$\backslash [L_s = I_s \times D_s + R_s + C_s, \backslash]$$

where (I_s) is business interruption per unit time, (D_s) duration, (R_s) response/recovery cost, and (C_s) customer, legal, or contractual consequence. Frequency and loss ranges come from internal episodes, near misses, external evidence, expert elicitation, and stress tests. Monte Carlo simulation can propagate uncertainty, but the result remains conditional on the scenario model.

Prioritization uses marginal risk reduction per constrained unit, adjusted for strategic dependencies:

$$\backslash [\text{Value}(j) = \frac{E[L \mid \text{current}] - E[L \mid \text{control}_j]}{\text{Cost}_j \times \text{Enablement}_j}, \backslash]$$

This does not reduce the portfolio to a greedy ratio. Some investments—asset identity, telemetry lineage, strong authentication, safe deployment—enable many later controls. The roadmap therefore balances immediate loss reduction, foundational leverage, regulatory obligations, and concentration risk.

Detection, incidents, and vulnerabilities in one portfolio

Detection metrics connect to scenario edges. Report eligible attack opportunities, sensor coverage, detection probability with intervals, time from first observable event to decision, analyst capacity, containment time, and missed/late evidence. Alert volume is a workload measure.

Incident outcomes use severity-adjusted business harm, time to containment and recovery, recurrence, and control-learning closure. Incident counts are shown with exposure and sensor health. A postmortem creates or updates graph evidence; it does not close until systemic actions have owners and validation criteria.

Vulnerabilities are clustered into remediation units and joined to deployment exposure, business service, exploit evidence, and compensating controls. The queue optimizes avoidable loss under engineering capacity, while mandatory policy lanes remain visible. Survival and recurrence reveal whether teams retire root causes or merely close tickets.

Safe runnable SQL lab: find unsupported executive claims

This SQLite lab builds a miniature claims/evidence graph and queries stale or insufficient assertions. It uses no production data.

```
CREATE TABLE claims(
  claim_id TEXT PRIMARY KEY, scenario TEXT, proposition TEXT,
  min_coverage REAL, max_age_days INTEGER, executive_visible INTEGER
);
CREATE TABLE evidence(
  evidence_id TEXT PRIMARY KEY, claim_id TEXT, observed_date TEXT,
  coverage REAL, result TEXT, independent_source TEXT
);

INSERT INTO claims VALUES
('C1','cloud takeover','Admin MFA blocks replay',0.995,30,1),
('C2','supply chain','Release artifacts are provenance-verified',0.990,14,1),
('C3','ransomware','Tier-0 recovery meets 8-hour objective',0.950,90,1);
INSERT INTO evidence VALUES
('E1','C1','2026-07-28',0.998,'pass','idp-log'),
('E2','C1','2026-07-29',0.997,'pass','red-team'),
('E3','C2','2026-06-01',0.999,'pass','ci-config'),
('E4','C3','2026-07-15',0.910,'pass','recovery-test');

WITH latest AS (
  SELECT c.claim_id, c.scenario, c.proposition, c.min_coverage, c.max_age_days,
    MAX(e.observed_date) AS last_seen,
    MIN(e.coverage) AS demonstrated_coverage,
    COUNT(DISTINCT e.independent_source) AS independent_sources
  FROM claims c LEFT JOIN evidence e USING(claim_id)
  WHERE c.executive_visible=1 GROUP BY c.claim_id
)
SELECT claim_id, scenario,
CASE
  WHEN last_seen IS NULL THEN 'NO_EVIDENCE'
  WHEN julianday('2026-08-20')-julianday(last_seen) > max_age_days THEN 'STALE'
  WHEN demonstrated_coverage < min_coverage THEN 'INSUFFICIENT_SCOPE'
  WHEN independent_sources < 2 THEN 'SINGLE_SOURCE'
  ELSE 'SUPPORTED'
END AS status
FROM latest ORDER BY claim_id;
```

Expected output:

```
C1|cloud takeover|SUPPORTED
C2|supply chain|STALE
C3|ransomware|INSUFFICIENT_SCOPE
```

The query is intentionally conservative: it uses minimum demonstrated coverage and requires two independent sources. In production, evidence applicability is interval- and population-specific, and dependence is more nuanced than counting source names. Still, this simple test catches a common governance failure: a current dashboard backed by stale evidence.

Executive reporting as decision support

The quarterly executive report begins with decisions, not telemetry. A one-page portfolio view contains:

1. material scenario movement and why;
2. controls whose evidence strengthened, weakened, or expired;
3. realized incidents and near misses linked to scenarios;
4. leading exposure changes such as internet-facing privilege or unsupported software;
5. investments requiring acceptance, acceleration, or stop decisions;
6. uncertainty and data-health exceptions.

Every statement is tagged as fact, estimate, assumption, or recommendation. Ranges replace false precision. The report may say: “Cloud takeover exposure likely decreased after phishing-resistant administrator authentication; implementation evidence covers 99.8% of eligible access and red-team replay was blocked, but two break-glass paths lack independent testing.” That is more actionable than “identity risk: 72/100.”

Board reporting uses fewer scenarios and emphasizes loss mechanisms, resilience, concentration, trend evidence, and management decisions. Operational drill-down remains available through the graph. Compliance mappings are generated from claims and evidence but do not become the enterprise risk model: passing a control test and reducing a material threat are related, not identical.

Operating model and measurement governance

A small central security data-science and evidence team owns ontology, causal standards, shared platforms, and portfolio synthesis. Embedded analysts own domain features and decisions in identity, cloud, product security, detection, and resilience. Security engineering owns control operation; business-service owners own risk decisions; internal audit independently tests selected claims.

Every production metric has a named owner, semantic version, review cadence, source SLO, and retirement condition. Every model has point-in-time reproducibility, calibration monitoring, rollback, and an intervention policy. Every experiment has an estimand and ethical boundary. The decision registry records the evidence available, alternatives considered, expected outcome, owner, review date, and what would reverse the decision.

The program avoids performance traps. Analysts are not rewarded for alert counts, red teams for raw findings, or engineering teams for ticket closure alone. Shared outcomes include demonstrated attack-path interruption, durable remediation, reduced recovery time, evidence freshness, and decision turnaround—balanced by user friction, privacy, cost, and reliability.

Twelve-month roadmap

Quarter 1: establish truth and choose the portfolio

- Approve the ontology, metric contract template, evidence classes, and decision registry.
- Identify the top five threat scenarios with business owners and loss ranges.
- Inventory telemetry dependencies and define risk-weighted source SLOs.
- Build canonical identity, asset, service, and control identifiers; quantify unresolved joins.
- Baseline current incident harm, detection coverage, vulnerability survival, and control-evidence freshness.

Exit criteria: scenario owners agree on attack paths; 80% of Tier-0 services map to accountable assets and identities; executive claims expose source and freshness; known data gaps have funded owners.

Quarter 2: prove two vertical slices

- Deliver an identity-compromise slice from event contract through intervention and friction measurement.
- Deliver a software-supply-chain slice from build provenance through release control claims and recovery drill.
- Launch randomized label audits and independent evidence sampling.
- Connect vulnerability remediation units to services and deployment exposure.
- Train domain teams to write falsifiable claims and uncertainty-aware decision memos.

Exit criteria: both slices support point-in-time replay, have tested failure modes, and produce a decision that changes resource allocation or policy.

Quarter 3: evaluate controls and scale operations

- Run causal or quasi-experimental evaluations for two high-cost controls.
- Deploy detection-capacity optimization and remediation survival reporting.
- Automate claim freshness, scope, and dependency checks.
- Conduct cloud takeover, ransomware recovery, and coding-agent containment drills.
- Establish model release gates, calibration review, and rollback across security analytics.

Exit criteria: at least two controls have credible effect and friction estimates; stale executive claims automatically downgrade; drill actions are validated rather than merely assigned.

Quarter 4: institutionalize portfolio decisions

- Produce the first evidence-linked annual threat portfolio and budget options.
- Quantify risk reduction ranges and foundational enablement from completed investments.
- Retire vanity metrics and duplicate dashboards through metric-contract governance.
- Add audit sampling and board drill-down to the evidence graph.
- Review team topology, skills, privacy controls, retention, and next-year scenario changes.

Exit criteria: the budget memo traces proposed spending to scenarios, claims, evidence gaps, expected outcomes, and owners; executives can inspect uncertainty; next-year priorities follow measured residual exposure rather than tool renewal dates.

Failure drills

Executive metric contradiction. Incident reporting says ransomware recovery improved while recovery exercises show Tier-0 regression. Trace definitions, populations, and censoring; suspend the aggregate claim; publish the contradiction and decision needed. Negative evidence is not a communications defect.

Semantic breaking change. A cloud provider changes principal identifiers, collapsing join coverage. Quarantine dependent metrics, version the adapter, replay the affected period, and annotate every evidence assertion generated during degradation.

Control monoculture. Three “independent” claims rely on one identity provider. Simulate provider compromise or outage, update dependency edges, and fund a recovery or alternate-verification path. Tool count is not defense independence.

Roadmap capture. A loud stakeholder requests an AI dashboard unrelated to top scenarios. Require a decision, user, evidence gap, and expected portfolio effect. Prototype only if it competes successfully against committed work; novelty is not priority.

Sources and Further Reading

- NIST, [Cybersecurity Framework 2.0](#), supplies Govern/Identify/Protect/Detect/Respond/Recover outcomes. The evidence graph makes those outcomes falsifiable rather than treating framework categories as metrics.
- NIST, [Risk Management Framework SP 800-37 Rev. 2](#), informs lifecycle authorization, continuous monitoring, and accountable risk decisions.
- NIST, [Security and Privacy Controls SP 800-53 Rev. 5](#), provides a control catalog; this chapter distinguishes catalog mapping, implementation evidence, operating evidence, and outcome evidence.
- NIST, [Guide for Conducting Risk Assessments SP 800-30 Rev. 1](#), supports threat, vulnerability, likelihood, and impact reasoning while preserving uncertainty.
- MITRE, [ATT&CK](#), supports attack-path and detection mappings. Technique coverage does not by itself prove prevention or detection effectiveness.

- Open Group, [Open FAIR Standard](#), offers a quantitative loss-event vocabulary useful for scenario ranges; estimates remain conditional on evidence and assumptions.
- NIST, [AI RMF 1.0](#), extends governance to AI-enabled controls and products, including validity, transparency, monitoring, and accountability.

Interview Practicum

You join a company after a material cloud breach. The board wants one cyber-risk score in 30 days. Security owns 140 dashboards, audit reports 92% control compliance, asset inventory joins only 68% of cloud resources to a business service, and the CISO must submit next year's budget in eight weeks. Design the response without promising a mature quantitative program in a month.

Worked Answer

I would not manufacture a scalar. In 30 days I would produce a decision-grade provisional portfolio. First, reconstruct the breach as a scenario and attack path, then select four or five material scenarios with business owners: likely cloud takeover, supply chain, ransomware, payment-data theft, and a company-specific case. For each I would state assets, entry conditions, loss mechanisms, controls, evidence, gaps, and uncertainty.

The 68% join rate is not a footnote; it bounds every service-level claim. I would prioritize Tier-0 and externally exposed resources, measure unresolved populations, and create a temporary “unknown ownership” risk segment rather than dropping those assets. The 92% compliance figure would be decomposed into design, implementation, operation, and outcome evidence. I would test freshness and independence for the claims most relevant to the breach.

For the board, I would present scenario ranges and confidence, recent movement, concentration, control evidence that survived or failed, recovery readiness, and three decisions. If a visual summary is required, I would use ordinal evidence states with drill-down, not arithmetic that implies unsupported precision. I would explicitly label facts, estimates, assumptions, and recommendations.

For the eight-week budget, I would compare options by marginal scenario reduction, foundational enablement, obligations, cost, and time to evidence. Likely first investments are identity and asset semantics, control-plane telemetry, strong administrator authentication, tested recovery, and a claims/evidence layer—not another dashboard. Each funded item receives an owner, falsifiable claim, measurement plan, and stop/review date.

At Principal level, I would use the crisis to simplify governance: freeze creation of uncontracted metrics, register executive claims, establish source SLOs, select two end-to-end vertical slices, and define a 12-month roadmap. I would tell the board what can be known in 30 days, what cannot, and which instrumentation will reduce uncertainty. Credibility comes from traceability and bounded claims, not a polished number.

Follow-Up Questions and Answers

What if the board insists on a number? Provide explicitly modeled scenario loss ranges and, if necessary, a portfolio distribution with assumptions and sensitivity—not an opaque 0–100 score. Pair it with the largest uncertainty drivers.

How do you choose the first vertical slice? Select a material scenario with an imminent decision, usable telemetry, an intervention owner, and measurable harm/friction. Avoid choosing only the easiest data.

Can compliance evidence populate the graph? Yes, with scope, time, method, and evidence class. A passed audit sample supports a bounded assertion; it does not automatically prove attack-path effectiveness.

How do you show program value after one year? Demonstrated control effects, retired exposure, faster containment/recovery, durable remediation, fresher independent evidence, and better decisions—alongside cost, friction, privacy, and remaining uncertainty.

Key Takeaways

- Build enterprise security measurement on versioned semantic facts, not dashboard names.
- Express controls as falsifiable, scoped claims connected to fresh and independent evidence.
- Manage a threat-scenario portfolio with attack paths, loss ranges, dependencies, and residual uncertainty.
- Executive reporting should expose decisions, evidence changes, assumptions, and contradictions.
- A credible roadmap alternates foundational data work with end-to-end decisions that prove practical value.

Field lesson: platform evidence can mature before outcome evidence

The sec-pr assurance program established a canonical population of 1,287 PR iterations, 1,428 scan runs, 1,098 candidate findings, 566 evidence artifacts, and 564 benchmark contracts. It also exposed 278 unresolved records through quarantine and preserved unknown outcomes instead of fabricating labels. For program leaders, this is a useful maturity pattern: first make the evidence supply chain observable and governed; then acquire adjudicated outcomes; only then authorize effectiveness and investment claims. Chapter 47 presents the complete vertical slice.

Manuscript Chapters

This directory contains the 48 chapter source files for *Data Science for Cybersecurity in the Age of AI Agents*.

The canonical structure is maintained in `../TOC.md`. Chapter files use the naming convention `chapter-01.md` through `chapter-48.md`.